

加我微信: [spring888java](#), 进群一起交流java相关各种技术!!

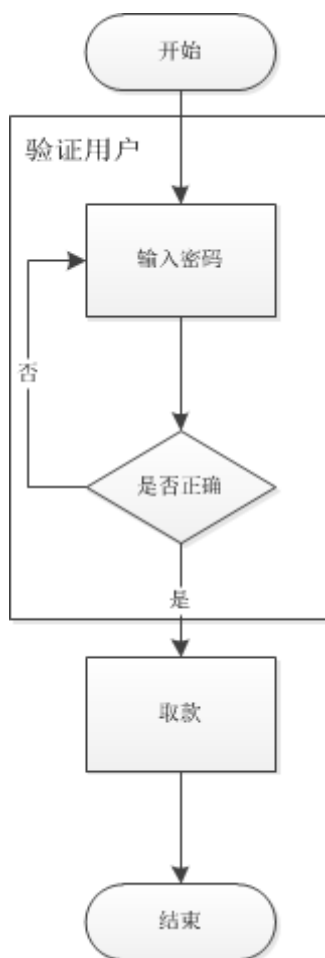
Spring系列第31篇: Aop概念详解

本文主要内容

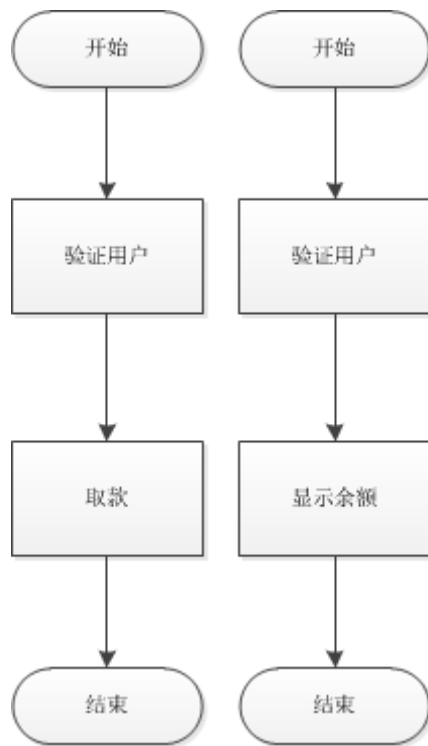
1. 什么是Aop?
2. Spring AOP中重要的一些概念详解
3. Spring AOP 硬编码实现

什么是AOP?

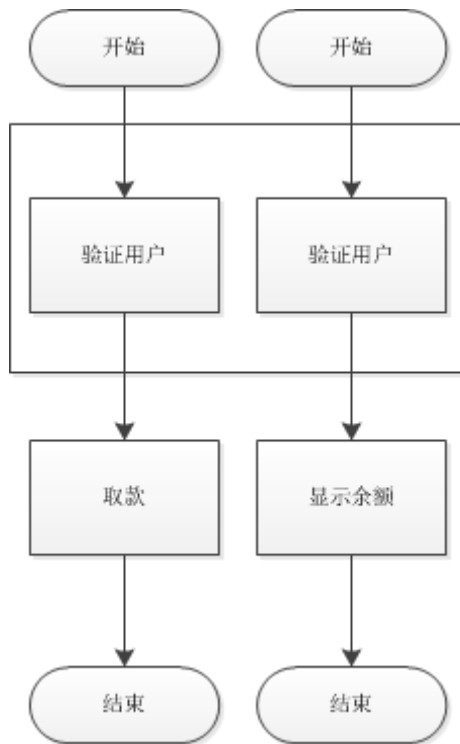
先看一下传统程序的流程, 比如银行系统会有一个取款流程



我们可以把方框里的流程合为一个, 另外系统还会有一个查询余额流程, 我们先把这两个流程放到一起:



有没有发现，这个两者有一个相同的验证流程，我们先把它们圈起来再说下一步：

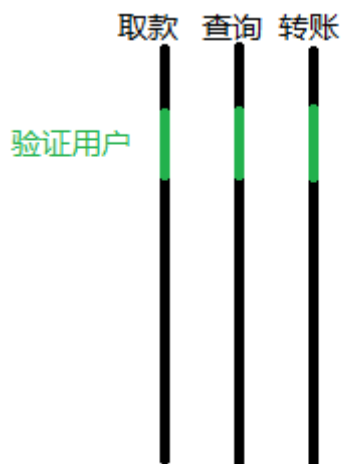


上面只是2个操作，如果有更多的操作，验证用户的功能是不是需要写很多次？

有没有想过可以把这个验证用户的代码是提取出来，不放到主流程里去呢，这就是AOP的作用了，有了AOP，你写代码时不要把这个验证用户步骤写进去，即完全不考虑验证用户，你写完之后，在另我一个地方，写好验证用户的代码，然后告诉Spring你要把这段代码加到哪几个地方，Spring就会帮你加过去，而不要你自己Copy过去，如果你有多个控制流呢，这个写代码的方法可以大大减少你的时间。

再举一个通用的例子，经常在debug的时候要打log吧，你也可以写好主要代码之后，把打log的代码写到另一个单独的地方，然后命令AOP把你的代码加过去，注意AOP不会把代码加到源文件里，但是它会正确的影响最终的机器代码。

现在大概明白了AOP了吗，我们来理一下头绪，上面那个方框像不像个平面，你可以把它当块板子，这块板子插入一些控制流程，这块板子就可以当成是AOP中的一个切面。所以AOP的本质是在一系列纵向的控制流程中，把那些相同的子流程提取成一个横向的面，这句话应该好理解吧，我们把纵向流程画成一条直线，然后把相同的部分以绿色突出，如下图左，而AOP相当于把相同的地方连一条横线，如下图右，这个图没画好，大家明白意思就行。



这个验证用户这个子流程就成了一个条线，也可以理解成一个切面，这里的切面只插了两三个流程，如果其它流程也需要这个子流程，也可以插到其它地方去。

通俗易懂的理解为：在程序中具有公共特性的某些类/某些方法上进行拦截，在方法执行的前面/后面/执行结果返回后增加执行一些方法。

先来考虑几个问题

1. aop中用什么来表示这些公共的功能？
2. aop中如何知道这些公共的功能用到哪些类的那些方法中去？
3. aop中需要将这些公共的功能用在目标方法的什么地方，前面？后面？还是其他什么地方？
4. aop底层是用什么实现的？

spring中有些概念，不是太好理解，带着这些问题，理解起来会容易很多，概念理解了，后面的路会容易很多，下面我们先来理解概念。

Spring中AOP一些概念

目标对象(target)

目标对象指将要被增强的对象，即包含主业务逻辑的类对象。

连接点(JoinPoint)

连接点，程序运行的某一个点，比如执行某个方法，在Spring AOP中Join Point总是表示一个方法的执行

代理对象(Proxy)

AOP中会通过代理的方式，对目标对象生成一个代理对象，代理对象中会加入需要增强功能，通过代理对象来间接的方式目标对象，起到增强目标对象的效果。

通知(Advice)

需要在目标对象中增强的功能，如上面说的：业务方法前验证用户的功能、方法执行之后打印方法的执行日志。

通知中有2个重要的信息：**方法的什么地方，执行什么操作**，这2个信息通过通知来指定。

方法的什么地方？之前、之后、包裹目标方法、方法抛出异常后等。

如：

在方法执行之前验证用户是否有效。

在方法执行之后，打印方法的执行耗时。

在方法抛出异常后，记录异常信息发送到mq。

切入点(Pointcut)

用来指定需要将通知使用到哪些地方，比如需要用在哪些类的哪些方法上，切入点就是做这个配置的。

切面 (Aspect)

通知 (Advice) 和切入点 (Pointcut) 的组合。切面来定义在哪些地方 (Pointcut) 执行什么操作 (Advice) 。

顾问 (Advisor)

Advisor 其实它就是 Pointcut 与 Advice 的组合，Advice 是要增强的逻辑，而增强的逻辑要在什么地方执行是通过Pointcut来指定的，所以 Advice 必需与 Pointcut 组合在一起，这就诞生了 Advisor 这个类，spring Aop中提供了一个Advisor接口将Pointcut 与 Advice 的组合起来。

Advisor有好几个称呼：顾问、通知器。

其中这4个：连接点(JoinPoint)、通知(advice)、切入点(pointcut)、顾问 (advisor)，在spring中都定义了接口和类来表示这些对象，下面我们一个个来看一下。

连接点 (JoinPoint)

JoinPoint接口

```
package org.aopalliance.intercept;

public interface Joinpoint {

    /**
     * 转到拦截器链中的下一个拦截器
     */
    Object proceed() throws Throwable;
}
```



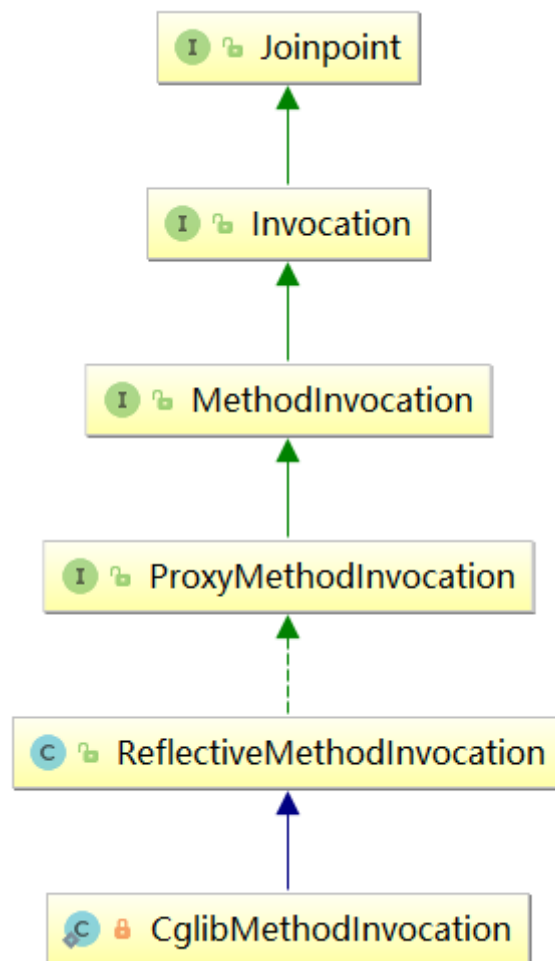
```

/**
 * 返回保存当前连接点静态部分【的对象】，这里一般指被代理的目标对象
 */
Object getThis();

/**
 * 返回此静态连接点 一般就为当前的Method(至少目前的唯一实现是MethodInvocation,所以连接点得静态部分肯定就是本方法)
 */
AccessibleObject getStaticPart();
}

```

几个重要的子接口和实现类，如下：



Invocation接口

```

package org.aopalliance.intercept;

/**
 * 此接口表示程序中的调用
 * 调用是一个连接点，可以被拦截器拦截。
 */
public interface Invocation extends Joinpoint {

    /**
     * 将参数作为数组对象获取。可以更改此数组中的元素值以更改参数。
     * 通常用来获取调用目标方法的参数
     */
}

```

```

    */
    Object[] getArguments();

}

```

MethodInvocation接口

```

package org.aopalliance.intercept;

import java.lang.reflect.Method;

/**
 * 方法调用的描述，在方法调用时提供给拦截器。
 * 方法调用是一个连接点，可以被方法拦截器拦截。
 */
public interface MethodInvocation extends Invocation {

    /**
     * 返回正在被调用得方法~~~ 返回的是当前Method对象。
     * 此时，效果同父类的AccessibleObject getStaticPart() 这个方法
     */
    Method getMethod();

}

```

通知(Advice)

通知中用来实现被增强的逻辑，通知中有2个关注点，再强调一下：方法的什么地方，执行什么操作。

Advice接口

通知的顶层接口，这个接口内部没有定义任何方法。

```

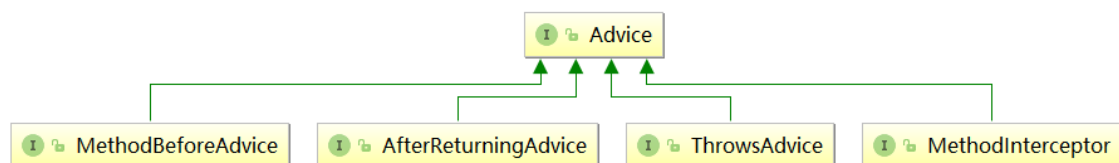
package org.aopalliance.aop;

public interface Advice {

}

```

Advice 4个子接口



MethodBeforeAdvice接口

方法执行前通知，需要在目标方法执行前执行一些逻辑的，可以通过这个实现。

通俗点说：需要在目标方法执行之前增强一些逻辑，可以通过这个接口来实现。before方法：在调用给定方法之前回调。

```
package org.springframework.aop;

public interface MethodBeforeAdvice extends BeforeAdvice {

    /**
     * 调用目标方法之前会先调用这个before方法
     * method: 需要执行的目标方法
     * args: 目标方法的参数
     * target: 目标对象
     */
    void before(Method method, Object[] args, @Nullable Object target) throws
    Throwable;
}
```

如同

```
public Object invoke(){
    调用MethodBeforeAdvice#before方法
    return 调用目标方法;
}
```

AfterReturningAdvice接口

方法执行后通知，需要在目标方法执行之后执行增强一些逻辑的，可以通过这个实现。

不过需要注意一点：目标方法正常执行后，才会回调这个接口，当目标方法有异常，那么这通知会被跳过。

```
package org.springframework.aop;

public interface AfterReturningAdvice extends AfterAdvice {

    /**
     * 目标方法执行之后会回调这个方法
     * method: 需要执行的目标方法
     * args: 目标方法的参数
     * target: 目标对象
     */
    void afterReturning(@Nullable Object returnValue, Method method, Object[]
    args, @Nullable Object target) throws Throwable;

}
```

如同

```

public Object invoke(){
    Object retVal = 调用目标方法;
    调用AfterReturningAdvice#afterReturning方法
    return retVal;
}

```

ThrowsAdvice接口

```

package org.springframework.aop;

public interface ThrowsAdvice extends AfterAdvice {

}

```

此接口上没有任何方法，因为方法由反射调用，实现类必须实现以下形式的方法，前3个参数是可选的，最后一个参数为需要匹配的异常的类型。

```

void afterThrowing([Method, args, target], ThrowableSubclass);

```

有效方法的一些例子如下：

```

public void afterThrowing(Exception ex)
public void afterThrowing(RemoteException)
public void afterThrowing(Method method, Object[] args, Object target, Exception
ex)
public void afterThrowing(Method method, Object[] args, Object target,
ServletException ex)

```

MethodInterceptor接口

方法拦截器，这个接口最强大，可以实现上面3种类型的通知，上面3种通知最终都通过适配模式将其转换为MethodInterceptor方式去执行。

```

package org.aopalliance.intercept;

@FunctionalInterface
public interface MethodInterceptor extends Interceptor {

    /**
     * 拦截目标方法的执行，可以在这个方法内部实现需要增强的逻辑，以及主动调用目标方法
     */
    Object invoke(MethodInvocation invocation) throws Throwable;

}

```

使用方式如：

```

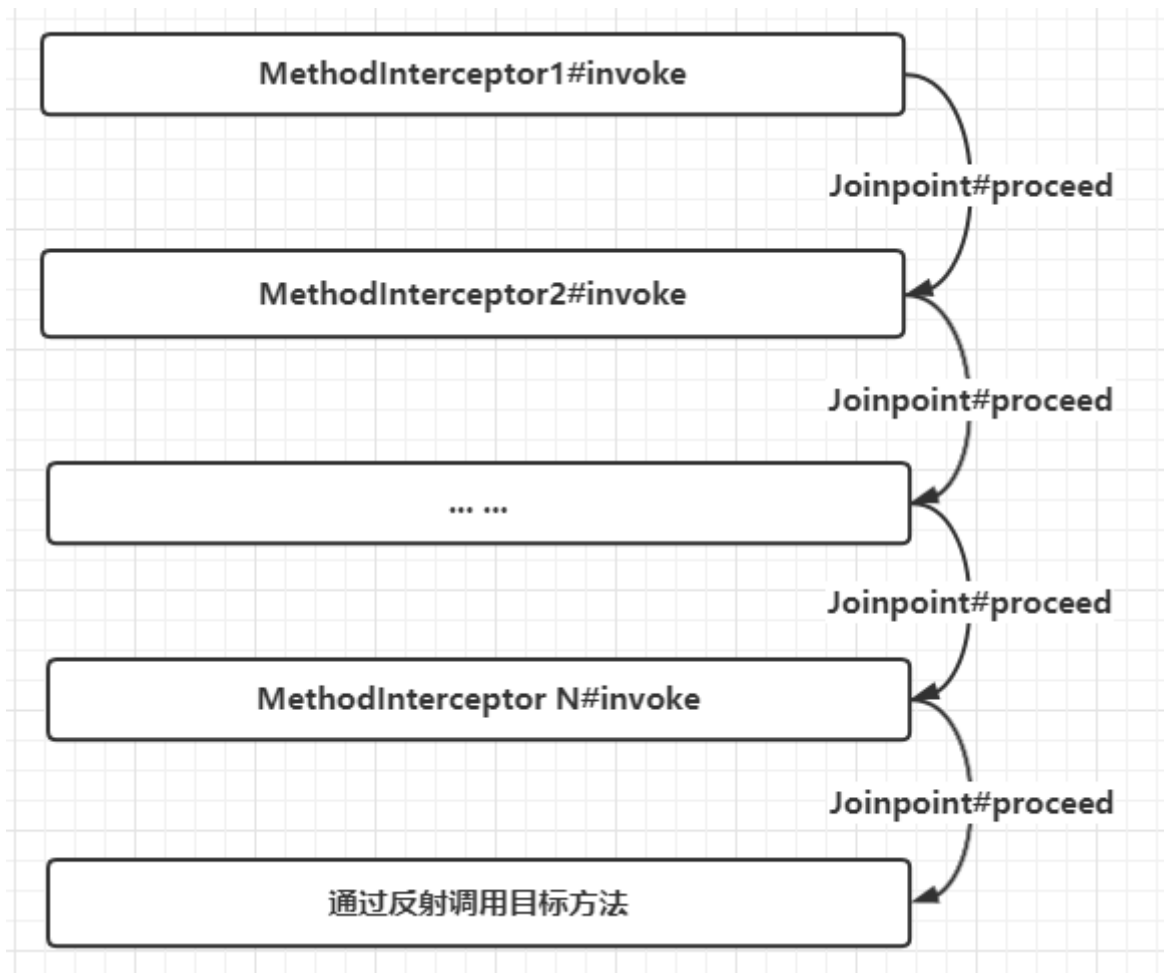
public class TracingInterceptor implements MethodInterceptor {
    Object invoke(MethodInvocation i) throws Throwable {
        System.out.println("method "+i.getMethod()+" is called on "+
            i.getThis()+" with args "+i.getArguments());
        Object ret=i.proceed();//转到拦截器链中的下一个拦截器
        System.out.println("method "+i.getMethod()+" returns "+ret);
        return ret;
    }
}

```

拦截器链

一个目标方法中可以添加很多Advice，这些Advice最终都会被转换为 `MethodInterceptor` 类型的方法拦截器，最终会有多个 `MethodInterceptor`，这些 `MethodInterceptor` 会组成一个方法调用链。

Aop内部会给目标对象创建一个代理，代理对象中会放入这些 `MethodInterceptor` 会组成一个方法调用链，当调用代理对象的方法的时候，会按顺序执行这些方法调用链，一个个执行，最后会通过反射再去调用目标方法，进而对目标方法进行增强。



切入点(PointCut)

通知 (Advice) 用来指定需要增强的逻辑，但是哪些类的哪些方法中需要使用这些通知呢？这个就是通过切入点来配置的，切入点在spring中对应了一个接口

PointCut接口

```
package org.springframework.aop;

public interface Pointcut {

    /**
     * 类过滤器，可以知道哪些类需要拦截
     */
    ClassFilter getClassFilter();

    /**
     * 方法匹配器，可以知道哪些方法需要拦截
     */
    MethodMatcher getMethodMatcher();

    /**
     * 匹配所有对象的 Pointcut，内部的2个过滤器默认都会返回true
     */
    Pointcut TRUE = TruePointcut.INSTANCE;

}
```

ClassFilter接口

比较简单，用来过滤类的

```
@FunctionalInterface
public interface ClassFilter {

    /**
     * 用来判断目标类型是否匹配
     */
    boolean matches(Class<?> clazz);

}
```

MethodMatcher接口

用来过滤方法的。

```
public interface MethodMatcher {

    /**
     * 执行静态检查给定方法是否匹配
     * @param method 目标方法
     * @param targetClass 目标对象类型
     */
    boolean matches(Method method, Class<?> targetClass);

    /**
     * 是否是动态匹配，即是否每次执行目标方法的时候都去验证一下
     */
    boolean isRuntime();

}
```

```

/**
 * 动态匹配验证的方法，比第一个matches方法多了一个参数args，这个参数是调用目标方法传入的
 * 参数
 */
boolean matches(Method method, Class<?> targetClass, Object... args);

/**
 * 匹配所有方法，这个内部的2个matches方法任何时候都返回true
 */
MethodMatcher TRUE = TrueMethodMatcher.INSTANCE;

}

```

我估计大家看MethodMatcher还是有点晕的，为什么需要2个matches方法？什么是动态匹配？

比如下面一个类

```

public class UserService{
    public void work(String userName){
        System.out.print(userName+"开始工作了!");
    }
}

```

work方法表示当前用户的工作方法，内部可以实现一些工作的逻辑。

我们希望通过aop对这个类进行增强，调用这个方法的时候，当传入的用户名是路人的粉丝的时候，需要先进行问候，其他用户的时候，无需问候，将这个问题的代码可以放在MethodBeforeAdvice中实现，这种情况就是当参数满足一定的条件了，才会使用这个通知，不满足的时候，通知无效，此时就可以使用上面的动态匹配来实现，MethodMatcher类中3个参数的matches方法可以用来对目标方法的参数做校验。

来看一下MethodMatcher过滤的整个过程

- 1.调用matches(Method method, Class<?> targetClass)方法，验证方法是否匹配
- 2.isRuntime方法是否为true，如果为false，则以第一步的结果为准，否则继续向下
- 3.调用matches(Method method, Class<?> targetClass, Object... args)方法继续验证，这个方法多了一个参数，可以对目标方法传入的参数进行校验。

通过上面的过程，大家可以看出来，如果isRuntime为false的时候，只需要对方法名称进行校验，当目标方法调用多次的时候，实际上第一步的验证结果是一样的，所以如果isRuntime为false的情况，可以将验证结果放在缓存中，提升效率，而spring内部就是这么做的，isRuntime为true的时候，需要每次都进行校验，效率会低一些，不过对性能的影响基本上可以忽略。

顾问(Advisor)

通知定义了需要做什么，切入点定义了在哪些类的哪些方法中执行通知，那么需要将他们2个组合起来才有效啊。

顾问 (Advisor) 就是做这个事情的

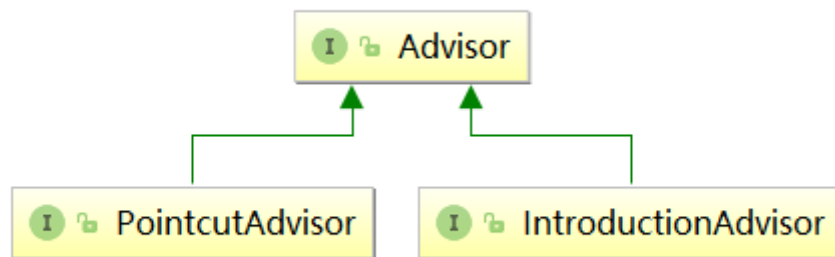
Advisor接口

```
package org.springframework.aop;

import org.aopalliance.aop.Advice;

/**
 * 包含AOP通知（在joinpoint处执行的操作）和确定通知适用性的过滤器（如切入点[PointCut]）的基本接口。
 * 这个接口不是供Spring用户使用的，而是为了支持不同类型的建议的通用性。
 */
public interface Advisor {
    /**
     * 返回引用的通知
     */
    Advice getAdvice();
}
```

上面这个接口通常不会直接使用，这个接口有2个子接口，通常我们会和这2个子接口来打交道，下面看一下这2个子接口。



PointcutAdvisor接口

通过名字就能看出来，这个和Pointcut有关，内部有个方法用来获取 `Pointcut`，AOP使用到的大部分Advisor都属于这种类型的。

在目标方法中实现各种增强功能基本上都是通过PointcutAdvisor来实现的。

```
package org.springframework.aop;

/**
 * 切入点类型的Advisor
 */
public interface PointcutAdvisor extends Advisor {

    /**
     * 获取顾问中使用的切入点
     */
    Pointcut getPointcut();
}
```


IntroductionAdvisor接口

这个接口，估计大家比较陌生，干什么的呢？

一个Java类，没有实现A接口，在不修改Java类的情况下，使其具备A接口的功能。可以通过IntroductionAdvisor给目标类引入更多接口的功能，这个功能是不是非常牛逼。

案例

上面都是一些概念，看起来比较枯燥乏味，下面来个使用硬编码的方式来用一下上面提到的一些类或者接口，加深理解。

来个类

```
package com.javacode2018.aop.demo3;

public class UserService {
    public void work(String userName) {
        System.out.println(userName + ",正在和路人甲java一起学Spring Aop,欢迎大家一起来!");
    }
}
```

下面通过aop来实现一些需求，对work方法进行增强。

案例1

需求：在work方法执行之前，打印一句：你好：userName

下面直接上代码，注释比较详细，就不细说了。

```
@Test
public void test1() {
    //定义目标对象
    UserService target = new UserService();
    //创建pointcut, 用来拦截UserService中的work方法
    Pointcut pointcut = new Pointcut() {
        @Override
        public ClassFilter getClassFilter() {
            //判断是否是UserService类型的
            return clazz -> UserService.class.isAssignableFrom(clazz);
        }

        @Override
        public MethodMatcher getMethodMatcher() {
            return new MethodMatcher() {
                @Override
                public boolean matches(Method method, Class<?> targetClass) {
                    //判断方法名称是否是work
                    return "work".equals(method.getName());
                }
            };
        }

        @Override
        public boolean isRuntime() {
            return false;
        }
    };
}
```

```

    }

    @Override
    public boolean matches(Method method, Class<?> targetClass,
Object... args) {
        return false;
    }
};

};

//创建通知，此处需要在方法之前执行操作，所以需要用到MethodBeforeAdvice类型的通知
MethodBeforeAdvice advice = (method, args, target1) -> System.out.println("你好:" + args[0]);

//创建Advisor，将pointcut和advice组装起来
DefaultPointcutAdvisor advisor = new DefaultPointcutAdvisor(pointcut,
advice);

//通过spring提供的代理创建工厂来创建代理
ProxyFactory proxyFactory = new ProxyFactory();
//为工厂指定目标对象
proxyFactory.setTarget(target);
//调用addAdvisor方法，为目标添加增强的功能，即添加Advisor，可以为目标添加很多个Advisor
proxyFactory.addAdvisor(advisor);
//通过工厂提供的方法来生成代理对象
UserService userServiceProxy = (UserService) proxyFactory.getProxy();

//调用代理的work方法
userServiceProxy.work("路人");
}

```

运行输出

```

你好:路人
路人,正在和路人甲java一起学Spring Aop,欢迎大家一起来!

```

上面是采用硬编码的方式来感受一下aop的用法，大家看了上面代码之后，估计会有疑问：我晕，这么复杂？？？

如果大家有使用过spring中的aop经验，可能只需要几行代码就实现了上面的功能，的确，spring中把整个功能简化了很多，不过我们得去了解他的内部是如何实现的，然后才能走的更远。

案例2

需求：统计一下work方法的耗时，将耗时输出

```

@Test
public void test2() {
    //定义目标对象
    UserService target = new UserService();
    //创建pointcut，用来拦截UserService中的work方法
    Pointcut pointcut = new Pointcut() {
        @Override
        public ClassFilter getClassFilter() {
            //判断是否是UserService类型的
            return clazz -> UserService.class.isAssignableFrom(clazz);
        }
    };
}

```

```

    }

    @Override
    public MethodMatcher getMethodMatcher() {
        return new MethodMatcher() {
            @Override
            public boolean matches(Method method, Class<?> targetClass) {
                //判断方法名称是否是work
                return "work".equals(method.getName());
            }

            @Override
            public boolean isRuntime() {
                return false;
            }

            @Override
            public boolean matches(Method method, Class<?> targetClass,
object... args) {
                return false;
            }
        };
    }
};

//创建通知，需要拦截方法的执行，所以需要用到MethodInterceptor类型的通知
MethodInterceptor advice = new MethodInterceptor() {
    @Override
    public Object invoke(MethodInvocation invocation) throws Throwable {
        System.out.println("准备调用:" + invocation.getMethod());
        long startTime = System.nanoTime();
        Object result = invocation.proceed();
        long endTime = System.nanoTime();
        System.out.println(invocation.getMethod() + ", 调用结束!");
        System.out.println("耗时(纳秒):" + (endTime - startTime));
        return result;
    }
};

//创建Advisor，将pointcut和advice组装起来
DefaultPointcutAdvisor advisor = new DefaultPointcutAdvisor(pointcut,
advice);

//通过spring提供的代理创建工厂来创建代理
ProxyFactory proxyFactory = new ProxyFactory();
//为工厂指定目标对象
proxyFactory.setTarget(target);
//调用addAdvisor方法，为目标添加增强的功能，即添加Advisor，可以为目标添加很多个Advisor
proxyFactory.addAdvisor(advisor);
//通过工厂提供的方法来生成代理对象
UserService userServiceProxy = (UserService) proxyFactory.getProxy();

//调用代理的work方法
userServiceProxy.work("路人");
}

```

运行输出

准备调用:`public void com.javacode2018.aop.demo3.UserService.work(java.lang.String)`
路人,正在和路人甲java一起学Spring Aop,欢迎大家一起来!
`public void com.javacode2018.aop.demo3.UserService.work(java.lang.String)`,调用结束!
耗时(纳秒):9526200

案例3

需求: userName中包含“粉丝”关键字,输出一句:感谢您一路的支持

此处需要用到 MethodMatcher 中的动态匹配了,通过参数来进行判断。

重点在于Pointcut中的getMethodMatcher方法,返回的MethodMatcher, @1必须返回true,此时才会进入到 @2 中对参数进行校验。

代码如下:

```
@Test
public void test2() {
    //定义目标对象
    UserService target = new UserService();
    //创建pointcut, 用来拦截UserService中的work方法
    Pointcut pointcut = new Pointcut() {
        @Override
        public ClassFilter getClassFilter() {
            //判断是否是UserService类型的
            return clazz -> UserService.class.isAssignableFrom(clazz);
        }

        @Override
        public MethodMatcher getMethodMatcher() {
            return new MethodMatcher() {
                @Override
                public boolean matches(Method method, Class<?> targetClass) {
                    //判断方法名称是否是work
                    return "work".equals(method.getName());
                }

                @Override
                public boolean isRuntime() {
                    return true; // @1: 注意这个地方要返回true
                }

                @Override
                public boolean matches(Method method, Class<?> targetClass,
Object... args) {
                    // @2: isRuntime为true的时候, 会执行这个方法
                    if (Objects.nonNull(args) && args.length == 1) {
                        String userName = (String) args[0];
                        return userName.contains("粉丝");
                    }
                    return false;
                }
            };
        }
    };

    //创建通知, 此处需要在方法之前执行操作, 所以需要用到MethodBeforeAdvice类型的通知
```

```
MethodBeforeAdvice advice = (method, args, target1) -> System.out.println("感谢您一路的支持!");

//创建Advisor, 将pointcut和advice组装起来
DefaultPointcutAdvisor advisor = new DefaultPointcutAdvisor(pointcut, advice);

//通过spring提供的代理创建工厂来创建代理
ProxyFactory proxyFactory = new ProxyFactory();
//为工厂指定目标对象
proxyFactory.setTarget(target);
//调用addAdvisor方法, 为目标添加增强的功能, 即添加Advisor, 可以为目标添加很多个Advisor
proxyFactory.addAdvisor(advisor);
//通过工厂提供的方法来生成代理对象
UserService userServiceProxy = (UserService) proxyFactory.getProxy();

//调用代理的work方法
userServiceProxy.work("粉丝: A");
}
```

运行输出

```
感谢您一路的支持!
粉丝: A, 正在和路人甲java一起学Spring Aop, 欢迎大家一起来!
```

本文案例代码入口

```
com.javacode2018.aop.demo3.Test3
```

上面的一些案例中都用到了 `ProxyFactory` 这个类, 内部将各种对象进行组装, 然后创建代理对象, `ProxyFactory` 这块关联的东西挺多的, 下一篇文章将详说这块的东西, 是非常重要的内容。

课后问题

对上面案例进行改造, 实现下面需求:

1. work方法执行之后, 打印一句: 再见: userName
2. 在work方法中抛出一个异常, 然后通过aop中的ThrowsAdvice类型的通知来拦截这个异常信息, 然后将异常错误信息打印出来

欢迎大家留言。

总结

这篇文章中主要介绍了一些概念性的东西, 不是太好理解, 建议大家多看几遍, 有关的类和接口大家也去看看源码, 理解起来也会容易一些, 也欢迎大家留言一起交流这块的东西。

案例源码

<https://gitee.com/javacode2018/spring-series>

路人甲java所有案例代码以后都会放到这个上面，大家watch一下，可以持续关注动态。

Spring系列

1. [Spring系列第1篇：为何要学spring?](#)
2. [Spring系列第2篇：控制反转（IoC）与依赖注入（DI）](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)
4. [Spring系列第4篇：xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇：创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇：玩转bean scope，避免跳坑里！](#)
7. [Spring系列第7篇：依赖注入之手动注入](#)
8. [Spring系列第8篇：自动注入（autowire）详解，高手在于坚持](#)
9. [Spring系列第9篇：depend-on到底是干什么的?](#)
10. [Spring系列第10篇：primary可以解决什么问题?](#)
11. [Spring系列第11篇：bean中的autowire-candidate又是干什么的?](#)
12. [Spring系列第12篇：lazy-init：bean延迟初始化](#)
13. [Spring系列第13篇：使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇：lookup-method和replaced-method比较陌生，怎么玩的?](#)
15. [Spring系列第15篇：代理详解（Java动态代理&cglib代理）?](#)
16. [Spring系列第16篇：深入理解java注解及spring对注解的增强（预备知识）](#)
17. [Spring系列第17篇：@Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇：@ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇：@import详解\(bean批量注册\)](#)
20. [Spring系列第20篇：@Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇：注解实现依赖注入（@Autowired、@Resource、@Primary、@Qualifier）](#)
22. [Spring系列第22篇：@Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇：Bean生命周期详解](#)
24. [Spring系列第24篇：父子容器详解](#)
25. [Spring系列第25篇：@Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇：国际化详解](#)
27. [Spring系列第27篇：spring事件机制详解](#)
28. [Spring系列第28篇：Bean循环依赖详解](#)
29. [Spring系列第29篇：BeanFactory扩展（BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor）](#)
30. [Spring系列第30篇：jdk动态代理和cglib代理](#)

更多好文章

1. [Java高并发系列（共34篇）](#)
2. [MySQL高手系列（共27篇）](#)
3. [Maven高手系列（共10篇）](#)
4. [Mybatis系列（共12篇）](#)
5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要，它是什么？怎么实现？](#)
7. [泛型，有点难度，会让很多人懵逼，那是因为你没有看这篇文章！](#)



领取福利，扫码发送java

- 1、1000本java电子书大全
- 2、2021版java面试通关手册
- 3、月薪5万必备的java微服务学习线路图及高清视频
- 4、邀请加入高质量java技术交流群

加我微信：spring888java，进群一起交流java相关各种技术！！

Spring系列第32篇：AOP核心源码、原理详解

本文目标：带领大家阅读aop的源码，深入理解aop的原理，内容有点长，消化需要大概一周时间，做好准备

阅读本文之前，需要先掌握下面3篇文章内容，不然会比较吃力。

1. [Spring系列第15篇：代理详解（java动态代理&CGLIB代理）](#)
2. [Spring系列第30篇：jdk动态代理和cglib代理](#)
3. [Spring系列第31篇：Aop概念详解](#)

本文主要分4部分

1. Aop原理介绍
2. 介绍aop相关的一些类
3. 通过源码详解aop代理的创建过程
4. 通过源码详解aop代理的调用过程
5. Aop代理一些特性的使用案例

Spring AOP原理

原理比较简单，主要就是使用jdk动态代理和cglib代理来创建代理对象，通过代理对象来访问目标对象，而代理对象中融入了增强的代码，最终起到对目标对象增强的效果。

aop相关的一些类

1. 连接点 (JoinPoint) 相关类
2. 通知 (Advice) 相关的类
3. 切入点 (Pointcut) 相关的类
4. 切面 (Advisor) 相关的类

连接点 (JoinPoint) 相关类

JoinPoint接口

这个接口表示一个通用的运行时连接点（在AOP术语中）

```
package org.aopalliance.intercept;

public interface Joinpoint {

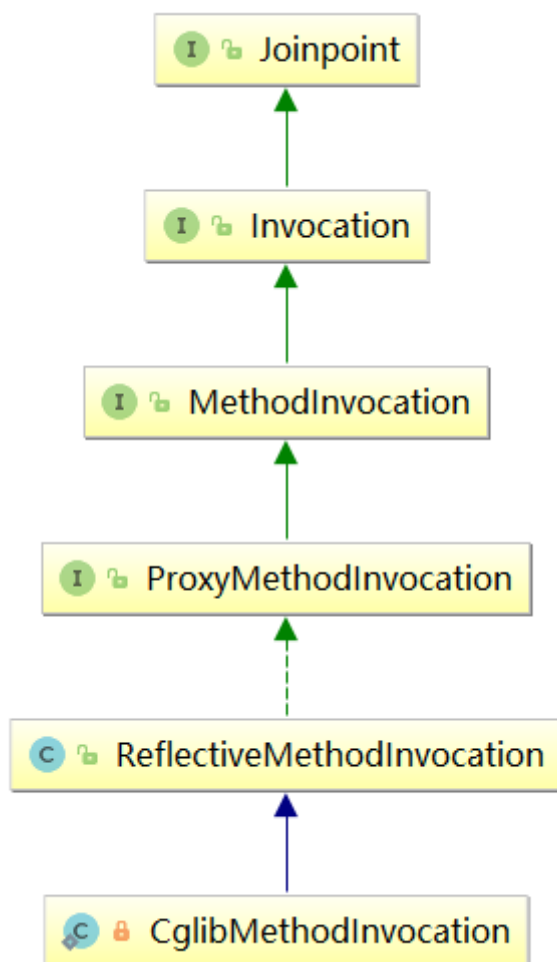
    /**
     * 转到拦截器链中的下一个拦截器
     */
    Object proceed() throws Throwable;

    /**
     * 返回保存当前连接点静态部分【的对象】，这里一般指被代理的目标对象
     */
    Object getThis();

    /**
     * 返回此静态连接点 一般就为当前的Method(至少目前的唯一实现是MethodInvocation,所以连接点得静态部分肯定就是本方法)
     */
    AccessibleObject getStaticPart();

}
```

几个重要的子接口和实现类，如下：



Invocation接口

此接口表示程序中的调用，调用是一个连接点，可以被拦截器拦截。

```
package org.aopalliance.intercept;

/**
 * 此接口表示程序中的调用
 * 调用是一个连接点，可以被拦截器拦截。
 */
public interface Invocation extends Joinpoint {

    /**
     * 将参数作为数组对象获取，可以更改此数组中的元素值以更改参数。
     * 通常用来获取调用目标方法的参数
     */
    Object[] getArguments();
}
```

MethodInvocation接口

用来表示连接点中方法的调用，可以获取调用过程中的目标方法。

```
package org.aopalliance.intercept;

import java.lang.reflect.Method;

/**
 * 方法调用的描述，在方法调用时提供给拦截器。
 * 方法调用是一个连接点，可以被方法拦截器拦截。
 */
public interface MethodInvocation extends Invocation {

    /**
     * 返回正在被调用得方法~~~ 返回的是当前Method对象。
     * 此时，效果同父类的AccessibleObject getStaticPart() 这个方法
     */
    Method getMethod();
}
```

ProxyMethodInvocation接口

表示代理方法的调用

```
public interface ProxyMethodInvocation extends MethodInvocation {

    /**
     * 获取被调用的代理对象
     */
    Object getProxy();

    /**
     * 克隆一个方法调用器MethodInvocation
     */
}
```

```

    */
    MethodInvocation invocableClone();

    /**
     * 克隆一个方法调用器MethodInvocation，并为方法调用器指定参数
     */
    MethodInvocation invocableClone(Object... arguments);

    /**
     * 设置要用于此链中任何通知的后续调用的参数。
     */
    void setArguments(Object... arguments);

    /**
     * 添加一些扩展用户属性，这些属性不在AOP框架内使用。它们只是作为调用对象的一部分保留，用于
     特殊的拦截器。
     */
    void setUserAttribute(String key, @Nullable Object value);

    /**
     * 根据key获取对应的用户属性
     */
    @Nullable
    Object getUserAttribute(String key);
}

```

通俗点理解：连接点表示方法的调用过程，内部包含了方法调用过程中的所有信息，比如被调用的方法、目标、代理对象、执行拦截器链等信息。

上面定义都是一些接口，最终有2个实现。

ReflectiveMethodInvocation

当代理对象是采用jdk动态代理创建的，通过代理对象来访问目标对象的方法的时，最终过程是由ReflectiveMethodInvocation来处理的，内部会通过递归调用方法拦截器，最终会调用到目标方法。

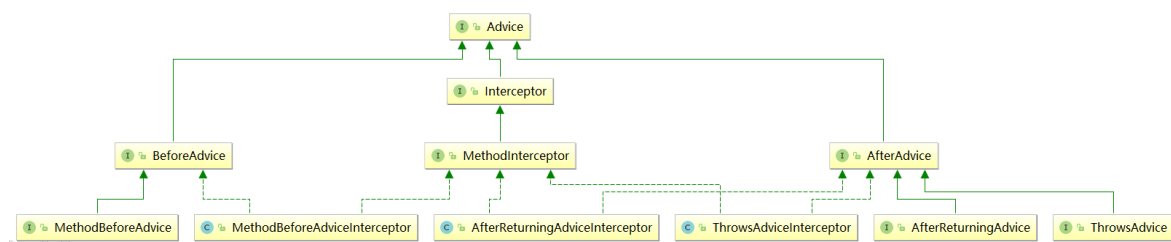
CglibMethodInvocation

功能和上面的类似，当代理对象是采用cglib创建的，通过代理对象来访问目标对象的方法的时，最终过程是由CglibMethodInvocation来处理的，内部会通过递归调用方法拦截器，最终会调用到目标方法。

这2个类源码稍后详解。

通知相关的类

通知用来定义需要增强的逻辑。



Advice接口

通知的底层接口

```
package org.aopalliance.aop;
public interface Advice {
}
```

BeforeAdvice接口

方法前置通知，内部空的

```
package org.springframework.aop;

public interface BeforeAdvice extends Advice {
}
```

Interceptor接口

此接口表示通用拦截器

```
package org.aopalliance.intercept;

public interface Interceptor extends Advice {
}
```

MethodInterceptor接口

方法拦截器，所有的通知均需要转换为MethodInterceptor类型的，最终多个MethodInterceptor组成一个方法拦截器连。

```
package org.aopalliance.intercept;

@FunctionalInterface
public interface MethodInterceptor extends Interceptor {
    /**
     * 拦截目标方法的执行，可以在这个方法内部实现需要增强的逻辑，以及主动调用目标方法
     */
    Object invoke(MethodInvocation invocation) throws Throwable;
}
```

AfterAdvice接口

后置通知的公共标记接口

```
package org.springframework.aop;

public interface AfterAdvice extends Advice {
}
```

MethodBeforeAdvice接口

方法执行前通知，需要在目标方法执行前执行一些逻辑的，可以通过这个实现。

通俗点说：需要在目标方法执行之前增强一些逻辑，可以通过这个接口来实现。before方法：在调用给定方法之前回调。

```
package org.springframework.aop;

public interface MethodBeforeAdvice extends BeforeAdvice {

    /**
     * 调用目标方法之前会先调用这个before方法
     * method: 需要执行的目标方法
     * args: 目标方法的参数
     * target: 目标对象
     */
    void before(Method method, Object[] args, @Nullable Object target) throws
    Throwable;
}
```

如同

```
public Object invoke(){
    调用MethodBeforeAdvice#before方法
    return 调用目标方法;
}
```

AfterReturningAdvice接口

方法执行后通知，需要在目标方法执行之后执行增强一些逻辑的，可以通过这个实现。

不过需要注意一点：目标方法正常执行后，才会回调这个接口，当目标方法有异常，那么这通知会被跳过。

```
package org.springframework.aop;

public interface AfterReturningAdvice extends AfterAdvice {

    /**
     * 目标方法执行之后会回调这个方法
     * method: 需要执行的目标方法
     * args: 目标方法的参数
     * target: 目标对象
     */
    void afterReturning(@Nullable Object returnValue, Method method, Object[]
    args, @Nullable Object target) throws Throwable;

}
```

如同

```
public Object invoke(){
    Object retVal = 调用目标方法;
    调用AfterReturningAdvice#afterReturning方法
    return retVal;
}
```

ThrowsAdvice接口

```
package org.springframework.aop;

public interface ThrowsAdvice extends AfterAdvice {

}
```

此接口上没有任何方法，因为方法由反射调用，实现类必须实现以下形式的方法，前3个参数是可选的，最后一个参数为需要匹配的异常的类型。

```
void afterThrowing([Method, args, target], ThrowableSubclass);
```

有效方法的一些例子如下：

```
public void afterThrowing(Exception ex)
public void afterThrowing(RemoteException)
public void afterThrowing(Method method, Object[] args, Object target, Exception
ex)
public void afterThrowing(Method method, Object[] args, Object target,
ServletException ex)
```

通知包装器

负责将各种非MethodInterceptor类型的通知(Advice)包装为MethodInterceptor类型。

刚才有说过：Aop中所有的Advice最终都会转换为MethodInterceptor类型的，组成一个方法调用链，然后执行

3个包装器类

- MethodBeforeAdviceInterceptor
- AfterReturningAdviceInterceptor
- ThrowsAdviceInterceptor

MethodBeforeAdviceInterceptor类

这个类实现了MethodInterceptor接口，负责将MethodBeforeAdvice方法前置通知包装为MethodInterceptor类型，创建这个类型的对象的时候需要传递一个MethodBeforeAdvice类型的参数，重点是invoke方法

```
package org.springframework.aop.framework.adapter;

@SuppressWarnings("serial")
public class MethodBeforeAdviceInterceptor implements MethodInterceptor,
BeforeAdvice, Serializable {

    private final MethodBeforeAdvice advice;
```

```

public MethodBeforeAdviceInterceptor(MethodBeforeAdvice advice) {
    Assert.notNull(advice, "Advice must not be null");
    this.advice = advice;
}

@Override
public Object invoke(MethodInvocation mi) throws Throwable {
    //负责调用前置通知的方法
    this.advice.before(mi.getMethod(), mi.getArguments(), mi.getThis());
    //继续执行方法调用链
    return mi.proceed();
}
}

```

AfterReturningAdviceInterceptor类

这个类实现了 `MethodInterceptor` 接口，负责将 `AfterReturningAdvice` 方法后置通知包装为 `MethodInterceptor` 类型，创建这个类型的对象的时候需要传递一个 `AfterReturningAdvice` 类型的参数，重点是 `invoke` 方法

```

public class AfterReturningAdviceInterceptor implements MethodInterceptor,
AfterAdvice, Serializable {

    private final AfterReturningAdvice advice;

    public AfterReturningAdviceInterceptor(AfterReturningAdvice advice) {
        Assert.notNull(advice, "Advice must not be null");
        this.advice = advice;
    }

    @Override
    public Object invoke(MethodInvocation mi) throws Throwable {
        //先执行方法调用链,可以获取目标方法的执行结果
        Object retVal = mi.proceed();
        //执行后置通知
        this.advice.afterReturning(retVal, mi.getMethod(), mi.getArguments(),
mi.getThis());
        //返回结果
        return retVal;
    }
}

```

ThrowsAdviceInterceptor类

这个类实现了 `MethodInterceptor` 接口，负责将 `ThrowsAdvice` 异常通知包装为 `MethodInterceptor` 类型，创建这个类型的对象的时候需要传递一个 `Object` 类型的参数，通常这个参数是 `ThrowsAdvice` 类型的，重点是 `invoke` 方法

```

package org.springframework.aop.framework.adapter;

```

```

public class ThrowsAdviceInterceptor implements MethodInterceptor, AfterAdvice {

    private static final String AFTER_THROWING = "afterThrowing";

    private final Object throwsAdvice;

    //创建ThrowsAdviceInterceptor
    public ThrowsAdviceInterceptor(Object throwsAdvice) {
        Assert.notNull(throwsAdvice, "Advice must not be null");
        this.throwsAdvice = throwsAdvice;
        //获取异常通知中定义的所有方法（public、默认的、protected、private）
        Method[] methods = throwsAdvice.getClass().getMethods();
        //轮询methods
        for (Method method : methods) {
            //方法名称为afterThrowing && 方法参数为1或者4
            if (method.getName().equals(AFTER_THROWING) &&
                (method.getParameterCount() == 1 ||
method.getParameterCount() == 4)) {
                //获取方法的最后一个参数类型
                Class<?> throwableParam = method.getParameterTypes()
[method.getParameterCount() - 1];
                //判断方法参数类型是不是Throwable类型的
                if (Throwable.class.isAssignableFrom(throwableParam)) {
                    // 缓存异常处理方法到map中（异常类型->异常处理方法）
                    this.exceptionHandlerMap.put(throwableParam, method);
                }
            }
        }
        //如果exceptionHandlerMap，抛出异常，所以最少要有一个异常处理方法
        if (this.exceptionHandlerMap.isEmpty()) {
            throw new IllegalArgumentException(
                "At least one handler method must be found in class [" +
throwsAdvice.getClass() + "]");
        }
    }

    /**
     * 获取异常通知中自定义的处理异常方法的数量
     */
    public int getHandlerMethodCount() {
        return this.exceptionHandlerMap.size();
    }

    @Override
    public Object invoke(MethodInvocation mi) throws Throwable {
        try {
            //调用通知链
            return mi.proceed();
        }
        catch (Throwable ex) {
            //获取异常通知中自定义的处理异常的方法
            Method handlerMethod = getExceptionHandler(ex);
            //当处理的方法不为空
            if (handlerMethod != null) {
                //调用异常处理方法
                invokeHandlerMethod(mi, ex, handlerMethod);
            }
        }
    }

```

```

        }
        //继续向外抛出异常
        throw ex; //@1
    }
}

/**
 * 获取throwsAdvice中处理exception参数指定的异常的方法
 */
@Nullable
private Method getExceptionHandler(Throwable exception) {
    //获取异常类型
    Class<?> exceptionClass = exception.getClass();
    //从缓存中获取异常类型对应的方法
    Method handler = this.exceptionHandlerMap.get(exceptionClass);
    //来一个循环，查询处理方法，循环条件：方法为空 && 异常类型!=Throwable
    while (handler == null && exceptionClass != Throwable.class) {
        //获取异常的父类型
        exceptionClass = exceptionClass.getSuperclass();
        //从缓存中查找异常对应的处理方法
        handler = this.exceptionHandlerMap.get(exceptionClass);
    }
    //将查找结果返回
    return handler;
}

//通过反射调用异常通知中的异常方法
private void invokeHandlerMethod(MethodInvocation mi, Throwable ex, Method
method) throws Throwable {
    //构建方法请求参数
    Object[] handlerArgs;
    //若只有1个参数，参数为：异常对象
    if (method.getParameterCount() == 1) {
        handlerArgs = new Object[] {ex};
    }
    else {
        //4个参数（方法、方法请求参数、目标对象、异常对象）
        handlerArgs = new Object[] {mi.getMethod(), mi.getArguments(),
mi.getThis(), ex};
    }
    try {
        //通过反射调用异常通知中的方法
        method.invoke(this.throwsAdvice, handlerArgs);
    }
    catch (InvocationTargetException targetEx) {
        throw targetEx.getTargetException();
    }
}
}
}

```

从上面可以看出，异常通知，自定义处理异常的方法有几个特点

1. 方法名称必须为 `afterThrowing`
2. 方法参数必须1个或4个，最后一个参数是 `Throwable` 类型或其子类型
3. 可以在异常处理中记录一些异常信息，这个还是比较有用的，但是注意一点目标方法抛出的异常最后还是会向外继续抛出 `@1`

光讲源码，大家看着枯燥乏味，来点案例。

先来一个类，用来模拟用户资金操作：充值、提现、查询资金余额；提现的时候余额不足的时候，会抛出异常。

```
package com.javacode2018.aop.demo4;

//模拟资金操作
public class FundService {
    //账户余额
    private double balance = 1000;

    //模拟提现
    double recharge(String userName, double price) {
        System.out.println(String.format("%s提现%s", userName, price));
        balance += price;
        return balance;
    }

    //模拟提现
    double cashOut(String userName, double price) {
        if (balance < price) {
            throw new RuntimeException("余额不足!");
        }
        System.out.println(String.format("%s提现%s", userName, price));
        balance -= price;
        return balance;
    }

    //获取余额
    double getBalance(String userName) {
        return balance;
    }
}
```

案例1：前置通知拦截非法访问

资金操作的所有方法都需要验证用户名，当用户名不是“路人”的时候，直接抛出非法访问异常。

```
package com.javacode2018.aop.demo4;

import org.junit.Test;
import org.springframework.aop.MethodBeforeAdvice;
import org.springframework.aop.framework.ProxyFactory;
import org.springframework.lang.Nullable;

import java.lang.reflect.Method;

public class AopTest4 {

    @Test
```

```

public void test1() {
    //代理工厂
    ProxyFactory proxyFactory = new ProxyFactory(new FundsService());
    //添加一个方法前置通知，判断用户名不是“路人”的时候，抛出非法访问异常
    proxyFactory.addAdvice(new MethodBeforeAdvice() {
        @Override
        public void before(Method method, Object[] args, @Nullable Object
target) throws Throwable {
            String userName = (String) args[0];
            //如果不是路人的时候，抛出非法访问异常
            if (!"路人".equals(userName)) {
                throw new RuntimeException(String.format("[%s]非法访问!",
userName));
            }
        }
    });
    //通过代理工厂创建代理
    FundsService proxy = (FundsService) proxyFactory.getProxy();
    //调用代理的方法
    proxy.recharge("路人", 100);
    proxy.recharge("张学友", 100);
}
}

```

运行输出

路人提现100.0

java.lang.RuntimeException: [张学友]非法访问!

```

    at com.javacode2018.aop.demo4.AopTest4$1.before(AopTest4.java:25)
    at
org.springframework.aop.framework.adapter.MethodBeforeAdviceInterceptor.invoke(M
ethodBeforeAdviceInterceptor.java:55)

```

案例2：通过异常通知记录异常

通过异常通知来捕获所有方法的运行，发现异常之后，通知开发修复bug。

```

public static class SendMsgThrowsAdvice implements ThrowsAdvice {
    //注意方法名称必须为afterThrowing
    public void afterThrowing(Method method, Object[] args, Object target,
RuntimeException e) {
        //监控到异常后发送消息通知开发者
        System.out.println("异常警报: ");
        System.out.println(String.format("method:[%s], args:[%s]",
method.toGenericString(), Arrays.stream(args).collect(Collectors.toList())));
        System.out.println(e.getMessage());
        System.out.println("请尽快修复bug! ");
    }
}

@Test
public void test2() {
    //代理工厂
    ProxyFactory proxyFactory = new ProxyFactory(new FundsService());
}

```

```
//添加一个异常通知，发现异常之后发送消息给开发者尽快修复bug
proxyFactory.addAdvice(new SendMsgThrowsAdvice());
//通过代理工厂创建代理
FundsService proxy = (FundsService) proxyFactory.getProxy();
//调用代理的方法
proxy.cashOut("路人", 2000);
}
```

运行输出

```
异常警报:
method:[double
com.javacode2018.aop.demo4.FundsService.cashOut(java.lang.String,double)], args:
[[路人, 2000.0]]
余额不足!
请尽快修复bug!

java.lang.RuntimeException: 余额不足!

    at com.javacode2018.aop.demo4.FundsService.cashOut(FundsService.java:18)
```

切入点(PointCut)相关类

通知（Advice）用来指定需要增强的逻辑，但是哪些类的哪些方法中需要使用这些通知呢？这个就是通过切入点来配置的。

  Pointcut
  getClassFilter() ClassFilter
  getMethodMatcher() JMatcher

  ClassFilter
  matches(Class<?>) boolean

  MethodMatcher
  matches(Method, Class<?>) boolean
  isRuntime() boolean
  matches(Method, Class<?>, Object...) boolean

PointCut接口

```
package org.springframework.aop;

public interface Pointcut {

    /**
     * 类过滤器，可以知道哪些类需要拦截
     */
    ClassFilter getClassFilter();

    /**
     * 方法匹配器，可以知道哪些方法需要拦截
     */
    MethodMatcher getMethodMatcher();

    /**
```

```

        * 匹配所有对象的 Pointcut，内部的2个过滤器默认都会返回true
        */
        Pointcut TRUE = TruePointcut.INSTANCE;
    }

```

ClassFilter接口

类过滤器。

```

@FunctionalInterface
public interface ClassFilter {

    /**
     * 用来判断目标类型是否匹配
     */
    boolean matches(Class<?> clazz);

}

```

MethodMatcher接口

方法过滤器。

```

public interface MethodMatcher {

    /**
     * 执行静态检查给定方法是否匹配
     * @param method 目标方法
     * @param targetClass 目标对象类型
     */
    boolean matches(Method method, Class<?> targetClass);

    /**
     * 是否是动态匹配，即是否每次执行目标方法的时候都去验证一下
     */
    boolean isRuntime();

    /**
     * 动态匹配验证的方法，比第一个matches方法多了一个参数args，这个参数是调用目标方法传入的
     参数
     */
    boolean matches(Method method, Class<?> targetClass, Object... args);

    /**
     * 匹配所有方法，这个内部的2个matches方法任何时候都返回true
     */
    MethodMatcher TRUE = TrueMethodMatcher.INSTANCE;

}

```

我估计大家看MethodMatcher还是有点晕的，为什么需要2个**maches**方法？什么是动态匹配？

比如下面一个类

```
public class UserService{
    public void work(String userName){
        System.out.print(userName+"开始工作了!");
    }
}
```

work方法表示当前用户的工作方法，内部可以实现一些工作的逻辑。

我们希望通过aop对这个类进行增强，调用这个方法的时候，当传入的用户名是路人的粉丝的时候，需要先进行问候，其他用户的时候，无需问候，将这个问题的代码可以放在MethodBeforeAdvice中实现，这种情况就是当参数满足一定的条件了，才会使用这个通知，不满足的时候，通知无效，此时就可以使用上面的动态匹配来实现，MethodMatcher类中3个参数的matches方法可以用来对目标方法的参数做校验。

来看一下 MethodMatcher 过滤的整个过程

- 1.调用matches(Method method, Class<?> targetClass)方法，验证方法是否匹配
- 2.isRuntime方法是否为true，如果为false，则以第一步的结果为准，否则继续向下
- 3.调用matches(Method method, Class<?> targetClass, Object... args)方法继续验证，这个方法多了一个参数，可以对目标方法传入的参数进行校验。

通过上面的过程，大家可以看出来，如果isRuntime为false的时候，只需要对方法名称进行校验，当目标方法调用多次的时候，实际上第一步的验证结果是一样的，所以如果isRuntime为false的情况，可以将验证结果放在缓存中，提升效率，而spring内部就是这么做的，isRuntime为false的时候，需要每次都进行校验，效率会低一些，不过对性能的影响基本上可以忽略。

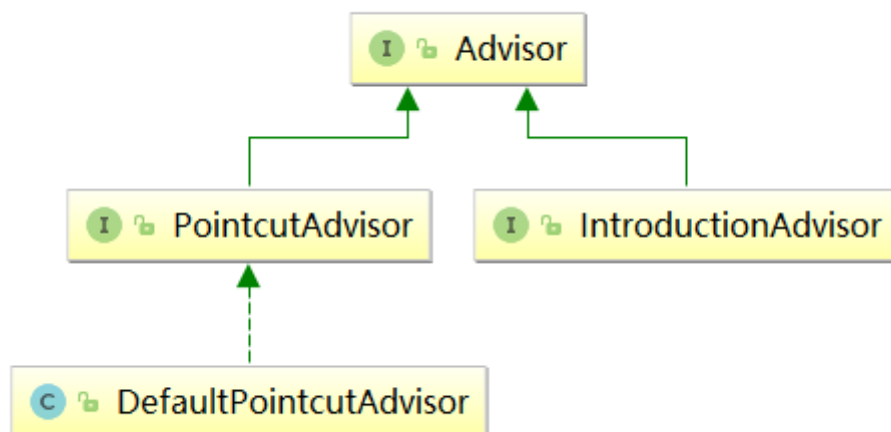
顾问(Advisor)

通知定义了需要做什么，切入点定义了在哪些类的哪些方法中执行通知，那么需要将他们2个组合起来才有效啊。

顾问 (Advisor) 就是做这个事情的。

在spring aop中，你可以将advisor理解为切面，切面中通常有2个关键信息：

1. 需要增强的目标方法列表，这个通过切入点(Pointcut)来指定
2. 需要在目标方法中增强的逻辑，这个通过(Advice)通知来指定



Advisor接口

```
package org.springframework.aop;

import org.aopalliance.aop.Advice;

/**
 * 包含AOP通知（在joinpoint处执行的操作）和确定通知适用性的过滤器（如切入点[PointCut]）的基本接口。
 * 这个接口不是供Spring用户使用的，而是为了支持不同类型的建议的通用性。
 */
public interface Advisor {
    /**
     * 返回引用的通知
     */
    Advice getAdvice();
}
```

上面这个接口通常不会直接使用，这个接口有2个子接口，通常我们会和这2个子接口来打交道，下面看一下这2个子接口。

PointcutAdvisor接口

通过名字就能看出来，这个和Pointcut有关，内部有个方法用来获取 `Pointcut`，AOP使用到的大部分Advisor都属于这种类型的。

在目标方法中实现各种增强功能基本上都是通过PointcutAdvisor来实现的。

```
package org.springframework.aop;

/**
 * 切入点类型的Advisor
 */
public interface PointcutAdvisor extends Advisor {
    /**
     * 获取顾问中使用的切入点
     */
    Pointcut getPointcut();
}
```

DefaultPointcutAdvisor类

PointcutAdvisor的默认实现，这是最常用的Advisor实现，它可以用于任何Pointcut和Advice类型，代码相当简单，里面定义了2个属性：pointcut和advisor，由使用者指定。

IntroductionAdvisor接口

这个接口，估计大家比较陌生，干什么的呢？

一个Java类，没有实现A接口，在不修改Java类的情况下，使其具备A接口的功能。可以通过IntroductionAdvisor给目标类引入更多接口的功能，这个功能是不是非常牛逼。

下面开始2个重点工作。

- 通过源码介绍aop中代理创建过程
- 通过源码介绍代理方法的调用执行过程

代理创建过程源码解析

先看一段代码

```
//代理工厂
ProxyFactory proxyFactory = new ProxyFactory(new FundsService());
//添加一个方法前置通知，判断用户名不是“路人”的时候，抛出非法访问异常
proxyFactory.addAdvice(new MethodBeforeAdvice() {
    @Override
    public void before(Method method, Object[] args, @Nullable Object target)
        throws Throwable {
        String userName = (String) args[0];
        //如果不是路人的时候，抛出非法访问异常
        if (!"路人".equals(userName)) {
            throw new RuntimeException(String.format("[%s]非法访问!", userName));
        }
    }
});
//通过代理工厂创建代理
FundsService proxy = (FundsService) proxyFactory.getProxy();
```

我们将上面代码拆分一下，变成下面这样

```
//1.创建代理所需参数配置（如：采用什么方式的代理、通知列表等）
AdvisedSupport advisedSupport = new AdvisedSupport();
//如：添加一个前置通知
advisedSupport.addAdvice(new MethodBeforeAdvice() {
    @Override
    public void before(Method method, Object[] args, @Nullable Object target)
        throws Throwable {
        String userName = (String) args[0];
        //如果不是路人的时候，抛出非法访问异常
        if (!"路人".equals(userName)) {
            throw new RuntimeException(String.format("[%s]非法访问!", userName));
        }
    }
});
//设置被代理的目标对象
FundsService target = new FundsService();
advisedSupport.setTarget(target);

//2.根据配置信息获取AopProxy对象，AopProxy用来负责创建最终的代理对象
// AopProxy接口有2个实现类（JDK动态代理、cglib代理）
// 具体最终会使用哪种方式，需要根据AdvisedSupport中指定的参数来判断
// 创建AopProxy使用了简单工厂模式
AopProxyFactory aopProxyFactory = new DefaultAopProxyFactory();
//通过AopProxy工厂获取AopProxy对象
AopProxy aopProxy = aopProxyFactory.createAopProxy(advisedSupport);

//3.通过AopProxy创建代理对象
```

```
Object proxy = aopProxy.getProxy();
```

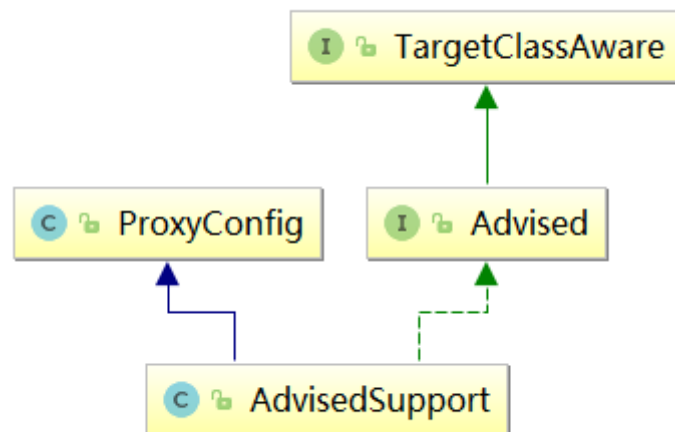
从上面可以看出创建代理有3个步骤。

创建代理3大步骤

1. 创建代理所需参数配置
2. 根据代理参数获取AopProxy对象
3. 通过AopProxy获取代理对象

创建代理所需参数配置

创建代理所需参数配置主要是通过 `AdvisedSupport` 这个类来做的，看一下类图，下面一个个来介绍。



根据代理参数获取AopProxy对象

TargetClassAware接口

比较简单的一个接口，定义了一个方法，用来获取目标对象类型。

所谓目标对象：就是被代理对象，比如上面的 `fundsservice` 对象。

```
package org.springframework.aop;

public interface TargetClassAware {
    @Nullable
    Class<?> getTargetClass();
}
```

ProxyConfig类

这个类比较关键了，代理配置类，内部包含了创建代理时需要配置的各种参数。

```
package org.springframework.aop.framework;

/**
 * 对外提供统一的代理参数配置类，以确保所有代理创建程序具有一致的属性
 */
public class ProxyConfig implements Serializable {

    // 标记是否直接对目标类进行代理，而不是通过接口产生代理
    private boolean proxyTargetClass = false;
```



```

    // 标记是否对代理进行优化。启动优化通常意味着在代理对象被创建后，增强的修改将不会生效，因此默认值为false。
    // 如果exposeProxy设置为true，即使optimize为true也会被忽略。
    private boolean optimize = false;

    // 标记是否需要阻止通过该配置创建的代理对象转换为Advised类型，默认值为false，表示代理对象可以被转换为Advised类型
    boolean opaque = false;

    // 标记代理对象是否应该被aop框架通过AopContext以ThreadLocal的形式暴露出去。
    // 当一个代理对象需要调用它自己的另外一个代理方法时，这个属性将非常有用。默认是false，以避免不必要的拦截。
    boolean exposeProxy = false;

    // 标记该配置是否需要被冻结，如果被冻结，将不可以修改增强的配置。
    // 当我们不希望调用方修改转换成Advised对象之后的代理对象时，这个配置将非常有用。
    private boolean frozen = false;

    //省略了属性的get set方法
}

```

Advised接口

这个接口中定义了操作Aop代理配置的各种方法（比如指定被代理的目标对象、添加通知、添加顾问等等）。

所有由spring aop创建的代理对象默认都会实现这个接口。

```

public interface Advised extends TargetClassAware {

    /**
     * 返回配置是否已冻结，被冻结之后，无法修改已创建好的代理对象中的通知
     */
    boolean isFrozen();

    /**
     * 是否对目标类直接创建代理，而不是对接口创建代理，通俗点讲：如果是通过cglib创建代理，此方法返回true，否则返回false
     */
    boolean isProxyTargetClass();

    /**
     * 获取配置中需要代理的接口列表
     */
    Class<?>[] getProxiedInterfaces();

    /**
     * 判断某个接口是否被代理
     */
    boolean isInterfaceProxied(Class<?> intf);

    /**
     * 设置被代理的目标源，创建代理的时候，通常需要传入被代理的对象，最终被代理的对象会被包装为TargetSource类型的
     */
}

```

```

void setTargetSource(TargetSource targetSource);

/**
 * 返回被代理的目标源
 */
TargetSource getTargetSource();

/**
 * 设置是否需要将代理暴露在ThreadLocal中，这样可以在线程中获取到被代理对象，这个配置挺有
用的，稍后会举例说明使用场景
 */
void setExposeProxy(boolean exposeProxy);

/**
 * 返回exposeProxy
 */
boolean isExposeProxy();

/**
 * 设置此代理配置是否经过预筛选，以便它只包含适用的顾问(匹配此代理的目标类)。
 * 默认设置是“假”。如果已经对advisor进行了预先筛选，则将其设置为“true”
 * 这意味着在为代理调用构建实际的advisor链时可以跳过ClassFilter检查。
 */
void setPreFiltered(boolean preFiltered);

/**
 * 返回preFiltered
 */
boolean isPreFiltered();

/**
 * 返回代理配置中干掉所有Advisor列表
 */
Advisor[] getAdvisors();

/**
 * 添加一个Advisor
 */
void addAdvisor(Advisor advisor) throws AopConfigException;

/**
 * 指定的位置添加一个Advisor
 */
void addAdvisor(int pos, Advisor advisor) throws AopConfigException;

/**
 * 移除一个Advisor
 */
boolean removeAdvisor(Advisor advisor);

/**
 * 移除指定位置的Advisor
 */
void removeAdvisor(int index) throws AopConfigException;

/**
 * 查找某个Advisor的位置
 */

```

```

    int indexOf(Advisor advisor);

    /**
     * 对advisor列表中的a替换为b
     */
    boolean replaceAdvisor(Advisor a, Advisor b) throws AopConfigException;

    /**
     * 添加一个通知
     */
    void addAdvice(Advice advice) throws AopConfigException;

    /**
     * 向指定的位置添加一个通知
     */
    void addAdvice(int pos, Advice advice) throws AopConfigException;

    /**
     * 移除一个通知
     */
    boolean removeAdvice(Advice advice);

    /**
     * 获取通知的位置
     */
    int indexOf(Advice advice);

    /**
     * 将代理配置转换为字符串，这个方便排错和调试使用的
     */
    String toProxyConfigString();
}

```

AdvisedSupport类

这个类是个重点，AOP代理配置管理器的基类，继承了 `ProxyConfig` 并且实现了 `Advised` 接口，创建aop代理之前，所有需要配置的信息都是通过这个类来操作的。

比如：设置是否为目标类创建代理、设置目标对象、配置通知列表等等。

```

package org.springframework.aop.framework;

public class AdvisedSupport extends ProxyConfig implements Advised {

    public static final TargetSource EMPTY_TARGET_SOURCE =
        EmptyTargetSource.INSTANCE;

    TargetSource targetSource = EMPTY_TARGET_SOURCE;

    /** 建议器是否已经针对特定的目标类进行筛选 */
    private boolean preFiltered = false;

    /** 调用链工厂，用来获取目标方法的调用链 */
    AdvisorChainFactory advisorChainFactory = new DefaultAdvisorChainFactory();

    /** 方法调用链缓存：以方法为键，以顾问链表为值的缓存。 */
}

```

```

private transient Map<MethodCacheKey, List<Object>> methodCache;

//代理对象需要实现的接口列表。保存在列表中以保持注册的顺序，以创建具有指定接口顺序的JDK代理。
private List<Class<?>> interfaces = new ArrayList<>();

//配置的顾问列表。所有添加的Advise对象都会被包装为Advisor对象
private List<Advisor> advisors = new ArrayList<>();

//数组更新了对advisor列表的更改，这更容易在内部操作。
private Advisor[] advisorArray = new Advisor[0];

//无参构造方法
public AdvisedSupport() {
    this.methodCache = new ConcurrentHashMap<>(32);
}

//有参构造方法，参数为：代理需要实现的接口列表
public AdvisedSupport(Class<?>... interfaces) {
    this();
    setInterfaces(interfaces);
}

//设置需要被代理的目标对象，目标对象会被包装为TargetSource格式的对象
public void setTarget(Object target) {
    setTargetSource(new SingletonTargetSource(target));
}

//设置被代理的目标源
@Override
public void setTargetSource(@Nullable TargetSource targetSource) {
    this.targetSource = (targetSource != null ? targetSource :
EMPTY_TARGET_SOURCE);
}

//获取被代理的目标源
@Override
public TargetSource getTargetSource() {
    return this.targetSource;
}

//设置被代理的目标类
public void setTargetClass(@Nullable Class<?> targetClass) {
    this.targetSource = EmptyTargetSource.forClass(targetClass);
}

//获取被代理的目标类型
@Override
@Nullable
public Class<?> getTargetClass() {
    return this.targetSource.getTargetClass();
}

/**
 * 设置此代理配置是否经过预筛选，这个什么意思呢：通过目标方法调用代理的时候，
 * 需要通过匹配的方式获取这个方法上的调用链列表，查找过程需要2个步骤：

```

* 第一步：类是否匹配，第二步：方法是否匹配，当这个属性为true的时候，会直接跳过第一步，这个懂了不

```
*/
@Override
public void setPreFiltered(boolean preFiltered) {
    this.preFiltered = preFiltered;
}

// 返回preFiltered
@Override
public boolean isPreFiltered() {
    return this.preFiltered;
}

/**
 * 设置顾问链工厂，当调用目标方法的时候，需要获取这个方法上匹配的Advisor列表，
 * 获取目标方法上匹配的Advisor列表的功能就是AdvisorChainFactory来负责的
 */
public void setAdvisorChainFactory(AdvisorChainFactory advisorChainFactory)
{
    Assert.notNull(advisorChainFactory, "AdvisorChainFactory must not be
null");
    this.advisorChainFactory = advisorChainFactory;
}

// 返回顾问链工厂对象
public AdvisorChainFactory getAdvisorChainFactory() {
    return this.advisorChainFactory;
}

//设置代理对象需要实现的接口
public void setInterfaces(Class<?>... interfaces) {
    Assert.notNull(interfaces, "Interfaces must not be null");
    this.interfaces.clear();
    for (Class<?> ifc : interfaces) {
        addInterface(ifc);
    }
}

//为代理对象添加需要实现的接口
public void addInterface(Class<?> intf) {
    Assert.notNull(intf, "Interface must not be null");
    if (!intf.isInterface()) {
        throw new IllegalArgumentException "[" + intf.getName() + "] is not
an interface");
    }
    if (!this.interfaces.contains(intf)) {
        this.interfaces.add(intf);
        adviceChanged();
    }
}

//移除代理对象需要实现的接口
public boolean removeInterface(Class<?> intf) {
    return this.interfaces.remove(intf);
}
```

```

//获取代理对象需要实现的接口列表
@Override
public Class<?>[] getProxiedInterfaces() {
    return ClassUtils.toClassArray(this.interfaces);
}

//判断代理对象是否需要实现某个接口
@Override
public boolean isInterfaceProxied(Class<?> intf) {
    for (Class<?> proxyIntf : this.interfaces) {
        if (intf.isAssignableFrom(proxyIntf)) {
            return true;
        }
    }
    return false;
}

//获取配置的所有顾问列表
@Override
public final Advisor[] getAdvisors() {
    return this.advisorArray;
}

//添加顾问
@Override
public void addAdvisor(Advisor advisor) {
    int pos = this.advisors.size();
    addAdvisor(pos, advisor);
}

//指定的位置添加顾问
@Override
public void addAdvisor(int pos, Advisor advisor) throws AopConfigException {
    //这块先忽略，以后讲解
    if (advisor instanceof IntroductionAdvisor) {
        validateIntroductionAdvisor((IntroductionAdvisor) advisor);
    }
    addAdvisorInternal(pos, advisor);
}

//移除指定的顾问
@Override
public boolean removeAdvisor(Advisor advisor) {
    int index = indexOf(advisor);
    if (index == -1) {
        return false;
    }
    else {
        removeAdvisor(index);
        return true;
    }
}

//移除指定位置的顾问
@Override
public void removeAdvisor(int index) throws AopConfigException {
    //当配置如果是冻结状态，是不允许对顾问进行修改的，否则会抛出异常
    if (isFrozen()) {

```

```

        throw new AopConfigException("Cannot remove Advisor: Configuration
is frozen.");
    }
    if (index < 0 || index > this.advisors.size() - 1) {
        throw new AopConfigException("Advisor index " + index + " is out of
bounds: " +
            "This configuration only has " + this.advisors.size() + "
advisors.");
    }
    //移除advisors中的顾问
    Advisor advisor = this.advisors.remove(index);
    if (advisor instanceof IntroductionAdvisor) {
        IntroductionAdvisor ia = (IntroductionAdvisor) advisor;
        // We need to remove introduction interfaces.
        for (Class<?> ifc : ia.getInterfaces()) {
            removeInterface(ifc);
        }
    }
    //更新advisorArray
    updateAdvisorArray();
    //通知已改变，内部会清除方法调用链缓存信息。
    adviceChanged();
}

@Override
public int indexOf(Advisor advisor) {
    Assert.notNull(advisor, "Advisor must not be null");
    return this.advisors.indexOf(advisor);
}

@Override
public boolean replaceAdvisor(Advisor a, Advisor b) throws
AopConfigException {
    Assert.notNull(a, "Advisor a must not be null");
    Assert.notNull(b, "Advisor b must not be null");
    int index = indexOf(a);
    if (index == -1) {
        return false;
    }
    removeAdvisor(index);
    addAdvisor(index, b);
    return true;
}

//批量添加顾问
public void addAdvisors(Advisor... advisors) {
    addAdvisors(Arrays.asList(advisors));
}

//批量添加顾问
public void addAdvisors(Collection<Advisor> advisors) {
    //配置如果是冻结状态，会抛出异常
    if (isFrozen()) {
        throw new AopConfigException("Cannot add advisor: Configuration is
frozen.");
    }
    if (!CollectionUtils.isEmpty(advisors)) {
        for (Advisor advisor : advisors) {

```

```

        if (advisor instanceof IntroductionAdvisor) {
            validateIntroductionAdvisor((IntroductionAdvisor) advisor);
        }
        Assert.notNull(advisor, "Advisor must not be null");
        this.advisors.add(advisor);
    }
    updateAdvisorArray();
    adviceChanged();
}
}

//此方法先忽略，用来为目标类引入接口的
private void validateIntroductionAdvisor(IntroductionAdvisor advisor) {
    advisor.validateInterfaces();
    // If the advisor passed validation, we can make the change.
    Class<?>[] ifcs = advisor.getInterfaces();
    for (Class<?> ifc : ifcs) {
        addInterface(ifc);
    }
}

//指定的位置添加顾问
private void addAdvisorInternal(int pos, Advisor advisor) throws
AopConfigException {
    Assert.notNull(advisor, "Advisor must not be null");
    if (isFrozen()) {
        throw new AopConfigException("Cannot add advisor: Configuration is
frozen.");
    }
    if (pos > this.advisors.size()) {
        throw new IllegalArgumentException(
            "Illegal position " + pos + " in advisor list with size " +
this.advisors.size());
    }
    this.advisors.add(pos, advisor);
    updateAdvisorArray();
    adviceChanged();
}

//将advisorArray和advisors保持一致
protected final void updateAdvisorArray() {
    this.advisorArray = this.advisors.toArray(new Advisor[0]);
}

//获取顾问列表
protected final List<Advisor> getAdvisorsInternal() {
    return this.advisors;
}

//添加通知
@Override
public void addAdvice(Advice advice) throws AopConfigException {
    int pos = this.advisors.size();
    addAdvice(pos, advice);
}

//指定的位置添加通知
@Override

```



```

public void addAdvice(int pos, Advice advice) throws AopConfigException {
    //此处会将advice通知包装为DefaultPointcutAdvisor类型的Advisor
    addAdvisor(pos, new DefaultPointcutAdvisor(advice));
}

//移除通知
@Override
public boolean removeAdvice(Advice advice) throws AopConfigException {
    int index = indexOf(advice);
    if (index == -1) {
        return false;
    }
    else {
        removeAdvisor(index);
        return true;
    }
}

//获取通知的位置
@Override
public int indexOf(Advice advice) {
    Assert.notNull(advice, "Advice must not be null");
    for (int i = 0; i < this.advisors.size(); i++) {
        Advisor advisor = this.advisors.get(i);
        if (advisor.getAdvice() == advice) {
            return i;
        }
    }
    return -1;
}

//是否包含某个通知
public boolean adviceIncluded(@Nullable Advice advice) {
    if (advice != null) {
        for (Advisor advisor : this.advisors) {
            if (advisor.getAdvice() == advice) {
                return true;
            }
        }
    }
    return false;
}

//获取当前配置中某种类型通知的数量
public int countAdvicesOfType(@Nullable Class<?> adviceClass) {
    int count = 0;
    if (adviceClass != null) {
        for (Advisor advisor : this.advisors) {
            if (adviceClass.isInstance(advisor.getAdvice())) {
                count++;
            }
        }
    }
    return count;
}

```

```

//基于当前配置，获取给定方法的方法调用链列表（即
org.aopalliance.intercept.MethodInterceptor对象列表）
public List<Object> getInterceptorsAndDynamicInterceptionAdvice(Method
method, @Nullable Class<?> targetClass) {
    MethodCacheKey cacheKey = new MethodCacheKey(method);
    //先从缓存中获取
    List<Object> cached = this.methodCache.get(cacheKey);
    //缓存中没有时，从advisorChainFactory中获取
    if (cached == null) {
        cached =
this.advisorChainFactory.getInterceptorsAndDynamicInterceptionAdvice(
        this, method, targetClass);
        this.methodCache.put(cacheKey, cached);
    }
    return cached;
}

//通知更改时调用，会清空当前方法调用链缓存
protected void adviceChanged() {
    this.methodCache.clear();
}

//将other中的配置信息复制到当前对象中
protected void copyConfigurationFrom(AdvisedSupport other) {
    copyConfigurationFrom(other, other.targetSource, new ArrayList<>
(other.advisors));
}

//将other中的配置信息复制到当前对象中
protected void copyConfigurationFrom(AdvisedSupport other, TargetSource
targetSource, List<Advisor> advisors) {
    copyFrom(other);
    this.targetSource = targetSource;
    this.advisorChainFactory = other.advisorChainFactory;
    this.interfaces = new ArrayList<>(other.interfaces);
    for (Advisor advisor : advisors) {
        if (advisor instanceof IntroductionAdvisor) {
            validateIntroductionAdvisor((IntroductionAdvisor) advisor);
        }
        Assert.notNull(advisor, "Advisor must not be null");
        this.advisors.add(advisor);
    }
    updateAdvisorArray();
    adviceChanged();
}

//构建此AdvisedSupport的仅配置副本，替换TargetSource。
AdvisedSupport getConfigurationOnlyCopy() {
    AdvisedSupport copy = new AdvisedSupport();
    copy.copyFrom(this);
    copy.targetSource = EmptyTargetSource.forClass(getTargetClass(),
getTargetSource().isStatic());
    copy.advisorChainFactory = this.advisorChainFactory;
    copy.interfaces = this.interfaces;
    copy.advisors = this.advisors;
    copy.updateAdvisorArray();
    return copy;
}

```

```
}
```

上面几个类有几个结论，这里说一下。

1. 配置中添加的Advice对象最终都会被转换为DefaultPointcutAdvisor对象，此时DefaultPointcutAdvisor未指定pointcut，大家可以去看一下DefaultPointcutAdvisor中pointcut有个默认值，默认会匹配任意类的任意方法。
2. 当配置被冻结的时候，即frozen为true的时，此时配置中的Advisor列表是不允许修改的。
3. 上面的getInterceptorsAndDynamicInterceptionAdvice方法，通过代理调用目标方法的时候，最后需要通过方法和目标类的类型，从当前配置中会获取匹配的方法拦截器列表，获取方法拦截器列表是由AdvisorChainFactory负责的。
getInterceptorsAndDynamicInterceptionAdvice会在调用代理的方法时会执行，稍后在执行阶段会详解。
4. 目标方法和其关联的方法拦截器列表会被缓存在methodCache中，当顾问列表有变化的时候，methodCache缓存会被清除。

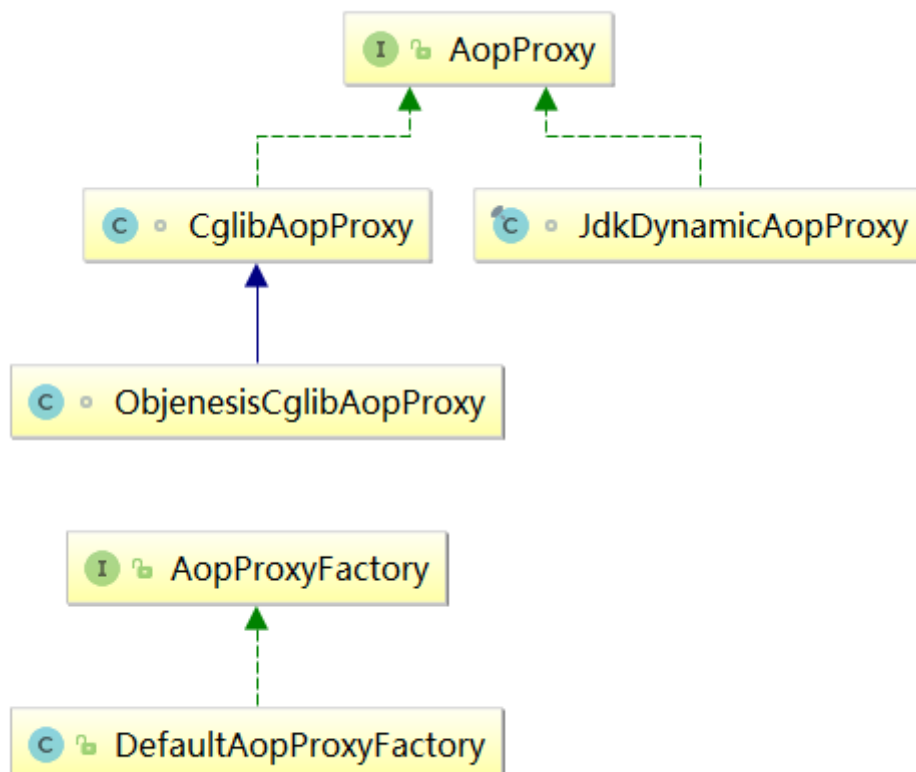
配置阶段完成之后，下面进入AopProxy获取阶段。

根据配置获取AopProxy

这个阶段对应的代码：

```
// 创建AopProxy使用了简单工厂模式
AopProxyFactory aopProxyFactory = new DefaultAopProxyFactory();
//通过AopProxy工厂获取AopProxy对象
AopProxy aopProxy = aopProxyFactory.createAopProxy(advisedSupport);
```

此阶段会根据AdvisedSupport中配置信息，判断具体是采用cglib的方式还是采用jdk动态代理的方式获取代理对象，先看一下涉及到的一些类。



AopProxy接口

这个接口定义了一个方法，用来创建最终的代理对象，这个接口有2个实现类

- CglibAopProxy：采用cglib的方式创建代理对象
- JdkDynamicAopProxy：采用jdk动态代理的方式创建代理对象

```
package org.springframework.aop.framework;

public interface AopProxy {

    /**
     * 创建一个新的代理对象
     */
    Object getProxy();

    /**
     * 创建一个新的代理对象
     */
    Object getProxy(@Nullable ClassLoader classLoader);

}
```

AopProxy的2个实现类，实现了上面定义的2个方法，稍后在代理的创建阶段详细介绍。

AopProxyFactory接口

通过名称就可以看出来，是一个工厂，负责创建AopProxy，使用的是简单工厂模式。

接口中定义了一个方法，会根据Aop的配置信息AdvisedSupport来获取AopProxy对象，主要是判断采用cglib的方式还是采用jdk动态代理的方式。

```
package org.springframework.aop.framework;

public interface AopProxyFactory {

    /**
     * 根据aop配置信息获取AopProxy对象
     */
    AopProxy createAopProxy(AdvisedSupport config) throws AopConfigException;

}
```

DefaultAopProxyFactory类

AopProxyFactory接口的默认实现，代码比较简单，我们来细看一下

```
package org.springframework.aop.framework;

/**
 * 默认AopProxyFactory实现，创建CGLIB代理或JDK动态代理。
 * 对于给定的AdvisedSupport实例，以下条件为真，则创建一个CGLIB代理：
 * optimize = true
 * proxyTargetClass = true
 * 未指定代理接口
 */
```

```

    * 通常，指定proxyTargetClass来强制执行CGLIB代理，或者指定一个或多个接口来使用JDK动态代理。
    */
    public class DefaultAopProxyFactory implements AopProxyFactory, Serializable {

        @Override
        public AopProxy createAopProxy(AdvisedSupport config) throws
AopConfigException {
            // optimize==true || proxyTargetClass 为true || 配置中没有需要代理的接口
            if (config.isOptimize() || config.isProxyTargetClass() ||
hasNoUserSuppliedProxyInterfaces(config)) {
                //获取需要被代理的类
                Class<?> targetClass = config.getTargetClass();
                if (targetClass == null) {
                    throw new AopConfigException("TargetSource cannot determine
target class: " +
                        "Either an interface or a target is required for proxy
creation.");
                }
                //如果被代理的类为接口 或者 被代理的类是jdk动态代理创建代理类，则采用
JdkDynamicAopProxy的方式，否则采用cglib代理的方式
                if (targetClass.isInterface() || Proxy.isProxyClass(targetClass)) {
                    //采用jdk动态代理的方式
                    return new JdkDynamicAopProxy(config);
                }
                //采用cglib代理的方式
                return new ObjenesisCglibAopProxy(config);
            }
            else {
                //采用jdk动态代理的方式
                return new JdkDynamicAopProxy(config);
            }
        }

        /**
         * 确定所提供的AdvisedSupport是否只指定了SpringProxy接口(或者根本没有指定代理接口)
         */
        private boolean hasNoUserSuppliedProxyInterfaces(AdvisedSupport config) {
            Class<?>[] ifcs = config.getProxiedInterfaces();
            return (ifcs.length == 0 || (ifcs.length == 1 &&
SpringProxy.class.isAssignableFrom(ifcs[0])));
        }
    }
}

```

代理创建阶段

到目前为止我们已经根据aop配置信息得到了AopProxy对象了，下面就可以调用AopProxy.getProxy方法获取代理对象了。

AopProxy.createAopProxy方法返回的结果有2种情况

- JdkDynamicAopProxy：以jdk动态代理的方式创建代理
- ObjenesisCglibAopProxy：以cglib的方式创建动态代理

项目详解这2个类的源码。

JdkDynamicAopProxy类

作用：采用jdk动态代理的方式创建代理对象，并处理代理对象的所有方法调用。

```
final class JdkDynamicAopProxy implements AopProxy, InvocationHandler,
Serializable {

    //代理的配置信息
    private final AdvisedSupport advised;

    //需要被代理的接口中是否定义了equals方法
    private boolean equalsDefined;

    //需要被代理的接口中是否定义了hashCode方法
    private boolean hashCodeDefined;

    //通过AdvisedSupport创建实例
    public JdkDynamicAopProxy(AdvisedSupport config) throws AopConfigException {
        Assert.notNull(config, "AdvisedSupport must not be null");
        if (config.getAdvisors().length == 0 && config.getTargetSource() ==
AdvisedSupport.EMPTY_TARGET_SOURCE) {
            throw new AopConfigException("No advisors and no TargetSource
specified");
        }
        this.advised = config;
    }

    //生成一个代理对象
    @Override
    public Object getProxy() {
        return getProxy(ClassUtils.getDefaultClassLoader());
    }

    //生成一个代理对象
    @Override
    public Object getProxy(@Nullable ClassLoader classLoader) {
        if (logger.isTraceEnabled()) {
            logger.trace("Creating JDK dynamic proxy: " +
this.advised.getTargetSource());
        }
        //@@0: 根据advised的信息获取代理需要被代理的所有接口列表
        Class<?>[] proxiedInterfaces =
AopProxyUtils.completeProxiedInterfaces(this.advised, true);
        //查找被代理的接口中是否定义了equals、hashCode方法
        findDefinedEqualsAndHashCodeMethods(proxiedInterfaces);
        /**
         * 这个大家应该很熟悉吧，通过jdk动态代理创建代理对象，注意最后一个参数是this
         * 表示当前类，当前类是InvocationHandler类型的，当调用代理对象的任何方法的时候
         * 都会被被当前类的 invoke 方法处理
         */
        return Proxy.newProxyInstance(classLoader, proxiedInterfaces, this);
    }

    //判断需要代理的接口中是否定义了这几个方法（equals、hashCode）
    private void findDefinedEqualsAndHashCodeMethods(Class<?>[]
proxiedInterfaces) {
        for (Class<?> proxiedInterface : proxiedInterfaces) {
```

```

//获取接口中定义的方法
Method[] methods = proxiedInterface.getDeclaredMethods();
for (Method method : methods) {
    //是否是equals方法
    if (AopUtils.isEqualsMethod(method)) {
        this.equalsDefined = true;
    }
    //是否是hashCode方法
    if (AopUtils.isHashCodeMethod(method)) {
        this.hashCodeDefined = true;
    }
    //如果发现这2个方法都定义了，结束循环查找
    if (this.equalsDefined && this.hashCodeDefined) {
        return;
    }
}
}
}

```

```

// 这个方法比较关键了，当在程序中调用代理对象的任何方法，最终都会被下面这个invoke方法处理
public Object invoke(Object proxy, Method method, Object[] args) throws
Throwable {
    //旧的代理对象
    Object oldProxy = null;
    //用来标记是否需要将代理对象暴露在ThreadLocal中
    boolean setProxyContext = false;
    //获取目标源
    TargetSource targetSource = this.advised.targetSource;
    //目标对象
    Object target = null;

    //下面进入代理方法的处理阶段
    try {
        // 处理equals方法：被代理的接口中没有定义equals方法 && 当前调用是equals方法
        if (!this.equalsDefined && AopUtils.isEqualsMethod(method)) {
            // 直接调用当前类中的equals方法
            return equals(args[0]);
        }
        // 处理hashCode方法：被代理的接口中没有定义hashCode方法 && 当前调用是
hashCode方法
        else if (!this.hashCodeDefined && AopUtils.isHashCodeMethod(method))
        {
            // 直接调用当前类中的hashCode方法
            return hashCode();
        }
        /**
         * 方法来源于 DecoratingProxy 接口，这个接口中定义了一个方法
         * 用来获取原始的被代理的目标类，主要是用在嵌套代理的情况下（所谓嵌套代理：代理对
象又被作为目标对象进行了代理）
         */
        else if (method.getDeclaringClass() == DecoratingProxy.class) {
            // 调用AopProxyUtils工具类的方法，内部通过循环遍历的方式，找到最原始的被代
理的目标类
            return AopProxyUtils.ultimateTargetClass(this.advised);
        }
        // 方法来源于 Advised 接口，代理对象默认情况下会实现 Advised 接口，可以通过代
理对象来动态向代理对象中添加通知等
    }
}

```

```

        else if (!this.advised.opaque &&
method.getDeclaringClass().isInterface() &&
        method.getDeclaringClass().isAssignableFrom(Advised.class))
{
    // this.advised是AdvisedSupport类型的，AdvisedSupport实现了Advised接
    口中的所有方法
    // 所以最终通过通过反射方式交给this.advised来响应当前调用
    return AopUtils.invokeJoinpointUsingReflection(this.advised,
method, args);
}
// 用来记录方法返回值
Object retVal;

//是否需要在threadLocal中暴露代理对象
if (this.advised.exposeProxy) {
    // 将代理对象暴露在上层文中，即暴露在threadLocal中，那么在当前线程中可以通
    过静态方法
    // AopContext#currentProxy获取当前被暴露的代理对象，这个是非常有用的，稍
    后用案例来讲解，瞬间就会明白
    oldProxy = AopContext.setCurrentProxy(proxy);
    // 将setProxyContext标记为true
    setProxyContext = true;
}

// 通过目标源获取目标对象
target = targetSource.getTarget();
// 获取目标对象类型
Class<?> targetClass = (target != null ? target.getClass() : null);

// @1: 获取当前方法的拦截器链
List<Object> chain =
this.advised.getInterceptorsAndDynamicInterceptionAdvice(method, targetClass);

// 拦截器链为空的情况下，表示这个方法上面没有找到任何增强的通知，那么会直接通过反
    射直接调用目标对象
    if (chain.isEmpty()) {
        // 获取方法请求的参数（有时候方法中有可变参数，所谓可变参数就是带有省略号
        (...)这种格式的参数，传入的参数类型和这种类型不一样的时候，会通过下面的
        adaptArgumentsIfNecessary方法进行转换）
        Object[] argsToUse =
AopProxyUtils.adaptArgumentsIfNecessary(method, args);
        //通过反射直接调用目标方法
        retVal = AopUtils.invokeJoinpointUsingReflection(target, method,
argsToUse);
    }
    else {
        // 创建一个方法调用器（包含了代理对象、目标对象、调用的方法、参数、目标类型、
        方法拦截器链）
        MethodInvocation invocation =
            new ReflectiveMethodInvocation(proxy, target, method,
args, targetClass, chain);
        // @3: 通过拦截器链一个个调用最终到目标方法的调用
        retVal = invocation.proceed();
    }

    // 下面会根据方法返回值的类型，做一些处理，比如方法返回的类型为自己，则最后需要将
    返回值置为代理对象
    Class<?> returnType = method.getReturnType();

```



```

        if (retVal != null && retVal == target &&
            returnType != Object.class && returnType.isInstance(proxy)
&&
!RawTargetAccess.class.isAssignableFrom(method.getDeclaringClass())) {
            // 将返回值设置为代理对象
            retVal = proxy;
        }
        // 方法的返回值类型returnType为原始类型（即int、byte、double等这种类型的） &&
        retVal为null,
        // 此时如果将null转换为原始类型会报错，所以此处直接抛出异常
        else if (retVal == null && returnType != void.TYPE &&
returnType.isPrimitive()) {
            throw new AopInvocationException(
                "Null return value from advice does not match primitive
return type for: " + method);
        }
        // 返回方法调用结果
        return retVal;
    }
    finally {
        // 目标对象不为null && 目标源不是静态的
        //所谓静态的，你可以理解为是否是单例的
        // isStatic为true，表示目标对象是单例的，同一个代理对象中所有方法共享一个目标对
        象
        // isStatic为false的时候，通常每次调用代理的方法，target对象是不一样的，所以方
        法调用万之后需要进行释放，可能有些资源清理，连接的关闭等操作
        if (target != null && !targetSource.isStatic()) {
            // 必须释放来自TargetSource中的目标对象
            targetSource.releaseTarget(target);
        }
        // setProxyContext为ture
        if (setProxyContext) {
            // 需要将旧的代理再放回上线文中
            AopContext.setCurrentProxy(oldProxy);
        }
    }
}
}
}
}

```

关于上面代码，有几点细说一下

@0: completeProxiedInterfaces方法

@0 处的代码如下，根据代理配置信息，获取需要被代理的所有接口

```

Class<?>[] proxiedInterfaces =
AopProxyUtils.completeProxiedInterfaces(this.advised, true);

```

AopProxyUtils.completeProxiedInterfaces 方法源码如下

```

static Class<?>[] completeProxiedInterfaces(AdvisedSupport advised, boolean
decoratingProxy) {
    //获取代理配置中需要被代理的接口

```

```

Class<?>[] specifiedInterfaces = advised.getProxiedInterfaces();
// 需要被代理的接口数量为0
if (specifiedInterfaces.length == 0) {
    // 获取需要被代理的目标类型
    Class<?> targetClass = advised.getTargetClass();
    //目标类型不为空
    if (targetClass != null) {
        //目标类型为接口
        if (targetClass.isInterface()) {
            //将其添加到需要代理的接口中
            advised.setInterfaces(targetClass);
        }
        // 目标类型为jdk动态代理创建的代理对象
        else if (Proxy.isProxyClass(targetClass)) {
            // 获取目标类型上的所有接口，将其添加到需要被代理的接口中
            advised.setInterfaces(targetClass.getInterfaces());
        }
        //再次获取代理配置中需要被代理的接口
        specifiedInterfaces = advised.getProxiedInterfaces();
    }
}
//判断SpringProxy接口是否已经在被代理的接口中
boolean addSpringProxy = !advised.isInterfaceProxied(SpringProxy.class);
//判断Advised接口是否已经在被代理的接口中
boolean addAdvised = !advised.isOpaque() &&
!advised.isInterfaceProxied(Advised.class);
//判断DecoratingProxy接口是否已经在被代理的接口中
boolean addDecoratingProxy = (decoratingProxy &&
!advised.isInterfaceProxied(DecoratingProxy.class));
//一个计数器，会根据上面三个boolean值做递增
int nonUserIfcCount = 0;
if (addSpringProxy) {
    nonUserIfcCount++;
}
if (addAdvised) {
    nonUserIfcCount++;
}
if (addDecoratingProxy) {
    nonUserIfcCount++;
}
// 下面就是构建所有需要被代理的接口
Class<?>[] proxiedInterfaces = new Class<?>[specifiedInterfaces.length +
nonUserIfcCount];
System.arraycopy(specifiedInterfaces, 0, proxiedInterfaces, 0,
specifiedInterfaces.length);
int index = specifiedInterfaces.length;
if (addSpringProxy) {
    proxiedInterfaces[index] = SpringProxy.class;
    index++;
}
if (addAdvised) {
    proxiedInterfaces[index] = Advised.class;
    index++;
}
if (addDecoratingProxy) {
    proxiedInterfaces[index] = DecoratingProxy.class;
}
return proxiedInterfaces;

```

```
}
```

上面的方法执行完毕之后，会得到一个被代理的接口列表，默认情况下会得到下面的一个列表

[开发者硬编码指定的需要被代理的接口列表, `SpringProxy`, `Advised`, `DecoratingProxy`]

最终创建出来的代理对象，默认会实现上面列的所有接口，后面3个接口是aop中自动给我们加上的。

@1: `getInterceptorsAndDynamicInterceptionAdvice`

这个方法位于 `AdvisedSupport` 中，根据方法和目标类型获取方法上面匹配的拦截器链

```
public List<Object> getInterceptorsAndDynamicInterceptionAdvice(Method method,
    @Nullable Class<?> targetClass) {
    //会先尝试从缓存中获取，如果获取不到，会从advisorChainFactory中获取，然后将其丢到缓存中
    MethodCacheKey cacheKey = new MethodCacheKey(method);
    List<Object> cached = this.methodCache.get(cacheKey);
    if (cached == null) {
        cached =
            this.advisorChainFactory.getInterceptorsAndDynamicInterceptionAdvice(
                this, method, targetClass);
        this.methodCache.put(cacheKey, cached);
    }
    return cached;
}
```

从 `advisorChainFactory` 中获取拦截器链稍后细说，我们把这个阶段叫做连接器链的获取阶段。

@3: `ReflectiveMethodInvocation.proceed()`

这个是一次会调用拦截器链，最终会调用到目标方法，获得目标方法的返回值，里面的细节见后面的 `代理方法调用处理阶段`

JdkDynamicAopProxy小结

1. 被创建的代理对象默认会实现 `SpringProxy`, `Advised`, `DecoratingProxy` 3个接口
2. `SpringProxy` 这个接口中没有任何方法，只是起一个标记作用，用来标记代理对象是使用spring aop创建的
3. 代理对象默认都会实现 `Advised` 接口，所以可以通过这个接口动态变更代理对象中的通知
4. `DecoratingProxy` 接口中定义了一个方法 `getDecoratedClass`，用来获取被代理的原始目标对象的类型

下面来看另外一个类: `ObjenesisCglibAopProxy`，这个继承了 `CglibAopProxy`，大部分逻辑都在 `CglibAopProxy` 中，所以我们主要看 `CglibAopProxy` 中代码。

CglibAopProxy类

作用：采用cglib代理的方式创建代理对象，并处理代理对象的所有方法调用。

以 `getProxy` 方法为入口，通过方法一个个来解说。

getProxy方法

```
public Object getProxy(@Nullable ClassLoader classLoader) {
    // 获取被代理的类
    Class<?> rootClass = this.advised.getTargetClass();

    // 代理对象的父类（cglib是采用继承的方式是创建代理对象的，所以将被代理的类作为代理对象的父类）
    Class<?> proxySuperClass = rootClass;
    // 判断被代理的类是不是cglib创建的类，如果是cglib创建的类，会将其父类作为被代理的类
    if (rootClass.getName().contains(ClassUtils.CGLIB_CLASS_SEPARATOR)) {
        proxySuperClass = rootClass.getSuperclass();
        // 添加需要被代理的接口
        Class<?>[] additionalInterfaces = rootClass.getInterfaces();
        for (Class<?> additionalInterface : additionalInterfaces) {
            this.advised.addInterface(additionalInterface);
        }
    }

    // 开始cglib创建代理，这个大家对cglib比较熟悉的一看就懂
    Enhancer enhancer = createEnhancer();
    // 设置被代理的父类
    enhancer.setSuperclass(proxySuperClass);
    // 设置被代理的接口[开发者硬编码指定的需要被代理的接口列表, SpringProxy, Advised]，这个比jdk动态代理的方式少了一个DecoratingProxy接口
    enhancer.setInterfaces(AopProxyUtils.completeProxiedInterfaces(this.advised));
    // 设置代理类类名生成策略
    enhancer.setNamingPolicy(SpringNamingPolicy.INSTANCE);
    // 设置字节码的生成策略
    enhancer.setStrategy(new ClassLoaderAwareGeneratorStrategy(classLoader));

    // @1: 获取Callback列表，这个稍后详解
    Callback[] callbacks = getCallbacks(rootClass);
    Class<?>[] types = new Class<?>[callbacks.length];
    for (int x = 0; x < types.length; x++) {
        types[x] = callbacks[x].getClass();
    }

    // @2: 设置CallbackFilter，CallbackFilter内部会判断被代理对象中的方法最终会被callbacks列表中的那个Callback来处理
    enhancer.setCallbackFilter(new ProxyCallbackFilter(
        this.advised.getConfigurationOnlyCopy(), this.fixedInterceptorMap,
        this.fixedInterceptorOffset));
    enhancer.setCallbackTypes(types);

    // 获取代理对象（内部会先创建代理类，然后会根据代理类生成一个代理对象）
    return createProxyClassAndInstance(enhancer, callbacks);
}
```

上面方法中有2个点比较难，需要说明，分别是 @1: `getCallbacks`方法 和 @2: 创建 `ProxyCallbackFilter`对象

@1: getCallbacks方法

通过被代理的类来获取 Callback 列表，Callback 是用来处理代理对象的方法调用的，代理对象中可能有很多方法，每个方法可能采用不同的处理方式，所以会有多个 Callback

```
private Callback[] getCallbacks(Class<?> rootClass) throws Exception {
    // 是否需要将代理暴露在threadLocal中
    boolean exposeProxy = this.advised.isExposeProxy();
    // 配置是否是冻结的
    boolean isFrozen = this.advised.isFrozen();
    // 被代理的目标对象是否是动态的（是否是单例的）
    boolean isStatic = this.advised.getTargetSource().isStatic();

    // 当方法上有需要执行的拦截器的时候，会用这个来处理
    Callback aopInterceptor = new DynamicAdvisedInterceptor(this.advised);

    // 当方法上没有需要执行的拦截器的时候，会使用targetInterceptor来处理，内部会通过反射直接
    // 调用目标对象的方法
    Callback targetInterceptor;
    /**
     * 这块根据是否需要暴露代理到threadLocal中以及目标对象是否是动态的，会创建不同的Callback
     * isStatic为true的时候，同一个代理的不同方法可能都是新的目标对象，所以当代理方法执行完毕
     * 之后，需要对目标对象进行释放
     */
    if (exposeProxy) {
        targetInterceptor = (isStatic ?
                               new
StaticUnadvisedExposedInterceptor(this.advised.getTargetSource().getTarget()) :
                               new
DynamicUnadvisedExposedInterceptor(this.advised.getTargetSource()));
    }
    else {
        targetInterceptor = (isStatic ?
                               new
StaticUnadvisedInterceptor(this.advised.getTargetSource().getTarget()) :
                               new
DynamicUnadvisedInterceptor(this.advised.getTargetSource()));
    }

    // targetDispatcher会直接调用目标方法
    Callback targetDispatcher = (isStatic ?
                                   new
StaticDispatcher(this.advised.getTargetSource().getTarget()) : new
SerializableNoop());

    Callback[] mainCallbacks = new Callback[] {
        aopInterceptor, // 处理匹配到拦截器的方法
        targetInterceptor, // 处理未匹配到拦截器的方法
        new SerializableNoop(),
        targetDispatcher, // 处理未匹配到拦截器的方法，和targetInterceptor有何不同呢？
        // 目标方法如果返回值的结果是目标对象类型的，会使用 targetInterceptor 处理，内部会返回代理对象
        this.advisedDispatcher, // 处理Advised接口中定义的方法
        new EqualsInterceptor(this.advised), // 处理equals方法
        new GetHashCodeInterceptor(this.advised) // 处理hashCode方法
    };

    Callback[] callbacks;
```

```

// 如果被代理的对象是单例的 && 配置是冻结的，此时会进行优化，怎么优化呢？
// 配置冻结的情况下，生成好的代理中通知是无法修改的，所以可以提前将每个方法对应的拦截器链找到给缓存起来
// 调用方法的时候，就直接从缓存中可以拿到方法对应的缓存信息，效率会高一些
if (isStatic && isFrozen) {
    Method[] methods = rootClass.getMethods();
    Callback[] fixedCallbacks = new Callback[methods.length];
    this.fixedInterceptorMap = new HashMap<>(methods.length);

    // 获取每个方法的调用链，然后给缓存在fixedInterceptorMap中
    for (int x = 0; x < methods.length; x++) {
        Method method = methods[x];
        List<Object> chain =
this.advised.getInterceptorsAndDynamicInterceptionAdvice(method, rootClass);
        fixedCallbacks[x] = new FixedChainStaticTargetInterceptor(
            chain, this.advised.getTargetSource().getTarget(),
this.advised.getTargetClass());
        this.fixedInterceptorMap.put(method, x);
    }
    callbacks = new Callback[mainCallbacks.length + fixedCallbacks.length];
    System.arraycopy(mainCallbacks, 0, callbacks, 0, mainCallbacks.length);
    System.arraycopy(fixedCallbacks, 0, callbacks, mainCallbacks.length,
fixedCallbacks.length);
    this.fixedInterceptorOffset = mainCallbacks.length;
}
else {
    callbacks = mainCallbacks;
}
return callbacks;
}

```

@2:创建ProxyCallbackFilter对象

```

enhancer.setCallbackFilter(new ProxyCallbackFilter(
    this.advised.getConfigurationOnlyCopy(), this.fixedInterceptorMap,
    this.fixedInterceptorOffset));

```

这块重点在于 ProxyCallbackFilter 中的 accept 方法，这个方法会根据目标放，获取目标对方最后会让callbacks列表中的哪个Callback处理，大家可以看一下源码，比较简单。

上面 getCallbacks 方法中涉及到了5个类如下

- DynamicAdvisedInterceptor
- StaticUnadvisedExposedInterceptor
- StaticUnadvisedInterceptor
- DynamicUnadvisedInterceptor
- StaticDispatcher

后面4个比较简单，大家可以去看一下源码，主要来看第一个类，基本上代理对象中的大部分自定义的方法都会进入到这个类的 intercept 方法中进行处理，代码如下

DynamicAdvisedInterceptor类

```
private static class DynamicAdvisedInterceptor implements MethodInterceptor,
Serializable {
    //代理配置信息
    private final AdvisedSupport advised;
    //构造器，需要一个AdvisedSupport
    public DynamicAdvisedInterceptor(AdvisedSupport advised) {
        this.advised = advised;
    }

    //这个方法是关键，用来处理代理对象中方法的调用
    public Object intercept(Object proxy, Method method, Object[] args,
        MethodProxy methodProxy) throws Throwable {
        //被暴露在threadLocal中旧的代理对象
        Object oldProxy = null;
        //用来标记代理对象是否被暴露在threadLocal中
        boolean setProxyContext = false;
        //目标对象
        Object target = null;
        //目标源
        TargetSource targetSource = this.advised.getTargetSource();
        try {
            //代理配置中是否需要将代理暴露在threadLocal中
            if (this.advised.exposeProxy) {
                //将代理对象暴露出去
                oldProxy = AopContext.setCurrentProxy(proxy);
                //将setProxyContext置为true
                setProxyContext = true;
            }
            //获取目标对象（即被代理的对象）
            target = targetSource.getTarget();
            Class<?> targetClass = (target != null ? target.getClass() : null);
            //@1: 获取当前方法的拦截器链
            List<Object> chain =
this.advised.getInterceptorsAndDynamicInterceptionAdvice(method, targetClass);
            //记录方法返回值
            Object retVal;
            //拦截器链不为空 && 方法是public类型的
            if (chain.isEmpty() && Modifier.isPublic(method.getModifiers())) {
                //获取方法调用参数
                Object[] argsToUse =
AopProxyUtils.adaptArgumentsIfNecessary(method, args);
                // 直接调用目标对象的方法
                retVal = methodProxy.invoke(target, argsToUse);
            }
            else {
                // 创建一个方法调用器（包含了代理对象、目标对象、调用的方法、参数、目标类型、
                // 方法拦截器链）
                // @2: 并执行方法调用器的process()方法，此方法会一次执行方法调用链，最终会
                // 调用目标方法，获取返回结果
                retVal = new CglibMethodInvocation(proxy, target, method, args,
                    targetClass, chain, methodProxy).process();
            }
            // 处理方法返回结果：会根据方法返回值的类型，做一些处理，比如方法返回的类型为自
            // 己，则最后需要将返回值置为代理对象
            retVal = processReturnValue(proxy, target, method, retVal);
        }
    }
}
```

```

        return retVal;
    }
    finally {
        // 目标对象不为null && 目标源不是静态的
        //所谓静态的，你可以理解为是否是单例的
        // isStatic为true，表示目标对象是单例的，同一个代理对象中所有方法共享一个目标对象
        // isStatic为false的时候，通常每次调用代理的方法，target对象是不一样的，所以方法调用完之后需要进行释放，可能有些资源清理，连接的关闭等操作
        if (target != null && !targetSource.isStatic()) {
            targetSource.releaseTarget(target);
        }
        // setProxyContext为ture
        if (setProxyContext) {
            // 需要将旧的代理再放回到上线文中
            AopContext.setCurrentProxy(oldProxy);
        }
    }
}
}
}

```

上面代码中2个重点：@1和@2

@1：获取当前方法的拦截器链，这个在JdkDynamicAopProxy的也有，稍后说。

@2：调用CglibMethodInvocation.proceed()，内部会一次调用方法拦截器链，最终会调用目标方法，获取目标方法返回值，这个稍后放在代理方法处理阶段详解。

下面来看一下方法拦截器链的获取。

方法拦截器链的获取

我们在创建代理的时候，增强的代码通常都放在Advise通知中，但是最终调用方法的时候，这些通知都会被转换为MethodInterceptor来执行，调用方法的过程中，需要先获取方法上匹配的所有方法连接器，然后依次执行，最终会调用到目标方法。

获取方法对应的拦截器链，对应下面这段代码

```

org.springframework.aop.framework.AdvisedSupport#getInterceptorsAndDynamicInterceptionAdvice

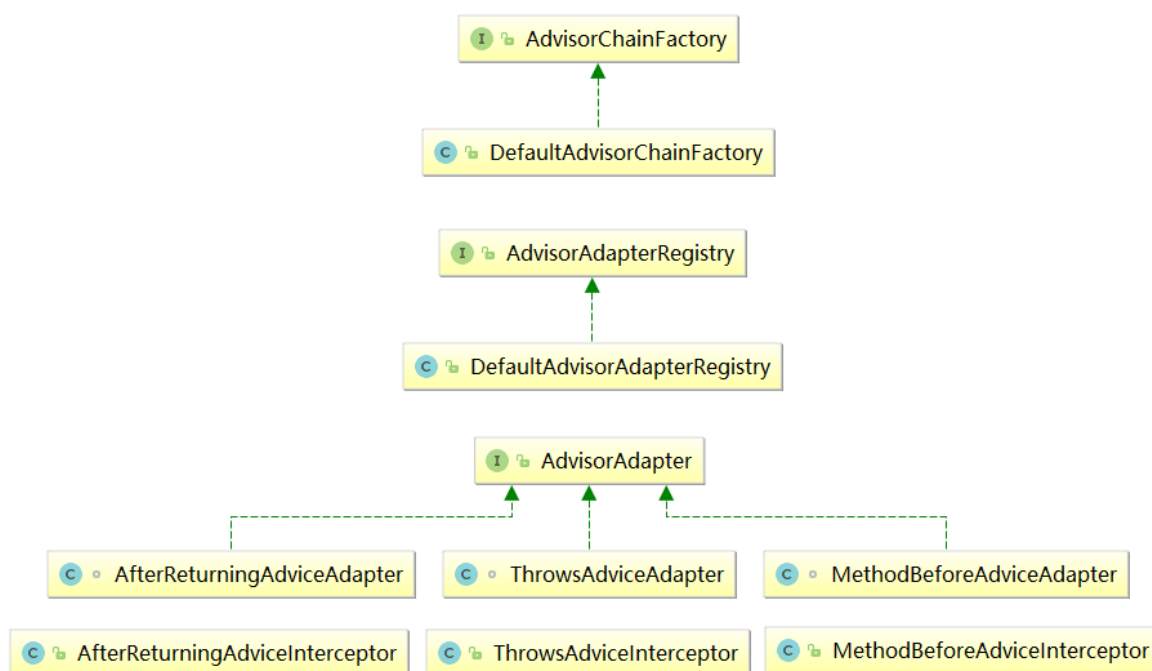
AdvisorChainFactory advisorChainFactory = new DefaultAdvisorChainFactory();

public List<Object> getInterceptorsAndDynamicInterceptionAdvice(Method method,
    @Nullable Class<?> targetClass) {
    MethodCacheKey cacheKey = new MethodCacheKey(method);
    List<Object> cached = this.methodCache.get(cacheKey);
    if (cached == null) {
        cached =
this.advisorChainFactory.getInterceptorsAndDynamicInterceptionAdvice(
            this, method, targetClass);
        this.methodCache.put(cacheKey, cached);
    }
    return cached;
}
}

```


会调用 `DefaultAdvisorChainFactory#getInterceptorsAndDynamicInterceptionAdvice` 方法 获取方法上匹配的拦截器链。

涉及到的类



AdvisorChainFactory接口

拦截器链工厂接口，定义了一个方法，用来获取方法匹配的拦截器链列表

```
package org.springframework.aop.framework;

public interface AdvisorChainFactory {

    /**
     * 获取方法匹配的拦截器链列表
     * @param config: 代理配置信息，里面包含了创建代理的所有信息，如：Advisor列表，此方法会
     从Advisor列表中找到和method匹配的
     * @param targetClass: 目标类
     */
    List<Object> getInterceptorsAndDynamicInterceptionAdvice(Advised config,
        Method method, @Nullable Class<?> targetClass);
}
```

DefaultAdvisorChainFactory类

AdvisorChainFactory接口的默认实现。

```
public class DefaultAdvisorChainFactory implements AdvisorChainFactory,
    Serializable {

    @Override
    public List<Object> getInterceptorsAndDynamicInterceptionAdvice(
        Advised config, Method method, @Nullable Class<?> targetClass) {

        // 获取Advisor适配器注册器，前面我们有提到过一个知识点：所有的Advisor最终都会转换为
        MethodInterceptor类型的，
        // 然后注册方法调用链去执行，AdvisorAdapterRegistry就是搞这个事情，
```

```

        // 其内部会将非MethodInterceptor类型通知通过适配器转换为MethodInterceptor类型
        AdvisorAdapterRegistry registry =
GlobalAdvisorAdapterRegistry.getInstance();
        //获取配置中的Advisor列表
        Advisor[] advisors = config.getAdvisors();
        List<Object> interceptorList = new ArrayList<>(advisors.length);
        //获取被调用方法所在类实际的类型
        Class<?> actualClass = (targetClass != null ? targetClass :
method.getDeclaringClass());
        Boolean hasIntroductions = null;
        //遍历Advisor列表，找到和actualClass和方法匹配的所有方法拦截器
        (MethodInterceptor) 链列表
        for (Advisor advisor : advisors) {
            //判断是否是PointcutAdvisor类型的，这种类型的匹配分为2个阶段，先看类是否匹配，
            然后再看方法是否匹配
            if (advisor instanceof PointcutAdvisor) {
                PointcutAdvisor pointcutAdvisor = (PointcutAdvisor) advisor;
                // 如果isPreFiltered为ture，表示类以及匹配过，不需要看类是否匹配了
                if (config.isPreFiltered() ||
pointcutAdvisor.getPointcut().getClassFilter().matches(actualClass)) {
                    MethodMatcher mm =
pointcutAdvisor.getPointcut().getMethodMatcher();
                    boolean match;
                    if (mm instanceof IntroductionAwareMethodMatcher) {
                        if (hasIntroductions == null) {
                            hasIntroductions =
hasMatchingIntroductions(advisors, actualClass);
                        }
                        match = ((IntroductionAwareMethodMatcher)
mm).matches(method, actualClass, hasIntroductions);
                    }
                    else {
                        //方法是否匹配
                        match = mm.matches(method, actualClass);
                    }
                    //方法匹配
                    if (match) {
                        // 通过AdvisorAdapterRegistry的getInterceptors将advisor转换
                        为MethodInterceptor列表
                        MethodInterceptor[] interceptors =
registry.getInterceptors(advisor);
                        //方法是否动态匹配
                        if (mm.isRuntime()) {
                            //轮询连接器，将其包装为
                            InterceptorAndDynamicMethodMatcher对象，后续方法调用的时候可以做动态匹配
                            for (MethodInterceptor interceptor : interceptors) {
                                interceptorList.add(new
InterceptorAndDynamicMethodMatcher(interceptor, mm));
                            }
                        }
                        else {
                            interceptorList.addAll(Arrays.asList(interceptors));
                        }
                    }
                }
            }
            else if (advisor instanceof IntroductionAdvisor) {
                IntroductionAdvisor ia = (IntroductionAdvisor) advisor;

```

```

        if (config.isPreFiltered() ||
            ia.getClassFilter().matches(actualClass)) {
            Interceptor[] interceptors =
                registry.getInterceptors(advisor);
            interceptorList.addAll(Arrays.asList(interceptors));
        }
    }
    else {
        Interceptor[] interceptors = registry.getInterceptors(advisor);
        interceptorList.addAll(Arrays.asList(interceptors));
    }
}

return interceptorList;
}
}

```

下面来看AdvisorAdapterRegistry这个接口。

AdvisorAdapterRegistry接口

AdvisorAdapter注册器，AdvisorAdapter可以将Advisor中的Advice适配为MethodInterceptor

```

package org.springframework.aop.framework.adapter;

public interface AdvisorAdapterRegistry {

    //将一个通知（Advice）包装为Advisor对象
    Advisor wrap(Object advice) throws UnknownAdviceTypeException;

    //根据Advisor获取方法MethodInterceptor列表
    MethodInterceptor[] getInterceptors(Advisor advisor) throws
        UnknownAdviceTypeException;

    //注册AdvisorAdapter，AdvisorAdapter可以将Advisor中的Advice适配为
    MethodInterceptor
    void registerAdvisorAdapter(AdvisorAdapter adapter);
}

```

DefaultAdvisorAdapterRegistry类

AdvisorAdapterRegistry的默认实现，目前里面做的事情主要是将负责将前置通知，异常通知，后置通知转换为MethodInterceptor类型的，源码比较简单，大家看一下就懂了。

```

public class DefaultAdvisorAdapterRegistry implements AdvisorAdapterRegistry,
    Serializable {

    //AdvisorAdapter转换器列表，AdvisorAdapter负责将Advisor中的Advice转换为
    MethodInterceptor类型的
    private final List<AdvisorAdapter> adapters = new ArrayList<>(3);

    //默认会注册3个AdvisorAdapter，这3个负责将前置通知，异常通知，后置通知转换为
    MethodInterceptor类型的
    public DefaultAdvisorAdapterRegistry() {
        registerAdvisorAdapter(new MethodBeforeAdviceAdapter());
        registerAdvisorAdapter(new AfterReturningAdviceAdapter());
    }
}

```

```

        registerAdvisorAdapter(new ThrowsAdviceAdapter());
    }

    @Override
    public Advisor wrap(Object adviceObject) throws UnknownAdviceTypeException {
        if (adviceObject instanceof Advisor) {
            return (Advisor) adviceObject;
        }
        if (!(adviceObject instanceof Advice)) {
            throw new UnknownAdviceTypeException(adviceObject);
        }
        Advice advice = (Advice) adviceObject;
        if (advice instanceof MethodInterceptor) {
            // So well-known it doesn't even need an adapter.
            return new DefaultPointcutAdvisor(advice);
        }
        //轮询adapters
        for (AdvisorAdapter adapter : this.adapters) {
            //adapter是否支持适配advice这个通知
            if (adapter.supportsAdvice(advice)) {
                return new DefaultPointcutAdvisor(advice);
            }
        }
        throw new UnknownAdviceTypeException(advice);
    }

    //将Advisor对象转换为MethodInterceptor列表，不过通常情况下一个advisor会返回一个
    MethodInterceptor
    @Override
    public MethodInterceptor[] getInterceptors(Advisor advisor) throws
    UnknownAdviceTypeException {
        List<MethodInterceptor> interceptors = new ArrayList<>(3);
        Advice advice = advisor.getAdvice();
        if (advice instanceof MethodInterceptor) {
            interceptors.add((MethodInterceptor) advice);
        }
        //轮询adapters
        for (AdvisorAdapter adapter : this.adapters) {
            //先看一下adapter是否支持适配advice这个通知
            if (adapter.supportsAdvice(advice)) {
                //如果匹配，这调用适配器的getInterceptor方法将advisor转换为
                MethodInterceptor
                interceptors.add(adapter.getInterceptor(advisor));
            }
        }
        if (interceptors.isEmpty()) {
            throw new UnknownAdviceTypeException(advisor.getAdvice());
        }
        return interceptors.toArray(new MethodInterceptor[0]);
    }

    @Override
    public void registerAdvisorAdapter(AdvisorAdapter adapter) {
        this.adapters.add(adapter);
    }
}

```

AdvisorAdapter接口

```
package org.springframework.aop.framework.adapter;

public interface AdvisorAdapter {
    //判断这个适配器支持advice这个通知么
    boolean supportsAdvice(Advice advice);

    //获取advisor对应的MethodInterceptor
    MethodInterceptor getInterceptor(Advisor advisor);
}
```

MethodBeforeAdviceAdapter类

适配 MethodBeforeAdvice 前置通知，负责将 MethodBeforeAdvice 类型的通知转换为 MethodBeforeAdviceInterceptor 类型的

```
class MethodBeforeAdviceAdapter implements AdvisorAdapter, Serializable {

    @Override
    public boolean supportsAdvice(Advice advice) {
        return (advice instanceof MethodBeforeAdvice);
    }

    @Override
    public MethodInterceptor getInterceptor(Advisor advisor) {
        MethodBeforeAdvice advice = (MethodBeforeAdvice) advisor.getAdvice();
        return new MethodBeforeAdviceInterceptor(advice);
    }

}
```

MethodBeforeAdviceInterceptor类

将 MethodBeforeAdvice 通知适配为 MethodInterceptor 类型的，代码很简单，大家一看就懂。

```
package org.springframework.aop.framework.adapter;

public class MethodBeforeAdviceInterceptor implements MethodInterceptor,
BeforeAdvice, Serializable {

    private final MethodBeforeAdvice advice;

    public MethodBeforeAdviceInterceptor(MethodBeforeAdvice advice) {
        Assert.notNull(advice, "Advice must not be null");
        this.advice = advice;
    }

    @Override
    public Object invoke(MethodInvocation mi) throws Throwable {
        //先调用前置通知
        this.advice.before(mi.getMethod(), mi.getArguments(), mi.getThis());
        //然后继续处理连接器连，内部会调用目标方法
        return mi.proceed();
    }

}
```

```
}  
  
}
```

AfterReturningAdviceAdapter类

适配 `AfterReturningAdvice` 后置通知，负责将 `AfterReturningAdvice` 类型的通知转换为 `AfterReturningAdviceInterceptor` 类型的

```
class AfterReturningAdviceAdapter implements AdvisorAdapter, Serializable {  
  
    @Override  
    public boolean supportsAdvice(Advice advice) {  
        return (advice instanceof AfterReturningAdvice);  
    }  
  
    @Override  
    public MethodInterceptor getInterceptor(Advisor advisor) {  
        AfterReturningAdvice advice = (AfterReturningAdvice)  
advisor.getAdvice();  
        return new AfterReturningAdviceInterceptor(advice);  
    }  
  
}
```

AfterReturningAdviceInterceptor类

将 `AfterReturningAdvice` 通知适配为 `MethodInterceptor` 类型的，代码很简单，大家一看就懂。

```
public class AfterReturningAdviceInterceptor implements MethodInterceptor,  
AfterAdvice, Serializable {  
  
    private final AfterReturningAdvice advice;  
  
    public AfterReturningAdviceInterceptor(AfterReturningAdvice advice) {  
        Assert.notNull(advice, "Advice must not be null");  
        this.advice = advice;  
    }  
  
    @Override  
    public Object invoke(MethodInvocation mi) throws Throwable {  
        //先调用拦截器链，内部会调用目标方法  
        Object retVal = mi.proceed();  
        //然后执行后置通知  
        this.advice.afterReturning(retVal, mi.getMethod(), mi.getArguments(),  
mi.getThis());  
        //返回结果  
        return retVal;  
    }  
  
}
```

ThrowsAdviceAdapter类

适配 ThrowsAdvice 前置通知，负责将 MethodBeforeAdvice 类型的通知转换为 MethodBeforeAdviceInterceptor 类型的

```
class ThrowsAdviceAdapter implements AdvisorAdapter, Serializable {

    @Override
    public boolean supportsAdvice(Advice advice) {
        return (advice instanceof ThrowsAdvice);
    }

    @Override
    public MethodInterceptor getInterceptor(Advisor advisor) {
        return new ThrowsAdviceInterceptor(advisor.getAdvice());
    }

}
```

MethodBeforeAdviceInterceptor类

将 ThrowsAdvice 通知适配为 MethodInterceptor 类型的，代码很简单，大家一看就懂。

```
package org.springframework.aop.framework.adapter;

public class ThrowsAdviceInterceptor implements MethodInterceptor, AfterAdvice {

    private static final String AFTER_THROWING = "afterThrowing";

    private final Object throwsAdvice;

    //创建ThrowsAdviceInterceptor
    public ThrowsAdviceInterceptor(Object throwsAdvice) {
        Assert.notNull(throwsAdvice, "Advice must not be null");
        this.throwsAdvice = throwsAdvice;
        //获取异常通知中定义的所有方法（public、默认的、protected、private）
        Method[] methods = throwsAdvice.getClass().getMethods();
        //轮询methods
        for (Method method : methods) {
            //方法名称为afterThrowing && 方法参数为1或者4
            if (method.getName().equals(AFTER_THROWING) &&
                (method.getParameterCount() == 1 ||
                 method.getParameterCount() == 4)) {
                //获取方法的最后一个参数类型
                Class<?> throwableParam = method.getParameterTypes()
                    [method.getParameterCount() - 1];
                //判断方法参数类型是不是Throwable类型的
                if (Throwable.class.isAssignableFrom(throwableParam)) {
                    // 缓存异常处理方法到map中（异常类型->异常处理方法）
                    this.exceptionHandlerMap.put(throwableParam, method);
                }
            }
        }
        //如果exceptionHandlerMap，抛出异常，所以最少要有一个异常处理方法
        if (this.exceptionHandlerMap.isEmpty()) {
            throw new IllegalArgumentException(
```

```

        "At least one handler method must be found in class [" +
throwsAdvice.getClass() + "]);
    }
}

/**
 * 获取异常通知中自定义的处理异常方法的数量
 */
public int getHandlerMethodCount() {
    return this.exceptionHandlerMap.size();
}

@Override
public Object invoke(MethodInvocation mi) throws Throwable {
    try {
        //调用通知链
        return mi.proceed();
    }
    catch (Throwable ex) {
        //获取异常通知中自定义的处理异常的方法
        Method handlerMethod = getExceptionHandler(ex);
        //当处理的方法不为空
        if (handlerMethod != null) {
            //调用异常处理方法
            invokeHandlerMethod(mi, ex, handlerMethod);
        }
        //继续向外抛出异常
        throw ex; //@1
    }
}

/**
 * 获取throwsAdvice中处理exception参数指定的异常的方法
 */
@Nullable
private Method getExceptionHandler(Throwable exception) {
    //获取异常类型
    Class<?> exceptionClass = exception.getClass();
    //从缓存中获取异常类型对应的方法
    Method handler = this.exceptionHandlerMap.get(exceptionClass);
    //来一个循环，查询处理方法，循环条件：方法为空 && 异常类型!=Throwable
    while (handler == null && exceptionClass != Throwable.class) {
        //获取异常的父类型
        exceptionClass = exceptionClass.getSuperclass();
        //从缓存中查找异常对应的处理方法
        handler = this.exceptionHandlerMap.get(exceptionClass);
    }
    //将查找结果返回
    return handler;
}

//通过反射调用异常通知中的异常方法
private void invokeHandlerMethod(MethodInvocation mi, Throwable ex, Method
method) throws Throwable {
    //构建方法请求参数
    Object[] handlerArgs;

```



```

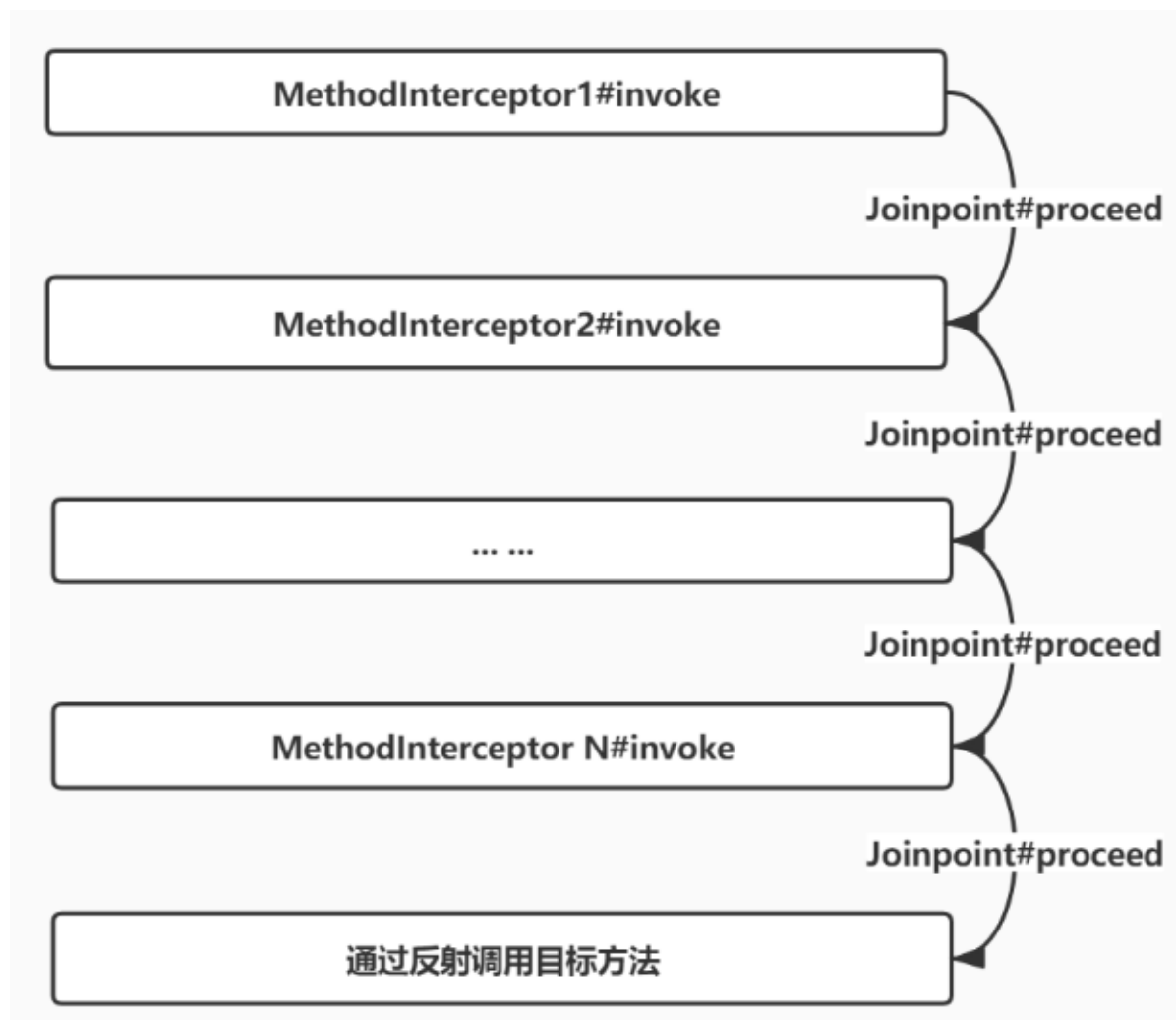
//若只有1个参数，参数为：异常对象
if (method.getParameterCount() == 1) {
    handlerArgs = new Object[] {ex};
}
else {
    //4个参数（方法、方法请求参数、目标对象、异常对象）
    handlerArgs = new Object[] {mi.getMethod(), mi.getArguments(),
mi.getThis(), ex};
}
try {
    //通过反射调用异常通知中的方法
    method.invoke(this.throwsAdvice, handlerArgs);
}
catch (InvocationTargetException targetEx) {
    throw targetEx.getTargetException();
}
}
}

```

代理方法的调用过程（拦截器链的执行）

拦截器链执行过程

到目前，已经获取到代理对象，接着会开始使用这个代理对象，在代理对象上执行一些方法调用，此时会依次调用此方法上的所有MethodInterceptor，最终会调用到目标上对应的方法，执行过程如下图



jdk动态代理方式创建代理最终会调用 `ReflectiveMethodInvocation#proceed` 方法。

cglib方式创建的代理最终会调用 `CglibAopProxy.CglibMethodInvocation#proceed` 方法。

下面来看一下这两个类的代码。

ReflectiveMethodInvocation类

```
package org.springframework.aop.framework;

public class ReflectiveMethodInvocation implements ProxyMethodInvocation,
Cloneable {

    //生成的代理对象
    protected final Object proxy;
    //被代理的目标对象
    protected final Object target;
    //被调用的方法
    protected final Method method;
    //调用方法传入参数
    protected Object[] arguments;
    //目标对象类型
    private final Class<?> targetClass;

    /**
     * 当期被调用的方法上匹配的 MethodInterceptor and
     InterceptorAndDynamicMethodMatcher 列表
     * 即方法调用链列表
     */
    protected final List<?> interceptorsAndDynamicMethodMatchers;

    //当前正在调用的连接器索引
    private int currentInterceptorIndex = -1;

    //构造器
    protected ReflectiveMethodInvocation(
        Object proxy, @Nullable Object target, Method method, @Nullable
        Object[] arguments,
        @Nullable Class<?> targetClass, List<Object>
        interceptorsAndDynamicMethodMatchers) {
        this.proxy = proxy;
        this.target = target;
        this.targetClass = targetClass;
        //获取桥接方法，关于什么是桥接方法，比较简单，百度一下，这里不做说明
        this.method = BridgeMethodResolver.findBridgedMethod(method);
        this.arguments = AopProxyUtils.adaptArgumentsIfNecessary(method,
        arguments);
        this.interceptorsAndDynamicMethodMatchers =
        interceptorsAndDynamicMethodMatchers;
    }

    //这里是重点，用来处理被调用的方法，会递归进行调用，所有的拦截器都执行完毕之后，会通过反射调
    用目标方法
    public Object proceed() throws Throwable {
        // 拦截器都执行完毕之后，通过反射调用目标对象中的方法
        if (this.currentInterceptorIndex ==
        this.interceptorsAndDynamicMethodMatchers.size() - 1) {
```

```

        return invokeJoinpoint();
    }

    //获取++this.currentInterceptorIndex指定的拦截器
    Object interceptorOrInterceptionAdvice =

    this.interceptorsAndDynamicMethodMatchers.get(++this.currentInterceptorIndex);
    //判断拦截器是否是InterceptorAndDynamicMethodMatcher，这种表示是动态拦截器，
    // 所谓动态拦截器就是要根据方法的参数的值来判断拦截器是否需要执行
    if (interceptorOrInterceptionAdvice instanceof
    InterceptorAndDynamicMethodMatcher) {
        InterceptorAndDynamicMethodMatcher dm =
            (InterceptorAndDynamicMethodMatcher)
            interceptorOrInterceptionAdvice;
        Class<?> targetClass = (this.targetClass != null ? this.targetClass
        : this.method.getDeclaringClass());
        //判断动态拦截器是否需要执行
        if (dm.methodMatcher.matches(this.method, targetClass,
        this.arguments)) {
            //执行当前拦截器的调用
            return dm.interceptor.invoke(this);
        }
        else {
            //如果不匹配，直接递归进入下一个拦截器的调用
            return proceed();
        }
    }
    else {
        //执行拦截器的调用
        return ((MethodInterceptor)
        interceptorOrInterceptionAdvice).invoke(this);
    }
}

//通过反射调用目标方法
@Nullable
protected Object invokeJoinpoint() throws Throwable {
    return AopUtils.invokeJoinpointUsingReflection(this.target, this.method,
    this.arguments);
}
}

```

ProxyFactory简化代理的创建

上面代理的整个创建过程和使用过程还是挺复杂的，spring在 `AdvisedSupport` 类的基础上又添加2个子类

- ProxyCreatorSupport
- ProxyFactory

通过这2个子类，将步骤稍微简化了一些，这2个类的代码比较简单，上面的如果理解了，看这2个类的代码会非常的轻松，源码这里就不细说了。

`ProxyCreatorSupport` 用来对代理的创建提供支持，内部添加了 `AopProxyFactory` 对象的引用，将代理的创建过程给简化了。

```
AopProxyFactory aopProxyFactory;
```

`ProxyFactory` 类继承了 `ProxyCreatorSupport`，让创建代理的过程更简单了，如果采用硬编码的方式，通常我们会使用 `ProxyFactory` 来创建代理对象，代码只需要下面几行了

```
//通过spring提供的代理创建工厂来创建代理
ProxyFactory proxyFactory = new ProxyFactory();
//ProxyFactory继承了AdvisedSupport类，所以可以直接可以通过ProxyFactory来设置创建代理需要的参数
//为工厂指定目标对象
proxyFactory.setTarget(target);
//添加顾问
proxyFactory.addAdvisor(advisor);
//调用proxyFactory.getProxy();创建代理
Object proxy = proxyFactory.getProxy();
```

案例

下面来一些案例，通过案例理解会更容易一些。

案例1

这个案例主要看一下生成的代理对象的一些信息。

```
package com.javacode2018.aop.demo5;

import com.javacode2018.aop.demo4.FundsService;
import org.springframework.aop.MethodBeforeAdvice;
import org.springframework.aop.framework.ProxyFactory;
import org.springframework.aop.support.DefaultPointcutAdvisor;
import org.springframework.lang.Nullable;

import java.lang.reflect.Method;

public class AopTest5 {
    public static void main(String[] args) {
        ProxyFactory proxyFactory = new ProxyFactory();
        proxyFactory.setTarget(new FundsService());
        proxyFactory.addAdvisor(new DefaultPointcutAdvisor(new
MethodBeforeAdvice() {
            @Override
            public void before(Method method, Object[] args, @Nullable Object
target) throws Throwable {
                System.out.println(method);
            }
        }));
        //创建代理对象
        Object proxy = proxyFactory.getProxy();
        System.out.println("代理对象的类型: " + proxy.getClass());
        System.out.println("代理对象的父类: " + proxy.getClass().getSuperclass());
        System.out.println("代理对象实现的接口列表");
        for (Class<?> cf : proxy.getClass().getInterfaces()) {
            System.out.println(cf);
        }
    }
}
```

```
    }  
    }  
}
```

运行输出

```
代理对象的类型: class  
com.javacode2018.aop.demo4.FundsService$$EnhancerBySpringCGLIB$$ad14161a  
代理对象的父类: class com.javacode2018.aop.demo4.FundsService  
代理对象实现的接口列表  
interface org.springframework.aop.SpringProxy  
interface org.springframework.aop.framework.Advised  
interface org.springframework.cglib.proxy.Factory
```

输出中可以看出默认帮我们实现了3个接口 [SpringProxy, Advised, Factory]

案例2：代理接口

有接口的情况默认会通过jdk动态代理的方式生成代理，下面来看一下。

来个接口

```
package com.javacode2018.aop.demo6;  
  
public interface IService {  
    void say(String name);  
}
```

实现类

```
package com.javacode2018.aop.demo6;  
  
public class Service implements IService {  
    @Override  
    public void say(String name) {  
        System.out.println("hello: " + name);  
    }  
}
```

测试案例

```
package com.javacode2018.aop.demo6;  
  
import org.junit.Test;  
import org.springframework.aop.MethodBeforeAdvice;  
import org.springframework.aop.framework.ProxyFactory;  
import org.springframework.lang.Nullable;  
  
import java.lang.reflect.Method;  
  
public class AopTest6 {  
    @Test  
    public void test1() {  
        Service target = new Service();
```

```

ProxyFactory proxyFactory = new ProxyFactory();
//设置需要被代理的对象
proxyFactory.setTarget(target);
//设置需要代理的接口
proxyFactory.addInterface(IService.class);

proxyFactory.addAdvice(new MethodBeforeAdvice() {
    @Override
    public void before(Method method, Object[] args, @Nullable Object
target) throws Throwable {
        System.out.println(method);
    }
});

IService proxy = (IService) proxyFactory.getProxy();
System.out.println("代理对象的类型: " + proxy.getClass());
System.out.println("代理对象的父类: " + proxy.getClass().getSuperclass());
System.out.println("代理对象实现的接口列表");
for (Class<?> cf : proxy.getClass().getInterfaces()) {
    System.out.println(cf);
}
//调用代理的方法
System.out.println("\n调用代理的方法");
proxy.say("spring aop");
}
}

```

运行输出

```

代理对象的类型: class com.sun.proxy.$Proxy4
代理对象的父类: class java.lang.reflect.Proxy
代理对象实现的接口列表
interface com.javacode2018.aop.demo6.IService
interface org.springframework.aop.SpringProxy
interface org.springframework.aop.framework.Advised
interface org.springframework.core.DecoratingProxy

调用代理的方法
public abstract void com.javacode2018.aop.demo6.IService.say(java.lang.String)
hello: spring aop

```

从第一行输出中可以看出是采用jdk动态代理方式创建的代理

第二行验证了，所有通过jdk动态代理方式创建的代理对象都是Proxy的子类

输出的接口列表中可以看出，默认帮我们实现了3个接口[SpringAop,Advised,DecoratingProxy]

案例3：强制使用cglib代理

在案例2中加入下面代码，设置 proxyTargetClass 为 true，会强制使用cglib代理。

```

//强制使用cglib代理
proxyFactory.setProxyTargetClass(true);

```

案例4：将代理暴露在threadLocal中

先来看一段代码，Service类中有2个方法，m1方法中会调用m2，通过aop代理对这个类创建了一个代理，通过代理来统计所有调用方法的耗时

```
package com.javacode2018.aop.demo7;

class Service {
    public void m1() {
        System.out.println("m1");
        this.m2();
    }

    public void m2() {
        System.out.println("m2");
    }
}

public class AopTest7 {
    @Test
    public void test1() {
        Service target = new Service();

        ProxyFactory proxyFactory = new ProxyFactory();
        proxyFactory.setTarget(target);

        proxyFactory.addAdvice(new MethodInterceptor() {
            @Override
            public Object invoke(MethodInvocation invocation) throws Throwable {
                long startTime = System.nanoTime();
                Object result = invocation.proceed();
                long endTime = System.nanoTime();
                System.out.println(String.format("%s方法耗时(纳秒):%s",
                    invocation.getMethod().getName(), endTime - startTime));
                return result;
            }
        });

        Service proxy = (Service) proxyFactory.getProxy();
        proxy.m1();
    }
}
```

运行输出

```
m1
m2
m1方法耗时(纳秒):11056000
```

为什么没有输出m2方法的耗时?

原因：m2方法是在m1方法中通过this的方式来调用的，this实际上指向的是上面代码中的 target 对象。

那么我们如何能让此处的m2也能被增强，你需要通过代理来调用m2方法才可以，可以将代理对象暴露在threadLocal中，然后在m1方法中获取到threadLocal中的代理对象，通过代理对象来调用m2就可以了。

需要调整改动2处。

第1处：配置代理创建时，将其暴露出去

```
proxyFactory.setExposeProxy(true);
```

第2处：m1中调用m2的方法需要修改为下面这样

```
((Service)AopContext.currentProxy()).m2();
```

再次执行，结果如下

```
m1  
m2  
m2方法耗时(纳秒):51200  
m1方法耗时(纳秒):12794500
```

这个功能还是挺有用的，以后我估计大家是可以用到的。

总结

本文内容比较多，大家好好吸收一下，有问题的欢迎留言。

下一篇文章将深入详解介绍spring中如何将aop搞成自动化的，东西也是比较多的，敬请期待。

案例源码

```
https://gitee.com/javacode2018/spring-series
```

路人甲java所有案例代码以后都会放到这个上面，大家watch一下，可以持续关注动态。

Spring系列

1. [Spring系列第1篇：为何要学spring?](#)
2. [Spring系列第2篇：控制反转（IoC）与依赖注入（DI）](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)
4. [Spring系列第4篇：xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇：创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇：玩转bean scope，避免跳坑里!](#)
7. [Spring系列第7篇：依赖注入之手动注入](#)
8. [Spring系列第8篇：自动注入（autowire）详解，高手在于坚持](#)
9. [Spring系列第9篇：depend-on到底是干什么的?](#)
10. [Spring系列第10篇：primary可以解决什么问题?](#)
11. [Spring系列第11篇：bean中的autowire-candidate又是干什么的?](#)
12. [Spring系列第12篇：lazy-init: bean延迟初始化](#)

13. [Spring系列第13篇：使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇：lookup-method和replaced-method比较陌生，怎么玩的？](#)
15. [Spring系列第15篇：代理详解（Java动态代理&cglib代理）？](#)
16. [Spring系列第16篇：深入理解java注解及spring对注解的增强（预备知识）](#)
17. [Spring系列第17篇：@Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇：@ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇：@import详解\(bean批量注册\)](#)
20. [Spring系列第20篇：@Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇：注解实现依赖注入（@Autowired、@Resource、@Primary、@Qualifier）](#)
22. [Spring系列第22篇：@Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇：Bean生命周期详解](#)
24. [Spring系列第24篇：父子容器详解](#)
25. [Spring系列第25篇：@Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇：国际化详解](#)
27. [Spring系列第27篇：spring事件机制详解](#)
28. [Spring系列第28篇：Bean循环依赖详解](#)
29. [Spring系列第29篇：BeanFactory扩展（BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor）](#)
30. [Spring系列第30篇：jdk动态代理和cglib代理](#)
31. [Spring系列第31篇：aop概念详解](#)

更多好文章

1. [Java高并发系列（共34篇）](#)
2. [MySQL高手系列（共27篇）](#)
3. [Maven高手系列（共10篇）](#)
4. [Mybatis系列（共12篇）](#)
5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要，它是什么？怎么实现？](#)
7. [泛型，有点难度，会让很多人懵逼，那是因为你没有看这篇文章！](#)

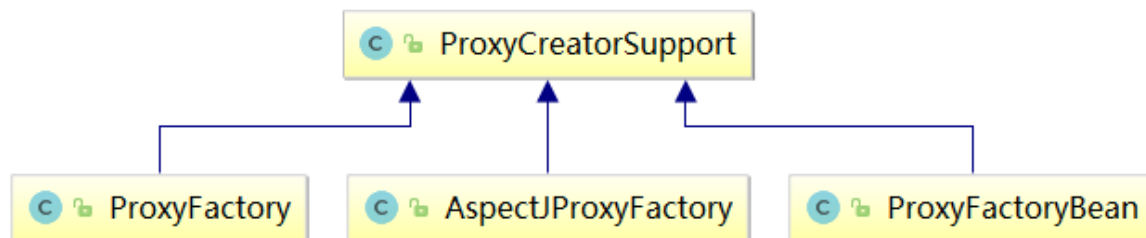


领取福利，扫码发送java

- 1、1000本java电子书大全
- 2、2021版java面试通关手册
- 3、月薪5万必备的java微服务学习线路图及高清视频
- 4、邀请加入高质量java技术交流群

加我微信：spring888java，进群一起交流java相关各种技术！！

Spring系列第33篇：ProxyFactoryBean创建AOP代理



ProxyFactory方式

这种是硬编码的方式，可以脱离spring直接使用，用到的比较多，自动化方式创建代理中都是依靠 `ProxyFactory` 来实现的，所以这种方式的原理大家一定要了解，上篇文章中已经有介绍过了，不清楚的可以去看一下：[Spring系列第32篇：AOP核心源码、原理详解](#)

AspectJProxyFactory方式

AspectJ是一个面向切面的框架，是目前最好用，最方便的AOP框架，Spring将其集成进来了，通过AspectJ提供的一些功能实现aop代理非常方便，下篇文章将详解。

ProxyFactoryBean方式

Spring环境中给指定的bean创建代理的一种方式，本文主要介绍这个。

ProxyFactoryBean

这个类实现了一个接口 `FactoryBean`，`FactoryBean` 不清楚的可以看一下：[Spring系列第5篇：创建bean实例这些方式你们都知道？](#)

`ProxyFactoryBean` 就是通过 `FactoryBean` 的方式来给指定的bean创建一个代理对象。

创建代理，有3个信息比较关键：

1. 需要增强的功能，这个放在通知（Advice）中实现
2. 目标对象（target）：表示你需要给哪个对象进行增强
3. 代理对象（proxy）：将增强的功能和目标对象组合在一起，然后形成的一个代理对象，通过代理对象来访问目标对象，起到对目标对象增强的效果。

使用 `ProxyFactoryBean` 也是围绕着3部分来的，`ProxyFactoryBean` 使用的步骤：

1. 创建 `ProxyFactoryBean` 对象
2. 通过 `ProxyFactoryBean.setTargetName` 设置目标对象的bean名称，目标对象是spring容器中的一个bean
3. 通过 `ProxyFactoryBean.setInterceptorNames` 添加需要增强的通知
4. 将 `ProxyFactoryBean` 注册到Spring容器，假设名称为 `proxyBean`
5. 从Spring查找名称为 `proxyBean` 的bean，这个bean就是生成好的代理对象

上案例。

来个类 `Service1`

```

package com.javacode2018.aop.demo8.test1;

public class Service1 {

    public void m1() {
        System.out.println("我是 m1 方法");
    }

    public void m2() {
        System.out.println("我是 m2 方法");
    }

}

```

需求

在spring容器中注册上面这个类的bean，名称为 `service1`，通过代理的方式来对这个bean进行增强，来2个通知

一个前置通知：在调用 `service1` 中的任意方法之前，输出一条信息：准备调用xxxx方法

一个环绕通知：复制统计所有方法的耗时。

下面是代码的实现

```

package com.javacode2018.aop.demo8.test1;

import org.aopalliance.intercept.MethodInterceptor;
import org.aopalliance.intercept.MethodInvocation;
import org.springframework.aop.MethodBeforeAdvice;
import org.springframework.aop.framework.ProxyFactoryBean;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.lang.Nullable;

import java.lang.reflect.Method;

@Configuration
public class MainConfig1 {
    //注册目标对象
    @Bean
    public Service1 service1() {
        return new Service1();
    }

    //注册一个前置通知
    @Bean
    public MethodBeforeAdvice beforeAdvice() {
        MethodBeforeAdvice advice = new MethodBeforeAdvice() {
            @Override
            public void before(Method method, Object[] args, @Nullable Object
target) throws Throwable {
                System.out.println("准备调用: " + method);
            }
        };
        return advice;
    }
}

```

```

//注册一个后置通知
@Bean
public MethodInterceptor costTimeInterceptor() {
    MethodInterceptor methodInterceptor = new MethodInterceptor() {
        @Override
        public Object invoke(MethodInvocation invocation) throws Throwable {
            long startTime = System.nanoTime();
            Object result = invocation.proceed();
            long endTime = System.nanoTime();
            System.out.println(invocation.getMethod() + ",耗时(纳秒): " +
(endTime - startTime));
            return result;
        }
    };
    return methodInterceptor;
}

//注册ProxyFactoryBean
@Bean
public ProxyFactoryBean service1Proxy() {
    //1.创建ProxyFactoryBean
    ProxyFactoryBean proxyFactoryBean = new ProxyFactoryBean();
    //2.设置目标对象的bean名称
    proxyFactoryBean.setTargetName("service1");
    //3.设置拦截器的bean名称列表，此处2个（advice1和advice2）
    proxyFactoryBean.setInterceptorNames("beforeAdvice",
"costTimeInterceptor");
    return proxyFactoryBean;
}
}

```

下面启动spring容器，并获取代理对象

```

AnnotationConfigApplicationContext context = new
AnnotationConfigApplicationContext(MainConfig1.class);
//获取代理对象，代理对象bean的名称为注册ProxyFactoryBean的名称，即：service1Proxy
Service1 bean = context.getBean("service1Proxy", Service1.class);
System.out.println("-----");
//调用代理的方法
bean.m1();
System.out.println("-----");
//调用代理的方法
bean.m2();

```

运行输出

```

-----
准备调用: public void com.javacode2018.aop.demo8.test1.Service1.m1()
我是 m1 方法
public void com.javacode2018.aop.demo8.test1.Service1.m1(),耗时(纳秒): 8680400
-----
准备调用: public void com.javacode2018.aop.demo8.test1.Service1.m2()
我是 m2 方法
public void com.javacode2018.aop.demo8.test1.Service1.m2(),耗时(纳秒): 82400

```

从输出中可以看到，目标对象 `service1` 已经被增强了。

ProxyFactoryBean中的interceptorNames

`interceptorNames` 用来指定拦截器的bean名称列表，常用的2种方式。

- 批量的方式
- 非批量的方式

批量的方式

使用方法

```
proxyFactoryBean.setInterceptorNames("需要匹配的bean名称*");
```

需要匹配的bean名称后面跟上一个`*`，可以用来批量的匹配，如：`interceptor*`，此时spring会从容器中找到下面2中类型的所有bean，bean名称以 `interceptor` 开头的将作为增强器

```
org.springframework.aop.Advisor  
org.aopalliance.intercept.Interceptor
```

这个地方使用的时候需要注意，批量的方式注册的时候，如果增强器的类型不是上面2种类型的，比如下面3种类型通知，我们需要将其包装为 `Advisor` 才可以，而 `MethodInterceptor` 是 `Interceptor` 类型的，可以不用包装为 `Advisor` 类型的。

```
MethodBeforeAdvice（方法前置通知）  
AfterReturningAdvice（方法后置通知）  
ThrowsAdvice（异常通知）
```

下面来个案例感受一下。

案例

下面批量注册2个增强器。

```
package com.javacode2018.aop.demo8.test2;  
  
import com.javacode2018.aop.demo8.test1.Service1;  
import org.aopalliance.intercept.MethodInterceptor;  
import org.aopalliance.intercept.MethodInvocation;  
import org.springframework.aop.Advisor;  
import org.springframework.aop.MethodBeforeAdvice;  
import org.springframework.aop.framework.ProxyFactoryBean;  
import org.springframework.aop.support.DefaultPointcutAdvisor;  
import org.springframework.context.annotation.Bean;  
import org.springframework.lang.Nullable;  
  
import java.lang.reflect.Method;  
  
public class MainConfig2 {  
    //注册目标对象  
    @Bean  
    public Service1 service1() {  
        return new Service1();  
    }  
}
```

```

//定义一个增强器: interceptor1, 内部是一个前置通知, 需要将其包装为Advisor类型的
@Bean
public Advisor interceptor1() {
    MethodBeforeAdvice advice = new MethodBeforeAdvice() {
        @Override
        public void before(Method method, Object[] args, @Nullable Object
target) throws Throwable {
            System.out.println("准备调用: " + method);
        }
    };
    DefaultPointcutAdvisor advisor = new DefaultPointcutAdvisor();
    advisor.setAdvice(advice);
    return advisor;
}

//定义一个增强器: interceptor2
@Bean
public MethodInterceptor interceptor2() {
    MethodInterceptor methodInterceptor = new MethodInterceptor() {
        @Override
        public Object invoke(MethodInvocation invocation) throws Throwable {
            long startTime = System.nanoTime();
            Object result = invocation.proceed();
            long endTime = System.nanoTime();
            System.out.println(invocation.getMethod() + ",耗时(纳秒): " +
(endTime - startTime));
            return result;
        }
    };
    return methodInterceptor;
}

//注册ProxyFactoryBean
@Bean
public ProxyFactoryBean service1Proxy() {
    //1.创建ProxyFactoryBean
    ProxyFactoryBean proxyFactoryBean = new ProxyFactoryBean();
    //2.设置目标对象的bean名称
    proxyFactoryBean.setTargetName("service1");
    //3.设置拦截器的bean名称列表, 此处批量注册
    proxyFactoryBean.setInterceptorNames("interceptor*"); //@1
    return proxyFactoryBean;
}
}

```

上面定义了2个增强器:

interceptor1: 前置通知, 包装为Advisor类型了

interceptor2: 环绕通知, MethodInterceptor类型的

测试代码

```

@Test
public void test2() {
    AnnotationConfigApplicationContext context = new
    AnnotationConfigApplicationContext(MainConfig2.class);
    //获取代理对象，代理对象bean的名称为注册ProxyFactoryBean的名称，即：service1Proxy
    Service1 bean = context.getBean("service1Proxy", Service1.class);
    System.out.println("-----");
    //调用代理的方法
    bean.m1();
    System.out.println("-----");
    //调用代理的方法
    bean.m2();
}

```

运行输出

```

-----
准备调用: public void com.javacode2018.aop.demo8.test1.Service1.m1()
我是 m1 方法
public void com.javacode2018.aop.demo8.test1.Service1.m1(),耗时(纳秒): 10326200
-----
准备调用: public void com.javacode2018.aop.demo8.test1.Service1.m2()
我是 m2 方法
public void com.javacode2018.aop.demo8.test1.Service1.m2(),耗时(纳秒): 52000

```

非批量的方式

用法

非批量的方式，需要注册多个增强器，需明确的指定多个增强器的bean名称，多个增强器按照参数中指定的顺序执行，如

```
proxyFactoryBean.setInterceptorNames("advice1","advice2");
```

advice1、advice2 对应的bean类型必须为下面列表中指定的类型，类型这块比匹配的方式范围广一些

```

MethodBeforeAdvice（方法前置通知）
AfterReturningAdvice（方法后置通知）
ThrowsAdvice（异常通知）
org.aopalliance.intercept.MethodInterceptor（环绕通知）
org.springframework.aop.Advisor（顾问）

```

下面来个案例。

案例

这次给 service1 来3个通知：前置、环绕、后置

```

package com.javacode2018.aop.demo8.test3;

import com.javacode2018.aop.demo8.test1.Service1;
import org.aopalliance.intercept.MethodInterceptor;

```



```

import org.aopalliance.intercept.MethodInvocation;
import org.springframework.aop.AfterReturningAdvice;
import org.springframework.aop.MethodBeforeAdvice;
import org.springframework.aop.framework.ProxyFactoryBean;
import org.springframework.context.annotation.Bean;
import org.springframework.lang.Nullable;

import java.lang.reflect.Method;

public class MainConfig3 {
    //注册目标对象
    @Bean
    public Service1 service1() {
        return new Service1();
    }

    //定义一个前置通知
    @Bean
    public MethodBeforeAdvice methodBeforeAdvice() {
        MethodBeforeAdvice methodBeforeAdvice = new MethodBeforeAdvice() {
            @Override
            public void before(Method method, Object[] args, @Nullable Object
target) throws Throwable {
                System.out.println("准备调用: " + method);
            }
        };
        return methodBeforeAdvice;
    }

    //定义一个环绕通知
    @Bean
    public MethodInterceptor methodInterceptor() {
        MethodInterceptor methodInterceptor = new MethodInterceptor() {
            @Override
            public Object invoke(MethodInvocation invocation) throws Throwable {
                long startTime = System.nanoTime();
                Object result = invocation.proceed();
                long endTime = System.nanoTime();
                System.out.println(invocation.getMethod() + ",耗时(纳秒): " +
(endTime - startTime));
                return result;
            }
        };
        return methodInterceptor;
    }

    //定义一个后置通知
    @Bean
    public AfterReturningAdvice afterReturningAdvice() {
        AfterReturningAdvice afterReturningAdvice = new AfterReturningAdvice() {
            @Override
            public void afterReturning(@Nullable Object returnValue, Method
method, Object[] args, @Nullable Object target) throws Throwable {
                System.out.println(method + ", 执行完毕!");
            }
        };
        return afterReturningAdvice;
    }
}

```

```
//注册ProxyFactoryBean
@Bean
public ProxyFactoryBean service1Proxy() {
    //1.创建ProxyFactoryBean
    ProxyFactoryBean proxyFactoryBean = new ProxyFactoryBean();
    //2.设置目标对象的bean名称
    proxyFactoryBean.setTargetName("service1");
    //3.设置拦截器的bean名称列表，此处批量注册
    proxyFactoryBean.setInterceptorNames("methodBeforeAdvice",
"methodInterceptor", "afterReturningAdvice");
    return proxyFactoryBean;
}
}
```

测试代码

```
@Test
public void test3() {
    AnnotationConfigApplicationContext context = new
AnnotationConfigApplicationContext(MainConfig3.class);
    //获取代理对象，代理对象bean的名称为注册ProxyFactoryBean的名称，即：service1Proxy
    Service1 bean = context.getBean("service1Proxy", Service1.class);
    System.out.println("-----");
    //调用代理的方法
    bean.m1();
    System.out.println("-----");
    //调用代理的方法
    bean.m2();
}
```

运行输出

```
-----
准备调用: public void com.javacode2018.aop.demo8.test1.Service1.m1()
我是 m1 方法
public void com.javacode2018.aop.demo8.test1.Service1.m1(), 执行完毕!
public void com.javacode2018.aop.demo8.test1.Service1.m1(), 耗时(纳秒): 12724100
-----
准备调用: public void com.javacode2018.aop.demo8.test1.Service1.m2()
我是 m2 方法
public void com.javacode2018.aop.demo8.test1.Service1.m2(), 执行完毕!
public void com.javacode2018.aop.demo8.test1.Service1.m2(), 耗时(纳秒): 76700
```

源码解析

重点在于下面这个方法

```
org.springframework.aop.framework.ProxyFactoryBean#getObject
```

源码：

```

public Object getObject() throws BeansException {
    //初始化advisor(拦截器)链
    initializeAdvisorChain();
    //是否是单例
    if (isSingleton()) {
        //创建单例代理对象
        return getSingletonInstance();
    }
    else {
        //创建多例代理对象
        return newPrototypeInstance();
    }
}

```

`initializeAdvisorChain` 方法，用来初始化advisor(拦截器)链，是根据 `interceptorNames` 配置，找到spring容器中符合条件的拦截器，将其放入创建aop代理的配置中

```

private synchronized void initializeAdvisorChain() throws AopConfigException,
BeansException {
    if (!ObjectUtils.isEmpty(this.interceptorNames)) {
        // 轮询 interceptorNames
        for (String name : this.interceptorNames) {
            //批量注册的方式：判断name是否以*结尾
            if (name.endsWith(GLOBAL_SUFFIX)) {
                //@1: 从容器中匹配查找匹配的增强器，将其添加到aop配置中
                addGlobalAdvisor((ListableBeanFactory) this.beanFactory,
                                name.substring(0, name.length() -
GLOBAL_SUFFIX.length()));
            }
            else {
                //非匹配的方式：按照name查找bean，将其包装为Advisor丢到aop配置中
                Object advice;
                //从容器中查找bean
                advice = this.beanFactory.getBean(name);
                //@2: 将advice添加到拦截器列表中
                addAdvisorOnChainCreation(advice, name);
            }
        }
    }
}

```

@1: `addGlobalAdvisor` 批量的方式Advisor，看一下源码，比较简单

```

/**
 * 添加所有全局拦截器和切入点，
 * 容器中所有类型为Advisor/Interceptor的bean，bean名称prefix开头的都会将其添加到拦截器链中
 */
private void addGlobalAdvisor(ListableBeanFactory beanFactory, String prefix) {
    //获取容器中所有类型为Advisor的bean
    String[] globalAdvisorNames =
        BeanFactoryUtils.beanNamesForTypeIncludingAncestors(beanFactory,
Advisor.class);
    //获取容器中所有类型为Interceptor的bean
    String[] globalInterceptorNames =
        BeanFactoryUtils.beanNamesForTypeIncludingAncestors(beanFactory,
Interceptor.class);
}

```

```

List<Object> beans = new ArrayList<>(globalAdvisorNames.length +
globalInterceptorNames.length);
Map<Object, String> names = new HashMap<>(beans.size());
for (String name : globalAdvisorNames) {
    Object bean = beanFactory.getBean(name);
    beans.add(bean);
    names.put(bean, name);
}
for (String name : globalInterceptorNames) {
    Object bean = beanFactory.getBean(name);
    beans.add(bean);
    names.put(bean, name);
}
//对beans进行排序, 可以实现Ordered接口, 排序规则: order asc
AnnotationAwareOrderComparator.sort(beans);
for (Object bean : beans) {
    String name = names.get(bean);
    //判断bean是否已prefix开头
    if (name.startsWith(prefix)) {
        //将其添加到拦截器链中
        addAdvisorOnChainCreation(bean, name);
    }
}
}

```

@2: addAdvisorOnChainCreation

```

private void addAdvisorOnChainCreation(Object next, String name) {
    //namedBeanToAdvisor用来将bean转换为advisor
    Advisor advisor = namedBeanToAdvisor(next);
    //将advisor添加到拦截器链中
    addAdvisor(advisor);
}

```

namedBeanToAdvisor 方法

```

private AdvisorAdapterRegistry advisorAdapterRegistry = new
DefaultAdvisorAdapterRegistry();

private Advisor namedBeanToAdvisor(Object next) {
    //将对象包装为Advisor对象
    return this.advisorAdapterRegistry.wrap(next);
}

```

对 AdvisorAdapterRegistry 不清楚的, 看一下上一篇文章: [Spring系列第32篇: AOP核心源码、原理详解](#)

advisorAdapterRegistry#wrap 方法将 adviceObject 包装为 Advisor 对象, 代码如下, 比较简单

```

public Advisor wrap(Object adviceObject) throws UnknownAdviceTypeException {
    if (adviceObject instanceof Advisor) {
        return (Advisor) adviceObject;
    }
    if (!(adviceObject instanceof Advice)) {
        throw new UnknownAdviceTypeException(adviceObject);
    }
}

```

```

Advice advice = (Advice) adviceObject;
if (advice instanceof MethodInterceptor) {
    return new DefaultPointcutAdvisor(advice);
}
for (AdvisorAdapter adapter : this.adapters) {
    if (adapter.supportsAdvice(advice)) {
        return new DefaultPointcutAdvisor(advice);
    }
}
throw new UnknownAdviceTypeException(advice);
}
}

```

总结

1. spring中创建代理主要分为2种：手动方式和自动化的方式
2. 手动方式采用硬编码的方式，一次只能给一个目标对象创建代理对象，相对来说灵活一下，对开发者来说更灵活一些，通常可以独立spring环境使用；自动化的方式主要在spring环境中使用，通常是匹配的方式来为符合条件的目标bean创建代理，使用起来更简单一些
3. 本文介绍的 ProxyFactoryBean 用来在spring环境中给指定的bean创建代理对象，用到的不是太多，大家可以作为了解即可

Spring系列

1. [Spring系列第1篇：为何要学spring?](#)
2. [Spring系列第2篇：控制反转 \(IoC\) 与依赖注入 \(DI\)](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)
4. [Spring系列第4篇：xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇：创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇：玩转bean scope，避免跳坑里!](#)
7. [Spring系列第7篇：依赖注入之手动注入](#)
8. [Spring系列第8篇：自动注入 \(autowire\) 详解，高手在于坚持](#)
9. [Spring系列第9篇：depend-on到底是干什么的?](#)
10. [Spring系列第10篇：primary可以解决什么问题?](#)
11. [Spring系列第11篇：bean中的autowire-candidate又是干什么的?](#)
12. [Spring系列第12篇：lazy-init: bean延迟初始化](#)
13. [Spring系列第13篇：使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇：lookup-method和replaced-method比较陌生，怎么玩的?](#)
15. [Spring系列第15篇：代理详解 \(Java动态代理&cglib代理\) ?](#)
16. [Spring系列第16篇：深入理解java注解及spring对注解的增强 \(预备知识\)](#)
17. [Spring系列第17篇：@Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇：@ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇：@import详解\(bean批量注册\)](#)
20. [Spring系列第20篇：@Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇：注解实现依赖注入 \(@Autowired、@Resource、@Primary、@Qualifier\)](#)
22. [Spring系列第22篇：@Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇：Bean生命周期详解](#)
24. [Spring系列第24篇：父子容器详解](#)
25. [Spring系列第25篇：@Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇：国际化详解](#)
27. [Spring系列第27篇：spring事件机制详解](#)

- 28. [Spring系列第28篇：Bean循环依赖详解](#)
- 29. [Spring系列第29篇：BeanFactory扩展（BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor）](#)
- 30. [Spring系列第30篇：jdk动态代理和cglib代理](#)
- 31. [Spring系列第31篇：aop概念详解](#)

更多好文章

- 1. [Java高并发系列（共34篇）](#)
- 2. [MySQL高手系列（共27篇）](#)
- 3. [Maven高手系列（共10篇）](#)
- 4. [Mybatis系列（共12篇）](#)
- 5. [聊聊db和缓存一致性常见的实现方式](#)
- 6. [接口幂等性这么重要，它是什么？怎么实现？](#)
- 7. [泛型，有点难度，会让很多人懵逼，那是因为你没有看这篇文章！](#)



领取福利，扫码发送java

- 1、1000本java电子书大全
- 2、2021版java面试通关手册
- 3、月薪5万必备的java微服务学习线路图及高清视频
- 4、邀请加入高质量java技术交流群

加我微信：spring888java，进群一起交流java相关各种技术！！

Spring系列第34篇：@Aspect中@Pointcut 12种用法

本文主要内容：掌握@Pointcut的12种用法。

Aop相关阅读

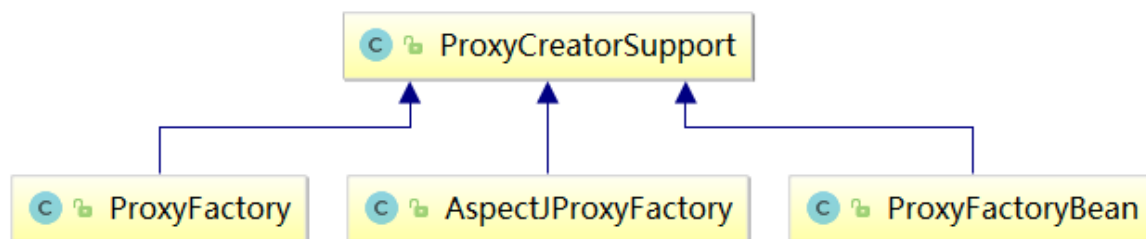
阅读本文之前，需要先掌握下面几篇篇文章内容，不然会比较吃力。

- 1. [Spring系列第15篇：代理详解（java动态代理&CGLIB代理）](#)
- 2. [Spring系列第30篇：jdk动态代理和cglib代理](#)
- 3. [Spring系列第31篇：Aop概念详解](#)
- 4. [Spring系列第32篇：AOP核心源码、原理详解](#)
- 5. [Spring系列第33篇：ProxyFactoryBean创建AOP代理](#)

本文继续AOP，目前手动Aop中三种方式已经介绍2种了，本文将介绍另外一种：

`AspectJProxyFactory`，可能大家对这个比较陌生，但是 `@Aspect` 这个注解大家应该很熟悉吧，通过这个注解在spring环境中实现aop特别的方便。

而 `AspectJProxyFactory` 这个类可以通过解析 `@Aspect` 标注的类来生成代理aop代理对象，对开发者来说，使创建代理变的更简洁了。



先了解几个概念

文中会涉及几个概念，先了解一下。

target

用来表示目标对象，即需要通过aop来增强的对象。

proxy

代理对象，target通过aop增强之后生成的代理对象。

AspectJ

AspectJ是什么？

AspectJ是一个面向切面的框架，是目前最好用，最方便的AOP框架，和spring中的aop可以集成在一起使用，通过AspectJ提供的一些功能实现aop代理变得非常方便。

AspectJ使用步骤

1. 创建一个类，使用`@Aspect`标注
2. `@Aspect`标注的类中，通过`@Pointcut`定义切入点
3. `@Aspect`标注的类中，通过`AspectJ`提供的一些通知相关的注解定义通知
4. 使用`AspectJProxyFactory`结合`@Aspect`标注的类，来生成代理对象

先来个案例，感受一下AspectJ是多么的方便。

来个类

```

package com.javacode2018.aop.demo9.test1;

public class Service1 {

    public void m1() {
        System.out.println("我是 m1 方法");
    }

    public void m2() {
        System.out.println(10 / 0);
        System.out.println("我是 m2 方法");
    }

}

```

通过 AspectJ 来对 Service1 进行增强，来2个通知，一个前置通知，一个异常通知，这2个通知需要对 Service1 中的所有方法生效，实现如下：

```

package com.javacode2018.aop.demo9.test1;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.AfterThrowing;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.Pointcut;

//@1: 这个类需要使用@Aspect进行标注
@Aspect
public class Aspect1 {

    //@2: 定义了一个切入点，可以匹配Service1中所有方法
    @Pointcut("execution(* com.javacode2018.aop.demo9.test1.Service1.*(..))")
    public void pointcut1() {
    }

    //@3: 定义了一个前置通知，这个通知对刚刚上面我们定义的切入点中的所有方法有效
    @Before(value = "pointcut1()")
    public void before(JoinPoint joinPoint) {
        //输出连接点的信息
        System.out.println("前置通知, " + joinPoint);
    }

    //@4: 定义了一个异常通知，这个通知对刚刚上面我们定义的切入点中的所有方法有效
    @AfterThrowing(value = "pointcut1()", throwing = "e")
    public void afterThrowing(JoinPoint joinPoint, Exception e) {
        //发生异常之后输出异常信息
        System.out.println(joinPoint + ",发生异常: " + e.getMessage());
    }

}

```

@1: 类上使用@Aspect标注

@2: 通过@Pointcut注解标注在方法上面，用来定义切入点

@3: 使用@Before标注在方法上面, 定义了一个前置通知, 通过value引用了上面已经定义的切入点, 表示这个通知会对Service1中的所有方法生效, 在通知中可以通过这个 类名.方法名() 引用 @Pointcut 定义的切入点, 表示这个通知对这些切入点有效, 若 @Before和@Pointcut 在一个类的时候, 直接通过 方法名() 引用当前类中定义的切入点

@4: 这个使用 @AfterThrowing 定义了一个异常通知, 也是对通过value引用了上面已经定义的切入点, 表示这个通知会对Service1中的所有方法生效, 若Service1中的方法抛出了Exception类型的异常, 都会回调 afterThrowing 方法。

来个测试类

```
package com.javacode2018.aop.demo9;

import com.javacode2018.aop.demo9.test1.Aspect1;
import com.javacode2018.aop.demo9.test1.Service1;
import org.junit.Test;
import org.springframework.aop.aspectj.annotation.AspectJProxyFactory;

public class AopTest9 {
    @Test
    public void test1() {
        try {
            //对应目标对象
            Service1 target = new Service1();
            //创建AspectJProxyFactory对象
            AspectJProxyFactory proxyFactory = new AspectJProxyFactory();
            //设置被代理的目标对象
            proxyFactory.setTarget(target);
            //设置标注了@Aspect注解的类
            proxyFactory.addAspect(Aspect1.class);
            //生成代理对象
            Service1 proxy = proxyFactory.getProxy();
            //使用代理对象
            proxy.m1();
            proxy.m2();
        } catch (Exception e) {
        }
    }
}
```

运行输出

```
前置通知, execution(void com.javacode2018.aop.demo9.test1.Service1.m1())
我是 m1 方法
前置通知, execution(void com.javacode2018.aop.demo9.test1.Service1.m2())
execution(void com.javacode2018.aop.demo9.test1.Service1.m2()),发生异常: / by zero
```

使用是不是特方便。

AspectJProxyFactory原理

@Aspect 标注的类上, 这个类中, 可以通过通过 @Pointcut 来定义切入点, 可以通过 @Before、@Around、@After、@AfterRunning、@AfterThrowing 标注在方法上来定义通知, 定义好了之后, 将 @Aspect 标注的这个类交给 AspectJProxyFactory 来解析生成 Advisor 链, 进而结合目标对象一起来生成代理对象, 大家可以去看一下源码, 比较简单, 这里就不多解释了。

本文的重点在 @Aspect 标注的类上，@Aspect 中有2个关键点比较重要

- @Pointcut：标注在方法上，用来定义切入点，有11种用法，本文主要讲解这11种用法。
- @Aspect类中定义通知：可以通过 @Before、@Around、@After、@AfterRunning、@AfterThrowing 标注在方法上来定义通知，这个下一篇介绍。

@Pointcut的12种用法

作用

用来标注在方法上来定义切入点。

定义

格式：@ 注解(value="表达标签 (表达式格式)")

如：

```
@Pointcut("execution(* com.javacode2018.aop.demo9.test1.Service1.*(..))")
```

表达式标签（10种）

- execution：用于匹配方法执行的连接点
- within：用于匹配指定类型内的方法执行
- this：用于匹配当前AOP代理对象类型的执行方法；注意是AOP代理对象的类型匹配，这样就可能包括引入接口的类型匹配
- target：用于匹配当前目标对象类型的执行方法；注意是目标对象的类型匹配，这样就不包括引入接口的类型匹配
- args：用于匹配当前执行的方法传入的参数为指定类型的执行方法
- @within：用于匹配所以持有指定注解类型内的方法
- @target：用于匹配当前目标对象类型的执行方法，其中目标对象持有指定的注解
- @args：用于匹配当前执行的方法传入的参数持有指定注解的执行
- @annotation：用于匹配当前执行方法持有指定注解的方法
- bean：Spring AOP扩展的，AspectJ没有对于指示符，用于匹配特定名称的Bean对象的执行方法

10种标签组成了12种用法

1、execution

使用 execution(方法表达式) 匹配方法执行。

execution格式

```
execution(modifiers-pattern? ret-type-pattern declaring-type-pattern? name-pattern(param-pattern) throws-pattern?)
```

- 其中带 ?号的 modifiers-pattern?, declaring-type-pattern?, hrows-pattern?是可选项
- ret-type-pattern,name-pattern, parameters-pattern是必选项
- modifier-pattern? 修饰符匹配，如public 表示匹配公有方法
- ret-type-pattern 返回值匹配，* 表示任何返回值，全路径的类名等
- declaring-type-pattern? 类路径匹配

- name-pattern 方法名匹配, * 代表所有, set*, 代表以set开头的所有方法
- (param-pattern) 参数匹配, 指定方法参数(声明的类型), (..)代表所有参数, (*,String)代表第一个参数为任何值,第二个为String类型, (..,String)代表最后一个参数是String类型
- throws-pattern? 异常类型匹配

举例说明

表达式	描述
public *.*(..)	任何公共方法的执行
* com.javacode2018..IPointcutService.* ()	com.javacode2018包及所有子包下IPointcutService接口中的任何无参方法
* com.javacode2018..*.*(..)	com.javacode2018包及所有子包下任何类的任何方法
* com.javacode2018..IPointcutService. ()	com.javacode2018包及所有子包下IPointcutService接口的任何只有一个参数方法
* com.javacode2018..IPointcutService+.* ()	com.javacode2018包及所有子包下IPointcutService接口及子类型的的任何无参方法
* Service1.*(String)	匹配Service1中只有1个参数的且参数类型是String的方法
* Service1.*(*,String)	匹配Service1中只有2个参数的且第二个参数类型是String的方法
* Service1.*(..,String)	匹配Service1中最后1个参数类型是String的方法

类型匹配语法

很多地方会按照类型的匹配, 先来说一下类型匹配的语法。

首先让我们了解下AspectJ类型匹配的通配符:

- *: 匹配任何数量字符
- ..: 匹配任何数量字符的重复, 如在类型模式中匹配任何数量子包; 而在方法参数模式中匹配任何数量参数 (0个或者多个参数)
- +: 匹配指定类型及其子类型; 仅能作为后缀放在类型模式后边

表达式	说明
java.lang.String	匹配String类型
java.*.String	匹配java包下的任何一级子包下的String类型，如匹配java.lang.String，但不匹配java.lang.ss.String
java..*	匹配java包及任何子包下的任何类型，如匹配java.lang.String、java.lang.annotation.Annotation
java.lang.*ing	匹配任何java.lang包下的以ing结尾的类型
java.lang.Number+	匹配java.lang包下的任何Number类型及其子类型，如匹配java.lang.Number，也匹配java.lang.Integer、java.math.BigInteger

2、within

用法

`within(类型表达式)`：目标对象target的类型是否和within中指定的类型匹配

表达式	描述
<code>within(com.javacode2018..*)</code>	com.javacode2018包及子包下的任何方法执行
<code>within(com.javacode2018..IPointcutService+)</code>	com.javacode2018包或所有子包下IPointcutService类型及子类型的任何方法
<code>within(com.javacode2018.Service1)</code>	匹配类com.javacode2018.Service1中定义的所有方法，不包含其子类中的方法

匹配原则

`target.getClass().equals(within表达式中指定的类型)`

案例

有2个类，父子关系

父类C1

```
package com.javacode2018.aop.demo9.test2;

public class C1 {
    public void m1() {
        System.out.println("我是m1");
    }

    public void m2() {
        System.out.println("我是m2");
    }
}
```

子类C2

```
package com.javacode2018.aop.demo9.test2;

public class C2 extends C1 {
    @Override
    public void m2() {
        super.m2();
    }

    public void m3() {
        System.out.println("我是m3");
    }
}
```

来个Aspect类

```
package com.javacode2018.aop.demo9.test2;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.Pointcut;

@Aspect
public class AspectTest2 {

    @Pointcut("within(C1)") //@1
    public void pc() {
    }

    @Before("pc()") //@2
    public void beforeAdvice(JoinPoint joinpoint) {
        System.out.println(joinpoint);
    }
}
```

注意@1匹配的类型是C1，也就是说被代理的对象的类型必须是C1类型的才行，需要和C1完全匹配

下面我们对C2创建代理

```
@Test
public void test2(){
    C2 target = new C2();
    AspectJProxyFactory proxyFactory = new AspectJProxyFactory();
    proxyFactory.setTarget(target);
    proxyFactory.addAspect(AspectTest2.class);

    C2 proxy = proxyFactory.getProxy();
    proxy.m1();
    proxy.m2();
    proxy.m3();
}
```

运行输出

```
我是m1  
我是m2  
我是m3
```

原因是目标对象是C2类型的，C2虽然是C1的子类，但是within中表达式指定的是要求类型必须是C1类型的才匹配。

如果将within表达式修改为下面任意一种就可以匹配了

```
@Pointcut("within(C1+)")  
@Pointcut("within(C2)")
```

再次运行输出

```
execution(void com.javacode2018.aop.demo9.test2.C1.m1())  
我是m1  
execution(void com.javacode2018.aop.demo9.test2.C2.m2())  
我是m2  
execution(void com.javacode2018.aop.demo9.test2.C2.m3())  
我是m3
```

3、this

用法

`this(类型全限定名)`：通过aop创建的代理对象的类型是否和this中指定的类型匹配；注意判断的目标是代理对象；this中使用的表达式必须是类型全限定名，不支持通配符。

匹配原则

```
如：this(x)，则代理对象proxy满足下面条件时会匹配  
x.getClass().isAssignableFrom(proxy.getClass())；
```

案例

来个接口

```
package com.javacode2018.aop.demo9.test3;  
  
public interface I1 {  
    void m1();  
}
```

来个实现类

```

package com.javacode2018.aop.demo9.test3;

public class Service3 implements I1 {

    @Override
    public void m1() {
        System.out.println("我是m1");
    }

}

```

来个@Aspect类

```

package com.javacode2018.aop.demo9.test3;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.Pointcut;

@Aspect
public class AspectTest3 {

    //@1: 匹配proxy是Service3类型的所有方法
    @Pointcut("this(Service3)")
    public void pc() {
    }

    @Before("pc()")
    public void beforeAdvice(JoinPoint joinpoint) {
        System.out.println(joinpoint);
    }

}

```

测试代码

```

@Test
public void test3() {
    Service3 target = new Service3();
    AspectJProxyFactory proxyFactory = new AspectJProxyFactory();
    proxyFactory.setTarget(target);
    //获取目标对象上的接口列表
    Class<?>[] allInterfaces = ClassUtils.getAllInterfaces(target);
    //设置需要代理的接口
    proxyFactory.setInterfaces(allInterfaces);
    proxyFactory.addAspect(AspectTest3.class);
    //获取代理对象
    Object proxy = proxyFactory.getProxy();
    //调用代理对象的方法
    ((I1) proxy).m1();

    System.out.println("proxy是否是jdk动态代理对象: " +
        AopUtils.isJdkDynamicProxy(proxy));
    System.out.println("proxy是否是cglib代理对象: " +
        AopUtils.isCglibProxy(proxy));
}

```

```
//判断代理对象是否是Service3类型的
system.out.println(Service3.class.isAssignableFrom(proxy.getClass()));
}
```

运行输出

```
我是m1
proxy是否是jdk动态代理对象: true
proxy是否是cglib代理对象: false
false
```

从输出中可以看出m1方法没有被增强，原因：this表达式要求代理对象必须是Service3类型的，输出中可以看出代理对象并不是Service3类型的，此处代理对象proxy是使用jdk动态代理生成的。

我们可以将代码调整一下，使用cglib来创建代理

```
proxyFactory.setProxyTargetClass(true);
```

再次运行，会发现m2被拦截了，结果如下

```
execution(void com.javacode2018.aop.demo9.test3.Service3.m1())
我是m1
proxy是否是jdk动态代理对象: false
proxy是否是cglib代理对象: true
true
```

4、target

用法

target(类型全限定名)：判断目标对象的类型是否和指定的类型匹配；注意判断的是目标对象的类型；表达式必须是类型全限定名，不支持通配符。

匹配原则

```
如:target(x)，则目标对象target满足下面条件时会匹配
x.getClass().isAssignableFrom(target.getClass());
```

案例

```
package com.javacode2018.aop.demo9.test4;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.Pointcut;

@Aspect
public class AspectTest4 {

    //@1: 目标类型必须是Service3类型的
    @Pointcut("target(com.javacode2018.aop.demo9.test3.Service3)")
```



```

    public void pc() {
    }

    @Before("pc()")
    public void beforeAdvice(JoinPoint joinpoint) {
        System.out.println(joinpoint);
    }

}

```

测试代码

```

@Test
public void test4() {
    Service3 target = new Service3();
    AspectJProxyFactory proxyFactory = new AspectJProxyFactory();
    proxyFactory.setProxyTargetClass(true);
    proxyFactory.setTarget(target);
    proxyFactory.addAspect(AspectTest4.class);
    //获取代理对象
    Object proxy = proxyFactory.getProxy();
    //调用代理对象的方法
    ((I1) proxy).m1();
    //判断target对象是否是Service3类型的
    System.out.println(Service3.class.isAssignableFrom(target.getClass()));
}

```

运行输出

```

execution(void com.javacode2018.aop.demo9.test3.Service3.m1())
我是m1
true

```

within、this、target对比

表达式标签	判断的对象	判断规则(x：指表达式中指定的类型)
within	target对象	target.getClass().equals(表达式中指定的类型)
this	proxy对象	x.getClass().isAssignableFrom(proxy.getClass());
target	target对象	x.getClass().isAssignableFrom(target.getClass());

5、args

用法

args(参数类型列表) 匹配当前执行的方法传入的参数是否为args中指定的类型；注意是匹配传入的参数类型，不是匹配方法签名的参数类型；参数类型列表中的参数必须是类型全限定名，不支持通配符；args属于动态切入点，也就是执行方法的时候进行判断的，这种切入点开销非常大，非特殊情况最好不要使用。

举例说明

表达式	描述
args(String)	匹配只有一个参数且传入的参数类型是String类型的方法
args(*,String)	匹配只有2个参数的且第2个参数类型是String的方法
args(..,String)	匹配最后1个参数类型是String的方法

案例

下面的m1方法参数是Object类型的。

```
package com.javacode2018.aop.demo9.test5;

public class Service5 {
    public void m1(Object object) {
        System.out.println("我是m1方法,参数: " + object);
    }
}
```

Aspect类

```
package com.javacode2018.aop.demo9.test5;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.Pointcut;

import java.util.Arrays;
import java.util.stream.Collectors;

@Aspect
public class AspectTest5 {
    //@1: 匹配只有1个参数其类型是String类型的
    @Pointcut("args(String)")
    public void pc() {
    }

    @Before("pc()")
    public void beforeAdvice(JoinPoint joinpoint) {
        System.out.println("请求参数: " +
            Arrays.stream(joinpoint.getArgs()).collect(Collectors.toList()));
    }
}
```

测试代码，调用2次m1方法，第一次传入一个String类型的，第二次传入一个int类型的，看看效果

```

@Test
public void test5() {
    Service5 target = new Service5();
    AspectJProxyFactory proxyFactory = new AspectJProxyFactory();
    proxyFactory.setTarget(target);
    proxyFactory.addAspect(AspectTest5.class);
    Service5 proxy = proxyFactory.getProxy();
    proxy.m1("路人");
    proxy.m1(100);
}

```

运行输出

```

请求参数: [路人]
我是m1方法, 参数: 路人
我是m1方法, 参数: 100

```

输出中可以看出, m1第一次调用被增强了, 第二次没有被增强。

args会在调用的过程中对参数实际的类型进行匹配, 比较耗时, 慎用。

6、@within

用法

@within(注解类型): 匹配指定的注解内定义的方法。

匹配规则

调用目标方法的时候, 通过java中 `Method.getDeclaringClass()` 获取当前的方法是哪个类中定义的, 然后会看这个类上是否有指定的注解。

```

被调用的目标方法Method对象.getDeclaringClass().getAnnotation(within中指定的注解类型) != null

```

来看3个案例。

案例1

目标对象上有@within中指定的注解, 这种情况时, 目标对象的所有方法都会被拦截。

来个注解

```

package com.javacode2018.aop.demo9.test9;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface Ann9 {
}

```

来个目标类，用@Ann9标注

```

package com.javacode2018.aop.demo9.test9;

@Ann9
public class S9 {
    public void m1() {
        System.out.println("我是m1方法");
    }
}

```

来个Aspect类

```

package com.javacode2018.aop.demo9.test9;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.Pointcut;

@Aspect
public class AspectTest9 {
    /**
     * 定义目标方法的类上有Ann9注解
     */
    @Pointcut("@within(Ann9)")
    public void pc() {
    }

    @Before("pc()")
    public void beforeAdvice(JoinPoint joinPoint) {
        System.out.println(joinPoint);
    }
}

```

测试代码

```

@Test
public void test9() {
    S9 target = new S9();
    AspectJProxyFactory proxyFactory = new AspectJProxyFactory();
    proxyFactory.setTarget(target);
    proxyFactory.addAspect(AspectTest9.class);
    S9 proxy = proxyFactory.getProxy();
    proxy.m1();
}

```

m1方法在类S9中定义的，S9上面有Ann9注解，所以匹配成功

运行输出

```

execution(void com.javacode2018.aop.demo9.test9.S9.m1())
我是m1方法

```

案例2

定义注解时未使用 `@Inherited`，说明子类无法继承父类上的注解，这个案例中我们将定义一个这样的注解，将注解放在目标类的父类上，来看一下效果。

定义注解Ann10

```

package com.javacode2018.aop.demo9.test10;

import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@Inherited
public @interface Ann10 {
}

```

来2个父子类

注意：

S10Parent为父类，并且使用了Anno10注解，内部定义了2个方法大家注意一下

而S10为代理的目标类，继承了S10Parent，内部重写了父类的m2方法，并且又新增了一个m3方法

```

package com.javacode2018.aop.demo9.test10;

@Ann10
class S10Parent {

    public void m1() {
        System.out.println("我是S10Parent.m1()方法");
    }

    public void m2() {
        System.out.println("我是S10Parent.m2()方法");
    }
}

```

```

public class S10 extends S10Parent {

    @Override
    public void m2() {
        System.out.println("我是S10.m2()方法");
    }

    public void m3() {
        System.out.println("我是S10.m3()方法");
    }

}

```

来个Aspect类

```

package com.javacode2018.aop.demo9.test10;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.Pointcut;

@Aspect
public class AspectTest10 {
    //匹配目标方法声明的类上有@Anno10注解
    @Pointcut("@within(com.javacode2018.aop.demo9.test10.Anno10)")
    public void pc() {
    }

    @Before("pc()")
    public void beforeAdvice(JoinPoint joinPoint) {
        System.out.println(joinPoint);
    }

}

```

测试用例

S10为目标类，依次执行代理对象的m1、m2、m3方法，最终会调用目标类target中对应的方法。

```

@Test
public void test10() {
    S10 target = new S10();
    AspectJProxyFactory proxyFactory = new AspectJProxyFactory();
    proxyFactory.setTarget(target);
    proxyFactory.addAspect(AspectTest10.class);
    S10 proxy = proxyFactory.getProxy();
    proxy.m1();
    proxy.m2();
    proxy.m3();
}

```

运行输出

```
execution(void com.javacode2018.aop.demo9.test10.S10Parent.m1())
我是S10Parent.m1()方法
我是S10.m2()方法
我是S10.m3()方法
```

分析结果

从输出中可以看出，只有m1方法被拦截了，其他2个方法没有被拦截。

确实是这样的，m1方法的是由S10Parent定义的，这个类上面有Ann10注解。

而m2方法虽然也在S10Parent中定义了，但是这个方法被子类S10重写了，所以调用目标对象中的m2方法的时候，此时发现m2方法是由S10定义的，而 `s10.class.getAnnotation(Ann10.class)` 为空，所以这个方法不会被拦截。

同样m3方法也是S10中定义的，也不会被拦截。

案例3

对案例2进行改造，在注解的定义上面加上 `@Inherited`，此时子类可以继承父类的注解，此时3个方法都会被拦截了。

下面上代码，下面代码为案例2代码的一个拷贝，不同地方只是注解的定义上多了 `@Inherited`

定义注解Ann11

```
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@Inherited
public @interface Ann11 {
}
```

2个父子类

```
package com.javacode2018.aop.demo9.test11;

@Ann11
class S11Parent {

    public void m1() {
        System.out.println("我是S11Parent.m1()方法");
    }

    public void m2() {
        System.out.println("我是S11Parent.m2()方法");
    }
}

public class S11 extends S11Parent {

    @Override
    public void m2() {
```

```

        System.out.println("我是S11.m2()方法");
    }

    public void m3() {
        System.out.println("我是S11.m3()方法");
    }
}

```

Aspect类

```

package com.javacode2018.aop.demo9.test11;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.Pointcut;

@Aspect
public class AspectTest11 {

    @Pointcut("@within(com.javacode2018.aop.demo9.test11.Ann11)")
    public void pc() {
    }

    @Before("pc()")
    public void beforeAdvice(JoinPoint joinPoint) {
        System.out.println(joinPoint);
    }
}

```

测试用例

```

@Test
public void test11() {
    S11 target = new S11();
    AspectJProxyFactory proxyFactory = new AspectJProxyFactory();
    proxyFactory.setTarget(target);
    proxyFactory.addAspect(AspectTest11.class);
    S11 proxy = proxyFactory.getProxy();
    proxy.m1();
    proxy.m2();
    proxy.m3();
}

```

运行输出

```

execution(void com.javacode2018.aop.demo9.test11.S11Parent.m1())
我是S11Parent.m1()方法
execution(void com.javacode2018.aop.demo9.test11.S11.m2())
我是S11.m2()方法
execution(void com.javacode2018.aop.demo9.test11.S11.m3())
我是S11.m3()方法

```

这次3个方法都被拦截了。

7、@target

用法

`@target(注解类型)`：判断目标对象`target`类型上是否有指定的注解；`@target`中注解类型也必须是全限定类型名。

匹配规则

```
target.class.getAnnotation(指定的注解类型) != null
```

2种情况可以匹配

- 注解直接标注在目标类上
- 注解标注在父类上，但是注解必须是可以继承的，即定义注解的时候，需要使用 `@Inherited` 标注

案例1

注解直接标注在目标类上，这种情况目标类会被匹配到。

自定义一个注解 `Ann6`

```
package com.javacode2018.aop.demo9.test6;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface Ann6 {
}
```

目标类 `S6` 上直接使用 `@Ann1`

```
package com.javacode2018.aop.demo9.test6;

@Ann6
public class S6 {
    public void m1() {
        System.out.println("我是m1");
    }
}
```

来个 `Aspect` 类

```
package com.javacode2018.aop.demo9.test6;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.Pointcut;

@Aspect
```

```

public class AspectTest6 {
    //①: 目标类上有@Ann1注解
    @Pointcut("@target(Ann1)")
    public void pc() {
    }

    @Before("pc()")
    public void beforeAdvice(JoinPoint joinPoint) {
        System.out.println(joinPoint);
    }
}

```

测试代码

```

@Test
public void test6() {
    S6 target = new S6();
    AspectJProxyFactory proxyFactory = new AspectJProxyFactory();
    proxyFactory.setTarget(target);
    proxyFactory.addAspect(AspectTest6.class);
    S6 proxy = proxyFactory.getProxy();
    proxy.m1();
    System.out.println("目标类上是否有 @Ann6 注解: " +
        (target.getClass().getAnnotation(Ann6.class) != null));
}

```

运行输出

```

execution(void com.javacode2018.aop.demo9.test6.S6.m1())
我是m1
目标类上是否有 @Ann6 注解: true

```

案例2

注解标注在父类上，注解上没有 `@Inherited`，这种情况下，目标类无法匹配到，下面看代码

注解Ann7

```

package com.javacode2018.aop.demo9.test7;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface Ann7 {
}

```

来2个父子类，父类上有 @Ann7，之类 S7 为目标类

```
package com.javacode2018.aop.demo9.test7;

import java.lang.annotation.Target;

@Ann7
class S7Parent {
}

public class S7 extends S7Parent {
    public void m1() {
        System.out.println("我是m1");
    }

    public static void main(String[] args) {
        System.out.println(S7.class.getAnnotation(Target.class));
    }
}
```

来个Aspect类

```
package com.javacode2018.aop.demo9.test7;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.Pointcut;

@Aspect
public class AspectTest7 {
    /**
     * 匹配目标类上有Ann7注解
     */
    @Pointcut("@target(com.javacode2018.aop.demo9.test7.Ann7)")
    public void pc() {
    }

    @Before("pc()")
    public void beforeAdvice(JoinPoint joinPoint) {
        System.out.println(joinPoint);
    }
}
```

测试代码

```

@Test
public void test7() {
    S7 target = new S7();
    AspectJProxyFactory proxyFactory = new AspectJProxyFactory();
    proxyFactory.setTarget(target);
    proxyFactory.addAspect(AspectTest7.class);
    S7 proxy = proxyFactory.getProxy();
    proxy.m1();
    System.out.println("目标类上是否有 @Ann7 注解: " +
(target.getClass().getAnnotation(Ann7.class) != null));
}

```

运行输出

```

我是m1
目标类上是否有 @Ann7 注解: false

```

分析结果

@Ann7标注在了父类上，但是@Ann7定义的时候没有使用 @Inherited，说明之类无法继承父类上面的注解，所以上面的目标类没有被拦截，下面我们将 @Ann7 的定义改一下，加上 @Inherited

```

package com.javacode2018.aop.demo9.test7;

import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@Inherited
public @interface Ann7 {
}

```

再次运行输出

```

execution(void com.javacode2018.aop.demo9.test7.S7.m1())
我是m1
目标类上是否有 @Ann7 注解: true

```

此时目标对象被拦截了。

8、@args

用法

@args(注解类型): 方法参数所属的类上有指定的注解；注意不是参数上有指定的注解，而是参数类型的类上有指定的注解。

案例1

```

@Pointcut("@args(Ann8)"): 匹配方法只有一个参数，并且参数所属的类上有Ann8注解

```

可以匹配下面的代码，m1方法的第一个参数类型是Car类型，Car类型上有注解Ann8

```
@Ann8
class Car {
}

public void m1(Car car) {
    System.out.println("我是m1");
}
```

案例2

@Pointcut("@args(*,Ann8)": 匹配方法只有2个参数，且第2个参数所属的类型上有Ann8注解

可以匹配下面代码

```
@Ann8
class Car {
}

public void m1(String name,Car car) {
    System.out.println("我是m1");
}
```

案例3

@Pointcut("@args(..,com.javacode2018.aop.demo9.test8.Ann8)": 匹配参数数量大于等于1，且最后一个参数所属的类型上有Ann8注解
@Pointcut("@args(*,com.javacode2018.aop.demo9.test8.Ann8,..)": 匹配参数数量大于等于2，且第2个参数所属的类型上有Ann8注解
@Pointcut("@args(..,com.javacode2018.aop.demo9.test8.Ann8,*)": 匹配参数数量大于等于2，且倒数第2个参数所属的类型上有Ann8注解

这个案例代码，大家自己写一下，体验一下。

9、@annotation

用法

@annotation(注解类型): 匹配被调用的方法上有指定的注解。

案例

定义一个注解，可以用在方法上

```

package com.javacode2018.aop.demo9.test12;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Ann12 {
}

```

定义2个类

S12Parent为父类，内部定义了2个方法，2个方法上都有@Ann12注解

S12是代理的目标类，也是S12Parent的子类，内部重写了m2方法，重写之后m2方法上并没有@Ann12注解，S12内部还定义2个方法m3和m4，而m3上面有注解@Ann12

```

package com.javacode2018.aop.demo9.test12;

class S12Parent {

    @Ann12
    public void m1() {
        System.out.println("我是S12Parent.m1()方法");
    }

    @Ann12
    public void m2() {
        System.out.println("我是S12Parent.m2()方法");
    }
}

public class S12 extends S12Parent {

    @Override
    public void m2() {
        System.out.println("我是S12.m2()方法");
    }

    @Ann12
    public void m3() {
        System.out.println("我是S12.m3()方法");
    }

    public void m4() {
        System.out.println("我是S12.m4()方法");
    }
}

```

来个Aspect类

当被调用的目标方法上有@Ann12注解的时，会被beforeAdvice处理。

```
package com.javacode2018.aop.demo9.test12;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.Pointcut;

@Aspect
public class AspectTest12 {

    @Pointcut("@annotation(com.javacode2018.aop.demo9.test12.Ann12)")
    public void pc() {
    }

    @Before("pc()")
    public void beforeAdvice(JoinPoint joinPoint) {
        System.out.println(joinPoint);
    }
}
```

测试用例

S12作为目标对象，创建代理，然后分别调用4个方法

```
@Test
public void test12() {
    S12 target = new S12();
    AspectJProxyFactory proxyFactory = new AspectJProxyFactory();
    proxyFactory.setTarget(target);
    proxyFactory.addAspect(AspectTest12.class);
    S12 proxy = proxyFactory.getProxy();
    proxy.m1();
    proxy.m2();
    proxy.m3();
    proxy.m4();
}
```

运行输出

```
execution(void com.javacode2018.aop.demo9.test12.S12Parent.m1())
我是S12Parent.m1()方法
我是S12.m2()方法
execution(void com.javacode2018.aop.demo9.test12.S12.m3())
我是S12.m3()方法
我是S12.m4()方法
```

分析结果

m1方法位于S12Parent中，上面有@Ann12注解，被拦截了，m3方法上有@Ann12注解，被拦截了，而m4上没有@Ann12注解，没有被拦截，这3个方法的执行结果都很容易理解。

重点在于m2方法的执行结果，没有被拦截，m2方法虽然在S12Parent中定义的时候也有@Ann12注解标注，但是这个方法被S1给重写了，在S1中定义的时候并没有@Ann12注解，代码中实际调用的是S1中的m2方法，发现这个方法上并没有@Ann12注解，所以没有被拦截。

10、bean

用法

bean(bean名称): 这个用在spring环境中，匹配容器中指定名称的bean。

案例

来个类BeanService

```
package com.javacode2018.aop.demo9.test13;

public class BeanService {
    private String beanName;

    public BeanService(String beanName) {
        this.beanName = beanName;
    }

    public void m1() {
        System.out.println(this.beanName);
    }
}
```

来个Aspect类

```
package com.javacode2018.aop.demo9.test13;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.Pointcut;
import org.springframework.stereotype.Component;

@Aspect
public class Aspect13 {
    //拦截spring容器中名称为beanService2的bean
    @Pointcut("bean(beanService2)")
    public void pc() {
    }

    @Before("pc()")
    public void beforeAdvice(JoinPoint joinPoint) {
        System.out.println(joinPoint);
    }
}
```


来个spring配置类

```
package com.javacode2018.aop.demo9.test13;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.EnableAspectJAutoProxy;

@Configuration
@EnableAspectJAutoProxy // 这个可以启用通过AspectJ方式自动为符合条件的bean创建代理
public class MainConfig13 {

    //将Aspect13注册到spring容器
    @Bean
    public Aspect13 aspect13() {
        return new Aspect13();
    }

    @Bean
    public BeanService beanService1() {
        return new BeanService("beanService1");
    }

    @Bean
    public BeanService beanService2() {
        return new BeanService("beanService2");
    }
}
```

这个配置类中有个 `@EnableAspectJAutoProxy`，这个注解大家可能比较陌生，这个属于aop中自动代理的范围，后面会有文章详细介绍这块，这里大家暂时先不用关注。

测试用例

下面启动spring容器，加载配置类MainConfig13，然后分别获取beanService1和beanService2，调用他们的m1方法，看看效果

```
@Test
public void test13() {
    AnnotationConfigApplicationContext context = new
    AnnotationConfigApplicationContext(MainConfig13.class);
    //从容器中获取beanService1
    BeanService beanService1 = context.getBean("beanService1",
    BeanService.class);
    beanService1.m1();
    //从容器中获取beanService2
    BeanService beanService2 = context.getBean("beanService2",
    BeanService.class);
    beanService2.m1();
}
```

运行输出

```
beanService1
execution(void com.javacode2018.aop.demo9.test13.BeanService.m1())
beanService2
```

beanService2的m1方法被拦截了。

11、reference pointcut

表示引用其他命名切入点。

有时，我们可以将切入专门放在一个类中集中定义。

其他地方可以通过引用的方式引入其他类中定义的切入点。

语法如下：

```
@Pointcut("完整包名类名.方法名称()")
```

若引用同一个类中定义切入点，包名和类名可以省略，直接通过方法就可以引用。

比如下面，我们可以将所有切入点定义在一个类中

```
package com.javacode2018.aop.demo9.test14;

import org.aspectj.lang.annotation.Pointcut;

public class AspectPcDefine {
    @Pointcut("bean(bean1)")
    public void pc1() {
    }

    @Pointcut("bean(bean2)")
    public void pc2() {
    }
}
```

下面顶一个Aspect类，来引用上面的切入点

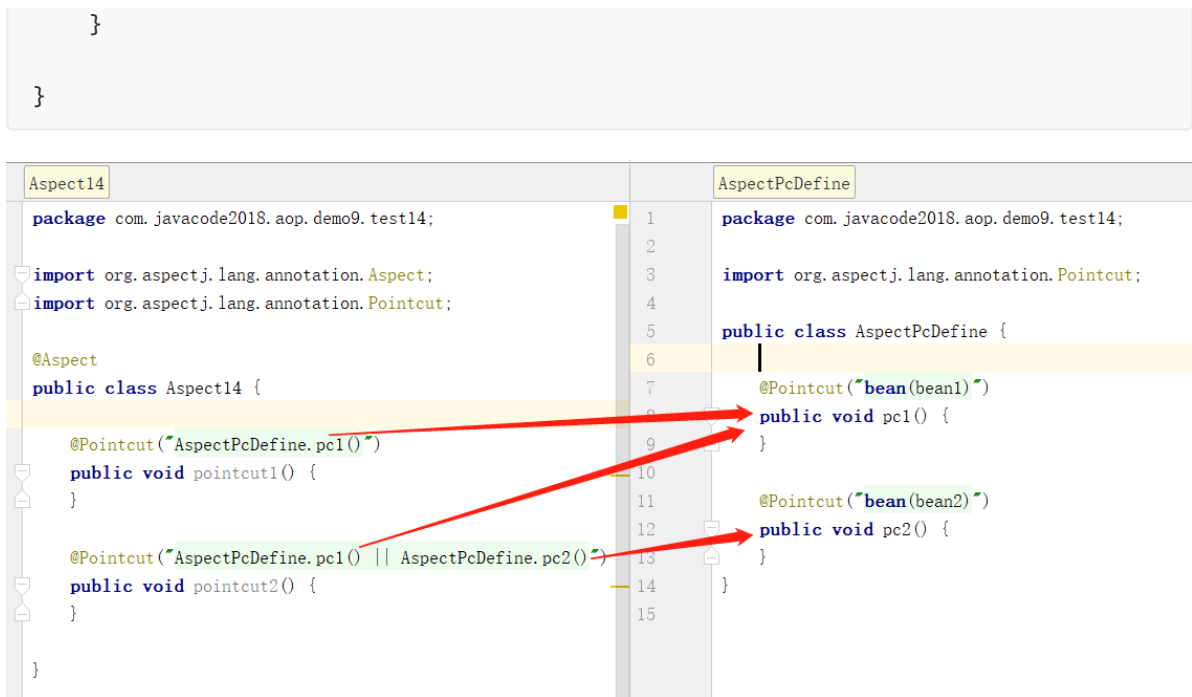
```
package com.javacode2018.aop.demo9.test14;

import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Pointcut;

@Aspect
public class Aspect14 {

    @Pointcut("com.javacode2018.aop.demo9.test14.AspectPcDefine.pc1()")
    public void pointcut1() {
    }

    @Pointcut("com.javacode2018.aop.demo9.test14.AspectPcDefine.pc1() || com.javacode2018.aop.demo9.test14.AspectPcDefine.pc2()")
    public void pointcut2() {
    }
}
```



12、组合型的pointcut

Pointcut定义时，还可以使用&&、||、!运算符。

- &&: 多个匹配都需要满足
- ||: 多个匹配中只需满足一个
- !: 匹配不满足的情况下

```

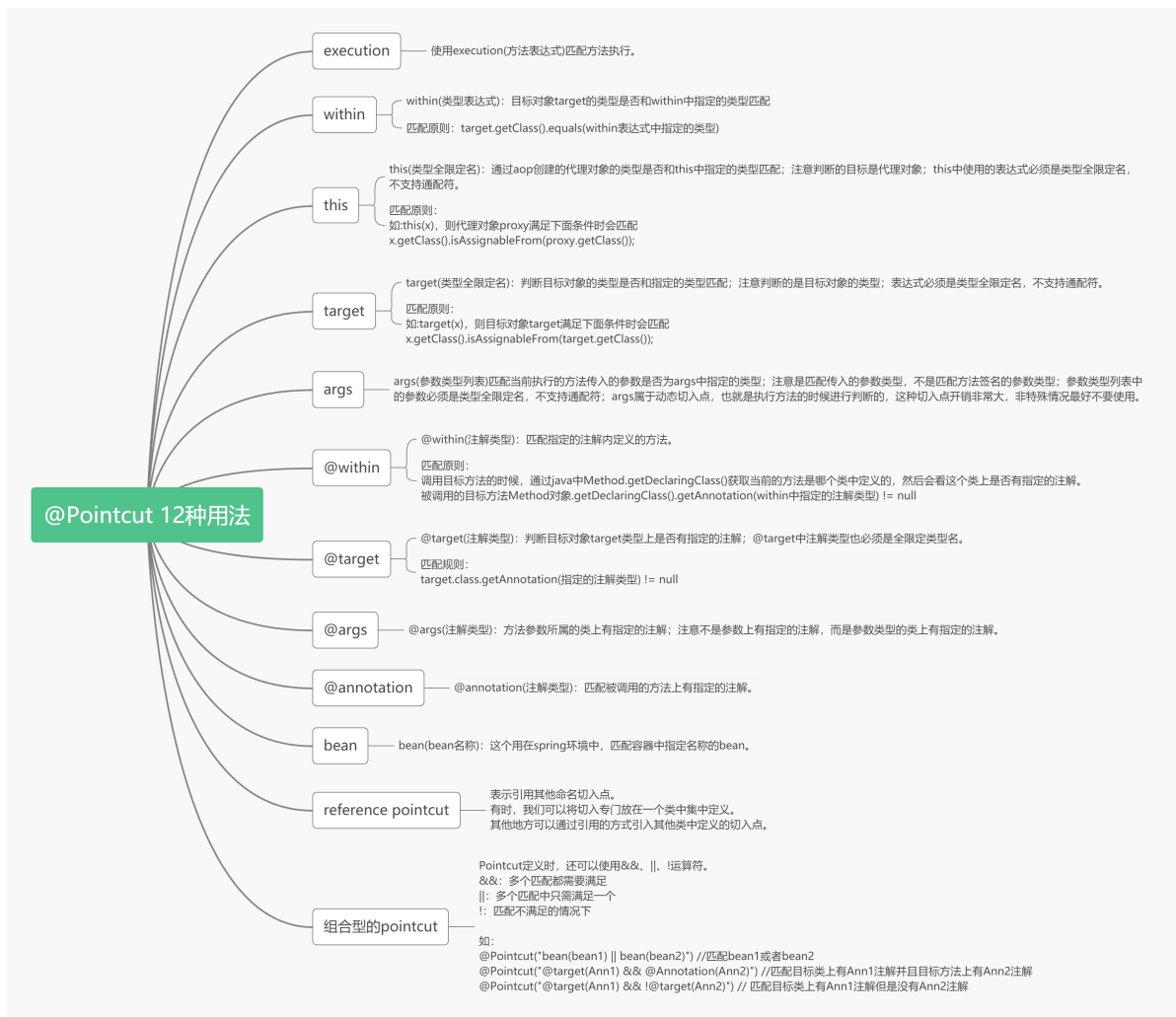
@Pointcut("bean(bean1) || bean(bean2)") //匹配bean1或者bean2
@Pointcut("@target(Ann1) && @Annotation(Ann2)") //匹配目标类上有Ann1注解并且目标方法上有Ann2注解
@Pointcut("@target(Ann1) && !@target(Ann2)") // 匹配目标类上有Ann1注解但是没有Ann2注解

```

总结

本文详解了@Pointcut的12种用法，案例大家一定要敲一遍，敲的过程中，会遇到问题，然后解决问题，才能够加深理解。

有问题的也欢迎大家留言交流，谢谢！



案例源码

<https://gitee.com/javacode2018/spring-series>

路人甲java所有案例代码以后都会放到这个上面, 大家watch一下, 可以持续关注动态。

Spring系列

1. [Spring系列第1篇: 为何要学spring?](#)
2. [Spring系列第2篇: 控制反转 \(IoC\) 与依赖注入 \(DI\)](#)
3. [Spring系列第3篇: Spring容器基本使用及原理](#)
4. [Spring系列第4篇: xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇: 创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇: 玩转bean scope, 避免跳坑里!](#)
7. [Spring系列第7篇: 依赖注入之手动注入](#)
8. [Spring系列第8篇: 自动注入 \(autowire\) 详解, 高手在于坚持](#)
9. [Spring系列第9篇: depend-on到底是干什么的?](#)
10. [Spring系列第10篇: primary可以解决什么问题?](#)
11. [Spring系列第11篇: bean中的autowire-candidate又是干什么的?](#)
12. [Spring系列第12篇: lazy-init: bean延迟初始化](#)
13. [Spring系列第13篇: 使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇: lookup-method和replaced-method比较陌生, 怎么玩的?](#)

15. [Spring系列第15篇：代理详解（Java动态代理&cglib代理）？](#)
16. [Spring系列第16篇：深入理解java注解及spring对注解的增强（预备知识）](#)
17. [Spring系列第17篇：@Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇：@ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇：@import详解\(bean批量注册\)](#)
20. [Spring系列第20篇：@Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇：注解实现依赖注入（@Autowired、@Resource、@Primary、@Qualifier）](#)
22. [Spring系列第22篇：@Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇：Bean生命周期详解](#)
24. [Spring系列第24篇：父子容器详解](#)
25. [Spring系列第25篇：@Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇：国际化详解](#)
27. [Spring系列第27篇：spring事件机制详解](#)
28. [Spring系列第28篇：Bean循环依赖详解](#)
29. [Spring系列第29篇：BeanFactory扩展（BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor）](#)
30. [Spring系列第30篇：jdk动态代理和cglib代理](#)
31. [Spring系列第31篇：aop概念详解](#)
32. [Spring系列第32篇：AOP核心源码、原理详解](#)
33. [Spring系列第33篇：ProxyFactoryBean创建AOP代理](#)

更多好文章

1. [Java高并发系列（共34篇）](#)
2. [MySQL高手系列（共27篇）](#)
3. [Maven高手系列（共10篇）](#)
4. [Mybatis系列（共12篇）](#)
5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要，它是什么？怎么实现？](#)
7. [泛型，有点难度，会让很多人懵逼，那是因为你没有看这篇文章！](#)



领取福利，扫码发送java

- 1、1000本java电子书大全
- 2、2021版java面试通关手册
- 3、月薪5万必备的java微服务学习线路图及高清视频
- 4、邀请加入高质量java技术交流群

加我微信：spring888java，进群一起交流java相关各种技术！！

Spring系列第35篇：@Aspect中5中通知详解

本文内容：详解@Aspect中5中通知的使用。

Aop相关阅读

阅读本文之前，需要先掌握下面几篇文章内容，不然会比较吃力。

1. [Spring系列第15篇：代理详解（java动态代理&CGLIB代理）](#)
2. [Spring系列第30篇：jdk动态代理和cglib代理](#)
3. [Spring系列第31篇：Aop概念详解](#)
4. [Spring系列第32篇：AOP核心源码、原理详解](#)
5. [Spring系列第33篇：ProxyFactoryBean创建AOP代理](#)
6. [Spring系列第34篇：@Aspect中@Pointcut 12种用法](#)

@Aspect中有5种通知

1. @Before：前置通知, 在方法执行之前执行
2. @Around：环绕通知, 围绕着方法执行
3. @After：后置通知, 在方法执行之后执行
4. @AfterReturning：返回通知, 在方法返回结果之后执行
5. @AfterThrowing：异常通知, 在方法抛出异常之后

这几种通知用起来都比较简单，都是通过注解的方式，将这些注解标注在@Aspect类的方法上，这些方法就会对目标方法进行拦截，下面我们一个个来看一下。

@Before：前置通知

介绍

定义一个前置通知

```
@Aspect
public class BeforeAspect {

    @Before("execution(* com.javacode2018.aop.demo10.test1.Service1.*(..))")
    public void before(JoinPoint joinPoint) {
        System.out.println("我是前置通知!");
    }
}
```

1. 类上需要使用 @Aspect 标注
2. 任意方法上使用 @Before 标注，将这个方法作为前置通知，目标方法被调用之前，会自动回调这个方法
3. 被 @Before 标注的方法参数可以为空，或者为 JoinPoint 类型，当为 JoinPoint 类型时，必须为第一个参数
4. 被 @Before 标注的方法名称可以随意命名，符合java规范就可以，其他通知也类似

@Before 中value的值为切入点表达式，也可以采用引用的方式指定切入点，如：

```
package com.javacode2018.aop.demo10.test1;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
```

```
import org.aspectj.lang.annotation.Pointcut;

@Aspect
public class BeforeAspect {

    @Pointcut("execution(* com.javacode2018.aop.demo10.test1.Service1.*(..))")
    public void pc() {
    }

    @Before("com.javacode2018.aop.demo10.test1.BeforeAspect.pc()")
    public void before(JoinPoint joinPoint) {
        System.out.println("我是前置通知!");
    }
}
```

此时，before方法上面的切入引用了pc方法上面的@Pointcut的值



案例

来个普通的service

```
package com.javacode2018.aop.demo10.test1;

public class Service1 {
    public String say(String name) {
        return "你好: " + name;
    }

    public String work(String name) {
        return "开始工作了: " + name;
    }
}
```

给上面的类定义一个前置通知，Service1中的所有方法执行执行，输出一段文字 我是前置通知！

```
package com.javacode2018.aop.demo10.test1;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.Pointcut;
```

```

@Aspect
public class BeforeAspect1 {

    @Pointcut("execution(* com.javacode2018.aop.demo10.test1.Service1.*(..))")
    public void pc() {
    }

    @Before("com.javacode2018.aop.demo10.test1.BeforeAspect1.pc()")
    public void before(JoinPoint joinPoint) {
        System.out.println("我是前置通知!");
    }
}

```

测试代码

```

package com.javacode2018.aop.demo10;

import com.javacode2018.aop.demo10.test1.BeforeAspect1;
import com.javacode2018.aop.demo10.test1.Service1;
import org.junit.Test;
import org.springframework.aop.aspectj.annotation.AspectJProxyFactory;

public class AopTest10 {

    @Test
    public void test1() {
        Service1 target = new Service1();
        Class<BeforeAspect1> aspectClass = BeforeAspect1.class;
        AspectJProxyFactory proxyFactory = new AspectJProxyFactory();
        proxyFactory.setTarget(target);
        proxyFactory.addAspect(aspectClass);
        Service1 proxy = proxyFactory.getProxy();
        System.out.println(proxy.say("路人"));
        System.out.println(proxy.work("路人"));
    }
}

```

运行输出

```

我是前置通知!
你好: 路人
我是前置通知!
开始工作了: 路人

```

对应的通知类

@Before通知最后会被解析为下面这个通知类

```
org.springframework.aop.aspectj.AspectJMethodBeforeAdvice
```

通知中获取被调方法信息

通知中如果想获取被调用方法的信息，分2种情况

1. 非环绕通知，可以将 `org.aspectj.lang.JoinPoint` 作为通知方法的第1个参数，通过这个参数获取被调用方法的信息
2. 如果是环绕通知，可以将 `org.aspectj.lang.ProceedingJoinPoint` 作为方法的第1个参数，通过这个参数获取被调用方法的信息

JoinPoint：连接点信息

```
org.aspectj.lang.JoinPoint
```

提供访问当前被通知方法的目标对象、代理对象、方法参数等数据：

```
package org.aspectj.lang;
import org.aspectj.lang.reflect.SourceLocation;

public interface JoinPoint {
    String toString();           //连接点所在位置的相关信息
    String toShortString();      //连接点所在位置的简短相关信息
    String toLongString();       //连接点所在位置的全部相关信息
    Object getThis();            //返回AOP代理对象
    Object getTarget();          //返回目标对象
    Object[] getArgs();          //返回被通知方法参数列表，也就是目前调用目标方法传入的参数
    Signature getSignature();    //返回当前连接点签名，这个可以用来获取目标方法的详细信息，
    //如方法Method对象等
    SourceLocation getSourceLocation(); //返回连接点方法所在类文件中的位置
    String getKind();            //连接点类型
    StaticPart getStaticPart();  //返回连接点静态部分
}
```

ProceedingJoinPoint：环绕通知连接点信息

用于环绕通知，内部主要关注2个方法，一个有参的，一个无参的，用来继续执行拦截器链上的下一个通知。

```
package org.aspectj.lang;
import org.aspectj.runtime.internal.AroundClosure;

public interface ProceedingJoinPoint extends JoinPoint {

    /**
     * 继续执行下一个通知或者目标方法的调用
     */
    public Object proceed() throws Throwable;

    /**
     * 继续执行下一个通知或者目标方法的调用
     */
    public Object proceed(Object[] args) throws Throwable;
}
```

Signature: 连接点签名信息

注意 `JoinPoint#getSignature()` 这个方法，用来获取连接点的签名信息，这个比较重要

```
signature getSignature();
```

通常情况，spring中的aop都是用来对方法进行拦截，所以通常情况下连接点都是一个具体的方法，`Signature` 有个子接口

```
org.aspectj.lang.reflect.MethodSignature
```

`JoinPoint#getSignature()` 都可以转换转换为 `MethodSignature` 类型，然后通过这个接口提供的一些方法来获取被调用的方法的详细信息。

下面对上面的前置通知的案例改造一下，获取被调用方法的详细信息，新建一个Aspect类：

```
BeforeAspect2
```

```
package com.javacode2018.aop.demo10.test2;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.Signature;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.Pointcut;
import org.aspectj.lang.reflect.MethodSignature;

import java.lang.reflect.Method;

@Aspect
public class BeforeAspect2 {

    @Pointcut("execution(* com.javacode2018.aop.demo10.test1.Service1.*(..))")
    public void pc() {
    }

    @Before("com.javacode2018.aop.demo10.test2.BeforeAspect2.pc()")
    public void before(JoinPoint joinPoint) {
        //获取连接点签名
        Signature signature = joinPoint.getSignature();
        //将其转换为方法签名
        MethodSignature methodSignature = (MethodSignature) signature;
        //通过方法签名获取被调用的目标方法
        Method method = methodSignature.getMethod();
        //输出方法信息
        System.out.println(method);
    }
}
```

测试用例

```

@Test
public void test2() {
    Service1 target = new Service1();
    Class<BeforeAspect2> aspectClass = BeforeAspect2.class;
    AspectJProxyFactory proxyFactory = new AspectJProxyFactory();
    proxyFactory.setTarget(target);
    proxyFactory.addAspect(aspectClass);
    Service1 proxy = proxyFactory.getProxy();
    System.out.println(proxy.say("路人"));
    System.out.println(proxy.work("路人"));
}

```

运行输出

```

public java.lang.String
com.javacode2018.aop.demo10.test1.Service1.say(java.lang.String)
你好: 路人
public java.lang.String
com.javacode2018.aop.demo10.test1.Service1.work(java.lang.String)
开始工作了: 路人

```

@Around: 环绕通知

介绍

环绕通知会包裹目标方法的执行，可以在通知内部调用 `ProceedingJoinPoint.process` 方法继续执行下一个拦截器。

用起来和@Before类似，但是有2点不一样

1. 若需要获取目标方法的信息，需要将ProceedingJoinPoint作为第一个参数
2. 通常使用Object类型作为方法的返回值，返回值也可以为void

特点

环绕通知比较特殊，其他4种类型的通知都可以用环绕通知来实现。

案例

通过环绕通知来统计方法的耗时。

```

package com.javacode2018.aop.demo10.test3;

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.Signature;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Pointcut;
import org.aspectj.lang.reflect.MethodSignature;

import java.lang.reflect.Method;

@Aspect
public class AroundAspect3 {

```

```

@Pointcut("execution(* com.javacode2018.aop.demo10.test1.Service1.*(..))")
public void pc() {
}

@Around("com.javacode2018.aop.demo10.test3.AroundAspect3.pc()")
public Object around(ProceedingJoinPoint joinPoint) throws Throwable {
    //获取连接点签名
    Signature signature = joinPoint.getSignature();
    //将其转换为方法签名
    MethodSignature methodSignature = (MethodSignature) signature;
    //通过方法签名获取被调用的目标方法
    Method method = methodSignature.getMethod();

    long startTime = System.nanoTime();
    //调用proceed方法，继续调用下一个通知
    Object returnValue = joinPoint.proceed();
    long endTime = System.nanoTime();
    long costTime = endTime - startTime;
    //输出方法信息
    System.out.println(String.format("%s, 耗时(纳秒): %s", method.toString(),
costTime));
    //返回方法的返回值
    return returnValue;
}
}

```

测试用例

```

@Test
public void test3() {
    Service1 target = new Service1();
    Class<AroundAspect3> aspectClass = AroundAspect3.class;
    AspectJProxyFactory proxyFactory = new AspectJProxyFactory();
    proxyFactory.setTarget(target);
    proxyFactory.addAspect(aspectClass);
    Service1 proxy = proxyFactory.getProxy();
    System.out.println(proxy.say("路人"));
    System.out.println(proxy.work("路人"));
}

```

运行输出

```

public java.lang.String
com.javacode2018.aop.demo10.test1.Service1.say(java.lang.String), 耗时(纳秒):
19000500
你好: 路人
public java.lang.String
com.javacode2018.aop.demo10.test1.Service1.work(java.lang.String), 耗时(纳秒):
59600
开始工作了: 路人

```

对应的通知类

@Around通知最后会被解析为下面这个通知类

```
org.springframework.aop.aspectj.AspectJAroundAdvice
```

@After：后置通知

介绍

后置通知，在方法执行之后执行，用法和前置通知类似。

特点

- 不管目标方法是否有异常，后置通知都会执行
- 这种通知无法获取方法返回值
- 可以使用 `JoinPoint` 作为方法的第一个参数，用来获取连接点的信息

案例

在 `Service1` 中任意方法执行完毕之后，输出一行日志。

```
package com.javacode2018.aop.demo10.test4;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.After;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Pointcut;
import org.aspectj.lang.reflect.MethodSignature;

@Aspect
public class AfterAspect4 {

    @Pointcut("execution(* com.javacode2018.aop.demo10.test1.Service1.*(..))")
    public void pc() {
    }

    @After("com.javacode2018.aop.demo10.test4.AfterAspect4.pc()")
    public void after(JoinPoint joinPoint) throws Throwable {
        MethodSignature methodSignature = (MethodSignature)
joinPoint.getSignature();
        System.out.println(String.format("%s, 执行完毕!",
methodSignature.getMethod()));
    }
}
```

测试案例

```

@Test
public void test4() {
    Service1 target = new Service1();
    Class<AfterAspect4> aspectClass = AfterAspect4.class;
    AspectJProxyFactory proxyFactory = new AspectJProxyFactory();
    proxyFactory.setTarget(target);
    proxyFactory.addAspect(aspectClass);
    Service1 proxy = proxyFactory.getProxy();
    System.out.println(proxy.say("路人"));
    System.out.println(proxy.work("路人"));
}

```

运行输出

```

public java.lang.String
com.javacode2018.aop.demo10.test1.Service1.say(java.lang.String),执行完毕!
你好: 路人
public java.lang.String
com.javacode2018.aop.demo10.test1.Service1.work(java.lang.String),执行完毕!
开始工作了: 路人

```

对应的通知类

@After通知最后会被解析为下面这个通知类

```
org.springframework.aop.aspectj.AspectJAfterAdvice
```

这个类中有 `invoke` 方法，这个方法内部会调用被通知的方法，其内部采用 `try..finally` 的方式实现的，所以不管目标方法是否有异常，通知一定会被执行。

```

@Override
public Object invoke(MethodInvocation mi) throws Throwable {
    try {
        //继续执行下一个拦截器
        return mi.proceed();
    }
    finally {
        //内部通过反射调用被@After标注的方法
        invokeAdviceMethod(getJoinPointMatch(), null, null);
    }
}

```

@AfterReturning: 返回通知

用法

返回通知，在方法返回结果之后执行。

特点

- 可以获取到方法的返回值
- 当目标方法返回异常的时候，这个通知不会被调用，这点和@After通知是有区别的

案例

后置通知中打印出方法及返回值信息。

```
package com.javacode2018.aop.demo10.test5;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.AfterReturning;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Pointcut;
import org.aspectj.lang.reflect.MethodSignature;

@Aspect
public class AfterReturningAspect5 {
    @Pointcut("execution(* com.javacode2018.aop.demo10.test1.Service1.*(..))")
    public void pc() {
    }

    @AfterReturning(value =
"com.javacode2018.aop.demo10.test5.AfterReturningAspect5.pc()", returning =
"retVal")
    public void afterReturning(JoinPoint joinPoint, Object retVal) throws
Throwable {
        MethodSignature methodSignature = (MethodSignature)
joinPoint.getSignature();
        System.out.println(String.format("%s返回值: %s",
methodSignature.getMethod(), retVal));
    }
}
```

注意@AfterReturning注解，用到了2个参数

- value：用来指定切入点
- returning：用来指定返回值对应方法的参数名称，返回值对应方法的第二个参数，名称为retVal

对应的通知类

@AfterReturning通知最后会被解析为下面这个通知类

```
org.springframework.aop.aspectj.AspectJAfterReturningAdvice
```

@AfterThrowing：异常通知

用法

在方法抛出异常之后会回调 `@AfterThrowing` 标注的方法。

`@AfterThrowing` 标注的方法可以指定异常的类型，当被调用的方法触发该异常及其子类型的异常之后，会触发异常方法的回调。也可以不指定异常类型，此时会匹配所有异常。

未指定异常类型

未指定异常类型，可以匹配所有异常类型，如下

```
@AfterThrowing(value = "切入点")
public void afterThrowing()
```

指定异常类型

通过 `@AfterThrowing` 的 `throwing` 指定参数异常参数名称，我们用方法的第二个参数用来接收异常，第二个参数名称为 `e`，下面的代码，当目标方法发生 `IllegalArgumentException` 异常及其子类型异常时，下面的方法会被回调。

```
@AfterThrowing(value =
    "com.javacode2018.aop.demo10.test6.AfterThrowingAspect6.pc()", throwing = "e")
public void afterThrowing(JoinPoint joinPoint, IllegalArgumentException e)
```

特点

- 不论异常是否被异常通知捕获，异常还会继续向外抛出。

案例

Service1中加了login方法，用户名不是 路人甲java 时抛出异常。

```
package com.javacode2018.aop.demo10.test1;

public class Service1 {
    public String say(String name) {
        return "你好: " + name;
    }

    public String work(String name) {
        return "开始工作了: " + name;
    }

    public boolean login(String name) {
        if (!"路人甲java".equals(name)) {
            throw new IllegalArgumentException("非法访问!");
        }
        return true;
    }
}
```

来个异常通知

```
package com.javacode2018.aop.demo10.test6;
```



```

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.AfterReturning;
import org.aspectj.lang.annotation.AfterThrowing;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Pointcut;
import org.aspectj.lang.reflect.MethodSignature;

@Aspect
public class AfterThrowingAspect6 {
    @Pointcut("execution(* com.javacode2018.aop.demo10.test1.Service1.*(..))")
    public void pc() {
    }

    @AfterThrowing(value =
"com.javacode2018.aop.demo10.test6.AfterThrowingAspect6.pc()", throwing = "e")
    public void afterThrowing(JoinPoint joinPoint, IllegalArgumentException e) {
        MethodSignature methodSignature = (MethodSignature)
joinPoint.getSignature();
        System.out.println(String.format("%s发生异常,异常信息: %s",
methodSignature.getMethod(), e.getMessage()));
    }
}

```

测试用例

```

@Test
public void test6() {
    Service1 target = new Service1();
    Class<AfterThrowingAspect6> aspectClass = AfterThrowingAspect6.class;
    AspectJProxyFactory proxyFactory = new AspectJProxyFactory();
    proxyFactory.setTarget(target);
    proxyFactory.addAspect(aspectClass);
    Service1 proxy = proxyFactory.getProxy();
    proxy.login("路人");
}

```

运行输出

```

public boolean
com.javacode2018.aop.demo10.test1.Service1.login(java.lang.String)发生异常,异常信息: 非法访问!

java.lang.IllegalArgumentException: 非法访问!

    at com.javacode2018.aop.demo10.test1.Service1.login(Service1.java:14)
    at
com.javacode2018.aop.demo10.test1.Service1$$FastClassBySpringCGLIB$$ea03ccbe.invoke(<generated>)
    at org.springframework.cglib.proxy.MethodProxy.invoke(MethodProxy.java:218)
    at
org.springframework.aop.framework.CglibAopProxy$CglibMethodInvocation.invokeJoin
point(CglibAopProxy.java:769)

```

对应的通知类

@AfterThrowing通知最后会被解析为下面这个通知类

```
org.springframework.aop.aspectj.AspectJAfterThrowingAdvice
```

来看一下这个类的 `invoke` 方法，这个方法是关键

```
@Override
public Object invoke(MethodInvocation mi) throws Throwable {
    try {
        //继续调用下一个拦截器链
        return mi.proceed();
    }
    catch (Throwable ex) {
        //判断ex和需要不糊的异常是否匹配
        if (shouldInvokeOnThrowing(ex)) {
            //通过反射调用@AfterThrowing标注的方法
            invokeAdviceMethod(getJoinPointMatch(), null, ex);
        }
        //继续向外抛出异常
        throw ex;
    }
}
```

几种通知对比

通知类型	执行时间点	可获取返回值	目标方法异常时是否会执行
@Before	方法执行之前	否	是
@Around	环绕方法执行	是	自己控制
@After	方法执行后	否	是
@AfterReturning	方法执行后	是	否
@AfterThrowing	方法发生异常后	否	是



案例源码

<https://gitee.com/javacode2018/spring-series>

路人甲java所有案例代码以后都会放到这个上面, 大家watch一下, 可以持续关注动态。

Spring系列

1. [Spring系列第1篇: 为何要学spring?](#)
2. [Spring系列第2篇: 控制反转 \(IoC\) 与依赖注入 \(DI\)](#)
3. [Spring系列第3篇: Spring容器基本使用及原理](#)
4. [Spring系列第4篇: xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇: 创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇: 玩转bean scope, 避免跳坑里!](#)
7. [Spring系列第7篇: 依赖注入之手动注入](#)
8. [Spring系列第8篇: 自动注入 \(autowire\) 详解, 高手在于坚持](#)
9. [Spring系列第9篇: depend-on到底是干什么的?](#)
10. [Spring系列第10篇: primary可以解决什么问题?](#)
11. [Spring系列第11篇: bean中的autowire-candidate又是干什么的?](#)
12. [Spring系列第12篇: lazy-init: bean延迟初始化](#)
13. [Spring系列第13篇: 使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇: lookup-method和replaced-method比较陌生, 怎么玩的?](#)
15. [Spring系列第15篇: 代理详解 \(Java动态代理&cglib代理\) ?](#)
16. [Spring系列第16篇: 深入理解java注解及spring对注解的增强 \(预备知识\)](#)
17. [Spring系列第17篇: @Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇: @ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇: @import详解\(bean批量注册\)](#)

20. [Spring系列第20篇: @Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇: 注解实现依赖注入 \(@Autowired、@Resource、@Primary、@Qualifier\)](#)
22. [Spring系列第22篇: @Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇: Bean生命周期详解](#)
24. [Spring系列第24篇: 父子容器详解](#)
25. [Spring系列第25篇: @Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇: 国际化详解](#)
27. [Spring系列第27篇: spring事件机制详解](#)
28. [Spring系列第28篇: Bean循环依赖详解](#)
29. [Spring系列第29篇: BeanFactory扩展 \(BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor\)](#)
30. [Spring系列第30篇: jdk动态代理和cglib代理](#)
31. [Spring系列第31篇: aop概念详解](#)
32. [Spring系列第32篇: AOP核心源码、原理详解](#)
33. [Spring系列第33篇: ProxyFactoryBean创建AOP代理](#)
34. [Spring系列第34篇: @Aspect中@Pointcut 12种用法](#)

更多好文章

1. [Java高并发系列 \(共34篇\)](#)
2. [MySQL高手系列 \(共27篇\)](#)
3. [Maven高手系列 \(共10篇\)](#)
4. [Mybatis系列 \(共12篇\)](#)
5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要, 它是什么? 怎么实现?](#)
7. [泛型, 有点难度, 会让很多人懵逼, 那是因为你没有看这篇文章!](#)



领取福利，扫码发送java

- 1、1000本java电子书大全
- 2、2021版java面试通关手册
- 3、月薪5万必备的java微服务学习线路图及高清视频
- 4、邀请加入高质量java技术交流群

加我微信: [spring888java](#), 进群一起交流java相关各种技术!!

Spring系列第36篇: @EnableAspectJAutoProxy、@Aspect中 通知顺序详解

这是aop最后一篇文章了, 本文带你深入理解@EnableAspectJAutoProxy, 这篇文章可能会颠覆你以前所掌握的一些知识, 让你醍醐灌顶, 欣喜若狂!

1、Aop相关阅读

阅读本文之前，需要先掌握下面几篇文章内容，不然会比较吃力。

1. [Spring系列第15篇：代理详解（java动态代理&CGLIB代理）](#)
2. [Spring系列第30篇：jdk动态代理和cglib代理](#)
3. [Spring系列第31篇：Aop概念详解](#)
4. [Spring系列第32篇：AOP核心源码、原理详解](#)
5. [Spring系列第33篇：ProxyFactoryBean创建AOP代理](#)
6. [Spring系列第34篇：@Aspect中@Pointcut 12种用法](#)
7. [Spring系列第35篇：@Aspect中5中通知详解](#)

目前为止，上面的文章基本上都是硬编码的方式一个个为目标对象创建代理的，但是，我们使用spring的过程中，可能需要对大量bean创建代理，比如我们需拦截所有的service的方法，打印耗时日志，对大量service bean做权限校验，做事务处理等等，这些功能都可以通过aop的方式来实现，若采用硬编码的方式一个个创建，那是相当难受的事情。

Spring中提供了批量的方式，为容器中符合条件的bean，自动创建代理对象，也就是我们本文要说的 `@EnableAspectJAutoProxy`。

2、@EnableAspectJAutoProxy自动为bean创建代理对象

`@EnableAspectJAutoProxy` 可以自动为spring容器中符合条件的bean创建代理对象，

`@EnableAspectJAutoProxy` 需要结合 `@Aspect` 注解一起使用。用法比较简单，下面我们通过案例来看一下。

先在 `com.javacode2018.aop.demo11.test1` 包中定义2个bean

UserService bean

```
package com.javacode2018.aop.demo11.test1;

import org.springframework.stereotype.Component;

@Component
public class UserService {
    public void say(){
        System.out.println("我是UserService");
    }
}
```

CarService bean

```

package com.javacode2018.aop.demo11.test1;

import org.springframework.stereotype.Component;

@Component
public class CarService {
    public void say() {
        System.out.println("我是CarService");
    }
}

```

通过Aspect来定义一个前置通知，需要拦截上面2个bean的所有方法，在方法执行之前输出一行日志

```

package com.javacode2018.aop.demo11.test1;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.Pointcut;
import org.springframework.stereotype.Component;

@Component //@1
@Aspect //@2
public class Aspect1 {

    @Pointcut("execution(* com.javacode2018.aop.demo11.test1..*(..))") //@3
    public void pc() {
    }

    @Before("com.javacode2018.aop.demo11.test1.Aspect1.pc()") //@4
    public void before(JoinPoint joinPoint) {
        System.out.println("我是前置通知,target:" + joinPoint.getTarget()); //@5
    }
}

```

Aspect1中有4个关键信息

@1: 使用 @Component 将这个类注册到spring容器;

@2: 使用 @Aspect 标注着是一个 AspectJ 来定义通知的配置类;

@3: 定义切入点，目前的配置，会拦截test1包及其子包中所有类的所有方法，而CarService和UserService刚好满足，所以会被拦截;

@4: 定义一个前置通知，这个通知会对@3定义的切入点起效;

@5: 目标方法执行执行，输出一行日志;

下面来一个spring配置类

```

package com.javacode2018.aop.demo11.test1;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.EnableAspectJAutoProxy;

@ComponentScan //@1
@EnableAspectJAutoProxy //@2
public class MainConfig1 {
}

```

@1: @ComponentScan 注解的作用会扫描当前包中的类，将标注有 @Component 的类注册到 spring 容器；

@2: @EnableAspectJAutoProxy 这个注解比较关键，用来启用自动代理的创建，简单点理解：会找到容器中所有标注有 @Aspect 注解的 bean 以及 Advisor 类型的 bean，会将他们转换为 Advisor 集合，spring 会通过 Advisor 集合对容器中满足切入点表达式的 bean 生成代理对象，整个都是 spring 容器启动的过程中自动完成的，原理稍后介绍。

下面来测试用例代码，启动 spring 容器，加载配置类，验证

```

package com.javacode2018.aop.demo11;

import com.javacode2018.aop.demo11.test1.CarService;
import com.javacode2018.aop.demo11.test1.MainConfig1;
import com.javacode2018.aop.demo11.test1.UserService;
import org.junit.Test;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class AspectTest11 {

    @Test
    public void test1() {
        AnnotationConfigApplicationContext context = new
AnnotationConfigApplicationContext();
        context.register(MainConfig1.class);
        context.refresh();
        UserService userService = context.getBean(UserService.class);
        userService.say();
        CarService carService = context.getBean(CarService.class);
        carService.say();
    }

}

```

运行输出

```

我是前置通知,target:com.javacode2018.aop.demo11.test1.UserService@dc7df28
我是UserService
我是前置通知,target:com.javacode2018.aop.demo11.test1.CarService@821330f
我是CarService

```

3、通知执行顺序

`@EnableAspectJAutoProxy` 允许spring容器中通过Advisor、`@Aspect` 来定义通知，当spring容器中存在多个Advisor、`@Aspect`时，组成的拦截器调用链顺序是什么样的呢？在介绍这个之前，我们需要先回顾一下aop中4种通知相关知识。

spring aop中4种通知 (Advice)

```
org.aopalliance.intercept.MethodInterceptor
org.springframework.aop.MethodBeforeAdvice
org.springframework.aop.AfterReturningAdvice
org.springframework.aop.ThrowsAdvice
```

所有的通知最终都需要转换为 `MethodInterceptor` 类型的通知，然后组成一个 `MethodInterceptor` 列表，我们称之为方法调用链或者拦截器链，上面列表中后面3通过下面的转换器将其包装为 `MethodInterceptor` 类型的通知：

```
org.springframework.aop.MethodBeforeAdvice ->
org.springframework.aop.framework.adapter.MethodBeforeAdviceInterceptor
org.springframework.aop.AfterReturningAdvice ->
org.springframework.aop.framework.adapter.AfterReturningAdviceInterceptor
org.springframework.aop.ThrowsAdvice ->
org.springframework.aop.framework.adapter.ThrowsAdviceInterceptor
```

下面我们再来看一下4种通知的用法和执行过程，以方便我们理解其执行顺序。

org.aopalliance.intercept.MethodInterceptor：方法拦截器

方法拦截器，这个比较强大，可以在方法执行前后执行一些增强操作，其他类型的通知最终都会被包装为 `MethodInterceptor` 来执行。

下面我们自定义一个 `MethodInterceptor`

```
class MyMethodInterceptor implements MethodInterceptor {
    @Override
    public Object invoke(MethodInvocation invocation) throws Throwable {
        System.out.println("我是MethodInterceptor start");
        //调用invocation.proceed()执行下一个拦截器
        Object result = invocation.proceed();
        System.out.println("我是MethodInterceptor end");
        //返回结果
        return result;
    }
}
```

org.springframework.aop.MethodBeforeAdvice：方法前置通知

方法前置通知，可以在方法之前定义增强操作。

下面我们自定义一个 `MethodBeforeAdvice`


```

class MyMethodBeforeAdvice implements MethodBeforeAdvice {

    @Override
    public void before(Method method, Object[] args, @Nullable Object target)
    throws Throwable {
        System.out.println("我是MethodBeforeAdvice");
    }
}

```

MethodBeforeAdvice最终会被包装为**MethodBeforeAdviceInterceptor**类型，然后放到拦截器链中去执行，通过**MethodBeforeAdviceInterceptor**代码可以理解**MethodBeforeAdvice**的执行过程

```

public class MethodBeforeAdviceInterceptor implements MethodInterceptor,
BeforeAdvice, Serializable {

    private final MethodBeforeAdvice advice;

    public MethodBeforeAdviceInterceptor(MethodBeforeAdvice advice) {
        this.advice = advice;
    }

    @Override
    public Object invoke(MethodInvocation mi) throws Throwable {
        //调用MethodBeforeAdvice的before方法，执行前置通知
        this.advice.before(mi.getMethod(), mi.getArguments(), mi.getThis());
        //执行下一个拦截器
        return mi.proceed();
    }
}

```

org.springframework.aop.AfterReturningAdvice：方法返回通知

方法返回通知，用来在方法执行完毕之后执行一些增强操作。

下面我们自定义一个AfterReturningAdvice

```

class MyAfterReturningAdvice implements AfterReturningAdvice {

    @Override
    public void afterReturning(@Nullable Object returnValue, Method method,
    Object[] args, @Nullable Object target) throws Throwable {
        System.out.println("我是AfterReturningAdvice");
    }
}

```

AfterReturningAdvice最终会被包装为**AfterReturningAdviceInterceptor**类型，然后放到拦截器链中去执行，通过**AfterReturningAdviceInterceptor**代码可以理解**AfterReturningAdvice**的执行过程

```

public class AfterReturningAdviceInterceptor implements MethodInterceptor,
AfterAdvice, Serializable {

    private final AfterReturningAdvice advice;

```

```

    public AfterReturningAdviceInterceptor(AfterReturningAdvice advice) {
        this.advice = advice;
    }

    @Override
    public Object invoke(MethodInvocation mi) throws Throwable {
        //执行下一个拦截器，可以获取目标方法的返回结果
        Object retVal = mi.proceed();
        //调用方法返回通知的afterReturning方法，会传入目标方法的返回值等信息
        this.advice.afterReturning(retVal, mi.getMethod(), mi.getArguments(),
            mi.getThis());
        return retVal;
    }
}

```

org.springframework.aop.ThrowsAdvice: 异常通知

当目标方法发生异常时，可以通过 ThrowsAdvice 来指定需要回调的方法，我们在此可以记录一些异常信息，或者将异常信息发送到监控系统等。

下面我们自定义一个ThrowsAdvice

```

/**
 * 用来定义异常通知
 * 方法名必须是afterThrowing，格式参考下面2种定义
 * 1. public void afterThrowing(Exception ex)
 * 2. public void afterThrowing(Method method, Object[] args, Object target,
 *    Exception ex)
 */
class MyThrowsAdvice implements ThrowsAdvice {
    public void afterThrowing(Method method, Object[] args, Object target,
        Exception ex) {
        System.out.println("我是ThrowsAdvice");
    }
}

```

ThrowsAdvice最终会被包装为ThrowsAdviceInterceptor类型，然后放到拦截器链中去执行，通过 ThrowsAdviceInterceptor代码可以理解ThrowsAdvice的执行过程，ThrowsAdviceInterceptor 构造参数传入一个自定义的 ThrowsAdvice 对象

```

public class ThrowsAdviceInterceptor implements MethodInterceptor, AfterAdvice {

    private final Object throwsAdvice;

    public ThrowsAdviceInterceptor(Object throwsAdvice) {
        this.throwsAdvice = throwsAdvice;
    }

    @Override
    public Object invoke(MethodInvocation mi) throws Throwable {
        try {
            return mi.proceed();
        } catch (Throwable ex) {
            //调用 ThrowsAdvice 中的 afterThrowing 方法来处理异常
        }
    }
}

```

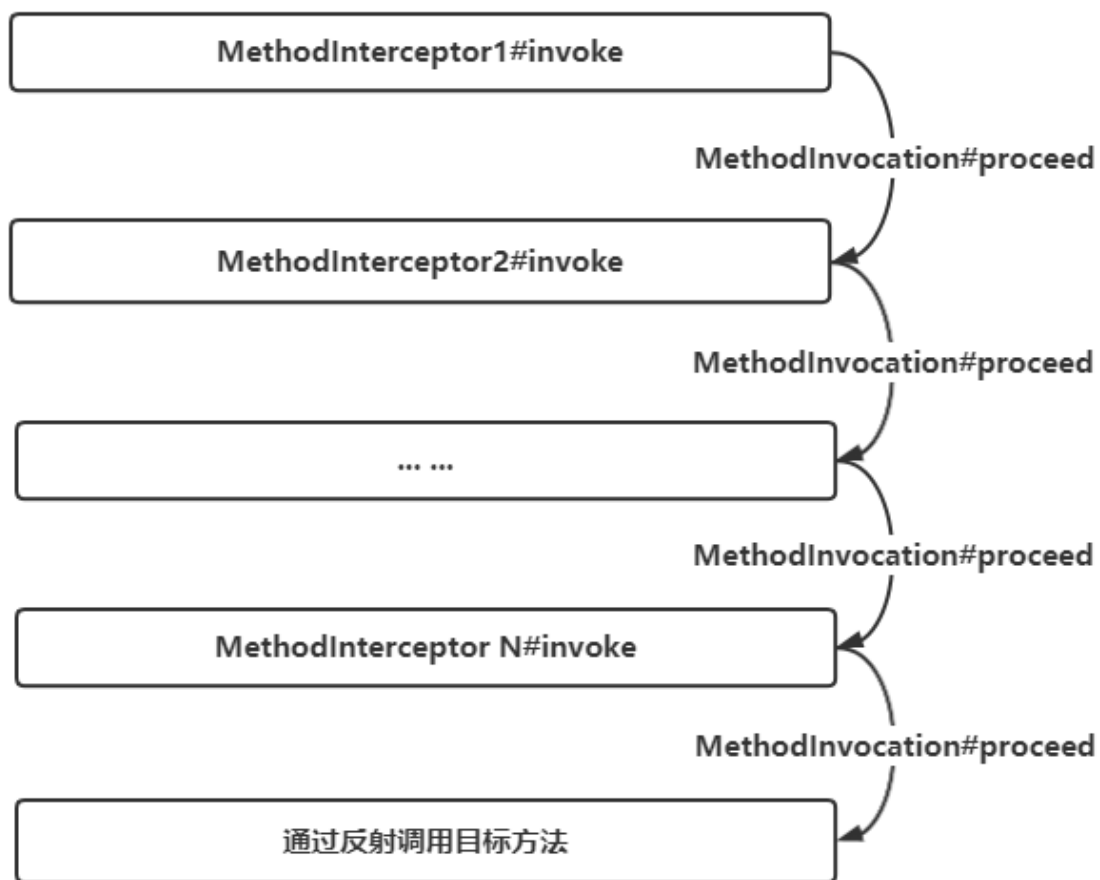
```

        this.throwsAdvice.afterThrowing(。。。);
        //将异常继续往外抛
        throw ex;
    }
}

```

拦截器链执行过程

假如目标方法上面有好几个通知，调用目标方法执行，spring会将所有的通知转换得到一个 `MethodInterceptor` 列表，然后依次按照下面的方式执行，会先调用第一个拦截器的 `MethodInterceptor#invoke(MethodInvocation invocation)` 方法，会传递一个 `MethodInvocation` 类型的参数，在此方法中，我们可以调用 `MethodInvocation#proceed` 方法去执行第二个拦截器，然后依次按照这样的过程执行，到了最后一个 `MethodInterceptor` 中，再次调用 `MethodInvocation#proceed` 时，会调用目标方法。



4种通知的执行顺序

结合上面的过程，假如目标方法上面依次添加了下面4种通知，我们来分析一下他们的执行过程

```

class MyMethodInterceptor implements MethodInterceptor {
    @Override
    public Object invoke(MethodInvocation invocation) throws Throwable {
        System.out.println("我是MethodInterceptor start");
        //调用invocation.proceed()执行下一个拦截器
        Object result = invocation.proceed();
        System.out.println("我是MethodInterceptor end");
        //返回结果
        return result;
    }
}

```

```

    }
}

class MyMethodBeforeAdvice implements MethodBeforeAdvice {

    @Override
    public void before(Method method, Object[] args, @Nullable Object target)
throws Throwable {
        System.out.println("我是MethodBeforeAdvice");
    }
}

class MyAfterReturningAdvice implements AfterReturningAdvice {

    @Override
    public void afterReturning(@Nullable Object returnValue, Method method,
Object[] args, @Nullable Object target) throws Throwable {
        System.out.println("我是AfterReturningAdvice");
    }
}

class MyThrowsAdvice implements ThrowsAdvice {
    public void afterThrowing(Method method, Object[] args, Object target,
Exception ex) {
        System.out.println("我是ThrowsAdvice");
    }
}

```

根据通知的规定，非 `MethodInterceptor` 类型的通知，都会被包装为 `MethodInterceptor` 类型的，上面除了第一个之外，其他3个都会被转换为 `MethodInterceptor`，转换之后变成了下面这样：

```

class MyMethodInterceptor implements MethodInterceptor {
    @Override
    public Object invoke(MethodInvocation mi) throws Throwable {
        System.out.println("我是MethodInterceptor start");
        //调用mi.proceed()执行下一个拦截器
        Object retVal = mi.proceed();
        System.out.println("我是MethodInterceptor end");
        //返回结果
        return retVal;
    }
}

class MyMethodBeforeAdvice implements MethodInterceptor {
    @Override
    public Object invoke(MethodInvocation mi) throws Throwable {
        System.out.println("我是MethodBeforeAdvice");
        //调用mi.proceed()执行下一个拦截器
        Object retVal = mi.proceed();
        return retVal;
    }
}

class MyAfterReturningAdvice implements MethodInterceptor {
    @Override
    public Object invoke(MethodInvocation mi) throws Throwable {
        //调用mi.proceed()执行下一个拦截器

```

```

        Object retVal = mi.proceed();
        System.out.println("我是AfterReturningAdvice");
        return retVal;
    }
}

class MyThrowsAdvice implements MethodInterceptor {
    @Override
    public Object invoke(MethodInvocation mi) throws Throwable {
        try {
            //调用mi.proceed()执行下一个拦截器
            return mi.proceed();
        } catch (Throwable ex) {
            System.out.println("我是ThrowsAdvice");
            throw ex;
        }
    }
}

```

根据通知链的执行过程，最终变成了下面这样：

```

System.out.println("我是MethodInterceptor start");
System.out.println("我是MethodBeforeAdvice");
Object retVal = null;
try {
    retVal = 通过反射调用目标方法获取返回值;
} catch (Throwable ex) {
    System.out.println("我是ThrowsAdvice");
    throw ex;
}
System.out.println("我是AfterReturningAdvice");
System.out.println("我是MethodInterceptor end");
return retVal;

```

将上面4个通知用到下面目标对象中

```

public static class Service3 {
    public String say(String name) {
        return "你好: " + name;
    }
}

```

执行下面代码生成代理，然后通过代理调用say方法

```

Service3 target = new Service3();
Service3 proxy = 对target通过aop生成代理对象;
System.out.println(proxy.say("路人"));

```

被4个拦截器链包裹之后，`System.out.println(proxy.say("路人"))`; 执行过程变成了下面这样

```

System.out.println("我是MethodInterceptor start");
System.out.println("我是MethodBeforeAdvice");
Object retVal = null;
try {
    retVal = target.say("路人");
} catch (Throwable ex) {
    System.out.println("我是ThrowsAdvice");
    throw ex;
}
System.out.println("我是AfterReturningAdvice");
System.out.println("我是MethodInterceptor end");
System.out.println(retVal);

```

再次简化

```

System.out.println("我是MethodInterceptor start");
System.out.println("我是MethodBeforeAdvice");
Object retVal = null;
try {
    retVal = "你好: " + name;
} catch (Throwable ex) {
    System.out.println("我是ThrowsAdvice");
    throw ex;
}
System.out.println("我是AfterReturningAdvice");
System.out.println("我是MethodInterceptor end");
System.out.println(retVal);

```

最终会输出

```

我是MethodInterceptor start
我是MethodBeforeAdvice
我是AfterReturningAdvice
我是MethodInterceptor end
你好: 路人

```

上案例代码，我们来看一下最终的执行结果是不是和我们分析的一样，下面为需要被代理的类 `Service3` 以及需要使用的4个通知。

```

package com.javacode2018.aop.demo11.test3;

import org.aopalliance.intercept.MethodInterceptor;
import org.aopalliance.intercept.MethodInvocation;
import org.springframework.aop.AfterReturningAdvice;
import org.springframework.aop.MethodBeforeAdvice;
import org.springframework.aop.ThrowsAdvice;
import org.springframework.lang.Nullable;

import java.lang.reflect.Method;

public class MoreAdvice {

    public static class Service3 {
        public String say(String name) {
            return "你好: " + name;
        }
    }
}

```

```

    }
}

public static class MyMethodInterceptor implements MethodInterceptor {
    @Override
    public Object invoke(MethodInvocation invocation) throws Throwable {
        System.out.println("我是MethodInterceptor start");
        //调用invocation.proceed()执行下一个拦截器
        Object result = invocation.proceed();
        System.out.println("我是MethodInterceptor end");
        //返回结果
        return result;
    }
}

public static class MyMethodBeforeAdvice implements MethodBeforeAdvice {

    @Override
    public void before(Method method, Object[] args, @Nullable Object
target) throws Throwable {
        System.out.println("我是MethodBeforeAdvice");
    }
}

public static class MyAfterReturningAdvice implements AfterReturningAdvice {

    @Override
    public void afterReturning(@Nullable Object returnValue, Method method,
Object[] args, @Nullable Object target) throws Throwable {
        System.out.println("我是AfterReturningAdvice");
    }
}

public static class MyThrowsAdvice implements ThrowsAdvice {
    public void afterThrowing(Method method, Object[] args, Object target,
Exception ex) {
        System.out.println("我是ThrowsAdvice");
    }
}
}

```

对应测试代码

```

@Test
public void test3() {
    //创建目标对象
    MoreAdvice.Service3 target = new MoreAdvice.Service3();
    //创建代理工厂，通过代理工厂来创建代理对象
    ProxyFactory proxyFactory = new ProxyFactory();
    proxyFactory.setTarget(target);
    //依次为目标对象添加4种通知
    proxyFactory.addAdvice(new MoreAdvice.MyMethodInterceptor());
    proxyFactory.addAdvice(new MoreAdvice.MyMethodBeforeAdvice());
    proxyFactory.addAdvice(new MoreAdvice.MyAfterReturningAdvice());
    proxyFactory.addAdvice(new MoreAdvice.MyThrowsAdvice());
    //获取到代理对象
    MoreAdvice.Service3 proxy = (MoreAdvice.Service3) proxyFactory.getProxy();
}

```

```
//通过代理对象访问目标方法say
system.out.println(proxy.say("路人"));
}
```

运行输出

```
我是MethodInterceptor start
我是MethodBeforeAdvice
我是AfterReturningAdvice
我是MethodInterceptor end
你好：路人
```

和我们上面分析的确实一模一样。

4、单个@Aspect中多个通知的执行顺序

@Aspect 标注的类中可以使用下面5种注解来定义通知

```
@Before
@Around
@After
@AfterReturning
@AfterThrowing
```

当单个 @Aspect 中定义了多种类型的通知时，@EnableAspectJAutoProxy内部会对其进行排序，排序顺序如下

```
@AfterThrowing
@AfterReturning
@After
@Around
@Before
```

下面我们来个 @Aspect 类，同时定义5种通知，然后来一步步分析一下其执行的属性。

```
package com.javacode2018.aop.demo11.test4;

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.*;
import org.springframework.stereotype.Component;

@Component
@Aspect
public class Aspect4 {
    @Pointcut("execution(* com.javacode2018.aop.demo11.test4.Service4.*(..))")
    public void pc() {
    }

    @Before("pc()")
    public void before() {
        System.out.println("@Before通知!");
    }
}
```



```

@Around("pc()")
public Object around(ProceedingJoinPoint joinPoint) throws Throwable {
    System.out.println("@Around通知start");
    Object result = joinPoint.proceed();
    System.out.println("@Around绕通知end");
    return result;
}

@After("pc()")
public void after() throws Throwable {
    System.out.println("@After通知!");
}

@AfterReturning("pc()")
public void afterReturning() throws Throwable {
    System.out.println("@AfterReturning通知!");
}

@AfterThrowing("pc()")
public void afterThrowing() {
    System.out.println("@AfterThrowing通知!");
}
}

```

上面会拦截 `com.javacode2018.aop.demo11.test4.Service4` 这个类中的所有方法，下面是 `Service4` 的源码。

```

package com.javacode2018.aop.demo11.test4;

import org.springframework.stereotype.Component;

@Component
public class Service4 {
    public String say(String name) {
        return "你好: " + name;
    }
}

```

来个spring的配置类，使用 `@EnableAspectJAutoProxy` 标注

```

package com.javacode2018.aop.demo11.test4;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.EnableAspectJAutoProxy;

@EnableAspectJAutoProxy
@ComponentScan
public class MainConfig4 {
}

```

测试代码

```

@Test
public void test4(){
    AnnotationConfigApplicationContext context = new
AnnotationConfigApplicationContext();
    context.register(MainConfig4.class);
    context.refresh();
    Service4 service4 = context.getBean(Service4.class);
    System.out.println(service4.say("路人"));
}

```

运行依次输出如下

```

@Around通知start
@Before通知!
@Around绕通知end
@After通知!
@AfterReturning通知!
你好: 路人

```

卧槽，这输出好像和我们上面说的不一样的，上面说的会按照下面的顺序执行，这到底是什么情况？

```

@AfterThrowing
@AfterReturning
@After
@Around
@Before

```

别急，排序规则和输出结果都没有问题，听我慢慢分析，下面的分析非常重要，注意看了

5、@Aspect中5种通知回顾

5种通知对应的Advice类

@Aspect中通过5中注解来定义通知，这些注解最终都需要转换为Advice去执行，转换关系如下

通知	对应的Advice类
@AfterThrowing	org.springframework.aop.aspectj.AspectJAfterThrowingAdvice
@AfterReturning	org.springframework.aop.aspectj.AspectJAfterReturningAdvice
@After	org.springframework.aop.aspectj.AspectJAfterAdvice
@Around	org.springframework.aop.aspectj.AspectJAroundAdvice
@Before	org.springframework.aop.aspectj.AspectJMethodBeforeAdvice

重点就在于表格右边的Advice类，当了解这些Advice的源码之后，他们的执行顺序大家就可以理解了，我们来看一下这些类的源码，重点看 `invoke` 方法

@AfterThrowing: AspectJAfterThrowingAdvice

```
public class AspectJAfterThrowingAdvice implements MethodInterceptor {

    @Override
    public Object invoke(MethodInvocation mi) throws Throwable {
        try {
            //执行下一个拦截器
            return mi.proceed();
        } catch (Throwable ex) {
            //通过反射调用@AfterThrowing标注的方法
            //继续抛出异常
            throw ex;
        }
    }
}
```

AspectJAfterThrowingAdvice 实现了 MethodInterceptor 接口，不需要进行包装。

@AfterReturning: AspectJAfterReturningAdvice

AspectJAfterReturningAdvice 源码：

```
public class AspectJAfterReturningAdvice implements AfterReturningAdvice {
    @Override
    public void afterReturning(@Nullable Object returnValue, Method method,
        Object[] args, @Nullable Object target) throws Throwable {
        // 调用@AfterReturning标注的方法
    }
}
```

AspectJAfterReturningAdvice 实现了 AfterReturningAdvice 接口，是一个方法返回通知，不是 MethodInterceptor 类型的，所以最终需包装为 MethodInterceptor 类型，变成下面这样

```
public class AspectJAfterReturningAdvice implements MethodInterceptor {
    @Override
    public Object invoke(MethodInvocation mi) throws Throwable {
        //执行下一个拦截器
        Object retVal = mi.proceed();
        //调用@AfterReturning标注的方法
        return retVal;
    }
}
```

@After: AspectJAfterAdvice

AspectJAfterAdvice 源码：

```

public class AspectJAfterAdvice implements MethodInterceptor {
    @Override
    public Object invoke(MethodInvocation mi) throws Throwable {
        try {
            //执行下一个拦截器
            return mi.proceed();
        } finally {
            //调用@After标注的方法
        }
    }
}

```

`AspectJAfterAdvice` 实现了 `MethodInterceptor` 接口，所以最终执行的时候不需要进行包装。

注意 `invoke` 方法内部使用了 `try...finally` 的方式，`@After` 方法的调用放在了 `finally` 中，所以不管是否有异常，`@After` 类型的通知都会被执行。

@Around: AspectJAroundAdvice

`AspectJAroundAdvice` 源码：

```

public class AspectJAroundAdvice extends AbstractAspectJAdvice implements
MethodInterceptor, Serializable {
    @Override
    public Object invoke(MethodInvocation mi) throws Throwable {
        return 调用 @Around标注的方法 ;
    }
}

```

`AspectJAroundAdvice` 实现了 `MethodInterceptor` 接口，最终执行的时候也不需要进行包装。

@Before: AspectJMethodBeforeAdvice

`AspectJMethodBeforeAdvice` 源码：

```

public class AspectJMethodBeforeAdvice implements MethodBeforeAdvice,
Serializable {
    @Override
    public void before(Method method, Object[] args, @Nullable Object target)
throws Throwable {
        invokeAdviceMethod(getJoinPointMatch(), null, null);
    }
}

```

`AspectJMethodBeforeAdvice` 实现了 `MethodBeforeAdvice` 接口，是一个前置通知，不是 `MethodInterceptor` 类型的，所以最终需包装为 `MethodInterceptor` 类型，变成下面这样

```

public class AspectJMethodBeforeAdvice implements MethodInterceptor {
    @Override
    public Object invoke(MethodInvocation mi) throws Throwable {
        //调用@Before标注的方法
        //执行下一个拦截器
        return mi.proceed();
    }
}

```

6、分析单个@Aspect中多个通知执行顺序

大家对@Aspect中5种通知内容理解之后，我们再回头看一下代码 Aspect4 中定义的5个通知

```

public class Aspect4 {
    @Pointcut("execution(* com.javacode2018.aop.demo11.test4.Service4.*(..))")
    public void pc() {
    }

    @Before("pc()")
    public void before() {
        System.out.println("@Before通知!");
    }

    @Around("pc()")
    public Object around(ProceedingJoinPoint joinPoint) throws Throwable {
        System.out.println("@Around通知start");
        Object result = joinPoint.proceed();
        System.out.println("@Around绕通知end");
        return result;
    }

    @After("pc()")
    public void after() throws Throwable {
        System.out.println("@After通知!");
    }

    @AfterReturning("pc()")
    public void afterReturning() throws Throwable {
        System.out.println("@AfterReturning通知!");
    }

    @AfterThrowing("pc()")
    public void afterThrowing() {
        System.out.println("@AfterThrowing通知!");
    }
}

```

我们给出的结论是，会按照下面的顺序执行

```
@AfterThrowing
@AfterReturning
@After
@Around
@Before
```

按照上面的顺序，一步步来分析。

先执行第1个通知 `@AfterThrowing`，变成下面这样

```
try {
    //执行下一个拦截器
    return mi.proceed();
} catch (Throwable ex) {
    System.out.println("@AfterThrowing通知!");
    //继续抛出异常
    throw ex;
}
```

`mi.processed()` 会执行第2个通知 `@AfterReturning`，变成了下面这样

```
try {
    //执行下一个拦截器
    Object retVal = mi.proceed();
    System.out.println("@AfterReturning通知!");
    return retVal;
} catch (Throwable ex) {
    System.out.println("@AfterThrowing通知!");
    //继续抛出异常
    throw ex;
}
```

继续 `mi.proceed()` 执行第3个通知 `@After`，变成了下面这样

```
try {
    Object result = null;
    try {
        //执行下一个拦截器
        result = mi.proceed();
    } finally {
        System.out.println("@After通知!");
    }
    System.out.println("@AfterReturning通知!");
    return retVal;
} catch (Throwable ex) {
    System.out.println("@AfterThrowing通知!");
    //继续抛出异常
    throw ex;
}
```

继续 `mi.proceed()` 执行第4个通知 `@Around`，变成了下面这样

```
try {
    Object result = null;
    try {
```

```

        System.out.println("@Around通知start");
        result = joinPoint.proceed();
        System.out.println("@Around绕通知end");
        return result;
    } finally {
        System.out.println("@After通知!");
    }
    System.out.println("@AfterReturning通知!");
    return retVal;
} catch (Throwable ex) {
    System.out.println("@AfterThrowing通知!");
    //继续抛出异常
    throw ex;
}

```

继续 `joinPoint.proceed()` 执行第5个通知 `@Before`，变成了下面这样

```

try {
    Object result = null;
    try {
        System.out.println("@Around通知start");
        System.out.println("@Before通知!");
        result = mi.proceed();
        System.out.println("@Around绕通知end");
        return result;
    } finally {
        System.out.println("@After通知!");
    }
    System.out.println("@AfterReturning通知!");
    return retVal;
} catch (Throwable ex) {
    System.out.println("@AfterThrowing通知!");
    //继续抛出异常
    throw ex;
}

```

继续 `joinPoint.proceed()` 会调用目标方法，变成了下面这样

```

try {
    Object result = null;
    try {
        System.out.println("@Around通知start");
        System.out.println("@Before通知!");
        result = // 通过反射调用目标方法; //@1
        System.out.println("@Around绕通知end");
        return result;
    } finally {
        System.out.println("@After通知!");
    }
    System.out.println("@AfterReturning通知!");
    return retVal;
} catch (Throwable ex) {
    System.out.println("@AfterThrowing通知!");
    //继续抛出异常
    throw ex;
}

```

将上面的 `@1` 替换为目标方法的调用，就变成下面这样了

```
try {
    Object result = null;
    try {
        System.out.println("@Around通知start");
        System.out.println("@Before通知!");
        result = service4.say("路人");
        System.out.println("@Around绕通知end");
        return result;
    } finally {
        System.out.println("@After通知!");
    }
    System.out.println("@AfterReturning通知!");
    return retVal;
} catch (Throwable ex) {
    System.out.println("@AfterThrowing通知!");
    //继续抛出异常
    throw ex;
}
```

所以最终输出

```
@Around通知start
@Before通知!
@Around绕通知end
@After通知!
@AfterReturning通知!
你好：路人
```

7、@EnableAspectJAutoProxy中为通知指定顺序

`@EnableAspectJAutoProxy` 用在spring环境中，可以通过 `@Aspect` 以及 `Advisor` 来定义多个通知，当spring容器中有多个 `@Aspect`、`Advisor` 时，他们的顺序是什么样的呢？

我们先看一下如何为 `@Aspect`、`自定义Advisor` 指定顺序。

为@Aspect指定顺序：用@Order注解

需要在 `@Aspect` 标注的类上使用 `@org.springframework.core.annotation.Order` 注解，值越小，通知的优先级越高。

```
@Aspect
@Order(1)
public class AspectOrder1{}
```

为Advisor指定顺序：实现Ordered接口

自定义的 `Advisor` 通过 `org.springframework.core.Ordered` 接口来指定顺序，这个接口有个 `public int getOrder()` 方法，用来返回通知的顺序。

spring为我们提供了一个 `Advisor` 类型的抽象类

`org.springframework.aop.support.AbstractPointcutAdvisor`，这个类实现了 `Ordered` 接口，spring中大部分 `Advisor` 会是继承 `AbstractPointcutAdvisor`，若需要自定义 `Advisor`，也可以继承这个类，这个类的 `getOrder` 方法比较关键，来看一下

```
public abstract class AbstractPointcutAdvisor implements PointcutAdvisor,
    Ordered, Serializable {

    @Nullable
    private Integer order;

    public void setOrder(int order) {
        this.order = order;
    }

    @Override
    public int getOrder() {
        //若当前Advisor指定了order，则直接返回
        if (this.order != null) {
            return this.order;
        }
        //获取当前类中配置的通知对象Advice
        Advice advice = getAdvice();
        //若advice实现了Ordered接口，这从advice中获取通知的顺序
        if (advice instanceof Ordered) {
            return ((Ordered) advice).getOrder();
        }
        //否则通知的优先级最低，Integer.MAX_VALUE
        return Ordered.LOWEST_PRECEDENCE;
    }
}
```

Spring为我们提供了一个默认的 `Advisor` 类: `DefaultPointcutAdvisor`，这个类就继承了 `AbstractPointcutAdvisor`，通常我们可以直接使用 `DefaultPointcutAdvisor` 来自定义通知。

8、多个@Aspect、Advisor排序规则

排序规则

- 1、在spring容器中获取@Aspect、Advisor类型的所有bean，得到一个列表 list1
- 2、对list1按照order的值升序排序，得到结果list2
- 3、然后再对list2中@Aspect类型的bean内部的通知进行排序，规则

```
@AfterThrowing
@AfterReturning
@After
@Around
@Before
```

- 4、最后运行的时候会得到上面排序产生的方法调用链列表去执行。

案例

下面我们定义2个@Aspect类，一个Advisor类，并且给这3个都指定，然后来验证一下通知执行的顺序。

先定义目标类

```
package com.javacode2018.aop.demo11.test2;

import org.springframework.stereotype.Component;

@Component
public class Service2 {
    public String say(String name) {
        return "你好: " + name;
    }
}
```

Aspect1: 第2个@Aspect

```
package com.javacode2018.aop.demo11.test2;

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.*;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Aspect
@Order(1)
@Component
public class MyAspect1 {

    @Pointcut("execution(* com.javacode2018.aop.demo11.test2.Service2.*(..))")
    public void pc() {
    }

    @Before("pc()")
    public void before() {
        System.out.println("MyAspect1 @Before通知!");
    }

    @Around("pc()")
    public Object around(ProceedingJoinPoint joinPoint) throws Throwable {
        System.out.println("MyAspect1 @Around通知start");
        Object result = joinPoint.proceed();
        System.out.println("MyAspect1 @Around绕通知end");
        return result;
    }

    @After("pc()")
    public void after() throws Throwable {
        System.out.println("MyAspect1 @After通知!");
    }

    @AfterReturning("pc()")
    public void afterReturning() throws Throwable {
    }
}
```

```

        System.out.println("MyAspect1 @AfterReturning通知!");
    }

    @AfterThrowing("pc()")
    public void afterThrowing() {
        System.out.println("MyAspect1 @AfterThrowing通知!");
    }
}

```

Aspect1: 第2个@Aspect

```

package com.javacode2018.aop.demo11.test2;

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.*;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Aspect
@Order(3)
@Component
public class MyAspect2 {

    @Pointcut("execution(* com.javacode2018.aop.demo11.test2.Service2.*(..))")
    public void pc() {
    }

    @Before("pc()")
    public void before() {
        System.out.println("MyAspect2 @Before通知!");
    }

    @Around("pc()")
    public Object around(ProceedingJoinPoint joinPoint) throws Throwable {
        System.out.println("MyAspect2 @Around通知start");
        Object result = joinPoint.proceed();
        System.out.println("MyAspect2 @Around绕通知end");
        return result;
    }

    @After("pc()")
    public void after() throws Throwable {
        System.out.println("MyAspect2 @After通知!");
    }

    @AfterReturning("pc()")
    public void afterReturning() throws Throwable {
        System.out.println("MyAspect2 @AfterReturning通知!");
    }

    @AfterThrowing("pc()")
    public void afterThrowing() {
        System.out.println("MyAspect2 @AfterThrowing通知!");
    }
}

```

自定义一个Advisor

```
package com.javacode2018.aop.demo11.test2;

import org.aopalliance.intercept.MethodInterceptor;
import org.aopalliance.intercept.MethodInvocation;
import org.springframework.aop.support.DefaultPointcutAdvisor;
import org.springframework.stereotype.Component;

@Component
public class Advisor1 extends DefaultPointcutAdvisor {

    public Advisor1() {
        MethodInterceptor methodInterceptor = new MethodInterceptor() {
            @Override
            public Object invoke(MethodInvocation invocation) throws Throwable {
                System.out.println("Advisor1 start");
                Object result = invocation.proceed();
                System.out.println("Advisor1 end");
                return result;
            }
        };
        this.setAdvice(methodInterceptor);
    }

    @Override
    public int getOrder() {
        return 2;
    }
}
```

来个spring配置类

标注 `@EnableAspectJAutoProxy` 来启用自动化的aop功能

```
package com.javacode2018.aop.demo11.test2;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.EnableAspectJAutoProxy;

@ComponentScan
@EnableAspectJAutoProxy
public class MainConfig2 {
}
```

测试代码

```

@Test
public void test2() {
    AnnotationConfigApplicationContext context = new
AnnotationConfigApplicationContext();
    context.register(MainConfig2.class);
    context.refresh();
    Service2 service2 = context.getBean(Service2.class);
    System.out.println(service2.say("路人"));
}

```

运行输出

```

MyAspect1 @Around通知start
MyAspect1 @Before通知!
Advisor1 start
MyAspect2 @Around通知start
MyAspect2 @Before通知!
MyAspect2 @Around绕通知end
MyAspect2 @After通知!
MyAspect2 @AfterReturning通知!
Advisor1 end
MyAspect1 @Around绕通知end
MyAspect1 @After通知!
MyAspect1 @AfterReturning通知!
你好: 路人

```

结果分析

下面我们一步步来推出结果为什么是上面这样。

先获取spring容器中@Aspect、Advisor类型的所有bean，根据其order升序排序，得到：

```

Aspect1: 顺序是1
Advisor1: 顺序是2
MyAspect2: 顺序是3

```

然后对每个 Aspect 内部的通知进行排序，根据单个 @Aspect 内部通知排序规则，可以得到：

```

Aspect1: 顺序是1
    @AfterThrowing
    @AfterReturning
    @After
    @Around
    @Before

Advisor1: 顺序是2

MyAspect2: 顺序是3
    @AfterThrowing
    @AfterReturning
    @After
    @Around
    @Before

```

下面将代码拿过来一步步填充。

先对 `Aspect1` 进行填充, 得到:

```
try {
    Object result = null;
    try {
        System.out.println("MyAspect1 @Around通知start");
        System.out.println("MyAspect1 @Before通知!");
        result = mi.proceed(); //@1
        System.out.println("MyAspect1 @Around绕通知end");
        return result;
    } finally {
        System.out.println("MyAspect1 @After通知!");
    }
    System.out.println("MyAspect1 @AfterReturning通知!");
    return retVal;
} catch (Throwable ex) {
    System.out.println("MyAspect1 @AfterThrowing通知!");
    //继续抛出异常
    throw ex;
}
```

@1 执行 `mi.proceed()` 会调用下一个拦截器, 即 `Advisor1` 中定义的拦截器, 然后会得到下面代码:

```
try {
    Object result = null;
    try {
        System.out.println("MyAspect1 @Around通知start");
        System.out.println("MyAspect1 @Before通知!");
        System.out.println("Advisor1 start");
        result = invocation.proceed(); //@2
        System.out.println("Advisor1 end");
        System.out.println("MyAspect1 @Around绕通知end");
        return result;
    } finally {
        System.out.println("MyAspect1 @After通知!");
    }
    System.out.println("MyAspect1 @AfterReturning通知!");
    return retVal;
} catch (Throwable ex) {
    System.out.println("MyAspect1 @AfterThrowing通知!");
    //继续抛出异常
    throw ex;
}
```

@2 执行 `mi.proceed()` 会调用下一个拦截器, 即 `Aspect2` 中定义的拦截器, 而 `Aspect2` 和 `Aspect1` 类似, 然后会得到下面代码:

```
try {
    Object result = null;
    try {
        System.out.println("MyAspect1 @Around通知start");
        System.out.println("MyAspect1 @Before通知!");
        System.out.println("Advisor1 start");
        try {
            try {
                System.out.println("MyAspect2 @Around通知start");
```

```

        System.out.println("MyAspect2 @Before通知!");
        result = mi.proceed(); //@3
        System.out.println("MyAspect2 @Around绕通知end");
        return result;
    } finally {
        System.out.println("MyAspect2 @After通知!");
    }
    System.out.println("MyAspect2 @AfterReturning通知!");
} catch (Throwable ex) {
    System.out.println("MyAspect2 @AfterThrowing通知!");
    //继续抛出异常
    throw ex;
}
System.out.println("Advisor1 end");
System.out.println("MyAspect1 @Around绕通知end");
return result;
} finally {
    System.out.println("MyAspect1 @After通知!");
}
System.out.println("MyAspect1 @AfterReturning通知!");
return retVal;
} catch (Throwable ex) {
    System.out.println("MyAspect1 @AfterThrowing通知!");
    //继续抛出异常
    throw ex;
}
}

```

@3 继续执行 `mi.proceed()`，此时会调用目标方法 `say("路人")`，然后就进化成下面这样了

```

try {
    Object result = null;
    try {
        System.out.println("MyAspect1 @Around通知start");
        System.out.println("MyAspect1 @Before通知!");
        System.out.println("Advisor1 start");
        try {
            try {
                System.out.println("MyAspect2 @Around通知start");
                System.out.println("MyAspect2 @Before通知!");
                result = "你好: 路人";
                System.out.println("MyAspect2 @Around绕通知end");
                return result;
            } finally {
                System.out.println("MyAspect2 @After通知!");
            }
            System.out.println("MyAspect2 @AfterReturning通知!");
        } catch (Throwable ex) {
            System.out.println("MyAspect2 @AfterThrowing通知!");
            //继续抛出异常
            throw ex;
        }
        System.out.println("Advisor1 end");
        System.out.println("MyAspect1 @Around绕通知end");
        return result;
    } finally {
        System.out.println("MyAspect1 @After通知!");
    }
}

```

```

        System.out.println("MyAspect1 @AfterReturning通知!");
        return retVal;
    } catch (Throwable ex) {
        System.out.println("MyAspect1 @AfterThrowing通知!");
        //继续抛出异常
        throw ex;
    }
}

```

再来和输出结果对比一下，是完全一致的。

```

MyAspect1 @Around通知start
MyAspect1 @Before通知!
Advisor1 start
MyAspect2 @Around通知start
MyAspect2 @Before通知!
MyAspect2 @Around绕通知end
MyAspect2 @After通知!
MyAspect2 @AfterReturning通知!
Advisor1 end
MyAspect1 @Around绕通知end
MyAspect1 @After通知!
MyAspect1 @AfterReturning通知!
你好：路人

```

9、@EnableAspectJAutoProxy另外2个功能

这个注解还有2个参数，大家看一下下面的注释，比较简单，就不用案例演示了。

```

public @interface EnableAspectJAutoProxy {

    /**
     * 是否基于类来创建代理，而不是基于接口来创建代理
     * 当为true的时候会使用cglib来直接对目标类创建代理对象
     * 默认为 false：即目标bean如果有接口的会采用jdk动态代理来创建代理对象，没有接口的目标
    bean，会采用cglib来创建代理对象
     */
    boolean proxyTargetClass() default false;

    /**
     * 是否需要将代理对象暴露在ThreadLocal中，当为true的时候
     * 可以通过org.springframework.aop.framework.AopContext#currentProxy获取当前代理
    对象
     */
    boolean exposeProxy() default false;
}

```

10、@EnableAspectJAutoProxy原理

@EnableAspectJAutoProxy 会在spring容器中注册一个bean


```
org.springframework.aop.aspectj.annotation.AnnotationAwareAspectJAutoProxyCreator
```

`AnnotationAwareAspectJAutoProxyCreator` 是 `BeanPostProcessor` 类型的，`BeanPostProcessor` 大家应该比较熟悉了，bean后置处理器，可以在bean声明周期中对bean进行操作，比如对bean生成代理等；而 `AnnotationAwareAspectJAutoProxyCreator` 就是对符合条件的bean，自动生成代理对象，源码就这里就不细说了，有兴趣的可以从 `postProcessAfterInitialization` 方法看，比较简单。

11、总结

今天内容还是挺多的，大家好好消化一下。

主要要掌握 `@EnableAspectJAutoProxy` 中多个 `@Aspect`、`Advisor` 时，通知的执行顺序，这个多看看，要理解其原理，记起来才会更容易，用起来也会更顺手。

如发现文章有错误、对内容有疑问，都可以在文章下面留言，或者加我微信（itsoku）交流，每周会挑选出一位热心小伙伴，送上一份精美的小礼品，快来关注我吧！

12、案例源码

<https://gitee.com/javacode2018/spring-series>

路人甲java所有案例代码以后都会放到这个上面，大家watch一下，可以持续关注动态。

13、Spring系列

1. [Spring系列第1篇：为何要学spring?](#)
2. [Spring系列第2篇：控制反转（IoC）与依赖注入（DI）](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)
4. [Spring系列第4篇：xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇：创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇：玩转bean scope，避免跳坑里！](#)
7. [Spring系列第7篇：依赖注入之手动注入](#)
8. [Spring系列第8篇：自动注入（autowire）详解，高手在于坚持](#)
9. [Spring系列第9篇：depend-on到底是干什么的?](#)
10. [Spring系列第10篇：primary可以解决什么问题?](#)
11. [Spring系列第11篇：bean中的autowire-candidate又是干什么的?](#)
12. [Spring系列第12篇：lazy-init：bean延迟初始化](#)
13. [Spring系列第13篇：使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇：lookup-method和replaced-method比较陌生，怎么玩的?](#)
15. [Spring系列第15篇：代理详解（Java动态代理&cglib代理）?](#)
16. [Spring系列第16篇：深入理解java注解及spring对注解的增强（预备知识）](#)
17. [Spring系列第17篇：@Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇：@ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇：@import详解\(bean批量注册\)](#)
20. [Spring系列第20篇：@Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇：注解实现依赖注入（@Autowired、@Resource、@Primary、@Qualifier）](#)
22. [Spring系列第22篇：@Scope、@DependsOn、@ImportResource、@Lazy 详解](#)

23. [Spring系列第23篇：Bean生命周期详解](#)
24. [Spring系列第24篇：父子容器详解](#)
25. [Spring系列第25篇：@Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇：国际化详解](#)
27. [Spring系列第27篇：spring事件机制详解](#)
28. [Spring系列第28篇：Bean循环依赖详解](#)
29. [Spring系列第29篇：BeanFactory扩展（BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor）](#)
30. [Spring系列第30篇：jdk动态代理和cglib代理](#)
31. [Spring系列第31篇：aop概念详解](#)
32. [Spring系列第32篇：AOP核心源码、原理详解](#)
33. [Spring系列第33篇：ProxyFactoryBean创建AOP代理](#)
34. [Spring系列第34篇：@Aspect中@Pointcut 12种用法](#)
35. [Spring系列第35篇：@Aspect中5中通知详解](#)
36. [Spring系列第36篇：@EnableAspectJAutoProxy、@Aspect中通知顺序详解](#)

14、更多好文章

1. [Java高并发系列（共34篇）](#)
2. [MySQL高手系列（共27篇）](#)
3. [Maven高手系列（共10篇）](#)
4. [Mybatis系列（共12篇）](#)
5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要，它是什么？怎么实现？](#)
7. [泛型，有点难度，会让很多人懵逼，那是因为你没有看这篇文章！](#)



领取福利，扫码发送java

- 1、1000本java电子书大全
- 2、2021版java面试通关手册
- 3、月薪5万必备的java微服务学习线路图及高清视频
- 4、邀请加入高质量java技术交流群

加我微信：spring888java，进群一起交流java相关各种技术！！

Spring系列第37篇：@EnableAsync & @Async 实现方法异步调用

1、本文内容

详解 @EnableAsync & @Async，主要分下面几个点进行介绍。

1. 作用
2. 用法

3. 获取异步执行结果
4. 自定义异步执行的线程池
5. 自定义异常处理
6. 线程隔离
7. 源码 & 原理

2、作用

spring容器中实现bean方法的异步调用。

比如有个logService的bean，logservice中有个log方法用来记录日志，当调用 `logService.log(msg)` 的时候，希望异步执行，那么可以通过 `@EnableAsync` & `@Async` 来实现。

3、用法

2步

1. 需要异步执行的方法上面使用 `@Async` 注解标注，若bean中所有的方法都需要异步执行，可以直接将 `@Async` 加载类上。
2. 将 `@EnableAsync` 添加在spring配置类上，此时 `@Async` 注解才会起效。

常见2种用法

1. 无返回值的
2. 可以获取返回值的

4、无返回值的

用法

方法返回值不是 `Future` 类型的，被执行时，会立即返回，并且无法获取方法返回值，如：

```
@Async
public void log(String msg) throws InterruptedException {
    System.out.println("开始记录日志," + System.currentTimeMillis());
    //模拟耗时2秒
    TimeUnit.SECONDS.sleep(2);
    System.out.println("日志记录完毕," + System.currentTimeMillis());
}
```

案例

实现日志异步记录的功能。

LogService.log方法用来异步记录日志，需要使用 `@Async` 标注

```
package com.javacode2018.async.demo1;

import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Component;
```

```

import java.util.concurrent.TimeUnit;

@Component
public class LogService {
    @Async
    public void log(String msg) throws InterruptedException {
        System.out.println(Thread.currentThread() + "开始记录日志," +
            System.currentTimeMillis());
        //模拟耗时2秒
        TimeUnit.SECONDS.sleep(2);
        System.out.println(Thread.currentThread() + "日志记录完毕," +
            System.currentTimeMillis());
    }
}

```

来个spring配置类，需要加上 `@EnableAsync` 开启bean方法的异步调用。

```

package com.javacode2018.async.demo1;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.EnableAspectJAutoProxy;
import org.springframework.scheduling.annotation.EnableAsync;

@ComponentScan
@EnableAsync
public class MainConfig1 {
}

```

测试代码

```

package com.javacode2018.async;

import com.javacode2018.async.demo1.LogService;
import com.javacode2018.async.demo1.MainConfig1;
import org.junit.Test;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

import java.util.concurrent.TimeUnit;

public class AsyncTest {

    @Test
    public void test1() throws InterruptedException {
        AnnotationConfigApplicationContext context = new
            AnnotationConfigApplicationContext();
        context.register(MainConfig1.class);
        context.refresh();
        LogService logService = context.getBean(LogService.class);
        System.out.println(Thread.currentThread() + " logService.log start," +
            System.currentTimeMillis());
        logService.log("异步执行方法!");
        System.out.println(Thread.currentThread() + " logService.log end," +
            System.currentTimeMillis());
    }
}

```

```
//休眠一下，防止@Test退出
    TimeUnit.SECONDS.sleep(3);
}

}
```

运行输出

```
Thread[main,5,main] logService.log start,1595223990417
Thread[main,5,main] logService.log end,1595223990432
Thread[SimpleAsyncTaskExecutor-1,5,main]开始记录日志,1595223990443
Thread[SimpleAsyncTaskExecutor-1,5,main]日志记录完毕,1595223992443
```

前2行输出，可以看出 `logService.log` 立即就返回了，后面2行来自于log方法，相差2秒左右。

前面2行在主线程中执行，后面2行在异步线程中执行。

5、获取异步返回值

用法

若需取异步执行结果，方法返回值必须为 `Future` 类型，使用spring提供的静态方法

`org.springframework.scheduling.annotation.AsyncResult#forValue` 创建返回值，如：

```
public Future<String> getGoodsInfo(long goodsId) throws InterruptedException {
    return AsyncResult.forValue(String.format("商品%s基本信息!", goodsId));
}
```

案例

场景：电商中商品详情页通常会有很多信息：商品基本信息、商品描述信息、商品评论信息，通过3个方法来或者这几个信息。

这3个方法之间无关联，所以可以采用异步的方式并行获取，提升效率。

下面是商品服务，内部3个方法都需要异步，所以直接在类上使用 `@Async` 标注了，每个方法内部休眠500毫秒，模拟一下耗时操作。

```
package com.javacode2018.async.demo2;

import org.springframework.scheduling.annotation.Async;
import org.springframework.scheduling.annotation.AsyncResult;
import org.springframework.stereotype.Component;

import java.util.Arrays;
import java.util.List;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;

@Async
@Component
public class GoodsService {
    //模拟获取商品基本信息，内部耗时500毫秒
```

```

    public Future<String> getGoodsInfo(long goodsId) throws InterruptedException
    {
        TimeUnit.MILLISECONDS.sleep(500);
        return AsyncResult.forValue(String.format("商品%s基本信息!", goodsId));
    }

    //模拟获取商品描述信息，内部耗时500毫秒
    public Future<String> getGoodsDesc(long goodsId) throws InterruptedException
    {
        TimeUnit.MILLISECONDS.sleep(500);
        return AsyncResult.forValue(String.format("商品%s描述信息!", goodsId));
    }

    //模拟获取商品评论信息列表，内部耗时500毫秒
    public Future<List<String>> getGoodsComments(long goodsId) throws
InterruptedException {
        TimeUnit.MILLISECONDS.sleep(500);
        List<String> comments = Arrays.asList("评论1", "评论2");
        return AsyncResult.forValue(comments);
    }
}

```

来个spring配置类，需要加上 @EnableAsync 开启bean方法的异步调用。

```

package com.javacode2018.async.demo2;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.scheduling.annotation.EnableAsync;

@ComponentScan
@EnableAsync
public class MainConfig2 {
}

```

测试代码

```

@Test
public void test2() throws InterruptedException, ExecutionException {
    AnnotationConfigApplicationContext context = new
AnnotationConfigApplicationContext();
    context.register(MainConfig2.class);
    context.refresh();
    GoodsService goodsService = context.getBean(GoodsService.class);

    long startTime = System.currentTimeMillis();
    System.out.println("开始获取商品的各種信息");

    long goodsId = 1L;
    Future<String> goodsInfoFuture = goodsService.getGoodsInfo(goodsId);
    Future<String> goodsDescFuture = goodsService.getGoodsDesc(goodsId);
    Future<List<String>> goodsCommentsFuture =
goodsService.getGoodsComments(goodsId);

    System.out.println(goodsInfoFuture.get());
    System.out.println(goodsDescFuture.get());
    System.out.println(goodsCommentsFuture.get());
}

```

```

        System.out.println("商品信息获取完毕,总耗时(ms): " + (System.currentTimeMillis()
- startTime));

        //休眠一下,防止@Test退出
        TimeUnit.SECONDS.sleep(3);
    }

```

运行输出

```

开始获取商品的各種信息
商品1基本信息!
商品1描述信息!
[评论1, 评论2]
商品信息获取完毕,总耗时(ms): 525

```

3个方法总计耗时500毫秒左右。

如果不采用异步的方式, 3个方法会同步执行, 耗时差不多1.5秒, 来试试, 将 `GoodsService` 上的 `@Async` 去掉, 然后再次执行测试案例, 输出

```

开始获取商品的各種信息
商品1基本信息!
商品1描述信息!
[评论1, 评论2]
商品信息获取完毕,总耗时(ms): 1503

```

这个案例大家可以借鉴一下, 按照这个思路可以去优化一下你们的代码, 方法之间无关联的可以采用异步的方式, 并行去获取, 最终耗时为最长的那个方法, 整体相对于同步的方式性能提升不少。

6、自定义异步执行的线程池

默认情况下, `@EnableAsync` 使用内置的线程池来异步调用方法, 不过我们也可以自定义异步执行任务的线程池。

有2种方式来自定义异步处理的线程池

方式1

在spring容器中定义一个线程池类型的bean, bean名称必须是taskExecutor

```

@Bean
public Executor taskExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(10);
    executor.setMaxPoolSize(100);
    executor.setThreadNamePrefix("my-thread-");
    return executor;
}

```

方式2

定义一个bean，实现 `AsyncConfigurer` 接口中的 `getAsyncExecutor` 方法，这个方法需要返回自定义的线程池，案例代码：

```
package com.javacode2018.async.demo3;

import com.javacode2018.async.demo1.LogService;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.context.annotation.Bean;
import org.springframework.lang.Nullable;
import org.springframework.scheduling.annotation.AsyncConfigurer;
import org.springframework.scheduling.annotation.EnableAsync;
import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;

import java.util.concurrent.Executor;

@EnableAsync
public class MainConfig3 {

    @Bean
    public LogService logService() {
        return new LogService();
    }

    /**
     * 定义一个AsyncConfigurer类型的bean，实现getAsyncExecutor方法，返回自定义的线程池
     *
     * @param executor
     * @return
     */
    @Bean
    public AsyncConfigurer asyncConfigurer(@Qualifier("logExecutors") Executor
executor) {
        return new AsyncConfigurer() {
            @Nullable
            @Override
            public Executor getAsyncExecutor() {
                return executor;
            }
        };
    }

    /**
     * 定义一个线程池，用来异步处理日志方法调用
     *
     * @return
     */
    @Bean
    public Executor logExecutors() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(10);
        executor.setMaxPoolSize(100);
        //线程名称前缀
        executor.setThreadNamePrefix("log-thread-"); // @1
        return executor;
    }
}
```



```
}
```

@1 自定义的线程池中线程名称前缀为 `log-thread-`，运行下面测试代码

```
@Test
public void test3() throws InterruptedException {
    AnnotationConfigApplicationContext context = new
AnnotationConfigApplicationContext();
    context.register(MainConfig3.class);
    context.refresh();
    LogService logService = context.getBean(LogService.class);
    System.out.println(Thread.currentThread() + " logService.log start," +
System.currentTimeMillis());
    logService.log("异步执行方法!");
    System.out.println(Thread.currentThread() + " logService.log end," +
System.currentTimeMillis());

    //休眠一下，防止@Test退出
    TimeUnit.SECONDS.sleep(3);
}
```

输出

```
Thread[main,5,main] logService.log start,1595228732914
Thread[main,5,main] logService.log end,1595228732921
Thread[log-thread-1,5,main] 开始记录日志,1595228732930
Thread[log-thread-1,5,main] 日志记录完毕,1595228734931
```

最后2行日志中线程名称是 `log-thread-`，正是我们自定义线程池中的线程。

7、自定义异常处理

异步方法若发生了异常，我们如何获取异常信息呢？此时可以通过自定义异常处理来解决。

异常处理分2种情况

1. 当返回值是Future的时候，方法内部有异常的时候，异常会向外抛出，可以对Future.get采用try..catch来捕获异常
2. 当返回值不是Future的时候，可以自定义一个bean，实现AsyncConfigurer接口中的getAsyncUncaughtExceptionHandler方法，返回自定义的异常处理器

情况1：返回值为Future类型

用法

通过try..catch来捕获异常，如下

```

try {
    Future<String> future = logService.mockException();
    System.out.println(future.get());
} catch (ExecutionException e) {
    System.out.println("捕获 ExecutionException 异常");
    //通过e.getCause获取实际的异常信息
    e.getCause().printStackTrace();
} catch (InterruptedException e) {
    e.printStackTrace();
}

```

案例

LogService中添加一个方法，返回值为Future，内部抛出一个异常，如下：

```

@Async
public Future<String> mockException() {
    //模拟抛出一个异常
    throw new IllegalArgumentException("参数有误!");
}

```

测试代码如下

```

@Test
public void test5() throws InterruptedException {
    AnnotationConfigApplicationContext context = new
    AnnotationConfigApplicationContext();
    context.register(MainConfig1.class);
    context.refresh();
    LogService logService = context.getBean(LogService.class);
    try {
        Future<String> future = logService.mockException();
        System.out.println(future.get());
    } catch (ExecutionException e) {
        System.out.println("捕获 ExecutionException 异常");
        //通过e.getCause获取实际的异常信息
        e.getCause().printStackTrace();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    //休眠一下，防止@Test退出
    TimeUnit.SECONDS.sleep(3);
}

```

运行输出

```

java.lang.IllegalArgumentException: 参数有误!
捕获 ExecutionException 异常
    at com.javacode2018.async.demo1.LogService.mockException(LogService.java:23)
    at
com.javacode2018.async.demo1.LogService$$FastClassBySpringCGLIB$$32a28430.invoke
(<generated>)
    at org.springframework.cglib.proxy.MethodProxy.invoke(MethodProxy.java:218)

```

情况2：无返回值异常处理

用法

当返回值不是Future的时候，可以自定义一个bean，实现 `AsyncConfigurer` 接口中的 `getAsyncUncaughtExceptionHandler` 方法，返回自定义的异常处理器，当目标方法执行过程中抛出异常的时候，此时会自动回调 `AsyncUncaughtExceptionHandler#handleUncaughtException` 这个方法，可以在这个方法中处理异常，如下：

```
@Bean
public AsyncConfigurer asyncConfigurer() {
    return new AsyncConfigurer() {
        @Nullable
        @Override
        public AsyncUncaughtExceptionHandler getAsyncUncaughtExceptionHandler()
        {
            return new AsyncUncaughtExceptionHandler() {
                @Override
                public void handleUncaughtException(Throwable ex, Method method,
                Object... params) {
                    //当目标方法执行过程中抛出异常的时候，此时会自动回调这个方法，可以在这个
                    方法中处理异常
                }
            };
        }
    };
}
```

案例

LogService中添加一个方法，内部抛出一个异常，如下：

```
@Async
public void mockNoReturnException() {
    //模拟抛出一个异常
    throw new IllegalArgumentException("无返回值的异常!");
}
```

来个spring配置类，通过 `AsyncConfigurer` 来自定义异常处理器 `AsyncUncaughtExceptionHandler`

```
package com.javacode2018.async.demo4;

import com.javacode2018.async.demo1.LogService;
import org.springframework.aop.interceptor.AsyncUncaughtExceptionHandler;
import org.springframework.context.annotation.Bean;
import org.springframework.lang.Nullable;
import org.springframework.scheduling.annotation.AsyncConfigurer;
import org.springframework.scheduling.annotation.EnableAsync;

import java.lang.reflect.Method;
import java.util.Arrays;

@EnableAsync
public class MainConfig4 {
```

```

@Bean
public LogService logService() {
    return new LogService();
}

@Bean
public AsyncConfigurer asyncConfigurer() {
    return new AsyncConfigurer() {
        @Nullable
        @Override
        public AsyncUncaughtExceptionHandler
getAsyncUncaughtExceptionHandler() {
            return new AsyncUncaughtExceptionHandler() {
                @Override
                public void handleUncaughtException(Throwable ex, Method
method, Object... params) {
                    String msg = String.format("方法[%s],参数[%s],发送异常了,异常
详细详细信息:", method, Arrays.asList(params));
                    System.out.println(msg);
                    ex.printStackTrace();
                }
            };
        }
    };
}
}
}

```

运行输出

```

方法[public void
com.javacode2018.async.demo1.LogService.mockNoReturnException()],参数[[]],发送异常
了,异常详细信息:
java.lang.IllegalArgumentException: 无返回值的异常!
    at
com.javacode2018.async.demo1.LogService.mockNoReturnException(LogService.java:29
)
    at
com.javacode2018.async.demo1.LogService$$FastClassBySpringCGLIB$$32a28430.invoke
(<generated>)
    at org.springframework.cglib.proxy.MethodProxy.invoke(MethodProxy.java:218)

```

8、线程池隔离

什么是线程池隔离？

一个系统中可能有很多业务，比如充值服务、提现服务或者其他服务，这些服务中都有一些方法需要异步执行，默认情况下他们会使用同一个线程池去执行，如果有一个业务量比较大，占用了线程池中的大量线程，此时会导致其他业务的方法无法执行，那么我们可以采用线程隔离的方式，对不同的业务使用不同的线程池，相互隔离，互不影响。

`@Async` 注解有个 `value` 参数，用来指定线程池的bean名称，方法运行的时候，就会采用指定的线程池来执行目标方法。

使用步骤

1. 在spring容器中，自定义线程池相关的bean
2. @Async("线程池bean名称")

案例

模拟2个业务：异步充值、异步提现；2个业务都采用独立的线程池来异步执行，互不影响。

异步充值服务

```
package com.javacode2018.async.demo5;

import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Component;

@Component
public class RechargeService {
    //模拟异步充值
    @Async(MainConfig5.RECHARGE_EXECUTORS_BEAN_NAME)
    public void recharge() {
        System.out.println(Thread.currentThread() + "模拟异步充值");
    }
}
```

异步提现服务

```
package com.javacode2018.async.demo5;

import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Component;

@Component
public class CashOutService {
    //模拟异步提现
    @Async(MainConfig5.CASHOUT_EXECUTORS_BEAN_NAME)
    public void cashOut() {
        System.out.println(Thread.currentThread() + "模拟异步提现");
    }
}
```

spring配置类

注意 @0、@1、@2、@3、@4 这几个地方的代码，采用线程池隔离的方式，注册了2个线程池，分别用来处理上面的2个异步业务。

```
package com.javacode2018.async.demo5;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.scheduling.annotation.EnableAsync;
import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;

import java.util.concurrent.Executor;

@EnableAsync //@0: 启用方法异步调用
```

```

@ComponentScan
public class MainConfig5 {

    //Q1: 值业务线程池bean名称
    public static final String RECHARGE_EXECUTORS_BEAN_NAME =
"rechargeExecutors";
    //Q2: 提现业务线程池bean名称
    public static final String CASHOUT_EXECUTORS_BEAN_NAME = "cashOutExecutors";

    /**
     * @3: 充值的线程池，线程名称以recharge-thread-开头
     * @return
     */
    @Bean(RECHARGE_EXECUTORS_BEAN_NAME)
    public Executor rechargeExecutors() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(10);
        executor.setMaxPoolSize(100);
        //线程名称前缀
        executor.setThreadNamePrefix("recharge-thread-");
        return executor;
    }

    /**
     * @4: 充值的线程池，线程名称以cashOut-thread-开头
     *
     * @return
     */
    @Bean(CASHOUT_EXECUTORS_BEAN_NAME)
    public Executor cashOutExecutors() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(10);
        executor.setMaxPoolSize(100);
        //线程名称前缀
        executor.setThreadNamePrefix("cashOut-thread-");
        return executor;
    }
}

```

测试代码

```

@Test
public void test7() throws InterruptedException {
    AnnotationConfigApplicationContext context = new
AnnotationConfigApplicationContext();
    context.register(MainConfig5.class);
    context.refresh();

    RechargeService rechargeService = context.getBean(RechargeService.class);
    rechargeService.recharge();
    CashOutService cashOutService = context.getBean(CashOutService.class);
    cashOutService.cashOut();

    //休眠一下，防止@Test退出
    TimeUnit.SECONDS.sleep(3);
}

```

运行输出

```
Thread[recharge-thread-1,5,main] 模拟异步充值  
Thread[cashOut-thread-1,5,main] 模拟异步提现
```

输出中可以看出2个业务使用的是不同的线程池执行的。

9、源码 & 原理

内部使用aop实现的，@EnableAsync会引入一个bean后置处理器：

`AsyncAnnotationBeanPostProcessor`，将其注册到spring容器，这个bean后置处理器在所有bean创建过程中，判断bean的类上是否有@Async注解或者类中是否有@Async标注的方法，如果有，会通过aop给这个bean生成代理对象，会在代理对象中添加一个切面：

`org.springframework.scheduling.annotation.AsyncAnnotationAdvisor`，这个切面中会引入一个拦截器：`AnnotationAsyncExecutionInterceptor`，方法异步调用的关键代码就是在这个拦截器的invoke方法中实现的，可以去看一下。

10、总结



11、案例源码

<https://gitee.com/javacode2018/spring-series>

路人甲java所有案例代码以后都会放到这个上面，大家watch一下，可以持续关注动态。

12、Spring系列

1. [Spring系列第1篇：为何要学spring?](#)
2. [Spring系列第2篇：控制反转（IoC）与依赖注入（DI）](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)
4. [Spring系列第4篇：xml中bean定义详解\(-\)](#)

5. [Spring系列第5篇：创建bean实例这些方式你们都知道？](#)
6. [Spring系列第6篇：玩转bean scope，避免跳坑里！](#)
7. [Spring系列第7篇：依赖注入之手动注入](#)
8. [Spring系列第8篇：自动注入（autowire）详解，高手在于坚持](#)
9. [Spring系列第9篇：depend-on到底是干什么的？](#)
10. [Spring系列第10篇：primary可以解决什么问题？](#)
11. [Spring系列第11篇：bean中的autowire-candidate又是干什么的？](#)
12. [Spring系列第12篇：lazy-init：bean延迟初始化](#)
13. [Spring系列第13篇：使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇：lookup-method和replaced-method比较陌生，怎么玩的？](#)
15. [Spring系列第15篇：代理详解（Java动态代理&cglib代理）？](#)
16. [Spring系列第16篇：深入理解java注解及spring对注解的增强（预备知识）](#)
17. [Spring系列第17篇：@Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇：@ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇：@import详解\(bean批量注册\)](#)
20. [Spring系列第20篇：@Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇：注解实现依赖注入（@Autowired、@Resource、@Primary、@Qualifier）](#)
22. [Spring系列第22篇：@Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇：Bean生命周期详解](#)
24. [Spring系列第24篇：父子容器详解](#)
25. [Spring系列第25篇：@Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇：国际化详解](#)
27. [Spring系列第27篇：spring事件机制详解](#)
28. [Spring系列第28篇：Bean循环依赖详解](#)
29. [Spring系列第29篇：BeanFactory扩展（BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor）](#)
30. [Spring系列第30篇：jdk动态代理和cglib代理](#)
31. [Spring系列第31篇：aop概念详解](#)
32. [Spring系列第32篇：AOP核心源码、原理详解](#)
33. [Spring系列第33篇：ProxyFactoryBean创建AOP代理](#)
34. [Spring系列第34篇：@Aspect中@Pointcut 12种用法](#)
35. [Spring系列第35篇：@Aspect中5中通知详解](#)
36. [Spring系列第36篇：@EnableAspectJAutoProxy、@Aspect中通知顺序详解](#)

13、更多好文章

1. [Java高并发系列（共34篇）](#)
2. [MySQL高手系列（共27篇）](#)
3. [Maven高手系列（共10篇）](#)
4. [Mybatis系列（共12篇）](#)
5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要，它是什么？怎么实现？](#)
7. [泛型，有点难度，会让很多人懵逼，那是因为你没有看这篇文章！](#)



领取福利，扫码发送java

- 1、1000本java电子书大全
- 2、2021版java面试通关手册
- 3、月薪5万必备的java微服务学习线路图及高清视频
- 4、邀请加入高质量java技术交流群

加我微信：spring888java，进群一起交流java相关各种技术！！

Spring系列第38篇：@Scheduled & @EnableScheduling定时器详解

spring中 @Scheduled & @EnableScheduling 这2个注解，可以用来快速开发定时器，使用特别的简单。

如何使用？

用法

- 1、需要定时执行的方法上加上@Scheduled注解，这个注解中可以指定定时执行的规则，稍后详细介绍。
- 2、Spring容器中使用@EnableScheduling开启定时任务的执行，此时spring容器才可以识别@Scheduled标注的方法，然后自动定时执行。

案例

db中有很多需要推送的任务，然后将其检索出来，推送到手机端，来个定时器，每秒一次从库中检测需要推送的消息，然后推送到手机端。

```
package com.javacode2018.scheduled.demo1;

import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

@Component
public class PushJob {

    //推送方法，每秒执行一次
    @Scheduled(fixedRate = 1000)
    public void push() throws InterruptedException {
        System.out.println("模拟推送消息，" + System.currentTimeMillis());
    }

}
```

来个spring配置类，需要使用 @EnableScheduling 标注

```

package com.javacode2018.scheduled.demo1;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.scheduling.annotation.EnableScheduling;

import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;

@ComponentScan
@EnableScheduling //在spring容器中启用定时任务的执行
public class MainConfig1 {

    @Bean
    public ScheduledExecutorService scheduledExecutorService() {
        return Executors.newScheduledThreadPool(20);
    }
}

```

测试类

```

package com.javacode2018.scheduled;

import com.javacode2018.scheduled.demo1.MainConfig1;
import org.junit.Test;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

import java.util.concurrent.TimeUnit;

public class ScheduledTest {
    @Test
    public void test1() throws InterruptedException {
        AnnotationConfigApplicationContext context = new
        AnnotationConfigApplicationContext();
        context.register(MainConfig1.class);
        context.refresh();
        //休眠一段时间，房子junit自动退出
        TimeUnit.SECONDS.sleep(10000);
    }
}

```

运行输出，每秒会输出一行，如下：

```

模拟推送消息，1595840822998
模拟推送消息，1595840823998
模拟推送消息，1595840824998
模拟推送消息，1595840825998
模拟推送消息，1595840826998
模拟推送消息，1595840827998
模拟推送消息，1595840828998

```

@Scheduled配置定时规则

@Scheduled可以用来配置定时器的执行规则，非常强大，@Scheduled中主要有8个参数，我们——来了解一下。

```
@Target({ElementType.METHOD, ElementType.ANNOTATION_TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Repeatable(Schedules.class)
public @interface Scheduled {

    String cron() default "";

    String zone() default "";

    long fixedDelay() default -1;

    String fixedDelayString() default "";

    long fixedRate() default -1;

    String fixedRateString() default "";

    long initialDelay() default -1;

    String initialDelayString() default "";

}
```

1. cron

该参数接收一个 cron表达式， cron表达式 是一个字符串，字符串以5或6个空格隔开，分开共6或7个域，每一个域代表一个含义。

cron表达式语法

```
[秒] [分] [小时] [日] [月] [周] [年]
```

注：[年]不是必须的域，可以省略[年]，则一共6个域

序号	说明	必填	允许填写的值	允许的通配符
1	秒	是	0-59	, - * /
2	分	是	0-59	, - * /
3	时	是	0-23	, - * /
4	日	是	1-31	, - * ? / L W
5	月	是	1-12 / JAN-DEC	, - * /
6	周	是	1-7 or SUN-SAT	, - * ? / L #
7	年	否	1970-2099	, - * /

通配符说明:

- `*` 表示所有值。例如:在分的字段上设置 `*`,表示每一分钟都会触发。
- `?` 表示不指定值。使用的场景为不需要关心当前设置这个字段的值。例如:要在每月的10号触发一个操作,但不关心是周几,所以需要周位置的那个字段设置为`?` 具体设置为 `0 0 0 10 * ?`
- `-` 表示区间。例如 在小时上设置 `"10-12"`,表示 10,11,12点都会触发。
- `,` 表示指定多个值,例如在周字段上设置 `"MON,WED,FRI"` 表示周一,周三和周五触发
- `/` 用于递增触发。如在秒上面设置`"5/15"` 表示从5秒开始,每增15秒触发(5,20,35,50)。在日字段上设置`'1/3'`所示每月1号开始,每隔三天触发一次。
- `L` 表示最后的意思。在日字段设置上,表示当月的最后一天(依据当前月份,如果是二月还会依据是否是闰年[leap]),在周字段上表示星期六,相当于`"7"`或`"SAT"`。如果在`"L"`前加上数字,则表示该数据的最后一个。例如在周字段上设置`"6L"`这样的格式,则表示`"本月最后一个星期五"`
- `W` 表示离指定日期的最近那个工作日(周一至周五)。例如在日字段上置`"15W"`,表示离每月15号最近的那个工作日触发。如果15号正好是周六,则找最近的周五(14号)触发,如果15号是周末,则找最近的下周一(16号)触发。如果15号正好在工作日(周一至周五),则就在该天触发。如果指定格式为`"1W"`,它则表示每月1号往后最近的工作日触发。如果1号正是周六,则将在3号下周一触发。(注, `"W"`前只能设置具体的数字,不允许区间`"-"`)。
- `#` 序号(表示每月的第几个周几),例如在周字段上设置`"6#3"`表示在每月的第三个周六。注意如果指定`"#5"`,正好第五周没有周六,则不会触发该配置(用在母亲节和父亲节再合适不过了);小提示: `'L'`和 `'W'`可以一组合使用。如果在日字段上设置`"LW"`,则表示在本月的最后一个工作日触发;周字段的设置,若使用英文字母是不区分大小写的,即`MON`与`mon`相同。

示例

每隔5秒执行一次: `* / 5 * * * * ?`

每隔1分钟执行一次: `0 * / 1 * * * * ?`

每天23点执行一次: `0 0 23 * * * ?`

每天凌晨1点执行一次: `0 0 1 * * * ?`

每月1号凌晨1点执行一次: `0 0 1 1 * * ?`

每月最后一天23点执行一次: `0 0 23 L * ?`

每周星期六凌晨1点实行一次: `0 0 1 ? * L`

在26分、29分、33分执行一次: `0 26,29,33 * * * ?`

每天的0点、13点、18点、21点都执行一次: `0 0 0,13,18,21 * * ?`

cron表达式使用占位符

另外, `cron` 属性接收的 `cron表达式` 支持占位符。

如: 配置文件:

```
time:
  cron: */5 * * * * *
  interval: 5
```

每5秒执行一次:

```

@Scheduled(cron="${time.cron}")
void testPlaceholder1() {
    System.out.println("Execute at " + System.currentTimeMillis());
}

@Scheduled(cron="*/${time.interval} * * * * *")
void testPlaceholder2() {
    System.out.println("Execute at " + System.currentTimeMillis());
}

```

2. zone

时区，接收一个 `java.util.TimeZone#ID`。`cron`表达式 会基于该时区解析。默认是一个空字符串，即取服务器所在地的时区。比如我们一般使用的时区 `Asia/Shanghai`。该字段我们一般留空。

3. fixedDelay

上一次执行完毕时间点之后多长时间再执行。

如：

```

@Scheduled(fixedDelay = 5000) //上一次执行完毕时间点之后5秒再执行

```

4. fixedDelayString

与 `3. fixedDelay` 意思相同，只是使用字符串的形式。唯一不同的是支持占位符。

如：

```

@Scheduled(fixedDelayString = "5000") //上一次执行完毕时间点之后5秒再执行

```

占位符的使用（配置文件中配置：time.fixedDelay=5000）

```

@Scheduled(fixedDelayString = "${time.fixedDelay}")
void testFixedDelayString() {
    System.out.println("Execute at " + System.currentTimeMillis());
}

```

5. fixedRate

上一次开始执行时间点之后多长时间再执行。

如：

```

@Scheduled(fixedRate = 5000) //上一次开始执行时间点之后5秒再执行

```

6. fixedRateString

与 `fixedRate` 意思相同，只是使用字符串的形式，唯一不同的是支持占位符。

7. initialDelay

第一次延迟多长时间后再执行。

如：

```
@Scheduled(initialDelay=1000, fixedRate=5000) //第一次延迟1秒后执行，之后按fixedRate
的规则每5秒执行一次
```

8. initialDelayString

与 `initialDelay` 意思相同，只是使用字符串的形式，唯一不同的是支持占位符。

@Schedules注解

这个注解不用多解释，看一下源码就知道作用了，当一个方法上面需要同时指定多个定时规则的时候，可以通过这个来配置

```
@Target({ElementType.METHOD, ElementType.ANNOTATION_TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface Schedules {
    Scheduled[] value();
}
```

如：

```
//2个定时器，500毫秒的，1000毫秒的
@Schedules({@Scheduled(fixedRate = 500), @Scheduled(fixedRate = 1000)})
public void push3() {
}
```

为定时器定义线程池

定时器默认情况下使用下面的线程池来执行定时任务的

```
new ScheduledThreadPoolExecutor(1)
```

只有一个线程，相当于只有一个干活的人，如果需要定时执行的任务太多，这些任务只能排队执行，会出现什么问题？

如果有些任务耗时比较长，导致其他任务排队时间比较长，不能有效的正常执行，直接影响到业务。

看下面代码，2个方法，都使用了 `@Scheduled(fixedRate = 1000)`，表示每秒执行一次，而 `push1` 方法中模拟耗时2秒，方法会中打印出线程名称、时间等信息，一会注意观察输出

```
package com.javacode2018.scheduled.demo2;

import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

import java.util.concurrent.TimeUnit;
```

```

@Component
public class PushJob {

    //推送方法，每秒执行一次
    @Scheduled(fixedRate = 1000)
    public void push1() throws InterruptedException {
        //休眠2秒，模拟耗时操作
        TimeUnit.SECONDS.sleep(2);
        System.out.println(Thread.currentThread().getName() + " push1 模拟推送消息, " + System.currentTimeMillis());
    }

    //推送方法，每秒执行一次
    @Scheduled(fixedRate = 1000)
    public void push2() {
        System.out.println(Thread.currentThread().getName() + " push2 模拟推送消息, " + System.currentTimeMillis());
    }

}

```

运行输出

```

pool-1-thread-1 push1 模拟推送消息, 1595902615507
pool-1-thread-1 push2 模拟推送消息, 1595902615507
pool-1-thread-1 push1 模拟推送消息, 1595902617507
pool-1-thread-1 push2 模拟推送消息, 1595902617507
pool-1-thread-1 push1 模拟推送消息, 1595902619508
pool-1-thread-1 push2 模拟推送消息, 1595902619508

```

注意上面的输出，线程名称都是 `pool-1-thread-1`，并且有个问题，`push2` 中2次输出时间间隔是2秒，这就是由于线程池中只有一个线程导致了排队执行而产生的问题。

可以通过自定义定时器中的线程池来解决这个问题，定义一个 `ScheduledExecutorService` 类型的 bean，名称为 `taskScheduler`

```

@Bean
public ScheduledExecutorService taskScheduler() {
    //设置需要并行执行的任务数量
    int corePoolSize = 20;
    return new ScheduledThreadPoolExecutor(corePoolSize);
}

```

此时问题就解决了，再次运行一下上面案例代码，结果如下，此时线程名称不一样了，且 `push2` 运行正常了

```

pool-1-thread-2 push2 模拟推送消息, 1595903154636
pool-1-thread-2 push2 模拟推送消息, 1595903155636
pool-1-thread-1 push1 模拟推送消息, 1595903156636
pool-1-thread-3 push2 模拟推送消息, 1595903156636
pool-1-thread-1 push2 模拟推送消息, 1595903157636

```

源码 & 原理

从 `EnableScheduling` 注解开始看，这个注解会导入 `SchedulingConfiguration` 类

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Import(SchedulingConfiguration.class)
@Documented
public @interface EnableScheduling {
}
```

`SchedulingConfiguration` 是一个配置类，内部定义了 `ScheduledAnnotationBeanPostProcessor` 类型的bean

```
@Configuration
@Role(BeanDefinition.ROLE_INFRASTRUCTURE)
public class SchedulingConfiguration {

    @Bean(name = TaskManagementConfigUtils.SCHEDULED_ANNOTATION_PROCESSOR_BEAN_NAME)
    @Role(BeanDefinition.ROLE_INFRASTRUCTURE)
    public ScheduledAnnotationBeanPostProcessor scheduledAnnotationProcessor() {
        return new ScheduledAnnotationBeanPostProcessor();
    }
}
```

`ScheduledAnnotationBeanPostProcessor` 是一个bean后置处理器，内部有个 `postProcessAfterInitialization` 方法，spring中任何bean在初始化完毕之后，会自动调用 `postProcessAfterInitialization` 方法，而 `ScheduledAnnotationBeanPostProcessor` 在这个方法中会解析bean中标注有 `@Scheduled` 注解的方法，这些方法也就是需要定时执行的方法。

`ScheduledAnnotationBeanPostProcessor` 还实现了一个接口：`SmartInitializingSingleton`，`SmartInitializingSingleton` 中有个方法 `afterSingletonsInstantiated` 会在spring容器中所有单例bean初始化完毕之后调用，定时器的装配及启动都是在这个方法中进行的。

```
org.springframework.scheduling.annotation.ScheduledAnnotationBeanPostProcessor#afterSingletonsInstantiated
```

案例源码

<https://gitee.com/javacode2018/spring-series>

路人甲java所有案例代码以后都会放到这个上面，大家watch一下，可以持续关注动态。

Spring系列

1. [Spring系列第1篇：为何要学spring?](#)
2. [Spring系列第2篇：控制反转（IoC）与依赖注入（DI）](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)
4. [Spring系列第4篇：xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇：创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇：玩转bean scope，避免跳坑里!](#)
7. [Spring系列第7篇：依赖注入之手动注入](#)
8. [Spring系列第8篇：自动注入（autowire）详解，高手在于坚持](#)
9. [Spring系列第9篇：depend-on到底是干什么的?](#)
10. [Spring系列第10篇：primary可以解决什么问题?](#)
11. [Spring系列第11篇：bean中的autowire-candidate又是干什么的?](#)
12. [Spring系列第12篇：lazy-init：bean延迟初始化](#)
13. [Spring系列第13篇：使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇：lookup-method和replaced-method比较陌生，怎么玩的?](#)
15. [Spring系列第15篇：代理详解（Java动态代理&cglib代理）?](#)
16. [Spring系列第16篇：深入理解java注解及spring对注解的增强（预备知识）](#)
17. [Spring系列第17篇：@Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇：@ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇：@import详解\(bean批量注册\)](#)
20. [Spring系列第20篇：@Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇：注解实现依赖注入（@Autowired、@Resource、@Primary、@Qualifier）](#)
22. [Spring系列第22篇：@Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇：Bean生命周期详解](#)
24. [Spring系列第24篇：父子容器详解](#)
25. [Spring系列第25篇：@Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇：国际化详解](#)
27. [Spring系列第27篇：spring事件机制详解](#)
28. [Spring系列第28篇：Bean循环依赖详解](#)
29. [Spring系列第29篇：BeanFactory扩展（BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor）](#)
30. [Spring系列第30篇：jdk动态代理和cglib代理](#)
31. [Spring系列第31篇：aop概念详解](#)
32. [Spring系列第32篇：AOP核心源码、原理详解](#)
33. [Spring系列第33篇：ProxyFactoryBean创建AOP代理](#)
34. [Spring系列第34篇：@Aspect中@Pointcut 12种用法](#)
35. [Spring系列第35篇：@Aspect中5中通知详解](#)
36. [Spring系列第36篇：@EnableAspectJAutoProxy、@Aspect中通知顺序详解](#)
37. [Spring系列第37篇：@EnableAsync & @Async 实现方法异步调用](#)

更多好文章

1. [Java高并发系列（共34篇）](#)
2. [MySql高手系列（共27篇）](#)
3. [Maven高手系列（共10篇）](#)
4. [Mybatis系列（共12篇）](#)
5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要，它是什么？怎么实现？](#)
7. [泛型，有点难度，会让很多人懵逼，那是因为你没有看这篇文章!](#)



领取福利，扫码发送java

- 1、1000本java电子书大全
- 2、2021版java面试通关手册
- 3、月薪5万必备的java微服务学习线路图及高清视频
- 4、邀请加入高质量java技术交流群

加我微信：spring888java，进群一起交流java相关各种技术！！

Spring系列第39篇：强大的Spel表达式

本文带你玩转spring中强大的spel表达式！

Spel概述

Spring表达式语言全称为“Spring Expression Language”，缩写为“SpEL”，类似于Struts2x中使用的OGNL表达式语言，能在运行时构建复杂表达式、存取对象图属性、对象方法调用等等，并且能与Spring功能完美整合，如能用来配置Bean定义。

表达式语言给静态Java语言增加了动态功能。

SpEL是单独模块，只依赖于core模块，不依赖于其他模块，可以单独使用。

Spel能干什么？

表达式语言一般是用最简单的形式完成最主要的工作，减少我们的工作量。

SpEL支持如下表达式：

一、基本表达式： 字面量表达式、关系，逻辑与算数运算表达式、字符串连接及截取表达式、三目运算及Elivis表达式、正则表达式、括号优先级表达式；

二、类相关表达式： 类类型表达式、类实例化、instanceof表达式、变量定义及引用、赋值表达式、自定义函数、对象属性存取及安全导航表达式、对象方法调用、Bean引用；

三、集合相关表达式： 内联List、内联数组、集合，字典访问、列表，字典，数组修改、集合投影、集合选择；不支持多维内联数组初始化；不支持内联字典定义；

四、其他表达式： 模板表达式。

注： SpEL表达式中的关键字是不区分大小写的。

SpEL基础

HelloWorld

首先准备支持SpEL的Jar包：“org.springframework.expression-3.0.5.RELEASE.jar”将其添加到类路径中。

SpEL在求表达式值时一般分为四步，其中第三步可选：首先构造一个解析器，其次解析器解析字符串表达式，在此构造上下文，最后根据上下文得到表达式运算后的值。

让我们看下代码片段吧：

```
package com.javacode2018.spel;

import org.junit.Test;
import org.springframework.expression.EvaluationContext;
import org.springframework.expression.Expression;
import org.springframework.expression.ExpressionParser;
import org.springframework.expression.spel.standard.SpelExpressionParser;
import org.springframework.expression.spel.support.StandardEvaluationContext;

public class SpelTest {
    @Test
    public void test1() {
        ExpressionParser parser = new SpelExpressionParser();
        Expression expression = parser.parseExpression("'Hello' + 'World').concat(#end)");
        EvaluationContext context = new StandardEvaluationContext();
        context.setVariable("end", "!");
        System.out.println(expression.getValue(context));
    }
}
```

输出

```
Hello world!
```

接下来让我们分析下代码：

- 1) 创建解析器：**SpEL使用ExpressionParser接口表示解析器，提供SpelExpressionParser默认实现；**
- 2) 解析表达式：使用ExpressionParser的parseExpression来解析相应的表达式为Expression对象。
- 3) 构造上下文：准备比如变量定义等等表达式需要的上下文数据。
- 4) 求值：通过Expression接口的getValue方法根据上下文获得表达式值。

是不是很简单，接下来让我们看下其具体实现及原理吧。

SpEL原理及接口

SpEL提供简单的接口从而简化用户使用，在介绍原理前让我们学习下几个概念：

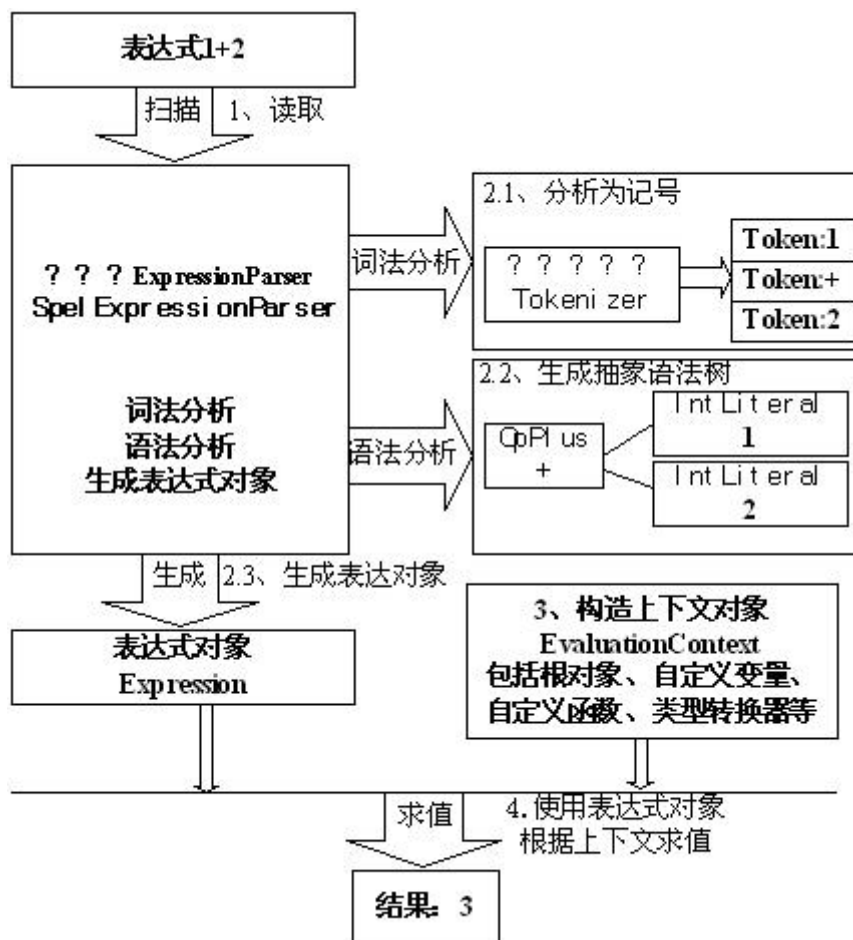
一、表达式： 表达式是表达式语言的核心，所以表达式语言都是围绕表达式进行的，从我们角度来看是“干什么”；

二、解析器： 用于将字符串表达式解析为表达式对象，从我们角度来看是“谁来干”；

三、上下文：表达式对象执行的环境，该环境可能定义变量、定义自定义函数、提供类型转换等等，从我们角度看是“在哪干”；

四、根对象及活动上下文对象：根对象是默认的活动上下文对象，活动上下文对象表示了当前表达式操作的对象，从我们角度看是“对谁干”。

理解了这些概念后，让我们看下SpEL如何工作的呢，如图5-1所示：



工作原理

1. 首先定义表达式：“1+2”；
2. 定义解析器ExpressionParser实现，SpEL提供默认实现SpELExpressionParser：
 - 2.1. SpELExpressionParser解析器内部使用Tokenizer类进行词法分析，即把字符串流分析为记号流，记号在SpEL使用Token类来表示；
 - 2.2. 有了记号流后，解析器便可根据记号流生成内部抽象语法树；在SpEL中语法树节点由SpELNode接口实现代表：如OpPlus表示加操作节点、IntLiteral表示int型字面量节点；使用SpELNode1实现组成了抽象语法树；
 - 2.3. 对外提供Expression接口来简化表示抽象语法树，从而隐藏内部实现细节，并提供getValue简单方法用于获取表达式值；SpEL提供默认实现为SpELExpression；
3. 定义表达式上下文对象（可选），SpEL使用EvaluationContext接口表示上下文对象，用于设置根对象、自定义变量、自定义函数、类型转换器等，SpEL提供默认实现StandardEvaluationContext；
4. 使用表达式对象根据上下文对象（可选）求值（调用表达式对象的getValue方法）获得结果。

接下来让我们看下SpEL的主要接口吧：

ExpressionParser接口

表示解析器，默认实现是org.springframework.expression.spel.standard包中的SpelExpressionParser类，使用parseExpression方法将字符串表达式转换为Expression对象，对于ParserContext接口用于定义字符串表达式是不是模板，及模板开始与结束字符：

```
public interface ExpressionParser {
    Expression parseExpression(String expressionString) throws ParseException;
    Expression parseExpression(String expressionString, ParserContext context)
    throws ParseException;
}
```

来看下示例：

```
@Test
public void testParserContext() {
    ExpressionParser parser = new SpelExpressionParser();
    ParserContext parserContext = new ParserContext() {
        @Override
        public boolean isTemplate() {
            return true;
        }

        @Override
        public String getExpressionPrefix() {
            return "#{";
        }

        @Override
        public String getExpressionSuffix() {
            return "}";
        }
    };
    String template = "#{ 'Hello ' }#{ 'World!' }";
    Expression expression = parser.parseExpression(template, parserContext);
    System.out.println(expression.getValue());
}
```

在此我们演示的是使用ParserContext的情况，此处定义了ParserContext实现：定义表达式是模块，表达式前缀为“#{”，后缀为“}”；使用parseExpression解析时传入的模板必须以“#{”开头，以“}”结尾，如“#{ 'Hello ' }#{ 'World!' }”。

默认传入的字符串表达式不是模板形式，如之前演示的Hello World。

EvaluationContext接口

表示上下文环境，默认实现是org.springframework.expression.spel.support包中的StandardEvaluationContext类，使用setRootObject方法来设置根对象，使用setVariable方法来注册自定义变量，使用registerFunction来注册自定义函数等等。

Expression接口

表示表达式对象，默认实现是org.springframework.expression.spel.standard包中的SpelExpression，提供getValue方法用于获取表达式值，提供setValue方法用于设置对象值。

了解了SpEL原理及接口，接下来的事情就是SpEL语法了。

SpEL语法

基本表达式

字面量表达式

SpEL支持的字面量包括：字符串、数字类型（int、long、float、double）、布尔类型、null类型。

类型	示例
字符串	<code>String str1 = parser.parseExpression("'Hello World!').getValue(String.class);</code>
数字类型	<code>int int1 = parser.parseExpression("1").getValue(Integer.class); long long1 = parser.parseExpression("-1L").getValue(long.class); float float1 = parser.parseExpression("1.1").getValue(Float.class); double double1 = parser.parseExpression("1.1E+2").getValue(double.class); int hex1 = parser.parseExpression("0xa").getValue(Integer.class); long hex2 = parser.parseExpression("0xaL").getValue(long.class);</code>
布尔类型	<code>boolean true1 = parser.parseExpression("true").getValue(boolean.class); boolean false1 = parser.parseExpression("false").getValue(boolean.class);</code>
null类型	<code>Object null1 = parser.parseExpression("null").getValue(Object.class);</code>

```
@Test
public void test2() {
    ExpressionParser parser = new SpelExpressionParser();

    String str1 = parser.parseExpression("'Hello
World!']").getValue(String.class);
    int int1 = parser.parseExpression("1").getValue(Integer.class);
    long long1 = parser.parseExpression("-1L").getValue(long.class);
    float float1 = parser.parseExpression("1.1").getValue(Float.class);
    double double1 = parser.parseExpression("1.1E+2").getValue(double.class);
    int hex1 = parser.parseExpression("0xa").getValue(Integer.class);
    long hex2 = parser.parseExpression("0xaL").getValue(long.class);
    boolean true1 = parser.parseExpression("true").getValue(boolean.class);
    boolean false1 = parser.parseExpression("false").getValue(boolean.class);
    Object null1 = parser.parseExpression("null").getValue(Object.class);

    System.out.println("str1=" + str1);
    System.out.println("int1=" + int1);
    System.out.println("long1=" + long1);
    System.out.println("float1=" + float1);
    System.out.println("double1=" + double1);
    System.out.println("hex1=" + hex1);
    System.out.println("hex2=" + hex2);
    System.out.println("true1=" + true1);
    System.out.println("false1=" + false1);
    System.out.println("null1=" + null1);
}
```

输出

```
str1=Hello world!
int1=1
long1=-1
float1=1.1
double1=110.0
hex1=10
hex2=10
true1=true
false1=false
null1=null
```

算数运算表达式

SpEL支持加(+)、减(-)、乘(*)、除(/)、求余 (%)、幂 (^) 运算。

类型	示例
加减乘除	<code>int result1 = parser.parseExpression("1+2-3*4/2").getValue(Integer.class); //-3</code>
求余	<code>int result2 = parser.parseExpression("4%3").getValue(Integer.class); //1</code>
幂运算	<code>int result3 = parser.parseExpression("2^3").getValue(Integer.class); //8</code>

SpEL还提供求余 (MOD) 和除 (DIV) 而外两个运算符，与“%”和“/”等价，不区分大小写。

关系表达式

等于 (==)、不等于 (!=)、大于(>)、大于等于(>=)、小于(<)、小于等于(<=)，区间 (between) 运算。

如 `parser.parseExpression("1>2").getValue(boolean.class);` 将返回false;

而 `parser.parseExpression("1 between {1, 2}").getValue(boolean.class);` 将返回true。

between运算符右边操作数必须是列表类型，且只能包含2个元素。第一个元素为开始，第二个元素为结束，区间运算是包含边界值的，即 `xxx>=list.get(0) && xxx<=list.get(1)`。

SpEL同样提供了等价的“EQ”、“NE”、“GT”、“GE”、“LT”、“LE”来表示等于、不等于、大于、大于等于、小于、小于等于，不区分大小写。

```
@Test
public void test3() {
    ExpressionParser parser = new SpelExpressionParser();
    boolean v1 = parser.parseExpression("1>2").getValue(boolean.class);
    boolean between1 = parser.parseExpression("1 between
{1,2}").getValue(boolean.class);
    System.out.println("v1=" + v1);
    System.out.println("between1=" + between1);
}
```

输出

```
v1=false
between1=true
```


逻辑表达式

且 (and或者&&)、或(or或者||)、非(!或NOT)。

```
@Test
public void test4() {
    ExpressionParser parser = new SpelExpressionParser();

    boolean result1 = parser.parseExpression("2>1 and (!true or !false)").getValue(boolean.class);
    boolean result2 = parser.parseExpression("2>1 && (!true || !false)").getValue(boolean.class);

    boolean result3 = parser.parseExpression("2>1 and (NOT true or NOT false)").getValue(boolean.class);
    boolean result4 = parser.parseExpression("2>1 && (NOT true || NOT false)").getValue(boolean.class);

    System.out.println("result1=" + result1);
    System.out.println("result2=" + result2);
    System.out.println("result3=" + result3);
    System.out.println("result4=" + result4);
}
```

输出

```
result1=true
result2=true
result3=true
result4=false
```

字符串连接及截取表达式

使用“+”进行字符串连接，使用“String'[0] [index]”来截取一个字符，目前只支持截取一个，如“Hello ' + 'World!'”得到“Hello World!”；而“Hello World![0]”将返回“H”。

三目运算

三目运算符“**表达式1?表达式2:表达式3**”用于构造三目运算表达式，如“2>1?true:false”将返回true；

Elivis运算符

Elivis运算符“**表达式1?:表达式2**”从Groovy语言引入用于简化三目运算符的，当表达式1为非null时则返回表达式1，当表达式1为null时则返回表达式2，简化了三目运算符方式“表达式1? 表达式1:表达式2”，如“null?:false”将返回false，而“true?:false”将返回true；

正则表达式

使用“str matches regex，如“'123' matches '\d{3}’”将返回true；

括号优先级表达式

使用“(表达式)”构造，括号里的具有高优先级。

类相关表达式

类类型表达式

使用“T(Type)”来表示java.lang.Class实例，“Type”必须是类全限定名，“java.lang”包除外，即该包下的类可以不指定包名；使用类类型表达式还可以进行访问类静态方法及类静态字段。

具体使用方法如下：

```
@Test
public void testClassTypeExpression() {
    ExpressionParser parser = new SpelExpressionParser();
    //java.lang包类访问
    Class<String> result1 =
    parser.parseExpression("T(String)").getValue(Class.class);
    System.out.println(result1);

    //其他包类访问
    String expression2 = "T(com.javacode2018.spel.SpelTest)";
    Class<SpelTest> value =
    parser.parseExpression(expression2).getValue(Class.class);
    System.out.println(value == SpelTest.class);

    //类静态字段访问
    int result3 =
    parser.parseExpression("T(Integer).MAX_VALUE").getValue(int.class);
    System.out.println(result3 == Integer.MAX_VALUE);

    //类静态方法调用
    int result4 =
    parser.parseExpression("T(Integer).parseInt('1')").getValue(int.class);
    System.out.println(result4);
}
```

输出

```
class java.lang.String
true
true
1
```

对于java.lang包里的可以直接使用“T(String)”访问；其他包必须是类全限定名；可以进行静态字段访问如“T(Integer).MAX_VALUE”；也可以进行静态方法访问如“T(Integer).parseInt('1')”。

类实例化

类实例化同样使用java关键字“new”，类名必须是全限定名，但java.lang包内的类型除外，如String、Integer。

```

@Test
public void testConstructorExpression() {
    ExpressionParser parser = new SpELExpressionParser();
    String result1 = parser.parseExpression("new String('路人甲java')").getValue(String.class);
    System.out.println(result1);

    Date result2 = parser.parseExpression("new java.util.Date()").getValue(Date.class);
    System.out.println(result2);
}

```

实例化完全跟Java内方式一样，运行输出

```

路人甲java
Tue Aug 03 20:22:43 CST 2020

```

instanceof表达式

SpEL支持instanceof运算符，跟Java内使用同义；如“'haha' instanceof T(String)”将返回true。

```

@Test
public void testInstanceOfExpression() {
    ExpressionParser parser = new SpELExpressionParser();
    Boolean value = parser.parseExpression("'路人甲' instanceof T(String)").getValue(Boolean.class);
    System.out.println(value);
}

```

输出

```
true
```

变量定义及引用

变量定义通过EvaluationContext接口的setVariable(variableName, value)方法定义；在表达式中使用"#variableName"引用；除了引用自定义变量，SpE还允许引用根对象及当前上下文对象，使用"#root"引用根对象，使用"#this"引用当前上下文对象；

```

@Test
public void testVariableExpression() {
    ExpressionParser parser = new SpELExpressionParser();
    EvaluationContext context = new StandardEvaluationContext();
    context.setVariable("name", "路人甲java");
    context.setVariable("lesson", "Spring系列");

    //获取name变量, lesson变量
    String name = parser.parseExpression("#name").getValue(context, String.class);
    System.out.println(name);
    String lesson = parser.parseExpression("#lesson").getValue(context, String.class);
}

```

```

System.out.println(lesson);

//StandardEvaluationContext构造器传入root对象，可以通过#root来访问root对象
context = new StandardEvaluationContext("我是root对象");
String rootObj = parser.parseExpression("#root").getValue(context,
String.class);
System.out.println(rootObj);

//#this用来访问当前上线文中的对象
String thisObj = parser.parseExpression("#this").getValue(context,
String.class);
System.out.println(thisObj);
}

```

输出

```

路人甲java
Spring系列
我是root对象
我是root对象

```

使用“#variable”来引用在EvaluationContext定义的变量；除了可以引用自定义变量，还可以使用“#root”引用根对象，“#this”引用当前上下文对象，此处“#this”即根对象。

自定义函数

目前只支持类静态方法注册为自定义函数；SpEL使用StandardEvaluationContext的registerFunction方法进行注册自定义函数，其实完全可以使用setVariable代替，两者其实本质是一样的；

```

@Test
public void testFunctionExpression() throws SecurityException,
NoSuchMethodException {
    //定义2个函数,registerFunction和setVariable都可以，不过从语义上面来看用
    registerFunction更恰当
    StandardEvaluationContext context = new StandardEvaluationContext();
    Method parseInt = Integer.class.getDeclaredMethod("parseInt", String.class);
    context.registerFunction("parseInt1", parseInt);
    context.setVariable("parseInt2", parseInt);

    ExpressionParser parser = new SpelExpressionParser();

    System.out.println(parser.parseExpression("#parseInt1('3')").getValue(context,
int.class));

    System.out.println(parser.parseExpression("#parseInt2('3')").getValue(context,
int.class));

    String expression1 = "#parseInt1('3') == #parseInt2('3')";
    boolean result1 = parser.parseExpression(expression1).getValue(context,
boolean.class);
    System.out.println(result1);
}

```

此处可以看出“registerFunction”和“setVariable”都可以注册自定义函数，但是两个方法的含义不一样，推荐使用“registerFunction”方法注册自定义函数。

运行输出

```
3
3
true
```

表达式赋值

使用 `Expression#setValue` 方法可以给表达式赋值

```
@Test
public void testAssignExpression1() {
    Object user = new Object() {
        private String name;

        public String getName() {
            return name;
        }

        public void setName(String name) {
            this.name = name;
        }

        @Override
        public String toString() {
            return "$classname{" +
                "name='" + name + '\'' +
                '}';
        }
    };
    {
        //user为root对象
        ExpressionParser parser = new SpelExpressionParser();
        EvaluationContext context = new StandardEvaluationContext(user);
        parser.parseExpression("#root.name").setValue(context, "路人甲java");
        System.out.println(parser.parseExpression("#root").getValue(context,
user.getClass()));
    }
    {
        //user为变量
        ExpressionParser parser = new SpelExpressionParser();
        EvaluationContext context = new StandardEvaluationContext();
        context.setVariable("user", user);
        parser.parseExpression("#user.name").setValue(context, "路人甲java");
        System.out.println(parser.parseExpression("#user").getValue(context,
user.getClass()));
    }
}
```

运行输出

```
$classname{name='路人甲java'}
$classname{name='路人甲java'}
```

对象属性存取及安全导航表达式

对象属性获取非常简单，即使用如“a.property.property”这种点缀式获取，SpEL对于属性名首字母是不区分大小写的；SpEL还引入了Groovy语言中的安全导航运算符“(对象|属性)?.属性”，用来避免“?.”前边的表达式为null时抛出空指针异常，而是返回null；修改对象属性值则可以通过赋值表达式或Expression接口的setValue方法修改。

```
public static class Car {
    private String name;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    @Override
    public String toString() {
        return "Car{" +
            "name='" + name + '\'' +
            '}';
    }
}

public static class User {
    private Car car;

    public Car getCar() {
        return car;
    }

    public void setCar(Car car) {
        this.car = car;
    }

    @Override
    public String toString() {
        return "User{" +
            "car=" + car +
            '}';
    }
}

@Test
public void test5() {
    User user = new User();
    EvaluationContext context = new StandardEvaluationContext();
    context.setVariable("user", user);

    ExpressionParser parser = new SpelExpressionParser();
    //使用.符号，访问user.car.name会报错，原因：user.car为空
    try {

        System.out.println(parser.parseExpression("#user.car.name").getValue(context,
            String.class));
    }
```

```

    } catch (EvaluationException | ParseException e) {
        System.out.println("出错了: " + e.getMessage());
    }
    //使用安全访问符号?.，可以规避null错误

    System.out.println(parser.parseExpression("#user?.car?.name").getValue(context,
String.class));

    Car car = new Car();
    car.setName("保时捷");
    user.setCar(car);

    System.out.println(parser.parseExpression("#user?.car?.toString()").getValue(context,
String.class));
}

```

运行输出

```

出错了: EL1007E: Property or field 'name' cannot be found on null
null
Car{name='保时捷'}

```

对象方法调用

对象方法调用更简单，跟Java语法一样；如“haha”.substring(2,4)将返回“ha”；而对于根对象可以直接调用方法；

Bean引用

SpEL支持使用“@”符号来引用Bean，在引用Bean时需要使用BeanResolver接口实现来查找Bean，Spring提供BeanFactoryResolver实现。

```

@Test
public void test6() {
    DefaultListableBeanFactory factory = new DefaultListableBeanFactory();
    User user = new User();
    Car car = new Car();
    car.setName("保时捷");
    user.setCar(car);
    factory.registerSingleton("user", user);

    StandardEvaluationContext context = new StandardEvaluationContext();
    context.setBeanResolver(new BeanFactoryResolver(factory));

    ExpressionParser parser = new SpELExpressionParser();
    User userBean = parser.parseExpression("@user").getValue(context,
User.class);
    System.out.println(userBean);
    System.out.println(userBean == factory.getBean("user"));
}

```

运行输出

```
User{car=Car{name='保时捷'}}
true
```

集合相关表达式

内联List

从Spring3.0.4开始支持内联List，使用{表达式,}定义内联List，如“{1,2,3}”将返回一个整型的ArrayList，而“{}”将返回空的List，对于字面量表达式列表，SpEL会使用java.util.Collections.unmodifiableList方法将列表设置为不可修改。

```
@Test
public void test7() {
    ExpressionParser parser = new SpelExpressionParser();
    //将返回不可修改的空List
    List<Integer> result2 = parser.parseExpression("{}").getValue(List.class);
    //对于字面量列表也将返回不可修改的List
    List<Integer> result1 = parser.parseExpression("{1,2,3}").getValue(List.class);
    Assert.assertEquals(new Integer(1), result1.get(0));
    try {
        result1.set(0, 2);
    } catch (Exception e) {
        e.printStackTrace();
    }
    //对于列表中只要有一个不是字面量表达式，将只返回原始List，
    //不会进行不可修改处理
    String expression3 = "{{1+2,2+4},{3,4+4}}";
    List<List<Integer>> result3 =
    parser.parseExpression(expression3).getValue(List.class);
    result3.get(0).set(0, 1);
    System.out.println(result3);
    //声明二维数组并初始化
    int[] result4 = parser.parseExpression("new int[2]{1,2}").getValue(int[].class);
    System.out.println(result4[1]);
    //定义一维数组并初始化
    int[] result5 = parser.parseExpression("new int[1]").getValue(int[].class);
    System.out.println(result5[0]);
}
```

输出

```
java.lang.UnsupportedOperationException
    at java.util.Collections$UnmodifiableList.set(Collections.java:1311)
    at com.javacode2018.spel.SpelTest.test7(SpelTest.java:315)
[[1, 6], [3, 8]]
2
0
```

内联数组

和Java 数组定义类似，只是在定义时进行多维数组初始化。

```
int[][][] result4 = parser.parseExpression("new int[1][2][3]{{1}{2}{3}}").getValue(int[][][].class);
```

集合，字典元素访问

SpEL目前支持所有集合类型和字典类型的元素访问，使用“集合[索引]”访问集合元素，使用“map[key]”访问字典元素；

```
//SpEL内联List访问
int result1 = parser.parseExpression("{1,2,3}[0]").getValue(int.class);

//SpEL目前支持所有集合类型的访问
Collection<Integer> collection = new HashSet<Integer>();
collection.add(1);
collection.add(2);

EvaluationContext context2 = new StandardEvaluationContext();
context2.setVariable("collection", collection);
int result2 = parser.parseExpression("#collection[1]").getValue(context2,
int.class);

//SpEL对Map字典元素访问的支持
Map<String, Integer> map = new HashMap<String, Integer>();
map.put("a", 1);

EvaluationContext context3 = new StandardEvaluationContext();
context3.setVariable("map", map);
int result3 = parser.parseExpression("#map['a']").getValue(context3, int.class);
```

列表，字典，数组元素修改

可以使用赋值表达式或Expression接口的setValue方法修改；

```
@Test
public void test8() {
    ExpressionParser parser = new SpelExpressionParser();

    //修改list元素值
    List<Integer> list = new ArrayList<Integer>();
    list.add(1);
    list.add(2);

    EvaluationContext context1 = new StandardEvaluationContext();
    context1.setVariable("collection", list);
    parser.parseExpression("#collection[1]").setValue(context1, 4);
}
```



```

    int result1 = parser.parseExpression("#collection[1]").getValue(context1,
int.class);
    System.out.println(result1);

    //修改map元素值
    Map<String, Integer> map = new HashMap<String, Integer>();
    map.put("a", 1);
    EvaluationContext context2 = new StandardEvaluationContext();
    context2.setVariable("map", map);
    parser.parseExpression("#map['a']").setValue(context2, 4);
    Integer result2 = parser.parseExpression("#map['a']").getValue(context2,
int.class);
    System.out.println(result2);
}

```

输出

```

4
4

```

集合投影

在SQL中投影指从表中选择出列，而在SpEL指根据集合中的元素中通过选择来构造另一个集合，该集合和原集合具有相同数量的元素；SpEL使用“(list|map) .![投影表达式]”来进行投影运算：

```

@Test
public void test9() {
    ExpressionParser parser = new SpELExpressionParser();

    //1.测试集合或数组
    List<Integer> list = new ArrayList<Integer>();
    list.add(4);
    list.add(5);
    EvaluationContext context1 = new StandardEvaluationContext();
    context1.setVariable("list", list);
    Collection<Integer> result1 = parser.parseExpression("#list.![
#this+1]").getValue(context1, Collection.class);
    result1.forEach(System.out::println);

    System.out.println("-----");
    //2.测试字典
    Map<String, Integer> map = new HashMap<String, Integer>();
    map.put("a", 1);
    map.put("b", 2);
    EvaluationContext context2 = new StandardEvaluationContext();
    context2.setVariable("map", map);
    List<Integer> result2 = parser.parseExpression("#map.![
value+1]").getValue(context2, List.class);
    result2.forEach(System.out::println);
}

```

对于集合或数组使用如上表达式进行投影运算，其中投影表达式中“#this”代表每个集合或数组元素，可以使用比如“#this.property”来获取集合元素的属性，其中“#this”可以省略。

Map投影最终只能得到List结果，如上所示，对于投影表达式中的“#this”将是Map.Entry，所以可以使用“value”来获取值，使用“key”来获取键。

集合选择

在SQL中指使用select进行选择行数据，而在SpEL指根据原集合通过条件表达式选择出满足条件的元素并构造为新的集合，SpEL使用“(list|map).?[选择表达式]”，其中选择表达式结果必须是boolean类型，如果true则选择的元素将添加到新集合中，false将不添加到新集合中。

```
@Test
public void test10() {
    ExpressionParser parser = new SpelExpressionParser();

    //1.测试集合或数组
    List<Integer> list = new ArrayList<Integer>();
    list.add(1);
    list.add(4);
    list.add(5);
    list.add(7);
    EvaluationContext context1 = new StandardEvaluationContext();
    context1.setVariable("list", list);
    Collection<Integer> result1 = parser.parseExpression("#list.?[#this>4]").getValue(context1, Collection.class);
    result1.forEach(System.out::println);

    System.out.println("-----");
}
```

输出

```
5
7
```

对于集合或数组选择，如“#collection.?[#this>4]”将选择出集合元素值大于4的所有元素。选择表达式必须返回布尔类型，使用“#this”表示当前元素。

```
//2.测试字典
Map<String, Integer> map = new HashMap<String, Integer>();
map.put("a", 1);
map.put("b", 2);
map.put("c", 3);
EvaluationContext context2 = new StandardEvaluationContext();
context2.setVariable("map", map);
Map<String, Integer> result2 = parser.parseExpression("#map.?[key!='a']").getValue(context2, Map.class);
result2.forEach((key, value) -> {
    System.out.println(key + ":" + value);
});
System.out.println("-----");
List<Integer> result3 = parser.parseExpression("#map.?[key!='a'].![value+1]").getValue(context2, List.class);
result3.forEach(System.out::println);
```

输出

```
b:2
c:3
-----
3
4
```

对于字典选择，如“#map.?[#this.key != 'a']”将选择键值不等于“a”的，其中选择表达式中“#this”是 Map.Entry 类型，而最终结果还是 Map，这点和投影不同；集合选择和投影可以一起使用，如“#map.?[key != 'a'].![value+1]”将首先选择键值不等于“a”的，然后在选出的 Map 中再进行“value+1”的投影。

表达式模板

模板表达式就是由字面量与一个或多个表达式块组成。每个表达式块由“前缀+表达式+后缀”形式组成，如“\${1+2}”即表达式块。在前边我们已经介绍了使用 ParserContext 接口实现来定义表达式是否是模板及前缀和后缀定义。在此就不多介绍了，如“Error \${#v0} \${#v1}”表达式表示由字面量“Error”、模板表达式“#v0”、模板表达式“#v1”组成，其中 v0 和 v1 表示自定义变量，需要在上下文定义。

解析表达式的时候需要指定模板，模板通过 ParserContext 接口来定义

```
public interface ParserContext {
    //是否是模板
    boolean isTemplate();
    //模板表达式前缀
    String getExpressionPrefix();
    //模板表达式后缀
    String getExpressionSuffix();
}
```

有个子类，我们直接可以拿来用：TemplateParserContext。

```
@Test
public void test11() {
    //创建解析器
    SpELExpressionParser parser = new SpELExpressionParser();
    //创建解析器上下文
    ParserContext context = new TemplateParserContext("%{", "}");
    Expression expression = parser.parseExpression("你好:%{#name},我们正在学习:%{#lesson}", context);

    //创建表达式计算上下文
    EvaluationContext evaluationContext = new StandardEvaluationContext();
    evaluationContext.setVariable("name", "路人甲java");
    evaluationContext.setVariable("lesson", "spring高手系列!");
    //获取值
    String value = expression.getValue(evaluationContext, String.class);
    System.out.println(value);
}
```

运行输出

```
你好:路人甲java,我们正在学习:spring高手系列!
```

在Bean定义中使用spel表达式

xml风格的配置

SpEL支持在Bean定义时注入，默认使用“#{SpEL表达式}”表示，其中“#root”根对象默认可以认为是ApplicationContext，只有ApplicationContext实现默认支持SpEL，获取根对象属性其实是获取容器中的Bean。

如：

```
<bean id="world" class="java.lang.String">
    <constructor-arg value="#{' world!'}"/>
</bean>

<bean id="hello1" class="java.lang.String">
    <constructor-arg value="#{'Hello'}#{world}"/>
</bean>

<bean id="hello2" class="java.lang.String">
    <constructor-arg value="#{'Hello' + world}"/>
</bean>

<bean id="hello3" class="java.lang.String">
    <constructor-arg value="#{'Hello' + @world}"/>
</bean>
```

模板默认以前缀“#{”开头，以后缀“}”结尾，且不允许嵌套，如“#{'Hello'#{world}}”错误，如“#{'Hello' + world}”中“world”默认解析为Bean。当然可以使用“@bean”引用了。

是不是很简单，除了XML配置方式，Spring还提供一种注解方式@Value，接着往下看吧。

注解风格的配置

基于注解风格的SpEL配置也非常简单，使用@Value注解来指定SpEL表达式，该注解可以放到字段、方法及方法参数上。

测试Bean类如下，使用@Value来指定SpEL表达式：

```
public class SpELBean {
    @Value("#{'Hello' + world}")
    private String value;
}
```

在Bean定义中SpEL的问题

如果有同学问“#{我不是SpEL表达式}”不是SpEL表达式，而是公司内部的模板，想换个前缀和后缀该如何实现呢？

我们使用BeanFactoryPostProcessor接口提供postProcessBeanFactory回调方法，它是在IoC容器创建好但还未进行任何Bean初始化时被ApplicationContext实现调用，因此在这个阶段把SpEL前缀及后缀改掉是安全的，具体代码如下：

```

package com.javacode2018.spel.test1;

import org.springframework.beans.BeansException;
import org.springframework.beans.factory.config.BeanExpressionResolver;
import org.springframework.beans.factory.config.BeanFactoryPostProcessor;
import org.springframework.beans.factory.config.ConfigurableListableBeanFactory;
import org.springframework.context.expression.StandardBeanExpressionResolver;
import org.springframework.stereotype.Component;

@Component
public class SpelBeanFactoryPostProcessor implements BeanFactoryPostProcessor {
    @Override
    public void postProcessBeanFactory(ConfigurableListableBeanFactory
beanFactory) throws BeansException {
        BeanExpressionResolver beanExpressionResolver =
beanFactory.getBeanExpressionResolver();
        if (beanExpressionResolver instanceof StandardBeanExpressionResolver) {
            StandardBeanExpressionResolver resolver =
(StandardBeanExpressionResolver) beanExpressionResolver;
            resolver.setExpressionPrefix("%{");
            resolver.setExpressionSuffix("}");
        }
    }
}

```

上测试代码

```

package com.javacode2018.spel.test1;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

@Component
public class LessonModel {
    @Value("你好,%{@name},%{@msg}")
    private String desc;

    @Override
    public String toString() {
        return "LessonModel{" +
            "desc='" + desc + '\'' +
            '}';
    }
}

```

@name: 容器中name的bean

@msg: 容器中msg的bean

下面我们来个配置类，顺便定义name和msg这2个bean，顺便扫描上面2个配置类

```

package com.javacode2018.spel.test1;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

```

```
@ComponentScan
@Configuration
public class MainConfig {
    @Bean
    public String name() {
        return "路粉";
    }

    @Bean
    public String msg() {
        return "欢迎和我一起学习java各种技术! ";
    }
}
```

测试用例

```
@Test
public void test12() {
    AnnotationConfigApplicationContext context = new
    AnnotationConfigApplicationContext();
    context.register(MainConfig.class);
    context.refresh();
    LessonModel lessonModel = context.getBean(LessonModel.class);
    System.out.println(lessonModel);
}
```

运行输出

```
LessonModel{desc='你好,路粉,欢迎和我一起学习java各种技术! '}
```

总结

1. Spel功能还是比较强大的，可以脱离spring环境独立运行
2. spel可以用在一些动态规则的匹配方面，比如监控系统中监控规则的动态匹配；其他的一些条件动态判断等等
3. 本文内容比较长，建议大家把案例都敲一遍，可以设置一些断点去研究一下源码，有问题的，欢迎大家留言交流。

案例源码

<https://gitee.com/javacode2018/spring-series>

路人甲java所有案例代码以后都会放到这个上面，大家watch一下，可以持续关注动态。

Spring系列

1. [Spring系列第1篇：为何要学spring?](#)
2. [Spring系列第2篇：控制反转（IoC）与依赖注入（DI）](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)

4. [Spring系列第4篇: xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇: 创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇: 玩转bean scope, 避免跳坑里!](#)
7. [Spring系列第7篇: 依赖注入之手动注入](#)
8. [Spring系列第8篇: 自动注入 \(autowire\) 详解, 高手在于坚持](#)
9. [Spring系列第9篇: depend-on到底是干什么的?](#)
10. [Spring系列第10篇: primary可以解决什么问题?](#)
11. [Spring系列第11篇: bean中的autowire-candidate又是干什么的?](#)
12. [Spring系列第12篇: lazy-init: bean延迟初始化](#)
13. [Spring系列第13篇: 使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇: lookup-method和replaced-method比较陌生, 怎么玩的?](#)
15. [Spring系列第15篇: 代理详解 \(Java动态代理&cglib代理\) ?](#)
16. [Spring系列第16篇: 深入理解java注解及spring对注解的增强 \(预备知识\)](#)
17. [Spring系列第17篇: @Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇: @ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇: @import详解\(bean批量注册\)](#)
20. [Spring系列第20篇: @Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇: 注解实现依赖注入 \(@Autowired、@Resource、@Primary、@Qualifier\)](#)
22. [Spring系列第22篇: @Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇: Bean生命周期详解](#)
24. [Spring系列第24篇: 父子容器详解](#)
25. [Spring系列第25篇: @Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇: 国际化详解](#)
27. [Spring系列第27篇: spring事件机制详解](#)
28. [Spring系列第28篇: Bean循环依赖详解](#)
29. [Spring系列第29篇: BeanFactory扩展 \(BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor\)](#)
30. [Spring系列第30篇: jdk动态代理和cglib代理](#)
31. [Spring系列第31篇: aop概念详解](#)
32. [Spring系列第32篇: AOP核心源码、原理详解](#)
33. [Spring系列第33篇: ProxyFactoryBean创建AOP代理](#)
34. [Spring系列第34篇: @Aspect中@Pointcut 12种用法](#)
35. [Spring系列第35篇: @Aspect中5中通知详解](#)
36. [Spring系列第36篇: @EnableAspectJAutoProxy、@Aspect中通知顺序详解](#)
37. [Spring系列第37篇: @EnableAsync & @Async 实现方法异步调用](#)
38. [Spring系列第38篇: @Scheduled & @EnableScheduling定时器详解](#)

更多好文章

1. [Java高并发系列 \(共34篇\)](#)
2. [MySQL高手系列 \(共27篇\)](#)
3. [Maven高手系列 \(共10篇\)](#)
4. [Mybatis系列 \(共12篇\)](#)
5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要, 它是什么? 怎么实现?](#)
7. [泛型, 有点难度, 会让很多人懵逼, 那是因为你没有看这篇文章!](#)



领取福利，扫码发送java

- 1、1000本java电子书大全
- 2、2021版java面试通关手册
- 3、月薪5万必备的java微服务学习线路图及高清视频
- 4、邀请加入高质量java技术交流群

加我微信：spring88java，进群一起交流java相关各种技术！！

Spring系列第40篇：缓存使用 (@EnableCaching、@Cacheable、 @CachePut、@CacheEvict、@Caching、 @CacheConfig)

本文主要详解spring中缓存的使用。

背景

缓存大家都了解过吧，主要用来提升系统查询速度。

比如电商中商品详情信息，这些信息通常不会经常变动但是会高频访问，我们可以将这些信息从db中拿出来放在缓存中（比如redis中、本地内存中），当获取的时候，先从缓存中获取，缓存中没有的时候，再从db中获取，然后将其再丢到缓存中，当商品信息被变更之后，可以将缓存中的信息剔除或者将最新的数据丢到缓存中。

Spring中提供了一整套的缓存解决方案，使用起来特别的容易，主要通过注解的方式使用缓存，常用的有5个注解，我们一个个来介绍。

本文中会大量用到spel表达式，对这块不熟悉的建议先看一下：[Spring中Spel详解](#)

@EnableCaching：启用缓存功能

开启缓存功能，配置类中需要加上这个注解，有了这个注解之后，spring才知道你需要使用缓存的功能，其他和缓存相关的注解才会有效，spring中主要是通过aop实现的，通过aop来拦截需要使用缓存的方法，实现缓存的功能。

@Cacheable：赋予缓存功能

作用

@Cacheable可以标记在一个方法上，也可以标记在一个类上。当标记在一个方法上时表示该方法是支持缓存的，当标记在一个类上时则表示该类所有的方法都是支持缓存的。对于一个支持缓存的方法，Spring会在其被调用后将其返回值缓存起来，以保证下次利用同样的参数来执行该方法时可以直接从缓存中获取结果，而不需要再次执行该方法。Spring在缓存方法的返回值时是以键值对进行缓存的，值就是方法的返回结果，至于键的话，Spring又支持两种策略，默认策略和自定义策略，这个稍后会进行说明。需要注意的是当一个支持缓存的方法在对象内部被调用时是不会触发缓存功能的。@Cacheable可以指定三个属性，value、key和condition。

```
@Target({ElementType.TYPE, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
@Inherited
@Documented
public @interface Cacheable {
    String[] value() default {};
    String[] cacheNames() default {};
    String key() default "";
    String condition() default "";
    String unless() default "";
}
```

value属性：指定Cache名称

value和cacheNames属性作用一样，必须指定其中一个，表示当前方法的返回值是会被缓存在哪个Cache上的，对应Cache的名称。其可以是一个Cache也可以是多个Cache，当需要指定多个Cache时其是一个数组。

可以将Cache想象为一个HashMap，系统中可以有很多个Cache，每个Cache有一个名字，你需要将方法的返回值放在哪个缓存中，需要通过缓存的名称来指定。

案例1

下面list方法加上了缓存的功能，将其结果放在缓存 `cache1` 中。

```
package com.javacode2018.cache.demo1;

import org.springframework.cache.annotation.Cacheable;
import org.springframework.stereotype.Component;

import java.util.Arrays;
import java.util.List;

@Component
public class ArticleService {

    @Cacheable(cacheNames = {"cache1"})
    public List<String> list() {
        System.out.println("获取文章列表!");
        return Arrays.asList("spring", "mysql", "java高并发", "maven");
    }
}
```

下面来个配置类 `MainConfig1`，必须加上 `@EnableCaching` 注解用来启用缓存功能。

然后在配置类中需要定义一个bean：缓存管理器，类型为 `CacheManager`，`CacheManager` 这个是个接口，有好几个实现（比如使用redis、ConcurrentMap来存储缓存信息），此处我们使用 `ConcurrentMapCacheManager`，内部使用ConcurrentHashMap将缓存信息直接存储在本地jvm内存中，不过线上环境一般是集群的方式，可以通过redis实现，下一篇文章介绍spring缓存集成redis。

```
package com.javacode2018.cache.demo1;

import org.springframework.cache.CacheManager;
import org.springframework.cache.annotation.EnableCaching;
import org.springframework.cache.concurrent.ConcurrentMapCacheManager;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@EnableCaching //①
@ComponentScan
@Configuration
public class MainConfig1 {

    //②: 缓存管理器
    @Bean
    public CacheManager cacheManager() {
        //创建缓存管理器(ConcurrentMapCacheManager: 其内部使用ConcurrentMap实现的)，构造器用来指定缓存的名称，可以指定多个
        ConcurrentMapCacheManager cacheManager = new
        ConcurrentMapCacheManager("cache1");
        return cacheManager;
    }

}
```

来个测试类，2次调用 `list` 方法看看效果

```
package com.javacode2018.cache;

import com.javacode2018.cache.demo1.ArticleService;
import com.javacode2018.cache.demo1.MainConfig1;
import org.junit.Test;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class CacheTest {

    @Test
    public void test1() {
        AnnotationConfigApplicationContext context = new
        AnnotationConfigApplicationContext();
        context.register(MainConfig1.class);
        context.refresh();
        ArticleService articleService = context.getBean(ArticleService.class);
        System.out.println(articleService.list());
        System.out.println(articleService.list());
    }

}
```

输出

获取文章列表!

[spring, mysql, java高并发, maven]

[spring, mysql, java高并发, maven]

从第一行可以看出，第一次进入到list方法内部了，第二次没有进入list方法内部，而是从缓存中获取的。

key属性：自定义key

key属性用来指定Spring缓存方法的返回结果时对应的key的，上面说了你可以将Cache理解为一个hashMap，缓存以key->value的形式存储在hashmap中，value就是需要缓存值（即方法的返回值）

key属性支持SpEL表达式；当我们没有指定该属性时，Spring将使用默认策略生成key（org.springframework.cache.interceptor.SimpleKeyGenerator），默认会方法参数创建key。

自定义策略是指我们可以通过SpEL表达式来指定我们的key，这里的SpEL表达式可以使用方法参数及它们对应的属性，使用方法参数时我们可以直接使用“#参数名”或者“#p参数index”。

Spring还为我们提供了一个root对象可以用来生成key，通过该root对象我们可以获取到以下信息。

属性名称	描述	示例
methodName	当前方法名	#root.methodName
method	当前方法	#root.method.name
target	当前被调用的对象	#root.target
targetClass	当前被调用的对象的class	#root.targetClass
args	当前方法参数组成的数组	#root.args[0]
caches	当前被调用的方法使用的Cache	#root.caches[0].name

这里我们主要看一下自定义策略。

案例2

ArticleService加入下面代码

```
@Cacheable(cacheNames = {"cache1"}, key = "#root.target.class.name+'-'+#page+'-'+#pageSize")
public String getPage(int page, int pageSize) {
    String msg = String.format("page-%s-pageSize-%s", page, pageSize);
    System.out.println("从db中获取数据: " + msg);
    return msg;
}
```

com.javacode2018.cache.CacheTest 新增测试用例

```
@Test
public void test2() {
    AnnotationConfigApplicationContext context = new
    AnnotationConfigApplicationContext();
    context.register(MainConfig1.class);
}
```

```

context.refresh();
ArticleService articleService = context.getBean(ArticleService.class);

//page=1,pageSize=10调用2次
System.out.println(articleService.getPage(1, 10));
System.out.println(articleService.getPage(1, 10));

//page=2,pageSize=10调用2次
System.out.println(articleService.getPage(2, 10));
System.out.println(articleService.getPage(2, 10));

{
    System.out.println("下面打印出cache1缓存中的key列表");
    ConcurrentMapCacheManager cacheManager =
context.getBean(ConcurrentMapCacheManager.class);
    ConcurrentMapCache cache1 = (ConcurrentMapCache)
cacheManager.getCache("cache1");
    cache1.getNativeCache().keySet().stream().forEach(System.out::println);
}
}

```

运行输出

```

从db中获取数据: page-1-pageSize-10
page-1-pageSize-10
page-1-pageSize-10
从db中获取数据: page-2-pageSize-10
page-2-pageSize-10
page-2-pageSize-10
下面打印出cache1缓存中的key列表
com.javacode2018.cache.demo1.ArticleService-getPage-1-10
com.javacode2018.cache.demo1.ArticleService-getPage-2-10

```

condition属性：控制缓存的使用条件

有时候，可能我们希望查询不走缓存，同时返回的结果也不要被缓存，那么就可以通过condition属性来实现，condition属性默认为空，表示将缓存所有的调用情形，其值是通过spel表达式来指定的，当为true时表示先尝试从缓存中获取；若缓存中不存在，则只需方法，并将方法返回值丢到缓存中；当为false的时候，不走缓存、直接执行方法、并且返回结果也不会丢到缓存中。

其值spel的写法和key属性类似。

案例3

ArticleService添加下面代码，方法的第二个参数cache用来控制是否走缓存，将condition的值指定为#cache

```

/**
 * 通过文章id获取文章
 *
 * @param id    文章id
 * @param cache 是否尝试从缓存中获取
 * @return
 */
@Cacheable(cacheNames = "cache1", key = "'getById'+#id", condition = "#cache")
public String getById(Long id, boolean cache) {
    System.out.println("获取数据!");
    return "spring缓存:" + UUID.randomUUID().toString();
}

```

来个测试用例，4次调用getById方法，前面2次和最后一次cache参数都是true，第3次为false

```

@Test
public void test3() {
    AnnotationConfigApplicationContext context = new
    AnnotationConfigApplicationContext();
    context.register(MainConfig1.class);
    context.refresh();
    ArticleService articleService = context.getBean(ArticleService.class);

    System.out.println(articleService.getById(1L, true));
    System.out.println(articleService.getById(1L, true));
    System.out.println(articleService.getById(1L, false));
    System.out.println(articleService.getById(1L, true));
}

```

运行输出

```

获取数据!
spring缓存:27e7c11a-26ed-4c8b-8444-78257daafed5
spring缓存:27e7c11a-26ed-4c8b-8444-78257daafed5
获取数据!
spring缓存:05ff7612-29cb-4863-b8bf-d1b7c2c192b7
spring缓存:27e7c11a-26ed-4c8b-8444-78257daafed5

```

从输出中可以看出，第1次和第3次，都进到方法里面去了，而2和4走了缓存，第一次执行完毕之后结果被丢到了缓存中，所以2和4这2次获取的结果和第1次是一样的。

unless属性：控制是否需要将结果丢到缓存中

用于否决方法缓存的SpEL表达式。与condition不同，此表达式是在调用方法后计算的，因此可以引用结果。默认值为""，这意味着缓存永远不会被否决。

前提是condition为空或者为true的情况下，unless才有效，condition为false的时候，unless无效，unless为true，方法返回结果不会丢到缓存中；unless为false，方法返回结果会丢到缓存中。

其值spel的写法和key属性类似。

案例4

下面来个案例，当返回结果为null的时候，不要将结果进行缓存，ArticleService添加下面代码

```
Map<Long, String> articleMap = new HashMap<>();
/**
 * 获取文章，先从缓存中获取，如果获取的结果为空，不要将结果放在缓存中
 *
 * @param id
 * @return
 */
@Cacheable(cacheNames = "cache1", key = "'findById'+#id", unless =
"#result==null")
public String findById(Long id) {
    this.articleMap.put(1L, "spring系列");
    System.out.println("----获取文章:" + id);
    return articleMap.get(id);
}
```

来个测试用例，4次调用findById，前面2次有数据，后面2次返回null，并将缓存中的key打印了出来

```
@Test
public void test4() {
    AnnotationConfigApplicationContext context = new
    AnnotationConfigApplicationContext();
    context.register(MainConfig1.class);
    context.refresh();
    ArticleService articleService = context.getBean(ArticleService.class);

    System.out.println(articleService.findById(1L));
    System.out.println(articleService.findById(1L));
    System.out.println(articleService.findById(3L));
    System.out.println(articleService.findById(3L));

    {
        System.out.println("下面打印出缓存cache1缓存中的key列表");
        ConcurrentMapCacheManager cacheManager =
        context.getBean(ConcurrentMapCacheManager.class);
        ConcurrentMapCache cache1 = (ConcurrentMapCache)
        cacheManager.getCache("cache1");
        cache1.getNativeCache().keySet().stream().forEach(System.out::println);
    }
}
```

运行输出

```
----获取文章:1
spring系列
spring系列
----获取文章:3
null
----获取文章:3
null
下面打印出缓存cache1缓存中的key列表
findById1
```

可以看出文章id为1的结果被缓存了，文件id为3的没有被缓存。

condition和unless对比

缓存的使用过程中有2个点：

1. 查询缓存中是否有数据
2. 如果缓存中没有数据，则去执行目标方法，然后将方法结果丢到缓存中。

spring中通过condition和unless对这2点进行干预。

condition作用域上面2个过程，当为true的时候，会尝试从缓存中获取数据，如果没有，会执行方法，然后将方法返回值丢到缓存中；如果为false，则直接调用目标方法，并且结果不会放在缓存中。

而unless在condition为true的情况下才有效，用来判断上面第2点中，是否不要将结果丢到缓存中，如果为true，则结果不会丢到缓存中，如果为false，则结果会丢到缓存中，并且unless中可以使用spel表达式通过#result来获取方法返回值。

@CachePut：将结果放入缓存

作用

@CachePut也可以标注在类或者方法上，被标注的方法每次都会被调用，然后方法执行完毕之后，会将方法结果丢到缓存中；当标注在类上，相当于在类的所有方法上标注了@CachePut。

有3种情况，结果不会丢到缓存

1. 当方法向外抛出的时候
2. condition的计算结果为false的时候
3. unless的计算结果为true的时候

源码和Cacheable类似，包含的参数类似的。

```
@Target({ElementType.TYPE, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
@Inherited
@Documented
public @interface CachePut {
    String[] value() default {};
    String[] cacheNames() default {};
    String key() default "";
    String condition() default "";
    String unless() default "";
}
```

- value和cacheNames：用来指定缓存名称，可以指定多个
- key：缓存的key，spel表达式，写法参考@Cacheable中的key
- condition：spel表达式，写法和@Cacheable中的condition一样，当为空或者计算结果为true的时候，方法的返回值才会丢到缓存中；否则结果不会丢到缓存中
- unless：当condition为空或者计算结果为true的时候，unless才会起效；true：结果不会被丢到缓存，false：结果会被丢到缓存。

案例5

来个案例，实现新增文章的操作，然后将文章丢到缓存中，注意下面@CachePut中的cacheNames、key 2个参数和案例4中findById方法上@Cacheable中的一样，说明他们共用一个缓存，key也是一样的，那么当add方法执行完毕之后，再去调用findById方法，则可以从缓存中直接获取到数据。

```
@CachePut(cacheNames = "cache1", key = "'findById'+#id")
public String add(Long id, String content) {
    System.out.println("新增文章:" + id);
    this.articleMap.put(id, content);
    return content;
}
```

测试用例

```
@Test
public void test5() {
    AnnotationConfigApplicationContext context = new
    AnnotationConfigApplicationContext();
    context.register(MainConfig1.class);
    context.refresh();
    ArticleService articleService = context.getBean(ArticleService.class);

    //新增3篇文章，由于add方法上面有@CachePut注解，所以新增之后会自动丢到缓存中
    articleService.add(1L, "java高并发系列");
    articleService.add(2L, "Maven高手系列");
    articleService.add(3L, "MySQL高手系列");

    //然后调用findById获取，看看是否会走缓存
    System.out.println("调用findById方法，会尝试从缓存中获取");
    System.out.println(articleService.findById(1L));
    System.out.println(articleService.findById(2L));
    System.out.println(articleService.findById(3L));

    {
        System.out.println("下面打印出cache1缓存中的key列表");
        ConcurrentMapCacheManager cacheManager =
        context.getBean(ConcurrentMapCacheManager.class);
        ConcurrentMapCache cache1 = (ConcurrentMapCache)
        cacheManager.getCache("cache1");
        cache1.getNativeCache().keySet().stream().forEach(System.out::println);
    }
}
```

运行输出

新增文章:1
新增文章:2
新增文章:3
调用findById方法，会尝试从缓存中获取
java高并发系列
Maven高手系列
MySQL高手系列
下面打印出缓存cache1中的key列表
findById3
findById2
findById1

看几眼输出结果，然后再来看一下findById方法的代码

```
@Cacheable(cacheNames = "cache1", key = "'findById'+#id", unless =
"#result==null")
public String findById(Long id) {
    this.articleMap.put(1L, "spring系列");
    System.out.println("----获取文章:" + id);
    return articleMap.get(id);
}
```

输出中并没有 ---- 这样的内容，说明调用findById方法获取结果是从缓存中得到的。

@CacheEvict：缓存清理

作用

用来清除缓存的，@CacheEvict也可以标注在类或者方法上，被标注在方法上，则目标方法被调用的时候，会清除指定的缓存；当标注在类上，相当于在类的所有方法上标注了@CacheEvict。

来看一下源码，每个参数的注释大家详细看一下。

```
public @interface CacheEvict {

    /**
     * cache的名称，和cacheNames效果一样
     */
    String[] value() default {};

    /**
     * cache的名称，和cacheNames效果一样
     */
    String[] cacheNames() default {};

    /**
     * 缓存的key，写法参考上面@Cacheable注解的key
     */
    String key() default "";

    /**
     * @CacheEvict 注解生效的条件，值为spel表达式，写法参考上面 @Cacheable注解中的
    condition
     */
}
```

```

String condition() default "";

/**
 * 是否清理 cacheNames 指定的缓存中的所有缓存信息，默认是false
 * 可以将一个cache想象为一个HashMap，当 allEntries 为true的时候，相当于
HashMap.clear()
 * 当 allEntries 为false的时候，只会干掉key对应的数据，相当于HashMap.remove(key)
 */
boolean allEntries() default false;

/**
 * 何事执行清除操作（方法执行前 or 方法执行成功之后）
 * true: @CacheEvict 标注的方法执行之前，执行清除操作
 * false: @CacheEvict 标注的方法执行成功之后，执行清除操作，当方法弹出异常的时候，不会执行清除操作
 */
boolean beforeInvocation() default false;
}

```

condition属性

@CacheEvict 注解生效的条件，值为spel表达式，写法参考上面 @Cacheable注解中的condition

会清除哪些缓存？

默认情况下会清除cacheNames指定的缓存中key参数指定的缓存信息。

但是当 allEntries 为true的时候，会清除 cacheNames 指定的缓存中的所有缓存信息。

具体什么时候清除缓存？

这个是通过 beforeInvocation 参数控制的，这个参数默认是false，默认会在目标方法成功执行之后执行清除操作，若方法向外抛出了异常，不会执行清理操作；

如果 beforeInvocation 为true，则方法被执行之前就会执行缓存清理操作，方法执行之后不会再执行了。

案例6

ArticleService中新增个方法，使用@CacheEvict标注，这个方法执行完毕之后，会清理cache1中key=findById+参数id的缓存信息，注意cacheNames和key两个参数的值和findById中这2个参数的值一样。

```

@CacheEvict(cacheNames = "cache1", key = "'findById'+#id") //@1
public void delete(Long id) {
    System.out.println("删除文章: " + id);
    this.articleMap.remove(id);
}

```

新增测试用例，注释比较清晰，就不解释了

```

@Test
public void test6() {
    AnnotationConfigApplicationContext context = new
    AnnotationConfigApplicationContext();
    context.register(MainConfig1.class);
}

```

```

context.refresh();
ArticleService articleService = context.getBean(ArticleService.class);

//第1次调用findById, 缓存中没有, 则调用方法, 将结果丢到缓存中
System.out.println(articleService.findById(1L));
//第2次调用findById, 缓存中存在, 直接从缓存中获取
System.out.println(articleService.findById(1L));

//执行删除操作, delete方法上面有@CacheEvict方法, 会清除缓存
articleService.delete(1L);

//再次调用findById方法, 发现缓存中没有了, 则会调用目标方法
System.out.println(articleService.findById(1L));
}

```

运行输出

```

----获取文章:1
spring系列
spring系列
删除文章: 1
----获取文章:1
spring系列

```

调用了3次findById, 第1次, 缓存中没有, 所以进到方法内部了, 然后将结果丢到缓存了, 第2次缓存中有, 所以从缓存获取, 然后执行了delete方法, 这个方法执行完毕之后, 会清除缓存中文章id为1L的文章信息, 最后执行第三次findById方法, 此时缓存中没有发现数据, 然后进到目标方法内部了, 目标方法内部输出了----内容。

@Caching: 缓存注解组

当我们在类上或者同一个方法上同时使用@Cacheable、@CachePut和@CacheEvict这几个注解中的多个的时候, 此时可以使用@Caching这个注解来实现。

```

@Target({ElementType.TYPE, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
@Inherited
@Documented
public @interface Caching {

    Cacheable[] cacheable() default {};

    CachePut[] put() default {};

    CacheEvict[] evict() default {};

}

```

@CacheConfig: 提取公共配置

这个注解标注在类上，可以将其他几个缓存注解（@Cacheable、@CachePut和@CacheEvic）的公共参数给提取出来放在@CacheConfig中。

比如当一个类中有很多方法都需要使用（@Cacheable、@CachePut和@CacheEvic）这些缓存注解的时候，大家可以看一下这3个注解的源码，他们有很多公共的属性，比如：cacheNames、keyGenerator、cacheManager、cacheResolver，若这些属性值都是一样的，可以将其提取出来，放在@CacheConfig中，不过这些注解（@Cacheable、@CachePut和@CacheEvic）中也可以指定属性的值对@CacheConfig中的属性值进行覆盖。

```
@CacheConfig(cacheNames = "cache1")
public class ArticleService {
    @Cacheable(key = "'findById'+#id")
    public String findById(Long id) {
        this.articleMap.put(1L, "spring系列");
        System.out.println("----获取文章:" + id);
        return articleMap.get(id);
    }
}
```

原理

spring中的缓存主要是利用spring中aop实现的，通过Aop对需要使用缓存的bean创建代理对象，通过代理对象拦截目标方法的执行，实现缓存功能。

重点在于@EnableCaching这个注解，可以从@Import这个注解看起

```
@Import(CachingConfigurationSelector.class)
public @interface EnableCaching {
}
```

最终会给需要使用缓存的bean创建代理对象，并且会在代理中添加一个拦截器

org.springframework.cache.interceptor.CacheInterceptor，这个类中的invoke方法是关键，会拦截所有缓存相关的目标方法的执行，大家可以去细看一下。

总结

Spring系列到此已经40篇了，能和我一起坚持到现在的，真的不容易。

还没有看完的朋友，建议大家按序都看一遍，文章最好按顺序看，前后知识点是有依赖的。

如果前面的文章都看过了，那么本文的原理，不用我介绍了，大家自己很容易就搞懂了。

案例源码

<https://gitee.com/javacode2018/spring-series>

路人甲java所有案例代码以后都会放到这个上面，大家watch一下，可以持续关注动态。

Spring系列

1. [Spring系列第1篇：为何要学spring?](#)
2. [Spring系列第2篇：控制反转 \(IoC\) 与依赖注入 \(DI\)](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)
4. [Spring系列第4篇：xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇：创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇：玩转bean scope，避免跳坑里!](#)
7. [Spring系列第7篇：依赖注入之手动注入](#)
8. [Spring系列第8篇：自动注入 \(autowire\) 详解，高手在于坚持](#)
9. [Spring系列第9篇：depend-on到底是干什么的?](#)
10. [Spring系列第10篇：primary可以解决什么问题?](#)
11. [Spring系列第11篇：bean中的autowire-candidate又是干什么的?](#)
12. [Spring系列第12篇：lazy-init: bean延迟初始化](#)
13. [Spring系列第13篇：使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇：lookup-method和replaced-method比较陌生，怎么玩的?](#)
15. [Spring系列第15篇：代理详解 \(Java动态代理&cglib代理\) ?](#)
16. [Spring系列第16篇：深入理解java注解及spring对注解的增强 \(预备知识\)](#)
17. [Spring系列第17篇：@Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇：@ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇：@import详解\(bean批量注册\)](#)
20. [Spring系列第20篇：@Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇：注解实现依赖注入 \(@Autowired、@Resource、@Primary、@Qualifier\)](#)
22. [Spring系列第22篇：@Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇：Bean生命周期详解](#)
24. [Spring系列第24篇：父子容器详解](#)
25. [Spring系列第25篇：@Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇：国际化详解](#)
27. [Spring系列第27篇：spring事件机制详解](#)
28. [Spring系列第28篇：Bean循环依赖详解](#)
29. [Spring系列第29篇：BeanFactory扩展 \(BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor\)](#)
30. [Spring系列第30篇：jdk动态代理和cglib代理](#)
31. [Spring系列第31篇：aop概念详解](#)
32. [Spring系列第32篇：AOP核心源码、原理详解](#)
33. [Spring系列第33篇：ProxyFactoryBean创建AOP代理](#)
34. [Spring系列第34篇：@Aspect中@Pointcut 12种用法](#)
35. [Spring系列第35篇：@Aspect中5中通知详解](#)
36. [Spring系列第36篇：@EnableAspectJAutoProxy、@Aspect中通知顺序详解](#)
37. [Spring系列第37篇：@EnableAsync & @Async 实现方法异步调用](#)
38. [Spring系列第38篇：@Scheduled & @EnableScheduling定时器详解](#)
39. [Spring系列第39篇：强大的Spel表达式](#)

更多好文章

1. [Java高并发系列 \(共34篇\)](#)
2. [MySql高手系列 \(共27篇\)](#)
3. [Maven高手系列 \(共10篇\)](#)
4. [Mybatis系列 \(共12篇\)](#)
5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要，它是什么？怎么实现？](#)

7. [泛型](#)，有点难度，会让很多人懵逼，那是因为你没有看这篇文章！



领取福利，扫码发送java

- 1、1000本java电子书大全
- 2、2021版java面试通关手册
- 3、月薪5万必备的java微服务学习线路图及高清视频
- 4、邀请加入高质量java技术交流群

加我微信：spring888java，进群一起交流java相关各种技术！！

Spring系列第41篇：@EnableCaching集成redis缓存

上篇文章主要介绍了[spring中缓存的使用](#)，不过文中的案例都是以本地内存作为存储介质的，但是实际上我们的项目上线之后，基本上都会采用集群的方式进行部署，如果将数据存储在本地内存中，集群之间是无法共享的，我们可以将数据存储在redis中，从而实现缓存的共享，下面我们一起来看下Spring中@EnableCaching如何对接redis。

安装redis

下载地址：<https://redis.io/download>

pom.xml中引入redis配置

```
<dependency>
  <groupId>org.redisson</groupId>
  <artifactId>redisson</artifactId>
  <version>3.13.3</version>
</dependency>
```

项目中创建redis配置文件

新建 com/javacode2018/cache/demo2/redis.yml，内容如下：

```
singleServerConfig:
  address: "redis://127.0.0.1:6379"
  password: null
  clientName: null
  database: 7 #选择使用哪个数据库0~15
  idleConnectionTimeout: 10000
  connectTimeout: 10000
  timeout: 3000
codec:
  class: "org.redisson.codec.JsonJacksonCodec"
```

创建redis相关的bean

```
package com.javacode2018.cache.demo2;

import org.redisson.Redisson;
import org.redisson.api.RedissonClient;
import org.redisson.config.Config;
import org.redisson.spring.cache.RedissonSpringCacheManager;
import org.springframework.cache.CacheManager;
import org.springframework.cache.annotation.EnableCaching;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;

import java.io.IOException;
import java.io.InputStream;
import java.util.Arrays;

@ComponentScan
@EnableCaching //①
public class MainConfig2 {
    @Bean //②
    public CacheManager cacheManager() throws IOException {
        RedissonSpringCacheManager cacheManager = new
        RedissonSpringCacheManager(this.redissonClient());
        cacheManager.setCacheNames(Arrays.asList("cache1"));
        return cacheManager;
    }

    @Bean //③
    public RedissonClient redissonClient() throws IOException {
        InputStream is =
        MainConfig2.class.getResourceAsStream("/com/javacode2018/cache/demo2/redis.yml");
        Config config = Config.fromYAML(is);
        return Redisson.create(config);
    }
}
```

①: 开启spring cache功能。

②: 自定义spring中cache管理器，这个地方我们定义了一个redis类型的管理器，底层使用redis来作为缓存的存储介质。

③: 通过redis.yml配置文件来创建一个RedissonClient，用于和redis进行交互。

来个测试类

```
package com.javacode2018.cache.demo2;

import org.springframework.cache.annotation.Cacheable;
import org.springframework.stereotype.Component;

import java.util.Arrays;
import java.util.List;

@Component
```

```

public class BookService {

    @Cacheable(cacheNames = "cache1", key = "#root.targetClass.name+'-
    '+#root.method.name")
    public List<String> list() {
        System.out.println("---模拟从db中获取数据---");
        return Arrays.asList("java高并发", "springboot", "springcloud");
    }

}

```

测试用例

```

@Test
public void test7() {
    AnnotationConfigApplicationContext context = new
    AnnotationConfigApplicationContext(MainConfig2.class);

    BookService bookService = context.getBean(BookService.class);
    System.out.println(bookService.list());
    System.out.println(bookService.list());

    {
        System.out.println("下面打印出cache1缓存中的key列表");
        RedissonSpringCacheManager cacheManager =
        context.getBean(RedissonSpringCacheManager.class);
        RedissonCache cache1 = (RedissonCache) cacheManager.getCache("cache1");
        cache1.getNativeCache().keySet().stream().forEach(System.out::println);
    }
}

```

运行输出

```

---模拟从db中获取数据---
[java高并发, springboot, springcloud]
[java高并发, springboot, springcloud]
下面打印出cache1缓存中的key列表
com.javacode2018.cache.demo2.BookService-list

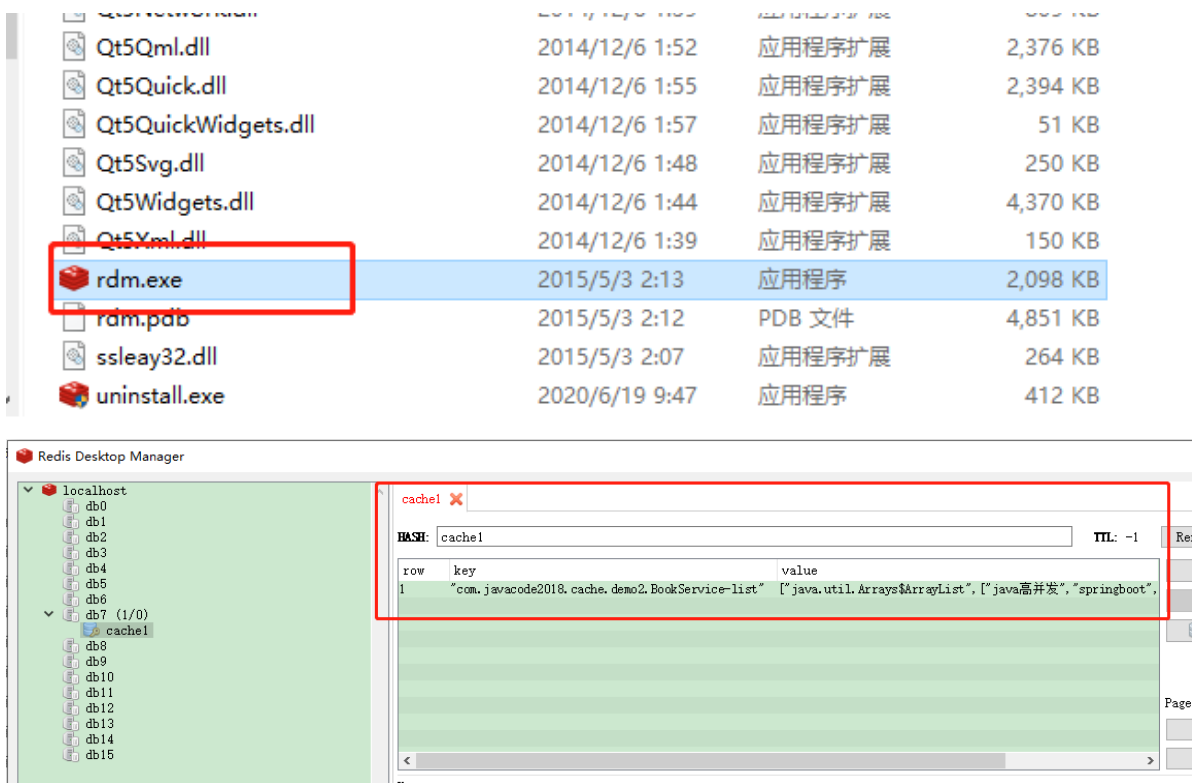
```

此时数据已经进入redis了，我们用redis客户端工具 RedisDesktopManager 来看一下。

RedisDesktopManager 下载地址

链接: <https://pan.baidu.com/s/1wCd-tk8dDDJnFIKciVIQsA>
提取码: x728

解压之后，点击下面的直接运行



案例源码

<https://gitee.com/javacode2018/spring-series>

路人甲java所有案例代码以后都会放到这个上面，大家watch一下，可以持续关注动态。

Spring系列

1. [Spring系列第1篇：为何要学spring?](#)
2. [Spring系列第2篇：控制反转 \(IoC\) 与依赖注入 \(DI\)](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)
4. [Spring系列第4篇：xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇：创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇：玩转bean scope，避免跳坑里!](#)
7. [Spring系列第7篇：依赖注入之手动注入](#)
8. [Spring系列第8篇：自动注入 \(autowire\) 详解，高手在于坚持](#)
9. [Spring系列第9篇：depend-on到底是干什么的?](#)
10. [Spring系列第10篇：primary可以解决什么问题?](#)
11. [Spring系列第11篇：bean中的autowire-candidate又是干什么的?](#)
12. [Spring系列第12篇：lazy-init: bean延迟初始化](#)
13. [Spring系列第13篇：使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇：lookup-method和replaced-method比较陌生，怎么玩的?](#)
15. [Spring系列第15篇：代理详解 \(Java动态代理&cglb代理\) ?](#)
16. [Spring系列第16篇：深入理解java注解及spring对注解的增强 \(预备知识\)](#)
17. [Spring系列第17篇：@Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇：@ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇：@import详解\(bean批量注册\)](#)
20. [Spring系列第20篇：@Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇：注解实现依赖注入 \(@Autowired、@Resource、@Primary、@Qualifier\)](#)

- 22. [Spring系列第22篇: @Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
- 23. [Spring系列第23篇: Bean生命周期详解](#)
- 24. [Spring系列第24篇: 父子容器详解](#)
- 25. [Spring系列第25篇: @Value【用法、数据来源、动态刷新】](#)
- 26. [Spring系列第26篇: 国际化详解](#)
- 27. [Spring系列第27篇: spring事件机制详解](#)
- 28. [Spring系列第28篇: Bean循环依赖详解](#)
- 29. [Spring系列第29篇: BeanFactory扩展 \(BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor\)](#)
- 30. [Spring系列第30篇: jdk动态代理和cglib代理](#)
- 31. [Spring系列第31篇: aop概念详解](#)
- 32. [Spring系列第32篇: AOP核心源码、原理详解](#)
- 33. [Spring系列第33篇: ProxyFactoryBean创建AOP代理](#)
- 34. [Spring系列第34篇: @Aspect中@Pointcut 12种用法](#)
- 35. [Spring系列第35篇: @Aspect中5中通知详解](#)
- 36. [Spring系列第36篇: @EnableAspectJAutoProxy、@Aspect中通知顺序详解](#)
- 37. [Spring系列第37篇: @EnableAsync & @Async 实现方法异步调用](#)
- 38. [Spring系列第38篇: @Scheduled & @EnableScheduling定时器详解](#)
- 39. [Spring系列第39篇: 强大的Spel表达式](#)
- 40. [Spring系列第40篇: 缓存使用 \(@EnableCaching、@Cacheable、@CachePut、@CacheEvict、@Caching、@CacheConfig\)](#)

更多好文章

- 1. [Java高并发系列 \(共34篇\)](#)
- 2. [MySql高手系列 \(共27篇\)](#)
- 3. [Maven高手系列 \(共10篇\)](#)
- 4. [Mybatis系列 \(共12篇\)](#)
- 5. [聊聊db和缓存一致性常见的实现方式](#)
- 6. [接口幂等性这么重要, 它是什么? 怎么实现?](#)
- 7. [泛型, 有点难度, 会让很多人懵逼, 那是因为你没有看这篇文章!](#)



领取福利, 扫码发送java

- 1、1000本java电子书大全
- 2、2021版java面试通关手册
- 3、月薪5万必备的java微服务学习线路图及高清视频
- 4、邀请加入高质量java技术交流群

加我微信: [spring888java](#), 进群一起交流java相关各种技术!!

Spring系列第42篇: JdbcTemplate实现增删改查如此简单?

本来这篇文章要写spring事务的，但是事务中大部分案例会用到JdbcTemplate相关的功能，所以先把JdbcTemplate拿出来说一下。

什么是JdbcTemplate?

大家来回顾一下，java中操作db最原始的方式就是纯jdbc了，是不是每次操作db都需要加载数据库驱动、获取连接、获取PreparedStatement、执行sql、关闭PreparedStatement、关闭连接等等，操作还是比较繁琐的，spring中提供了一个模块，对jdbc操作进行了封装，使其更简单，就是本文要讲的JdbcTemplate，JdbcTemplate是Spring对JDBC的封装，目的是使JDBC更加易于使用。

下面我们来看一下JdbcTemplate到底怎么玩的？

JdbcTemplate使用步骤

1. 创建数据源DataSource
2. 创建JdbcTemplate，new JdbcTemplate(dataSource)
3. 调用JdbcTemplate的方法操作db，如增删改查

```
public class DataSourceUtils {
    public static DataSource getDataSource() {
        org.apache.tomcat.jdbc.pool.DataSource dataSource = new
org.apache.tomcat.jdbc.pool.DataSource();
        dataSource.setDriverClassName("com.mysql.jdbc.Driver");
        dataSource.setUrl("jdbc:mysql://localhost:3306/javacode2018?
characterEncoding=UTF-8");
        dataSource.setUsername("root");
        dataSource.setPassword("root123");
        dataSource.setInitialSize(5);
        return dataSource;
    }
}

@Test
public void test0() {
    //1.创建数据源DataSource
    DataSource dataSource = DataSourceUtils.getDataSource();
    //2.创建JdbcTemplate, new JdbcTemplate(dataSource)
    JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);
    //3.调用JdbcTemplate的方法操作db，如增删改查
    List<Map<String, Object>> maps = jdbcTemplate.queryForList("select * from
t_user");
    System.out.println(maps);
}
```

输出

```
[{id=114, name=路人}, {id=115, name=java高并发}, {id=116, name=spring系列}]
```

t_user表数据

```
mysql> select id,name from t_user;
+-----+-----+
| id | name          |
+-----+-----+
| 114 | 路人          |
| 115 | java高并发    |
| 116 | spring系列    |
+-----+-----+
3 rows in set (0.00 sec)
```

上面查询返回了 `t_user` 表所有的记录，返回了一个集合，集合中是一个Map，Map表示一行记录，key为列名，value为列对应的值。

有没有感觉到特别的方便，只需要 `jdbcTemplate.queryForList("select * from t_user")` 这么简单的一行代码，数据就被获取到了。

下面我们继续探索更强大更好用的功能。

增加、删除、修改操作

`JdbcTemplate`中以`update`开头的方法，用来执行增、删、改操作，下面来看几个常用的。

无参情况

Api

```
int update(final String sql)
```

案例

```
@Test
public void test1() {
    DataSource dataSource = DataSourceUtils.getDataSource();
    JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);
    int updateRows = jdbcTemplate.update("INSERT INTO t_user (name) VALUE ('maven系列')");
    System.out.println("影响行数: " + updateRows);
}
```

有参情况1

Api

```
int update(String sql, Object... args)
```

案例

`sql`中使用`?`作为占位符。

```

@Test
public void test2() {
    DataSource dataSource = DataSourceUtils.getDataSource();
    JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);
    int updateRows = jdbcTemplate.update("INSERT INTO t_user (name) VALUE (?)",
"mybatis系列");
    System.out.println("影响行数: " + updateRows);
}

```

有参情况2

Api

```

int update(String sql, PreparedStatementSetter pss)

```

通过PreparedStatementSetter来设置参数，是个函数式接口，内部有个setValues方法会传递一个PreparedStatement参数，我们可以通这个参数手动的设置参数的值。

案例

```

@Test
public void test3() {
    DataSource dataSource = DataSourceUtils.getDataSource();
    JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);
    int updateRows = jdbcTemplate.update("INSERT INTO t_user (name) VALUE (?)",
new PreparedStatementSetter() {
        @Override
        public void setValues(PreparedStatement ps) throws SQLException {
            ps.setString(1, "mysql系列");
        }
    });
    System.out.println("影响行数: " + updateRows);
}

```

获取自增列的值

Api

```

public int update(final PreparedStatementCreator psc, final KeyHolder
generatedKeyHolder)

```

案例

```

@Test
public void test4() {
    DataSource dataSource = DataSourceUtils.getDataSource();
    JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);
    String sql = "INSERT INTO t_user (name) VALUE (?)";
    KeyHolder keyHolder = new GeneratedKeyHolder();
    int rowCount = jdbcTemplate.update(new PreparedStatementCreator() {
        @Override

```

```

        public PreparedStatement createPreparedStatement(Connection con) throws
SQLException {
            //手动创建PreparedStatement, 注意第二个参数:
            Statement.RETURN_GENERATED_KEYS
            PreparedStatement ps = con.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS);
            ps.setString(1, "获取自增列的值");
            return ps;
        }
    }, keyHolder);
    System.out.println("新记录id: " + keyHolder.getKey().intValue());
}

```

输出

新记录id: 122

```

mysql> select id,name from t_user;
+-----+-----+
| id | name                |
+-----+-----+
| 114 | 路人                |
| 115 | java高并发          |
| 116 | spring系列          |
| 117 | maven系列           |
| 118 | mysql系列           |
| 122 | 获取自增列的值      |
+-----+-----+
6 rows in set (0.00 sec)

```

批量增删改操作

Api

```

int[] batchUpdate(final String[] sql);
int[] batchUpdate(String sql, List<Object[]> batchArgs);
int[] batchUpdate(String sql, List<Object[]> batchArgs, int[] argTypes);

```

案例

```

@Test
public void test5() {
    DataSource dataSource = DataSourceUtils.getDataSource();
    JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);
    List<Object[]> list = Arrays.asList(
        new Object[]{"刘德华"},
        new Object[]{"郭富城"},
        new Object[]{"张学友"},
        new Object[]{"黎明"});
    int[] updateRows = jdbcTemplate.batchUpdate("INSERT INTO t_user (name) VALUE
(?)", list);
    for (int updateRow : updateRows) {

```

```
        System.out.println(updateRow);
    }
}
```

查询操作

查询一行单行

Api

```
/**
 * sql: 执行的sql, 如果有参数, 参数占位符?
 * requiredType: 返回的一列数据对应的java类型, 如String
 * args: ?占位符对应的参数列表
 */
<T> T queryForObject(String sql, Class<T> requiredType, @Nullable Object...
args)
```

案例

```
@Test
public void test6() {
    DataSource dataSource = DataSourceUtils.getDataSource();
    JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);
    String name = jdbcTemplate.queryForObject("select name from t_user where id
= ?", String.class, 114);
    System.out.println(name);
}
```

输出

路人

db中对应数据

```
mysql> select name from t_user where id = 114;
+-----+
| name  |
+-----+
| 路人  |
+-----+
1 row in set (0.00 sec)
```

使用注意

若queryForObject中sql查询无结果时, 会报错

如id为0的记录不存在

```
mysql> select name from t_user where id = 0;
Empty set (0.00 sec)
```

```

@Test
public void test7() {
    DataSource dataSource = DataSourceUtils.getDataSource();
    JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);
    String name = jdbcTemplate.queryForObject("select name from t_user where id
= ?", String.class, 0);
    System.out.println(name);
}

```

运行，会弹出一个异常 `EmptyResultDataAccessException`，期望返回一条记录，但实际上却没有找到记录，和期望结果不符，所以报错了

```

org.springframework.dao.EmptyResultDataAccessException: Incorrect result size:
expected 1, actual 0

    at
org.springframework.dao.support.DataAccessUtils.nullableSingleResult(DataAccessu
tils.java:97)
    at
org.springframework.jdbc.core.JdbcTemplate.queryForObject(JdbcTemplate.java:784)

```

这种如何解决呢，需要用到查询多行的方式解决了，即下面要说的 `queryForList` 相关的方法，无结果的时候会返回一个空的List，我们可以在这个空的List上面做文章。

查询一行多列

Api

以 `queryForList` 开头的方法。

```

<T> List<T> queryForList(String sql, Class<T> elementType);
<T> List<T> queryForList(String sql, Class<T> elementType, @Nullable Object...
args);
<T> List<T> queryForList(String sql, Object[] args, Class<T> elementType);
<T> List<T> queryForList(String sql, Object[] args, int[] argTypes, Class<T>
elementType);

```

注意：

上面这个T虽然是泛型，但是只支持 `Integer.class` `String.class` 这种单数据类型的，自己定义的Bean不支持。（所以用来查询单列数据）

elementType: 查询结果需要转换为哪种类型？如 `String`、`Integer`、`Double`。

案例

```

@Test
public void test8() {
    DataSource dataSource = DataSourceUtils.getDataSource();
    JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);
    //<T> List<T> queryForList(String sql, Class<T> elementType);
    List<String> list1 = jdbcTemplate.queryForList("select name from t_user
where id>131", String.class);
    System.out.println("list1:" + list1);
}

```



```

//<T> List<T> queryForList(String sql, Class<T> elementType, @Nullable
Object... args);
List<String> list2 = jdbcTemplate.queryForList("select name from t_user
where id>?", String.class, 131);
System.out.println("list2:" + list2);

//<T> List<T> queryForList(String sql, Object[] args, Class<T> elementType);
List<String> list3 = jdbcTemplate.queryForList("select name from t_user
where id>?", new Object[]{131}, String.class);
System.out.println("list3:" + list3);

//<T> List<T> queryForList(String sql, Object[] args, int[] argTypes,
Class<T> elementType);
List<String> list4 = jdbcTemplate.queryForList("select name from t_user
where id>?", new Object[]{131}, new int[]{java.sql.Types.INTEGER},
String.class);
System.out.println("list4:" + list4);
}

```

输出

```

list1:[郭富城, 张学友, 黎明]
list2:[郭富城, 张学友, 黎明]
list3:[郭富城, 张学友, 黎明]
list4:[郭富城, 张学友, 黎明]

```

sql结果:

```

mysql> select name from t_user where id>131;
+-----+
| name      |
+-----+
| 郭富城    |
| 张学友    |
| 黎明      |
+-----+
3 rows in set (0.00 sec)

```

查询单行记录，将记录转换成一个对象

Api

```

<T> T queryForObject(String sql, RowMapper<T> rowMapper);
<T> T queryForObject(String sql, Object[] args, RowMapper<T> rowMapper);
<T> T queryForObject(String sql, Object[] args, int[] argTypes, RowMapper<T>
rowMapper);
<T> T queryForObject(String sql, RowMapper<T> rowMapper, Object... args);

```

上面这些方法的参数中都有一个rowMapper参数，行映射器，可以将当前行的结果映射为一个自定义的对象。

```

@FunctionalInterface
public interface RowMapper<T> {

    /**
     * @param ResultSet 结果集
     * @param 当前结果集的第几行
     * @return 当前行的结果对象，将当前行的结果映射为一个自定义的对象返回
     */
    @Nullable
    T mapRow(ResultSet rs, int rowNum) throws SQLException;

}

```

JdbcTemplate内部会遍历ResultSet，然后循环调用RowMapper#mapRow，得到当前行的结果，将其丢到List中返回，如下：

```

List<T> results = new ArrayList<>();
int rowNum = 0;
while (rs.next()) {
    results.add(this.rowMapper.mapRow(rs, rowNum++));
}
return results;

```

案例

```

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@ToString
public class User {
    private Integer id;
    private String name;
}

```

```

@Test
public void test9() {
    DataSource dataSource = DataSourceUtils.getDataSource();
    JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);
    String sql = "select id,name from t_user where id = ?";
    //查询id为34的用户信息
    User user = jdbcTemplate.queryForObject(sql, new RowMapper<User>() {
        @Nullable
        @Override
        public User mapRow(ResultSet rs, int rowNum) throws SQLException {
            User user = new User();
            user.setId(rs.getInt(1));
            user.setName(rs.getString(1));
            return user;
        }
    }, 134);
    System.out.println(user);
}

```

输出

```
User(id=134, name=134)
```

使用注意

当queryForObject中sql查询无结果的时候，会报错，必须要返回一行记录

查询单行记录，返回指定的javabean

RowMapper 有个实现了类 BeanPropertyRowMapper，可以将结果映射为javabean。

```
@Test
public void test10() {
    DataSource dataSource = DataSourceUtils.getDataSource();
    JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);
    String sql = "select id,name from t_user where id = ?";
    //查询id为34的用户信息
    RowMapper<User> rowMapper = new BeanPropertyRowMapper<>(User.class);
    User user = jdbcTemplate.queryForObject(sql, rowMapper, 134);
    System.out.println(user);
}
```

查询多列多行，每行结果为一个Map

Api

```
List<Map<String, Object>> queryForList(String sql);
List<Map<String, Object>> queryForList(String sql, Object... args);
```

每行结果为一个Map，key为列名小写，value为列对应的值。

案例

```
@Test
public void test11() {
    DataSource dataSource = DataSourceUtils.getDataSource();
    JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);
    String sql = "select id,name from t_user where id>?";
    List<Map<String, Object>> maps = jdbcTemplate.queryForList(sql, 130);
    System.out.println(maps);
}
```

输出

```
[{id=131, name=刘德华}, {id=132, name=郭富城}, {id=133, name=张学友}, {id=134, name=黎明}]
```

查询多列多行，将结果映射为javabean

Api

```
<T> List<T> query(String sql, RowMapper<T> rowMapper, @Nullable Object... args)
```

案例

```
@Test
public void test12() {
    DataSource dataSource = DataSourceUtils.getDataSource();
    JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);
    String sql = "select id,name from t_user where id>?";
    List<User> maps = jdbcTemplate.query(sql, new RowMapper<User>() {
        @Nullable
        @Override
        public User mapRow(ResultSet rs, int rowNum) throws SQLException {
            User user = new User();
            user.setId(rs.getInt(1));
            user.setName(rs.getString(1));
            return user;
        }
    }, 130);
    System.out.println(maps);
}
```

运行输出

```
[User(id=131, name=刘德华), User(id=132, name=郭富城), User(id=133, name=张学友),
User(id=134, name=黎明)]
```

更简单的方式，使用 BeanPropertyRowMapper

```
@Test
public void test13() {
    DataSource dataSource = DataSourceUtils.getDataSource();
    JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);
    String sql = "select id,name from t_user where id>?";
    List<User> maps = jdbcTemplate.query(sql, new BeanPropertyRowMapper<User>
(User.class), 130);
    System.out.println(maps);
}
```

输出

```
[User(id=131, name=刘德华), User(id=132, name=郭富城), User(id=133, name=张学友),
User(id=134, name=黎明)]
```

总结

1. 使用注意：JdbcTemplate中的getObject开头的方法，要求sql必须返回一条记录，否则会报错

2. BeanPropertyRowMapper可以将行记录映射为javabean
3. JdbcTemplate采用模板的方式操作jdbc变的特别的容易，代码特别的简洁，不过其内部没有动态sql的功能，即通过参数，动态生成指定的sql，mybatis在动态sql方面做的比较好，大家用的时候可以根据需求进行选择。

案例源码

git地址:

<https://gitee.com/javacode2018/spring-series>

本文案例对应源码:

spring-series\lesson-003-

jdbcTemplate\src\main\java\com\javacode2018\jdbcTemplate\demo1\Demo1Test.java

路人甲java所有案例代码以后都会放到这个上面，大家watch一下，可以持续关注动态。

Spring系列

1. [Spring系列第1篇：为何要学spring?](#)
2. [Spring系列第2篇：控制反转（IoC）与依赖注入（DI）](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)
4. [Spring系列第4篇：xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇：创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇：玩转bean scope，避免跳坑里!](#)
7. [Spring系列第7篇：依赖注入之手动注入](#)
8. [Spring系列第8篇：自动注入（autowire）详解，高手在于坚持](#)
9. [Spring系列第9篇：depend-on到底是干什么的?](#)
10. [Spring系列第10篇：primary可以解决什么问题?](#)
11. [Spring系列第11篇：bean中的autowire-candidate又是干什么的?](#)
12. [Spring系列第12篇：lazy-init: bean延迟初始化](#)
13. [Spring系列第13篇：使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇：lookup-method和replaced-method比较陌生，怎么玩的?](#)
15. [Spring系列第15篇：代理详解（Java动态代理&cglib代理）?](#)
16. [Spring系列第16篇：深入理解java注解及spring对注解的增强（预备知识）](#)
17. [Spring系列第17篇：@Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇：@ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇：@import详解\(bean批量注册\)](#)
20. [Spring系列第20篇：@Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇：注解实现依赖注入（@Autowired、@Resource、@Primary、@Qualifier）](#)
22. [Spring系列第22篇：@Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇：Bean生命周期详解](#)
24. [Spring系列第24篇：父子容器详解](#)
25. [Spring系列第25篇：@Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇：国际化详解](#)
27. [Spring系列第27篇：spring事件机制详解](#)
28. [Spring系列第28篇：Bean循环依赖详解](#)
29. [Spring系列第29篇：BeanFactory扩展（BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor）](#)
30. [Spring系列第30篇：jdk动态代理和cglib代理](#)

31. [Spring系列第31篇：aop概念详解](#)
32. [Spring系列第32篇：AOP核心源码、原理详解](#)
33. [Spring系列第33篇：ProxyFactoryBean创建AOP代理](#)
34. [Spring系列第34篇：@Aspect中@Pointcut 12种用法](#)
35. [Spring系列第35篇：@Aspect中5中通知详解](#)
36. [Spring系列第36篇：@EnableAspectJAutoProxy、@Aspect中通知顺序详解](#)
37. [Spring系列第37篇：@EnableAsync & @Async 实现方法异步调用](#)
38. [Spring系列第38篇：@Scheduled & @EnableScheduling定时器详解](#)
39. [Spring系列第39篇：强大的Spel表达式](#)
40. [Spring系列第40篇：缓存使用 \(@EnableCaching、@Cacheable、@CachePut、@CacheEvict、@Caching、@CacheConfig\)](#)
41. [Spring系列第41篇：@EnableCaching集成redis缓存](#)

更多好文章

1. [Java高并发系列 \(共34篇\)](#)
2. [MySQL高手系列 \(共27篇\)](#)
3. [Maven高手系列 \(共10篇\)](#)
4. [Mybatis系列 \(共12篇\)](#)
5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要，它是什么？怎么实现？](#)
7. [泛型，有点难度，会让很多人懵逼，那是因为你没有看这篇文章！](#)



领取福利，扫码发送java

- 1、1000本java电子书大全
- 2、2021版java面试通关手册
- 3、月薪5万必备的java微服务学习线路图及高清视频
- 4、邀请加入高质量java技术交流群

加我微信：spring888java，进群一起交流java相关各种技术！！

Spring系列第43篇：spring中编程式事务怎么用的？

本文开始，大概用10篇左右的文章来详解spring中事务的使用，吃透spring事务。

本文内容

详解spring中编程式事务的使用。

spring中使用事务的2种方式

spring使事务操作变的异常容易了，spring中控制事务主要有2种方式

- **编程式事务**：硬编码的方式
- **声明式事务**：大家比较熟悉的注解@Transactional的方式

编程式事务

什么是编程式事务？

通过硬编码的方式使用spring中提供的事务相关的类来控制事务。

编程式事务主要有2种用法

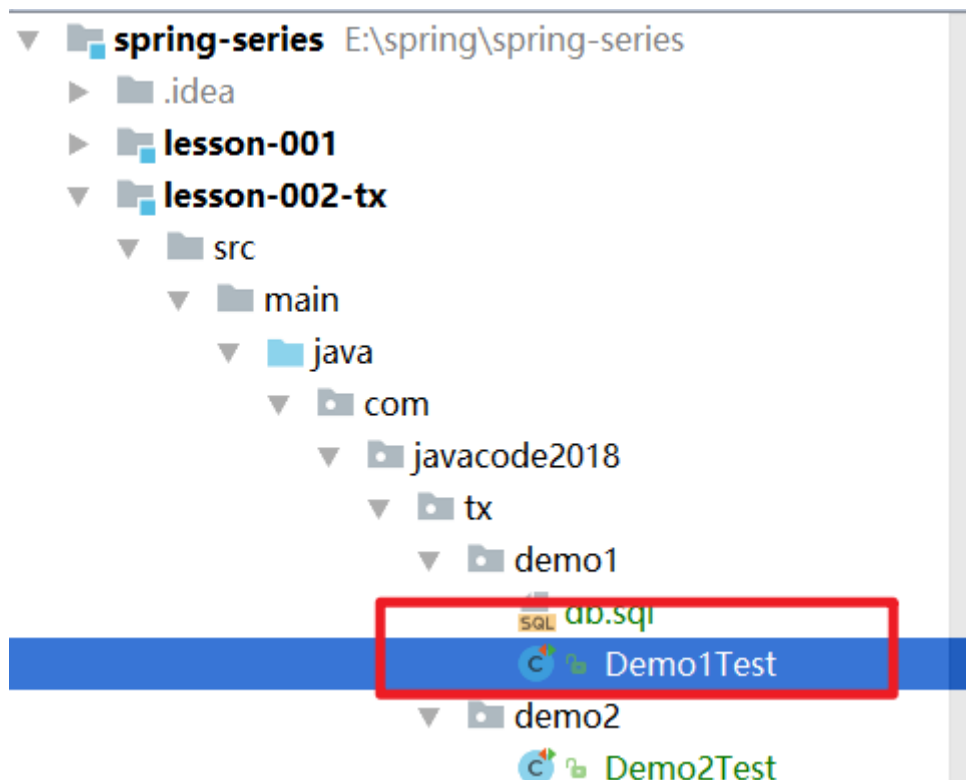
- 方式1：通过PlatformTransactionManager控制事务
- 方式2：通过TransactionTemplate控制事务

方式1： PlatformTransactionManager

这种是最原始的方式，代码量比较大，后面其他方式都是对这种方式的封装。

直接看案例，有详细的注释，大家一看就懂。

案例代码位置



准备sql

```

DROP DATABASE IF EXISTS javacode2018;
CREATE DATABASE IF NOT EXISTS javacode2018;

USE javacode2018;
DROP TABLE IF EXISTS t_user;
CREATE TABLE t_user(
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(256) NOT NULL DEFAULT '' COMMENT '姓名'
);

```

maven配置

```

<!-- JdbcTemplate需要的 -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-jdbc</artifactId>
    <version>5.2.3.RELEASE</version>
</dependency>
<!-- spring 事务支持 -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-tx</artifactId>
    <version>5.2.3.RELEASE</version>
</dependency>

```

测试代码

代码中会用到JdbcTemplate，对这个不理解的可以看一下：[JdbcTemplate使用详解](#)

```

@Test
public void test1() throws Exception {
    //定义一个数据源
    org.apache.tomcat.jdbc.pool.DataSource dataSource = new
    org.apache.tomcat.jdbc.pool.DataSource();
    dataSource.setDriverClassName("com.mysql.jdbc.Driver");
    dataSource.setUrl("jdbc:mysql://localhost:3306/javacode2018?
    characterEncoding=UTF-8");
    dataSource.setUsername("root");
    dataSource.setPassword("root123");
    dataSource.setInitialSize(5);
    //定义一个JdbcTemplate，用来方便执行数据库增删改查
    JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);
    //1.定义事务管理器，给其指定一个数据源（可以把事务管理器想象为一个人，这个人来负责事务的控制操作）
    PlatformTransactionManager platformTransactionManager = new
    DataSourceTransactionManager(dataSource);
    //2.定义事务属性：TransactionDefinition，TransactionDefinition可以用来配置事务的属性信息，比如事务隔离级别、事务超时时间、事务传播方式、是否是只读事务等等。
    TransactionDefinition transactionDefinition = new
    DefaultTransactionDefinition();
    //3.开启事务：调用platformTransactionManager.getTransaction开启事务操作，得到事务状态(TransactionStatus)对象
    TransactionStatus transactionStatus =
    platformTransactionManager.getTransaction(transactionDefinition);
    //4.执行业务操作，下面就执行2个插入操作

```



```

try {
    System.out.println("before:" + jdbcTemplate.queryForList("SELECT * from
t_user"));
    jdbcTemplate.update("insert into t_user (name) values (?)", "test1-1");
    jdbcTemplate.update("insert into t_user (name) values (?)", "test1-2");
    //5.提交事务: platformTransactionManager.commit
    platformTransactionManager.commit(transactionStatus);
} catch (Exception e) {
    //6.回滚事务: platformTransactionManager.rollback
    platformTransactionManager.rollback(transactionStatus);
}
System.out.println("after:" + jdbcTemplate.queryForList("SELECT * from
t_user"));
}

```

运行输出

```

before:[]
after:[{id=1, name=test1-1}, {id=2, name=test1-2}]

```

代码分析

代码中主要有5个步骤

步骤1: 定义事务管理器PlatformTransactionManager

事务管理器相当于一个管理员，这个管理员就是用来帮你控制事务的，比如开启事务，提交事务，回滚事务等等。

spring中使用PlatformTransactionManager这个接口来表示事务管理器，

```

public interface PlatformTransactionManager {

    //获取一个事务（开启事务）
    TransactionStatus getTransaction(@Nullable TransactionDefinition definition)
        throws TransactionException;

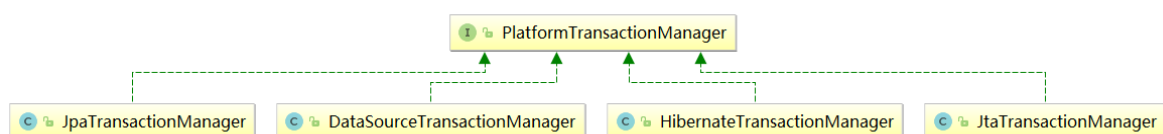
    //提交事务
    void commit(TransactionStatus status) throws TransactionException;

    //回滚事务
    void rollback(TransactionStatus status) throws TransactionException;

}

```

PlatformTransactionManager多个实现类，用来应对不同的环境



JpaTransactionManager: 如果你用jpa来操作db，那么需要用这个管理器来帮你控制事务。

DataSourceTransactionManager: 如果你用是指定数据源的方式, 比如操作数据库用的是: JdbcTemplate、mybatis、ibatis, 那么需要用这个管理器来帮你控制事务。

HibernateTransactionManager: 如果你用hibernate来操作db, 那么需要用这个管理器来帮你控制事务。

JtaTransactionManager: 如果你用的是java中的jta来操作db, 这种通常是分布式事务, 此时需要用这种管理器来控制事务。

上面案例代码中我们使用的是JdbcTemplate来操作db, 所以用的是 DataSourceTransactionManager 这个管理器。

```
PlatformTransactionManager platformTransactionManager = new
DataSourceTransactionManager(dataSource);
```

步骤2: 定义事务属性TransactionDefinition

定义事务属性, 比如事务隔离级别、事务超时时间、事务传播方式、是否是只读事务等等。

spring中使用TransactionDefinition接口来表示事务的定义信息, 有个子类比较常用: DefaultTransactionDefinition。

关于事务属性细节比较多, 篇幅比较长, 后面会专门有文章来详解。

步骤3: 开启事务

调用事务管理器的 `getTransaction` 方法, 即可以开启一个事务

```
TransactionStatus transactionStatus =
platformTransactionManager.getTransaction(transactionDefinition);
```

这个方法会返回一个 TransactionStatus 表示事务状态的一个对象, 通过 TransactionStatus 提供的一些方法可以用来控制事务的一些状态, 比如事务最终是需要回滚还是需要提交。

执行了 `getTransaction` 后, spring内部会执行一些操作, 为了方便大家理解, 咱们看看伪代码:

```
//有一个全局共享的threadLocal对象 resources
static final ThreadLocal<Map<Object, Object>> resources = new NamedThreadLocal<>
("Transactional resources");
//获取一个db的连接
DataSource datasource = platformTransactionManager.getDataSource();
Connection connection = datasource.getConnection();
//设置手动提交事务
connection.setAutoCommit(false);
Map<Object, Object> map = new HashMap<>();
map.put(datasource, connection);
resources.set(map);
```

上面代码, 将数据源datasource和connection映射起来放在了ThreadLocal中, ThreadLocal大家应该比较熟悉, 用于在同一个线程中共享数据; 后面我们可以通过resources这个ThreadLocal获取datasource其对应的connection对象。

步骤4：执行业务操作

我们使用JdbcTemplate插入了2条记录。

```
JdbcTemplate.update("insert into t_user (name) values (?)", "test1-1");  
JdbcTemplate.update("insert into t_user (name) values (?)", "test1-2");
```

大家看一下创建JdbcTemplate的代码，需要指定一个datasource

```
JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);
```

再看看创建事务管理器的代码

```
PlatformTransactionManager platformTransactionManager = new  
DataSourceTransactionManager(dataSource);
```

2者用到的是同一个dataSource，而事务管理器开启事务的时候，会创建一个连接，将dataSource和connection映射之后丢在了ThreadLocal中，而JdbcTemplate内部执行db操作的时候，也需要获取连接，JdbcTemplate会以自己内部的dataSource去上面的threadlocal中找有没有关联的连接，如果有直接拿来用，若没找到将重新创建一个连接，而此时是可以找到的，那么JdbcTemplate就参与到spring的事务中了。

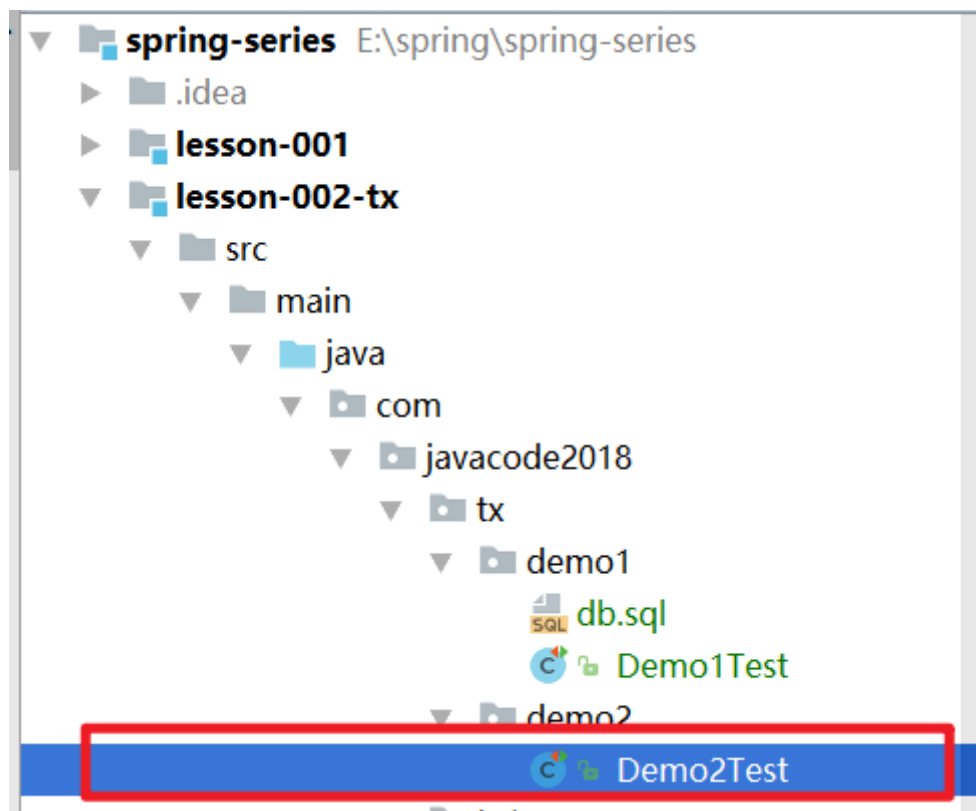
步骤5：提交 or 回滚

```
//5.提交事务: platformTransactionManager.commit  
platformTransactionManager.commit(transactionStatus);  
  
//6.回滚事务: platformTransactionManager.rollback  
platformTransactionManager.rollback(transactionStatus);
```

方式2：TransactionTemplate

方式1中部分代码是可以重用的，所以spring对其进行了优化，采用模板方法模式就其进行封装，主要省去了提交或者回滚事务的代码。

案例代码位置



测试代码

```
@Test
public void test1() throws Exception {
    //定义一个数据源
    org.apache.tomcat.jdbc.pool.DataSource dataSource = new
    org.apache.tomcat.jdbc.pool.DataSource();
    dataSource.setDriverClassName("com.mysql.jdbc.Driver");
    dataSource.setUrl("jdbc:mysql://localhost:3306/javacode2018?
    characterEncoding=UTF-8");
    dataSource.setUsername("root");
    dataSource.setPassword("root123");
    dataSource.setInitialSize(5);
    //定义一个JdbcTemplate，用来方便执行数据库增删改查
    JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);
    //1.定义事务管理器，给其指定一个数据源（可以把事务管理器想象为一个人，这个人来负责事务的控制操作）
    PlatformTransactionManager platformTransactionManager = new
    DataSourceTransactionManager(dataSource);
    //2.定义事务属性：TransactionDefinition，TransactionDefinition可以用来配置事务的属性信息，比如事务隔离级别、事务超时时间、事务传播方式、是否是只读事务等等。
    DefaultTransactionDefinition transactionDefinition = new
    DefaultTransactionDefinition();
    transactionDefinition.setTimeout(10); //如：设置超时时间10s
    //3.创建TransactionTemplate对象
    TransactionTemplate transactionTemplate = new
    TransactionTemplate(platformTransactionManager, transactionDefinition);
    /**
     * 4.通过TransactionTemplate提供的方法执行业务操作
     * 主要有2个方法：
     * （1）.executeWithoutResult(Consumer<TransactionStatus> action)：没有返回值的，需传递一个Consumer对象，在accept方法中做业务操作
     * （2）.<T> T execute(TransactionCallback<T> action)：有返回值的，需要传递一个TransactionCallback对象，在doInTransaction方法中做业务操作
     */
}
```

```

    * 调用execute方法或者executeWithoutResult方法执行完毕之后，事务管理器会自动提交事务或者回滚事务。
    * 那么什么时候事务会回滚，有2种方式：
    * （1）transactionStatus.setRollbackOnly();将事务状态标注为回滚状态
    * （2）execute方法或者executeWithoutResult方法内部抛出异常
    * 什么时候事务会提交？
    * 方法没有异常 && 未调用过transactionStatus.setRollbackOnly();
    */
    transactionTemplate.executeWithoutResult(new Consumer<TransactionStatus>() {
        @Override
        public void accept(TransactionStatus transactionStatus) {
            jdbcTemplate.update("insert into t_user (name) values (?)",
"transactionTemplate-1");
            jdbcTemplate.update("insert into t_user (name) values (?)",
"transactionTemplate-2");
        }
    });
    System.out.println("after:" + jdbcTemplate.queryForList("SELECT * from
t_user"));
}

```

运行输出

```
after:[{id=1, name=transactionTemplate-1}, {id=2, name=transactionTemplate-2}]
```

代码分析

TransactionTemplate，主要有2个方法：

executeWithoutResult：无返回值场景

executeWithoutResult(Consumer<TransactionStatus> action)：没有返回值的，需传递一个Consumer对象，在accept方法中做业务操作

```

transactionTemplate.executeWithoutResult(new Consumer<TransactionStatus>() {
    @Override
    public void accept(TransactionStatus transactionStatus) {
        //执行业务操作
    }
});

```

execute：有返回值场景

<T> T execute(TransactionCallback<T> action)：有返回值的，需要传递一个TransactionCallback对象，在doInTransaction方法中做业务操作

```

Integer result = transactionTemplate.execute(new TransactionCallback<Integer>()
{
    @Nullable
    @Override
    public Integer doInTransaction(TransactionStatus status) {
        return jdbcTemplate.update("insert into t_user (name) values (?)",
            "executewithoutResult-3");
    }
});

```

通过上面2个方法，事务管理器会自动提交事务或者回滚事务。

什么时候事务会回滚，有2种方式

方式1

在execute或者executeWithoutResult内部执行 `transactionStatus.setRollbackOnly()`; 将事务状态标注为回滚状态，spring会自动让事务回滚

方式2

execute方法或者executeWithoutResult方法内部抛出任意异常即可回滚。

什么时候事务会提交？

方法没有异常 && 未调用过 `transactionStatus.setRollbackOnly()`;

编程式事务正确的使用姿势

如果大家确实想在系统中使用编程式事务，那么可以参考下面代码，使用spring来管理对象，更简洁一些。

先来个配置类，将事务管理器 `PlatformTransactionManager`、事务模板 `TransactionTemplate` 都注册到spring中，重用。

```

package com.javacode2018.tx.demo3;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.support.TransactionTemplate;

import javax.sql.DataSource;

@Configuration
@ComponentScan
public class MainConfig3 {
    @Bean
    public DataSource dataSource() {

```

```

        org.apache.tomcat.jdbc.pool.DataSource dataSource = new
org.apache.tomcat.jdbc.pool.DataSource();
        dataSource.setDriverClassName("com.mysql.jdbc.Driver");
        dataSource.setUrl("jdbc:mysql://localhost:3306/javacode2018?
characterEncoding=UTF-8");
        dataSource.setUsername("root");
        dataSource.setPassword("root123");
        dataSource.setInitialSize(5);
        return dataSource;
    }

    @Bean
    public JdbcTemplate jdbcTemplate(DataSource dataSource) {
        return new JdbcTemplate(dataSource);
    }

    @Bean
    public PlatformTransactionManager transactionManager(DataSource dataSource)
{
        return new DataSourceTransactionManager(dataSource);
    }

    @Bean
    public TransactionTemplate transactionTemplate(PlatformTransactionManager
transactionManager) {
        return new TransactionTemplate(transactionManager);
    }
}

```

通常我们会将业务操作放在service中，所以我们也来个service：UserService。

```

package com.javacode2018.tx.demo3;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;
import org.springframework.transaction.support.TransactionTemplate;

import java.util.List;

@Component
public class UserService {
    @Autowired
    private JdbcTemplate jdbcTemplate;
    @Autowired
    private TransactionTemplate transactionTemplate;

    //模拟业务操作1
    public void bus1() {
        this.transactionTemplate.executeWithoutResult(transactionStatus -> {
            //先删除表数据
            this.jdbcTemplate.update("delete from t_user");
            //调用bus2
            this.bus2();
        });
    }
}

```

```

//模拟业务操作2
public void bus2() {
    this.transactionTemplate.executewithoutResult(transactionStatus -> {
        this.jdbcTemplate.update("insert into t_user (name) VALUE (?)",
"java");
        this.jdbcTemplate.update("insert into t_user (name) VALUE (?)",
"spring");
        this.jdbcTemplate.update("insert into t_user (name) VALUE (?)",
"mybatis");
    });
}

//查询表中所有数据
public List userList() {
    return jdbcTemplate.queryForList("select * from t_user");
}
}

```

bus1中会先删除数据，然后调用bus2，此时bus1中的所有操作和bus2中的所有操作会被放在一个事务中执行，这是spring内部默认实现的，bus1中调用 `executewithoutResult` 的时候，会开启一个事务，而内部又会调用bus2，而bus2内部也调用了 `executewithoutResult`，bus内部会先判断一下上线文环境中有没有事务，如果有就直接参与到已存在的事务中，刚好发现有bus1已开启的事务，所以就直接参与到bus1的事务中了，最终bus1和bus2会在一个事务中运行。

上面bus1代码转换为sql脚本如下：

```

start transaction; //开启事务
delete from t_user;
insert into t_user (name) VALUE ('java');
insert into t_user (name) VALUE ('spring');
insert into t_user (name) VALUE ('mybatis');
commit;

```

来个测试案例，看一下效果

```

package com.javacode2018.tx.demo3;

import org.junit.Test;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Demo3Test {
    @Test
    public void test1() {
        AnnotationConfigApplicationContext context = new
AnnotationConfigApplicationContext(MainConfig3.class);
        UserService userService = context.getBean(UserService.class);
        userService.bus1();
        System.out.println(userService.userList());
    }
}

```

运行 test1() 输出

```
[{id=18, name=java}, {id=19, name=spring}, {id=20, name=mybatis}]
```


上面代码中，bus1或者bus2中，如果有异常或者执行 `transactionStatus.setRollbackOnly()`，此时整个事务都会回滚，大家可以去试试！

总结一下

大家看了之后，会觉得这样用好复杂啊，为什么要这么玩？

的确，看起来比较复杂，代码中融入了大量spring的代码，耦合性比较强，不利于扩展，本文的目标并不是让大家以后就这么用，主要先让大家从硬编码上了解spring中事务是如何控制的，后面学起来才会更容易。

我们用的最多的是声明式事务，声明式事务的底层还是使用上面这种方式来控制事务的，只不过对其进行封装，让我们用起来更容易些。

下篇文章将详解声明式事务的使用。

案例源码

git地址：

<https://gitee.com/javacode2018/spring-series>

本文案例对应源码模块：`lesson-002-tx`

路人甲java所有案例代码以后都会放到这个上面，大家watch一下，可以持续关注动态。

Spring系列

1. [Spring系列第1篇：为何要学spring？](#)
2. [Spring系列第2篇：控制反转（IoC）与依赖注入（DI）](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)
4. [Spring系列第4篇：xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇：创建bean实例这些方式你们都知道？](#)
6. [Spring系列第6篇：玩转bean scope，避免跳坑里！](#)
7. [Spring系列第7篇：依赖注入之手动注入](#)
8. [Spring系列第8篇：自动注入（autowire）详解，高手在于坚持](#)
9. [Spring系列第9篇：depend-on到底是干什么的？](#)
10. [Spring系列第10篇：primary可以解决什么问题？](#)
11. [Spring系列第11篇：bean中的autowire-candidate又是干什么的？](#)
12. [Spring系列第12篇：lazy-init：bean延迟初始化](#)
13. [Spring系列第13篇：使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇：lookup-method和replaced-method比较陌生，怎么玩的？](#)
15. [Spring系列第15篇：代理详解（Java动态代理&cglib代理）？](#)
16. [Spring系列第16篇：深入理解java注解及spring对注解的增强（预备知识）](#)
17. [Spring系列第17篇：@Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇：@ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇：@import详解\(bean批量注册\)](#)
20. [Spring系列第20篇：@Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇：注解实现依赖注入（@Autowired、@Resource、@Primary、@Qualifier）](#)
22. [Spring系列第22篇：@Scope、@DependsOn、@ImportResource、@Lazy 详解](#)

23. [Spring系列第23篇：Bean生命周期详解](#)
24. [Spring系列第24篇：父子容器详解](#)
25. [Spring系列第25篇：@Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇：国际化详解](#)
27. [Spring系列第27篇：spring事件机制详解](#)
28. [Spring系列第28篇：Bean循环依赖详解](#)
29. [Spring系列第29篇：BeanFactory扩展（BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor）](#)
30. [Spring系列第30篇：jdk动态代理和cglib代理](#)
31. [Spring系列第31篇：aop概念详解](#)
32. [Spring系列第32篇：AOP核心源码、原理详解](#)
33. [Spring系列第33篇：ProxyFactoryBean创建AOP代理](#)
34. [Spring系列第34篇：@Aspect中@Pointcut 12种用法](#)
35. [Spring系列第35篇：@Aspect中5中通知详解](#)
36. [Spring系列第36篇：@EnableAspectJAutoProxy、@Aspect中通知顺序详解](#)
37. [Spring系列第37篇：@EnableAsync & @Async 实现方法异步调用](#)
38. [Spring系列第38篇：@Scheduled & @EnableScheduling定时器详解](#)
39. [Spring系列第39篇：强大的Spel表达式](#)
40. [Spring系列第40篇：缓存使用（@EnableCaching、@Cacheable、@CachePut、@CacheEvict、@Caching、@CacheConfig）](#)
41. [Spring系列第41篇：@EnableCaching集成redis缓存](#)
42. [Spring系列第42篇：玩转JdbcTemplate](#)

更多好文章

1. [Java高并发系列（共34篇）](#)
2. [MySql高手系列（共27篇）](#)
3. [Maven高手系列（共10篇）](#)
4. [Mybatis系列（共12篇）](#)
5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要，它是什么？怎么实现？](#)
7. [泛型，有点难度，会让很多人懵逼，那是因为你没有看这篇文章！](#)

加我微信：spring888java，进群一起交流java相关各种技术！！

Spring系列第44篇：spring声明事务怎么用的？

spring事务有2种用法：**编程式事务**和**声明式事务**。

编程式事务[上一篇](#)文章中已经介绍了，不熟悉的建议先看一下编程式事务的用法。

这篇主要介绍声明式事务的用法，我们在工作中基本上用的都是声明式事务，所以这篇文章是比较重要的，建议各位打起精神，正式开始。

什么是声明式事务？

所谓声明式事务，就是通过配置的方式，比如通过配置文件（xml）或者注解的方式，告诉spring，哪些方法需要spring帮忙管理事务，然后开发者只用关注业务代码，而事务的事情spring自动帮我们控制。

比如注解的方式，只需在方法上面加一个 @Transaction 注解，那么方法执行之前spring会自动开启一个事务，方法执行完毕之后，会自动提交或者回滚事务，而方法内部没有任何事务相关代码，用起来特别的方法。

```
@Transaction
public void insert(String userName){
    this.jdbcTemplate.update("insert into t_user (name) values (?)", userName);
}
```

声明式事务的2种实现方式

1. **配置文件的方式**，即在spring xml文件中进行统一配置，开发者基本上就不用关注事务的事情了，代码中无需关心任何和事务相关的代码，一切交给spring处理。
2. **注解的方式**，只需在需要spring来帮忙管理事务的方法上加上@Transaction注解就可以了，注解的方式相对来说更简洁一些，都需要开发者自己去进行配置，可能有些同学对spring不是太熟悉，所以配置这个有一定的风险，做好代码review就可以了。

配置文件的方式这里就不讲了，用的相对比较少，我们主要掌握注解的方式如何使用，就可以了。

声明式事务注解方式5个步骤

1、启用Spring的注释驱动事务管理功能

在spring配置类上加上 @EnableTransactionManagement 注解

```
@EnableTransactionManagement
public class MainConfig4 {
}
```

简要介绍一下原理：当spring容器启动的时候，发现有@EnableTransactionManagement注解，此时会拦截所有bean的创建，扫描看一下bean上是否有@Transaction注解（类、或者父类、或者接口、或者方法中有这个注解都可以），如果有这个注解，spring会通过aop的方式给bean生成代理对象，代理对象中会增加一个拦截器，拦截器会拦截bean中public方法执行，会在方法执行之前启动事务，方法执行完毕之后提交或者回滚事务。稍后会专门有一篇文章带大家看这块的源码。

如果有兴趣的可以自己先去读一下源码，主要是下面这个这方法会

```
org.springframework.transaction.interceptor.TransactionInterceptor#invoke
```

再看看 EnableTransactionManagement 的源码

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Import(TransactionManagementConfigurationSelector.class)
public @interface EnableTransactionManagement {
}
```

```

/**
 * spring是通过aop的方式对bean创建代理对象来实现事务管理的
 * 创建代理对象有2种方式，jdk动态代理和cglib代理
 * proxyTargetClass: 为true的时候，就是强制使用cglib来创建代理
 */
boolean proxyTargetClass() default false;

/**
 * 用来指定事务拦截器的顺序
 * 我们知道一个方法上可以添加很多拦截器，拦截器是可以指定顺序的
 * 比如你可以自定义一些拦截器，放在事务拦截器之前或者之后执行，就可以通过order来控制
 */
int order() default Ordered.LOWEST_PRECEDENCE;
}

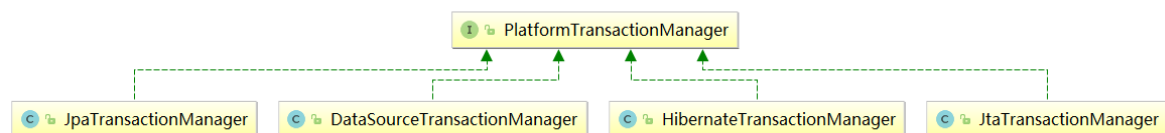
```

2、定义事务管理器

事务交给spring管理，那么你一定要创建一个或者多个事务管理者，有这些管理者来管理具体的事务，比如启动事务、提交事务、回滚事务，这些都是管理者来负责的。

spring中使用PlatformTransactionManager这个接口来表示事务管理者。

PlatformTransactionManager多个实现类，用来应对不同的环境



JpaTransactionManager：如果你用jpa来操作db，那么需要用这个管理器来帮你控制事务。

DataSourceTransactionManager：如果你用是指定数据源的方式，比如操作数据库用的是：JdbcTemplate、mybatis、ibatis，那么需要用这个管理器来帮你控制事务。

HibernateTransactionManager：如果你用hibernate来操作db，那么需要用这个管理器来帮你控制事务。

JtaTransactionManager：如果你用的是java中的jta来操作db，这种通常是分布式事务，此时需要用这种管理器来控制事务。

比如：我们用的是mybatis或者jdbctemplate，那么通过下面方式定义一个事务管理器。

```

@Bean
public PlatformTransactionManager transactionManager(DataSource dataSource) {
    return new DataSourceTransactionManager(dataSource);
}

```

3、需使用事务的目标上加@Transactional注解

- @Transactional放在接口上，那么接口的实现类中所有public都被spring自动加上事务
- @Transactional放在类上，那么当前类以及其下无限级子类中所有public方法将被spring自动加上事务
- @Transactional放在public方法上，那么该方法将被spring自动加上事务
- 注意：@Transactional只对public方法有效

下面我们看一下@Transactional源码：

```

@Target({ElementType.TYPE, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
@Inherited
@Documented
public @interface Transactional {

    /**
     * 指定事务管理器的bean名称，如果容器中有多个事务管理器PlatformTransactionManager，
     * 那么你得告诉spring，当前配置需要使用哪个事务管理器
     */
    @AliasFor("transactionManager")
    String value() default "";

    /**
     * 同value，value和transactionManager选配一个就行，也可以为空，如果为空，默认会从容器
    中按照类型查找一个事务管理器bean
     */
    @AliasFor("value")
    String transactionManager() default "";

    /**
     * 事务的传播属性
     */
    Propagation propagation() default Propagation.REQUIRED;

    /**
     * 事务的隔离级别，就是制定数据库的隔离级别，数据库隔离级别大家知道么？不知道的可以去补一下
     */
    Isolation isolation() default Isolation.DEFAULT;

    /**
     * 事务执行的超时时间（秒），执行一个方法，比如有问题，那我不可能等你一天吧，可能最多我只能
    等你10秒
     * 10秒后，还没有执行完毕，就弹出一个超时异常吧
     */
    int timeout() default TransactionDefinition.TIMEOUT_DEFAULT;

    /**
     * 是否是只读事务，比如某个方法中只有查询操作，我们可以指定事务是只读的
     * 设置了这个参数，可能数据库会做一些性能优化，提升查询速度
     */
    boolean readOnly() default false;

    /**
     * 定义零(0)个或多个异常类，这些异常类必须是Throwable的子类，当方法抛出这些异常及其子类异
    常的时候，spring会让事务回滚
     * 如果不配做，那么默认会在 RuntimeException 或者 Error 情况下，事务才会回滚
     */
    Class<? extends Throwable>[] rollbackFor() default {};

    /**
     * 和 rollbackFor 作用一样，只是这个地方使用的是类名
     */
    String[] rollbackForClassName() default {};

    /**
     * 定义零(0)个或多个异常类，这些异常类必须是Throwable的子类，当方法抛出这些异常的时候，事
    务不会回滚

```

```

    */
    Class<? extends Throwable>[] noRollbackFor() default {};

    /**
     * 和 noRollbackFor 作用一样，只是这个地方使用的是类名
     */
    String[] noRollbackForClassName() default {};

}

```

参数介绍

参数	描述
value	指定事务管理器的bean名称，如果容器中有多事务管理器 PlatformTransactionManager，那么你得告诉spring，当前配置需要使用哪个事务管理器
transactionManager	同value，value和transactionManager选配一个就行，也可以为空，如果为空，默认会从容器中按照类型查找一个事务管理器bean
propagation	事务的传播属性，下篇文章详细介绍
isolation	事务的隔离级别，就是制定数据库的隔离级别，数据库隔离级别大家知道么？不知道的可以去补一下
timeout	事务执行的超时时间（秒），执行一个方法，比如有问题，那我不可能等你一天吧，可能最多我只能等你10秒 10秒后，还没有执行完毕，就弹出一个超时异常吧
readOnly	是否是只读事务，比如某个方法中只有查询操作，我们可以指定事务是只读的 设置了这个参数，可能数据库会做一些性能优化，提升查询速度
rollbackFor	定义零(0)个或更多异常类，这些异常类必须是Throwable的子类，当方法抛出这些异常及其子类异常的时候，spring会让事务回滚 如果不配做，那么默认会在 RuntimeException 或者 Error 情况下，事务才会回滚
rollbackForClassName	同 rollbackFor，只是这个地方使用的是类名
noRollbackFor	定义零(0)个或更多异常类，这些异常类必须是Throwable的子类，当方法抛出这些异常的时候，事务不会回滚
noRollbackForClassName	同 noRollbackFor，只是这个地方使用的是类名

4、执行db业务操作

在@Transaction标注类或者目标方法上执行业务操作，此时这些方法会自动被spring进行事务管理。

如，下面的insertBatch操作，先删除数据，然后批量插入数据，方法上加上了@Transactional注解，此时这个方法会自动受spring事务控制，要么都成功，要么都失败。

```

@Component
public class UserService {
    @Autowired
    private JdbcTemplate jdbcTemplate;
}

```

```
//先清空表中数据，然后批量插入数据，要么都成功要么都失败
@Transactional
public void insertBatch(String... names) {
    jdbcTemplate.update("truncate table t_user");
    for (String name : names) {
        jdbcTemplate.update("INSERT INTO t_user(name) VALUES (?)", name);
    }
}
}
```

5、启动spring容器，使用bean执行业务操作

```
@Test
public void test1() {
    AnnotationConfigApplicationContext context = new
    AnnotationConfigApplicationContext();
    context.register(MainConfig4.class);
    context.refresh();

    UserService userService = context.getBean(UserService.class);
    userService.insertBatch("java高并发系列", "mysql系列", "maven系列", "mybatis系
列");
}
```

案例1

准备数据库

```
DROP DATABASE IF EXISTS javacode2018;
CREATE DATABASE IF NOT EXISTS javacode2018;

USE javacode2018;
DROP TABLE IF EXISTS t_user;
CREATE TABLE t_user(
    id int PRIMARY KEY AUTO_INCREMENT,
    name varchar(256) NOT NULL DEFAULT '' COMMENT '姓名'
);
```

spring配置类

```
package com.javacode2018.tx.demo4;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;
import org.springframework.transaction.support.TransactionTemplate;
```



```

import javax.sql.DataSource;

@EnableTransactionManagement //@1
@Configuration
@ComponentScan
public class MainConfig4 {
    //定义一个数据源
    @Bean
    public DataSource dataSource() {
        org.apache.tomcat.jdbc.pool.DataSource dataSource = new
org.apache.tomcat.jdbc.pool.DataSource();
        dataSource.setDriverClassName("com.mysql.jdbc.Driver");
        dataSource.setUrl("jdbc:mysql://localhost:3306/javacode2018?
characterEncoding=UTF-8");
        dataSource.setUsername("root");
        dataSource.setPassword("root123");
        dataSource.setInitialSize(5);
        return dataSource;
    }

    //定义一个JdbcTemplate,用来执行db操作
    @Bean
    public JdbcTemplate jdbcTemplate(DataSource dataSource) {
        return new JdbcTemplate(dataSource);
    }

    //定义我一个事物管理器
    @Bean
    public PlatformTransactionManager transactionManager(DataSource dataSource)
{ //@2
        return new DataSourceTransactionManager(dataSource);
    }
}

```

@1: 使用@EnableTransactionManagement注解开启spring事务管理

@2: 定义事务管理器

来个业务类

```

package com.javacode2018.tx.demo4;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.Map;

@Component
public class UserService {
    @Autowired
    private JdbcTemplate jdbcTemplate;

    //先清空表中数据，然后批量插入数据，要么都成功要么都失败
    @Transactional //@1

```



```

    public int insertBatch(String... names) {
        int result = 0;
        jdbcTemplate.update("truncate table t_user");
        for (String name : names) {
            result += jdbcTemplate.update("INSERT INTO t_user(name) VALUES (?)",
name);
        }
        return result;
    }

    //获取所有用户信息
    public List<Map<String, Object>> userList() {
        return jdbcTemplate.queryForList("SELECT * FROM t_user");
    }
}

```

@1: insertBatch方法上加上了@Transactional注解，让spring来自动为这个方法加上事务

测试类

```

package com.javacode2018.tx.demo4;

import org.junit.Test;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Demo4Test {
    @Test
    public void test1() {
        AnnotationConfigApplicationContext context = new
AnnotationConfigApplicationContext();
        context.register(MainConfig4.class);
        context.refresh();

        UserService userService = context.getBean(UserService.class);
        //先执行插入操作
        int count = userService.insertBatch(
            "java高并发系列",
            "mysql系列",
            "maven系列",
            "mybatis系列");
        System.out.println("插入成功（条）：" + count);
        //然后查询一下
        System.out.println(userService.userList());
    }
}

```

运行输出

```

插入成功（条）：4
[[{id=1, name=java高并发系列}, {id=2, name=mysql系列}, {id=3, name=maven系列}, {id=4,
name=mybatis系列}]

```

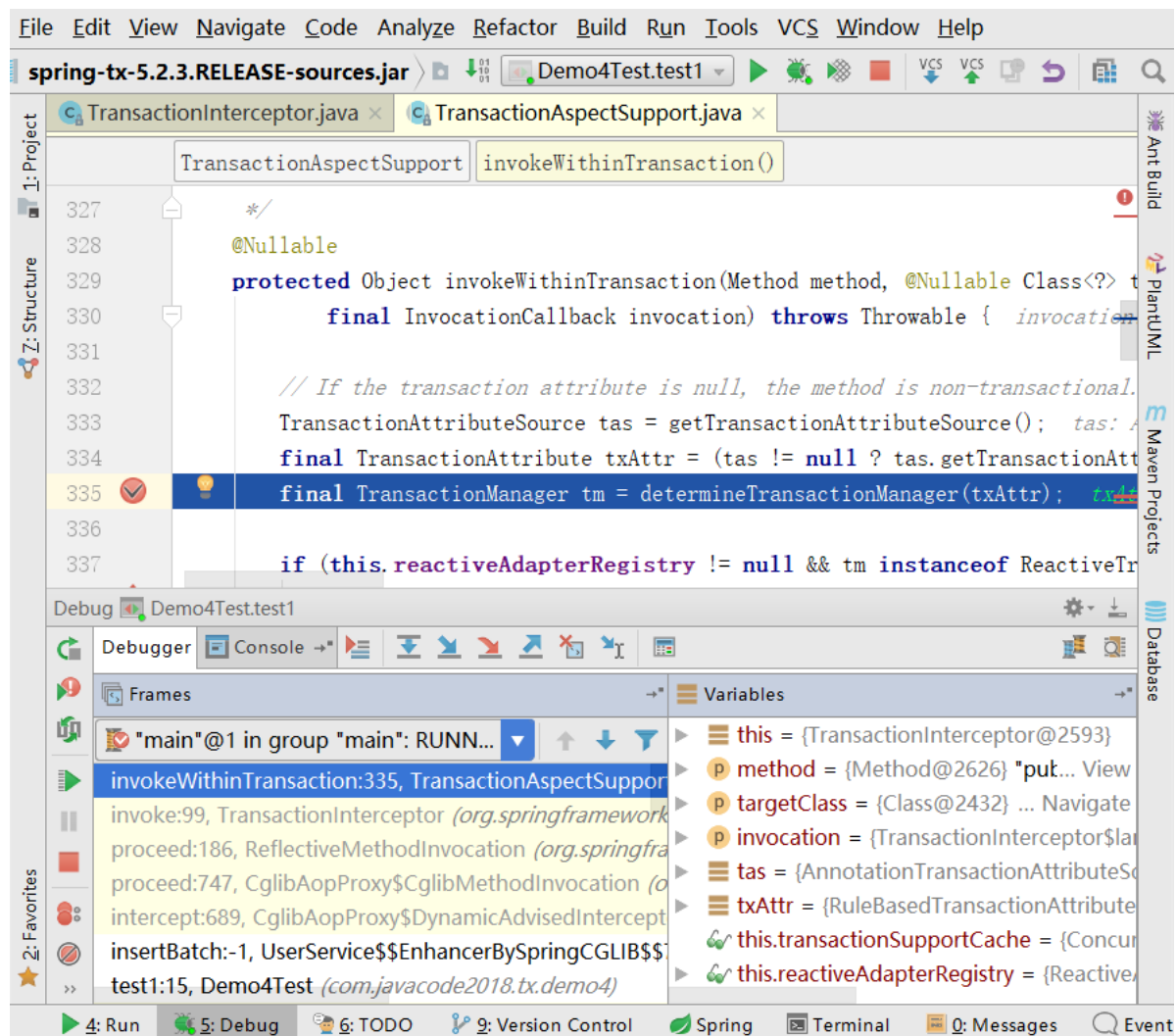
有些朋友可能会问，如何知道这个被调用的方法有没有使用事务？下面我们就来看一下。

如何确定方法有没有用到spring事务

方式1：断点调试

spring事务是由TransactionInterceptor拦截器处理的，最后会调用下面这个方法，设置个断点就可以看到详细过程了。

```
org.springframework.transaction.interceptor.TransactionAspectSupport#invokeWithinTransaction
```



方式2：看日志

spring处理事务的过程，有详细的日志输出，开启日志，控制台就可以看到事务的详细过程了。

添加maven配置

```
<dependency>
  <groupId>ch.qos.logback</groupId>
  <artifactId>logback-classic</artifactId>
  <version>1.2.3</version>
</dependency>
```

src\main\resources新建logback.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <pattern>[%d{MM-dd HH:mm:ss.SSS}][%thread{20}:%${PID:- }][%X{trace_id}][%level][%logger{56}:%line:%method\
(\)]:%msg%n#####n#####n</pattern>
    </encoder>
  </appender>

  <logger name="org.springframework" level="debug">
    <appender-ref ref="STDOUT" />
  </logger>
</configuration>
```

再来运行一下案例1

```
[09-10 11:20:38.830][main: ][][DEBUG]
[o.s.jdbc.datasource.DataSourceTransactionManager:370:getTransaction()]:Creating
new transaction with name [com.javacode2018.tx.demo4.UserService.insertBatch]:
PROPAGATION_REQUIRED, ISOLATION_DEFAULT
#####
[09-10 11:20:39.120][main: ][][DEBUG]
[o.s.jdbc.datasource.DataSourceTransactionManager:265:doBegin()]:Acquired
Connection
[ProxyConnection[PooledConnection[com.mysql.jdbc.JDBC4Connection@65fe9e33]]] for
JDBC transaction
#####
[09-10 11:20:39.125][main: ][][DEBUG]
[o.s.jdbc.datasource.DataSourceTransactionManager:283:doBegin()]:Switching JDBC
Connection
[ProxyConnection[PooledConnection[com.mysql.jdbc.JDBC4Connection@65fe9e33]]] to
manual commit
#####
[09-10 11:20:39.139][main: ][][DEBUG]
[org.springframework.jdbc.core.JdbcTemplate:502:update()]:Executing SQL update
[truncate table t_user]
#####
[09-10 11:20:39.169][main: ][][DEBUG]
[org.springframework.jdbc.core.JdbcTemplate:860:update()]:Executing prepared SQL
update
#####
[09-10 11:20:39.169][main: ][][DEBUG]
[org.springframework.jdbc.core.JdbcTemplate:609:execute()]:Executing prepared
SQL statement [INSERT INTO t_user(name) VALUES (?)]
#####
[09-10 11:20:39.234][main: ][][DEBUG]
[org.springframework.jdbc.core.JdbcTemplate:860:update()]:Executing prepared SQL
update
#####
[09-10 11:20:39.235][main: ][][DEBUG]
[org.springframework.jdbc.core.JdbcTemplate:609:execute()]:Executing prepared
SQL statement [INSERT INTO t_user(name) VALUES (?)]
```

```

#####
[09-10 11:20:39.236][main: ][DEBUG]
[org.springframework.jdbc.core.JdbcTemplate:860:update()]:Executing prepared SQL
update
#####
[09-10 11:20:39.237][main: ][DEBUG]
[org.springframework.jdbc.core.JdbcTemplate:609:execute()]:Executing prepared
SQL statement [INSERT INTO t_user(name) VALUES (?)]
#####
[09-10 11:20:39.238][main: ][DEBUG]
[org.springframework.jdbc.core.JdbcTemplate:860:update()]:Executing prepared SQL
update
#####
[09-10 11:20:39.239][main: ][DEBUG]
[org.springframework.jdbc.core.JdbcTemplate:609:execute()]:Executing prepared
SQL statement [INSERT INTO t_user(name) VALUES (?)]
#####
[09-10 11:20:39.241][main: ][DEBUG]
[o.s.jdbc.datasource.DataSourceTransactionManager:741:processCommit()]:Initiatin
g transaction commit
#####
[09-10 11:20:39.241][main: ][DEBUG]
[o.s.jdbc.datasource.DataSourceTransactionManager:328:doCommit()]:Committing
JDBC transaction on Connection
[ProxyConnection[PooledConnection[com.mysql.jdbc.JDBC4Connection@65fe9e33]]]
#####
[09-10 11:20:39.244][main: ][DEBUG]
[o.s.jdbc.datasource.DataSourceTransactionManager:387:doCleanupAfterCompletion()
]:Releasing JDBC Connection
[ProxyConnection[PooledConnection[com.mysql.jdbc.JDBC4Connection@65fe9e33]]]
after transaction
#####
插入成功（条）：4
[09-10 11:20:39.246][main: ][DEBUG]
[org.springframework.jdbc.core.JdbcTemplate:427:query()]:Executing SQL query
[SELECT * FROM t_user]
#####
[09-10 11:20:39.247][main: ][DEBUG]
[org.springframework.jdbc.datasource.DataSourceUtils:115:doGetConnection()]:Fetc
hing JDBC Connection from DataSource
#####
[{id=1, name=java高并发系列}, {id=2, name=mysql系列}, {id=3, name=maven系列}, {id=4,
name=mybatis系列}]

```

来理解一下日志

insertBatch方法上有@Transaction注解，所以会被拦截器拦截，下面是在insertBatch方法调用之前，创建了一个事务。

```

36ms [09-10 11:20:38.790][main: ][DEBUG][o.s.beans.factory.support
.DefaultListableBeanFactory:806:createArgumentArray()]:Autowiring by type from bean name
36ms 'transactionManager' via factory method to bean named 'dataSource'
#####
[09-10 11:20:38.830][main: ][DEBUG][o.s.jdbc.datasource
.DataSourceTransactionManager:370:getTransaction()]:Creating new transaction with name
com.javacode2018.tx.demo4.UserService.insertBatch: PROPAGATION_REQUIRED, ISOLATION_DEFAULT
#####
Thu Sep 10 11:20:38 CST 2020 WARN: Establishing SSL connection without server's identity
verification is not recommended. According to MySQL 5.5.45+, 5.6.26+ and 5.7.6+ requirements

```

insertBatch方法上@Transaction注解参数都是默认值，@Transaction注解中可以通过 value或者 transactionManager 来指定事务管理器，但是没有指定，此时spring会在容器中按照事务管理器类型找一个默认的，刚好我们在spring容器中定义了一个，所以直接拿来用了。事务管理器我们用的是 new DataSourceTransactionManager(dataSource)，从事务管理器的datasource中获取一个数据库连接，然后通过连接设置事务为手动提交，然后将 (datasource->这个连接)丢到ThreadLocal中了，具体为什么，可以看[上一篇文章](#)。

```
[09-10 11:20:39.120][main: ][][DEBUG][o.s.jdbc.datasource
.DataSourceTransactionManager:265:doBegin()]:Acquired Connection
[ProxyConnection[PooledConnection[com.mysql.jdbc.JDBC4Connection@65fe9e33]]] for JDBC
transaction
#####
[09-10 11:20:39.125][main: ][][DEBUG][o.s.jdbc.datasource
.DataSourceTransactionManager:283:doBegin()]:Switching JDBC Connection
[ProxyConnection[PooledConnection[com.mysql.jdbc.JDBC4Connection@65fe9e33]]] to manual commit
#####
```

从事务管理器的datasource中获取一个数据库连接

开启connection手动提交事务

下面就正是进入insertBatch方法内部了，通过jdbcTemplate执行一些db操作，jdbcTemplate内部会通过datasource到上面的threadlocal中拿到spring事务那个连接，然后执行db操作。

```
#####
[09-10 11:20:39.159][main: ][][DEBUG][org.springframework.jdbc.core.JdbcTemplate:502:update()
]:Executing SQL update [truncate table t_user]
#####
[09-10 11:20:39.169][main: ][][DEBUG][org.springframework.jdbc.core.JdbcTemplate:860:update()
]:Executing prepared SQL update
#####
[09-10 11:20:39.169][main: ][][DEBUG][org.springframework.jdbc.core.JdbcTemplate:609:execute()
]:Executing prepared SQL statement [INSERT INTO t_user(name) VALUES (?)]
#####
[09-10 11:20:39.234][main: ][][DEBUG][org.springframework.jdbc.core.JdbcTemplate:860:update()
]:Executing prepared SQL update
#####
[09-10 11:20:39.235][main: ][][DEBUG][org.springframework.jdbc.core.JdbcTemplate:609:execute()
]:Executing prepared SQL statement [INSERT INTO t_user(name) VALUES (?)]
#####
[09-10 11:20:39.236][main: ][][DEBUG][org.springframework.jdbc.core.JdbcTemplate:860:update()
]:Executing prepared SQL update
#####
[09-10 11:20:39.237][main: ][][DEBUG][org.springframework.jdbc.core.JdbcTemplate:609:execute()
]:Executing prepared SQL statement [INSERT INTO t_user(name) VALUES (?)]
#####
[09-10 11:20:39.238][main: ][][DEBUG][org.springframework.jdbc.core.JdbcTemplate:860:update()
]:Executing prepared SQL update
#####
```

通过jdbcTemplate执行db操作

最后insertBatch方法执行完毕之后，没有任何异常，那么spring就开始通过数据库连接提交事务了。

```
[09-10 11:20:39.241][main: ][][DEBUG][o.s.jdbc.datasource
.DataSourceTransactionManager:741:processCommit()]:Initiating transaction commit
#####
[09-10 11:20:39.241][main: ][][DEBUG][o.s.jdbc.datasource
.DataSourceTransactionManager:328:doCommit()]:Committing JDBC transaction on Connection
[ProxyConnection[PooledConnection[com.mysql.jdbc.JDBC4Connection@65fe9e33]]]
#####
[09-10 11:20:39.244][main: ][][DEBUG][o.s.jdbc.datasource
.DataSourceTransactionManager:387:doCleanupAfterCompletion()]:Releasing JDBC Connection
[ProxyConnection[PooledConnection[com.mysql.jdbc.JDBC4Connection@65fe9e33]]] after transaction
#####
```

开始启动事务提交

通过connection提交事务

释放数据库连接

总结

本文讲解了一下spring中编程式事务的使用步骤。

主要涉及到了2个注解：

@EnableTransactionManagement：开启spring事务管理功能

@Transaction：将其加在需要spring管理事务的类、方法、接口上，只会对public方法有效。

大家再消化一下，有问题，欢迎留言交流。

下篇文章将详细介绍事务的传播属性，敬请期待。

案例源码

git地址：

<https://gitee.com/javacode2018/spring-series>

本文案例对应源码：`spring-series\lesson-002-tx\src\main\java\com\javacode2018\tx\demo4`

路人甲java所有案例代码以后都会放到这个上面，大家watch一下，可以持续关注动态。

Spring系列

1. [Spring系列第1篇：为何要学spring？](#)
2. [Spring系列第2篇：控制反转（IoC）与依赖注入（DI）](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)
4. [Spring系列第4篇：xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇：创建bean实例这些方式你们都知道？](#)
6. [Spring系列第6篇：玩转bean scope，避免跳坑里！](#)
7. [Spring系列第7篇：依赖注入之手动注入](#)
8. [Spring系列第8篇：自动注入（autowire）详解，高手在于坚持](#)
9. [Spring系列第9篇：depend-on到底是干什么的？](#)
10. [Spring系列第10篇：primary可以解决什么问题？](#)
11. [Spring系列第11篇：bean中的autowire-candidate又是干什么的？](#)
12. [Spring系列第12篇：lazy-init：bean延迟初始化](#)
13. [Spring系列第13篇：使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇：lookup-method和replaced-method比较陌生，怎么玩的？](#)
15. [Spring系列第15篇：代理详解（Java动态代理&cglib代理）？](#)
16. [Spring系列第16篇：深入理解java注解及spring对注解的增强（预备知识）](#)
17. [Spring系列第17篇：@Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇：@ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇：@import详解\(bean批量注册\)](#)
20. [Spring系列第20篇：@Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇：注解实现依赖注入（@Autowired、@Resource、@Primary、@Qualifier）](#)
22. [Spring系列第22篇：@Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇：Bean生命周期详解](#)
24. [Spring系列第24篇：父子容器详解](#)
25. [Spring系列第25篇：@Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇：国际化详解](#)

27. [Spring系列第27篇：spring事件机制详解](#)
28. [Spring系列第28篇：Bean循环依赖详解](#)
29. [Spring系列第29篇：BeanFactory扩展（BeanFactoryPostProcessor、
BeanDefinitionRegistryPostProcessor）](#)
30. [Spring系列第30篇：jdk动态代理和cglib代理](#)
31. [Spring系列第31篇：aop概念详解](#)
32. [Spring系列第32篇：AOP核心源码、原理详解](#)
33. [Spring系列第33篇：ProxyFactoryBean创建AOP代理](#)
34. [Spring系列第34篇：@Aspect中@Pointcut 12种用法](#)
35. [Spring系列第35篇：@Aspect中5中通知详解](#)
36. [Spring系列第36篇：@EnableAspectJAutoProxy、@Aspect中通知顺序详解](#)
37. [Spring系列第37篇：@EnableAsync & @Async 实现方法异步调用](#)
38. [Spring系列第38篇：@Scheduled & @EnableScheduling定时器详解](#)
39. [Spring系列第39篇：强大的Spel表达式](#)
40. [Spring系列第40篇：缓存使用（@EnableCaching、@Cacheable、@CachePut、@CacheEvict、
@Caching、@CacheConfig）](#)
41. [Spring系列第41篇：@EnableCaching集成redis缓存](#)
42. [Spring系列第42篇：玩转JdbcTemplate](#)
43. [Spring系列第43篇：spring中编程式事务怎么用的？](#)

更多好文章

1. [Java高并发系列（共34篇）](#)
2. [MySQL高手系列（共27篇）](#)
3. [Maven高手系列（共10篇）](#)
4. [Mybatis系列（共12篇）](#)
5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要，它是什么？怎么实现？](#)
7. [泛型，有点难度，会让很多人懵逼，那是因为你没有看这篇文章！](#)

加我微信：spring888java，进群一起交流java相关各种技术！！

Spring系列第45篇：详解Spring事务中7种传播行为

本文详解Spring事务中的7种传播行为，还是比较重要的。

环境

2. jdk1.8
3. Spring 5.2.3.RELEASE
4. mysql5.7

什么是事务传播行为？

事务的传播行为用来描述：系统中的一些方法交由spring来管理事务，当这些方法之间出现嵌套调用的时候，事务所表现出来的行为是什么样的？

比如下面2个类，Service1中的m1方法和Service2中的m2方法上面都有@Transactional注解，说明这2个方法由spring来控制事务。

但是注意m1中2行代码，先执行了一个insert，然后调用service2中的m2方法，service2中的m2方法也执行了一个insert。

那么大家觉得这2个insert会在一个事务中运行么？也就是说此时事务的表现行为是什么样的呢？这个就是spring事务的传播行为来控制的事情，不同的传播行为，表现会不一样，可能他们会在一个事务中执行，也可能不会在一个事务中执行，这就需要看传播行为的配置了。

```
@Component
public class Service1 {
    @Autowired
    private Service2 service2;

    @Autowired
    private JdbcTemplate jdbcTemplate;

    @Transactional
    public void m1() {
        this.jdbcTemplate.update("INSERT into t1 values ('m1')");
        this.service2.m2();
    }
}

@Component
public class Service2 {
    @Autowired
    private JdbcTemplate jdbcTemplate;

    @Transactional
    public void m2() {
        this.jdbcTemplate.update("INSERT into t1 values ('m2')");
    }
}
```

如何配置事务传播行为？

通过@Transactional注解中的propagation属性来指定事务的传播行为：

```
Propagation propagation() default Propagation.REQUIRED;
```

7种传播行为

Propagation是个枚举，有7种值，如下：

事务传播行为类型	说明
REQUIRED	如果当前事务管理器中没有事务，就新建一个事务，如果已经存在一个事务中，加入到这个事务中。这是最常见的选择，是默认的传播行为。
SUPPORTS	支持当前事务，如果当前事务管理器中没有事务，就以非事务方式执行。
MANDATORY	使用当前的事务，如果当前事务管理器中没有事务，就抛出异常。
REQUIRES_NEW	新建事务，如果当前事务管理器中存在事务，把当前事务挂起，然后会新建一个事务。
NOT_SUPPORTED	以非事务方式执行操作，如果当前事务管理器中存在事务，就把当前事务挂起。
NEVER	以非事务方式执行，如果当前事务管理器中存在事务，则抛出异常。
NESTED	如果当前事务管理器中存在事务，则在嵌套事务内执行；如果当前事务管理器中没有事务，则执行与PROPAGATION_REQUIRED类似的操作。

注意：这7种传播行为有个前提，他们的事务管理器是同一个的时候，才会有上面描述中的表现行为。

下面通过案例对7中表现行为来做说明，在看案例之前，先回顾几个知识点

1、Spring声明式事务处理事务的过程

spring声明式事务是通过事务拦截器TransactionInterceptor拦截目标方法，来实现事务管理的功能的，事务管理器处理过程大致如下：

```
1、获取事务管理器
2、通过事务管理器开启事务
try{
    3、调用业务方法执行db操作
    4、提交事务
}catch(RuntimeException | Error){
    5、回滚事务
}
```

2、何时事务会回滚？

默认情况下，目标方法抛出RuntimeException或者Error的时候，事务会被回滚。

3、Spring事务管理器中的Connection和业务中操作db的Connection如何使用同一个的？

以DataSourceTransactionManager为事务管理器，操作db使用JdbcTemplate来说明一下。

创建DataSourceTransactionManager和JdbcTemplate的时候都需要指定dataSource，需要将他俩的dataSource指定为同一个对象。

当事务管理器开启事务的时候，会通过dataSource.getConnection()方法获取一个db连接connection，然后将dataSource->connection丢到一个Map中，然后将map放到ThreadLocal中。

当JdbcTemplate执行sql的时候，以JdbcTemplate.dataSource去上面的ThreadLocal中查找，是否有可用的连接，如果有，就直接拿来用了，否则调用JdbcTemplate.dataSource.getConnection()方法获取一个连接来用。

所以spring中可以确保事务管理器中的Connection和JdbcTemplate中操作db的Connection是同一个，这样才能确保spring可以控制事务。

代码验证

准备db

```
DROP DATABASE IF EXISTS javacode2018;
CREATE DATABASE IF NOT EXISTS javacode2018;

USE javacode2018;
DROP TABLE IF EXISTS user1;
CREATE TABLE user1(
    id int PRIMARY KEY AUTO_INCREMENT,
    name varchar(64) NOT NULL DEFAULT '' COMMENT '姓名'
);

DROP TABLE IF EXISTS user2;
CREATE TABLE user2(
    id int PRIMARY KEY AUTO_INCREMENT,
    name varchar(64) NOT NULL DEFAULT '' COMMENT '姓名'
);
```

spring配置类MainConfig6

准备JdbcTemplate和事务管理器。

```
package com.javacode2018.tx.demo6;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;

import javax.sql.DataSource;

@EnableTransactionManagement //开启spring事务管理功能
@Configuration //指定当前类是一个spring配置类
@ComponentScan //开启bean扫描注册
public class MainConfig6 {
    //定义一个数据源
    @Bean
    public DataSource dataSource() {
        org.apache.tomcat.jdbc.pool.DataSource dataSource = new
        org.apache.tomcat.jdbc.pool.DataSource();
        dataSource.setDriverClassName("com.mysql.jdbc.Driver");
    }
}
```

```

        dataSource.setUrl("jdbc:mysql://localhost:3306/javacode2018?
characterEncoding=UTF-8");
        dataSource.setUsername("root");
        dataSource.setPassword("root123");
        dataSource.setInitialSize(5);
        return dataSource;
    }

    //定义一个JdbcTemplate,用来执行db操作
    @Bean
    public JdbcTemplate jdbcTemplate(DataSource dataSource) {
        return new JdbcTemplate(dataSource);
    }

    //定义我一个事务管理器
    @Bean
    public PlatformTransactionManager transactionManager(DataSource dataSource)
    {
        return new DataSourceTransactionManager(dataSource);
    }
}

```

来3个service

后面的案例中会在这3个service中使用spring的事务来演示效果。

User1Service

```

package com.javacode2018.tx.demo6;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;

@Component
public class User1Service {
    @Autowired
    private JdbcTemplate jdbcTemplate;
}

```

User2Service

```

package com.javacode2018.tx.demo6;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;

@Component
public class User2Service {
    @Autowired
    private JdbcTemplate jdbcTemplate;
}

```

TxService

```
package com.javacode2018.tx.demo6;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class TxService {
    @Autowired
    private User1Service user1Service;
    @Autowired
    private User2Service user2Service;
}
```

测试用例Demo6Test

before方法会在每个@Test标注的方法之前执行一次，这个方法主要用来做一些准备工作：启动spring容器、清理2个表中的数据；after方法会在每个@Test标注的方法执行完毕之后执行一次，我们在这个里面输出2个表的数据；方便查看的测试用例效果。

```
package com.javacode2018.tx.demo6;

import org.junit.Before;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Demo6Test {

    private TxService txService;
    private JdbcTemplate jdbcTemplate;

    //每个@Test用例执行之前先启动一下spring容器，并清理一下user1、user2中的数据
    @Before
    public void before() {
        AnnotationConfigApplicationContext context = new
        AnnotationConfigApplicationContext(MainConfig6.class);
        txService = context.getBean(TxService.class);
        jdbcTemplate = context.getBean(JdbcTemplate.class);
        jdbcTemplate.update("truncate table user1");
        jdbcTemplate.update("truncate table user2");
    }

    @After
    public void after() {
        System.out.println("user1表数据: " + jdbcTemplate.queryForList("SELECT *
        from user1"));
        System.out.println("user2表数据: " + jdbcTemplate.queryForList("SELECT *
        from user2"));
    }
}
```

1、REQUIRED

User1Service

添加1个方法，事务传播行为：REQUIRED

```
@Transactional(propagation = Propagation.REQUIRED)
public void required(String name) {
    this.jdbcTemplate.update("insert into user1(name) VALUES (?)", name);
}
```

User2Service

添加2个方法，事务传播行为：REQUIRED，注意第2个方法内部最后一行会抛出一个异常。

```
@Transactional(propagation = Propagation.REQUIRED)
public void required(String name) {
    this.jdbcTemplate.update("insert into user1(name) VALUES (?)", name);
}

@Transactional(propagation = Propagation.REQUIRED)
public void required_exception(String name) {
    this.jdbcTemplate.update("insert into user1(name) VALUES (?)", name);
    throw new RuntimeException();
}
```

场景1 (1-1)

外围方法没有事务，外围方法内部调用2个REQUIRED级别的事务方法。

案例中都是在TxService的方法中去调用另外2个service，所以TxService中的方法统称外围方法，另外2个service中的方法称内部方法。

验证方法1

TxService添加

```
public void notransaction_exception_required_required() {
    this.user1Service.required("张三");
    this.user2Service.required("李四");
    throw new RuntimeException();
}
```

测试用例，Demo6Test中添加

```
@Test
public void notransaction_exception_required_required() {
    txService.notransaction_exception_required_required();
}
```

运行输出

```
user1表数据: [{id=1, name=张三}]
user2表数据: [{id=1, name=李四}]
```

验证方法2

TxService添加

```
public void notransaction_required_required_exception() {
    this.user1Service.required("张三");
    this.user2Service.required_exception("李四");
}
```

测试用例，Demo6Test中添加

```
@Test
public void notransaction_required_required_exception() {
    txService.notransaction_required_required_exception();
}
```

运行输出

```
user1表数据: [{id=1, name=张三}]
user2表数据: []
```

结果分析

验证方法序号	数据库结果	结果分析
1	“张三”、“李四”均插入。	外围方法未开启事务，插入“张三”、“李四”方法在自己的事务中独立运行，外围方法异常不影响内部插入“张三”、“李四”方法独立的事务。
2	“张三”插入，“李四”未插入。	外围方法没有事务，插入“张三”、“李四”方法都在自己的事务中独立运行，所以插入“李四”方法抛出异常只会回滚插入“李四”方法，插入“张三”方法不受影响。

结论

通过这两个方法我们证明了在外围方法未开启事务的情况下 `Propagation.REQUIRED` 修饰的内部方法会新开启自己的事务，且开启的事务相互独立，互不干扰。

场景2（1-2）

外围方法开启事务（`Propagation.REQUIRED`），这个使用频率特别高。

验证方法1

TxService添加

```
@Transactional(propagation = Propagation.REQUIRED)
public void transaction_exception_required_required() {
    user1Service.required("张三");
    user2Service.required("李四");
    throw new RuntimeException();
}
```

测试用例，Demo6Test中添加

```
@Test
public void transaction_exception_required_required() {
    txService.transaction_exception_required_required();
}
```

运行输出

```
user1表数据: []
user2表数据: []
```

验证方法2

TxService添加

```
@Transactional(propagation = Propagation.REQUIRED)
public void transaction_required_required_exception() {
    user1Service.required("张三");
    user2Service.required_exception("李四");
}
```

测试用例，Demo6Test中添加

```
@Test
public void transaction_required_required_exception() {
    txService.transaction_required_required_exception();
}
```

运行输出

```
user1表数据: []
user2表数据: []
```

验证方法3

TxService添加

```
@Transactional(propagation = Propagation.REQUIRED)
public void transaction_required_required_exception_try() {
    user1Service.required("张三");
    try {
        user2Service.required_exception("李四");
    } catch (Exception e) {
        System.out.println("方法回滚");
    }
}
```

测试用例， Demo6Test中添加

```
@Test
public void transaction_required_required_exception_try() {
    txService.transaction_required_required_exception_try();
}
```

运行输出

```
方法回滚
user1表数据: []
user2表数据: []
```

结果分析

验证方法序号	数据库结果	结果分析
1	“张三”、“李四”均未插入。	外围方法开启事务，内部方法加入外围方法事务，外围方法回滚，内部方法也要回滚。
2	“张三”、“李四”均未插入。	外围方法开启事务，内部方法加入外围方法事务，内部方法抛出异常回滚，外围方法感知异常致使整体事务回滚。
3	“张三”、“李四”均未插入。	外围方法开启事务，内部方法加入外围方法事务，内部方法抛出异常回滚，即使方法被catch不被外围方法感知，整个事务依然回滚。

结论

以上试验结果我们证明在外围方法开启事务的情况下 `Propagation.REQUIRED` 修饰的内部方法会加入到外围方法的事务中，所有 `Propagation.REQUIRED` 修饰的内部方法和外围方法均属于同一事务，只要一个方法回滚，整个事务均回滚。

2、PROPAGATION_REQUIRES_NEW

User1Service

添加1个方法，事务传播行为：REQUIRES_NEW

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void requires_new(String name) {
    this.jdbcTemplate.update("insert into user1(name) VALUES (?)", name);
}
```

User2Service

添加2个方法，事务传播行为：REQUIRES_NEW，注意第2个方法内部最后一行会抛出一个异常。

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void requires_new(String name) {
    this.jdbcTemplate.update("insert into user2(name) VALUES (?)", name);
}

@Transactional(propagation = Propagation.REQUIRES_NEW)
public void requires_new_exception(String name) {
    this.jdbcTemplate.update("insert into user2(name) VALUES (?)", name);
    throw new RuntimeException();
}
```

场景1 (2-1)

外围方法没有事务。

验证方法1

TxService添加

```
public void notransaction_exception_requiresNew_requiresNew(){
    user1Service.requires_new("张三");
    user2Service.requires_new("李四");
    throw new RuntimeException();
}
```

Demo6Test中添加

```
@Test
public void notransaction_exception_requiresNew_requiresNew() {
    txService.notransaction_exception_requiresNew_requiresNew();
}
```

运行输出

user1表数据: [{id=1, name=张三}]
user2表数据: [{id=1, name=李四}]

验证方法2

TxService添加

```
public void notransaction_requiresNew_requiresNew_exception(){
    user1Service.requires_new("张三");
    user2Service.requires_new_exception("李四");
}
```

测试用例，Demo6Test中添加

```
@Test
public void notransaction_requiresNew_requiresNew_exception() {
    txService.notransaction_requiresNew_requiresNew_exception();
}
```

运行输出

user1表数据: [{id=1, name=张三}]
user2表数据: []

结果分析

验证方法序号	数据库结果	结果分析
1	“张三”插入，“李四”插入。	外围方法没有事务，插入“张三”、“李四”方法都在自己的事务中独立运行,外围方法抛出异常回滚不会影响内部方法。
2	“张三”插入，“李四”未插入	外围方法没有开启事务，插入“张三”方法和插入“李四”方法分别开启自己的事务，插入“李四”方法抛出异常回滚，其他事务不受影响。

结论

通过这两个方法我们证明了在外围方法未开启事务的情况下 `Propagation.REQUIRES_NEW` 修饰的内部方法会新开启自己的事务，且开启的事务相互独立，互不干扰。

场景2 (2-2)

外围方法开启事务。

验证方法1

TxService添加

```
@Transactional(propagation = Propagation.REQUIRED)
public void transaction_exception_required_requiresNew_requiresNew() {
    user1Service.required("张三");
    user2Service.requires_new("李四");
    user2Service.requires_new("王五");
    throw new RuntimeException();
}
```

测试用例， **Demo6Test**中添加

```
@Test
public void transaction_exception_required_requiresNew_requiresNew() {
    txService.transaction_exception_required_requiresNew_requiresNew();
}
```

运行输出

```
user1表数据: []
user2表数据: [{id=1, name=李四}, {id=2, name=王五}]
```

验证方法2

TxService添加

```
@Transactional(propagation = Propagation.REQUIRED)
public void transaction_required_requiresNew_requiresNew_exception() {
    user1Service.required("张三");
    user2Service.requires_new("李四");
    user2Service.requires_new_exception("王五");
}
```

Demo6Test中添加

```
@Test
public void transaction_required_requiresNew_requiresNew_exception() {
    txService.transaction_required_requiresNew_requiresNew_exception();
}
```

运行输出

```
user1表数据: []  
user2表数据: [{id=1, name=李四}]
```

验证方法3

TxService添加

```
@Transactional(propagation = Propagation.REQUIRED)  
public void transaction_required_requiresNew_requiresNew_exception_try(){  
    user1Service.required("张三");  
    user2Service.requires_new("李四");  
    try {  
        user2Service.requires_new_exception("王五");  
    } catch (Exception e) {  
        System.out.println("回滚");  
    }  
}
```

Demo6Test中添加

```
@Test  
public void transaction_required_requiresNew_requiresNew_exception_try() {  
    txService.transaction_required_requiresNew_requiresNew_exception_try();  
}
```

运行输出

```
回滚  
user1表数据: [{id=1, name=张三}]  
user2表数据: [{id=1, name=李四}]
```

结果分析

验证方法序号	数据库结果	结果分析
1	“张三”未插入，“李四”插入，“王五”插入。	外围方法开启事务，插入“张三”方法和外围方法一个事务，插入“李四”方法、插入“王五”方法分别在独立的新建事务中，外围方法抛出异常只回滚和外围方法同一事务的方法，故插入“张三”的方法回滚。
2	“张三”未插入，“李四”插入，“王五”未插入。	外围方法开启事务，插入“张三”方法和外围方法一个事务，插入“李四”方法、插入“王五”方法分别在独立的新建事务中。插入“王五”方法抛出异常，首先插入“王五”方法的事务被回滚，异常继续抛出被外围方法感知，外围方法事务亦被回滚，故插入“张三”方法也被回滚。
3	“张三”插入，“李四”插入，“王五”未插入。	外围方法开启事务，插入“张三”方法和外围方法一个事务，插入“李四”方法、插入“王五”方法分别在独立的新建事务中。插入“王五”方法抛出异常，首先插入“王五”方法的事务被回滚，异常被catch不会被外围方法感知，外围方法事务不回滚，故插入“张三”方法插入成功。

结论

在外围方法开启事务的情况下 `Propagation.REQUIRES_NEW` 修饰的内部方法依然会单独开启独立事务，且与外部方法事务也独立，内部方法之间、内部方法和外部方法事务均相互独立，互不干扰。

3、PROPAGATION_NESTED

User1Service

添加1个方法，事务传播行为：NESTED

```
@Transactional(propagation = Propagation.NESTED)
public void nested(String name) {
    this.jdbcTemplate.update("insert into user1(name) VALUES (?)", name);
}
```

User2Service

添加2个方法，事务传播行为：NESTED，注意第2个方法内部最后一行会抛出一个异常。

```
@Transactional(propagation = Propagation.NESTED)
public void nested(String name) {
    this.jdbcTemplate.update("insert into user2(name) VALUES (?)", name);
}

@Transactional(propagation = Propagation.NESTED)
public void nested_exception(String name) {
    this.jdbcTemplate.update("insert into user2(name) VALUES (?)", name);
    throw new RuntimeException();
}
```

场景1 (3-1)

外围方法没有事务。

验证方法1

TxService添加

```
public void notransaction_exception_nested_nested(){
    user1Service.nested("张三");
    user2Service.nested("李四");
    throw new RuntimeException();
}
```

Demo6Test中添加

```
@Test
public void notransaction_exception_nested_nested() {
    txService.notransaction_exception_nested_nested();
}
```

运行输出

```
user1表数据: [{id=1, name=张三}]
user2表数据: [{id=1, name=李四}]
```

验证方法2

TxService添加

```
public void notransaction_nested_nested_exception(){
    user1Service.nested("张三");
    user2Service.nested_exception("李四");
}
```

测试用例， Demo6Test中添加

```
@Test
public void notransaction_nested_nested_exception() {
    txService.notransaction_nested_nested_exception();
}
```

运行输出

```
user1表数据: [{id=1, name=张三}]
user2表数据: []
```

结果分析

验证方法序号	数据库结果	结果分析
1	“张三”、“李四”均插入。	外围方法未开启事务，插入“张三”、“李四”方法在自己的事务中独立运行，外围方法异常不影响内部插入“张三”、“李四”方法独立的事务。
2	“张三”插入，“李四”未插入。	外围方法没有事务，插入“张三”、“李四”方法都在自己的事务中独立运行，所以插入“李四”方法抛出异常只会回滚插入“李四”方法，插入“张三”方法不受影响。

结论

通过这两个方法我们证明了在外围方法未开启事务的情况下 `Propagation.NESTED` 和 `Propagation.REQUIRED` 作用相同，修饰的内部方法都会新开启自己的事务，且开启的事务相互独立，互不干扰。

场景2 (3-1)

外围方法开启事务。

验证方法1

TxService添加

```
@Transactional
public void transaction_exception_nested_nested(){
    user1Service.nested("张三");
    user2Service.nested("李四");
    throw new RuntimeException();
}
```

测试用例，Demo6Test中添加

```
@Test
public void transaction_exception_nested_nested() {
    txService.transaction_exception_nested_nested();
}
```

运行输出

```
user1表数据: []
user2表数据: []
```

验证方法2

TxService添加

```
@Transactional
public void transaction_nested_nested_exception(){
    user1Service.nested("张三");
    user2Service.nested_exception("李四");
}
```

Demo6Test中添加

```
@Test
public void transaction_nested_nested_exception() {
    txService.transaction_nested_nested_exception();
}
```

运行输出

```
user1表数据: []
user2表数据: []
```

验证方法3

TxService添加

```
@Transactional
public void transaction_nested_nested_exception_try(){
    user1Service.nested("张三");
    try {
        user2Service.nested_exception("李四");
    } catch (Exception e) {
        System.out.println("方法回滚");
    }
}
```


Demo6Test中添加

```
@Test
public void transaction_nested_nested_exception_try() {
    txService.transaction_nested_nested_exception_try();
}
```

运行输出

方法回滚
user1表数据: [{id=1, name=张三}]
user2表数据: []

结果分析

验证方法序号	数据库结果	结果分析
1	“张三”、“李四”均未插入。	外围方法开启事务，内部事务为外围事务的子事务，外围方法回滚，内部方法也要回滚。
2	“张三”、“李四”均未插入。	外围方法开启事务，内部事务为外围事务的子事务，内部方法抛出异常回滚，且外围方法感知异常致使整体事务回滚。
3	“张三”插入、“李四”未插入。	外围方法开启事务，内部事务为外围事务的子事务，插入“李四”内部方法抛出异常，可以单独对子事务回滚。

结论

以上试验结果我们证明在外围方法开启事务的情况下 `Propagation.NESTED` 修饰的内部方法属于外部事务的子事务，外围主事务回滚，子事务一定回滚，而内部子事务可以单独回滚而不影响外围主事务和其他子事务。

内部事务原理

以mysql为例，mysql中有个[savepoint的功能](#)，NESTED内部事务就是通过这个实现的。

REQUIRED,REQUIRES_NEW,NESTED比较

由“场景2（1-2）”和“场景2（3-2）”对比，我们可知：

REQUIRED和NESTED修饰的内部方法都属于外围方法事务，如果外围方法抛出异常，这两种方法的事务都会被回滚。但是REQUIRED是加入外围方法事务，所以和外围事务同属于一个事务，一旦REQUIRED事务抛出异常被回滚，外围方法事务也将被回滚。而NESTED是外围方法的子事务，有单独的保存点，所以NESTED方法抛出异常被回滚，不会影响到外围方法的事务。

由“场景2（2-2）”和“场景2（3-2）”对比，我们可知：

REQUIRES_NEW和NESTED都可以做到内部方法事务回滚而不影响外围方法事务。但是因为NESTED是嵌套事务，所以外围方法回滚之后，作为外围方法事务的子事务也会被回滚。而REQUIRES_NEW是通过开启新的事务实现的，内部事务和外围事务是两个事务，外围事务回滚不会影响内部事务。

其他几个传播行为

REQUIRED,REQUIRES_NEW,NESTED这几个算是比较特殊的，比较常用的，剩下的5个传播行为，大家可以自己练练。

模拟用例

介绍了这么多事务传播行为，我们在实际工作中如何应用呢？下面我来举一个示例：

假设我们有一个注册的方法，方法中调用添加积分的方法，如果我们希望添加积分不会影响注册流程（即添加积分执行失败回滚不能使注册方法也回滚），我们会这样写：

```
@Service
public class UserServiceImpl implements UserService {

    @Transactional
    public void register(User user){

        try {
            membershipPointService.addPoint(Point point);
        } catch (Exception e) {
            //省略...
        }
        //省略...
    }
    //省略...
}
```

我们还规定注册失败要影响 addPoint() 方法（注册方法回滚添加积分方法也需要回滚），那么 addPoint() 方法就需要这样实现：

```
@Service
public class MembershipPointServiceImpl implements MembershipPointService{

    @Transactional(propagation = Propagation.NESTED)
    public void addPoint(Point point){
        try {
            recordService.addRecord(Record record);
        } catch (Exception e) {
            //省略...
        }
        //省略...
    }
    //省略...
}
```

我们注意到了在 addPoint() 中还调用了 addRecord() 方法，这个方法用来记录日志。他的实现如下：

```
@Service
public class RecordServiceImpl implements RecordService{

    @Transactional(propagation = Propagation.NOT_SUPPORTED)
    public void addRecord(Record record){

        //省略...
    }
    //省略...
}
```

我们注意到 `addRecord()` 方法中 `propagation = Propagation.NOT_SUPPORTED`，因为对于日志无所谓精确，可以多一条也可以少一条，所以 `addRecord()` 方法本身和外围 `addPoint()` 方法抛出异常都不会使 `addRecord()` 方法回滚，并且 `addRecord()` 方法抛出异常也不会影响外围 `addPoint()` 方法的执行。

通过这个例子相信大家对于事务传播行为的使用有了更加直观的认识，通过各种属性的组合确实能让我们的业务实现更加灵活多样。

总结

通过本文，相信大家对spring事务7种传播行为都有了深入的了解，希望对大家有所帮助，也欢迎大家留言交流!!!

案例源码

git地址：
<https://gitee.com/javacode2018/spring-series>

本文案例对应源码：`spring-series\lesson-002-tx\src\main\java\com\javacode2018\tx\demo6`

路人甲java所有案例代码以后都会放到这个上面，大家watch一下，可以持续关注动态。

Spring系列

1. [Spring系列第1篇：为何要学spring?](#)
2. [Spring系列第2篇：控制反转 \(IoC\) 与依赖注入 \(DI\)](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)
4. [Spring系列第4篇：xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇：创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇：玩转bean scope，避免跳坑里!](#)
7. [Spring系列第7篇：依赖注入之手动注入](#)
8. [Spring系列第8篇：自动注入 \(autowire\) 详解，高手在于坚持](#)
9. [Spring系列第9篇：depend-on到底是干什么的?](#)
10. [Spring系列第10篇：primary可以解决什么问题?](#)
11. [Spring系列第11篇：bean中的autowire-candidate又是干什么的?](#)
12. [Spring系列第12篇：lazy-init: bean延迟初始化](#)
13. [Spring系列第13篇：使用继承简化bean配置\(abstract & parent\)](#)

14. [Spring系列第14篇：lookup-method和replaced-method比较陌生，怎么玩的？](#)
15. [Spring系列第15篇：代理详解（Java动态代理&cglib代理）？](#)
16. [Spring系列第16篇：深入理解java注解及spring对注解的增强（预备知识）](#)
17. [Spring系列第17篇：@Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇：@ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇：@import详解\(bean批量注册\)](#)
20. [Spring系列第20篇：@Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇：注解实现依赖注入（@Autowired、@Resource、@Primary、@Qualifier）](#)
22. [Spring系列第22篇：@Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇：Bean生命周期详解](#)
24. [Spring系列第24篇：父子容器详解](#)
25. [Spring系列第25篇：@Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇：国际化详解](#)
27. [Spring系列第27篇：spring事件机制详解](#)
28. [Spring系列第28篇：Bean循环依赖详解](#)
29. [Spring系列第29篇：BeanFactory扩展（BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor）](#)
30. [Spring系列第30篇：jdk动态代理和cglib代理](#)
31. [Spring系列第31篇：aop概念详解](#)
32. [Spring系列第32篇：AOP核心源码、原理详解](#)
33. [Spring系列第33篇：ProxyFactoryBean创建AOP代理](#)
34. [Spring系列第34篇：@Aspect中@Pointcut 12种用法](#)
35. [Spring系列第35篇：@Aspect中5中通知详解](#)
36. [Spring系列第36篇：@EnableAspectJAutoProxy、@Aspect中通知顺序详解](#)
37. [Spring系列第37篇：@EnableAsync & @Async 实现方法异步调用](#)
38. [Spring系列第38篇：@Scheduled & @EnableScheduling定时器详解](#)
39. [Spring系列第39篇：强大的Spel表达式](#)
40. [Spring系列第40篇：缓存使用（@EnableCaching、@Cacheable、@CachePut、@CacheEvict、@Caching、@CacheConfig）](#)
41. [Spring系列第41篇：@EnableCaching集成redis缓存](#)
42. [Spring系列第42篇：玩转JdbcTemplate](#)
43. [Spring系列第43篇：spring中编程式事务怎么用的？](#)
44. [Spring系列第44篇：详解spring声明式事务\(@Transactional\)](#)
45. [Spring系列第45篇：带你吃透Spring事务7种传播行为](#)

更多好文章

1. [Java高并发系列（共34篇）](#)
2. [MySql高手系列（共27篇）](#)
3. [Maven高手系列（共10篇）](#)
4. [Mybatis系列（共12篇）](#)
5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要，它是什么？怎么实现？](#)
7. [泛型，有点难度，会让很多人懵逼，那是因为你没有看这篇文章！](#)

加我微信：spring88java，进群一起交流java相关各种技术！！

Spring系列第46篇：Spring如何管理多数据源事务？

本篇内容：通过原理和大量案例带大家吃透Spring多数据源事务。

Spring中通过事务管理器来控制事务，每个数据源都需要指定一个事务管理器，如果我们的项目中需要操作多个数据库，那么需要我们配置多个数据源，也就需要配置多个数据管理器。

多数据源事务使用2个步骤

1、为每个数据源定义一个事务管理器

如下面代码，有2个数据源分别连接数据库ds1和ds2，然后为每个数据源定义了1个事务管理器，此时spring容器中有2个数据源和2个事务管理器。

```
//数据源1
@Bean
public DataSource dataSource1() {
    org.apache.tomcat.jdbc.pool.DataSource dataSource = new
org.apache.tomcat.jdbc.pool.DataSource();
    dataSource.setDriverClassName("com.mysql.jdbc.Driver");
    dataSource.setUrl("jdbc:mysql://localhost:3306/ds1?characterEncoding=UTF-
8");
    dataSource.setUsername("root");
    dataSource.setPassword("root123");
    dataSource.setInitialSize(5);
    return dataSource;
}

//事务管理器1，对应数据源1
@Bean
public PlatformTransactionManager
transactionManager1(@Qualifier("dataSource1")DataSource dataSource) {
    return new DataSourceTransactionManager(dataSource);
}

//数据源2
@Bean
public DataSource dataSource2() {
    org.apache.tomcat.jdbc.pool.DataSource dataSource = new
org.apache.tomcat.jdbc.pool.DataSource();
    dataSource.setDriverClassName("com.mysql.jdbc.Driver");
    dataSource.setUrl("jdbc:mysql://localhost:3306/ds2?characterEncoding=UTF-
8");
    dataSource.setUsername("root");
    dataSource.setPassword("root123");
    dataSource.setInitialSize(5);
    return dataSource;
}

//事务管理器2，对应数据源2
@Bean
public PlatformTransactionManager
transactionManager2(@Qualifier("dataSource2")DataSource dataSource) {
```

```
return new DataSourceTransactionManager(dataSource);  
}
```

2、指定事务的管理器bean名称

使用@Transaction中时，需通过@Transaction注解的value或transactionManager属性指定事务管理器bean名称，如：

```
@Transactional(transactionManager = "transactionManager1", propagation =  
Propagation.REQUIRED)  
public void required(String name) {  
    this.jdbcTemplate1.update("insert into user1(name) VALUES (?)", name);  
}
```

这里补充一下，之前我们使用@Transactional的时候，并没有通过value或者transactionManager设置事务管理器，这是为什么？

这是因为我们在spring容器中只定义了一个事务管理器，spring启动事务的时候，默认会按类型在容器中查找事务管理器，刚好容器中只有一个，就拿过来用了，如果有多个的时候，如果你不指定，spring是不知道具体要用哪个事务管理器的。

多数据源事务的使用就这么简单，下面我们来看案例，案例才是精华。

事务管理器运行过程

这里先给大家解释一下REQUIRED传播行为下，事务管理器的大致的运行过程，方便理解后面的案例代码。

```
Service1中:  
@Transactional(transactionManager = "transactionManager1", propagation =  
Propagation.REQUIRED)  
public void m1(){  
    this.jdbcTemplate1.update("insert into user1(name) VALUES ('张三')");  
    service2.m2();  
}  
  
Service2中:  
@Transactional(transactionManager = "transactionManager1", propagation =  
Propagation.REQUIRED)  
public void m2(){  
    this.jdbcTemplate1.update("insert into user1(name) VALUES ('李四')");  
}
```

spring事务中有个resources的ThreadLocal，static修饰的，用来存放共享的资源，稍后过程中会用到。

```
private static final ThreadLocal<Map<Object, Object>> resources = new  
NamedThreadLocal<>("Transactional resources");
```

下面看m1方法简化版的事务过程：

1、TransactionInterceptor拦截m1方法

2、获取m1方法的事务配置信息：事务管理器bean名称：transactionManager1，事务传播行为：REQUIRED

3、从spring容器中找到事务管理器transactionManager1，然后问一下transactionManager1，当前上下文中有无事务，显然现在是没有的

4、创建一个新的事务

```
//获取事务管理器对应的数据源，即dataSource1
DataSource dataSource1 = transactionManager1.getDataSource();
//即从dataSource1中获取一个连接
Connection conn = transactionManager1.dataSource1.getConnection();
//开启事务手动提交
conn.setAutoCommit(false);
//将dataSource1->conn放入map中
map.put(dataSource1,conn);
//将map丢到上面的resources ThreadLocal中
resources.set(map);
```

5、下面来带m1放的第一行代码：this.jdbcTemplate1.update("insert into user1(name) VALUES ('张三')");

6、jdbcTemplate内部需要获取数据连接，获取连接的过程

```
//从resources这个ThreadLocal中获取到map
Map map = resources.get();
//通过jdbcTemplate1.datasource从map看一下没有可用的连接
Connection conn = map.get(jdbcTemplate1.datasource);
//如果从map没有找到连接，那么重新从jdbcTemplate1.datasource中获取一个
//大家应该可以看出来，jdbcTemplate1和transactionManager1指定的是同一个dataSource，索引这个地方conn是不为null的
if(conn==null){
    conn = jdbcTemplate1.datasource.getConnection();
}
```

7、通过上面第6步获取的conn执行db操作，插入张三

8、下面来到m1方法的第2行代码：service2.m2();

9、m2方法上面也有@Transactional,TransactionInterceptor拦截m2方法

10、获取m2方法的事务配置信息：事务管理器bean名称：transactionManager1，事务传播行为：REQUIRED

11、从spring容器中找到事务管理器transactionManager1，然后问一下transactionManager1，当前上下文中有无事务，显然是有的，m1开启的事务正在执行中，所以m2方法就直接加入这个事务了

12、下面来带m2放的第一行代码：this.jdbcTemplate1.update("insert into user1(name) VALUES ('李四')");

13、jdbcTemplate内部需要获取数据连接，获取连接的过程

```
//从resources这个ThreadLocal中获取到map
Map map = resources.get();
//通过jdbcTemplate1.datasource从map看一下没有可用的连接
Connection conn = map.get(jdbcTemplate1.datasource);
//如果从map没有找到连接，那么重新从jdbcTemplate1.datasource中获取一个
//大家应该可以看出来，jdbcTemplate1和transactionManager1指定的是同一个dataSource，索引这个地方conn是不为null的
if(conn==null){
    conn = jdbcTemplate1.datasource.getConnection();
}
```

14、通过第13步获取的conn执行db操作，插入李四

15、最终TransactionInterceptor发现2个方法都执行完毕了，没有异常，执行事务提交操作，如下

```
//获取事务管理器对应的数据源，即dataSource1
DataSource dataSource1 = transactionManager1.getDataSource();
//从resources这个ThreadLocal中获取到map
Map map = resources.get();
//通过map拿到事务管理器开启的连接
Connection conn = map.get(dataSource1);
//通过conn提交事务
conn.commit();
```



```
//管理连接
conn.close();
16、清理ThreadLocal中的连接：通过map.remove(dataSource1)将连接从resource ThreadLocal中
移除
17、清理事务
```

从上面代码中可以看出：整个过程中有2个地方需要用到数据库连接Connection对象，第1个地方是：spring事务拦截器启动事务的时候会从datasource中获取一个连接，通过这个连接开启事务手动提交，第2个地方是：最终执行sql操作的时候，也需要用到一个连接。那么必须确保这两个连接必须是同一个连接的时候，执行sql的操作才会受spring事务控制，那么如何确保这2个是同一个连接呢？从代码中可以看出必须让事务管理器中的datasource和JdbcTemplate中的datasource必须是同一个，那么最终2个连接就是同一个对象。

这里顺便回答一下群友问的一个问题：**什么是事务挂起操作？**

这里以事务传播行为REQUIRED_NEW为例说明一下，REQUIRED_NEW表示不管当前事务管理器中是否有事务，都会重新开启一个事务，如果当前事务管理器中有事务，会把当前事务挂起。

所谓挂起，你可以这么理解：**对当前存在事务的现场生成一个快照，然后将事务现场清理干净，然后重新开启一个新事务，新事务执行完毕之后，将事务现场清理干净，然后再根据前面的快照恢复旧事务。**

下面我们再回到本文的内容，多数据源事务管理。

事务管理器如何判断当前是否有事务？

简化版的过程如下：

```
Map map=resource的ThreadLocal.get();
DataSource datasource = transactionManager.getDataSource();
Connection conn = map.get(datasource);
//如果conn不为空，就表示当前有事务
if(conn!=null){
}
```

从这段代码可以看出：判断是否存在事务，主要和datasource有关，和事务管理器无关，即使是不同的事务管理器，只要事务管理器的datasource是一样的，那么就可以发现当前存在的事务。

事务管理器的运行过程和如何判断是否有事务，这2点大家一定要理解，这个理解了，后面的案例理解起来会容易很多。

下面上案例。

案例源码

git地址：
<https://gitee.com/javacode2018/spring-series>

本文案例对应源码：

案例1: `spring-series\lesson-002-tx\src\main\java\com\javacode2018\tx\demo7`

案例2: `spring-series\lesson-002-tx\src\main\java\com\javacode2018\tx\demo8`

案例1

准备案例代码

1、准备db

2个数据库: ds1、ds2

每个库中2个表: user1、user2

```
DROP DATABASE IF EXISTS ds1;
CREATE DATABASE if NOT EXISTS ds1;

USE ds1;
DROP TABLE IF EXISTS user1;
CREATE TABLE user1(
    id int PRIMARY KEY AUTO_INCREMENT,
    name varchar(64) NOT NULL DEFAULT '' COMMENT '姓名'
);

DROP TABLE IF EXISTS user2;
CREATE TABLE user2(
    id int PRIMARY KEY AUTO_INCREMENT,
    name varchar(64) NOT NULL DEFAULT '' COMMENT '姓名'
);

DROP DATABASE IF EXISTS ds2;
CREATE DATABASE if NOT EXISTS ds2;

USE ds2;
DROP TABLE IF EXISTS user1;
CREATE TABLE user1(
    id int PRIMARY KEY AUTO_INCREMENT,
    name varchar(64) NOT NULL DEFAULT '' COMMENT '姓名'
);

DROP TABLE IF EXISTS user2;
CREATE TABLE user2(
    id int PRIMARY KEY AUTO_INCREMENT,
    name varchar(64) NOT NULL DEFAULT '' COMMENT '姓名'
);
```

2、spring配置类

定义2个数据源: dataSource1、dataSource2, 分别用来连接数据库ds1和ds2

定义2个JdbcTemplate: jdbcTemplate1、jdbcTemplate2, 分别关联dataSource1和dataSource2

2个数据源对应2个事务管理器: transactionManager1、transactionManager2, 分别用来管理2个数据源的事务

6个bean的名称

数据源	JdbcTemplate	事务管理器
dataSource1	jdbcTemplate1	transactionManager1
dataSource2	jdbcTemplate2	transactionManager2

源码如下:

```
package com.javacode2018.tx.demo7;

import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;

import javax.sql.DataSource;

@EnableTransactionManagement //开启spring事务管理功能
@Configuration //指定当前类是一个spring配置类
@ComponentScan //开启bean扫描注册
public class MainConfig7 {
    //定义数据源1, 连接数据库: ds1
    @Bean
    public DataSource dataSource1() {
        org.apache.tomcat.jdbc.pool.DataSource dataSource = new
org.apache.tomcat.jdbc.pool.DataSource();
        dataSource.setDriverClassName("com.mysql.jdbc.Driver");
        dataSource.setUrl("jdbc:mysql://localhost:3306/ds1?
characterEncoding=UTF-8");
        dataSource.setUsername("root");
        dataSource.setPassword("root123");
        dataSource.setInitialSize(5);
        return dataSource;
    }

    //定义一个JdbcTemplate, 对应数据源dataSource1, 用来操作数据库: ds1
    @Bean
    public JdbcTemplate jdbcTemplate1(@Qualifier("dataSource1") DataSource
dataSource) {
        return new JdbcTemplate(dataSource);
    }

    //定义事务管理器transactionManager1, 对应数据源dataSource1, 用来管理数据库ds1中的事务
    @Bean
    public PlatformTransactionManager
transactionManager1(@Qualifier("dataSource1") DataSource dataSource) {
        return new DataSourceTransactionManager(dataSource);
    }

    //定义数据源2, 连接数据库: ds2
    @Bean
    public DataSource dataSource2() {
```

```

        org.apache.tomcat.jdbc.pool.DataSource dataSource = new
org.apache.tomcat.jdbc.pool.DataSource();
        dataSource.setDriverClassName("com.mysql.jdbc.Driver");
        dataSource.setUrl("jdbc:mysql://localhost:3306/ds2?
characterEncoding=UTF-8");
        dataSource.setUsername("root");
        dataSource.setPassword("root123");
        dataSource.setInitialSize(5);
        return dataSource;
    }

    //定义一个JdbcTemplate，对应数据源dataSource2，用来操作数据库：ds2
    @Bean
    public JdbcTemplate jdbcTemplate2(@Qualifier("dataSource2") DataSource
dataSource) {
        return new JdbcTemplate(dataSource);
    }

    //定义事务管理器transactionManager2，对应数据源dataSource2，用来管理数据库ds2中的事务
    @Bean
    public PlatformTransactionManager
transactionManager2(@Qualifier("dataSource2") DataSource dataSource) {
        return new DataSourceTransactionManager(dataSource);
    }
}

```

下面来定义4个service，分别用来操作2个库中的4张表。

3、Ds1User1Service

用来操作ds1.user1表，注意下面代码中@Transactional注解中transactionManager的值为transactionManager1

```

package com.javacode2018.tx.demo7;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

@Component
public class Ds1User1Service {
    @Autowired
    private JdbcTemplate jdbcTemplate1;

    @Transactional(transactionManager = "transactionManager1", propagation =
Propagation.REQUIRED)
    public void required(String name) {
        this.jdbcTemplate1.update("insert into user1(name) VALUES (?)", name);
    }
}

```

同样，再来定义操作另外3个表的service

4、Ds1User2Service

用来操作ds1.user2表，事务管理器也是transactionManager1

```
package com.javacode2018.tx.demo7;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

@Component
public class Ds1User2Service {
    @Autowired
    private JdbcTemplate jdbcTemplate1;

    @Transactional(transactionManager = "transactionManager1", propagation =
Propagation.REQUIRED)
    public void required(String name) {
        this.jdbcTemplate1.update("insert into user2(name) VALUES (?)", name);
    }
}
```

5、Ds2User1Service

用来操作ds2.user1表，对应事务管理器transactionManager2

```
package com.javacode2018.tx.demo7;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

@Component
public class Ds2User1Service {
    @Autowired
    private JdbcTemplate jdbcTemplate2;

    @Transactional(transactionManager = "transactionManager2", propagation =
Propagation.REQUIRED)
    public void required(String name) {
        this.jdbcTemplate2.update("insert into user1(name) VALUES (?)", name);
    }
}
```

6、Ds2User2Service

用来操作ds2.user2表，对应事务管理器transactionManager2

```
package com.javacode2018.tx.demo7;
```

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

@Component
public class Ds2User2Service {
    @Autowired
    private JdbcTemplate jdbcTemplate2;

    @Transactional(transactionManager = "transactionManager2", propagation =
Propagation.REQUIRED)
    public void required(String name) {
        this.jdbcTemplate2.update("insert into user2(name) VALUES (?)", name);
    }
}

```

7、Tx1Service

```

package com.javacode2018.tx.demo7;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class Tx1Service {
    @Autowired
    private Ds1User1Service ds1User1Service;
    @Autowired
    private Ds1User2Service ds1User2Service;
}

```

8、Tx2Service

```

package com.javacode2018.tx.demo7;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class Tx2Service {
    @Autowired
    private Ds2User1Service ds2User1Service;
    @Autowired
    private Ds2User2Service ds2User2Service;
}

```

9、测试类Demo7Test

```

package com.javacode2018.tx.demo7;

import org.junit.After;
import org.junit.Before;

```

```

import
org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.jdbc.core.JdbcTemplate;

public class Demo7Test {

    private Tx1Service txService1;
    private JdbcTemplate jdbcTemplate1;
    private JdbcTemplate jdbcTemplate2;

    // @Before标注的方法会在任意@Test方法执行之前执行，我们这在这里清理一下2库中4张表的数据
    @Before
    public void before() {
        AnnotationConfigApplicationContext context = new
        AnnotationConfigApplicationContext(MainConfig7.class);
        txService1 = context.getBean(Tx1Service.class);
        this.jdbcTemplate1 = context.getBean("jdbcTemplate1",
        JdbcTemplate.class);
        this.jdbcTemplate2 = context.getBean("jdbcTemplate2",
        JdbcTemplate.class);
        jdbcTemplate1.update("truncate table ds1.user1");
        jdbcTemplate1.update("truncate table ds1.user2");
        jdbcTemplate2.update("truncate table ds2.user1");
        jdbcTemplate2.update("truncate table ds2.user2");
    }

    // @After标注的方法会在任意@Test方法执行完毕之后执行，我们在此处输出4张表的数据，用来查看
    测试案例之后，表中的数据清空
    @After
    public void after() {
        System.out.println("ds1.user1表数据: " +
        this.jdbcTemplate1.queryForList("SELECT * from user1"));
        System.out.println("ds1.user2表数据: " +
        this.jdbcTemplate1.queryForList("SELECT * from user2"));
        System.out.println("ds2.user1表数据: " +
        this.jdbcTemplate2.queryForList("SELECT * from user1"));
        System.out.println("ds2.user2表数据: " +
        this.jdbcTemplate2.queryForList("SELECT * from user2"));
    }

}

```

代码验证

1、场景1

外围方法和内部方法使用相同的事务管理器，传播行为都是REQUIRED。

Tx1Service中添加代码

```
@Transactional(transactionManager = "transactionManager1", propagation =
Propagation.REQUIRED)
public void test1() {
    this.ds1User1Service.required("张三");
    this.ds1User2Service.required("李四");
    throw new RuntimeException();
}
```

方法、事务管理器、事务管理器对应数据源、操作db的jdbcTemplate中数据源对应关系。

方法	事务管理器	事务管理器对应数据源	jdbcTemplate对应数据源
test1	transactionManager1	datasource1	-
ds1User1Service.required	transactionManager1	datasource1	datasource1
this.ds1User2Service.required	transactionManager1	datasource1	datasource1

Demo7Test中添加测试用例

```
@Test
public void test1() {
    this.txService1.test1();
}
```

运行输出

```
ds1.user1表数据: []
ds1.user2表数据: []
ds2.user1表数据: []
ds2.user2表数据: []
```

结论分析

数据库结果	结果分析
“张三”、“李四”均为插入	外围方法和内部方法使用同一个事务管理器transactionManager1，且事务管理器和jdbcTemplate的datasource都是同一个，外围方法会开启事务，内部方法加入外围方法事务，外围方法弹出异常导致事务回滚，内部方法跟着回滚了。

2、场景2

外部方法和内部方法使用不同的事务管理器。

Tx1Service中添加代码

```
@Transactional(transactionManager = "transactionManager2", propagation =
Propagation.REQUIRED)
public void test2() {
    this.ds1User1Service.required("张三");
    this.ds1User2Service.required("李四");
    throw new RuntimeException();
}
```

方法、事务管理器、事务管理器对应数据源、操作db的jdbctemplate中数据源对应关系。

方法	事务管理器	事务管理器对应数据源	jdbctemplate对应数据源
test2	transactionManager2	datasource2	-
ds1User1Service.required("张三");	transactionManager1	datasource1	datasource1
this.ds1User2Service.required("李四");	transactionManager1	datasource1	datasource1

Demo7Test中添加测试用例

```
@Test
public void test2() {
    this.txService1.test2();
}
```

运行输出

```
ds1.user1表数据: [{id=1, name=张三}]
ds1.user2表数据: [{id=1, name=李四}]
ds2.user1表数据: []
ds2.user2表数据: []
```

结论分析

数据库结果	结果分析
"张三"、"李四"均插入	外围方法test2和内部两个required方法用到的不是同一个事务管理器，内部的2个方法在各自的事务中执行，不受外部方法事务的控制。

3、场景3

Tx1Service中添加代码


```

@Autowired
private Ds2User1Service ds2User1Service;
@Autowired
private Ds2User2Service ds2User2Service;

@Transactional(transactionManager = "transactionManager1", propagation =
Propagation.REQUIRED)
public void test3() {
    this.ds1User1Service.required("张三");
    this.ds1User2Service.required("李四");
    this.ds2User1Service.required("王五");
    this.ds2User2Service.required("赵六");
    throw new RuntimeException();
}

```

方法、事务管理器、事务管理器对应数据源、操作db的jdbctemplate中数据源对应关系。

方法	事务管理器	事务管理器对应数据源	jdbctemplate对应数据源
test3	transactionManager1	datasource1	-
this.ds1User1Service.required("张三");	transactionManager1	datasource1	datasource1
this.ds1User2Service.required("李四");	transactionManager1	datasource1	datasource1
this.ds2User1Service.required("王五");	transactionManager2	datasource2	datasource2
this.ds2User2Service.required("赵六");	transactionManager2	datasource2	datasource2

Demo7Test中添加测试用例

```

@Test
public void test3() {
    this.txService1.test3();
}

```

运行输出

```

ds1.user1表数据: []
ds1.user2表数据: []
ds2.user1表数据: [{id=1, name=王五}]
ds2.user2表数据: [{id=1, name=赵六}]

```

结论分析

"张三"、"李四"都未插入，"王五"、"赵六"插入成功。

外围方法和内部的前2个required方法事务管理器都是transactionManager1，所以他们3个在一个事务中执行；而内部的后2个required方法事务管理器是transactionManager2，他们分别在自己的事务中执行，不受外围方法事务的控制，外围方法感受到了异常，回滚事务，只会导致内部的前2个required方法回滚。

4、场景4

Tx2Service中加入代码

```
@Transactional(transactionManager = "transactionManager2", propagation =
Propagation.REQUIRED)
public void test1() {
    this.ds2User1Service.required("王五");
    this.ds2User2Service.required("赵六");
}
```

Tx1Service中加入代码

```
@Autowired
private Tx2Service tx2Service;

@Transactional(transactionManager = "transactionManager1", propagation =
Propagation.REQUIRED)
public void test4() {
    this.ds1User1Service.required("张三");
    this.ds1User2Service.required("李四");
    this.tx2Service.test1();
    throw new RuntimeException();
}
```

方法、事务管理器、事务管理器对应数据源、操作db的jdbcTemplate中数据源对应关系。

方法	事务管理器	事务管理器对应数据源	jdbcTemplate对应数据源
test4	transactionManager1	datasource1	-
this.ds1User1Service.required	transactionManager1	datasource1	datasource1
this.ds1User2Service.required	transactionManager1	datasource1	datasource1
this.tx2Service.test1()	transactionManager2	datasource2	-
this.ds2User1Service.required	transactionManager2	datasource2	datasource2
this.ds2User2Service.required	transactionManager2	datasource2	datasource2

Demo7Test中添加测试用例

```
@Test
public void test4() {
    this.txService1.test4();
}
```

运行输出

```
ds1.user1表数据: []
ds1.user2表数据: []
ds2.user1表数据: [{id=1, name=王五}]
ds2.user2表数据: [{id=1, name=赵六}]
```

结论分析

"张三"、"李四"都未插入，"王五"、"赵六"插入成功。

分析一下过程

- 1、test4在事务管理器transactionManager1中开启事务tm1，并将连接放入resourceThreadLocal中(datasource1->conn1)
- 2、this.ds1User1Service.required("张三")事务管理器是transactionManager1，所以会加入事务tm1中，通过jdbcTemplate1插入张三，由于jdbcTemplate1.datasource是datasource1，所以会获取到threadLocal中的conn1来插入数据
- 3、this.ds1User2Service.required("李四")事务管理器是transactionManager1，所以会加入事务tm1中，通过jdbcTemplate1插入张三，由于jdbcTemplate1.datasource是datasource1，所以会获取到threadLocal中的conn1来插入数据
- 4、执行this.tx2Service.test1()，这个方法事务管理器是transactionManager2，所以会重新开启一个事务tm2，并将连接放入resourceThreadLocal中(datasource2->conn2)
- 5、this.ds2User1Service.required("王五")事务管理器是transactionManager2，通过所以会加入事务tm2中，通过jdbcTemplate1插入张三，由于jdbcTemplate1.datasource是datasource2，所以会获取到threadLocal中的conn2来插入数据
- 6、this.ds2User2Service.required("赵六")事务管理器是transactionManager2，所以会加入事务tm2中，通过jdbcTemplate1插入张三，由于jdbcTemplate1.datasource是datasource2，所以会获取到threadLocal中的conn2来插入数据
- 7、tm2提交
- 8、tm1发现test4抛出异常，tm1执行回滚

5、场景5

Tx2Service中加入代码

```
@Transactional(transactionManager = "transactionManager2", propagation =
Propagation.REQUIRED)
public void test2() {
    this.ds2User1Service.required("王五");
    this.ds2User2Service.required("赵六");
    throw new RuntimeException();
}
```

Tx1Service中加入代码

```
@Transactional(transactionManager = "transactionManager1", propagation =
Propagation.REQUIRED)
public void test5() {
    this.ds1User1Service.required("张三");
    this.ds1User2Service.required("李四");
    this.tx2Service.test2();
}
```

方法、事务管理器、事务管理器对应数据源、操作db的jdbcTemplate中数据源对应关系。

方法	事务管理器	事务管理器对应数据源	jdbcTemplate对应数据源
test4	transactionManager1	datasource1	-
this.ds1User1Service.required	transactionManager1	datasource1	datasource1
this.ds1User2Service.required	transactionManager1	datasource1	datasource1
this.tx2Service.test1()	transactionManager2	datasource2	-
this.ds2User1Service.required	transactionManager2	datasource2	datasource2
this.ds2User2Service.required	transactionManager2	datasource2	datasource2

Demo7Test中添加测试用例

```
@Test
public void test5() {
    this.txService1.test5();
}
```

运行输出

```
ds1.user1表数据: []
ds1.user2表数据: []
ds2.user1表数据: []
ds2.user2表数据: []
```

结论分析

4个表都未插入数据。

外围方法test5通过事务管理器transactionManager1开启了事务tm1，内部方法插入“张三”，“李四”加入了tm1事务，而test2通过事务管理器transactionManager2又开启了一个事务tm2，test2内部方法插入“王五”，“赵六”加入了tm2事务，test2内部抛出了异常，tm2和tm1都感受到了这个异常，所以2个事务都进行了回滚操作。

案例2

spring配置类

```
package com.javacode2018.tx.demo8;

import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;

import javax.sql.DataSource;
```

```

@EnableTransactionManagement //开启spring事务管理功能
@Configuration //指定当前类是一个spring配置类
@ComponentScan //开启bean扫描注册
public class MainConfig8 {
    //定义数据源1，连接数据库：ds1
    @Bean
    public DataSource dataSource1() {
        org.apache.tomcat.jdbc.pool.DataSource dataSource = new
org.apache.tomcat.jdbc.pool.DataSource();
        dataSource.setDriverClassName("com.mysql.jdbc.Driver");
        dataSource.setUrl("jdbc:mysql://localhost:3306/ds1?
characterEncoding=UTF-8");
        dataSource.setUsername("root");
        dataSource.setPassword("root123");
        dataSource.setInitialSize(5);
        return dataSource;
    }

    //定义一个jdbcTemplate1
    @Bean
    public JdbcTemplate jdbcTemplate1(DataSource dataSource) {
        return new JdbcTemplate(dataSource);
    }

    //定义事务管理器transactionManager1
    @Bean
    public PlatformTransactionManager transactionManager1(DataSource dataSource)
{
        return new DataSourceTransactionManager(dataSource);
    }

    //定义jdbcTemplate2
    @Bean
    public JdbcTemplate jdbcTemplate2(DataSource dataSource) {
        return new JdbcTemplate(dataSource);
    }

    //定义事务管理器transactionManager2
    @Bean
    public PlatformTransactionManager transactionManager2(DataSource dataSource)
{
        return new DataSourceTransactionManager(dataSource);
    }
}

```

上面代码中

- 定义了1个数据源：dataSource1
- 2个jdbcTemplate：jdbcTemplate1和jdbcTemplate2，他们的datasource都是dataSource1
- 2个事务管理器：transactionManager1和transactionManager2，他们的datasource都是dataSource1

有同学发现这样写是不是很奇怪，不是说一个数据源定义一个事务管理器么，这什么操作？

不急，我们这样写，是为了让你更深入了解其原理。

User2Service

内部的required方法操作db用的是jdbcTemplate2，事务管理器为transactionManager2

```
package com.javacode2018.tx.demo8;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

@Component
public class User2Service {
    @Autowired
    private JdbcTemplate jdbcTemplate2;

    @Transactional(transactionManager = "transactionManager2", propagation =
Propagation.REQUIRED)
    public void required() {
        this.jdbcTemplate2.update("insert into user2(name) VALUES (?)", "李四");
    }
}
```

User2Service

内部的required方法操作db用的是jdbcTemplate1，事务管理器为transactionManager1，并且会调用user2Service的required方法。

```
package com.javacode2018.tx.demo8;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

@Component
public class User1Service {
    @Autowired
    private JdbcTemplate jdbcTemplate1;

    @Autowired
    private User2Service user2Service;

    @Transactional(transactionManager = "transactionManager1", propagation =
Propagation.REQUIRED)
    public void required() {
        this.jdbcTemplate1.update("insert into user1(name) VALUES (?)", "张三");
        this.user2Service.required();
        throw new RuntimeException();
    }
}
```

大家觉得required方法执行完毕之后，会是什么结果？

- A: 张三未插入、李四插入成功
- B: 张三、李四均为插入

大家先思考一下，先别看下面的执行结果，可以参考事务管理器的执行过程分析一下结果。

好了，我们上测试用例。

Demo8Test

```
package com.javacode2018.tx.demo8;

import org.junit.After;
import org.junit.Before;
import org.junit.Test;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.jdbc.core.JdbcTemplate;

public class Demo8Test {

    private UserService user1Service;
    private JdbcTemplate jdbcTemplate1;

    //@Before标注的方法会在任意@Test方法执行之前执行，我们这在这里清理一下2库中4张表的数据
    @Before
    public void before() {
        AnnotationConfigApplicationContext context = new
        AnnotationConfigApplicationContext(MainConfig8.class);
        this.user1Service = context.getBean(UserService.class);
        this.jdbcTemplate1 = context.getBean("jdbcTemplate1",
        JdbcTemplate.class);
        jdbcTemplate1.update("truncate table ds1.user1");
        jdbcTemplate1.update("truncate table ds1.user2");
    }

    //@After标注的方法会在任意@Test方法执行完毕之后执行，我们在此处输出4张表的数据，用来查看
    测试案例之后，表中的数据清空
    @After
    public void after() {
        System.out.println("ds1.user1表数据: " +
        this.jdbcTemplate1.queryForList("SELECT * from user1"));
        System.out.println("ds1.user2表数据: " +
        this.jdbcTemplate1.queryForList("SELECT * from user2"));
    }

    @Test
    public void test1() {
        this.user1Service.required();
    }

}
```

运行输出

```
ds1.user1表数据: []  
ds1.user2表数据: []
```

结果是都没有插入。

结果分析

分析一下执行过程

```
1、this.user1Service.required();  
2、事务拦截器拦截user1Service.required()方法，事务配置信息：（事务管理器：  
transactionManager1，传播行为REQUIRED）  
3、问一下transactionManager1，当前是否有事务，transactionManager2看了一下，发现没有，那么  
重新创建一个事务tm1，通过transactionManager1中的datasource，即datasource1重新获取一个连  
接：conn1，然后丢到resourceThreadLocal中(datasource1->conn1)  
4、执行this.jdbcTemplate1.update("insert into user1(name) VALUES (?)", "张三")，由  
于jdbcTemplate1中的datasource是datasource1，所以会从resourceThreadLocal中拿到conn1连  
接来执行sql  
5、执行this.user2Service.required();  
6、事务拦截器拦截user1Service.required()方法，事务配置信息：（事务管理器：  
transactionManager2，传播行为REQUIRED）  
7、问一下transactionManager2，当前是否有事务？大家在回头看一下事务管理器是如何判断当前是否有  
事务的，由于transactionManager2和transactionManager1用到的都是datasource1，所以  
transactionManager2会发现当前是存在事务的，即tm1  
8、执行this.jdbcTemplate2.update("insert into user2(name) VALUES (?)", "李四")，由  
于jdbcTemplate2中的datasource也是datasource1，所以会从resourceThreadLocal中拿到conn1  
连接来执行sql  
9、最终整个操作过程中只有一个事务tm1，一个连接conn1，通过conn1执行2个插入操作  
10、执行throw new RuntimeException();抛出异常  
11、tm1感受到了异常，所以会执行回滚操作，最终都插入失败
```

总结一下

1、本文介绍了多数据源事务的使用，2个步骤：先为每个数据源定义一个事务管理器，然后在@Transactional中指定具体要使用哪个事务管理器。

2、事务管理器运行过程、事务管理器如何判断当前是否有事务，这2点非常非常重要，大家再看一下。

有问题欢迎留言交流。

案例源码

```
git地址：  
https://gitee.com/javacode2018/spring-series
```

本文案例对应源码：

```
spring-series\lesson-002-tx\src\main\java\com\javacode2018\tx\demo7  
spring-series\lesson-002-tx\src\main\java\com\javacode2018\tx\demo8
```

路人甲java所有案例代码以后都会放到这个上面，大家watch一下，可以持续关注动态。

Spring系列

1. [Spring系列第1篇：为何要学spring?](#)
2. [Spring系列第2篇：控制反转 \(IoC\) 与依赖注入 \(DI\)](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)
4. [Spring系列第4篇：xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇：创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇：玩转bean scope，避免跳坑里!](#)
7. [Spring系列第7篇：依赖注入之手动注入](#)
8. [Spring系列第8篇：自动注入 \(autowire\) 详解，高手在于坚持](#)
9. [Spring系列第9篇：depend-on到底是干什么的?](#)
10. [Spring系列第10篇：primary可以解决什么问题?](#)
11. [Spring系列第11篇：bean中的autowire-candidate又是干什么的?](#)
12. [Spring系列第12篇：lazy-init: bean延迟初始化](#)
13. [Spring系列第13篇：使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇：lookup-method和replaced-method比较陌生，怎么玩的?](#)
15. [Spring系列第15篇：代理详解 \(Java动态代理&cglib代理\) ?](#)
16. [Spring系列第16篇：深入理解java注解及spring对注解的增强 \(预备知识\)](#)
17. [Spring系列第17篇：@Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇：@ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇：@import详解\(bean批量注册\)](#)
20. [Spring系列第20篇：@Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇：注解实现依赖注入 \(@Autowired、@Resource、@Primary、@Qualifier\)](#)
22. [Spring系列第22篇：@Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇：Bean生命周期详解](#)
24. [Spring系列第24篇：父子容器详解](#)
25. [Spring系列第25篇：@Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇：国际化详解](#)
27. [Spring系列第27篇：spring事件机制详解](#)
28. [Spring系列第28篇：Bean循环依赖详解](#)
29. [Spring系列第29篇：BeanFactory扩展 \(BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor\)](#)
30. [Spring系列第30篇：jdk动态代理和cglib代理](#)
31. [Spring系列第31篇：aop概念详解](#)
32. [Spring系列第32篇：AOP核心源码、原理详解](#)
33. [Spring系列第33篇：ProxyFactoryBean创建AOP代理](#)
34. [Spring系列第34篇：@Aspect中@Pointcut 12种用法](#)
35. [Spring系列第35篇：@Aspect中5中通知详解](#)
36. [Spring系列第36篇：@EnableAspectJAutoProxy、@Aspect中通知顺序详解](#)
37. [Spring系列第37篇：@EnableAsync & @Async 实现方法异步调用](#)
38. [Spring系列第38篇：@Scheduled & @EnableScheduling定时器详解](#)
39. [Spring系列第39篇：强大的Spel表达式](#)
40. [Spring系列第40篇：缓存使用 \(@EnableCaching、@Cacheable、@CachePut、@CacheEvict、@Caching、@CacheConfig\)](#)
41. [Spring系列第41篇：@EnableCaching集成redis缓存](#)
42. [Spring系列第42篇：玩转jdbcTemplate](#)
43. [Spring系列第43篇：spring中程式事务怎么用的?](#)
44. [Spring系列第44篇：详解spring声明式事务\(@Transactional\)](#)
45. [Spring系列第45篇：带你吃透Spring事务7种传播行为](#)

更多好文章

1. [Java高并发系列 \(共34篇\)](#)
2. [MySql高手系列 \(共27篇\)](#)
3. [Maven高手系列 \(共10篇\)](#)
4. [Mybatis系列 \(共12篇\)](#)
5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要，它是什么？怎么实现？](#)
7. [泛型，有点难度，会让很多人懵逼，那是因为你没有看这篇文章！](#)

加我微信：spring888java，进群一起交流java相关各种技术！！

Spring系列第47篇：spring事务源码解析

本文主要内容：Spring编程式事务源码深度解析，理解spring事务的本质

开始本文之前，有些必备的知识需要大家先了解一下

1. [玩转JdbcTemplate](#)
2. [详解Spring编程式事务](#)
3. [详解Spring声明式事务\(@EnableTransactionManagement、@Transactional\)](#)
4. [详解Spring事务7种传播行为](#)
5. [详解Spring多数据源事务](#)

目录

环境

回顾一下编程式事务用法

编程式事务过程

- 1、定义事务属性信息(TransactionDefinition)
- 2、定义事务管理器(PlatformTransactionManager)
- 3、获取事务
 - 3.1、getTransaction：获取事务
 - 3.2、doGetTransaction：获取事务对象
 - 3.3、@3.1-7：doBegin 开启事务
 - 3.4、@3.1-8：prepareSynchronization 准备事务同步
 - 3.5、小结
- 4、事务方法中执行增删改查
- 5、提交事务
- 6、回滚事务
- 7、存在事务的情况如何走？
 - 7.1、判断是否有事务：isExistingTransaction
 - 7.2、若当前存在事务
 - 7.3、事务挂起：suspend
 - 7.4、事务恢复：resume
- 8、事务执行过程中的回调接口： TransactionSynchronization
 - 8.1、作用
 - 8.2、扩展点的用法
 - 1、定义事务TransactionSynchronization对象
 - 2、将TransactionSynchronization注册到当前事务中
 - 3、回调扩展点TransactionSynchronization中的方法
 - 8.3、来个案例
 - 1、执行sql
 - 2、案例代码
 - 3、输出

总结一下

案例源码

环境

1. jdk1.8
2. Spring版本： 5.2.3.RELEASE
3. mysql5.7

回顾一下编程式事务用法

```
@Test
public void test1() throws Exception {
    //定义一个数据源
```

```

org.apache.tomcat.jdbc.pool.DataSource dataSource = new
org.apache.tomcat.jdbc.pool.DataSource();
dataSource.setDriverClassName("com.mysql.jdbc.Driver");
dataSource.setUrl("jdbc:mysql://localhost:3306/javacode2018?
characterEncoding=UTF-8");
dataSource.setUsername("root");
dataSource.setPassword("root123");
dataSource.setInitialSize(5);
//定义一个JdbcTemplate，用来方便执行数据库增删改查
JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);
//1.定义事务管理器，给其指定一个数据源（可以把事务管理器想象为一个人，这个人来负责事务的控制操作）
PlatformTransactionManager platformTransactionManager = new
DataSourceTransactionManager(dataSource);
//2.定义事务属性：TransactionDefinition，TransactionDefinition可以用来配置事务的属性信息，比如事务隔离级别、事务超时时间、事务传播方式、是否是只读事务等等。
TransactionDefinition transactionDefinition = new
DefaultTransactionDefinition();
//3.获取事务：调用platformTransactionManager.getTransaction开启事务操作，得到事务状态(TransactionStatus)对象
TransactionStatus transactionStatus =
platformTransactionManager.getTransaction(transactionDefinition);
//4.执行业务操作，下面就执行2个插入操作
try {
    System.out.println("before:" + jdbcTemplate.queryForList("SELECT * from
t_user"));
    jdbcTemplate.update("insert into t_user (name) values (?)", "test1-1");
    jdbcTemplate.update("insert into t_user (name) values (?)", "test1-2");
    //5.提交事务：platformTransactionManager.commit
    platformTransactionManager.commit(transactionStatus);
} catch (Exception e) {
    //6.回滚事务：platformTransactionManager.rollback
    platformTransactionManager.rollback(transactionStatus);
}
System.out.println("after:" + jdbcTemplate.queryForList("SELECT * from
t_user"));
}

```

编程式事务过程

编程式事务过程，我们简化了一下，如下：

```

1、定义事务属性信息：TransactionDefinition transactionDefinition = new
DefaultTransactionDefinition();
2、定义事务管理器：PlatformTransactionManager platformTransactionManager = new
DataSourceTransactionManager(dataSource);
3、获取事务：TransactionStatus transactionStatus =
platformTransactionManager.getTransaction(transactionDefinition);
4、执行sql操作：比如上面通过JdbcTemplate的各种方法执行各种sql操作
5、提交事务(platformTransactionManager.commit)或者回滚事务
(platformTransactionManager.rollback)

```

下面通过源码来解析上面4步操作，带大家理解原理。

1、定义事务属性信息(TransactionDefinition)

事务启动的过程中需要定义事务的一些配置信息，如：事务传播行为、隔离级别、超时时间、是否是只读事务、事务名称，spring中使用**TransactionDefinition**接口表示事务定义信息，下面看一下TransactionDefinition接口源码，主要有5个信息

- 事务传播行为
- 事务隔离级别
- 事务超时时间
- 是否是只读事务
- 事务名称

```
public interface TransactionDefinition {

    //传播行为:REQUIRED
    int PROPAGATION_REQUIRED = 0;

    //传播行为:REQUIRED
    int PROPAGATION_SUPPORTS = 1;

    //传播行为:REQUIRED
    int PROPAGATION_MANDATORY = 2;

    //传播行为:REQUIRED
    int PROPAGATION_REQUIRES_NEW = 3;

    //传播行为:REQUIRED
    int PROPAGATION_NOT_SUPPORTED = 4;

    //传播行为:REQUIRED
    int PROPAGATION_NEVER = 5;

    //传播行为:REQUIRED
    int PROPAGATION_NESTED = 6;

    //默认隔离级别
    int ISOLATION_DEFAULT = -1;

    //隔离级别：读未提交
    int ISOLATION_READ_UNCOMMITTED = 1;

    //隔离级别：读已提交
    int ISOLATION_READ_COMMITTED = 2;

    //隔离级别：可重复读
    int ISOLATION_REPEATABLE_READ = 4;

    //隔离级别：序列化的方式
    int ISOLATION_SERIALIZABLE = 8;

    //默认超时时间
    int TIMEOUT_DEFAULT = -1;

    //返回事务传播行为，默认是REQUIRED
    default int getPropagationBehavior() {
        return PROPAGATION_REQUIRED;
    }

    //返回事务的隔离级别
```

```

default int getIsolationLevel() {
    return ISOLATION_DEFAULT;
}

//返回事务超时时间(秒)
default int getTimeout() {
    return -1;
}

//是否是只读事务
default boolean isReadOnly() {
    return false;
}

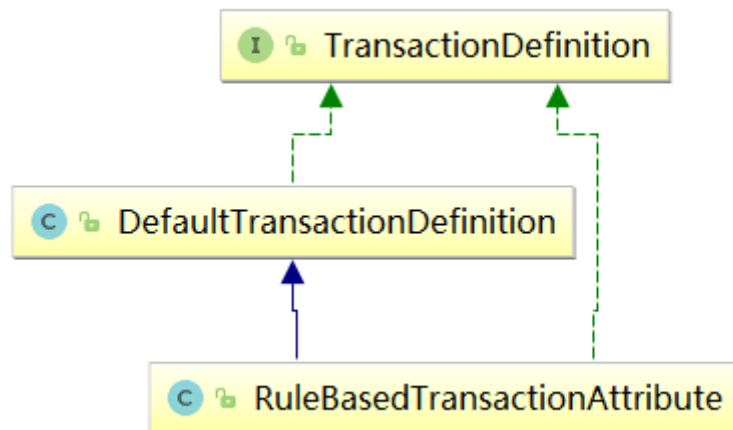
//获取事务名称
@Nullable
default String getName() {
    return null;
}

//获取默认的事务定义信息
static TransactionDefinition withDefaults() {
    return StaticTransactionDefinition.INSTANCE;
}
}

```

TransactionDefinition接口的实现类，比较多，我们重点关注用的比较多的2个

- **DefaultTransactionDefinition**：TransactionDefinition接口的默认的一个实现，编程式事务中通常可以使用这个
- **RuleBasedTransactionAttribute**：声明式事务中用到的是这个，这个里面对于事务回滚有一些动态匹配的规则，稍后在声明式事务中去讲。



编程式事务中通常使用DefaultTransactionDefinition，如下：

```

DefaultTransactionDefinition transactionDefinition = new
DefaultTransactionDefinition();
//设置事务传播行为
transactionDefinition.setPropagationBehavior(TransactionDefinition.PROPROPAGATION_S
UPPORTS);
//设置事务隔离级别
transactionDefinition.setIsolationLevel(TransactionDefinition.ISOLATION_READ_COM
MITTED);
//设置是否是只读事务
transactionDefinition.setReadOnly(true);
//设置事务超时时间(s)，事务超过了指定的时间还未结束，会抛异常
transactionDefinition.setTimeout(5);
//设置事务名称，这个名称可以随便设置，不过最好起个有意义的名字，在debug的过程中会输出
transactionDefinition.setName("class完整类名.方法名称");

```

下面进入第2步，定义事务管理器。

2、定义事务管理器(PlatformTransactionManager)

事务管理器，这是个非常重要的角色，可以把这货想象为一个人，spring就是靠这个人来管理事务的，负责：**获取事务、提交事务、回滚事务**，Spring中用**PlatformTransactionManager**接口表示事务管理器，接口中有三个方法

```

public interface PlatformTransactionManager {

    //通过事务管理器获取一个事务，返回TransactionStatus： 内部包含事务的状态信息
    TransactionStatus getTransaction(@Nullable TransactionDefinition definition)
    throws TransactionException;

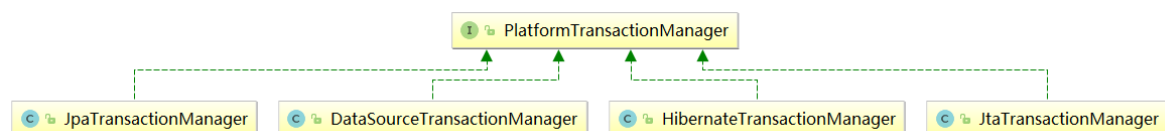
    //根据事务的状态信息提交事务
    void commit(TransactionStatus status) throws TransactionException;

    //根据事务的状态信息回滚事务
    void rollback(TransactionStatus status) throws TransactionException;

}

```

PlatformTransactionManager有多个实现类，用来应对不同的环境，比如你操作db用的是hibernate或者mybatis，那么用到的事务管理器是不一样的，常见的事务管理器实现有下面几个



JpaTransactionManager：如果你用jpa来操作db，那么需要用这个管理器来帮你控制事务。

DataSourceTransactionManager：如果你用是指定数据源的方式，比如操作数据库用的是：JdbcTemplate、mybatis、ibatis，那么需要用这个管理器来帮你控制事务。

HibernateTransactionManager：如果你用hibernate来操作db，那么需要用这个管理器来帮你控制事务。

JtaTransactionManager：如果你用的是java中的jta来操作db，这种通常是分布式事务，此时需要用这种管理器来控制事务。

我们的案例中使用的是JdbcTemplate来操作db，所以用的是 `DataSourceTransactionManager` 这个管理器。

```
PlatformTransactionManager platformTransactionManager = new
DataSourceTransactionManager(dataSource);
```

下面进入第3步，通过事务管理器来开启事务。

3、获取事务

下面源码我们以REQUIRED中嵌套一个REQUIRED_NEW来进行说明，也就是事务中嵌套一个新的事务。

3.1、getTransaction：获取事务

通过事务管理器的getTransaction方法开启事务，传递一个TransactionDefinition参数

```
TransactionStatus transactionStatus =
platformTransactionManager.getTransaction(transactionDefinition);
```

事务管理器我们用的是DataSourceTransactionManager，下面我们看一下DataSourceTransactionManager.getTransaction源码

```
org.springframework.jdbc.datasource.DataSourceTransactionManager

public final TransactionStatus getTransaction(@Nullable TransactionDefinition
definition) throws TransactionException {

    //事务定义信息，若传入的definition如果为空，取默认的
    TransactionDefinition def = (definition != null ? definition :
TransactionDefinition.withDefaults());

    //3.1-1: 获取事务对象
    Object transaction = doGetTransaction();
    boolean debugEnabled = logger.isDebugEnabled();

    //3.1-2: 当前是否存在事务
    if (isExistingTransaction(transaction)) {
        //3.1-3: 如果当前存在事务，走这里
        return handleExistingTransaction(def, transaction, debugEnabled);
    }

    // 当前没有事务，走下面的代码
    // 若事务传播级别是PROPAGATION_MANDATORY：要求必须存在事务，若当前没有事务，弹出异常
    if (def.getPropagationBehavior() ==
TransactionDefinition.PROPAGATION_MANDATORY) {
        throw new IllegalTransactionStateException(
            "No existing transaction found for transaction marked with
propagation 'mandatory'");
    } else if (def.getPropagationBehavior() ==
TransactionDefinition.PROPAGATION_REQUIRED ||
```



```

        def.getPropagationBehavior() ==
TransactionDefinition.PROPAGATION_REQUIRES_NEW ||
        def.getPropagationBehavior() ==
TransactionDefinition.PROPAGATION_NESTED) {
    //事务传播行为
    (PROPAGATION_REQUIRED|PROPAGATION_REQUIRES_NEW|PROPAGATION_NESTED)走这里
    //@3.1-4: 挂起事务
    SuspendedResourcesHolder suspendedResources = suspend(null);
    try {
        //@3.1-5: 是否开启新的事务同步
        boolean newSynchronization = (getTransactionSynchronization() !=
SYNCHRONIZATION_NEVER);
        //@3.1-6: 创建事务状态对象
        DefaultTransactionStatus,DefaultTransactionStatus是TransactionStatus的默认实现
        DefaultTransactionStatus status = newTransactionStatus(def,
transaction, true, newSynchronization, debugEnabled, suspendedResources);
        //@3.1-7: doBegin用于开始事务
        doBegin(transaction, def);
        //@3.1-8: 准备事务同步
        prepareSynchronization(status, def);
        //@3.1-9: 返回事务状态对象
        return status;
    } catch (RuntimeException | Error ex) {
        //@3.1-10: 出现 (RuntimeException|Error) 恢复被挂起的事务
        resume(null, suspendedResources);
        throw ex;
    }
} else {
    //@3.1-11: 其他事务传播行为的走这里 (PROPAGATION_SUPPORTS、
PROPAGATION_NOT_SUPPORTED、PROPAGATION_NEVER)
    boolean newSynchronization = (getTransactionSynchronization() ==
SYNCHRONIZATION_ALWAYS);
    return prepareTransactionStatus(def, null, true, newSynchronization,
debugEnabled, null);
}
}

```

下面来看一下@3.1-1: doGetTransaction方法, 用来获取事务对象

3.2、doGetTransaction: 获取事务对象

```

org.springframework.jdbc.datasource.DataSourceTransactionManager

protected Object doGetTransaction() {
    //@3.2-1: 创建数据源事务对象
    DataSourceTransactionObject txObject = new DataSourceTransactionObject();
    //@3.2-2: 是否支持内部事务
    txObject.setSavepointAllowed(isNestedTransactionAllowed());
    //@3.2-4: ConnectionHolder表示jdbc连接持有者，简单理解：数据的连接被丢到
    ConnectionHolder中了，ConnectionHolder中提供了一些方法来返回里面的连接，此处调用
    TransactionSynchronizationManager.getResource方法来获取ConnectionHolder对象
    ConnectionHolder conHolder = (ConnectionHolder)
    TransactionSynchronizationManager.getResource(observeOnDataSource());
    //@3.2-5: 将conHolder丢到DataSourceTransactionObject中，第二个参数表示是否是一个新的
    连接，明显不是的吗，新的连接需要通过datasource来获取，通过datasource获取的连接才是新的连接
    txObject.setConnectionHolder(conHolder, false);
    return txObject;
}

```

下面来看一下@3.2-4的代码，这个是重点了，这个用到了一个新的类

TransactionSynchronizationManager:事务同步管理器，什么叫同步？一个事务过程中，被调用的方法都在一个线程中串行执行，就是同步；这个类中用到了很多ThreadLocal，用来在线程中存储事务相关的一些信息，来瞅一眼

```

public abstract class TransactionSynchronizationManager {

    //存储事务资源信息
    private static final ThreadLocal<Map<Object, Object>> resources =
        new NamedThreadLocal<>("Transactional resources");

    //存储事务过程中的一些回调接口(TransactionSynchronization接口，这个可以在事务的过程中
    给开发者提供一些回调用的)
    private static final ThreadLocal<Set<TransactionSynchronization>>
    synchronizations =
        new NamedThreadLocal<>("Transaction synchronizations");

    //存储当前正在运行的事务的名称
    private static final ThreadLocal<String> currentTransactionName =
        new NamedThreadLocal<>("Current transaction name");

    //存储当前正在运行的事务是否是只读的
    private static final ThreadLocal<Boolean> currentTransactionReadOnly =
        new NamedThreadLocal<>("Current transaction read-only status");

    //存储当前正在运行的事务的隔离级别
    private static final ThreadLocal<Integer> currentTransactionIsolationLevel =
        new NamedThreadLocal<>("Current transaction isolation level");

    //存储当前正在运行的事务是否是活动状态，事务启动的时候会被激活
    private static final ThreadLocal<Boolean> actualTransactionActive =
        new NamedThreadLocal<>("Actual transaction active");

    //还有很多静态方法，主要是用来操作上面这些ThreadLocal的，这里就不列出的，大家可以去看看
}

```

下面来看TransactionSynchronizationManager.getResource代码

```
ConnectionHolder conHolder = (ConnectionHolder)
TransactionSynchronizationManager.getResource(observeOnDataSource());
```

observeOnDataSource()会返回事务管理器中的datasource对象

```
protected DataSource obtainDataSource() {
    DataSource dataSource = getDataSource();
    Assert.state(dataSource != null, "No DataSource set");
    return dataSource;
}
```

下面看TransactionSynchronizationManager.getResource的源码

```
public static Object getResource(Object key) {
    //通过TransactionSynchronizationUtils.unwrapResourceIfNecessary(key) 获取一个
    actualKey，我们传入的是datasource，实际上最后actualKey和传入的datasource是一个对象
    Object actualKey =
    TransactionSynchronizationUtils.unwrapResourceIfNecessary(key);
    //调用doGetResource方法获取对应的value
    Object value = doGetResource(actualKey);
    return value;
}
```

doGetResource(actualKey)方法如下，内部会从resources这个ThreadLocal中获取获取数据ConnectionHolder对象，到目前为止，根本没有看到向resource中放入过数据，获取获取到的conHolder肯定是null了

```
static final ThreadLocal<Map<Object, Object>> resources = new NamedThreadLocal<>
("Transactional resources");

private static Object doGetResource(Object actualKey) {
    Map<Object, Object> map = resources.get();
    if (map == null) {
        return null;
    }
    Object value = map.get(actualKey);
    return value;
}
```

TransactionSynchronizationManager.getResource：可以理解为从resource ThreadLocal中查找transactionManager.datasource绑定的ConnectionHolder对象

到此，Object transaction = doGetTransaction()方法执行完毕，下面我们再回到getTransaction方法，第一次进来，上下文中是没有事务的，所以会走下面的@3.1-4的代码，当前没有事务，导致没有事务需要挂起，所以suspend方法内部可以先忽略

```
org.springframework.transaction.support.AbstractPlatformTransactionManager#getTransaction

public final TransactionStatus getTransaction(@Nullable TransactionDefinition
definition)
    throws TransactionException {

    //事务定义信息，若传入的definition如果为空，取默认的
```

```

TransactionDefinition def = (definition != null ? definition :
TransactionDefinition.withDefaults());

//@3.1-1: 获取事务对象
Object transaction = doGetTransaction();

if (def.getPropagationBehavior() ==
TransactionDefinition.PROPAGATION_REQUIRED ||
    def.getPropagationBehavior() ==
TransactionDefinition.PROPAGATION_REQUIRES_NEW ||
    def.getPropagationBehavior() ==
TransactionDefinition.PROPAGATION_NESTED) {
    //事务传播行为
    (PROPAGATION_REQUIRED|PROPAGATION_REQUIRES_NEW|PROPAGATION_NESTED)走这里
    //@3.1-4: 挂起事务
    SuspendedResourcesHolder suspendedResources = suspend(null);
    try {
        //@3.1-5: 是否开启新的事务同步
        boolean newSynchronization = (getTransactionSynchronization() !=
SYNCHRONIZATION_NEVER);
        //@3.1-6: 创建事务状态对象
        DefaultTransactionStatus status = newTransactionStatus(
            def, transaction, true, newSynchronization, debugEnabled,
            suspendedResources);
        //@3.1-7: doBegin用于开始事务
        doBegin(transaction, def);
        //@3.1-8: 准备事务同步
        prepareSynchronization(status, def);
        //@3.1-9: 返回事务状态对象
        return status;
    } catch (RuntimeException | Error ex) {
        //@3.1-10: 出现 (RuntimeException|Error) 恢复被挂起的事务
        resume(null, suspendedResources);
        throw ex;
    }
}
}

```

然后会执行下面代码

```

//@3.1-5: 是否开启新的事务同步，事务同步是干嘛的，是spring在事务过程中给开发者预留的一些扩展
点，稍后细说；大家先这么理解，每个新的事务newSynchronization都是true，开一个一个新的事务就会
启动一个新的同步
boolean newSynchronization = (getTransactionSynchronization() !=
SYNCHRONIZATION_NEVER);
//@3.1-6: 创建事务状态对象DefaultTransactionStatus,DefaultTransactionStatus是
TransactionStatus的默认实现
DefaultTransactionStatus status = newTransactionStatus(def, transaction, true,
newSynchronization, debugEnabled, suspendedResources);
//@3.1-7: doBegin用于开始事务
doBegin(transaction, def);
//@3.1-8: 准备事务同步
prepareSynchronization(status, def);
//@3.1-9: 返回事务状态对象
return status;

```

上面过程我们重点来说一下@3.1-7 和 @3.1-8

3.3、@3.1-7: doBegin 开启事务

```
org.springframework.jdbc.datasource.DataSourceTransactionManager#doBegin

protected void doBegin(Object transaction, TransactionDefinition definition) {
    //数据源事务对象
    DataSourceTransactionObject txObject = (DataSourceTransactionObject)
transaction;
    //数据库连接
    Connection con = null;
    try {
        //txObject.hasConnectionHolder()用来判断txObject.connectionHolder!=null, 现在肯定是null, 所以txObject.hasConnectionHolder()返回false
        if (!txObject.hasConnectionHolder() ||
            txObject.getConnectionHolder().isSynchronizedWithTransaction())
        {
            //调用transactionManager.datasource.getConnection()获取一个数据库连接
            Connection newCon = obtainDataSource().getConnection();
            //将数据库连接丢到一个ConnectionHolder中, 放到txObject中, 注意第2个参数是true, 表示第一个参数的ConnectionHolder是新创建的
            txObject.setConnectionHolder(new ConnectionHolder(newCon), true);
        }
        //连接中启动事务同步
        txObject.getConnectionHolder().setSynchronizedWithTransaction(true);
        //获取连接
        con = txObject.getConnectionHolder().getConnection();
        //获取隔离级别
        Integer previousIsolationLevel =
DataSourceUtils.prepareConnectionForTransaction(con, definition);
        //设置隔离级别
        txObject.setPreviousIsolationLevel(previousIsolationLevel);
        //设置是否是只读
        txObject.setReadOnly(definition.isReadOnly());

        //判断连接是否是自动提交的, 如果是自动提交的将其置为手动提交
        if (con.getAutoCommit()) {
            //在txObject中存储一下连接自动提交老的值, 用于在事务执行完毕之后, 还原一下
            //Connection的autoCommit的值
            txObject.setMustRestoreAutoCommit(true);
            //设置手动提交
            con.setAutoCommit(false);
        }
        //准备事务连接
        prepareTransactionalConnection(con, definition);
        //设置事务活动开启
        txObject.getConnectionHolder().setTransactionActive(true);

        //根据事务定义信息获取事务超时时间
        int timeout = determineTimeout(definition);
        if (timeout != TransactionDefinition.TIMEOUT_DEFAULT) {
            //设置连接的超时时间
            txObject.getConnectionHolder().setTimeoutInSeconds(timeout);
        }
        //txObject中的ConnectionHolder是否是一个新的, 确实是新的, 所以这个地方返回true
        if (txObject.isNewConnectionHolder()) {
```

```

        //将datasource->ConnectionHolder丢到resource ThreadLocal的map中
        TransactionSynchronizationManager.bindResource(observeOnDataSource(),
txObject.getConnectionHolder());
    }
}
}

```

重点来看一下下面这段代码

```

TransactionSynchronizationManager.bindResource(observeOnDataSource(),
txObject.getConnectionHolder());

```

源码

```

org.springframework.transaction.support.TransactionSynchronizationManager#bindResource

public static void bindResource(Object key, Object value) throws
IllegalStateException {
    Object actualKey =
TransactionSynchronizationUtils.unwrapResourceIfNecessary(key);
    Map<Object, Object> map = resources.get();
    if (map == null) {
        map = new HashMap<>();
        resources.set(map);
    }
    map.put(actualKey, value);
}

```

上面这段代码执行完毕之后，datasource->ConnectionHolder(conn)被放到resources ThreadLocal的map中了。

3.4、@3.1-8: prepareSynchronization 准备事务同步

```

//@3.1-8: 准备事务同步
prepareSynchronization(status, def);

```

源码如下，大家看一下，这个方法主要的作用是，开启一个新事务的时候，会将事务的状态、隔离级别、是否是只读事务、事务名称丢到TransactionSynchronizationManager中的各种对应的ThreadLocal中，方便在当前线程中共享这些数据。

```

org.springframework.transaction.support.AbstractPlatformTransactionManager#prepareSynchronization

protected void prepareSynchronization(DefaultTransactionStatus status,
TransactionDefinition definition) {
    //如果是一个新的事务，status.isNewSynchronization()将返回true
    if (status.isNewSynchronization()) {

        TransactionSynchronizationManager.setActualTransactionActive(status.hasTransaction());
        TransactionSynchronizationManager.setCurrentTransactionIsolationLevel(

```

```

        definition.getIsolationLevel() !=
TransactionDefinition.ISOLATION_DEFAULT ?
        definition.getIsolationLevel() : null);

    TransactionSynchronizationManager.setCurrentTransactionReadOnly(definition.isReadOnly());

    TransactionSynchronizationManager.setCurrentTransactionName(definition.getName());

    // @3.4-1: 初始化事务同步
    TransactionSynchronizationManager.initSynchronization();
}
}

```

@3.4-1: 初始化事务同步

```

org.springframework.transaction.support.TransactionSynchronizationManager

private static final ThreadLocal<Set<TransactionSynchronization>>
synchronizations =
    new NamedThreadLocal<>("Transaction synchronizations");

// 获取同步是否启动，新事务第一次进来synchronizations.get()是null，所以这个方法返回的是false
public static boolean isSynchronizationActive() {
    return (synchronizations.get() != null);
}

// 初始化事务同步，主要就是在synchronizations ThreadLocal中放一个LinkedHashSet
public static void initSynchronization() throws IllegalStateException {
    if (isSynchronizationActive()) {
        throw new IllegalStateException("Cannot activate transaction
synchronization - already active");
    }
    synchronizations.set(new LinkedHashSet<>());
}

```

3.5、小结

获取事务的过程已经结束了，我们来看一下这个过程中做的一些关键的事情

- 1、获取db连接：从事务管理器的datasource中调用getConnection获取一个新的数据库连接，将连接置为手动提交
- 2、将datasource关联连接丢到ThreadLocal中：将第一步中获取到的连丢到ConnectionHolder中，然后将事务管理器的datasource->ConnectionHolder丢到了resource ThreadLocal中，这样我们可以通过datasource在ThreadLocal中获取到关联的数据库连接
- 3、准备事务同步：将事务的一些信息放到ThreadLocal中

4、事务方法中执行增删改查

以下面这个插入操作来看一下这个插入是如何参与到spring事务中的。

```

jdbcTemplate.update("insert into t_user (name) values (?)", "test1-1");

```

最终会进入到jdbcTemplate#execute方法里，无用代码我们给剔除，重点内部关注下面获取连接的方法

```

org.springframework.jdbc.core.JdbcTemplate#execute(org.springframework.jdbc.core
.PreparedStatementCreator,
org.springframework.jdbc.core.PreparedStatementCallback<T>){
    //获取数据库连接
    Connection con = DataSourceUtils.getConnection(observeOnDataSource());
    //通过conn执行db操作
}

```

observeOnDataSource()会返回jdbcTemplate.datasource对象，下面重点来看DataSourceUtils.getConnection源码，最终会进入下面这个方法

```

org.springframework.jdbc.datasource.DataSourceUtils#doGetConnection

public static Connection doGetConnection(DataSource dataSource) throws
SQLException {
    //用jdbcTemplate.dataSource从TransactionSynchronizationManager的resource
ThreadLocal中获取对应的ConnectionHolder对象，在前面获取事务环节中，
transactionManager.dataSource->ConnectionHolder被丢到resource ThreadLocal，而
jdbcTemplate.dataSource和transactionManager.dataSource是同一个对象，所以是可以获取到
ConnectionHolder的，此时就会使用事务开启是的数据库连接
    ConnectionHolder conHolder = (ConnectionHolder)
TransactionSynchronizationManager.getResource(dataSource);
    //conHolder不为空 && conHolder中有数据库连接对象
    if (conHolder != null && (conHolder.hasConnection() ||
conHolder.isSynchronizedWithTransaction())) {
        //返回conHolder中的数据库连接对象
        return conHolder.getConnection();
    }

    //如果上面获取不到连接，会走这里，这里将会调用
jdbcTemplate.dataSource.getConnection()从数据源中获取一个新的db连接
    Connection con = fetchConnection(dataSource);
    //将连接返回
    return con;
}

```

可以得出一个结论：如果要让最终执行的sql受spring事务控制，那么事务管理器中datasource对象必须和jdbcTemplate.datasource是同一个，这个结论在其他文章中说过很多次了，这里大家算是搞明白了吧。

5、提交事务

调用事务管理器的commit方法，提交事务

```
platformTransactionManager.commit(transactionStatus);
```

commit源码

```

org.springframework.transaction.support.AbstractPlatformTransactionManager#commit

public final void commit(TransactionStatus status) throws TransactionException {
    //事务是否已经完成，此时还未完成，如果事务完成了，再来调用commit方法会报错
}

```



```

        if (status.isCompleted()) {
            throw new IllegalStateException(
                "Transaction is already completed - do not call commit or
rollback more than once per transaction");
        }
        //事务状态
        DefaultTransactionStatus defStatus = (DefaultTransactionStatus) status;
        //defStatus.rollbackOnly是否是true, 如果是true, 说明事务状态被标注了需要回滚, 此时走回
        滚逻辑
        if (defStatus.isLocalRollbackOnly()) {
            //走回滚逻辑
            processRollback(defStatus, false);
            return;
        }
        //提交事务过程
        processCommit(defStatus);
    }
}

```

processCommit源码

```

org.springframework.transaction.support.AbstractPlatformTransactionManager#proces
ssCommit

private void processCommit(DefaultTransactionStatus status) throws
TransactionException {
    try {
        try {
            //提交之前的回调 (给开发提供的扩展点)
            triggerBeforeCommit(status);
            //事务完成之前的回调 (给开发提供的扩展点)
            triggerBeforeCompletion(status);
            //是否是新事务, 如果是新事务, 将执行提交操作, 比如传播行为是REQUIRED中嵌套了一个
            REQUIRED, 那么内部的事务就不是新的事务, 外部的事务是新事务
            if (status.isNewTransaction()) {
                //@5-1: 执行提交操作
                doCommit(status);
            }
        } catch (UnexpectedRollbackException ex) {
            //事务完成之后执行的回调 (给开发提供的扩展点)
            triggerAfterCompletion(status,
TransactionSynchronization.STATUS_ROLLED_BACK);
            throw ex;
        } catch (RuntimeException | Error ex) {
            //提交过程中有异常, 执行回滚操作
            doRollbackOnCommitException(status, ex);
            throw ex;
        }
        try {
            //事务commit之后, 执行一些回调 (给开发提供的扩展点)
            triggerAfterCommit(status);
        } finally {
            //事务完成之后, 执行一些回调 (给开发提供的扩展点)
            triggerAfterCompletion(status,
TransactionSynchronization.STATUS_COMMITTED);
        }
    } finally {
        //事务执行完毕之后, 执行一些清理操作
    }
}

```

```

        cleanupAfterCompletion(status);
    }
}

```

上面这个方法看起来挺长的，重点会做**3件事情**：

1、给开发提供的扩展点：以trigger开头的方法，是留给开发的扩展点，可以在事务执行的过程中执行一些回调，主要是在事务提交之前，提交之后，回滚之前，回滚之后，可以执行一些回调，也就是事务同步要干的事情，这个扩展点稍后说。

2、通过connection执行commit操作，对应上面的 @5-1 代码：doCommit(status);

3、完成之后执行清理操作：finally中执行 cleanupAfterCompletion(status);

来看看**doCommit(status)方法**，内部主要就是调用connection的commit()提交事务，如下：

```

org.springframework.jdbc.datasource.DataSourceTransactionManager#doCommit

protected void doCommit(DefaultTransactionStatus status) {
    DataSourceTransactionObject txObject = (DataSourceTransactionObject)
status.getTransaction();
    //从ConnectionHolder中获取Connection
    Connection con = txObject.getConnectionHolder().getConnection();
    //执行commit，提交数据库事务
    con.commit();
}

```

cleanupAfterCompletion(status)：清理操作

```

org.springframework.transaction.support.AbstractPlatformTransactionManager#cleanupAfterCompletion

private void cleanupAfterCompletion(DefaultTransactionStatus status) {
    //将事务状态置为已完成
    status.setCompleted();
    //是否是新的事务同步
    if (status.isNewSynchronization()) {
        //将TransactionSynchronizationManager中的那些ThreadLocal中的数据都清除，会调用ThreadLocal的remove()方法清除数据
        TransactionSynchronizationManager.clear();
    }
    //是否是新事务
    if (status.isNewTransaction()) {
        //执行清理操作
        doCleanupAfterCompletion(status.getTransaction());
    }
    //是否有被挂起的事务
    if (status.getSuspendedResources() != null) {
        Object transaction = (status.hasTransaction() ? status.getTransaction() : null);
        //恢复被挂起的事务
        resume(transaction, (SuspendedResourcesHolder)
status.getSuspendedResources());
    }
}

```

doCleanupAfterCompletion源码

```

org.springframework.jdbc.datasource.DataSourceTransactionManager#doCleanupAfterCompletion

protected void doCleanupAfterCompletion(Object transaction) {
    DataSourceTransactionObject txObject = (DataSourceTransactionObject)
transaction;

    //是否是一个新的ConnectionHolder，如果是新的事务，那么ConnectionHolder是新的
    if (txObject.isNewConnectionHolder()) {
        //将transactionManager.datasource->ConnectionHolder从resource ThreadLocal
中干掉
        TransactionsSynchronizationManager.unbindResource(observeOnDataSource());
    }
    //下面重置Connection，将Connection恢复到最原始的状态
    Connection con = txObject.getConnectionHolder().getConnection();
    try {
        if (txObject.isMustRestoreAutoCommit()) {
            //自动提交
            con.setAutoCommit(true);
        }
        //恢复connection的隔离级别、是否是只读事务
        DataSourceUtils.resetConnectionAfterTransaction(
            con, txObject.getPreviousIsolationLevel(),
txObject.isReadOnly());
    } catch (Throwable ex) {
        logger.debug("Could not reset JDBC Connection after transaction", ex);
    }
    //是否是新的连接
    if (txObject.isNewConnectionHolder()) {
        //释放连接，内部会调用conn.close()方法
        DataSourceUtils.releaseConnection(con, this.dataSource);
    }
    //还原ConnectionHolder到最初的状态
    txObject.getConnectionHolder().clear();
}

```

总结一下，清理工作主要做的事情就是释放当前线程占有的一切资源，然后将被挂起的事务恢复。

6、回滚事务

回滚的操作和提交的操作差不多的，源码我就不讲了，大家自己去看一下。

7、存在事务的情况如何走？

下面来看另外一个流程，REQUIRED中嵌套一个REQUIRED_NEW，然后走到REQUIRED_NEW的时候，代码是如何运行的？大致的过程如下

- 1、判断上线文中是否有事务
- 2、挂起当前事务
- 3、开启新事务，并执行新事务
- 4、恢复被挂起的事务

7.1、判断是否有事务：isExistingTransaction

判断上线文中是否有事务，比较简单，如下：

```
org.springframework.jdbc.datasource.DataSourceTransactionManager#isExistingTransaction

protected boolean isExistingTransaction(Object transaction) {
    DataSourceTransactionObject txObject = (DataSourceTransactionObject)
transaction;
    //txObject.connectionHolder!=null && connectionHolder事务处于开启状态(上面我们介绍过在doBegin开启事务的时候connectionHolder.transactionActive会被置为true)
    return (txObject.hasConnectionHolder() &&
txObject.getConnectionHolder().isTransactionActive());
}
```

7.2、若当前存在事务

我们再来看一下获取事务中，有事务如何走

```
org.springframework.transaction.support.AbstractPlatformTransactionManager#getTransaction

public final TransactionStatus getTransaction(@Nullable TransactionDefinition
definition) throws TransactionException {
    //获取事务
    Object transaction = doGetTransaction();

    //是否存在事务
    if (isExistingTransaction(transaction)) {
        //存在事务会走这里
        return handleExistingTransaction(def, transaction, debugEnabled);
    }
}
```

当前存在事务，然后会进入handleExistingTransaction方法

```
org.springframework.transaction.support.AbstractPlatformTransactionManager#handleExistingTransaction

private TransactionStatus handleExistingTransaction(TransactionDefinition
definition, Object transaction, boolean debugEnabled) throws
TransactionException {
    //当前有事务，被嵌套的事务传播行为是PROPAGATION_NEVER，抛出异常
    if (definition.getPropagationBehavior() ==
TransactionDefinition.PROPAGATION_NEVER) {
        throw new IllegalTransactionStateException(
            "Existing transaction found for transaction marked with
propagation 'never'");
    }

    if (definition.getPropagationBehavior() ==
TransactionDefinition.PROPAGATION_NOT_SUPPORTED) {
```

```

        //当前有事务，被嵌套的事务传播行为是PROPAGATION_NOT_SUPPORTED，那么将先调用
suspend将当前事务挂起，然后以无事务的方式运行被嵌套的事务
        //挂起当前事务
        Object suspendedResources = suspend(transaction);
        boolean newSynchronization = (getTransactionSynchronization() ==
SYNCHRONIZATION_ALWAYS);
        //以无事务的方式运行
        return prepareTransactionStatus(
            definition, null, false, newSynchronization, debugEnabled,
suspendedResources);
    }

    if (definition.getPropagationBehavior() ==
TransactionDefinition.PROPAGATION_REQUIRES_NEW) {
        //被嵌套的事务传播行为是PROPAGATION_REQUIRES_NEW，那么会先挂起当前事务，然后会重新
开启一个新的事务
        //挂起当前事务
        SuspendedResourcesHolder suspendedResources = suspend(transaction);
        try {
            //下面的过程我们就不在再介绍了，之前有介绍过
            boolean newSynchronization = (getTransactionSynchronization() !=
SYNCHRONIZATION_NEVER);
            DefaultTransactionStatus status = newTransactionStatus(
                definition, transaction, true, newSynchronization,
debugEnabled, suspendedResources);
            doBegin(transaction, definition);
            prepareSynchronization(status, definition);
            return status;
        } catch (RuntimeException | Error beginEx) {
            resumeAfterBeginException(transaction, suspendedResources, beginEx);
            throw beginEx;
        }
    }
    //其他的传播行为走下面。。。暂时省略了
}
}

```

下面重点看事务挂起和事务的恢复操作。

7.3、事务挂起：suspend

事务挂起调用事务管理器的suspend方法，源码如下，主要做的事情：将当前事务中的一切信息保存到SuspendedResourcesHolder对象中，相当于事务的快照，后面恢复的时候用；然后将事务现场清理干净，主要是将一堆存储在ThreadLocal中的事务数据干掉。

```

org.springframework.transaction.support.AbstractPlatformTransactionManager#suspend
nd

protected final SuspendedResourcesHolder suspend(@Nullable Object transaction)
throws TransactionException {
    //当前事务同步是否被激活，如果是新事务，这个返回的是true
    if (TransactionSynchronizationManager.isSynchronizationActive()) {
        //挂起事务同步，这个地方会可以通过TransactionSynchronization接口给开发者提供了扩展
点，稍后我们会单独介绍TransactionSynchronization接口，这个接口专门用来在事务执行过程中做回调
的
        List<TransactionSynchronization> suspendedSynchronizations =
doSuspendSynchronization();
        try {

```

```

        Object suspendedResources = null;
        if (transaction != null) {
            // @1: 获取挂起的资源
            suspendedResources = doSuspend(transaction);
        }
        // 下面就是获取当前事务的各种信息(name, readOnly, 事务隔离级别, 是否被激活)
        String name =
TransactionSynchronizationManager.getCurrentTransactionName();
        TransactionSynchronizationManager.setCurrentTransactionName(null);
        boolean readOnly =
TransactionSynchronizationManager.isCurrentTransactionReadOnly();

        TransactionSynchronizationManager.setCurrentTransactionReadOnly(false);
        Integer isolationLevel =
TransactionSynchronizationManager.getCurrentTransactionIsolationLevel();

        TransactionSynchronizationManager.setCurrentTransactionIsolationLevel(null);
        boolean wasActive =
TransactionSynchronizationManager.isActualTransactionActive();
        TransactionSynchronizationManager.setActualTransactionActive(false);
        return new SuspendedResourcesHolder(
            suspendedResources, suspendedSynchronizations, name,
            readOnly, isolationLevel, wasActive);
    }
}

```

下面来看看 @1: doSuspend(transaction) 源码，主要就是将datasource->connectionHolder从 resource ThreadLocal中解绑，然后将connectionHolder返回，下面这个方法实际上返回的就是 connectionHolder对象

```

org.springframework.jdbc.datasource.DataSourceTransactionManager#doSuspend

protected Object doSuspend(Object transaction) {
    DataSourceTransactionObject txObject = (DataSourceTransactionObject)
transaction;
    //将connectionHolder置为null
    txObject.setConnectionHolder(null);
    //将datasource->connectionHolder从resource ThreadLocal中解绑，并返回被解绑的
connectionHolder对象
    return TransactionSynchronizationManager.unbindResource(observeOnDataSource());
}

```

此时，当前的事务被挂起了，然后开启一个新的事务，新的事务的过程上面已经介绍过了，下面我们来看事务的恢复过程。

7.4、事务恢复：resume

事务挂起调用事务管理器的resume方法，源码如下，主要做的事情：通过 SuspendedResourcesHolder对象中，将被挂起的事务恢复，SuspendedResourcesHolder对象中保存了被挂起的事务所有信息，所以可以通过这个对象来恢复事务。

```

org.springframework.transaction.support.AbstractPlatformTransactionManager#resume

```

```

protected final void resume(@Nullable Object transaction, @Nullable
SuspendedResourcesHolder resourcesHolder)
    throws TransactionException {
    if (resourcesHolder != null) {
        Object suspendedResources = resourcesHolder.suspendedResources;
        if (suspendedResources != null) {
            //恢复被挂起的资源，也就是将datasource->connectionHolder绑定到resource
ThreadLocal中
            doResume(transaction, suspendedResources);
        }
        List<TransactionSynchronization> suspendedSynchronizations =
resourcesHolder.suspendedSynchronizations;
        //下面就是将数据恢复到各种ThreadLocal中
        if (suspendedSynchronizations != null) {

            TransactionSynchronizationManager.setActualTransactionActive(resourcesHolder.wa
sActive);

            TransactionSynchronizationManager.setCurrentTransactionIsolationLevel(resources
Holder.isolationLevel);

            TransactionSynchronizationManager.setCurrentTransactionReadOnly(resourcesHolder
.readOnly);

            TransactionSynchronizationManager.setCurrentTransactionName(resourcesHolder.nam
e);

            //恢复事务同步（将事务扩展点恢复）
            doResumeSynchronization(suspendedSynchronizations);

        }
    }
}

```

8、事务执行过程中的回调接口：TransactionSynchronization

8.1、作用

spring事务运行的过程中，给开发者预留了一些扩展点，在事务执行的不同阶段，将回调扩展点中的一些方法。

比如我们想在事务提交之前、提交之后、回滚之前、回滚之后做一些事务，那么可以通过扩展点来实现。

8.2、扩展点的用法

1、定义事务TransactionSynchronization对象

TransactionSynchronization接口中的方法在spring事务执行的过程中会自动被回调

```

public interface TransactionSynchronization extends Flushable {

    //提交状态
    int STATUS_COMMITTED = 0;
}

```

```

//回滚状态
int STATUS_ROLLED_BACK = 1;

//状态未知，比如事务提交或者回滚的过程中发生了异常，那么事务的状态是未知的
int STATUS_UNKNOWN = 2;

//事务被挂起的时候会调用被挂起事务中所有TransactionSynchronization的resume方法
default void suspend() {
}

//事务恢复的过程中会调用被恢复的事务中所有TransactionSynchronization的resume方法
default void resume() {
}

//清理操作
@Override
default void flush() {
}

//事务提交之前调用
default void beforeCommit(boolean readOnly) {
}

//事务提交或者回滚之前调用
default void beforeCompletion() {
}

//事务commit之后调用
default void afterCommit() {
}

//事务完成之后调用
default void afterCompletion(int status) {
}

}

```

2、将TransactionSynchronization注册到当前事务中

通过下面静态方法将事务扩展点TransactionSynchronization注册到当前事务中

```

TransactionSynchronizationManager.registerSynchronization(transactionSynchronization)

```

看一下源码，很简单，丢到ThreadLocal中了


```

private static final ThreadLocal<Set<TransactionSynchronization>>
synchronizations =
    new NamedThreadLocal<>("Transaction synchronizations");

public static void registerSynchronization(TransactionSynchronization
synchronization)
    throws IllegalStateException {
    Set<TransactionSynchronization> synchs = synchronizations.get();
    if (synchs == null) {
        throw new IllegalStateException("Transaction synchronization is not
active");
    }
    synchs.add(synchronization);
}

```

当有多个TransactionSynchronization的时候，可以指定其顺序，可以实现org.springframework.core.Ordered接口，来指定顺序，从小大的排序被调用，TransactionSynchronization有个默认适配器TransactionSynchronizationAdapter，这个类实现了Ordered接口，所以，如果我们要使用的时候，直接使用TransactionSynchronizationAdapter这个类。

3、回调扩展点TransactionSynchronization中的方法

TransactionSynchronization中的方法是spring事务管理器自动调用的，本文上面有提交到，事务管理器在事务提交或者事务回滚的过程中，有很多地方会调用trigger开头的方法，这个trigger方法内部就会遍历当前事务中的transactionSynchronization列表，然后调用transactionSynchronization内部的一些指定的方法。

以事务提交的源码为例，来看一下

```

private void processCommit(DefaultTransactionStatus status) throws
TransactionException {
    triggerBeforeCommit(status);
    triggerBeforeCompletion(status);
    //....其他代码省略
}

```

triggerBeforeCommit(status)源码

```

protected final void triggerBeforeCommit(DefaultTransactionStatus status) {
    if (status.isNewSynchronization()) {

        TransactionSynchronizationUtils.triggerBeforeCommit(status.isReadOnly());
    }
}

```

TransactionSynchronizationUtils.triggerBeforeCommit 源码

```

public static void triggerBeforeCommit(boolean readOnly) {
    for (TransactionSynchronization synchronization :
        TransactionSynchronizationManager.getSynchronizations()) {
        synchronization.beforeCommit(readOnly);
    }
}

```

8.3、来个案例

1、执行sql

```
DROP DATABASE IF EXISTS javacode2018;
CREATE DATABASE if NOT EXISTS javacode2018;

USE javacode2018;
DROP TABLE IF EXISTS t_user;
CREATE TABLE t_user(
    id int PRIMARY KEY AUTO_INCREMENT,
    name varchar(256) NOT NULL DEFAULT '' COMMENT '姓名'
);
```

2、案例代码

代码比较简单，不多解释了，运行测试方法m0

```
package com.javacode2018.tx.demo9;

import org.junit.Before;
import org.junit.Test;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.TransactionDefinition;
import org.springframework.transaction.TransactionStatus;
import org.springframework.transaction.support.DefaultTransactionDefinition;
import
org.springframework.transaction.support.TransactionSynchronizationAdapter;
import
org.springframework.transaction.support.TransactionSynchronizationManager;

/**
 * 公众号：路人甲java，工作10年的前阿里P7，所有文章以系列的方式呈现，带领大家成为java高手，
 * 目前已出：java高并发系列、mysql高手系列、Maven高手系列、mybatis系列、spring系列，
 * 正在连载springcloud系列，欢迎关注！
 */
public class Demo9Test {
    JdbcTemplate jdbcTemplate;
    PlatformTransactionManager platformTransactionManager;

    @Before
    public void before() {
        //定义一个数据源
        org.apache.tomcat.jdbc.pool.DataSource dataSource = new
org.apache.tomcat.jdbc.pool.DataSource();
        dataSource.setDriverClassName("com.mysql.jdbc.Driver");
        dataSource.setUrl("jdbc:mysql://localhost:3306/javacode2018?
characterEncoding=UTF-8");
        dataSource.setUsername("root");
        dataSource.setPassword("root123");
        dataSource.setInitialSize(5);
        //定义一个JdbcTemplate，用来方便执行数据库增删改查
        this.jdbcTemplate = new JdbcTemplate(dataSource);
```

```

        this.platformTransactionManager = new
DataSourceTransactionManager(dataSource);
        this.jdbcTemplate.update("truncate table t_user");
    }

    @Test
    public void m0() throws Exception {
        System.out.println("PROPAGATION_REQUIRED start");
        //2.定义事务属性: TransactionDefinition, TransactionDefinition可以用来配置事务
        的属性信息, 比如事务隔离级别、事务超时时间、事务传播方式、是否是只读事务等等。
        TransactionDefinition transactionDefinition = new
DefaultTransactionDefinition(TransactionDefinition.PROPAGATION_REQUIRED);
        //3.开启事务: 调用platformTransactionManager.getTransaction开启事务操作, 得到
        事务状态(TransactionStatus)对象
        TransactionStatus transactionStatus =
platformTransactionManager.getTransaction(transactionDefinition);
        this.addSynchronization("ts-1", 2);
        this.addSynchronization("ts-2", 1);
        //4.执行业务操作, 下面就执行2个插入操作
        jdbcTemplate.update("insert into t_user (name) values (?)", "test1-1");
        jdbcTemplate.update("insert into t_user (name) values (?)", "test1-2");
        this.m1();
        //5.提交事务: platformTransactionManager.commit
        System.out.println("PROPAGATION_REQUIRED 准备commit");
        platformTransactionManager.commit(transactionStatus);
        System.out.println("PROPAGATION_REQUIRED commit完毕");

        System.out.println("after:" + jdbcTemplate.queryForList("SELECT * from
t_user"));
    }

    public void m1() {
        System.out.println("PROPAGATION_REQUIRES_NEW start");
        TransactionDefinition transactionDefinition = new
DefaultTransactionDefinition(TransactionDefinition.PROPAGATION_REQUIRES_NEW);
        TransactionStatus transactionStatus =
platformTransactionManager.getTransaction(transactionDefinition);
        jdbcTemplate.update("insert into t_user (name) values (?)", "test2-1");
        jdbcTemplate.update("insert into t_user (name) values (?)", "test2-2");
        this.addSynchronization("ts-3", 2);
        this.addSynchronization("ts-4", 1);
        System.out.println("PROPAGATION_REQUIRES_NEW 准备commit");
        platformTransactionManager.commit(transactionStatus);
        System.out.println("PROPAGATION_REQUIRES_NEW commit完毕");
    }

    public void addSynchronization(final String name, final int order) {
        if (TransactionSynchronizationManager.isSynchronizationActive()) {
            TransactionSynchronizationManager.registerSynchronization(new
TransactionSynchronizationAdapter() {
                @Override
                public int getOrder() {
                    return order;
                }
            })

            @Override
            public void suspend() {
                System.out.println(name + ":suspend");
            }
        }
    }

```

```

    }

    @Override
    public void resume() {
        System.out.println(name + ":resume");
    }

    @Override
    public void flush() {
        System.out.println(name + ":flush");
    }

    @Override
    public void beforeCommit(boolean readOnly) {
        System.out.println(name + ":beforeCommit:" + readOnly);
    }

    @Override
    public void beforeCompletion() {
        System.out.println(name + ":beforeCompletion");
    }

    @Override
    public void afterCommit() {
        System.out.println(name + ":afterCommit");
    }

    @Override
    public void afterCompletion(int status) {
        System.out.println(name + ":afterCompletion:" + status);
    }
    });
}
}
}
}

```

3、输出

```

PROPAGATION_REQUIRED start
PROPAGATION_REQUIRES_NEW start
ts-2:suspend
ts-1:suspend
PROPAGATION_REQUIRES_NEW 准备commit
ts-4:beforeCommit:false
ts-3:beforeCommit:false
ts-4:beforeCompletion
ts-3:beforeCompletion
ts-4:afterCommit
ts-3:afterCommit
ts-4:afterCompletion:0
ts-3:afterCompletion:0
ts-2:resume
ts-1:resume
PROPAGATION_REQUIRES_NEW commit完毕
PROPAGATION_REQUIRED 准备commit
ts-2:beforeCommit:false

```

```
ts-1:beforeCommit:false
ts-2:beforeCompletion
ts-1:beforeCompletion
ts-2:afterCommit
ts-1:afterCommit
ts-2:afterCompletion:0
ts-1:afterCompletion:0
PROPAGATION_REQUIRED commit完毕
after:[{id=1, name=test1-1}, {id=2, name=test1-2}, {id=3, name=test2-1}, {id=4,
name=test2-2}]
```

输出配合案例源码，大家理解一下。

总结一下

今天的内容挺长的，辛苦大家了，不过我相信spring事务这块吃透了，会让你收获很多，加油！

事务方面有任何问题的欢迎给我留言，下篇文章将解析声明式事务的源码，敬请期待！！**

案例源码

git地址：

<https://gitee.com/javacode2018/spring-series>

本文案例对应源码：

spring-series\lesson-002-tx\src\main\java\com\javacode2018\tx\demo9

路人甲java所有案例代码以后都会放到这个上面，大家watch一下，可以持续关注动态。

Spring系列

1. [Spring系列第1篇：为何要学spring?](#)
2. [Spring系列第2篇：控制反转 \(IoC\) 与依赖注入 \(DI\)](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)
4. [Spring系列第4篇：xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇：创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇：玩转bean scope，避免跳坑里！](#)
7. [Spring系列第7篇：依赖注入之手动注入](#)
8. [Spring系列第8篇：自动注入 \(autowire\) 详解，高手在于坚持](#)
9. [Spring系列第9篇：depend-on到底是干什么的?](#)
10. [Spring系列第10篇：primary可以解决什么问题?](#)
11. [Spring系列第11篇：bean中的autowire-candidate又是干什么的?](#)
12. [Spring系列第12篇：lazy-init: bean延迟初始化](#)
13. [Spring系列第13篇：使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇：lookup-method和replaced-method比较陌生，怎么玩的?](#)
15. [Spring系列第15篇：代理详解 \(Java动态代理&cglib代理\) ?](#)
16. [Spring系列第16篇：深入理解java注解及spring对注解的增强 \(预备知识\)](#)
17. [Spring系列第17篇：@Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇：@ComponentScan、@ComponentScans详解\(bean批量注册\)](#)

19. [Spring系列第18篇: @import详解\(bean批量注册\)](#)
20. [Spring系列第20篇: @Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇: 注解实现依赖注入 \(@Autowired、@Resource、@Primary、@Qualifier\)](#)
22. [Spring系列第22篇: @Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇: Bean生命周期详解](#)
24. [Spring系列第24篇: 父子容器详解](#)
25. [Spring系列第25篇: @Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇: 国际化详解](#)
27. [Spring系列第27篇: spring事件机制详解](#)
28. [Spring系列第28篇: Bean循环依赖详解](#)
29. [Spring系列第29篇: BeanFactory扩展 \(BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor\)](#)
30. [Spring系列第30篇: jdk动态代理和cglib代理](#)
31. [Spring系列第31篇: aop概念详解](#)
32. [Spring系列第32篇: AOP核心源码、原理详解](#)
33. [Spring系列第33篇: ProxyFactoryBean创建AOP代理](#)
34. [Spring系列第34篇: @Aspect中@Pointcut 12种用法](#)
35. [Spring系列第35篇: @Aspect中5中通知详解](#)
36. [Spring系列第36篇: @EnableAspectJAutoProxy、@Aspect中通知顺序详解](#)
37. [Spring系列第37篇: @EnableAsync & @Async 实现方法异步调用](#)
38. [Spring系列第38篇: @Scheduled & @EnableScheduling定时器详解](#)
39. [Spring系列第39篇: 强大的Spel表达式](#)
40. [Spring系列第40篇: 缓存使用 \(@EnableCaching、@Cacheable、@CachePut、@CacheEvict、@Caching、@CacheConfig\)](#)
41. [Spring系列第41篇: @EnableCaching集成redis缓存](#)
42. [Spring系列第42篇: 玩转JdbcTemplate](#)
43. [Spring系列第43篇: spring中编程式事务怎么用的?](#)
44. [Spring系列第44篇: 详解spring声明式事务\(@Transactional\)](#)
45. [Spring系列第45篇: 带你吃透Spring事务7种传播行为](#)
46. [Spring系列第46篇: Spring如何管理多数据源事务?](#)

更多好文章

1. [Java高并发系列 \(共34篇\)](#)
2. [MySQL高手系列 \(共27篇\)](#)
3. [Maven高手系列 \(共10篇\)](#)
4. [Mybatis系列 \(共12篇\)](#)
5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要, 它是什么? 怎么实现?](#)
7. [泛型, 有点难度, 会让很多人懵逼, 那是因为你没有看这篇文章!](#)

加我微信: [spring888java](#), 进群一起交流java相关各种技术!!

Spring系列第48篇: @Transaction 事务源码解析

大家好, 今天咱们通过源码来了解一下spring中@Transaction事务的原理。

开始本文之前, 下面这些知识需提前了解下

1、[吃透Spring AOP](#)

2、[Spring编程式事务源码解析](#)

在这里插播两句，整个系列前后知识是有依赖的，大家最好按顺序阅读，这样不会出现无法理解的情况，若跳着读，可能会比较懵。。。

1、环境

1. jdk1.8
2. Spring版本：5.2.3.RELEASE
3. mysql5.7

2、@Transaction 事务的用法

咱们先来回顾一下，@Transaction 事务的用法，特别简单，2个步骤

- 1、在需要让spring管理事务的方法上添加 @Transaction 注解
- 2、在spring配置类上添加 @EnableTransactionManagement 注解，这步特别重要，别给忘了，有了这个注解之后，@Transaction标注的方法才会生效。

3、@Transaction事务原理

原理比较简单，内部是通过spring aop的功能，通过拦截器拦截 @Transaction 方法的执行，在方法前后添加事务的功能。

4、@EnableTransactionManagement注解作用

@EnableTransactionManagement注解会开启spring自动管理事务的功能，有了这个注解之后，spring容器启动的过程中，会拦截所有bean的创建过程，判断bean 是否需要让spring来管理事务，即判断bean中是否有@Transaction注解，判断规则如下

- 1、一直沿着当前bean的类向上找，先从当前类中，然后父类、父类的父类，当前类的接口、接口父接口，父接口的父接口，一直向上找，一下这些类型上面是否有 @Transaction注解
- 2、类的任意public方法上面是否有@Transaction注解

如果bean满足上面任意一个规则，就会被spring容器通过aop的方式创建代理，代理中会添加一个拦截器

```
org.springframework.transaction.interceptor.TransactionInterceptor
```

TransactionInterceptor 拦截器是关键，它会拦截@Transaction方法的执行，在方法执行前后添加事务的功能，这个拦截器中大部分都是编程式事务的代码，若 [编程式事务的源码](#) 大家看懂了，这个拦截器源码看起来就是小儿科了。

5、@EnableTransactionManagement源码解析

```

@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Import(TransactionManagementConfigurationSelector.class) //@1
public @interface EnableTransactionManagement {

    // 是基于类的代理（cglib），还是基于接口的代理（jdk动态代理），默认为false，表示是基于
    jdk动态代理
    boolean proxyTargetClass() default false;

    //通知的模式，默认是通过aop的方式
    AdviceMode mode() default AdviceMode.PROXY;

    // 我们知道这个注解的功能最终是通过aop的方式来实现的，对bean创建了一个代理，代理中添加了一个
    拦截器
    // 当代理中还有其他拦截器的时候，可以通过order这个属性来指定事务拦截器的顺序
    // 默认值是 LOWEST_PRECEDENCE = Integer.MAX_VALUE,拦截器的执行顺序是order升序
    int order() default Ordered.LOWEST_PRECEDENCE;

}

```

注意 @1 这个代码

```

@Import(TransactionManagementConfigurationSelector.class)

```

用到了@Import注解，对这个注解不熟悉的可以看一下[Spring系列第18篇：@import详解\(bean批量注册\)](#)，这个注解的value是TransactionManagementConfigurationSelector，看一下这个类的源码，重点是他的 selectImports 方法，这个方法会返回一个类名数组，spring容器启动过程中会自动调用这个方法，将这个方法指定的类注册到spring容器中；方法的参数是AdviceMode，这个就是@EnableTransactionManagement注解中mode属性的值，默认是PROXY，所以会走到@1代码处

```

public class TransactionManagementConfigurationSelector extends
AdviceModeImportSelector<EnableTransactionManagement> {

    @Override
    protected String[] selectImports(AdviceMode adviceMode) {
        switch (adviceMode) {
            case PROXY: //@1
                return new String[] {AutoProxyRegistrar.class.getName(),
ProxyTransactionManagementConfiguration.class.getName()};
            case ASPECTJ:
                return new String[] {determineTransactionAspectClass()};
            default:
                return null;
        }
    }
}

```

最终会在spring容器中注册下面这2个bean

```

AutoProxyRegistrar
ProxyTransactionManagementConfiguration

```


下面来看一下这2个类的代码。

AutoProxyRegistrar

这个类实现了ImportBeanDefinitionRegistrar接口，这个接口中有个方法

`registerBeanDefinitions`，spring容器在启动过程中会调用这个方法，开发者可以在这个方法中做一些bean注册的事情，而AutoProxyRegistrar在这个方法中主要做的事情就是下面 @1 的代码，大家可以点进去看看，这里我就不点进去了，这个代码的作用就是在容器中做了一个非常关键的bean：InfrastructureAdvisorAutoProxyCreator，这个类之前在aop中介绍过，是bean后置处理器，会拦截所有bean的创建，对符合条件的bean创建代理。

```
public class AutoProxyRegistrar implements ImportBeanDefinitionRegistrar {

    @Override
    public void registerBeanDefinitions(AnnotationMetadata
importingClassMetadata, BeanDefinitionRegistry registry) {
        boolean candidateFound = false;
        Set<String> annTypes = importingClassMetadata.getAnnotationTypes();
        for (String annType : annTypes) {
            AnnotationAttributes candidate =
AnnotationConfigUtils.attributesFor(importingClassMetadata, annType);
            if (candidate == null) {
                continue;
            }
            Object mode = candidate.get("mode");
            Object proxyTargetClass = candidate.get("proxyTargetClass");
            if (mode != null && proxyTargetClass != null && AdviceMode.class ==
mode.getClass() &&
                Boolean.class == proxyTargetClass.getClass()) {
                candidateFound = true;
                if (mode == AdviceMode.PROXY) {

AopConfigUtils.registerAutoProxyCreatorIfNecessary(registry); // @1
                    if ((Boolean) proxyTargetClass) {

AopConfigUtils.forceAutoProxyCreatorToUseClassProxying(registry);
                        return;
                    }
                }
            }
        }
    }
}
```

说的简单点：AutoProxyRegistrar的作用就是启用spring aop的功能，对符合条件的bean创建代理。

ProxyTransactionManagementConfiguration

```
@Configuration(proxyBeanMethods = false)
public class ProxyTransactionManagementConfiguration extends
AbstractTransactionManagementConfiguration {
    //注册bean：事务顾问（spring aop中拦截器链就是一个个的Advisor对象）
    @Bean(name = TransactionManagementConfigUtils.TRANSACTION_ADVISOR_BEAN_NAME)
    @Role(BeanDefinition.ROLE_INFRASTRUCTURE)
```

```

    public BeanFactoryTransactionAttributeSourceAdvisor transactionAdvisor(
        TransactionAttributeSource transactionAttributeSource,
        TransactionInterceptor transactionInterceptor) {
        BeanFactoryTransactionAttributeSourceAdvisor advisor = new
BeanFactoryTransactionAttributeSourceAdvisor();
        advisor.setTransactionAttributeSource(transactionAttributeSource);
        //设置事务拦截器
        advisor.setAdvice(transactionInterceptor);
        if (this.enableTx != null) {
            //设置aop中事务拦截器的顺序
            advisor.setOrder(this.enableTx.<Integer>getNumber("order"));
        }
        return advisor;
    }

    //注册bean:TransactionAttributeSource, TransactionAttributeSource用来获取获取事
务属性配置信息: TransactionAttribute
    @Bean
    @Role(BeansDefinition.ROLE_INFRASTRUCTURE)
    public TransactionAttributeSource transactionAttributeSource() { //@1
        return new AnnotationTransactionAttributeSource();
    }

    //注册bean: 事务拦截器
    @Bean
    @Role(BeansDefinition.ROLE_INFRASTRUCTURE)
    public TransactionInterceptor transactionInterceptor(
        TransactionAttributeSource transactionAttributeSource) {
        TransactionInterceptor interceptor = new TransactionInterceptor();
        interceptor.setTransactionAttributeSource(transactionAttributeSource);
        //拦截器中设置事务管理器, txManager可以为空
        if (this.txManager != null) {
            interceptor.setTransactionManager(this.txManager);
        }
        return interceptor;
    }
}

```

是个配置类，代码比较简单，注册了3个bean，最重要的一点就是添加了事务事务拦截器：TransactionInterceptor。

AutoProxyRegistrar负责启用aop的功能，而ProxyTransactionManagementConfiguration负责在aop中添加事务拦截器，二者结合起来的效果就是：对@Transaction标注的bean创建代理对象，代理对象中通过TransactionInterceptor拦截器来实现事务管理的功能。

再看下代码 @1，注册了一个TransactionAttributeSource类型的bean

TransactionAttributeSource接口源码：

```

public interface TransactionAttributeSource {

    /**
     * 确定给定的类是否是这个TransactionAttributeSource元数据格式中的事务属性的候选类。
     * 如果此方法返回false，则不会遍历给定类上的方法，以进行getTransactionAttribute内省。
     * 因此，返回false是对不受影响的类的优化，而返回true仅仅意味着类需要对给定类上的每个方法进行完全自省。
     */
}

```

```

    /**/
    default boolean isCandidateClass(Class<?> targetClass) {
        return true;
    }

    //返回给定方法的事务属性，如果该方法是非事务性的，则返回null。
    TransactionAttribute getTransactionAttribute(Method method, @Nullable
    Class<?> targetClass);
}

```

getTransactionAttribute方法用来获取指定方法上的事务属性信息TransactionAttribute，大家对TransactionDefinition比较熟悉吧，用来配置事务属性信息的，而TransactionAttribute继承了TransactionDefinition接口，源码如下，而TransactionAttribute中新定义了2个方法，一个方法用来指定事务管理器bean名称的，一个用来判断给定的异常是否需要回滚事务

```

public interface TransactionAttribute extends TransactionDefinition {

    //事务管理器的bean名称
    @Nullable
    String getQualifier();

    //判断指定的异常是否需要回滚事务
    boolean rollbackOn(Throwable ex);
}

```

TransactionAttributeSource接口有个实现类AnnotationTransactionAttributeSource，负责将@Transaction解析为TransactionAttribute对象，大家可以去这个类中设置一下断点看一下@Transaction注解查找的顺序，这样可以深入理解@Transaction放在什么地方才会让事务起效。

AnnotationTransactionAttributeSource内部最会委托给

SpringTransactionAnnotationParser#parseTransactionAnnotation方法来解析@Transaction注解，进而得到事务属性配置信息：RuleBasedTransactionAttribute，代码如下：

```

org.springframework.transaction.annotation.SpringTransactionAnnotationParser

protected TransactionAttribute parseTransactionAnnotation(AnnotationAttributes
attributes) {
    RuleBasedTransactionAttribute rbta = new RuleBasedTransactionAttribute();

    Propagation propagation = attributes.getEnum("propagation");
    rbta.setPropagationBehavior(propagation.value());
    Isolation isolation = attributes.getEnum("isolation");
    rbta.setIsolationLevel(isolation.value());
    rbta.setTimeout(attributes.getNumber("timeout").intValue());
    rbta.setReadOnly(attributes.getBoolean("readOnly"));
    rbta.setQualifier(attributes.getString("value"));

    //回滚规则
    List<RollbackRuleAttribute> rollbackRules = new ArrayList<>();
    for (Class<?> rbRule : attributes.getClassArray("rollbackFor")) {
        rollbackRules.add(new RollbackRuleAttribute(rbRule));
    }
    for (String rbRule : attributes.getStringArray("rollbackForClassName")) {
        rollbackRules.add(new RollbackRuleAttribute(rbRule));
    }
}

```

```

    }
    for (Class<?> rbRule : attributes.getClassArray("noRollbackFor")) {
        rollbackRules.add(new NoRollbackRuleAttribute(rbRule));
    }
    for (String rbRule : attributes.getStringArray("noRollbackForClassName")) {
        rollbackRules.add(new NoRollbackRuleAttribute(rbRule));
    }
    rbta.setRollbackRules(rollbackRules);

    return rbta;
}

```

下面来看重点了事务拦截器。

6、TransactionInterceptor

负责拦截@Transaction方法的执行，在方法执行之前开启spring事务，方法执行完毕之后提交或者回滚事务。

在讲这个类的源码之前，先提几个问题，大家带着问题去看代码，理解更深一些。

- 1、事务管理器是如何获取的？
- 2、什么情况下事务会提交？
- 3、什么异常会导致事务回滚？

6.1、invokeWithinTransaction方法

这个方法是事务拦截器的入口，需要spring管理事务的业务方法会被这个方法拦截，大家可以设置断点跟踪一下

```

protected Object invokeWithinTransaction(Method method, @Nullable Class<?>
targetClass,
        final InvocationCallback invocation) throws Throwable {

    TransactionAttributeSource tas = getTransactionAttributeSource();
    // @6-1: 获取事务属性配置信息：通过
    TransactionAttributeSource.getTransactionAttribute解析@Trasaction注解得到事务属性配置
    信息
    final TransactionAttribute txAttr = (tas != null ?
    tas.getTransactionAttribute(method, targetClass) : null);
    // @6-2: 获取事务管理器
    final TransactionManager tm = determineTransactionManager(txAttr);

    // 将事务管理器tx转换为 PlatformTransactionManager
    PlatformTransactionManager ptm = asPlatformTransactionManager(tm);

    if (txAttr == null || !(ptm instanceof
    CallbackPreferringPlatformTransactionManager)) {
        // createTransactionIfNecessary内部，这里就不说了，内部主要就是使用spring事务硬
        编码的方式开启事务，最终会返回一个TransactionInfo对象
        TransactionInfo txInfo = createTransactionIfNecessary(ptm, txAttr,
        joinpointIdentification);
        // 业务方法返回值
        Object retVal;
    }
}

```

```

    try {
        //调用aop中的下一个拦截器，最终会调用到业务目标方法，获取到目标方法的返回值
        retVal = invocation.proceedWithInvocation();
    }
    catch (Throwable ex) {
        //6-3: 异常情况下，如何走？可能只需提交，也可能只需回滚，这个取决于事务的配置
        completeTransactionAfterThrowing(txInfo, ex);
        throw ex;
    }
    finally {
        //清理事务信息
        cleanupTransactionInfo(txInfo);
    }
    //6-4: 业务方法返回之后，只需事务提交操作
    commitTransactionAfterReturning(txInfo);
    //返回执行结果
    return retVal;
}
}

```

6.2、获取事务管理器

```

//@6-2: 获取事务管理器
final TransactionManager tm = determineTransactionManager(txAttr);

```

determineTransactionManager源码如下:

```

org.springframework.transaction.interceptor.TransactionAspectSupport#determineTransactionManager

protected TransactionManager determineTransactionManager(@Nullable
TransactionAttribute txAttr) {
    // txAttr == null || this.beanFactory == null , 返回拦截器中配置的事务管理器
    if (txAttr == null || this.beanFactory == null) {
        return getTransactionManager();
    }

    //qualifier就是@Transactional注解中通过value或者transactionManager来指定事务管理器的
    的bean名称
    String qualifier = txAttr.getQualifier();
    if (StringUtils.hasText(qualifier)) {
        //从spring容器中查找[beanName:qualifier,type:TransactionManager]的bean
        return determineQualifiedTransactionManager(this.beanFactory,
qualifier);
    }
    else if (StringUtils.hasText(this.transactionManagerBeanName)) {
        //从spring容器中查找
[beanName:this.transactionManagerBeanName,type:TransactionManager]的bean
        return determineQualifiedTransactionManager(this.beanFactory,
this.transactionManagerBeanName);
    }
    else {
        //最后通过类型TransactionManager在spring容器中找事务管理器
        TransactionManager defaultManager = getTransactionManager();
        if (defaultManager == null) {

```

```

        defaultTransactionManager =
this.transactionManagerCache.get(DEFAULT_TRANSACTION_MANAGER_KEY);
        if (defaultTransactionManager == null) {
            defaultTransactionManager =
this.beanFactory.getBean(TransactionManager.class);
            this.transactionManagerCache.putIfAbsent(
                DEFAULT_TRANSACTION_MANAGER_KEY, defaultTransactionManager);
        }
    }
    return defaultTransactionManager;
}
}

```

从上面可知，事务管理器的查找顺序：

- 1、先看@Transactional中是否通过value或者transactionManager指定了事务管理器
- 2、TransactionInterceptor.transactionManagerBeanName是否有值，如果有，将通过这个值查找事务管理器
- 3、如果上面2种都没有，将从spring容器中查找TransactionManager类型的事务管理器

6.3、异常情况下，如何走？

```

try{
    //....
}catch (Throwable ex) {
    //6-3: 异常情况下，如何走？可能只需提交，也可能只需回滚，这个取决于事务的配置
    completeTransactionAfterThrowing(txInfo, ex);
    throw ex;
}

```

源码中可以看出，发生异常了会进入 `completeTransactionAfterThrowing` 方法，`completeTransactionAfterThrowing` 源码如下

```

protected void completeTransactionAfterThrowing(@Nullable TransactionInfo
txInfo, Throwable ex) {
    if (txInfo != null && txInfo.getTransactionStatus() != null) {
        //6-3-1: 判断事务是否需要回滚
        if (txInfo.transactionAttribute != null &&
txInfo.transactionAttribute.rollbackOn(ex)) {
            //通过事务管理器回滚事务

            txInfo.getTransactionManager().rollback(txInfo.getTransactionStatus());
        }
        else {
            //通过事务管理器提交事务

            txInfo.getTransactionManager().commit(txInfo.getTransactionStatus());
        }
    }
}

```

注意上面的 @6-3-1 代码，判断事务是否需要回滚，调用的是

`transactionAttribute.rollbackOn(ex)`，最终会进入下面这个方法内部

```
org.springframework.transaction.interceptor.RuleBasedTransactionAttribute#rollbackOn

public boolean rollbackOn(Throwable ex) {
    RollbackRuleAttribute winner = null;
    int deepest = Integer.MAX_VALUE;

    // @Transactional 中可以通过 rollbackFor 指定需要回滚的异常列表，通过 noRollbackFor 属性指定不需要回滚的异常
    // 根据 @Transactional 中指定的回滚规则判断 ex 类型的异常是否需要回滚
    if (this.rollbackRules != null) {
        for (RollbackRuleAttribute rule : this.rollbackRules) {
            int depth = rule.getDepth(ex);
            if (depth >= 0 && depth < deepest) {
                deepest = depth;
                winner = rule;
            }
        }
    }
    // 若 @Transactional 注解中没有匹配到，这走默认的规则，将通过 super.rollbackOn 来判断
    if (winner == null) {
        return super.rollbackOn(ex);
    }

    return !(winner instanceof NoRollbackRuleAttribute);
}
```

`super.rollbackOn(ex)`源码如下，可以看出默认情况下，异常类型是 `RuntimeException` 或者 `Error` 的情况下，事务才会回滚。

```
@Override
public boolean rollbackOn(Throwable ex) {
    return (ex instanceof RuntimeException || ex instanceof Error);
}
```

6.4、没有异常如何走？

```
//6-4: 业务方法返回之后，只需事务提交操作
commitTransactionAfterReturning(txInfo);
```

没有异常的情况下会进入 `commitTransactionAfterReturning` 方法，

`commitTransactionAfterReturning` 源码如下，比较简单，就是调用事务管理器的 `commit` 方法提交事务

```
protected void commitTransactionAfterReturning(@Nullable TransactionInfo txInfo)
{
    if (txInfo != null && txInfo.getTransactionStatus() != null) {
        txInfo.getTransactionManager().commit(txInfo.getTransactionStatus());
    }
}
```

源码解析的差不多了，建议大家设置断点跟踪一下，加深对事务原理的理解。

7、重点回顾

- 1、使用@Transaction的时候，一定别忘记@EnableTransactionManagement注解，否则事务不起效
- 2、@Transaction的功能主要是通过aop来实现的，关键代码在TransactionInterceptor拦截器中
- 3、默认情况下，事务只会在 RuntimeException 或 Error 异常下回滚，可以通过@Transaction来配置其他需要回滚或不需要回滚的异常类型

8、课堂讨论

下面代码的执行结果是什么？为什么？

```
@Component
public class ServiceA {

    @Autowired
    ServiceB serviceB;

    @Transactional
    public void m1() {
        try {
            serviceB.m2();
        } catch (Exception e) {
            e.printStackTrace();
        }
        "insert into t_user (name) values ('张三')";
    }
}

@Component
public class ServiceB {

    @Transactional
    public void m2() {
        "insert into t_user (name) values ('李四')";
        throw new RuntimeException("手动抛出异常!");
    }
}
```

欢迎留言和我分享你的想法。如果有收获，也欢迎你把这篇文章分享给你的朋友。

Spring系列

1. [Spring系列第1篇：为何要学spring?](#)
2. [Spring系列第2篇：控制反转 \(IoC\) 与依赖注入 \(DI\)](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)
4. [Spring系列第4篇：xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇：创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇：玩转bean scope，避免跳坑里!](#)
7. [Spring系列第7篇：依赖注入之手动注入](#)
8. [Spring系列第8篇：自动注入 \(autowire\) 详解，高手在于坚持](#)
9. [Spring系列第9篇：depend-on到底是干什么的?](#)
10. [Spring系列第10篇：primary可以解决什么问题?](#)
11. [Spring系列第11篇：bean中的autowire-candidate又是干什么的?](#)
12. [Spring系列第12篇：lazy-init: bean延迟初始化](#)
13. [Spring系列第13篇：使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇：lookup-method和replaced-method比较陌生，怎么玩的?](#)
15. [Spring系列第15篇：代理详解 \(Java动态代理&cglb代理\) ?](#)
16. [Spring系列第16篇：深入理解java注解及spring对注解的增强 \(预备知识\)](#)
17. [Spring系列第17篇：@Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇：@ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇：@import详解\(bean批量注册\)](#)
20. [Spring系列第20篇：@Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇：注解实现依赖注入 \(@Autowired、@Resource、@Primary、@Qualifier\)](#)
22. [Spring系列第22篇：@Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇：Bean生命周期详解](#)
24. [Spring系列第24篇：父子容器详解](#)
25. [Spring系列第25篇：@Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇：国际化详解](#)
27. [Spring系列第27篇：spring事件机制详解](#)
28. [Spring系列第28篇：Bean循环依赖详解](#)
29. [Spring系列第29篇：BeanFactory扩展 \(BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor\)](#)
30. [Spring系列第30篇：jdk动态代理和cglb代理](#)
31. [Spring系列第31篇：aop概念详解](#)
32. [Spring系列第32篇：AOP核心源码、原理详解](#)
33. [Spring系列第33篇：ProxyFactoryBean创建AOP代理](#)
34. [Spring系列第34篇：@Aspect中@Pointcut 12种用法](#)
35. [Spring系列第35篇：@Aspect中5中通知详解](#)
36. [Spring系列第36篇：@EnableAspectJAutoProxy、@Aspect中通知顺序详解](#)
37. [Spring系列第37篇：@EnableAsync & @Async 实现方法异步调用](#)
38. [Spring系列第38篇：@Scheduled & @EnableScheduling定时器详解](#)
39. [Spring系列第39篇：强大的Spel表达式](#)
40. [Spring系列第40篇：缓存使用 \(@EnableCaching、@Cacheable、@CachePut、@CacheEvict、@Caching、@CacheConfig\)](#)
41. [Spring系列第41篇：@EnableCaching集成redis缓存](#)
42. [Spring系列第42篇：玩转jdbcTemplate](#)
43. [Spring系列第43篇：spring中程式事务怎么用的?](#)
44. [Spring系列第44篇：详解spring声明式事务\(@Transactional\)](#)
45. [Spring系列第45篇：带你吃透Spring事务7种传播行为](#)
46. [Spring系列第46篇：Spring如何管理多数据源事务?](#)
47. [Spring系列第47篇：spring事务源码解析](#)

更多好文章

1. [Java高并发系列（共34篇）](#)
2. [MySql高手系列（共27篇）](#)
3. [Maven高手系列（共10篇）](#)
4. [Mybatis系列（共12篇）](#)
5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要，它是什么？怎么实现？](#)
7. [泛型，有点难度，会让很多人懵逼，那是因为你没有看这篇文章！](#)

加我微信：spring888java，进群一起交流java相关各种技术！！

Spring系列第49篇：实战篇：手把手带你实现事务消息！

1、本文2个目的

- 1、讨论一下消息投递的5种方式
- 2、带你手写代码，实现事务消息的投递

2、讨论一下消息投递的5种方式

2.1、业务场景

电商中有这样的一个场景：商品下单之后，需给用户送积分，订单表和积分表分别在不同的db中，涉及到分布式事务的问题。

我们通过可靠消息来解决这个问题：

1. 商品下单成功之后送积分的操作，我们使用mq来实现
2. 商品下单成功之后，投递一条消息到mq，积分系统消费消息，给用户增加积分

我们主要讨论一下，商品下单及投递消息到mq的操作，如何实现？每种方式优缺点？

2.2、方式一

过程

- step1：开启本地事务
- step2：生成购物订单
- step3：投递消息到mq
- step4：提交本地事务

这种方式是将发送消息放在了事务提交之前。

可能存在的问题

- **step3发生异常**：导致step4失败，商品下单失败，直接影响到商品下单业务
- **step4发生异常，其他step成功**：商品下单失败，消息投递成功，给用户增加了积分

2.3、方式二

下面我们换种方式，我们将发送消息放到事务之后进行。

过程

- **step1**：开启本地事务
- **step2**：生成购物订单
- **step3**：提交本地事务
- **step4**：投递消息到mq

可能会出现的问题

step4发生异常，其他step成功：导致商品下单成功，投递消息失败，用户未增加积分

上面两种是比较常见的做法，也是最容易出错的。

2.4、方式三

- **step1**：开启本地事务
- **step2**：生成购物订单
- **step3**：本地库中插入一条需要发送消息的记录t_msg_record
- **step3**：提交本地事务
- **step5**：新增一个定时器，轮询t_msg_record，将待发送的记录投递到mq中

这种方式借助了数据库的事务，业务和消息记录作为了一个原子操作，业务成功之后，消息日志必定是存在的。解决了前两种方式遇到的问题。如果我们的业务系统比较单一，可以采用这种方式。

对于微服务化的情况，上面这种方式不是太好，每个服务都需要上面的操作；也不利于扩展。

2.5、方式四

增加一个**消息服务及消息库**，负责消息的落库、将消息发送投递到mq。

- **step1**：开启本地事务
- **step2**：生成购物订单
- **step3**：当前事务库插入一条日志：生成一个唯一的业务id (msg_order_id)，将msg_order_id和订单关联起来保存到当前事务所在的库中
- **step4**：调用消息服务：携带msg_order_id，将消息先落地入库，此时消息的状态为待发送状态，返回消息id(msg_id)
- **step5**：提交本地事务
- **step6**：如果上面都成功，调用消息服务，将消息投递到mq中；如果上面有失败的情况，则调用消息服务取消消息的发送

能想到上面这种方式，已经算是有很大进步了，我们继续分析一下可能存在的问题：

1. 系统中增加了一个消息服务，商品下单操作依赖于该服务，业务对该服务依赖性比较高，当消息服务不可用时，整个业务将不可用。

2. 若step6失败，消息将处于待发送状态，此时业务方需要提供一个回查接口（通过msg_order_id查询），验证业务是否执行成功；消息服务需新增一个定时任务，对于状态为待发送状态的消息做补偿处理，检查一下业务是否处理成功；从而确定消息是投递还是取消发送
3. step4依赖于消息服务，如果消息服务性能不佳，会导致当前业务的事务提交时间延长，**容易产生死锁，并导致并发性能降低**。我们通常是比较忌讳在事务中做远程调用处理的，远程调用的性能和时间往往不可控，会导致当前事务变为一个大事务，从而引发其他故障。

2.6、方式五

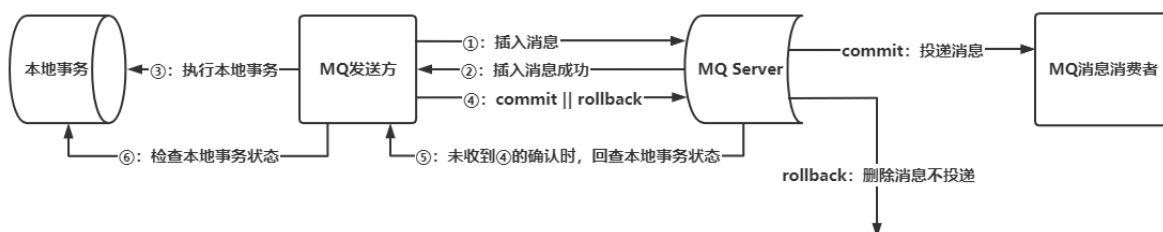
在以上方式中，我们继续改进，进而出现了更好的一种方式：

- **step1**：生成一个全局唯一业务消息id (bus_msg_id)，调用消息服务，携带bus_msg_id，将消息先落地入库，此时消息的状态为待发送状态，返回消息id (msg_id)
- **step2**：开启本地事务
- **step3**：生成购物订单
- **step4**：当前事务库插入一条日志（将step3中的业务和bus_msg_id关联起来）
- **step5**：提交本地事务
- **step6**：分2种情况：如果上面都成功，调用消息服务，将消息投递到mq中；如果上面有失败的情况，则调用消息服务取消消息的发送

若step6失败，消息将处于待发送状态，此时业务方需要提供一个回查接口（通过bus_msg_id查询），验证业务是否执行成功；

消息服务需新增一个定时任务，对于状态为待发送状态的消息做补偿处理，检查一下业务是否处理成功；从而确定消息是投递还是取消发送。

方式五和方式四对比，比较好的一个地方：将调用消息服务，消息落地操作，放在了事务之外进行，这点小的改进其实算是一个非常好的优化，减少了本地事务的执行时间，从而可以提升并发量，阿里有个消息中间件RocketMQ就支持方式5这种，大家可以去用用。



下面我们通过代码来实现方式4。

3、方式4代码实现

3.1、准备数据库

3张表：

t_user：业务库中的用户表，一会用来模拟用户注册，注册成功之后投递消息。

t_msg_order：消息订单表，这个表放在业务库中，业务操作中若需要发送消息，则在业务操作的事务中同时向t_msg_order表插入一条数据，若业务操作成功，那么t_msg_order表肯定也会成功插入一条数据，发送消息的时候携带上t_msg_order的id，消息服务可以通过这个id去业务库回查t_msg_order中的记录，如果记录存在，则说明业务操作成功了。

t_msg: 消息表, 所有发送的消息信息放在这个表中, 主要字段有: 消息内容, msg_order_id: 来源于 t_msg_order表的id

```
DROP DATABASE IF EXISTS javacode2018;
CREATE DATABASE if NOT EXISTS javacode2018;

USE javacode2018;
DROP TABLE IF EXISTS t_user;
CREATE TABLE t_user(
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(256) NOT NULL DEFAULT '' COMMENT '姓名'
);

DROP TABLE IF EXISTS t_msg;
CREATE TABLE t_msg(
    id INT PRIMARY KEY AUTO_INCREMENT,
    msg VARCHAR(256) NOT NULL DEFAULT '' COMMENT '消息内容, 可以json格式的数据',
    msg_order_id BIGINT NOT NULL DEFAULT 0 COMMENT '消息订单id',
    status SMALLINT NOT NULL DEFAULT 0 COMMENT '消息状态,0:待投递, 1: 已发送, 2: 取消发送'
) COMMENT '消息表';

DROP TABLE IF EXISTS t_msg_order;
CREATE TABLE t_msg_order(
    id INT PRIMARY KEY AUTO_INCREMENT,
    ref_type BIGINT NOT NULL DEFAULT 0 COMMENT '关联业务类型',
    ref_id VARCHAR(256) NOT NULL DEFAULT '' COMMENT '关联业务id (ref_type & ref_id 唯一)'
) COMMENT '消息订单表,放在业务库中';

alter table t_msg_order add UNIQUE INDEX idx1 (ref_type,ref_id);
```

3.2、关键java代码

配置类MainConfig11

```
package com.javacode2018.tx.demo11;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;

import javax.sql.DataSource;

/**
 * 公众号: 路人甲Java, 工作10年的前阿里P7分享Java、算法、数据库方面的技术干货!
 * <a href="http://www.itsoku.com">个人博客</a>
 */
@ComponentScan
@EnableTransactionManagement
public class MainConfig11 {
    @Bean
    public DataSource dataSource() {
```

```

        org.apache.tomcat.jdbc.pool.DataSource dataSource = new
org.apache.tomcat.jdbc.pool.DataSource();
        dataSource.setDriverClassName("com.mysql.jdbc.Driver");
        dataSource.setUrl("jdbc:mysql://localhost:3306/javacode2018?
characterEncoding=UTF-8");
        dataSource.setUsername("root");
        dataSource.setPassword("root123");
        dataSource.setInitialSize(5);
        return dataSource;
    }

    //定义一个jdbcTemplate
    @Bean
    public JdbcTemplate jdbcTemplate(DataSource dataSource) {
        return new JdbcTemplate(dataSource);
    }

    //定义事务管理器transactionManager
    @Bean
    public PlatformTransactionManager transactionManager(DataSource dataSource)
    {
        return new DataSourceTransactionManager(dataSource);
    }
}

```

MsgModel

```

package com.javacode2018.tx.demo11;

import lombok.*;

/**
 * 公众号：路人甲Java，工作10年的前阿里P7分享Java、算法、数据库方面的技术干货！
 * <a href="http://www.itsoku.com">个人博客</a>
 */
@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
@ToString
public class MsgModel {
    private Long id;
    //消息内容
    private String msg;
    //消息订单id
    private Long msg_order_id;
    //消息状态,0:待投递, 1: 已发送, 2: 取消发送
    private Integer status;
}

```

MsgOrderModel

```
package com.javacode2018.tx.demo11;

import lombok.*;

/**
 * 公众号：路人甲Java，工作10年的前阿里P7分享Java、算法、数据库方面的技术干货！
 * <a href="http://www.itsoku.com">个人博客</a>
 */
@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class MsgOrderModel {
    private Long id;
    //关联业务类型
    private Integer ref_type;
    //关联业务id (ref_type & ref_id 唯一)
    private String ref_id;
}
```

MsgOrderService

提供了对t_msg_order表的一些操作，2个方法，一个用来插入数据，一个用来查询。

```
package com.javacode2018.tx.demo11;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.BeanPropertyRowMapper;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.Objects;

/**
 * 公众号：路人甲Java，工作10年的前阿里P7分享Java、算法、数据库方面的技术干货！
 * <a href="http://www.itsoku.com">个人博客</a>
 */
@Component
public class MsgOrdersService {
    @Autowired
    private JdbcTemplate jdbcTemplate;

    /**
     * 插入消息订单
     *
     * @param ref_type
     * @param ref_id
     * @return
     */
    @Transactional
    public MsgOrderModel insert(Integer ref_type, String ref_id) {
```

```

        MsgOrderModel msgOrderModel =
MsgOrderModel.builder().ref_type(ref_type).ref_id(ref_id).build();
        //插入消息
        this.jdbcTemplate.update("insert into t_msg_order (ref_type,ref_id)
values (?,?)",
                                ref_type,
                                ref_id
                                );
        //获取消息订单id
        msgOrderModel.setId(this.jdbcTemplate.queryForObject("SELECT
LAST_INSERT_ID()", Long.class));
        return msgOrderModel;
    }

    /**
     * 根据消息id获取消息
     *
     * @param id
     * @return
     */
    public MsgOrderModel getById(Long id) {
        List<MsgOrderModel> list = this.jdbcTemplate.query("select * from
t_msg_order where id = ? limit 1", new BeanPropertyRowMapper<MsgOrderModel>
(MsgOrderModel.class), id);
        return Objects.nonNull(list) && !list.isEmpty() ? list.get(0) : null;
    }
}

```

MsgService

消息服务，提供了对t_msg表的一些操作以及消息投递的一些方法

方法	说明
addMsg	添加消息，消息会落库，处于待发送状态
confirmSendMsg	确定投递消息，事务成功后可以调用
cancelSendMsg	取消投递消息，事务回滚可以调用

代码：

```

package com.javacode2018.tx.demo11;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.BeanPropertyRowMapper;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.Objects;

/**

```


* 公众号：路人甲Java，工作10年的前阿里P7分享Java、算法、数据库方面的技术干货！

* 个人博客

*/

@Component

public class MsgService {

 //添加一条消息(独立的事务中执行)

 @Transactional(propagation = Propagation.REQUIRES_NEW)

 public Long addMsg(String msg, Long msg_order_id, boolean isSend) {

 MsgModel msgModel =

MsgModel.builder().msg(msg).msg_order_id(msg_order_id).status(0).build();

 //先插入消息

 Long msg_id = this.insert(msgModel).getId();

 if (isSend) {

 //如果需要投递，则调用投递的方法

 this.confirmSendMsg(msg_id);

 }

 return msg_id;

 }

/**

 * 确认消息投递(不需要事务)

 *

 * @param msg_id 消息id

 */

 @Transactional(propagation = Propagation.NOT_SUPPORTED)

 public void confirmSendMsg(Long msg_id) {

 MsgModel msgModel = this.getById(msg_id);

 //向mq中投递消息

 System.out.println(String.format("投递消息: %s", msgModel));

 //将消息状态置为已投递

 this.updateStatus(msg_id, 1);

 }

/**

 * 取消消息投递(不需要事务)

 *

 * @param msg_id 消息id

 */

 @Transactional(propagation = Propagation.NOT_SUPPORTED)

 public void cancelSendMsg(Long msg_id) {

 MsgModel msgModel = this.getById(msg_id);

 System.out.println(String.format("取消投递消息: %s", msgModel));

 //将消息状态置为取消投递

 this.updateStatus(msg_id, 2);

 }

@Autowired

private JdbcTemplate jdbcTemplate;

/**

 * 插入消息

 *

 * @param msgModel

 * @return

 */

private MsgModel insert(MsgModel msgModel) {

 //插入消息

```

        this.jdbcTemplate.update("insert into t_msg (msg,msg_order_id,status)
values (?,?,?)",
        msgModel.getMsg(),
        msgModel.getMsg_order_id(),
        msgModel.getStatus());
        //获取消息id
        msgModel.setId(this.jdbcTemplate.queryForObject("SELECT
LAST_INSERT_ID()", Long.class));
        System.out.println("插入消息: " + msgModel);
        return msgModel;
    }

    /**
     * 根据消息id获取消息
     *
     * @param id
     * @return
     */
    private MsgModel getById(Long id) {
        List<MsgModel> list = this.jdbcTemplate.query("select * from t_msg where
id = ? limit 1", new BeanPropertyRowMapper<MsgModel>(MsgModel.class), id);
        return Objects.nonNull(list) && !list.isEmpty() ? list.get(0) : null;
    }

    /**
     * 更新消息状态
     *
     * @param id
     * @param status
     */
    private void updateStatus(long id, int status) {
        this.jdbcTemplate.update("update t_msg set status = ? where id = ?",
status, id);
    }
}

```

消息投递器MsgSender

消息投递器，给业务方使用，内部只有一个方法，用来发送消息。

若上下文没有事务，则消息落地之后立即投递；若存在事务，则消息投递分为2步走：消息先落地，事务执行完毕之后再确定是否投递，用到了事务扩展点：**TransactionSynchronization**，事务执行完毕之后会回调TransactionSynchronization接口中的afterCompletion方法，在这个方法中确定是否投递消息。对事务扩展点**TransactionSynchronization**不熟悉的建议先看一下这篇文章：[Spring系列第47篇：spring事务源码解析](#)

```

package com.javacode2018.tx.demo11;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
import org.springframework.transaction.support.TransactionSynchronization;
import
org.springframework.transaction.support.TransactionSynchronizationAdapter;
import
org.springframework.transaction.support.TransactionSynchronizationManager;

```

```

/**
 * 公众号：路人甲Java，工作10年的前阿里P7分享Java、算法、数据库方面的技术干货！
 * <a href="http://www.itsoku.com">个人博客</a>
 * 消息发送器，所有使用者调用send方法发送消息
 */
@Component
public class MsgSender {
    @Autowired
    private MsgOrderService msgOrderService;
    @Autowired
    private MsgService msgService;

    //发送消息
    public void send(String msg, int ref_type, String ref_id) {
        MsgOrderModel msgOrderModel = this.msgOrderService.insert(ref_type,
ref_id);

        Long msg_order_id = msgOrderModel.getId();
        //TransactionSynchronizationManager.isSynchronizationActive 可以用来判断事
务同步是否开启了
        boolean isSynchronizationActive =
TransactionSynchronizationManager.isSynchronizationActive();
        /**
         * 若事务同步开启了，那么可以在事务同步中添加事务扩展点，则先插入消息，暂不发送，则在事
务扩展点中添加回调
         * 事务结束之后会自动回调扩展点TransactionSynchronizationAdapter的
afterCompletion()方法
         * 咱们在这个方法中确定是否投递消息
         */
        if (isSynchronizationActive) {
            final Long msg_id = this.msgService.addMsg(msg, msg_order_id,
false);
            TransactionSynchronizationManager.registerSynchronization(new
TransactionSynchronizationAdapter() {
                @Override
                public void afterCompletion(int status) {
                    //代码走到这里时，事务已经完成了（可能是回滚了、或者是提交了）
                    //看一下消息关联的订单是否存在，如果存在，说明事务是成功的，业务是执行成
功的，那么投递消息
                    if (msgOrderService.getById(msg_order_id) != null) {
                        System.out.println(String.format("准备投递消息，
{msg_id:%s}", msg_id));
                        //事务成功：投递消息
                        msgService.confirmSendMsg(msg_id);
                    } else {
                        System.out.println(String.format("准备取消投递消息，
{msg_id:%s}", msg_id));
                        //事务是不：取消投递消息
                        msgService.cancelSendMsg(msg_id);
                    }
                }
            });
        } else {
            //无事务的，直接插入并投递消息
            this.msgService.addMsg(msg, msg_order_id, true);
        }
    }
}

```

```
}
```

3.3、测试（3种场景）

3.3.1、场景1：业务成功，消息投递成功

UserService

下面的register方法是有事务的，内部会插入一条用户信息，然后会投递一条消息

```
package com.javacode2018.tx.demo11;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

/**
 * 公众号：路人甲Java，工作10年的前阿里P7分享Java、算法、数据库方面的技术干货！
 * <a href="http://www.itsoku.com">个人博客</a>
 */
@Component
public class UserService {
    @Autowired
    private JdbcTemplate jdbcTemplate;
    //消息投递器
    @Autowired
    private MsgSender msgSender;

    /**
     * 模拟用户注册成功，顺便发送消息
     */
    @Transactional
    public void register(Long user_id, String user_name) {
        //先插入用户
        this.jdbcTemplate.update("insert into t_user(id,name) VALUES (?,?)",
            user_id, user_name);
        System.out.println(String.format("用户注册: [user_id:%s,user_name:%s]",
            user_id, user_name));
        //发送消息
        String msg = String.format("[user_id:%s,user_name:%s]", user_id,
            user_name);
        //调用投递器的send方法投递消息
        this.msgSender.send(msg, 1, user_id.toString());
    }
}
```

测试类

```
package com.javacode2018.tx.demo11;

import org.junit.Before;
import org.junit.Test;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
```

```

import org.springframework.jdbc.core.JdbcTemplate;

/**
 * 公众号：路人甲Java，工作10年的前阿里P7分享Java、算法、数据库方面的技术干货！
 * <a href="http://www.itsoku.com">个人博客</a>
 */
public class Demo11Test {

    private AnnotationConfigApplicationContext context;
    private UserService userService;
    private JdbcTemplate jdbcTemplate;

    @Before
    public void before() {
        this.context = new
AnnotationConfigApplicationContext(MainConfig11.class);
        userService = context.getBean(UserService.class);
        this.jdbcTemplate = context.getBean("jdbcTemplate", JdbcTemplate.class);
        jdbcTemplate.update("truncate table t_user");
        jdbcTemplate.update("truncate table t_msg");
        jdbcTemplate.update("truncate table t_msg_order");
    }

    @Test
    public void test1() {
        this.userService.register(1L, "路人");
    }

}

```

运行输出

```

用户注册: [user_id:1,user_name:路人]
插入消息: MsgModel(id=1, msg=[user_id:1,user_name:路人], msg_order_id=1, status=0)
准备投递消息,{msg_id:1}
投递消息: MsgModel(id=1, msg=[user_id:1,user_name:路人], msg_order_id=1, status=0)

```

3.3.2、场景2：业务失败，消息取消投递

UserService中添加代码

手动抛出异常，让事务回滚。

```

/**
 * 模拟用户注册失败，咱们通过弹出异常让事务回滚，结果也会导致消息发送被取消
 *
 * @param user_id
 * @param user_name
 */
@Transactional
public void registerFail(Long user_id, String user_name) {
    this.register(user_id, user_name);
    throw new RuntimeException("故意失败!");
}

```

Demo11Test添加用例

```
@Test
public void test2() {
    this.userService.registerFail(1L, "张三");
}
```

运行输出

弹出了异常，信息比较多，我们截了关键的部分，如下，可以看出事务被回滚了，消息被取消投递了。

```
用户注册: [user_id:1,user_name:张三]
插入消息: MsgModel(id=1, msg=[user_id:1,user_name:张三], msg_order_id=1, status=0)
准备取消投递消息, {msg_id:1}
取消投递消息: MsgModel(id=1, msg=[user_id:1,user_name:张三], msg_order_id=1,
status=0)

java.lang.RuntimeException: 故意失败!

    at com.javacode2018.tx.demo11.UserService.registerFail(UserService.java:44)
    at
com.javacode2018.tx.demo11.UserService$$FastClassBySpringCGLIB$$5dd21f5c.invoke(
<generated>)
```

3.3.3、嵌套事务

事务发送是跟随当前所在的事务的，当前事务提交了，消息一定会被投递出去，当前事务是不，消息会被取消投递。

下面看嵌套事务的代码

UserService中添加代码

注意下面方法的事务传播行为是：REQUIRES_NEW，当前如果有事务，会重启一个事务。

```
//事务传播属性是REQUIRES_NEW,会在独立的事务中运行
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void registerRequiresNew(Long user_id, String user_name) {
    this.register(user_id, user_name);
}
```

添加一个类UserService1

```
package com.javacode2018.tx.demo11;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

/**
 * 公众号：路人甲Java，工作10年的前阿里P7分享Java、算法、数据库方面的技术干货！
 * <a href="http://www.itsoku.com">个人博客</a>
 */
@Component
public class UserService1 {
```

```

@Autowired
private UserService userService;

@Autowired
private MsgSender msgSender;

//嵌套事务案例
@Transactional
public void nested() {
    this.msgSender.send("消息1", 2, "1");
    //registerRequiresNew事务传播属性是REQUIRES_NEW:会在新事务中运行
    this.userService.registerRequiresNew(1L, "张三");
    //registerFail事务传播属性是默认的, 会在当前事务中运行, registerFail弹出异常会导致
    //当前事务回滚
    this.userService.registerFail(2L, "李四");
}
}

```

nested 是外围方法, 这个方法上有@Transactional, 运行的时候会开启一个事务, 内部3行代码:

@1: 发送消息, 会在当前事务中执行

@2: registerRequiresNew事务传播行为是REQUIRES_NEW, 所以会重启一个事务

@3: registerFail事务传播行为是默认的REQUIRED, 会参与到nested()开启的事务中运行, registerFail方法内部会抛出一个异常, 最终会导致外部方法事务回滚。

上面方法需要投递3条消息, 而@1和@3投递的消息由于事务回滚会导致消息被回滚, 而@2在独立的事务中执行, @2的消息会投递成功, 下面来看看执行结果, 是不是和分析的一致。

Demo11Test添加用例

```

@Test
public void test3() {
    UserService1 userService1 = this.context.getBean(UserService1.class);
    userService1.nested();
}

```

运行输出

```

插入消息: MsgModel(id=1, msg=消息1, msg_order_id=1, status=0)
用户注册: [user_id:1,user_name:张三]
插入消息: MsgModel(id=2, msg=[user_id:1,user_name:张三], msg_order_id=2, status=0)
准备投递消息, {msg_id:2}
投递消息: MsgModel(id=2, msg=[user_id:1,user_name:张三], msg_order_id=2, status=0)
用户注册: [user_id:2,user_name:李四]
插入消息: MsgModel(id=3, msg=[user_id:2,user_name:李四], msg_order_id=3, status=0)
准备取消投递消息, {msg_id:1}
取消投递消息: MsgModel(id=1, msg=消息1, msg_order_id=1, status=0)
准备取消投递消息, {msg_id:3}
取消投递消息: MsgModel(id=3, msg=[user_id:2,user_name:李四], msg_order_id=3,
status=0)

```

java.lang.RuntimeException: 故意失败!

at com.javacode2018.tx.demo11.UserService.registerFail(UserService.java:44)

大家细看一下结果，和分析的是一致的。

3.4、小结

事务消息分2步走，先落库，此时消息待投递，等到事务执行完毕之后，再确定是否投递，用到的关键技术点是事务扩展接口：TransactionSynchronization，事务执行完毕之后会自动回调接口中的afterCompletion方法。

遗留的一个问题：消息补偿操作

当事务消息刚落库，此时处于待投递状态，系统刚好down机了，此时系统恢复之后，需要有个定时器来处理这种消息，拿着消息中的msg_order_id去业务库查一下订单是否存在，如果存在，则投递消息，否则取消投递，这个留给大家去实现。

4、总结

好了，今天的内容就到此就讲完了，我们一块来总结回顾一下，你需要重点掌握的内容。

- 1、消息投递的5种方式的推演，要熟练掌握其优缺点
- 2、方式4中事务消息的代码实现，需要大家掌握

消息服务使用频率挺高的，通常作为系统中的基础服务使用，大家可以尝试一下开发一个独立的消息服务，提供给其他服务使用。

欢迎留言和我分享你的想法，如果有收获，也欢迎你把这篇文章分享给你的朋友，谢谢！

5、案例源码

git地址：

<https://gitee.com/javacode2018/spring-series>

本文案例对应源码：

spring-series\lesson-002-tx\src\main\java\com\javacode2018\tx\demo11

路人甲java所有案例代码以后都会放到这个上面，大家watch一下，可以持续关注动态。

6、Spring系列

1. [Spring系列第1篇：为何要学spring?](#)
2. [Spring系列第2篇：控制反转（IoC）与依赖注入（DI）](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)
4. [Spring系列第4篇：xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇：创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇：玩转bean scope，避免跳坑里！](#)
7. [Spring系列第7篇：依赖注入之手动注入](#)
8. [Spring系列第8篇：自动注入（autowire）详解，高手在于坚持](#)
9. [Spring系列第9篇：depend-on到底是干什么的?](#)
10. [Spring系列第10篇：primary可以解决什么问题?](#)
11. [Spring系列第11篇：bean中的autowire-candidate又是干什么的?](#)
12. [Spring系列第12篇：lazy-init: bean延迟初始化](#)

13. [Spring系列第13篇：使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇：lookup-method和replaced-method比较陌生，怎么玩的？](#)
15. [Spring系列第15篇：代理详解（Java动态代理&cglib代理）？](#)
16. [Spring系列第16篇：深入理解java注解及spring对注解的增强（预备知识）](#)
17. [Spring系列第17篇：@Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇：@ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇：@Import详解\(bean批量注册\)](#)
20. [Spring系列第20篇：@Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇：注解实现依赖注入（@Autowired、@Resource、@Primary、@Qualifier）](#)
22. [Spring系列第22篇：@Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇：Bean生命周期详解](#)
24. [Spring系列第24篇：父子容器详解](#)
25. [Spring系列第25篇：@Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇：国际化详解](#)
27. [Spring系列第27篇：spring事件机制详解](#)
28. [Spring系列第28篇：Bean循环依赖详解](#)
29. [Spring系列第29篇：BeanFactory扩展（BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor）](#)
30. [Spring系列第30篇：jdk动态代理和cglib代理](#)
31. [Spring系列第31篇：aop概念详解](#)
32. [Spring系列第32篇：AOP核心源码、原理详解](#)
33. [Spring系列第33篇：ProxyFactoryBean创建AOP代理](#)
34. [Spring系列第34篇：@Aspect中@Pointcut 12种用法](#)
35. [Spring系列第35篇：@Aspect中5中通知详解](#)
36. [Spring系列第36篇：@EnableAspectJAutoProxy、@Aspect中通知顺序详解](#)
37. [Spring系列第37篇：@EnableAsync & @Async 实现方法异步调用](#)
38. [Spring系列第38篇：@Scheduled & @EnableScheduling定时器详解](#)
39. [Spring系列第39篇：强大的Spel表达式](#)
40. [Spring系列第40篇：缓存使用（@EnableCaching、@Cacheable、@CachePut、@CacheEvict、@Caching、@CacheConfig）](#)
41. [Spring系列第41篇：@EnableCaching集成redis缓存](#)
42. [Spring系列第42篇：玩转JdbcTemplate](#)
43. [Spring系列第43篇：spring中编程式事务怎么用的？](#)
44. [Spring系列第44篇：详解spring声明式事务\(@Transactional\)](#)
45. [Spring系列第45篇：带你吃透Spring事务7种传播行为](#)
46. [Spring系列第46篇：Spring如何管理多数据源事务？](#)
47. [Spring系列第47篇：spring编程式事务源码解析](#)
48. [Spring系列第48篇：@Transaction 事务源码解析](#)

7、更多好文章

1. [Java高并发系列（共34篇）](#)
2. [MySQL高手系列（共27篇）](#)
3. [Maven高手系列（共10篇）](#)
4. [Mybatis系列（共12篇）](#)
5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要，它是什么？怎么实现？](#)
7. [泛型，有点难度，会让很多人懵逼，那是因为你没有看这篇文章！](#)

加我微信：spring88java，进群一起交流java相关各种技术！！

Spring系列第50篇：spring事务拦截器顺序如何控制？

1、前言

咱们知道 Spring 事务是通过aop的方式添加了一个事务拦截器，事务拦截器会拦截目标方法的执行，在方法执行前后添加了事务控制。

那么spring事务拦截器的顺序如何控制呢，若我们自己也添加了一些拦截器，此时事务拦截器和自定义拦截器共存的时候，他们的顺序是怎么执行的？如何手动来控制他们的顺序？？

可能有些朋友会问，控制他们的顺序，这个功能有什么用呢？为什么要学这个

学会了这些，你可以实现很多牛逼的功能，比如

- 1、读写分离
- 2、通用幂等框架
- 3、分布式事务框架

对这些有兴趣么？那么咱们继续。

2、事务拦截器顺序设置

@EnableTransactionManagement 注解有个 order属性，默认值是Integer.MAX_VALUE，用来指定事务拦截器的顺序，值越小，拦截器的优先级越高，如：

```
@EnableTransactionManagement(order = 2)
```

下面来看案例。

3、案例

我们自定义2个拦截器：一个放在事务拦截器之前执行，一个放在事务拦截器之后执行

拦截器	顺序
TransactionInterceptorBefore	1
@EnableTransactionManagement 事务拦截器	2
TransactionInterceptorAfter	3

3.1、准备sql

```

DROP DATABASE IF EXISTS javacode2018;
CREATE DATABASE if NOT EXISTS javacode2018;

USE javacode2018;
DROP TABLE IF EXISTS t_user;
CREATE TABLE t_user(
    id int PRIMARY KEY AUTO_INCREMENT,
    name varchar(256) NOT NULL DEFAULT '' COMMENT '姓名'
);

```

3.2、Spring配置类MainConfig10

@1: 开启了事务管理功能，并且设置了事务拦截器的顺序是2，spring事务拦截器完整类名是

```
org.springframework.transaction.interceptor.TransactionInterceptor
```

@2: 开启aop功能

```

package com.javacode2018.tx.demo10;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.EnableAspectJAutoProxy;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;

import javax.sql.DataSource;

@Configuration //说明当前类是一个配置类
@ComponentScan //开启bean自动扫描注册功能
@EnableTransactionManagement(order = 2) //@1: 设置事务拦截器的顺序是2
@EnableAspectJAutoProxy // @2: 开启@Aspect Aop功能
public class MainConfig10 {
    @Bean
    public DataSource dataSource() {
        org.apache.tomcat.jdbc.pool.DataSource dataSource = new
org.apache.tomcat.jdbc.pool.DataSource();
        dataSource.setDriverClassName("com.mysql.jdbc.Driver");
        dataSource.setUrl("jdbc:mysql://localhost:3306/javacode2018?
characterEncoding=UTF-8");
        dataSource.setUsername("root");
        dataSource.setPassword("root123");
        dataSource.setInitialSize(5);
        return dataSource;
    }

    //定义一个jdbcTemplate
    @Bean
    public JdbcTemplate jdbcTemplate(DataSource dataSource) {
        return new JdbcTemplate(dataSource);
    }

    //定义事务管理器transactionManager

```

```

@Bean
public PlatformTransactionManager transactionManager(DataSource dataSource)
{
    return new DataSourceTransactionManager(dataSource);
}
}

```

3.3、定义一个有事务的Service类

addUser方法上面添加了@Transactional注解，表示使用spring来管理事务，方法内部向db中插入了一条数据，为了方便分析结果，方法内部输出了2行日志

```

package com.javacode2018.tx.demo10;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

@Component
public class UserService {
    @Autowired
    private JdbcTemplate jdbcTemplate;

    @Transactional
    public void addUser() {
        System.out.println("-----UserService.addUser start");
        this.jdbcTemplate.update("insert into t_user(name) VALUES (?)", "张三");
        System.out.println("-----UserService.addUser end");
    }
}

```

3.4、自定义第1个拦截器，放在事务拦截器之前执行

下面通过Aspect的方式定义了一个拦截器，顺序通过@Order(1)设置的是1，那么这个拦截器会在事务拦截器之前执行。

```

package com.javacode2018.tx.demo10;

import org.aopalliance.intercept.Joinpoint;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Pointcut;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Component
@Aspect
@Order(1) //@1
public class TransactionInterceptorBefore {

    @Pointcut("execution(* com.javacode2018.tx.demo10.UserService.*(..))")
    public void pointcut() {
    }
}

```

```

@Around("pointcut()")
public Object tsBefore(ProceedingJoinPoint joinPoint) throws Throwable {
    System.out.println("-----before start!!!");
    Object result = joinPoint.proceed();
    System.out.println("-----before end!!!");
    return result;
}
}

```

3.4、自定义第2个拦截器，放在事务拦截器后面执行

这个拦截器的order是3，会在事务拦截器后面执行。

```

package com.javacode2018.tx.demo10;

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Pointcut;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Component
@Aspect
@Order(2)
public class TransactionInterceptorAfter {

    @Pointcut("execution(* com.javacode2018.tx.demo10.UserService.*(..))")
    public void pointcut() {
    }

    @Around("pointcut()")
    public Object tsAfter(ProceedingJoinPoint joinPoint) throws Throwable {
        System.out.println("-----after start!!!");
        Object result = joinPoint.proceed();
        System.out.println("-----after end!!!");
        return result;
    }
}

```

3.5、添加测试类

```

package com.javacode2018.tx.demo10;

import org.junit.Before;
import org.junit.Test;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.jdbc.core.JdbcTemplate;

public class Demo10Test {

    private UserService userService;

    private JdbcTemplate jdbcTemplate;
}

```

```

@Before
public void before() {
    AnnotationConfigApplicationContext context = new
AnnotationConfigApplicationContext(MainConfig10.class);
    userService = context.getBean(UserService.class);
    this.jdbcTemplate = context.getBean("jdbcTemplate", JdbcTemplate.class);
    jdbcTemplate.update("truncate table t_user");
}

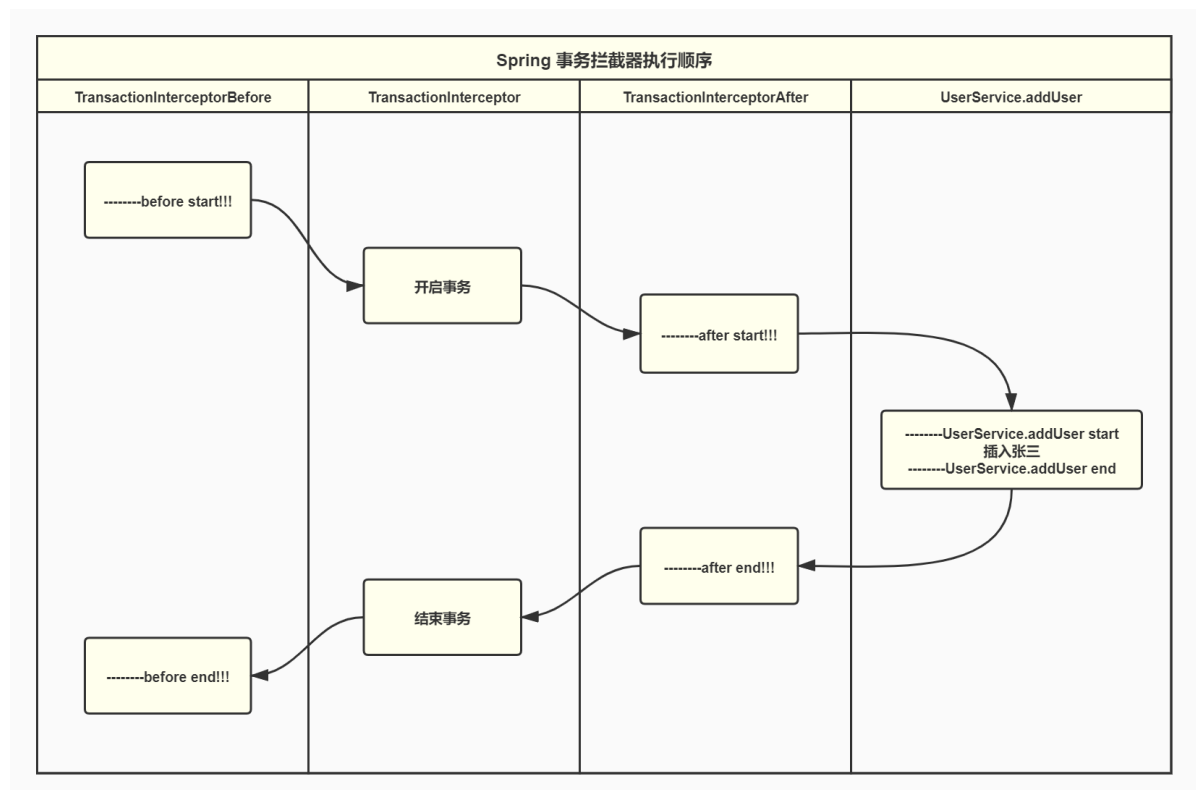
@Test
public void test1() {
    this.userService.addUser();
}
}

```

3.6、分析test1方法代码执行顺序

咱们先不执行，下分析一下test1方法执行顺序，test1方法内部会调用userService的addUser方法，这个方法会被3个拦截器拦截。

自定义的2个拦截器和事务拦截器TransactionInterceptor拦截，执行顺序如下：



下面来运行一下，看看结果和我们分析的是否一致。

3.7、运行test1输出

```

-----before start!!!
-----after start!!!
-----UserService.addUser start
-----UserService.addUser end
-----after end!!!
-----before end!!!

```

结果和上图中一致，大家可以在3个拦截器中设置一下断点，调试一下可以看到更详细的信息，可加深理解。

4、总结

今天的内容算是比较简单的，重点要掌握如何设置事务拦截器的顺序，@EnableTransactionManagement 有个 order属性，默认值是Integer.MAX_VALUE，用来指定事务拦截器的顺序，值越小，拦截器的优先级越高。

后面我们会通过这个功能实现读写分离，通用幂等性的功能。

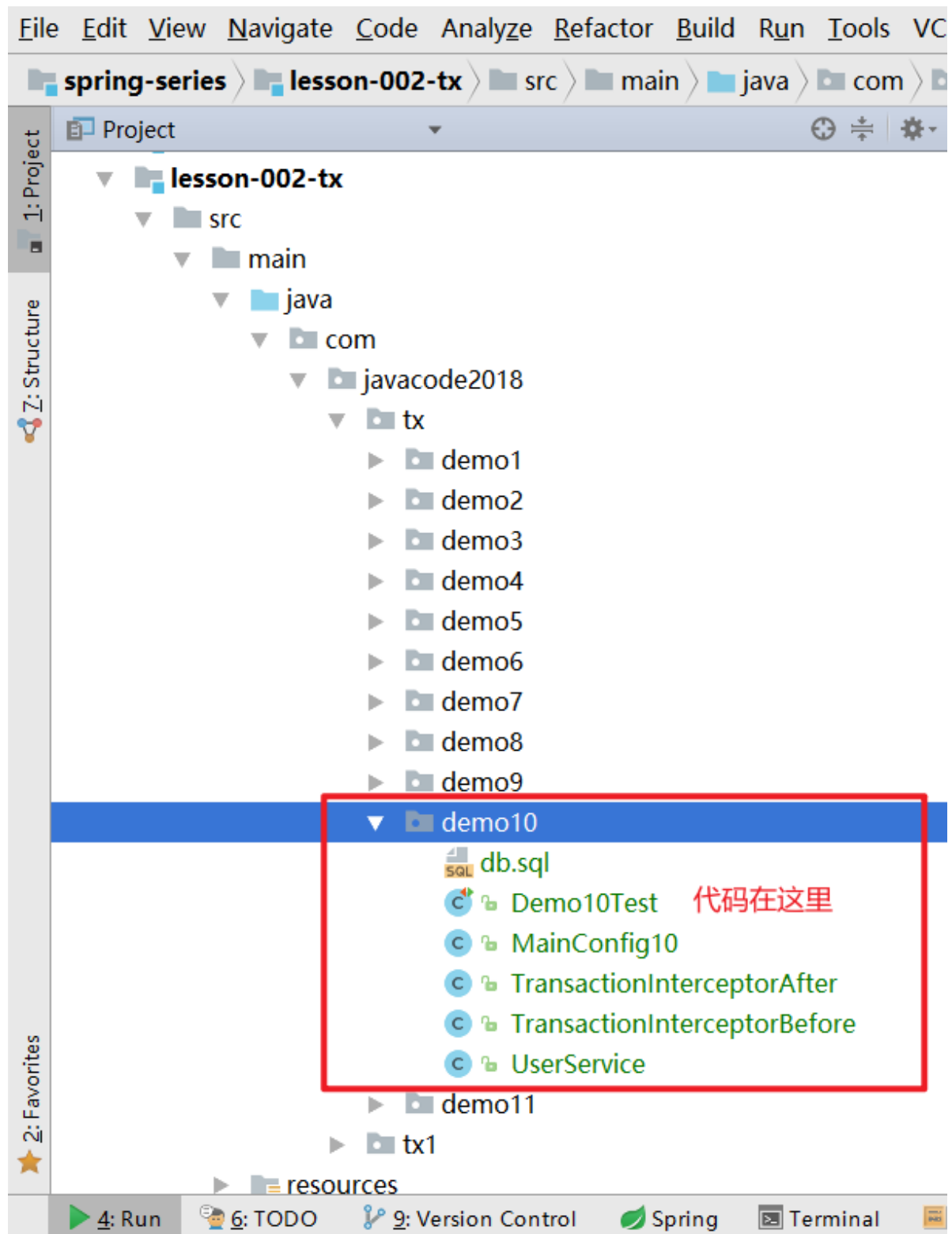
5、案例源码

git地址：

<https://gitee.com/javacode2018/spring-series>

本文案例对应源码：

spring-series\lesson-002-tx\src\main\java\com\javacode2018\tx\demo10



路人甲java所有案例代码以后都会放到这个上面，大家watch一下，可以持续关注动态。

6、Spring系列

1. [Spring系列第1篇：为何要学spring?](#)
2. [Spring系列第2篇：控制反转（IoC）与依赖注入（DI）](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)
4. [Spring系列第4篇：xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇：创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇：玩转bean scope，避免跳坑里!](#)
7. [Spring系列第7篇：依赖注入之手动注入](#)

8. [Spring系列第8篇：自动注入（autowire）详解，高手在于坚持](#)
9. [Spring系列第9篇：depend-on到底是干什么的？](#)
10. [Spring系列第10篇：primary可以解决什么问题？](#)
11. [Spring系列第11篇：bean中的autowire-candidate又是干什么的？](#)
12. [Spring系列第12篇：lazy-init：bean延迟初始化](#)
13. [Spring系列第13篇：使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇：lookup-method和replaced-method比较陌生，怎么玩的？](#)
15. [Spring系列第15篇：代理详解（Java动态代理&cglib代理）？](#)
16. [Spring系列第16篇：深入理解java注解及spring对注解的增强（预备知识）](#)
17. [Spring系列第17篇：@Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇：@ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇：@import详解\(bean批量注册\)](#)
20. [Spring系列第20篇：@Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇：注解实现依赖注入（@Autowired、@Resource、@Primary、@Qualifier）](#)
22. [Spring系列第22篇：@Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇：Bean生命周期详解](#)
24. [Spring系列第24篇：父子容器详解](#)
25. [Spring系列第25篇：@Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇：国际化详解](#)
27. [Spring系列第27篇：spring事件机制详解](#)
28. [Spring系列第28篇：Bean循环依赖详解](#)
29. [Spring系列第29篇：BeanFactory扩展（BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor）](#)
30. [Spring系列第30篇：jdk动态代理和cglib代理](#)
31. [Spring系列第31篇：aop概念详解](#)
32. [Spring系列第32篇：AOP核心源码、原理详解](#)
33. [Spring系列第33篇：ProxyFactoryBean创建AOP代理](#)
34. [Spring系列第34篇：@Aspect中@Pointcut 12种用法](#)
35. [Spring系列第35篇：@Aspect中5中通知详解](#)
36. [Spring系列第36篇：@EnableAspectJAutoProxy、@Aspect中通知顺序详解](#)
37. [Spring系列第37篇：@EnableAsync & @Async 实现方法异步调用](#)
38. [Spring系列第38篇：@Scheduled & @EnableScheduling定时器详解](#)
39. [Spring系列第39篇：强大的Spel表达式](#)
40. [Spring系列第40篇：缓存使用（@EnableCaching、@Cacheable、@CachePut、@CacheEvict、@Caching、@CacheConfig）](#)
41. [Spring系列第41篇：@EnableCaching集成redis缓存](#)
42. [Spring系列第42篇：玩转JdbcTemplate](#)
43. [Spring系列第43篇：spring中编程式事务怎么用的？](#)
44. [Spring系列第44篇：详解spring声明式事务\(@Transactional\)](#)
45. [Spring系列第45篇：带你吃透Spring事务7种传播行为](#)
46. [Spring系列第46篇：Spring如何管理多数据源事务？](#)
47. [Spring系列第47篇：spring编程式事务源码解析](#)
48. [Spring系列第48篇：@Transaction 事务源码解析](#)
49. [Spring系列第49篇：通过Spring事务实现MQ中的事务消息](#)

7、更多好文章

1. [Java高并发系列（共34篇）](#)
2. [MySQL高手系列（共27篇）](#)
3. [Maven高手系列（共10篇）](#)
4. [Mybatis系列（共12篇）](#)

5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要，它是什么？怎么实现？](#)
7. [泛型，有点难度，会让很多人懵逼，那是因为你没有看这篇文章！](#)

加我微信：spring888java，进群一起交流java相关各种技术！！

Spring系列第51篇：导致 Spring 事务失效常见的几种情况

这算是spring事务第9篇文章了，花了这么多篇文章介绍事务这块的知识，说明事务这块的东西确实比较多、知识点比较细，也非常重要，希望大家能够重视起来，吃透这块的知识。

本文2个目的：

- 1、使用spring事务的过程中，哪些情况会导致事务失效？
- 2、遇到事务相关bug时，有哪些方法可以快速定位bug？
- 3、文末有福利

1、事务失效的7种情况

1. 未启用spring事务管理功能
2. 方法不是public类型的
3. 数据源未配置事务管理器
4. 自身调用问题
5. 异常类型错误
6. 异常被吞了
7. 业务和spring事务代码必须在一个线程中

1.1、未启用spring事务管理功能

@EnableTransactionManagement 注解用来启用spring事务自动管理事务的功能，这个注解千万不要忘记写了。

1.2、方法不是public类型的

@Transaction 可以用在类上、接口上、public方法上，如果将@Trasaction用在了非public方法上，事务将无效。

1.3、数据源未配置事务管理器

spring是通过事务管理器了来管理事务的，一定不要忘记配置事务管理器了，要注意为每个数据源配置一个事务管理器：

```
@Bean
public PlatformTransactionManager transactionManager(dataSource dataSource) {
    return new DataSourceTransactionManager(dataSource);
}
```

1.4、自身调用问题

spring是通过aop的方式，对需要spring管理事务的bean生成了代理对象，然后通过代理对象拦截了目标方法的执行，在方法前后添加了事务的功能，所以必须通过代理对象调用目标方法的时候，事务才会起效。

看下面代码，大家思考一个问题：当外部直接调用m1的时候，m2方法的事务会生效么？

```
@Component
public class UserService {
    public void m1(){
        this.m2();
    }

    @Transactional
    public void m2(){
        //执行db操作
    }
}
```

显然不会生效，因为m1中通过this的方式调用了m2方法，而this并不是代理对象，this.m2()不会被事务拦截器，所以事务是无效的，如果外部直接调用通过UserService这个bean来调用m2方法，事务是有效的，上面代码可以做一下调整，如下，@1在UserService中注入了自己，此时m1中的m2事务是生效的

```
@Component
public class UserService {
    @Autowired //①
    private UserService userService;

    public void m1() {
        this.userService.m2();
    }

    @Transactional
    public void m2() {
        //执行db操作
    }
}
```

重点：必须通过代理对象访问方法，事务才会生效。

1.5、异常类型错误

spring事务回滚的机制：对业务方法进行try catch，当捕获到有指定的异常时，spring自动对事务进行回滚，那么问题来了，哪些异常spring会回滚事务呢？

并不是任何异常情况下，spring都会回滚事务，默认情况下，RuntimeException和Error的情况下，spring事务才会回滚。

也可以自定义回滚的异常类型：

```
@Transactional(rollbackFor = {异常类型列表})
```

1.6、异常被吞了

当业务方法抛出异常，spring感知到异常的时候，才会做事务回滚的操作，若方法内部将异常给吞了，那么事务无法感知到异常了，事务就不会回滚了。

如下代码，事务操作2发生了异常，但是被捕获了，此时事务并不会被回滚

```
@Transactional
public void m1(){
    事务操作1
    try{
        事务操作2，内部抛出了异常
    }catch(Exception e){

    }
}
```

1.7、业务和spring事务代码必须在一个线程中

spring事务实现中使用了ThreadLocal，ThreadLocal大家应该知道吧，可以实现同一个线程中数据共享，必须是同一个线程的时候，数据才可以共享，这就要求业务代码必须和spring事务的源码执行过程必须在一个线程中，才会受spring事务的控制，比如下面代码，方法内部的子线程内部执行的事务操作将不受m1方法上spring事务的控制，这个大家一定要注意

```
@Transactional
public void m1() {
    new Thread() {
        一系列事务操作
    }.start();
}
```

2、如何快速定位事务相关bug？

2种方式

方式1：看日志

如果你使用了logback或者log4j来输出日志，可以修改一下日志级别为debug模式，可以看到事务的详细执行日志，帮助你定位错误

方式2：调试代码

如果你对源码比较了解，那么你会知道被spring管理事务的业务方法，执行的时候都会被TransactionInterceptor拦截器拦截，会进入到它的invoke方法中，咱们可以在invoke方法中设置一些断点，可以看到详细的执行过程，排错也就比较容易了。

整体上来说，还是需要你深入理解原理，原理了解了，写代码的时候本身就会避免很多坑。

3、Spring系列

1. [Spring系列第1篇：为何要学spring?](#)
2. [Spring系列第2篇：控制反转 \(IoC\) 与依赖注入 \(DI\)](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)
4. [Spring系列第4篇：xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇：创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇：玩转bean scope，避免跳坑里!](#)
7. [Spring系列第7篇：依赖注入之手动注入](#)
8. [Spring系列第8篇：自动注入 \(autowire\) 详解，高手在于坚持](#)
9. [Spring系列第9篇：depend-on到底是干什么的?](#)
10. [Spring系列第10篇：primary可以解决什么问题?](#)
11. [Spring系列第11篇：bean中的autowire-candidate又是干什么的?](#)
12. [Spring系列第12篇：lazy-init: bean延迟初始化](#)
13. [Spring系列第13篇：使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇：lookup-method和replaced-method比较陌生，怎么玩的?](#)
15. [Spring系列第15篇：代理详解 \(Java动态代理&cglib代理\) ?](#)
16. [Spring系列第16篇：深入理解java注解及spring对注解的增强 \(预备知识\)](#)
17. [Spring系列第17篇：@Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇：@ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇：@import详解\(bean批量注册\)](#)
20. [Spring系列第20篇：@Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇：注解实现依赖注入 \(@Autowired、@Resource、@Primary、@Qualifier\)](#)
22. [Spring系列第22篇：@Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇：Bean生命周期详解](#)
24. [Spring系列第24篇：父子容器详解](#)
25. [Spring系列第25篇：@Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇：国际化详解](#)
27. [Spring系列第27篇：spring事件机制详解](#)
28. [Spring系列第28篇：Bean循环依赖详解](#)
29. [Spring系列第29篇：BeanFactory扩展 \(BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor\)](#)
30. [Spring系列第30篇：jdk动态代理和cglib代理](#)
31. [Spring系列第31篇：aop概念详解](#)
32. [Spring系列第32篇：AOP核心源码、原理详解](#)
33. [Spring系列第33篇：ProxyFactoryBean创建AOP代理](#)
34. [Spring系列第34篇：@Aspect中@Pointcut 12种用法](#)
35. [Spring系列第35篇：@Aspect中5中通知详解](#)

36. [Spring系列第36篇: @EnableAspectJAutoProxy、@Aspect中通知顺序详解](#)
37. [Spring系列第37篇: @EnableAsync & @Async 实现方法异步调用](#)
38. [Spring系列第38篇: @Scheduled & @EnableScheduling定时器详解](#)
39. [Spring系列第39篇: 强大的Spel表达式](#)
40. [Spring系列第40篇: 缓存使用 \(@EnableCaching、@Cacheable、@CachePut、@CacheEvict、@Caching、@CacheConfig\)](#)
41. [Spring系列第41篇: @EnableCaching集成redis缓存](#)
42. [Spring系列第42篇: 玩转JdbcTemplate](#)
43. [Spring系列第43篇: spring中编程式事务怎么用的?](#)
44. [Spring系列第44篇: 详解spring声明式事务\(@Transactional\)](#)
45. [Spring系列第45篇: 带你吃透Spring事务7种传播行为](#)
46. [Spring系列第46篇: Spring如何管理多数据源事务?](#)
47. [Spring系列第47篇: spring编程式事务源码解析](#)
48. [Spring系列第48篇: @Transaction 事务源码解析](#)
49. [Spring系列第49篇: 通过Spring事务实现MQ中的事务消息](#)
50. [Spring系列第50篇: spring事务拦截器顺序如何控制?](#)
51. [Spring系列第51篇: 导致 Spring 事务失效常见的几种情况](#)

4、更多好文章

1. [Java高并发系列 \(共34篇\)](#)
2. [MySQL高手系列 \(共27篇\)](#)
3. [Maven高手系列 \(共10篇\)](#)
4. [Mybatis系列 \(共12篇\)](#)
5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要, 它是什么? 怎么实现?](#)
7. [泛型, 有点难度, 会让很多人懵逼, 那是因为你没有看这篇文章!](#)

5、福利

这边整理了几百本计算机相关的电子书, 常见的基本上都有, 获取方式, 扫码发送: **电子书**



加我微信：spring888java，进群一起交流java相关各种技术！！

Spring系列第52篇：Spring实现数据库读写分离

1、背景

大多数系统都是读多写少，为了降低数据库的压力，可以对主库创建多个从库，从库自动从主库同步数据，程序中将写的操作发送到主库，将读的操作发送到从库去执行。

今天的主要目标：**通过spring实现读写分离。**

读写分离需实现下面2个功能：

- 1、读的方法，由调用者来控制具体是读从库还是主库
- 2、有事务的方法，内部的所有读写操作都走主库

2、思考3个问题

- 1、读的方法，由调用者来控制具体是读从库还是主库，如何实现？

可以给所有读的方法添加一个参数，来控制读从库还是主库。

- 2、数据源如何路由？

spring-jdbc 包中提供了一个抽象类：AbstractRoutingDataSource，实现了javax.sql.DataSource接口，我们用这个类来作为数据源类，重点是这个类可以用来做数据源的路由，可以在其内部配置多个真实的数据源，最终用哪个数据源，由开发者来决定。

AbstractRoutingDataSource中有个map，用来存储多个目标数据源

```
private Map<Object, DataSource> resolvedDataSources;
```

比如主从库可以这么存储

```
resolvedDataSources.put("master", 主库数据源);  
resolvedDataSources.put("slave", 从库数据源);
```

AbstractRoutingDataSource中还有抽象方法 determineCurrentLookupKey，将这个方法的返回值作为key到上面的resolvedDataSources中查找对应的数据源，作为当前操作db的数据源

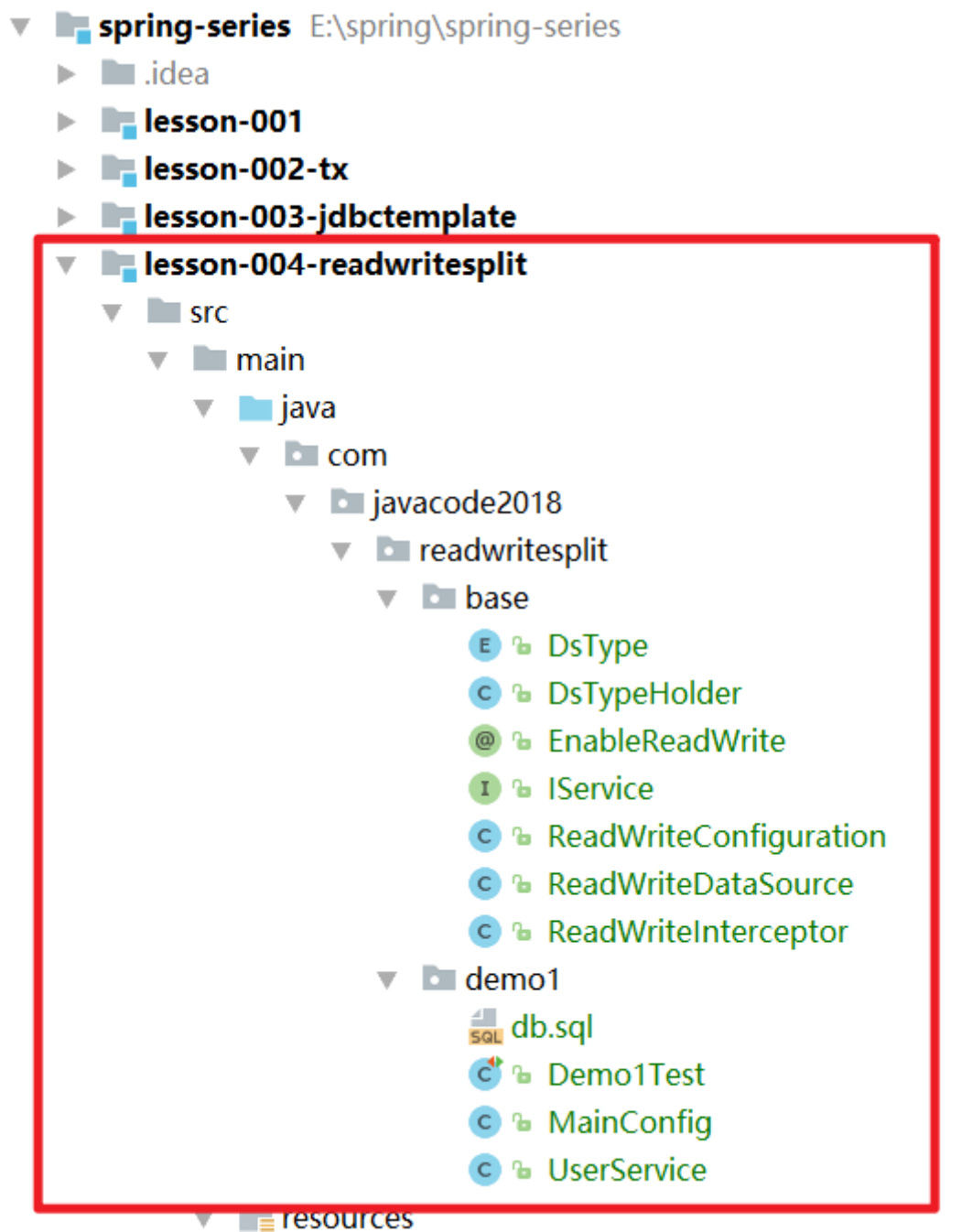
```
protected abstract Object determineCurrentLookupKey();
```

3、读写分离在哪控制？

读写分离属于一个通用的功能，可以通过spring的aop来实现，添加一个拦截器，拦截目标方法的之前，在目标方法执行之前，获取一下当前需要走哪个库，将这个标志存储在ThreadLocal中，将这个标志作为AbstractRoutingDataSource.determineCurrentLookupKey()方法的返回值，拦截器中在目标方法执行完毕之后，将这个标志从ThreadLocal中清除。

3、代码实现

3.1、工程结构图



3.2、DsType

表示数据源类型，有2个值，用来区分是主库还是从库。

```
package com.javacode2018.readwritesplit.base;

public enum DsType {
    MASTER, SLAVE;
}
```

3.3、DsTypeHolder

内部有个ThreadLocal，用来记录当前走主库还是从库，将这个标志放在dsTypeThreadLocal中

```
package com.javacode2018.readwritesplit.base;

public class DsTypeHolder {
    private static ThreadLocal<DsType> dsTypeThreadLocal = new ThreadLocal<>();
}
```

```

    public static void master() {
        dsTypeThreadLocal.set(DsType.MASTER);
    }

    public static void slave() {
        dsTypeThreadLocal.set(DsType.SLAVE);
    }

    public static DsType getDsType() {
        return dsTypeThreadLocal.get();
    }

    public static void clearDsType() {
        dsTypeThreadLocal.remove();
    }
}

```

3.4、IService接口

这个接口起到标志的作用，当某个类需要启用读写分离的时候，需要实现这个接口，实现这个接口的类都会被读写分离拦截器拦截。

```

package com.javacode2018.readwritesplit.base;

//需要实现读写分离的service需要实现该接口
public interface IService {
}

```

3.5、ReadWriteDataSource

读写分离数据源，继承ReadWriteDataSource，注意其内部的determineCurrentLookupKey方法，从上面的ThreadLocal中获取当前需要走主库还是从库的标志。

```

package com.javacode2018.readwritesplit.base;

import org.springframework.jdbc.datasource.lookup.AbstractRoutingDataSource;
import org.springframework.lang.Nullable;

public class ReadWriteDataSource extends AbstractRoutingDataSource {
    @Nullable
    @Override
    protected Object determineCurrentLookupKey() {
        return DsTypeHolder.getDsType();
    }
}

```

3.6、ReadWriteInterceptor

读写分离拦截器，需放在事务拦截器前面执行，通过@1代码我们将此拦截器的顺序设置为Integer.MAX_VALUE - 2，稍后我们将事务拦截器的顺序设置为Integer.MAX_VALUE - 1，事务拦截器的执行顺序是从小到达的，所以，ReadWriteInterceptor会在事务拦截器org.springframework.transaction.interceptor.TransactionInterceptor之前执行。

由于业务方法中存在相互调用的情况，比如service1.m1中调用service2.m2，而service2.m2中调用了service2.m3，我们只需要在m1方法执行之前，获取具体要用哪个数据源就可以了，所以下面代码中会在第一次进入这个拦截器的时候，记录一下走主库还是从库。

下面方法中会获取当前目标方法的最后一个参数，最后一个参数可以是DsType类型的，开发者可以通过这个参数来控制具体走主库还是从库。

```
package com.javacode2018.readwritesplit.base;

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Pointcut;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

import java.util.Objects;

@Aspect
@Order(Integer.MAX_VALUE - 2) //@1
@Component
public class ReadWriteInterceptor {

    @Pointcut("target(IService)")
    public void pointcut() {
    }

    //获取当前目标方法的最后一个参数
    private Object getLastArgs(final ProceedingJoinPoint pjp) {
        Object[] args = pjp.getArgs();
        if (Objects.nonNull(args) && args.length > 0) {
            return args[args.length - 1];
        } else {
            return null;
        }
    }

    @Around("pointcut()")
    public Object around(final ProceedingJoinPoint pjp) throws Throwable {
        //判断是否是第一次进来,用于处理事务嵌套
        boolean isFirst = false;
        try {
            if (DsTypeHolder.getDSType() == null) {
                isFirst = true;
            }
            if (isFirst) {
                Object lastArgs = getLastArgs(pjp);
                if (DsType.SLAVE.equals(lastArgs)) {
                    DsTypeHolder.slave();
                } else {
                    DsTypeHolder.master();
                }
            }
            return pjp.proceed();
        } finally {
            //退出的时候,清理
            if (isFirst) {

```

```

        DsTypeHolder.clearDsType();
    }
}
}
}

```

3.7、ReadWriteConfiguration

spring配置类，作用

- 1、@3：用来将com.javacode2018.readwritesplit.base包中的一些类注册到spring容器中，比如上面的拦截器ReadWriteInterceptor
- 2、@1：开启spring aop的功能
- 3、@2：开启spring自动管理事务的功能，@EnableTransactionManagement的order用来指定事务拦截器org.springframework.transaction.interceptor.TransactionInterceptor顺序，在这里我们将order设置为Integer.MAX_VALUE - 1，而上面ReadWriteInterceptor的order是Integer.MAX_VALUE - 2，所以ReadWriteInterceptor会在事务拦截器之前执行。

```

package com.javacode2018.readwritesplit.base;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.EnableAspectJAutoProxy;
import org.springframework.transaction.annotation.EnableTransactionManagement;

@Configuration
@EnableAspectJAutoProxy //①
@EnableTransactionManagement(proxyTargetClass = true, order = Integer.MAX_VALUE
- 1) //②
@ComponentScan(basePackageClasses = IService.class) //③
public class ReadWriteConfiguration {
}

```

3.8、@EnableReadWrite

这个注解用来开启读写分离的功能，@1通过@Import将ReadWriteConfiguration导入到spring容器了，这样就会自动启用读写分离的功能。业务中需要使用读写分离，只需要在spring配置类中加上@EnableReadWrite注解就可以了。

```

package com.javacode2018.readwritesplit.base;

import org.springframework.context.annotation.Import;

import java.lang.annotation.*;

@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Import(ReadWriteConfiguration.class) //①
public @interface EnableReadWrite {
}

```

4、案例

读写分离的关键代码写完了，下面我们来上案例验证一下效果。

4.1、执行sql脚本

下面准备2个数据库：javacode2018_master（主库）、javacode2018_slave（从库）

2个库中都创建一个t_user表，分别插入了一条数据，稍后用这个数据来验证走的是主库还是从库。

```
DROP DATABASE IF EXISTS javacode2018_master;
CREATE DATABASE IF NOT EXISTS javacode2018_master;

USE javacode2018_master;
DROP TABLE IF EXISTS t_user;
CREATE TABLE t_user (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(256) NOT NULL DEFAULT ''
    COMMENT '姓名'
);

INSERT INTO t_user (name) VALUE ('master库');

DROP DATABASE IF EXISTS javacode2018_slave;
CREATE DATABASE IF NOT EXISTS javacode2018_slave;

USE javacode2018_slave;
DROP TABLE IF EXISTS t_user;
CREATE TABLE t_user (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(256) NOT NULL DEFAULT ''
    COMMENT '姓名'
);

INSERT INTO t_user (name) VALUE ('slave库');
```

4.2、spring配置类

@1：启用读写分离

masterDs()方法：定义主库数据源

slaveDs()方法：定义从库数据源

dataSource()：定义读写分离路由数据源

后面还有2个方法用来定义JdbcTemplate和事务管理器，方法中都通过@Qualifier("dataSource")限定了注入的bean名称为dataSource：即注入了上面dataSource()返回的读写分离路由数据源。

```
package com.javacode2018.readwritesplit.demo1;

import com.javacode2018.readwritesplit.base.DsType;
import com.javacode2018.readwritesplit.base.EnableReadwrite;
import com.javacode2018.readwritesplit.base.ReadwriteDataSource;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
```

```

import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.transaction.PlatformTransactionManager;

import javax.sql.DataSource;
import java.util.HashMap;
import java.util.Map;

@EnableReadWrite @@1
@Configuration
@ComponentScan
public class MainConfig {
    //主库数据源
    @Bean
    public DataSource masterDs() {
        org.apache.tomcat.jdbc.pool.DataSource dataSource = new
org.apache.tomcat.jdbc.pool.DataSource();
        dataSource.setDriverClassName("com.mysql.jdbc.Driver");
        dataSource.setUrl("jdbc:mysql://localhost:3306/javacode2018_master?
characterEncoding=UTF-8");
        dataSource.setUsername("root");
        dataSource.setPassword("root123");
        dataSource.setInitialSize(5);
        return dataSource;
    }

    //从库数据源
    @Bean
    public DataSource slaveDs() {
        org.apache.tomcat.jdbc.pool.DataSource dataSource = new
org.apache.tomcat.jdbc.pool.DataSource();
        dataSource.setDriverClassName("com.mysql.jdbc.Driver");
        dataSource.setUrl("jdbc:mysql://localhost:3306/javacode2018_slave?
characterEncoding=UTF-8");
        dataSource.setUsername("root");
        dataSource.setPassword("root123");
        dataSource.setInitialSize(5);
        return dataSource;
    }

    //读写分离路由数据源
    @Bean
    public ReadWriteDataSource dataSource() {
        ReadWriteDataSource dataSource = new ReadWriteDataSource();
        //设置主库为默认的库，当路由的时候没有在datasource那个map中找到对应的数据源的时候，
会使用这个默认的数据源
        dataSource.setDefaultTargetDataSource(this.masterDs());
        //设置多个目标库
        Map<Object, Object> targetDataSources = new HashMap<>();
        targetDataSources.put(DsType.MASTER, this.masterDs());
        targetDataSources.put(DsType.SLAVE, this.slaveDs());
        dataSource.setTargetDataSources(targetDataSources);
        return dataSource;
    }

    //JdbcTemplate, dataSource为上面定义的注入读写分离的数据源
    @Bean

```

```

    public JdbcTemplate jdbcTemplate(@Qualifier("dataSource") DataSource
dataSource) {
        return new JdbcTemplate(dataSource);
    }

    //定义事务管理器，dataSource为上面定义的注入读写分离的数据源
    @Bean
    public PlatformTransactionManager
transactionManager(@Qualifier("dataSource") DataSource dataSource) {
        return new DataSourceTransactionManager(dataSource);
    }
}

```

4.3、UserService

这个类就相当于我们平时写的service，我是为了方法，直接在里面使用了JdbcTemplate来操作数据库，真实的项目操作db会放在dao里面。

getUserNameById方法：通过id查询name。

insert方法：插入数据，这个内部的所有操作都会走主库，为了验证是不是查询也会走主库，插入数据之后，我们会调用this.userService.getUserNameById(id, DsType.SLAVE)方法去执行查询操作，第二个参数故意使用SLAVE，如果查询有结果，说明走的是主库，否则走的是从库，这里为什么需要通过this.userService来调用getUserNameById？

this.userService最终是个代理对象，通过代理对象访问其内部的方法，才会被读写分离的拦截器拦截。

```

package com.javacode2018.readwritesplit.demo1;

import com.javacode2018.readwritesplit.base.DsType;
import com.javacode2018.readwritesplit.base.IService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Component
public class UserService implements IService {

    @Autowired
    private JdbcTemplate jdbcTemplate;

    @Autowired
    private UserService userService;

    @Transactional(propagation = Propagation.SUPPORTS, readOnly = true)
    public String getUserNameById(long id, DsType dsType) {
        String sql = "select name from t_user where id=?";
        List<String> list = this.jdbcTemplate.queryForList(sql, String.class,
id);
        return (list != null && list.size() > 0) ? list.get(0) : null;
    }

    //这个insert方法会走主库，内部的所有操作都会走主库

```

```

@Transactional
public void insert(long id, String name) {
    System.out.println(String.format("插入数据{id:%s, name:%s}", id, name));
    this.jdbcTemplate.update("insert into t_user (id,name) values (?,?)",
id, name);
    String userName = this.userService.getUserNameById(id, DsType.SLAVE);
    System.out.println("查询结果: " + userName);
}
}

```

4.4、测试用例

```

package com.javacode2018.readwritesplit.demo1;

import com.javacode2018.readwritesplit.base.DsType;
import org.junit.Before;
import org.junit.Test;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Demo1Test {

    UserService userService;

    @Before
    public void before() {
        AnnotationConfigApplicationContext context = new
AnnotationConfigApplicationContext();
        context.register(MainConfig.class);
        context.refresh();
        this.userService = context.getBean(UserService.class);
    }

    @Test
    public void test1() {
        System.out.println(this.userService.getUserNameById(1, DsType.MASTER));
        System.out.println(this.userService.getUserNameById(1, DsType.SLAVE));
    }

    @Test
    public void test2() {
        long id = System.currentTimeMillis();
        System.out.println(id);
        this.userService.insert(id, "张三");
    }
}

```

test1方法执行2次查询，分别查询主库和从库，输出：

```

master库
slave库

```

是不是很爽，由开发者自己控制具体走主库还是从库。

test2执行结果如下，可以看出查询到了刚刚插入的数据，说明insert中所有操作都走的是主库。

1604905117467

插入数据{id:1604905117467, name:张三}

查询结果: 张三

5、案例源码

git地址:

<https://gitee.com/javacode2018/spring-series>

本文案例对应源码:

spring-series\lesson-004-readwritesplit

大家star一下，所有系列代码都会在这个里面。

6、Spring系列

1. [Spring系列第1篇: 为何要学spring?](#)
2. [Spring系列第2篇: 控制反转 \(IoC\) 与依赖注入 \(DI\)](#)
3. [Spring系列第3篇: Spring容器基本使用及原理](#)
4. [Spring系列第4篇: xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇: 创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇: 玩转bean scope, 避免跳坑里!](#)
7. [Spring系列第7篇: 依赖注入之手动注入](#)
8. [Spring系列第8篇: 自动注入 \(autowire\) 详解, 高手在于坚持](#)
9. [Spring系列第9篇: depend-on到底是干什么的?](#)
10. [Spring系列第10篇: primary可以解决什么问题?](#)
11. [Spring系列第11篇: bean中的autowire-candidate又是干什么的?](#)
12. [Spring系列第12篇: lazy-init: bean延迟初始化](#)
13. [Spring系列第13篇: 使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇: lookup-method和replaced-method比较陌生, 怎么玩的?](#)
15. [Spring系列第15篇: 代理详解 \(Java动态代理&cglib代理\)?](#)
16. [Spring系列第16篇: 深入理解java注解及spring对注解的增强 \(预备知识\)](#)
17. [Spring系列第17篇: @Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇: @ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇: @import详解\(bean批量注册\)](#)
20. [Spring系列第20篇: @Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇: 注解实现依赖注入 \(@Autowired、@Resource、@Primary、@Qualifier\)](#)
22. [Spring系列第22篇: @Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇: Bean生命周期详解](#)
24. [Spring系列第24篇: 父子容器详解](#)
25. [Spring系列第25篇: @Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇: 国际化详解](#)
27. [Spring系列第27篇: spring事件机制详解](#)
28. [Spring系列第28篇: Bean循环依赖详解](#)
29. [Spring系列第29篇: BeanFactory扩展 \(BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor\)](#)
30. [Spring系列第30篇: jdk动态代理和cglib代理](#)
31. [Spring系列第31篇: aop概念详解](#)

32. [Spring系列第32篇：AOP核心源码、原理详解](#)
33. [Spring系列第33篇：ProxyFactoryBean创建AOP代理](#)
34. [Spring系列第34篇：@Aspect中@Pointcut 12种用法](#)
35. [Spring系列第35篇：@Aspect中5中通知详解](#)
36. [Spring系列第36篇：@EnableAspectJAutoProxy、@Aspect中通知顺序详解](#)
37. [Spring系列第37篇：@EnableAsync & @Async 实现方法异步调用](#)
38. [Spring系列第38篇：@Scheduled & @EnableScheduling定时器详解](#)
39. [Spring系列第39篇：强大的Spel表达式](#)
40. [Spring系列第40篇：缓存使用 \(@EnableCaching、@Cacheable、@CachePut、@CacheEvict、@Caching、@CacheConfig\)](#)
41. [Spring系列第41篇：@EnableCaching集成redis缓存](#)
42. [Spring系列第42篇：玩转jdbcTemplate](#)
43. [Spring系列第43篇：spring中编程式事务怎么用的？](#)
44. [Spring系列第44篇：详解spring声明式事务\(@Transactional\)](#)
45. [Spring系列第45篇：带你吃透Spring事务7种传播行为](#)
46. [Spring系列第46篇：Spring如何管理多数据源事务？](#)
47. [Spring系列第47篇：spring编程式事务源码解析](#)
48. [Spring系列第48篇：@Transaction 事务源码解析](#)
49. [Spring系列第49篇：通过Spring事务实现MQ中的事务消息](#)
50. [Spring系列第50篇：spring事务拦截器顺序如何控制？](#)
51. [Spring系列第51篇：导致 Spring 事务失效常见的几种情况](#)
52. [Spring系列第52篇：Spring实现数据库读写分离](#)
53. [Spring系列第53篇：Spring集成MyBatis](#)
54. [Spring系列第54篇：集成junit](#)
55. [Spring系列第55篇：spring上下文生命周期](#)
56. [Spring系列第56篇：一文搞懂spring到底为什么要用三级缓存？](#)

7、更多好文章

1. [Java高并发系列（共34篇）](#)
2. [MySQL高手系列（共27篇）](#)
3. [Maven高手系列（共10篇）](#)
4. [Mybatis系列（共12篇）](#)
5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要，它是什么？怎么实现？](#)
7. [泛型，有点难度，会让很多人懵逼，那是因为你没有看这篇文章！](#)

8、福利

这边整理了几百本计算机相关的电子书，常见的基本上都有，获取方式，扫码发送：**电子书**



加我微信: [spring888java](#), 进群一起交流java相关各种技术!!

Spring系列第53篇: Spring集成MyBatis

1、本文内容

本文主要介绍mybatis和spring集成的两种方式, 对MyBatis不熟悉的, 建议先看一下[MyBatis高手系列](#)
目前注解的方式我们用的比较多, 所以主要介绍注解的方式, xml的方式这里就暂时不介绍了。

注解的方式mybatis集成spring主要有2种方式:

- 1、方式1: mapper xml文件放在resource目录, 和Mapper接口不在一个目录的情况
- 2、方式2: mapper xml文件和Mapper接口在同一个目录

还会介绍另外一个点: **多数据库的时候, 如何配置?**

2、案例代码

git地址:
<https://gitee.com/javacode2018/spring-series>

方式1源码: `spring-series\lesson-005-mybatis`
方式2源码: `spring-series\lesson-006-mybatis`

3、准备db脚本

```

/*创建数据库javacode2018*/
DROP DATABASE IF EXISTS `javacode2018`;
CREATE DATABASE `javacode2018`;
USE `javacode2018`;

/*创建表结构*/
DROP TABLE IF EXISTS `t_user`;
CREATE TABLE `t_user` (
  `id` BIGINT AUTO_INCREMENT PRIMARY KEY COMMENT '主键，用户id，自动增长',
  `name` VARCHAR(32) NOT NULL DEFAULT '' COMMENT '姓名'
) COMMENT '用户表';

SELECT * FROM t_user;

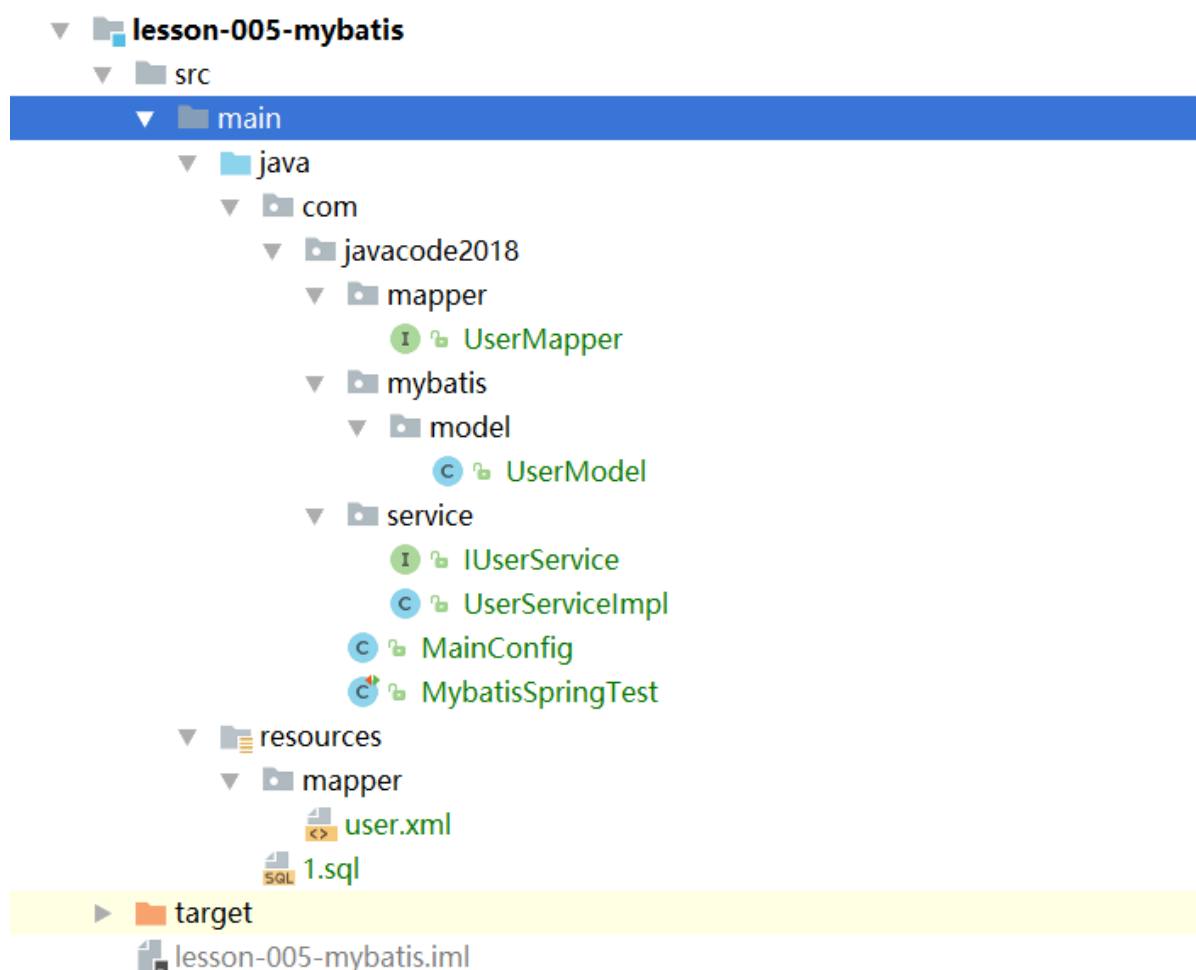
```

下面我们来通过mybatis对上面的表执行插入和查询的操作。

4、方式1

4.1、项目结构

这种方式，Mapper接口和mapper xml文件放在不同的位置，mapper xml文件放在resource/mapper目录，如下图



4.2、UserModel

```
package com.javacode2018.mybatis.model;

import lombok.*;

/**
 * 公众号：路人甲Java，工作10年的前阿里P7分享Java、算法、数据库方面的技术干货！
 * <a href="http://www.itsoku.com">个人博客</a>
 */
@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
@ToString
public class UserModel {
    private Long id;
    private String name;
}
```

4.3、UserMapper

这个是mapper接口，类上面需要添加**@Mapper注解**，用来标注这是一个Mapper接口

```
package com.javacode2018.mapper;

import com.javacode2018.mybatis.model.UserModel;
import org.apache.ibatis.annotations.Mapper;

import java.util.List;

/**
 * 公众号：路人甲Java，工作10年的前阿里P7分享Java、算法、数据库方面的技术干货！
 * <a href="http://www.itsoku.com">个人博客</a>
 */
@Mapper
public interface UserMapper {
    void insert(UserModel userModel);
    List<UserModel> getList();
}
```

4.4、UserMapper.xml

在spring-series\lesson-005-mybatis\src\main\resources\mapper中添加UserMapper.xml文件，这个文件名大家可以随意起名，不一定非要叫UserMapper，具体位置大家可以看看上面的项目结构图

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="com.javacode2018.mapper.UserMapper">

    <insert id="insert" parameterType="com.javacode2018.mybatis.model.UserModel"
        keyProperty="id" useGeneratedKeys="true">
        <![CDATA[ INSERT INTO `t_user` (name) VALUES (#{name})]]>
    </insert>
</mapper>
```

```

</insert>

<select id="getList" resultType="com.javacode2018.mybatis.model.UserModel">
    <![CDATA[
        SELECT id,name FROM t_user
    ]]>
</select>

</mapper>

```

4.5、IUserService

```

package com.javacode2018.service;

import com.javacode2018.mybatis.model.UserModel;

import java.util.List;

/**
 * 公众号：路人甲Java，工作10年的前阿里P7分享Java、算法、数据库方面的技术干货！
 * <a href="http://www.itsoku.com">个人博客</a>
 */
public interface IUserService {
    /**
     * 插入用户信息
     *
     * @param userModel
     * @return
     */
    UserModel insert(UserModel userModel);

    /**
     * 查询用户所有记录
     *
     * @return
     */
    List<UserModel> getList();
}

```

4.6、UserServiceImpl

IUserService的实现类，内部将Usermapper通过@Autowired注入进来，通过userMapper来访问数据库，userMapper实际上是mybatis创建的一个代理对象。

```

package com.javacode2018.service;

import com.javacode2018.mapper.UserMapper;
import com.javacode2018.mybatis.model.UserModel;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

/**

```

```

* 公众号：路人甲Java，工作10年的前阿里P7分享Java、算法、数据库方面的技术干货！
* <a href="http://www.itsoku.com">个人博客</a>
*/
@Service
public class UserServiceImpl implements IUserService {
    @Autowired
    private UserMapper userMapper;

    @Transactional(rollbackFor = Exception.class)
    @Override
    public UserModel insert(UserModel userModel) {
        userMapper.insert(userModel);
        return userModel;
    }

    @Transactional(propagation = Propagation.SUPPORTS, readOnly = true)
    @Override
    public List<UserModel> getList() {
        return userMapper.getList();
    }
}

```

4.7、spring配置类MainConfig

```

package com.javacode2018;

import com.javacode2018.mapper.UserMapper;
import org.apache.ibatis.annotations.Mapper;
import org.mybatis.spring.SqlSessionFactoryBean;
import org.mybatis.spring.annotation.MapperScan;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.io.Resource;
import org.springframework.core.io.support.PathMatchingResourcePatternResolver;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.TransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;

import javax.sql.DataSource;
import java.io.IOException;

/**
 * 公众号：路人甲Java，工作10年的前阿里P7分享Java、算法、数据库方面的技术干货！
 * <a href="http://www.itsoku.com">个人博客</a>
 */
@EnableTransactionManagement //开启事务支持
@ComponentScan
@Configuration
// @MapperScan这个注解是关键，会扫描标记有@Mapper注解的Mapper接口，将其注册到spring容器中
@MapperScan(basePackageClasses = {UserMapper.class}, annotationClass =
Mapper.class)
public class MainConfig {
    //定义数据源
    @Bean

```

```

    public DataSource dataSource() {
        org.apache.tomcat.jdbc.pool.DataSource dataSource = new
org.apache.tomcat.jdbc.pool.DataSource();
        dataSource.setDriverClassName("com.mysql.jdbc.Driver");
        dataSource.setUrl("jdbc:mysql://localhost:3306/javacode2018?
characterEncoding=UTF-8");
        dataSource.setUsername("root");
        dataSource.setPassword("root123");
        dataSource.setInitialSize(5);
        return dataSource;
    }

    //定义事务管理器
    @Bean
    public TransactionManager transactionManager(DataSource dataSource) {
        return new DataSourceTransactionManager(dataSource);
    }

    //定义SqlSessionFactoryBean，用来创建SqlSessionFactory
    @Bean
    public SqlSessionFactoryBean sqlSessionFactoryBean(DataSource dataSource)
throws IOException {
        SqlSessionFactoryBean sqlSessionFactoryBean = new
SqlSessionFactoryBean();
        sqlSessionFactoryBean.setDataSource(dataSource);
        //获取所有mapper.xml文件
        Resource[] resources = new
PathMatchingResourcePatternResolver().getResources("classpath*:mapper/**/*.xml")
;
        sqlSessionFactoryBean.setMapperLocations(resources);
        return sqlSessionFactoryBean;
    }
}

```

这个类中有2个关键信息：

1、**@MapperScan注解**：这个注解是关键，会扫描标记有@Mapper注解的Mapper接口类，然后给Mapper接口生成代理对象，将其注册到spring容器中，这个注解有几个属性需要注意下

- value或者basePackages：String类型的数组，用来指定扫描的包
- basePackageClasses：可以指定一个类，扫描范围为这个类所在的包及其所有子包
- sqlSessionFactoryRef：用来指定sqlSessionFactory的bean名称，当我们的系统中需要操作多个库的时候，每个库对应一个SqlSessionFactory，此时可以通过这个属性指定需要使用哪个SqlSessionFactory。

2、**定义SqlSessionFactoryBean**：通过名字大家可以看出，这个是用来生成SqlSessionFactory的，内部需要指定数据源和本地mapper xml的位置，我们将mapper xml文件放在resource/mapper文件中，此处我们采用通配符的方式，加载classpath中mapper目录及子目录中的所有xml文件

```

Resource[] resources = new
PathMatchingResourcePatternResolver().getResources("classpath*:mapper/**/*.xml")
;
sqlSessionFactoryBean.setMapperLocations(resources);

```


4.8、测试效果

来个测试类

```
package com.javacode2018;

import com.javacode2018.mybatis.model.UserModel;
import com.javacode2018.service.IUserService;
import org.junit.Test;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

import java.util.List;

/**
 * 公众号：路人甲Java，工作10年的前阿里P7分享Java、算法、数据库方面的技术干货！
 * <a href="http://www.itsoku.com">个人博客</a>
 */
public class MybatisSpringTest {

    @Test
    public void insert() {
        AnnotationConfigApplicationContext context = new
        AnnotationConfigApplicationContext(MainConfig.class);
        IUserService userService = context.getBean(IUserService.class);
        UserModel userModel = UserModel.builder().name("张三").build();
        userService.insert(userModel);
        System.out.println(userModel);
    }

    @Test
    public void getList() {
        AnnotationConfigApplicationContext context = new
        AnnotationConfigApplicationContext(MainConfig.class);
        IUserService userService = context.getBean(IUserService.class);
        List<UserModel> userModelList = userService.getList();
        System.out.println(userModelList);
    }
}
```

insert方法用来插入数据，执行输出

```
UserModel(id=1, name=张三)
```

getList方法用来查询表中所有数据，运行输出

```
[UserModel(id=1, name=张三)]
```

4.8、小结：回顾步骤

主要分为3步

1、Mapper接口上添加@Mapper注解

```
@Mapper
public interface UserMapper
```

2、定义mapper xml文件，如将user.xml文件放在了spring-series\lesson-005-mybatis\src\main\resources\mapper目录

3、spring配置类中添加@MapperScan注解，用来扫描@Mapper标注的类，将其注册到spring容器中

```
@MapperScan(basePackageClasses = {UserMapper.class}, annotationClass =  
Mapper.class)
```

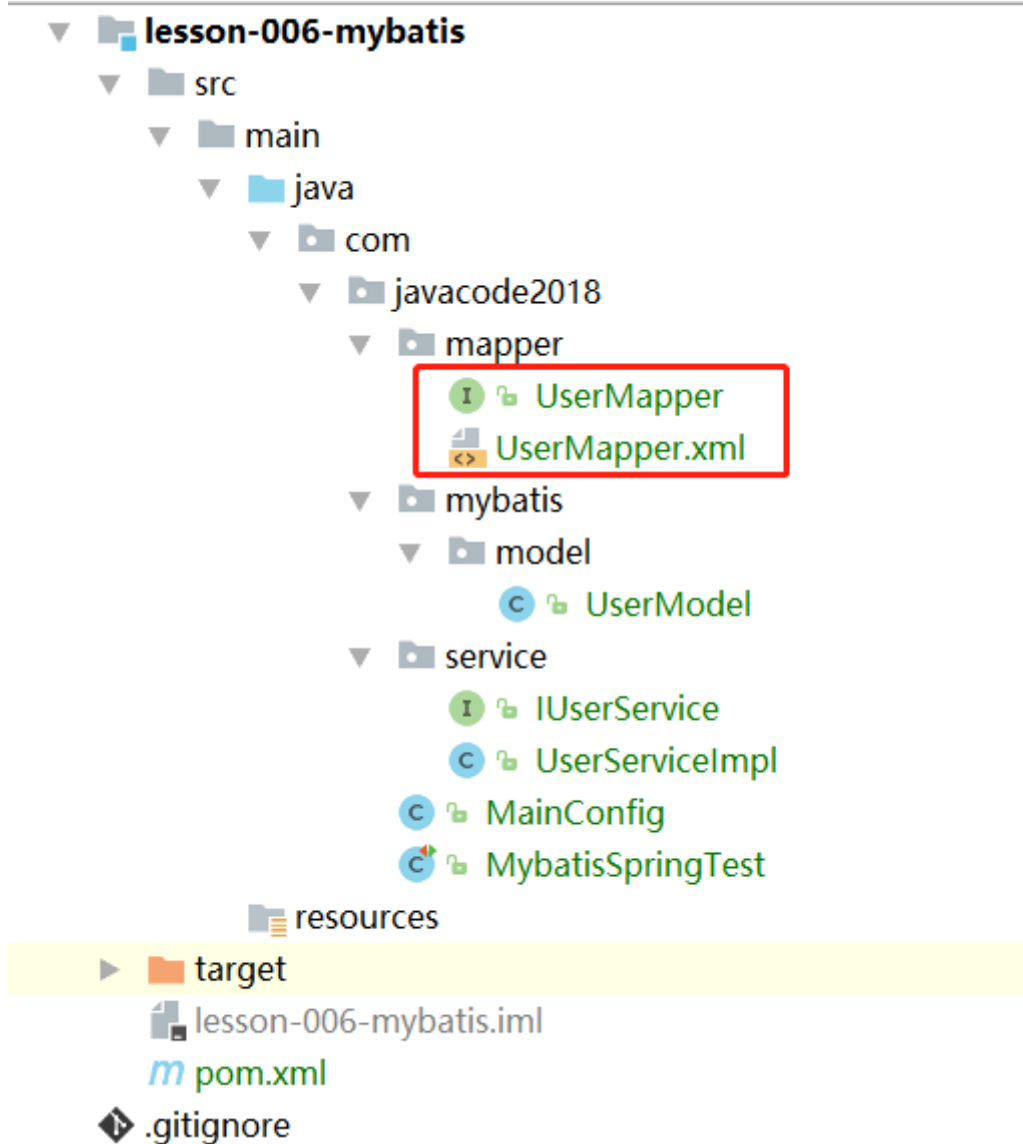
4、定义SqlSessionFactoryBean

```
@Bean  
public SqlSessionFactoryBean sqlSessionFactoryBean(DataSource dataSource) throws  
IOException {  
    SqlSessionFactoryBean sqlSessionFactoryBean = new SqlSessionFactoryBean();  
    //指定数据源  
    sqlSessionFactoryBean.setDataSource(dataSource);  
    //获取所有mapper.xml文件  
    Resource[] resources = new  
    PathMatchingResourcePatternResolver().getResources("classpath*:mapper/**/*.xml")  
    ;  
    //设置mapper的位置  
    sqlSessionFactoryBean.setMapperLocations(resources);  
    return sqlSessionFactoryBean;  
}
```

5、方式2

5.1、项目结构

这种方式，将mapper xml文件和mapper接口放在同一个目录，下面我们新建一个项目lesson-006-mybatis，结构如下



对比两个图，主要的区别：方式1中将UserMapper.xml文件放在了src\main\resources\mapper目录中，而这种将UserMapper.xml和UserMapper接口放在了同一个目录。

5.2、修改SqlSessionFactoryBean的定义

需要修改MainConfig中SqlSessionFactoryBean的定义，如下，更简洁了，不需要在指定mapper xml的位置了，这里需要注意一点，方式2中将mapper xml文件和mapper接口放在一个目录的时候，**这两个文件的名字必须一样**，这样在定义SqlSessionFactoryBean的时候才不需要指定mapper xml的位置。

```
@Bean
public SqlSessionFactoryBean sqlSessionFactoryBean(DataSource dataSource) throws
IOException {
    SqlSessionFactoryBean sqlSessionFactoryBean = new SqlSessionFactoryBean();
    sqlSessionFactoryBean.setDataSource(dataSource);
    return sqlSessionFactoryBean;
}
```

5.3、调整一下pom.xml配置

pom文件中需要加入下面配置，这样maven在打包的时候，才会将src/main/java目录中的xml打包到目标目录中，否则打包之后这些xml都丢失了，这个一定要注意。

```
<build>
```

```

<resources>
  <resource>
    <directory>${project.basedir}/src/main/java</directory>
    <includes>
      <include>**/*.xml</include>
    </includes>
  </resource>
  <resource>
    <directory>${project.basedir}/src/main/resources</directory>
    <includes>
      <include>**/*</include>
    </includes>
  </resource>
</resources>
</build>

```

5.4、小结

如果使用方式2需要注意下面2点：

- 1、mapper接口和mapper xml必须同名，且放在同一个目录
- 2、需要在pom.xml文件中加入指定的配置，否则打包之后xml丢失了

6、集成多个数据源

有时候我们的系统中需要用到多个数据源，每个数据源对应一个SqlSessionFactory，@MapperScan注解中可以通过sqlSessionTemplateRef来指定SqlSessionFactory的bean名称。

6.1、多数据源开发步骤

1、步骤1：@MapperScan中指定SqlSessionFactory的bean名称

每个db对应一个模块，通过包区分不同的模块，每个模块中指定一个spring的配置类，配置类需配置3个bean：数据源、事务管理器、SqlSessionFactory，下面是一个模块的spring配置类，注意下面代码的@MapperScan注解，当系统中有多个sqlSessionFactory的时候需要用sqlSessionFactoryRef属性指定了sqlSessionFactory的bean名称。

由于有多个数据源，所以代码中需要通过@Qualifier(DATASOURCE_BEAN_NAME)来限定注入的datasource的bean名称。

```

package com.javacode2018.mybatis.module1;

import org.apache.ibatis.annotations.Mapper;
import org.mybatis.spring.SqlSessionFactoryBean;
import org.mybatis.spring.annotation.MapperScan;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.transaction.TransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;

import javax.sql.DataSource;

```

```

import java.io.IOException;

@Configuration
@MapperScan(annotationClass = Mapper.class, sqlSessionFactoryRef =
Module1SpringConfig.SQL_SESSION_FACTORY_BEAN_NAME)
@ComponentScan
@EnableTransactionManagement
public class Module1SpringConfig {
    public final static String DATASOURCE_BEAN_NAME = "dataSourcemodule1";
    public final static String TRANSACTION_MANAGER_BEAN_NAME =
"transactionManagermodule1";
    public final static String SQL_SESSION_FACTORY_BEAN_NAME =
"sqlSessionFactorymodule1";

    //定义数据源
    @Bean
    public DataSource dataSourcemodule1() {
        org.apache.tomcat.jdbc.pool.DataSource dataSource = new
org.apache.tomcat.jdbc.pool.DataSource();
        dataSource.setDriverClassName("com.mysql.jdbc.Driver");
        dataSource.setUrl("jdbc:mysql://localhost:3306/module1?
characterEncoding=UTF-8");
        dataSource.setUsername("root");
        dataSource.setPassword("root123");
        dataSource.setInitialSize(5);
        return dataSource;
    }

    //定义事务管理器
    @Bean
    public TransactionManager
transactionManagermodule1(@Qualifier(DATASOURCE_BEAN_NAME) DataSource
dataSource) {
        return new DataSourceTransactionManager(dataSource);
    }

    //定义SqlSessionFactoryBean，用来创建SqlSessionFactory
    @Bean
    public SqlSessionFactoryBean
sqlSessionFactorymodule1(@Qualifier(DATASOURCE_BEAN_NAME) DataSource dataSource)
throws IOException {
        SqlSessionFactoryBean sqlSessionFactoryBean = new
SqlSessionFactoryBean();
        sqlSessionFactoryBean.setDataSource(dataSource);
        return sqlSessionFactoryBean;
    }
}

```

2、步骤2：指定事务管理器

每个db对应一个datasource，每个datasource需要指定一个事务管理器，通过@Transaction注解的transactionManager属性指定事务管理器，如下

```

@Transactional(transactionManager =
Module1SpringConfig.TRANSACTION_MANAGER_BEAN_NAME, rollbackFor =
Exception.class)

```

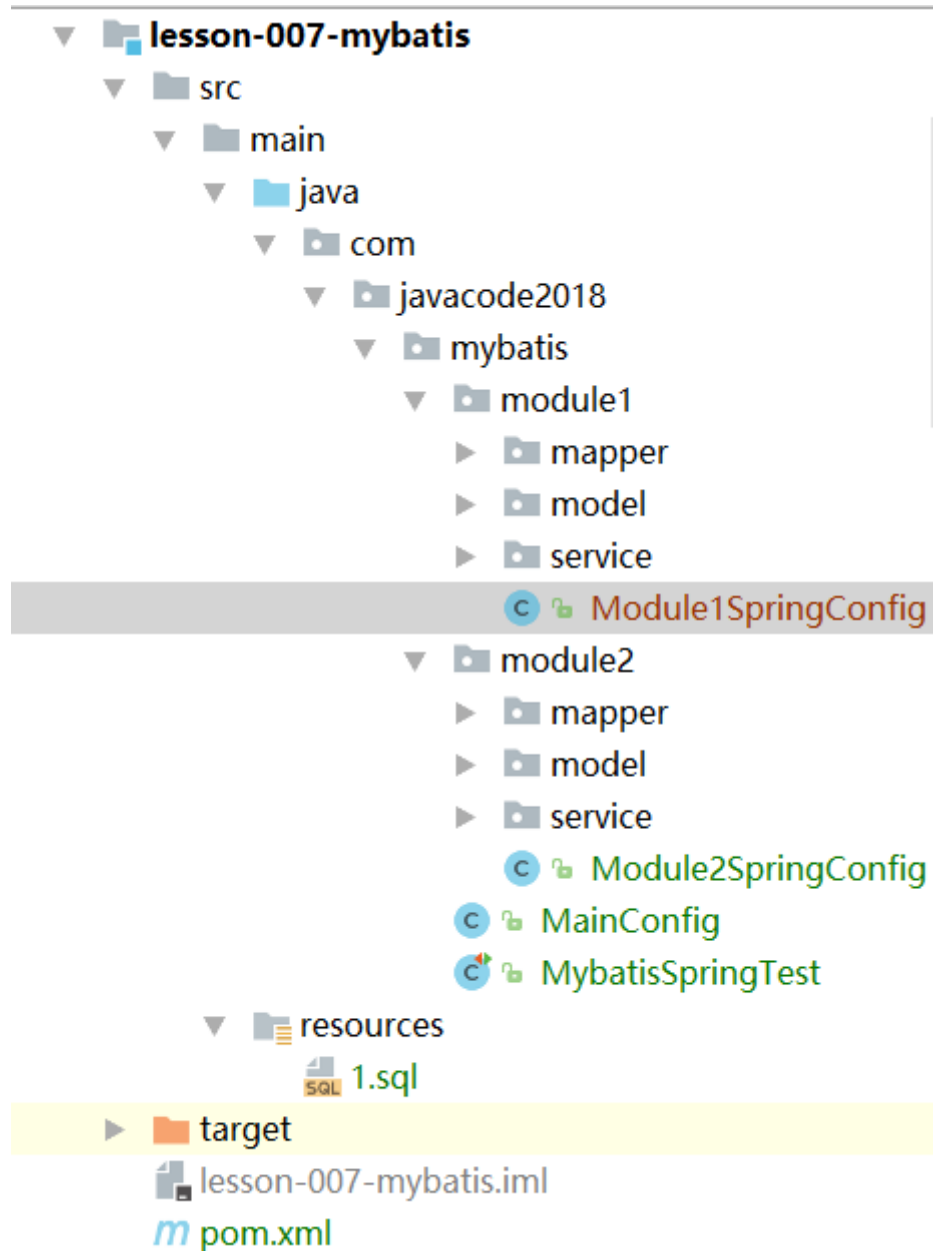
6.2、案例

下面我们通过案例来感受一下多个db的时候如何实现的。

1、案例代码

```
spring-series\lesson-007-mybatis
```

2、项目结构图



- 案例中我们准备了2个数据库：module1和module2，每个库中创建了一个t_user表
- module1库对应的所有类放在com.javacode2018.mybatis.module1包中
- module2库对应的所有类放在com.javacode2018.mybatis.module2包中
- Module1SpringConfig是module1的spring配置类
- Module2SpringConfig是module2的spring配置类

3、项目如何运行？

1、导入项目到idea中

2、执行sql脚本

```
/*创建2个库：module1和module2，每个库中创建一个t_user表*/
DROP DATABASE IF EXISTS `module1`;
CREATE DATABASE `module1`;

DROP DATABASE IF EXISTS `module2`;
CREATE DATABASE `module2`;

/*创建表结构*/
DROP TABLE IF EXISTS module1.t_user;
CREATE TABLE module1.t_user (
  id BIGINT AUTO_INCREMENT PRIMARY KEY COMMENT '主键，用户id，自动增长',
  `name` VARCHAR(32) NOT NULL DEFAULT '' COMMENT '姓名'
) COMMENT '用户表';

DROP TABLE IF EXISTS module2.t_user;
CREATE TABLE module2.t_user (
  id BIGINT AUTO_INCREMENT PRIMARY KEY COMMENT '主键，用户id，自动增长',
  `name` VARCHAR(32) NOT NULL DEFAULT '' COMMENT '姓名'
) COMMENT '用户表';

SELECT * FROM module1.t_user;
SELECT * FROM module2.t_user;
```

3、运行测试用例

```
com.javacode2018.mybatis.MybatisSpringTest
```

这个用例中有2个方法，大家可以运行一下，看看2个db的时候，效果如何。

6.3、小结

系统中集成多个数据源这种方式，大家作为了解就可以了，不建议这么使用，通常一个业务库对应一个module，模块与模块之间相互隔离，独立部署，便于维护扩展。

7、Spring集成MyBatis的原理

有些朋友可能想研究spring中是如何集成mybatis的，对这块比较感兴趣，这里我就不细说了，如果大家有兴趣的，可以先看一下我前面写的[MyBatis高手系列](#)，以及目前spring系列所有的文章，那么mybatis集成spring的原理，可以从@MapperScan注解下手，你会秒懂的！！！这里就留个大家去研究了。

再插2句，一直有很多粉丝说让我写springboot系列，为什么我要花这么多时间来写spring系列？写了这么多，用到的有多少？

大家如果将spring的原理吃透了，springboot不是小意思么，所以建议大家先把spring系列的所有文章吃透，那么springboot即使我还没写，你们自己去看一下源码也是很容易的懂了。看不懂是因为欠缺的太多，spring中的很多原理你还没有掌握，所以看springboot会懵逼，一点点来吧，先把基础打牢，基础ok了，后面学东西会很快的，慢就是快，细品！！！！

8、总结

本文介绍了spring中集成mybatis的2种方式，2种方式的重点都在上面的小结中，相对来说还是挺简单的，具体用哪种方式，大家自由选择。

9、Spring系列

1. [Spring系列第1篇：为何要学spring?](#)
2. [Spring系列第2篇：控制反转（IoC）与依赖注入（DI）](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)
4. [Spring系列第4篇：xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇：创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇：玩转bean scope，避免跳坑里！](#)
7. [Spring系列第7篇：依赖注入之手动注入](#)
8. [Spring系列第8篇：自动注入（autowire）详解，高手在于坚持](#)
9. [Spring系列第9篇：depend-on到底是干什么的?](#)
10. [Spring系列第10篇：primary可以解决什么问题?](#)
11. [Spring系列第11篇：bean中的autowire-candidate又是干什么的?](#)
12. [Spring系列第12篇：lazy-init：bean延迟初始化](#)
13. [Spring系列第13篇：使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇：lookup-method和replaced-method比较陌生，怎么玩的?](#)
15. [Spring系列第15篇：代理详解（Java动态代理&cglib代理）?](#)
16. [Spring系列第16篇：深入理解java注解及spring对注解的增强（预备知识）](#)
17. [Spring系列第17篇：@Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇：@ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇：@import详解\(bean批量注册\)](#)
20. [Spring系列第20篇：@Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇：注解实现依赖注入（@Autowired、@Resource、@Primary、@Qualifier）](#)
22. [Spring系列第22篇：@Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇：Bean生命周期详解](#)
24. [Spring系列第24篇：父子容器详解](#)
25. [Spring系列第25篇：@Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇：国际化详解](#)
27. [Spring系列第27篇：spring事件机制详解](#)
28. [Spring系列第28篇：Bean循环依赖详解](#)
29. [Spring系列第29篇：BeanFactory扩展（BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor）](#)
30. [Spring系列第30篇：jdk动态代理和cglib代理](#)
31. [Spring系列第31篇：aop概念详解](#)
32. [Spring系列第32篇：AOP核心源码、原理详解](#)
33. [Spring系列第33篇：ProxyFactoryBean创建AOP代理](#)
34. [Spring系列第34篇：@Aspect中@Pointcut 12种用法](#)
35. [Spring系列第35篇：@Aspect中5中通知详解](#)
36. [Spring系列第36篇：@EnableAspectJAutoProxy、@Aspect中通知顺序详解](#)
37. [Spring系列第37篇：@EnableAsync & @Async 实现方法异步调用](#)
38. [Spring系列第38篇：@Scheduled & @EnableScheduling定时器详解](#)
39. [Spring系列第39篇：强大的Spel表达式](#)
40. [Spring系列第40篇：缓存使用（@EnableCaching、@Cacheable、@CachePut、@CacheEvict、@Caching、@CacheConfig）](#)
41. [Spring系列第41篇：@EnableCaching集成redis缓存](#)
42. [Spring系列第42篇：玩转JdbcTemplate](#)

43. [Spring系列第43篇：spring中编程式事务怎么用的？](#)
44. [Spring系列第44篇：详解spring声明式事务\(@Transactional\)](#)
45. [Spring系列第45篇：带你吃透Spring事务7种传播行为](#)
46. [Spring系列第46篇：Spring如何管理多数据源事务？](#)
47. [Spring系列第47篇：spring编程式事务源码解析](#)
48. [Spring系列第48篇：@Transaction 事务源码解析](#)
49. [Spring系列第49篇：通过Spring事务实现MQ中的事务消息](#)
50. [Spring系列第50篇：spring事务拦截器顺序如何控制？](#)
51. [Spring系列第51篇：导致 Spring 事务失效常见的几种情况](#)
52. [Spring系列第52篇：Spring实现数据库读写分离](#)

10、更多好文章

1. [Java高并发系列（共34篇）](#)
2. [MySql高手系列（共27篇）](#)
3. [Maven高手系列（共10篇）](#)
4. [Mybatis系列（共12篇）](#)
5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要，它是什么？怎么实现？](#)
7. [泛型，有点难度，会让很多人懵逼，那是因为你没有看这篇文章！](#)

11、福利

这边整理了几百本计算机相关的电子书，下面是部分截图，常见的基本上都有，获取方式，扫描文末二维码，发送：**电子书**



C和指针.pdf



658381 Java工...



[www.ebook23....



[www.ebook23....



[www.ebook23....



[www.ebook23....



[www.ebook23....



[www.ebook23....



[www.ebook23....



[www.javazx.co...



Effective Java中...



Elasticsearch权...



HBase不睡觉书...



HBase实战.pdf



Head First 设计...



Java 8实战.pdf



Java JDK 8学习...



JAVA 遗传算法编..



JavaParser-JUG...



javaparsersite...



Java编程宝典.pdf



Java编程思想(第...



Java并发编程之...



Java常用算法手...



JAVA程序员面试...



Java多线程编程...



JAVA多线程设计...



Java高手真经(应...



Java国际认证(SC...



Java经典编程30...



java经典实例 第...



JAVA面试题解惑...



Java深度历险.pdf



java网络编程精...



Java线程(第三版)...



Java线程与并发...



Java性能优化权...



JAVA语言导学.pdf



Java自学网 [jav...



JVM+实用参数...



linux学习.pdf



Maven实战.pdf



MongoDB权威...



MySQL 5.6从零...



MySQL管理之道...



MySQL技术精粹...



MySQL技术内幕...



Neo4j权威指南...



Netty权威指南.p...



Spring Boot 2...



SPRING 实战 (...



Spring.Batch批...



《Apache Kafka...



编写高质量代码...



超越java: 探讨...



程序员实用算法...



大公司最爱问的9...



大型分布式网站...



代码大全 第2版...



分布式Java应用...



分布式服务框架...



分布式系统常用...



疯狂Java: 突破...



高效算法: 竞赛...



互联网创业核心...



架构解密: 从分...



Offer 面试指南



解析 Java 虚拟机...

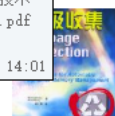


程序员面试揭秘...



面试宝典

名称: 互联网创业核心技术-构建可伸缩的web应用.pdf
大小: 120.84MB
修改时间: 2020-10-20 14:01



架构解密: 从分...

剑指offer 名企面...



面试宝典.pdf

解析Java虚拟机...



面试资料整理.pdf

进军硅谷 程序员...



排序与查找.pdf

尽在双11: 阿里...



七周七并发模型_...

精通Spring 4.x ...



人人都是架构师+...

垃圾收集.pdf



深入Linux内核架...



深入RabbitMQ...



深入分析Java W...



深入理解ElasticS...



深入理解Java虚...



深入理解Spring ...



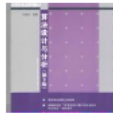
实战Java高并发...



数据结构与算法...



算法的乐趣.pdf



算法设计与分析 ...



透视JAVA--反编...



图解Java多线程...



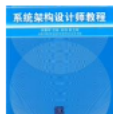
图解TCP_IP_第5...



图解数据结构-使...



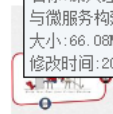
网络IO模型.pdf



系统架构设计师...



这样编码才规范 ...



自己动手写Java...



最新-黑马月入12...



加我微信: spring888java, 进群一起交流java相关各种技术!!

Spring系列第54篇: 集成junit

通常我们写完代码之后，为了确保代码的正确性，都需要自己测试一遍，看一下代码的运行结果和我们期望的结果是不是一样的，也就是我们常说的单元测试，java中最常用的单元测试框架是junit，本文主要介绍3个内容：

- 1、玩转junit
- 2、spring集成junit
- 3、开发工具中使用junit

1、背景

我们写了一个工具类，有2个方法

```
package com.javacode2018.junit.demo1;

public class MathUtils {
    /**
     * 获取最大的数字
     *
     * @param args
     * @return
     */
    public static int max(int... args) {
        int result = Integer.MIN_VALUE;
        for (int arg : args) {
            result = result > arg ? result : arg;
        }
        return result;
    }

    /**
     * 获取最小的数字
     *
     * @param args
     * @return
     */
    public static int min(int... args) {
        int result = Integer.MAX_VALUE;
        for (int arg : args) {
            result = result < arg ? result : arg;
        }
        return result;
    }
}
```

然后我们想测试这两个方法，下面我们来写测试代码，如下，测试一下max方法和min方法的结果和我们期望的结果是否一致，不一致的时候，输出一段文字

```
package com.javacode2018.junit.demo1;

public class MathUtilsTest1 {
    public static void main(String[] args) {
        testMax();
    }
}
```

```

        testMin();
    }

    public static void testMax() {
        int result = MathUtils.max(1, 2, 3);
        if (result != 3) {
            System.out.println(String.format("max 方法有问题，期望结果是3，实际结果
是%d", result));
        }
    }

    public static void testMin() {
        int result = MathUtils.min(1, 2, 3);
        if (result != 1) {
            System.out.println(String.format("min 方法有问题，期望结果是3，实际结果
是%d", result));
        }
    }
}

```

上面我们要测试的方法就2个，若需测试的方法很多的时候，咱们需要写大量的这种测试代码，工作量还是蛮大的，而junit做的事情和上面差不多，都是用来判断被测试的方法和期望的结果是否一致，不一致的时候给出提示，不过junit用起来更容易一些，还有各种开发用到的ide（eclipse、idea）结合的更好一些，用起来特别的顺手。

2、junit用法详解

2.1、使用步骤

1)、添加junit maven配置，这里我们就用4.13，你们也可以用最新的

```

<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13</version>
</dependency>

```

2)、写测试用例，在写好的测试方法上面添加@Test注解，比如我们需要对上面案例中的max方法进行测试，通常会新建一个测试类，类名为被测试的类加上Test后缀，即：MathUtilsTest，然后在这个类我们需要写max方法的测试方法，如下，需要我们在max方法上面加上@Test注解

```

package com.javacode2018.junit.demo1;

import org.junit.Assert;
import org.junit.Test;

public class MathUtilsTest {
    @Test
    public void max() throws Exception {
        int result = MathUtils.max(1, 2, 3);
        //判断测试结果和我们期望的结果是否一致
        Assert.assertEquals(result, 3);
    }
}

```

3、运行测试用例，现在测试代码都写好了，下面我们写个类来启动测试用例，这里需要使用JUnitCore.runClasses方法来运行测试用例，如下：

```

package com.javacode2018.junit.demo1;

import org.junit.runner.JUnitCore;
import org.junit.runner.Result;
import org.junit.runner.notification.Failure;

public class Demo1TestRunner {
    public static void main(String[] args) {
        //使用JUnitCore.runClasses方法传入测试用例的类，然后获取测试用例的运行结果
        Result result = JUnitCore.runClasses(MathUtilsTest.class);
        //获取失败的用例
        for (Failure failure : result.getFailures()) {
            System.out.println(failure);
        }
        //获取所有测试用例是否执行成功
        System.out.println(result.wasSuccessful());
    }
}

```

2.2、同时运行多个测试用例

可以一个测试类中写多个测试方法，每个方法上加上@Test注解就可以了，然后通过JUnitCore来执行就可以，下面代码中我们写2个方法对MathUtils中的max和min方法都进行测试，我们故意将执行结果和期望结果搞成不一致的，运行下面代码，然后看看运行结果。

```

package com.javacode2018.junit.demo2;

import com.javacode2018.junit.demo1.MathUtils;
import org.junit.Assert;
import org.junit.Test;
import org.junit.runner.JUnitCore;
import org.junit.runner.Result;
import org.junit.runner.manipulation.Ordering;
import org.junit.runner.notification.Failure;

public class MathUtilsTest2 {
    @Test
    public void max() throws Exception {

```

```

        int result = MathUtils.max(1, 2, 3);
        //判断测试结果和我们期望的结果是否一致
        Assert.assertEquals(result, 1);
    }

    @Test
    public void min() throws Exception {
        int result = MathUtils.min(1, 2, 3);
        //判断测试结果和我们期望的结果是否一致
        Assert.assertEquals(result, 3);
    }

    public static void main(String[] args) {
        Result result = JUnitCore.runClasses(MathUtilsTest2.class);

        System.out.println("失败用例个数:" + result.getFailures().size());
        for (Failure failure : result.getFailures()) {
            System.out.println(failure);
        }
        System.out.println("运行测试用例个数:" + result.getRunCount());
        System.out.println("运行测试用例总耗时(ms):" + result.getRuntime());
        System.out.println("测试用例是否都成功了: " + result.wasSuccessful());
    }
}

```

运行输出如下，运行了2个用例，失败了2个，测试的详细信息都被输出了

```

失败用例个数:2
max(com.javacode2018.junit.demo2.MathUtilsTest2): expected:<3> but was:<1>
min(com.javacode2018.junit.demo2.MathUtilsTest2): expected:<1> but was:<3>
运行测试用例个数:2
运行测试用例总耗时(ms):11
测试用例是否都成功了: false

```

2.3、使用断言

什么是断言？

断言是用来判断程序的运行结果和我们期望的结果是不是一致的，如果不一致，会抛出异常，断言中有3个信息比较关键

- 1、被测试的数据
- 2、期望的数据
- 3、抛出异常

断言提供的方法将被测试的数据和期望的数据进行对比，如果不一样的时候，将抛出异常，程序可以捕获这个异常，这样就可以知道测试失败了。

junit中的org.junit.Assert类中提供了大量静态方法，用来判断被测试的数据和期望的数据是否一致，不一致，将抛出异常，这里随便列几个大家看一下吧


```
//判断condition如果不是true，将抛出异常，异常的提示信息是message
public static void assertTrue(String message, boolean condition)

//判断expected和actual是否相等，如果不相等，将抛出异常
public static void assertEquals(Object expected, Object actual)
```

用法，如：

```
int result = MathUtils.max(1, 2, 3);
//判断测试结果和我们期望的结果是否一致
Assert.assertEquals(result, 1);
```

2.4、测试套件：批量运行测试用例

到目前为止，我们还只能一次运行一个测试类，如下

```
JUnitCore.runClasses(MathUtilsTest2.class)
```

但是在实际项目中，我们可能会有很多测试类，需要批量运行。

比如下面有2个测试类

MathUtilsTest3001.java

```
package com.javacode2018.junit.demo3;

import com.javacode2018.junit.demo1.MathUtils;
import org.junit.Assert;
import org.junit.Test;

public class MathUtilsTest3001 {
    @Test
    public void max() throws Exception {
        int result = MathUtils.max(1, 2, 3);
        //判断测试结果和我们期望的结果是否一致
        Assert.assertEquals(result, 3);
    }

    @Test
    public void min() throws Exception {
        int result = MathUtils.min(1, 2, 3);
        //判断测试结果和我们期望的结果是否一致
        Assert.assertEquals(result, 1);
    }
}
```

MathUtilsTest3002.java

```
package com.javacode2018.junit.demo3;

import com.javacode2018.junit.demo1.MathUtils;
import org.junit.Assert;
import org.junit.Test;
import org.junit.runner.RunWith;
import org.junit.runner.RunWith;
import org.junit.runner.Result;
```



```
import org.junit.runner.notification.Failure;

public class MathUtilsTest3002 {
    @Test
    public void max() throws Exception {
        int result = MathUtils.max(100, 99, 200);
        //判断测试结果和我们期望的结果是否一致
        Assert.assertEquals(result, 200);
    }

    @Test
    public void min() throws Exception {
        int result = MathUtils.min(1, -1, 10);
        //判断测试结果和我们期望的结果是否一致
        Assert.assertEquals(result, -1);
    }
}
```

现在我们希望同时运行上面2个测试类，我们可以这么做，创建一个AllTest.java类，注意这个类上有2个注解比较特殊，都是junit提供的，@RunWith表示这是一个测试套件类，需要批量运行测试类，具体要运行哪些测试类呢，通过@Suite.SuiteClasses来指定

```
package com.javacode2018.junit.demo3;

import org.junit.runner.RunWith;
import org.junit.runners.Suite;

@RunWith(Suite.class)
@Suite.SuiteClasses({MathUtilsTest3001.class, MathUtilsTest3002.class})
public class AllTest {
}
```

下面来个启动类，将AllTest传递给JUnitCore.runClasses

```
package com.javacode2018.junit.demo3;

import com.javacode2018.junit.demo1.MathUtilsTest;
import com.javacode2018.junit.demo2.MathUtilsTest2;
import org.junit.runner.JUnitCore;
import org.junit.runner.Result;
import org.junit.runner.notification.Failure;

public class Demo3TestRunner {
    public static void main(String[] args) {
        //①: 传入AllTest
        Result result = JUnitCore.runClasses(AllTest.class);

        System.out.println("失败用例个数:" + result.getFailures().size());
        for (Failure failure : result.getFailures()) {
            System.out.println(failure);
        }
        System.out.println("运行测试用例个数:" + result.getRunCount());
        System.out.println("运行测试用例总耗时(ms):" + result.getRuntime());
        System.out.println("测试用例是否都成功了: " + result.wasSuccessful());
    }
}
```

运行输出

```
失败用例个数:0
运行测试用例个数:4
运行测试用例总耗时(ms):12
测试用例是否都成功了: true
```

测试套件中不仅可以包含基本的测试类，而且可以包含其它的测试套件，这样可以很方便的分层管理不同模块的单元测试代码，比如下面代码，Module2Test和Module2Test都是测试套件

```
@RunWith(Suite.class)
@Suite.SuiteClasses({Test1.class, Test2.class})
public class Module2Test {
}

@RunWith(Suite.class)
@Suite.SuiteClasses({Test1.class, Test2.class})
public class Module2Test {
}

@RunWith(Suite.class)
@Suite.SuiteClasses({Module2Test.class, Module2Test.class, Test3.java})
public class AllTest {
}

//运行AllTest
JUnitCore.runClasses(AllTest.class);
```

2.5、Junit常用注解

1) @Test注解

@Test: 将一个普通方法修饰成一个测试方法

@Test(expected=xx.class): xx.class 表示异常类，表示测试的方法抛出此异常时，认为是正常的测试通过的

@Test(timeout = 毫秒数): 测试方法执行时间是否符合预期

2) @BeforeClass

会在所有的方法执行前被执行，static 方法（全局只会执行一次，而且是第一个运行）

3) @AfterClass

会在所有的方法执行之后进行执行，static 方法（全局只会执行一次，而且是最后一个运行）

4) @Before

会在每一个测试方法被运行前执行一次

5) @After

会在每一个测试方法运行后被执行一次

6) @Ignore

所修饰的测试方法会被测试运行器忽略

7) @RunWith

可以更改测试运行器 org.junit.runner.Runner

下面的案例，基本上用到了上面所有的注解，大家结合输出理解一下。

```
package com.javacode2018.junit.demo4;

import com.javacode2018.junit.demo1.MathUtils;
import org.junit.*;
import org.junit.runner.JUnit4;
import org.junit.runner.Result;
import org.junit.runner.notification.Failure;

import java.util.concurrent.TimeUnit;

public class MathUtilsTest4 {

    @BeforeClass
    public static void bc() {
        System.out.println("@BeforeClass");
        System.out.println("-----");
    }

    @AfterClass
    public static void ac() {
        System.out.println("@AfterClass");
    }

    @Before
    public void bf() {
        System.out.println("@Before:" + this);
    }

    @After
    public void af() {
        System.out.println("@After:" + this);
        System.out.println("#####");
    }

    @Test
    public void max() throws Throwable {
        System.out.println("max():" + this);
        int result = MathUtils.max(1, 2, 3);
        //判断测试结果和我们期望的结果是否一致
        Assert.assertEquals(result, 3);
    }

    @Test
    public void min() throws Exception {
        System.out.println("min():" + this);
        int result = MathUtils.min(1, 2, 3);
        //判断测试结果和我们期望的结果是否一致
        Assert.assertEquals(result, 1);
    }
}
```

```

    }

    //方法运行时间超过了timeout，表示测试用例运行失败
    @Test(timeout = 1000)
    public void timeOutTest() throws InterruptedException {
        System.out.println("timeOutTest():" + this);
        TimeUnit.SECONDS.sleep(2000);
    }

    //方法若未抛出expected指定的异常，表示测试用例运行失败
    @Test(expected = NullPointerException.class)
    public void expectedTest() {
        System.out.println("expectedTest():" + this);
        new RuntimeException("异常不匹配");
    }

    @Test
    @Ignore
    public void ignoredMethod() {
        System.out.println("我是被忽略的方法");
    }

    public static void main(String[] args) {
        Result result = JUnitCore.runClasses(MathUtilsTest4.class);
        System.out.println("-----");
        System.out.println("运行测试用例个数:" + result.getRunCount());
        System.out.println("失败用例个数:" + result.getFailures().size());
        for (Failure failure : result.getFailures()) {
            System.out.println(failure);
        }
        System.out.println("运行测试用例总耗时(ms):" + result.getRuntime());
        System.out.println("测试用例是否都成功了: " + result.wasSuccessful());

    }
}

```

运行结果如下

```

@BeforeClass
-----
@Before:com.javacode2018.junit.demo4.MathUtilsTest4@78e03bb5
timeOutTest():com.javacode2018.junit.demo4.MathUtilsTest4@78e03bb5
@After:com.javacode2018.junit.demo4.MathUtilsTest4@78e03bb5
#####
@Before:com.javacode2018.junit.demo4.MathUtilsTest4@48533e64
max():com.javacode2018.junit.demo4.MathUtilsTest4@48533e64
@After:com.javacode2018.junit.demo4.MathUtilsTest4@48533e64
#####
@Before:com.javacode2018.junit.demo4.MathUtilsTest4@7e0b37bc
min():com.javacode2018.junit.demo4.MathUtilsTest4@7e0b37bc
@After:com.javacode2018.junit.demo4.MathUtilsTest4@7e0b37bc
#####
@Before:com.javacode2018.junit.demo4.MathUtilsTest4@1a93a7ca
expectedTest():com.javacode2018.junit.demo4.MathUtilsTest4@1a93a7ca
@After:com.javacode2018.junit.demo4.MathUtilsTest4@1a93a7ca
#####
@AfterClass

```

```
-----
运行测试用例个数:4
失败用例个数:3
timeOutTest(com.javacode2018.junit.demo4.MathUtilsTest4): test timed out after
1000 milliseconds
max(com.javacode2018.junit.demo4.MathUtilsTest4): hah
expectedTest(com.javacode2018.junit.demo4.MathUtilsTest4): Expected exception:
java.lang.NullPointerException
运行测试用例总耗时(ms):1018
测试用例是否都成功了: false
```

从输出中可以看出

- @BeforeClass和@AfterClass标注的方法只会运行一次
- 每个@Test标注的方法运行之前会先运行@Before标注的方法，然后运行@Test标注的这个方法，之后再运行@After
- 从this的输出看出，每个@Test运行的时候，当前类的实例都会重新创建一个新的
- 不论@Test标注的方法是否异常，@AfterClass、@After标注的方法都会执行，且异常会被淹没，输出中看不到异常信息

2.6、参数化测试

JUnit 4 引入了一个新的功能**参数化测试**。

参数化测试允许开发人员使用不同的值反复运行同一个测试，你将遵循 5 个步骤来创建**参数化测试**。

- 用 @RunWith(Parameterized.class) 来注释 test 类。
- 创建一个由 @Parameters 注释的公共的静态方法，它返回一个对象的集合(数组)来作为测试数据集。
- 创建一个公共的构造函数，它接受和一行测试数据相等同的东西。
- 为每一列测试数据创建一个实例变量。
- 用实例变量作为测试数据的来源来创建你的测试用例。

可能大家看了上面的理解，还是比较迷糊。

比如我们com.javacode2018.junit.demo1.MathUtils#max测试下面几组数组

```
1, 2, 3
100, 99, 80
30, -1, 100
```

我们可以这么写

```
package com.javacode2018.junit.demo5;

import com.javacode2018.junit.demo1.MathUtils;
import org.junit.Assert;
import org.junit.Test;
import org.junit.runner.RunWith;
import org.junit.runner.RunWith;
import org.junit.runner.notification.Failure;
import org.junit.runners.Parameterized;

import java.util.ArrayList;
```

```

import java.util.Arrays;
import java.util.List;

@RunWith(Parameterized.class)
public class MathUtilsTest5 {

    public static class TestData {
        int[] testData; //测试数据
        int expectedValue; //预期的结果

        public TestData(int[] testData, int expectedValue) {
            this.testData = testData;
            this.expectedValue = expectedValue;
        }

        @Override
        public String toString() {
            return "TestData{" +
                "testData=" + Arrays.toString(testData) +
                ", expectedValue=" + expectedValue +
                '}';
        }
    }

    private TestData testData;

    @Parameterized.Parameters
    public static List<TestData> initTestData() {
        System.out.println("initTestData()");
        //key: 期望的结果, value: max方法需要测试的数据
        List<TestData> result = new ArrayList<>();
        result.add(new TestData(new int[]{1, 2, 3}, 3));
        result.add(new TestData(new int[]{100, 99, 80}, 100));
        result.add(new TestData(new int[]{30, -1, 100}, 100));
        return result;
    }

    public MathUtilsTest5(TestData testData) {
        System.out.println("MathUtilsTest5构造器:" + testData);
        this.testData = testData;
    }

    @Test
    public void maxTest() throws Throwable {
        System.out.println(this.hashCode() + ",maxTest():" + this.testData);
        int result = MathUtils.max(this.testData.testData);
        //判断测试结果和我们期望的结果是否一致
        Assert.assertEquals(this.testData.expectedValue, result);
        System.out.println("#####");
    }

    public static void main(String[] args) {
        Result result = JUnitCore.runClasses(MathUtilsTest5.class);
        System.out.println("-----");
        System.out.println("运行测试用例个数:" + result.getRunCount());
        System.out.println("失败用例个数:" + result.getFailures().size());
        for (Failure failure : result.getFailures()) {
            System.out.println(failure);
        }
    }
}

```

```

    }
    System.out.println("运行测试用例总耗时(ms):" + result.getRuntime());
    System.out.println("测试用例是否都成功了: " + result.wasSuccessful());

}
}

```

为了方便大家理解代码的运行过程，代码中添加了很多日志输出，运行结果如下，结合代码和输出，理解很容易

```

initTestData()
MathUtilsTest5构造器:TestData{testData=[1, 2, 3], expectedValue=3}
721748895,maxTest():TestData{testData=[1, 2, 3], expectedValue=3}
#####
MathUtilsTest5构造器:TestData{testData=[100, 99, 80], expectedValue=100}
463345942,maxTest():TestData{testData=[100, 99, 80], expectedValue=100}
#####
MathUtilsTest5构造器:TestData{testData=[30, -1, 100], expectedValue=100}
195600860,maxTest():TestData{testData=[30, -1, 100], expectedValue=100}
#####
-----
运行测试用例个数:3
失败用例个数:0
运行测试用例总耗时(ms):12
测试用例是否都成功了: true

```

3、Spring集成junit

spring集成junit比较简单，下面我们来个案例感受一下。

3.1、加入maven配置

```

<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-test</artifactId>
    <version>5.2.6.RELEASE</version>
</dependency>
<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13</version>
</dependency>

```

3.2、来个spring的入口配置类

```

package com.javacode2018.springjunit;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class MainConfig {

    @Bean

```

```

    public String name() {
        return "路人甲java";
    }

    @Bean
    public int age() {
        return 30;
    }
}

```

3.3、来个junit测试类集成spring

下面我们来个测试类，对上面的MainConfig中注册的2个bean进行测试

```

package com.javacode2018.springjunit;

import org.junit.Assert;
import org.junit.Test;
import org.junit.runner.RunWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.test.context.ContextConfiguration;
import org.springframework.test.context.junit4.SpringRunner;

@RunWith(SpringRunner.class) //@1
@ContextConfiguration(classes = {MainConfig.class}) //@2
public class MainConfigTest {

    @Autowired
    private String name;

    @Autowired
    private int age;

    @Test
    public void nameTest() {
        System.out.println(this.name);
        Assert.assertEquals("路人甲java", this.name);
    }

    @Test
    public void ageTest() {
        System.out.println(this.age);
        Assert.assertEquals(30, this.age);
    }
}

```

注意上面的@1和@2的代码了，这就是spring集成junit的关键代码，@RunWith的值是org.springframework.test.context.junit4.SpringRunner，@ContextConfiguration注解通过classes属性指定spring启动类，如果是xml的方式，可以使用另外一个属性value或者locations来指定入口xml的位置。

上面代码中name和age属性上面都有@Autowired注解，这2个属性会被自动从spring容器中注入进来。

3.4、来个测试启动类

```
package com.javacode2018.springjunit;

import org.junit.runner.JUnitCore;
import org.junit.runner.Result;
import org.junit.runner.notification.Failure;

public class TestRunner {

    public static void main(String[] args) {
        Result result = JUnitCore.runClasses(MainConfigTest.class);
        System.out.println("-----");
        System.out.println("运行测试用例个数:" + result.getRunCount());
        System.out.println("失败用例个数:" + result.getFailures().size());
        for (Failure failure : result.getFailures()) {
            System.out.println(failure);
        }
        System.out.println("运行测试用例总耗时(ms):" + result.getRuntime());
        System.out.println("测试用例是否都成功了: " + result.wasSuccessful());
    }
}
```

运行输出

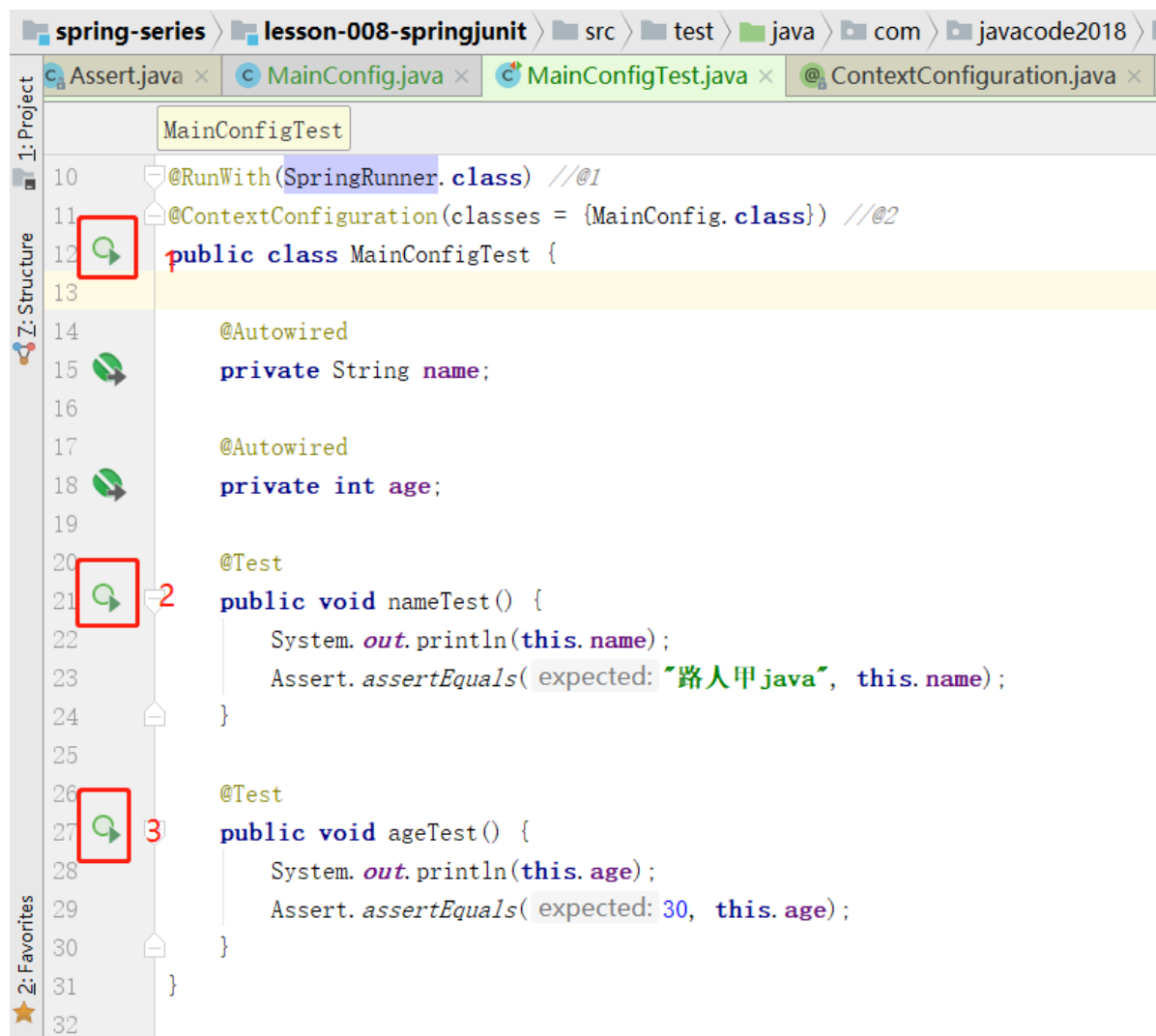
```
30
路人甲 java
-----
运行测试用例个数:2
失败用例个数:0
运行测试用例总耗时(ms):422
测试用例是否都成功了: true
```

4、开发工具中使用junit

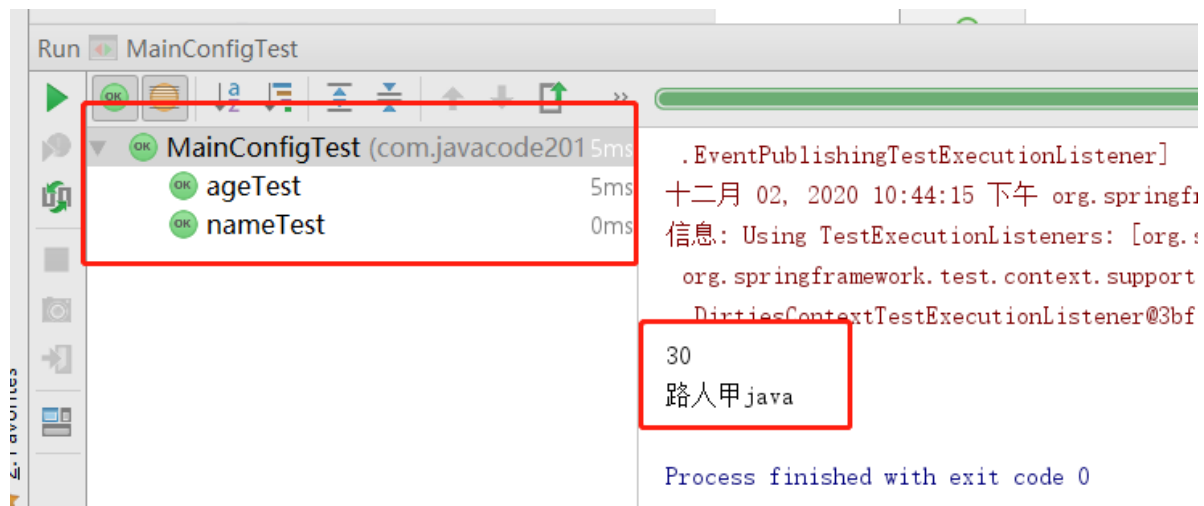
上面介绍的所有案例，都是通过main方法中用JUnitCore.runClasses来运行测试用例的，实际上有更简单的方式。

java的常用开发工具有eclipse和idea，这两个工具都将junit集成好了，通过开发工具继承的功能，运行测试用例更方便，不需要我们写JUnitCore.runClasses代码了。

我们来演示一下，如下图



点击1处的蓝色箭头，即可运行当前测试类中的所有用例，即2个测试方法，运行效果如下，左边显示了运行的测试用例列表，右边的显示了输出信息。



图中的2和3也可以点击，只会运行选中的那个测试方法，比如点击2，只会运行nameTest()这个用例，效果如下



5、总结

- 1、本文详细介绍了junit的用法，常用的注解有@Test、@BeforeClass、@AfterClass、@Before、@After、@Ignore、@RunWith，这些都要掌握
- 2、spring中集成junit，主要的配置就是在测试类上面需要加上下面代码

```
@RunWith(SpringRunner.class) //@1
@SpringBootTest(classes = {MainConfig.class}) //@2
```

- 3、开发工具中使用junit更容易

6、案例源码

git地址：
<https://gitee.com/javacode2018/spring-series>

本文案例对应源码：
spring-series\lesson-008-junit
spring-series\lesson-008-springjunit

大家star一下，所有系列代码都会在这个里面，还有所有原创文章的连接也在里面，方便查阅！！

7、Spring系列

1. [Spring系列第1篇：为何要学spring?](#)
2. [Spring系列第2篇：控制反转 \(IoC\) 与依赖注入 \(DI\)](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)
4. [Spring系列第4篇：xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇：创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇：玩转bean scope，避免跳坑里！](#)
7. [Spring系列第7篇：依赖注入之手动注入](#)
8. [Spring系列第8篇：自动注入 \(autowire\) 详解，高手在于坚持](#)

9. [Spring系列第9篇: depend-on到底是干什么的?](#)
10. [Spring系列第10篇: primary可以解决什么问题?](#)
11. [Spring系列第11篇: bean中的autowire-candidate又是干什么的?](#)
12. [Spring系列第12篇: lazy-init: bean延迟初始化](#)
13. [Spring系列第13篇: 使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇: lookup-method和replaced-method比较陌生, 怎么玩的?](#)
15. [Spring系列第15篇: 代理详解 \(Java动态代理&cglib代理\) ?](#)
16. [Spring系列第16篇: 深入理解java注解及spring对注解的增强 \(预备知识\)](#)
17. [Spring系列第17篇: @Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇: @ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇: @import详解\(bean批量注册\)](#)
20. [Spring系列第20篇: @Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇: 注解实现依赖注入 \(@Autowired、@Resource、@Primary、@Qualifier\)](#)
22. [Spring系列第22篇: @Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇: Bean生命周期详解](#)
24. [Spring系列第24篇: 父子容器详解](#)
25. [Spring系列第25篇: @Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇: 国际化详解](#)
27. [Spring系列第27篇: spring事件机制详解](#)
28. [Spring系列第28篇: Bean循环依赖详解](#)
29. [Spring系列第29篇: BeanFactory扩展 \(BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor\)](#)
30. [Spring系列第30篇: jdk动态代理和cglib代理](#)
31. [Spring系列第31篇: aop概念详解](#)
32. [Spring系列第32篇: AOP核心源码、原理详解](#)
33. [Spring系列第33篇: ProxyFactoryBean创建AOP代理](#)
34. [Spring系列第34篇: @Aspect中@Pointcut 12种用法](#)
35. [Spring系列第35篇: @Aspect中5中通知详解](#)
36. [Spring系列第36篇: @EnableAspectJAutoProxy、@Aspect中通知顺序详解](#)
37. [Spring系列第37篇: @EnableAsync & @Async 实现方法异步调用](#)
38. [Spring系列第38篇: @Scheduled & @EnableScheduling定时器详解](#)
39. [Spring系列第39篇: 强大的Spel表达式](#)
40. [Spring系列第40篇: 缓存使用 \(@EnableCaching、@Cacheable、@CachePut、@CacheEvict、@Caching、@CacheConfig\)](#)
41. [Spring系列第41篇: @EnableCaching集成redis缓存](#)
42. [Spring系列第42篇: 玩转jdbcTemplate](#)
43. [Spring系列第43篇: spring中编程式事务怎么用的?](#)
44. [Spring系列第44篇: 详解spring声明式事务\(@Transactional\)](#)
45. [Spring系列第45篇: 带你吃透Spring事务7种传播行为](#)
46. [Spring系列第46篇: Spring如何管理多数据源事务?](#)
47. [Spring系列第47篇: spring编程式事务源码解析](#)
48. [Spring系列第48篇: @Transaction 事务源码解析](#)
49. [Spring系列第49篇: 通过Spring事务实现MQ中的事务消息](#)
50. [Spring系列第50篇: spring事务拦截器顺序如何控制?](#)
51. [Spring系列第51篇: 导致 Spring 事务失效常见的几种情况](#)
52. [Spring系列第52篇: Spring实现数据库读写分离](#)
53. [Spring系列第53篇: Spring集成MyBatis](#)

8、更多好文章

1. [Java高并发系列 \(共34篇\)](#)

2. [MySQL高手系列（共27篇）](#)
3. [Maven高手系列（共10篇）](#)
4. [Mybatis系列（共12篇）](#)
5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要，它是什么？怎么实现？](#)
7. [泛型，有点难度，会让很多人懵逼，那是因为你没有看这篇文章！](#)

9、福利

这边整理了几百本计算机相关的电子书，下面是部分截图，常见的基本上都有，获取方式，扫描文末二维码，发送：**电子书**



C和指针.pdf



658381 Java工...



[www.ebook23....



[www.ebook23....



[www.ebook23....



[www.ebook23....



[www.ebook23....



[www.ebook23....



[www.ebook23....



[www.javazx.co...



Effective Java中...



Elasticsearch权...



HBase不睡觉书...



HBase实战.pdf



Head First 设计...



Java 8实战.pdf



Java JDK 8学习...



JAVA 遗传算法编..



JavaParser-JUG...



javaparsersite...



Java编程宝典.pdf



Java编程思想(第...



Java并发编程之...



Java常用算法手...



JAVA程序员面试...



Java多线程编程...



JAVA多线程设计...



Java高手真经(应...



Java国际认证(SC...



Java经典编程30...



java经典实例 第...



JAVA面试题解惑...



Java深度历险.pdf



java网络编程精...



Java线程(第三版)...



Java线程与并发...



Java性能优化权...



JAVA语言导学.pdf



Java自学网 [jav...



JVM+实用参数...



linux学习.pdf



Maven实战.pdf



MongoDB权威...



MySQL 5.6从零...



MySQL管理之道...



MySQL技术精粹...



MySQL技术内幕...



Neo4j权威指南...



Netty权威指南.p...



Spring Boot 2...



SPRING 实战 (...



Spring.Batch批...



《Apache Kafka...



编写高质量代码...



超越java: 探讨...



程序员实用算法...



大公司最爱问的9...



大型分布式网站...



代码大全 第2版...



分布式Java应用...



分布式服务框架...



分布式系统常用...



疯狂Java: 突破...



高效算法: 竞赛...



互联网创业核心...



架构解密: 从分...



Offer 面试指南



解析 Java 虚拟机...

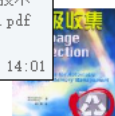


程序员面试揭秘...



面试宝典

名称: 互联网创业核心技术-构建可伸缩的web应用.pdf
大小: 120.84MB
修改时间: 2020-10-20 14:01



架构解密: 从分...

剑指offer 名企面...



面试宝典.pdf

解析Java虚拟机...



面试资料整理.pdf

进军硅谷 程序员...



排序与查找.pdf

尽在双11: 阿里...



七周七并发模型_...

精通Spring 4.x ...



人人都是架构师+...

垃圾收集.pdf



深入Linux内核架...



深入RabbitMQ...



深入分析Java W...



深入理解ElasticS...



深入理解Java虚...



深入理解Spring ...



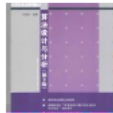
实战Java高并发...



数据结构与算法...



算法的乐趣.pdf



算法设计与分析 ...



透视JAVA--反编...



图解Java多线程...



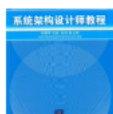
图解TCP_IP_第5...



图解数据结构-使...



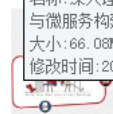
网络IO模型.pdf



系统架构设计师...



这样编码才规范 ...



自己动手写Java...



最新-黑马月入12...



加我微信: spring888java, 进群一起交流java相关各种技术!!

Spring系列第55篇: spring上下文生命周期

本文主要内容：带大家掌握spring应用上下文的生命周期。

为什么需要掌握这个？

- 1、应对面试，面试中经常会问到
- 2、项目中想扩展spring的，那么这部分内容必须掌握
- 3、更容易阅读spring源码

1、什么是spring应用上下文？

接口 `org.springframework.context.ApplicationContext` 表示spring上下文，下面2个实现类

```
org.springframework.context.support.ClassPathXmlApplicationContext  
org.springframework.context.annotation.AnnotationConfigApplicationContext
```

2、应用上文生命周期(14个阶段)

- 1、创建spring应用上下文
- 2、上下文启动准备阶段
- 3、BeanFactory创建阶段
- 4、BeanFactory准备阶段
- 5、BeanFactory后置处理阶段
- 6、BeanFactory注册BeanPostProcessor阶段
- 7、初始化内建Bean：MessageSource
- 8、初始化内建Bean：Spring事件广播器
- 9、Spring应用上下文刷新阶段
- 10、Spring事件监听器注册阶段
- 11、单例bean实例化阶段
- 12、BeanFactory初始化完成阶段
- 13、Spring应用上下文启动完成阶段
- 14、Spring应用上下文关闭阶段

3、Spring应用上下文的使用

看下这段代码，是不是很熟悉，这就是spring上下文最常见的用法，稍后我们以这段代码为例，结合spring源码，来细说每个阶段的细节。

```
@Configuration  
@ComponentScan  
public class MainConfig {  
    public static void main(String[] args) {
```



```

//1.创建spring上下文
AnnotationConfigApplicationContext configApplicationContext = new
AnnotationConfigApplicationContext();
//2.上下文中注册bean
configApplicationContext.register(MainConfig.class);
//3.刷新spring上下文，内部会启动spring上下文
configApplicationContext.refresh();
//4.关闭spring上下文
System.out.println("stop ok!");
    }
}

```

4、阶段1：创建Spring应用上下文

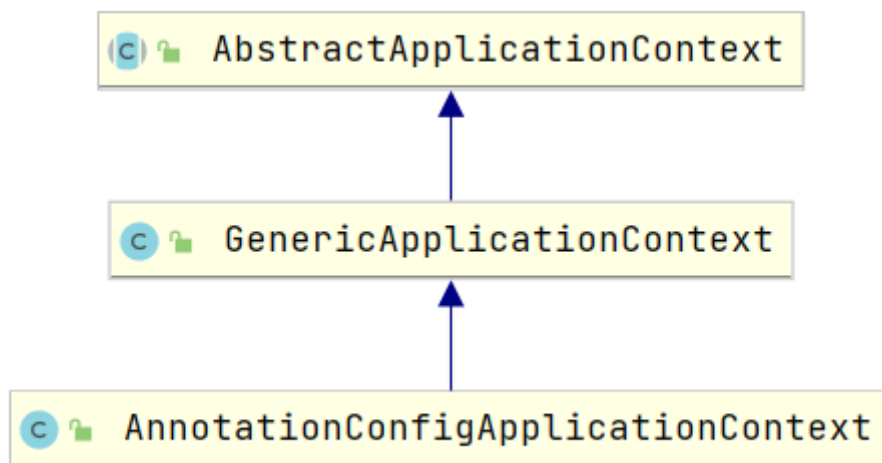
对应这段代码

```

AnnotationConfigApplicationContext configApplicationContext = new
AnnotationConfigApplicationContext();

```

看一下其类图，这里主要列出了 `AnnotationConfigApplicationContext` 2个父类



当调用子类的构造器的时候，默认会自动先调用父类的构造器，先来看一下 `GenericApplicationContext` 构造器源码，如下，将 `beanFactory` 创建好了。

```

public GenericApplicationContext() {
    this.beanFactory = new DefaultListableBeanFactory();
}

```

再看看 `AnnotationConfigApplicationContext` 构造器源码，如下：

```

public AnnotationConfigApplicationContext() {
    //创建AnnotatedBeanDefinitionReader: 用来读取及注册通过注解方式定义的bean
    this.reader = new AnnotatedBeanDefinitionReader(this); //①
    //创建ClassPathBeanDefinitionScanner: bean定义扫描器, 可以扫描包中的类, 对满足条件的
    类, 会将其注册到spring容器中
    this.scanner = new ClassPathBeanDefinitionScanner(this);
}

```

①: new AnnotatedBeanDefinitionReader(this), 进入这个方法内部, 最终会走到下面这个方法, 非常关键的一个方法, 会向spring容器中注册5个关键的bean, 这几个bean都是非常很重要的。

```

org.springframework.context.annotation.AnnotationConfigUtils#registerAnnotationC
onfigProcessors(org.springframework.beans.factory.support.BeanDefinitionRegistry
, java.lang.Object)

```

```

public static Set<BeanDefinitionHolder> registerAnnotationConfigProcessors(
    BeanDefinitionRegistry registry, @Nullable Object source) {

    //1、注册ConfigurationClassPostProcessor, 这是个非常关键的类, 实现了
    BeanDefinitionRegistryPostProcessor接口
    // ConfigurationClassPostProcessor这个类主要做的事情: 负责所有bean的注册, 如果想看
    bean注册源码的, 可以在其postProcessBeanDefinitionRegistry方法中设置断点
    if
    (!registry.containsBeanDefinition(CONFIGURATION_ANNOTATION_PROCESSOR_BEAN_NAME))
    {
        RootBeanDefinition def = new
        RootBeanDefinition(ConfigurationClassPostProcessor.class);
        def.setSource(source);
        beanDefs.add(registerPostProcessor(registry, def,
        CONFIGURATION_ANNOTATION_PROCESSOR_BEAN_NAME));
    }

    //2、注册AutowiredAnnotationBeanPostProcessor: 负责处理@Autowired注解
    if
    (!registry.containsBeanDefinition(AUTOWIRED_ANNOTATION_PROCESSOR_BEAN_NAME)) {
        RootBeanDefinition def = new
        RootBeanDefinition(AutowiredAnnotationBeanPostProcessor.class);
        def.setSource(source);
        beanDefs.add(registerPostProcessor(registry, def,
        AUTOWIRED_ANNOTATION_PROCESSOR_BEAN_NAME));
    }

    //3、注册CommonAnnotationBeanPostProcessor: 负责处理@Resource注解
    if (jsr250Present &&
    !registry.containsBeanDefinition(COMMON_ANNOTATION_PROCESSOR_BEAN_NAME)) {
        RootBeanDefinition def = new
        RootBeanDefinition(CommonAnnotationBeanPostProcessor.class);
        def.setSource(source);
        beanDefs.add(registerPostProcessor(registry, def,
        COMMON_ANNOTATION_PROCESSOR_BEAN_NAME));
    }

    //4、注册EventListenerMethodProcessor: 负责处理@EventListener标注的方法, 即事件处理
    器
    if (!registry.containsBeanDefinition(EVENT_LISTENER_PROCESSOR_BEAN_NAME)) {

```

```

        RootBeanDefinition def = new
RootBeanDefinition(EventListenerMethodProcessor.class);
        def.setSource(source);
        beanDefs.add(registerPostProcessor(registry, def,
EVENT_LISTENER_PROCESSOR_BEAN_NAME));
    }

    //5、注册DefaultEventListenerFactory: 负责将@EventListener标注的方法包装为
ApplicationListener对象
    if (!registry.containsBeanDefinition(EVENT_LISTENER_FACTORY_BEAN_NAME)) {
        RootBeanDefinition def = new
RootBeanDefinition(DefaultEventListenerFactory.class);
        def.setSource(source);
        beanDefs.add(registerPostProcessor(registry, def,
EVENT_LISTENER_FACTORY_BEAN_NAME));
    }

    return beanDefs;
}

```

再来捋一下上面这段代码，主要向spring容器中注册了5个关键的bean

1、ConfigurationClassPostProcessor：这是个非常关键的类，建议去看一下他的源码，基本上我们自定义的bean都是通过这个类注册的，下面这些注解都是在这个类中处理的

```

@Configuration
@Component
@PropertySource
@PropertySources
@ComponentScan
@ComponentScans
@Import
@ImportResource
@Bean

```

2、AutowiredAnnotationBeanPostProcessor：负责处理@Autowired注解

3、注册CommonAnnotationBeanPostProcessor：负责处理@Resource注解

4、注册EventListenerMethodProcessor：负责处理@EventListener标注的方法，即事件处理器

5、注册DefaultEventListenerFactory：负责将@EventListener标注的方法包装为ApplicationListener对象

4、阶段2~阶段13

阶段2到阶段13，这中间的12个阶段都位于refresh方法中，所以refresh方法非常很重要，需要多花时间研究这个方法。

refresh方法源码如下：

```

org.springframework.context.support.AbstractApplicationContext#refresh

@Override
public void refresh() throws BeansException, IllegalStateException {
    // 阶段2: Spring应用上下文启动准备阶段

```

```

prepareRefresh();

// 阶段3: BeanFactory创建阶段
ConfigurableListableBeanFactory beanFactory = obtainFreshBeanFactory();

// 阶段4: BeanFactory准备阶段
prepareBeanFactory(beanFactory);

try {
    // 阶段5: BeanFactory后置处理阶段
    postProcessBeanFactory(beanFactory);

    // 阶段6: BeanFactory注册BeanPostProcessor阶段
    invokeBeanFactoryPostProcessors(beanFactory);

    // 阶段7: 注册BeanPostProcessor
    registerBeanPostProcessors(beanFactory);

    // 阶段8: 初始化内建Bean: MessageSource
    initMessageSource();

    // 阶段9: 初始化内建Bean: Spring事件广播器
    initApplicationEventMulticaster();

    // 阶段10: Spring应用上下文刷新阶段，由子类实现
    onRefresh();

    // 阶段11: Spring事件监听器注册阶段
    registerListeners();

    // 阶段12: 实例化所有剩余的（非lazy init）单例。
    finishBeanFactoryInitialization(beanFactory);

    // 阶段13: 刷新完成阶段
    finishRefresh();
}
}

```

阶段2: Spring应用上下文启动准备阶段

```

protected void prepareRefresh() {
    // 标记启动时间
    this.startupDate = System.currentTimeMillis();
    // 是否关闭: false
    this.closed.set(false);
    // 是否启动: true
    this.active.set(true);

    // 初始化PropertySource，留个子类去实现的，可以在这个方法中扩展属性配置信息，丢到
    this.environment中
    initPropertySources();

    // 验证环境配置中是否包含必须的配置参数信息，比如我们希望上下文启动中，this.environment
    中必须要有某些属性的配置，才可以启动，那么可以在这个里面验证
    getEnvironment().validateRequiredProperties();

    // earlyApplicationListeners用来存放早期的事件监听器
}

```

```

        if (this.earlyApplicationListeners == null) {
            this.earlyApplicationListeners = new LinkedHashSet<>
(this.applicationListeners);
        }
        else {
            // Reset local application listeners to pre-refresh state.
            this.applicationListeners.clear();
            this.applicationListeners.addAll(this.earlyApplicationListeners);
        }

        // earlyApplicationEvents用来存放早期的事件
        this.earlyApplicationEvents = new LinkedHashSet<>();
    }

```

这里说一下什么是早期事件？什么是早期的事件监听器呢？

当使用spring上下文发布事件的时候，如下代码

```
applicationContext.publishEvent(事件对象)
```

其内部最终会用到 `AbstractApplicationContext` 下面这个属性来发布事件，但是可能此时 `applicationEventMulticaster` 还没有创建好，是空对象，所以此时是无法广播事件的，那么此时发布的时间就是早期的时间，就会被放在 `this.earlyApplicationEvents` 中暂时存放着，当时间广播器 `applicationEventMulticaster` 创建好了之后，才会会将早期的事件广播出去

```
private ApplicationEventMulticaster applicationEventMulticaster;
```

同理，当 `applicationEventMulticaster` 还未准备好的时候，调用下面下面代码向spring上下文中添加事件监听器的时候，这时放进去的监听器就是早期的事件监听器。

```
applicationContext.addApplicationListener(事件监听器对象)
```

说了这么多，理解起来很简单，就是当事件广播器 `applicationEventMulticaster` 还未准备好的时候，此时向上下文中添加的事件就是早期的事件，会被放到 `this.earlyApplicationEvents` 中，此时这个事件暂时没办法广播。

阶段3：BeanFactory创建阶段

这个阶段负责将BeanFactory创建好，返回给spring应用上下文

```
ConfigurableListableBeanFactory beanFactory = obtainFreshBeanFactory();
```

`obtainFreshBeanFactory` 源码：

```

protected ConfigurableListableBeanFactory obtainFreshBeanFactory() {
    //刷新BeanFactory，由子类实现
    refreshBeanFactory();
    //返回spring上下文中创建好的BeanFacotry
    return getBeanFactory();
}

```

阶段4: BeanFactory准备阶段

```
prepareBeanFactory(beanFactory);
```

源码如下

```
protected void prepareBeanFactory(ConfigurableListableBeanFactory beanFactory) {
    // 设置类加载器
    beanFactory.setBeanClassLoader(getClassLoader());
    // 设置spel表达式解析器
    beanFactory.setBeanExpressionResolver(new
StandardBeanExpressionResolver(beanFactory.getBeanClassLoader()));
    // 设置属性编辑器注册器
    beanFactory.addPropertyEditorRegistrar(new ResourceEditorRegistrar(this,
getEnvironment()));

    // @1: 添加一个BeanPostProcessor: ApplicationContextAwareProcessor, 当你的bean实
现了spring中xxxAware这样的接口, 那么bean创建的过程中, 将由
ApplicationContextAwareProcessor来回调这些接口, 将对象注入进去
    beanFactory.addBeanPostProcessor(new
ApplicationContextAwareProcessor(this));
    // @2: 自动注入的时候, 下面这些接口定义的方法, 将会被跳过自动注入, 为什么? 因为这部分接口将
由ApplicationContextAwareProcessor来回调注入
    beanFactory.ignoreDependencyInterface(EnvironmentAware.class);
    beanFactory.ignoreDependencyInterface(EmbeddedValueResolverAware.class);
    beanFactory.ignoreDependencyInterface(ResourceLoaderAware.class);
    beanFactory.ignoreDependencyInterface(ApplicationEventPublisherAware.class);
    beanFactory.ignoreDependencyInterface(MessageSourceAware.class);
    beanFactory.ignoreDependencyInterface(ApplicationContextAware.class);

    // @3: 注册依赖注入的时候查找的对象, 比如你的bean中想用ResourceLoader, 那么可以写:
@Autowired private ResourceLoader resourceLoader, 那么spring容器会将spring容器上下文注
入进去
    beanFactory.registerResolvableDependency(BeanFactory.class, beanFactory);
    beanFactory.registerResolvableDependency(ResourceLoader.class, this);
    beanFactory.registerResolvableDependency(ApplicationEventPublisher.class,
this);
    beanFactory.registerResolvableDependency(ApplicationContext.class, this);

    // @4: 注入一个beanpostprocessor: ApplicationListenerDetector
    beanFactory.addBeanPostProcessor(new ApplicationListenerDetector(this));

    // @5: 将Environment注册到spring容器, 对应的bean名称是environment
    if (!beanFactory.containsLocalBean(ENVIRONMENT_BEAN_NAME)) {
        beanFactory.registerSingleton(ENVIRONMENT_BEAN_NAME, getEnvironment());
    }
    // @6: 将系统属性注册到spring容器, 对应的bean名称是systemProperties
    if (!beanFactory.containsLocalBean(SYSTEM_PROPERTIES_BEAN_NAME)) {
        beanFactory.registerSingleton(SYSTEM_PROPERTIES_BEAN_NAME,
getEnvironment().getSystemProperties());
    }
    // @7: 将系统环境变量配置信息注入到spring容器, 对应的bean名称是systemEnvironment
    if (!beanFactory.containsLocalBean(SYSTEM_ENVIRONMENT_BEAN_NAME)) {
```

```
        beanFactory.registerSingleton(SYSTEM_ENVIRONMENT_BEAN_NAME,
        getEnvironment().getSystemEnvironment());
    }
}
```

先来看看@1的代码

```
beanFactory.addBeanPostProcessor(new ApplicationContextAwareProcessor(this));
```

向spring容器中注入了一个ApplicationContextAwareProcessor，这是一个BeanPostProcessor（Bean处理器），这个接口中定义了很多方法，会在bean创建的不同阶段被调用，常用来扩展bean的创建过程。想升入了解的朋友，可以先去看一下这篇文章：[Bean生命周期](#)。

在回到ApplicationContextAwareProcessor这个接口上来，看一下其的源码，大家就知道是干什么的，如下，当我们的bean实现了Aware这样的接口的时候，接口中的方法会被回调，用来在自定义的bean中注入spring上下文的一些对象，比如我们在我们的bean中用到Environment，那么只需实现EnvironmentAware接口，那么Environment会被自动注入进去

```
if (bean instanceof EnvironmentAware) {
    ((EnvironmentAware)
    bean).setEnvironment(this.applicationContext.getEnvironment());
}
if (bean instanceof EmbeddedValueResolverAware) {
    ((EmbeddedValueResolverAware)
    bean).setEmbeddedValueResolver(this.embeddedValueResolver);
}
if (bean instanceof ResourceLoaderAware) {
    ((ResourceLoaderAware) bean).setResourceLoader(this.applicationContext);
}
if (bean instanceof ApplicationEventPublisherAware) {
    ((ApplicationEventPublisherAware)
    bean).setApplicationEventPublisher(this.applicationContext);
}
if (bean instanceof MessageSourceAware) {
    ((MessageSourceAware) bean).setMessageSource(this.applicationContext);
}
if (bean instanceof ApplicationContextAware) {
    ((ApplicationContextAware)
    bean).setApplicationContext(this.applicationContext);
}
```

再来看看@3的代码

```
beanFactory.registerResolvableDependency(BeanFactory.class, beanFactory);
beanFactory.registerResolvableDependency(ResourceLoader.class, this);
beanFactory.registerResolvableDependency(ApplicationEventPublisher.class, this);
beanFactory.registerResolvableDependency(ApplicationContext.class, this);
```

这个用来向spring容器中添加依赖查找的对象，上面代码的第1行是添加了BeanFactory，当我们的bean中想用到BeanFactory的时候，只需要按照下面这么写，那么就会被自动注入，即使因为上面第1行代码将BeanFactory添加到依赖查找列表中了

```
@Autowired
private BeanFactory beanFactory;
```

再来看看 @4 的代码

```
beanFactory.addBeanPostProcessor(new ApplicationListenerDetector(this));
```

ApplicationListenerDetector也是一个BeanPostProcessor，这个类是干嘛的？处理自定义的事件监听器的。

当我们的bean实现了ApplicationListener接口，是一个事件监听器的时候，那么这个bean创建的过程中将会被ApplicationListenerDetector处理，会将我们这个bean添加到spring上下文容器的事件监听器列表中。



再来看看@5的代码

```
if (!beanFactory.containsLocalBean(ENVIRONMENT_BEAN_NAME)) {
    beanFactory.registerSingleton(ENVIRONMENT_BEAN_NAME, getEnvironment());
}
```

将Environment作为单例注册到spring容器单例列表中，对应的bean名称是environment，当我们的bean中需要用到Environment的时候，可以使用下面的写法，此时spring容器创建bean的过程中，就会从单例的bean列表中找到Environment，将其注入到下面的属性中。

```
@Autowired
private Environment environment
```

我们还可以通过 environment 名称从spring容器中查找到Environment对象，如下：

```
applicationContext.getBean("environment", Environment.class)
```

再来看看@6、@7代码

这个同@5的代码，这里说一下

```
getEnvironment().getSystemProperties() -- 对应 --> System.getProperties()
getEnvironment().getSystemEnvironment() -- 对应 --> System.getenv()
```

这里说一下System.getProperties()、System.getenv()这两个是干嘛的。

下面这个命令大家都用过吧，大家主要看一下-D，这个可以设置一些启动参数，-D后面跟的这些参数，会被放在System.getProperties()中了。

```
java -jar -D参数=值
```

System.getenv()用来获取环境变量信息的。

阶段5: BeanFactory后置处理阶段

```
postProcessBeanFactory(beanFactory);
```

这个方法留给子类实现的，此时beanFactory已经创建好了，但是容器中的bean还没有被实例化，子类可以实现这个方法，可以对BeanFactory做一些特殊的配置，比如可以添加一些自定义BeanPostProcessor等等，主要是留给子类去扩展的。

阶段6: BeanFactory注册BeanPostProcessor阶段

```
invokeBeanFactoryPostProcessors(beanFactory);
```

调用BeanFactoryPostProcessor，BeanFactoryPostProcessor是干什么的？

建议一定要先看一下这篇文章：[Spring系列第29篇：BeanFactory扩展（BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor）](#)

这个阶段主要就是从spring容器中找到BeanFactoryPostProcessor接口的所有实现类，然后调用，完成所有bean注册的功能，注意是bean注册，即将bean的定义信息转换为BeanDefinition对象，然后注册到spring容器中，此时bean还未被实例化，下面继续。

大家再回头看一下阶段1，阶段1在spring容器中注册了一个非常关键的bean：ConfigurationClassPostProcessor，代码如下

```
if
(!registry.containsBeanDefinition(CONFIGURATION_ANNOTATION_PROCESSOR_BEAN_NAME))
{
    RootBeanDefinition def = new
    RootBeanDefinition(ConfigurationClassPostProcessor.class);
    def.setSource(source);
    beanDefs.add(registerPostProcessor(registry, def,
    CONFIGURATION_ANNOTATION_PROCESSOR_BEAN_NAME));
}
```

ConfigurationClassPostProcessor就实现了BeanFactoryPostProcessor接口，所以ConfigurationClassPostProcessor会在阶段6中被调用，下面这些注解都是在这个类中处理的，所以想研究下面这些注解原理的，直接去看ConfigurationClassPostProcessor的源码，非常关键重要的一个类，研究spring源码，此类必看

```
@Configuration
@Component
@PropertySource
@PropertySources
@ComponentScan
@ComponentScans
@Import
@ImportResource
@Bean
```

通常，阶段6执行完毕之后，我们所有自定义的bean都已经被注册到spring容器中了，被转换为BeanDefinition丢到BeanFactory中了，此时bean还未被实例化。

阶段7：注册BeanPostProcessor

```
registerBeanPostProcessors(beanFactory);
```

注册BeanPostProcessor，这个阶段会遍历spring容器bean定义列表，把所有实现了BeanPostProcessor接口的bean撸出来，然后将他们添加到spring容器的BeanPostProcessor列表中。

BeanPostProcessor是bean后置处理器，其内部提供了很多方法，用来对bean创建的过程进行扩展的。

阶段8：初始化内建Bean：MessageSource

```
initMessageSource();
```

MessageSource 是用来处理国际化的，这个阶段会将MessageSource创建好，如果想扩展或者实现国际化的，可以看看这个方法的源码。

关于国际化的可以看这篇文章：[Spring系列第26篇：国际化详解](#)

阶段9：初始化内建Bean：Spring事件广播器

```
initApplicationEventMulticaster();
```

ApplicationEventMulticaster是事件广播器，用来广播事件的，这个阶段会将ApplicationEventMulticaster创建好，如果想自定义事件广播器的，可以看看这个方法的源码，关于spring事件的使用，看这里：[Spring系列第27篇：spring事件机制详解](#)。

阶段10：Spring应用上下文刷新阶段，由子类实现

```
onRefresh();
```

用来初始化其他特殊的bean，由子类去实现。

阶段11：Spring事件监听器注册阶段

```
registerListeners();
```

注册事件监听器到事件广播器中，看一下其源码，如下：

```
protected void registerListeners() {  
    // @1:先注册静态指定的侦听器,即将spring上下文中添加的时间监听器,添加到时间广播器  
    (ApplicationEventMulticaster) 中  
    for (ApplicationListener<?> listener : getApplicationListeners()) {  
        getApplicationEventMulticaster().addApplicationListener(listener);  
    }  
}
```

```

    }

    // @2:将spring容器中定义的事件监听器，添加到时间广播器
    (ApplicationEventMulticaster) 中
    String[] listenerBeanNames = getBeanNamesForType(ApplicationListener.class,
true, false);
    for (String listenerBeanName : listenerBeanNames) {

        getApplicationEventMulticaster().addApplicationListenerBean(listenerBeanName);
    }

    // @3:此时事件广播器准备好了，所以此时发布早期的事件（早期的事件由于事件广播器还未被创建，
    所以先被放在了earlyApplicationEvents中，而此时广播器创建好了，所以将早期的时间发布一下）
    Set<ApplicationEvent> earlyEventsToProcess = this.earlyApplicationEvents;
    this.earlyApplicationEvents = null;
    if (earlyEventsToProcess != null) {
        for (ApplicationEvent earlyEvent : earlyEventsToProcess) {
            getApplicationEventMulticaster().multicastEvent(earlyEvent);
        }
    }
}
}

```

这里主要说一下@1，将上下文中的事件监听器添加到事件广播器中，看看下面源码就懂了

```

org.springframework.context.support.AbstractApplicationContext

Set<ApplicationListener<?>> applicationListeners = new LinkedHashSet<>();

public Collection<ApplicationListener<?>> getApplicationListeners() {
    return this.applicationListeners;
}

@Override
public void addApplicationListener(ApplicationListener<?> listener) {
    Assert.notNull(listener, "ApplicationListener must not be null");
    if (this.applicationEventMulticaster != null) {
        this.applicationEventMulticaster.addApplicationListener(listener);
    }
    this.applicationListeners.add(listener);
}

```

再看看@3，广播早期的事件，早期的时候，由于事件广播器 `this.applicationEventMulticaster` 还是空，所以事件被放在了 `this.earlyApplicationEvents` 这个集合中并没有广播，等到这个时候才广播早期的事件，所以用事件的时候，大家需要注意，如果你在阶段11之前，调用了下面方法发布事件，那么可能此时你看不到事件产生的效果

```
applicationContext.publishEvent(事件对象)
```

所以如果你的bean实现了下面这些接口，不建议再实现

`org.springframework.context.ApplicationListener` 接口，否则会让你莫名其妙的感觉。

```

org.springframework.beans.factory.config.BeanFactoryPostProcessor
org.springframework.beans.factory.config.BeanPostProcessor

```

阶段12：实例化所有剩余的（非lazy init）单例

```
finishBeanFactoryInitialization(beanFactory);
```

这个方法中将实例化所有单例的bean（不包含需要延迟实例化的bean），来看看其源码：

```
protected void finishBeanFactoryInitialization(ConfigurableListableBeanFactory
beanFactory) {
    // 添加${}表达式解析器
    if (!beanFactory.hasEmbeddedValueResolver()) {
        beanFactory.addEmbeddedValueResolver(strVal ->
getEnvironment().resolvePlaceholders(strVal));
    }

    // 冻结所有bean定义，表示已注册的bean定义不会被进一步修改或后处理。这允许工厂主动缓存bean
定义元数据。
    beanFactory.freezeConfiguration();

    // @1: 实例化所有单例bean（不包含需延迟实例化的bean），通常scope=singleton的bean都会在下
面这个方法中完成初始化。
    beanFactory.preInstantiateSingletons();
}
```

重点看 @1 的代码，这里会调用 beanFactory 的 preInstantiateSingletons() 方法，进去看源码，源码位于 DefaultListableBeanFactory 这个类中，如下：

```
org.springframework.beans.factory.support.DefaultListableBeanFactory#preInstanti
atesSingletons

public void preInstantiateSingletons() throws BeansException {
    // beanDefinitionNames表示当前BeanFactory中定义的bean名称列表，
    List<String> beanNames = new ArrayList<>(this.beanDefinitionNames);

    // @1: 循环实例化bean
    for (String beanName : beanNames) {
        //这里根据beanName来来实例化bean，代码省略了。。。
    }

    // @2: 变量bean，若bean实现了SmartInitializingSingleton接口，将调用其
afterSingletonsInstantiated()方法
    for (String beanName : beanNames) {
        Object singletonInstance = getSingleton(beanName);
        if (singletonInstance instanceof SmartInitializingSingleton) {
            final SmartInitializingSingleton smartSingleton =
(SmartInitializingSingleton) singletonInstance;
            smartSingleton.afterSingletonsInstantiated();
        }
    }
}
```

上面这个方法主要做了2个事情：

1、完成所有单例bean的实例化：对应上面的代码 @1，循环遍历beanNames列表，完成所有单例bean的实例化工作，这个循环完成之后，所有单例bean已经实例化完毕了，被放在spring容器缓存起来了。

2、回调 `SmartInitializingSingleton` 接口：对应上面的代码 @2，此时所有的单例bean已经实例化好了，此时遍历所有单例bean，若bean实现了 `SmartInitializingSingleton` 接口，这个接口中有个 `afterSingletonsInstantiated` 方法，此时会回调这个方法，若我们想在所有单例bean创建完毕之后，做一些事情，可以实现这个接口。

阶段13：刷新完成阶段

```
finishRefresh();
```

进去看看其源码

```
protected void finishRefresh() {
    // @1: 清理一些资源缓存
    clearResourceCaches();

    // @2: 为此上下文初始化生命周期处理器
    initLifecycleProcessor();

    // @3: 首先将刷新传播到生命周期处理器
    getLifecycleProcessor().onRefresh();

    // @4: 发布ContextRefreshedEvent事件
    publishEvent(new ContextRefreshedEvent(this));
}
```

先来看@2的代码

```
initLifecycleProcessor();
```

源码如下

```
protected void initLifecycleProcessor() {

    ConfigurableListableBeanFactory beanFactory = getBeanFactory();
    //spring容器中是否有名称为"lifecycleProcessor"的bean
    if (beanFactory.containsLocalBean("lifecycleProcessor")) {
        //从spring容器中找到LifecycleProcessor，赋给this.lifecycleProcessor
        this.lifecycleProcessor =
            beanFactory.getBean("lifecycleProcessor", LifecycleProcessor.class);
    }
    else {
        //如果spring容器中没有自定义lifecycleProcessor，那么创建一个默认的
        DefaultLifecycleProcessor
            defaultProcessor = new
            DefaultLifecycleProcessor();
        defaultProcessor.setBeanFactory(beanFactory);
        //设置当前spring应用上下文的生命周期处理器
        this.lifecycleProcessor = defaultProcessor;
        //将其注册到spring容器中
        beanFactory.registerSingleton("lifecycleProcessor",
            this.lifecycleProcessor);
    }
}
```

再来看@3的代码

```
getLifecycleProcessor().onRefresh();
```

调用 `LifecycleProcessor` 的 `onRefresh` 方法，这里要先说一下

`org.springframework.context.LifecycleProcessor` 这个接口，生命周期处理器，接口中定义了2个方法，而 `onClose` 方法会在上下文关闭中会被调用，稍后会看到

```
public interface LifecycleProcessor extends Lifecycle {

    /**
     * 上下文刷新通知
     */
    void onRefresh();

    /**
     * 上下文关闭通知
     */
    void onClose();

}
```

先来看看 `onRefresh` 方法，这个接口有个默认实现

`org.springframework.context.support.DefaultLifecycleProcessor`，所以默认情况下，我们就看 `DefaultLifecycleProcessor` 中的 `onRefresh` 源码，如下

```
public void onRefresh() {
    startBeans(true);
    this.running = true;
}
```

进入 `startBeans(true)` 内部看看，如下，代码看起来可能有点绕，实际上很简单，就是从容器中找到所有实现 `org.springframework.context.Lifecycle` 接口的bean，然后调用他们的 `start` 方法。

```
private void startBeans(boolean autoStartupOnly) {
    Map<String, Lifecycle> lifecycleBeans = getLifecycleBeans();
    Map<Integer, LifecycleGroup> phases = new HashMap<>();
    lifecycleBeans.forEach((beanName, bean) -> {
        if (!autoStartupOnly || (bean instanceof SmartLifecycle &&
            ((SmartLifecycle) bean).isAutoStartup())) {
            int phase = getPhase(bean);
            LifecycleGroup group = phases.get(phase);
            if (group == null) {
                group = new LifecycleGroup(phase, this.timeoutPerShutdownPhase,
                    lifecycleBeans, autoStartupOnly);
                phases.put(phase, group);
            }
            group.add(beanName, bean);
        }
    });
    if (!phases.isEmpty()) {
        List<Integer> keys = new ArrayList<>(phases.keySet());
        Collections.sort(keys);
    }
}
```

```

        for (Integer key : keys) {
            phases.get(key).start();
        }
    }
}

```

再来看@4的代码

```
publishEvent(new ContextRefreshedEvent(this));
```

发布 `ContextRefreshedEvent` 事件，想在这个阶段做点事情的，可以监听这个事件。

小结下

这个阶段主要干了3个事情：

- 1、初始化当前上下文中的生命周期处理器 `LifecycleProcessor`。
- 2、调用 `LifecycleProcessor.onRefresh()` 方法，若没有自定义 `LifecycleProcessor`，那么会走 `DefaultLifecycleProcessor`，其内部会调用spring容器中所有实现 `Lifecycle` 接口的bean的 `start` 方法。
- 3、发布 `ContextRefreshedEvent` 事件。

4、阶段14：Spring应用上下文关闭阶段

```
applicationContext.close()
```

最终会进入到 `doClose()` 方法，代码如下

```

org.springframework.context.support.AbstractApplicationContext#doClose

protected void doClose() {
    // 判断是不是需要关闭（active为true的时候，才能关闭，并用cas确保并发情况下只能有一个执行成功）
    if (this.active.get() && this.closed.compareAndSet(false, true)) {

        // @1: 发布关闭事件ContextClosedEvent
        publishEvent(new ContextClosedEvent(this));

        // @2: 调用生命周期处理器的onClose方法
        if (this.lifecycleProcessor != null) {
            this.lifecycleProcessor.onClose();
        }

        // @3: 销毁上下文的BeanFactory中所有缓存的单例
        destroyBeans();

        // @4: 关闭BeanFactory本身
        closeBeanFactory();

        // @5: 就给子类去扩展的
        onClose();
    }
}

```

```

// @6: 恢复事件监听器列表至刷新之前的状态，即将早期的事件监听器还原
if (this.earlyApplicationListeners != null) {
    this.applicationListeners.clear();
    this.applicationListeners.addAll(this.earlyApplicationListeners);
}

// @7: 标记活动状态为: false
this.active.set(false);
}
}

```

先来看@1的代码

```
publishEvent(new ContextClosedEvent(this));
```

发布 ContextClosedEvent 事件。

再来看@2的代码

```

if (this.lifecycleProcessor != null) {
    this.lifecycleProcessor.onClose();
}

```

调用生命周期处理器的 onClose 方法，这里我们直接看 DefaultLifecycleProcessor 的 onClose()，看看其源码，如下

```

public void onClose() {
    stopBeans();
    //将running设置为false
    this.running = false;
}

```

stopBeans() 源码如下，一大片，简单点理解：就是从容器中找到所有实现 org.springframework.context.Lifecycle 接口的bean，然后调用他们的 close() 方法。

```

private void stopBeans() {
    Map<String, Lifecycle> lifecycleBeans = getLifecycleBeans();
    Map<Integer, LifecycleGroup> phases = new HashMap<>();
    lifecycleBeans.forEach((beanName, bean) -> {
        int shutdownPhase = getPhase(bean);
        LifecycleGroup group = phases.get(shutdownPhase);
        if (group == null) {
            group = new LifecycleGroup(shutdownPhase,
this.timeoutPerShutdownPhase, lifecycleBeans, false);
            phases.put(shutdownPhase, group);
        }
        group.add(beanName, bean);
    });
    if (!phases.isEmpty()) {
        List<Integer> keys = new ArrayList<>(phases.keySet());
        keys.sort(Collections.reverseOrder());
        for (Integer key : keys) {
            phases.get(key).stop();
        }
    }
}

```



```
}  
}
```

再来看@3的代码

```
destroyBeans();
```

销毁上下文的BeanFactory中所有缓存的单例bean，主要干2件事情：

1、调用bean的销毁方法

比如bean实现了 `org.springframework.beans.factory.DisposableBean` 接口，此时会被调用。还有bean中有些方法标注了 `@PreDestroy` 注解的，此时也会被调用。

2、将bean的信息从BeanFactory中清理掉

再来看@4的代码

```
closeBeanFactory();
```

源码如下，主要就是想当前spring应用上下文中的beanFactory属性还原。

```
org.springframework.context.support.AbstractRefreshableApplicationContext#closeBeanFactory  
  
@Override  
protected final void closeBeanFactory() {  
    synchronized (this.beanFactoryMonitor) {  
        if (this.beanFactory != null) {  
            this.beanFactory.setSerializationId(null);  
            this.beanFactory = null;  
        }  
    }  
}
```

5、总结

本文详细介绍了spring应用上下文的生命周期，非常重要，内容和细节稍微比较多，建议大家结合源码多看几遍，平时闲的时候，也可以回头再看几遍，加深理解，有问题的欢迎留言交流。

6、案例源码

git地址：
<https://gitee.com/javacode2018/spring-series>

大家star一下，所有系列代码都会在这个里面，还有所有原创文章的连接也在里面，方便查阅！！

7、Spring系列

1. [Spring系列第1篇：为何要学spring?](#)
2. [Spring系列第2篇：控制反转（IoC）与依赖注入（DI）](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)
4. [Spring系列第4篇：xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇：创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇：玩转bean scope，避免跳坑里!](#)
7. [Spring系列第7篇：依赖注入之手动注入](#)
8. [Spring系列第8篇：自动注入（autowire）详解，高手在于坚持](#)
9. [Spring系列第9篇：depend-on到底是干什么的?](#)
10. [Spring系列第10篇：primary可以解决什么问题?](#)
11. [Spring系列第11篇：bean中的autowire-candidate又是干什么的?](#)
12. [Spring系列第12篇：lazy-init: bean延迟初始化](#)
13. [Spring系列第13篇：使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇：lookup-method和replaced-method比较陌生，怎么玩的?](#)
15. [Spring系列第15篇：代理详解（Java动态代理&cglb代理）?](#)
16. [Spring系列第16篇：深入理解java注解及spring对注解的增强（预备知识）](#)
17. [Spring系列第17篇：@Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇：@ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇：@import详解\(bean批量注册\)](#)
20. [Spring系列第20篇：@Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇：注解实现依赖注入（@Autowired、@Resource、@Primary、@Qualifier）](#)
22. [Spring系列第22篇：@Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇：Bean生命周期详解](#)
24. [Spring系列第24篇：父子容器详解](#)
25. [Spring系列第25篇：@Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇：国际化详解](#)
27. [Spring系列第27篇：spring事件机制详解](#)
28. [Spring系列第28篇：Bean循环依赖详解](#)
29. [Spring系列第29篇：BeanFactory扩展（BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor）](#)
30. [Spring系列第30篇：jdk动态代理和cglb代理](#)
31. [Spring系列第31篇：aop概念详解](#)
32. [Spring系列第32篇：AOP核心源码、原理详解](#)
33. [Spring系列第33篇：ProxyFactoryBean创建AOP代理](#)
34. [Spring系列第34篇：@Aspect中@Pointcut 12种用法](#)
35. [Spring系列第35篇：@Aspect中5中通知详解](#)
36. [Spring系列第36篇：@EnableAspectJAutoProxy、@Aspect中通知顺序详解](#)
37. [Spring系列第37篇：@EnableAsync & @Async 实现方法异步调用](#)
38. [Spring系列第38篇：@Scheduled & @EnableScheduling定时器详解](#)
39. [Spring系列第39篇：强大的Spel表达式](#)
40. [Spring系列第40篇：缓存使用（@EnableCaching、@Cacheable、@CachePut、@CacheEvict、@Caching、@CacheConfig）](#)
41. [Spring系列第41篇：@EnableCaching集成redis缓存](#)
42. [Spring系列第42篇：玩转JdbcTemplate](#)
43. [Spring系列第43篇：spring中程式事务怎么用的?](#)
44. [Spring系列第44篇：详解spring声明式事务\(@Transactional\)](#)
45. [Spring系列第45篇：带你吃透Spring事务7种传播行为](#)
46. [Spring系列第46篇：Spring如何管理多数据源事务?](#)
47. [Spring系列第47篇：spring程式事务源码解析](#)
48. [Spring系列第48篇：@Transaction 事务源码解析](#)

- 49. [Spring系列第49篇：通过Spring事务实现MQ中的事务消息](#)
- 50. [Spring系列第50篇：spring事务拦截器顺序如何控制？](#)
- 51. [Spring系列第51篇：导致 Spring 事务失效常见的几种情况](#)
- 52. [Spring系列第52篇：Spring实现数据库读写分离](#)
- 53. [Spring系列第53篇：Spring集成MyBatis](#)
- 54. [Spring系列第54篇：集成junit](#)

8、更多好文章

- 1. [Java高并发系列（共34篇）](#)
- 2. [MySql高手系列（共27篇）](#)
- 3. [Maven高手系列（共10篇）](#)
- 4. [Mybatis系列（共12篇）](#)
- 5. [聊聊db和缓存一致性常见的实现方式](#)
- 6. [接口幂等性这么重要，它是什么？怎么实现？](#)
- 7. [泛型，有点难度，会让很多人懵逼，那是因为你没有看这篇文章！](#)

9、文末扫码领取《2020版java面试通关手册》

下面给大家送上面试通关手册，按分类都整理好了，查看特别的方便，便于快速突破自己薄弱的环节，拿下offer。

- ☐  2020年BATJ JAVA经典必考面试题库
- ☐  2020年JAVA常见面试题库
- ☐  2020年Java各知识点综合面试题
- ☐  2020年JAVA核心面试题库
- ☐  2020年互联网一线大厂面试题库

按分类都整理好了，便于查看，快速各个击破

- | | |
|--|---|
| ☐  ActiveMQ消息中间件 | ☐  SpringCloud |
| ☐  Dubbo | ☐  SpringMVC |
| ☐  JVM | ☐  Tomcat |
| ☐  Kafka | ☐  Zookeeper |
| ☐  Linux | ☐  并发编程 |
| ☐  MongoDB | ☐  多线程 |
| ☐  MyBatis | ☐  集合框架 |
| ☐  MySQL | ☐  乐观锁与悲观锁 |
| ☐  Netty | ☐  设计模式 |
| ☐  Nginx | ☐  数据库 |
| ☐  RabbitMQ消息中间件 | ☐  性能优化 |
| ☐  Redis | |
| ☐  Spring | |
| ☐  SpringBoot | |

详细的分类列表

☐ 文件名

- ☐  zookeeper面试题.pdf
- ☐  Zookeeper+分布式过程.pdf
- ☐  zookeeper面试专题.pdf
- ☐  Zookeeper篇.pdf

☐ 文件名

- ☐  Netty面试题.pdf
- ☐  Netty面试专题.pdf

☐ 文件名

- ☐  Spring面试专题及答案.pdf
- ☐  Spring面试题.pdf
- ☐  Spring面试题（含答案）.pdf
- ☐  Spring面试专题.pdf
- ☐  Spring源码深度解析.pdf

☐ 文件名

- ☐  JVM.pdf
- ☐  JVM常见面试题指南.pdf
- ☐  JVM面试题.pdf
- ☐  JVM面试专题.pdf
- ☐  JVM执行子系统.pdf
- ☐  JVM性能优化相关问题.pdf

☐ 文件名

- ☐  Redis面试题.pdf
- ☐  Redis面试题（二）.pdf
- ☐  Redis面试题（含答案）.pdf
- ☐  Redis面试专题.pdf
- ☐  Redis篇.pdf
- ☐  Redis实战.pdf
- ☐  Redis面试专题（二）.pdf

☐ 文件名

☐  SpringBoot面试题.pdf

☐  SpringBoot面试专题.pdf

☐ 文件名

☐  SpringCloud面试题.pdf

☐  SpringCloud面试专题.pdf

☐ 文件名

☐  面试必备之乐观锁与悲观锁.pdf

☐ 文件名

☐  并发编程面试题.pdf

☐  并发编程面试专题.pdf

☐  并发面试题.pdf

☐ 文件名

☐  MongoDB面试专题.pdf

☐  MongoDB篇.pdf

☐  MongoDB面试题.pdf

☐ 文件名

☐  SpringMVC面试题.pdf

☐  SpringMVC面试专题.pdf

☐  看透Spring MVC源代码分析与实践 .pdf

☐ 文件名

☐  java面试题_设计模式(26题).pdf

☐  设计模式面试题.pdf

☐  设计模式面试专题.pdf

☐ 文件名

☐  数据库.pdf

获取方式：扫描下面二维码，发送：通关



加我微信：spring888java，进群一起交流java相关各种技术！！

Spring系列第56篇：面试官：循环依赖不用三级缓存可以么？

今天来聊一个面试中经常会被问到的问题，咱们一起必须把这个问题搞懂。

问题：spring中为什么需要用三级缓存来解决这个问题？用二级缓存可以么？

我先给出答案：不可用。

这里先声明下：

本文未指明bean scope默认情况下，所有bean都是单例的，即scope是singleton，即下面所有问题都是在单例的情况下分析的。

代码中注释很详细，一定要注意多看代码中的注释。

1、循环依赖相关问题

- 1、什么是循环依赖？
- 2、循环依赖的注入对象的2种方式：构造器的方式、setter的方式
- 3、构造器的方式详解
- 4、spring是如何知道有循环依赖的？
- 5、setter方式详解
- 6、需注意循环依赖注入的是半成品

7、为什么必须用三级缓存？

2、什么是循环依赖？

A依赖于B，B依赖于A，比如下面代码

```
public class A {  
    private B b;  
}  
  
public class B {  
    private A a;  
}
```

3、循环依赖注入对象的2种方式

3.1、构造器的方式

通过构造器相互注入对方，代码如下

```
public class A {  
    private B b;  
  
    public A(B b) {  
        this.b = b;  
    }  
}  
  
public class B {  
    private A a;  
  
    public B(A a) {  
        this.a = a;  
    }  
}
```

3.2、setter的方式

通过setter方法注入对方，代码如下

```
public class A {  
    private B b;  
  
    public B getB() {  
        return b;  
    }  
  
    public void setB(B b) {  
        this.b = b;  
    }  
}
```



```
public class B {  
    private A a;  
  
    public A getA() {  
        return a;  
    }  
  
    public void setA(A a) {  
        this.a = a;  
    }  
}
```

4、构造器的方式详解

4.1、构造器的方式知识点

- 1、构造器的方式如何注入？
- 2、循环依赖，构造器的方式，spring的处理过程是什么样的？
- 3、循环依赖构造器的方式案例代码解析

4.2、构造器的方式如何注入？

再来看一下下面这2个类，相互依赖，通过构造器的方式相互注入对方。

```
public class A {  
    private B b;  
  
    public A(B b) {  
        this.b = b;  
    }  
}  
  
public class B {  
    private A a;  
  
    public B(A a) {  
        this.a = a;  
    }  
}
```

大家来思考一个问题：**2个类都只能创建一个对象，大家试着用硬编码的方式看看可以创建这2个类的对象么？**

我想大家一眼就看出来了，无法创建。

创建A的时候需要先有B，而创建B的时候需要先有A，导致无法创建成功。

4.3、循环依赖，构造器的方式，spring的处理过程是什么样的？

spring在创建bean之前，会将当前正在创建的bean名称放在一个列表中，这个列表我们就叫做singletonsCurrentlyInCreation，用来记录正在创建中的bean名称列表，创建完毕之后，会将其从singletonsCurrentlyInCreation列表中移除，并且会将创建好的bean放到另外一个单例列表中，这个列表叫做singletonObjects，下面看一下这两个集合的代码，如下：

代码位于org.springframework.beans.factory.support.DefaultSingletonBeanRegistry类中

```
//用来存放正在创建中的bean名称列表
private final Set<String> singletonsCurrentlyInCreation =
    Collections.newSetFromMap(new ConcurrentHashMap<>(16));

//用来存放已经创建好的单例bean，key为bean名称，value为bean的实例
private final Map<String, Object> singletonObjects = new ConcurrentHashMap<>
(256);
```

下面我们来看下面2个bean的创建过程

```
@Component
public class A {
    private B b;

    public A(B b) {
        this.b = b;
    }
}

@Component
public class B {
    private A a;

    public B(A a) {
        this.a = a;
    }
}
```

过程如下

- 1、从`singletonObjects`查看是否有a，此时没有
- 2、准备创建a
- 3、判断a是否在`singletonsCurrentlyInCreation`列表，此时明显不在，则将a加入`singletonsCurrentlyInCreation`列表
- 4、调用a的构造器A(B b)创建A
- 5、spring发现A的构造器需要用到b
- 6、则向spring容器查找b，从`singletonObjects`查看是否有b，此时没有
- 7、spring准备创建b
- 8、判断b是否在`singletonsCurrentlyInCreation`列表，此时明显不在，则将b加入`singletonsCurrentlyInCreation`列表
- 9、调用b的构造器B(A a)创建b
- 10、spring发现B的构造器需要用到a，则向spring容器查找a
- 11、则向spring容器查找a，从`singletonObjects`查看是否有a，此时没有
- 12、准备创建a
- 13、判断a是否在`singletonsCurrentlyInCreation`列表，上面第3步中a被放到了这个列表，此时a在这个列表中，走到这里了，说明a已经存在创建列表中了，此时程序又来创建a，说明这么一直走下去会死循环，此时spring会弹出异常，终止bean的创建操作。

4.4、通过这个过程，我们得到了2个结论

1、循环依赖如果是构造器的方式，bean无法创建成功，这个前提是bean都是单例的，bean如果是多例的，大家自己可以分析分析。

2、spring是通过`singletonsCurrentlyInCreation`这个列表来发现循环依赖的，这个列表会记录创建中的bean，当发现bean在这个列表中存在了，说明有循环依赖，并且这个循环依赖是无法继续走下去的，如果继续走下去，会进入死循环，此时spring会抛出异常让系统终止。

判断循环依赖的源码在下面这个位置，`singletonsCurrentlyInCreation`是Set类型的，Set的add方法返回false，说明被add的元素在Set中已经存在了，然后会抛出循环依赖的异常。

```
org.springframework.beans.factory.support.DefaultSingletonBeanRegistry#beforeSingletonCreation

private final Set<String> singletonsCurrentlyInCreation =
    Collections.newSetFromMap(new ConcurrentHashMap<>(16));

protected void beforeSingletonCreation(String beanName) {
    //bean名称已经存在创建列表中，则抛出循环依赖异常
    if (!this.inCreationCheckExclusions.contains(beanName) &&
        !this.singletonsCurrentlyInCreation.add(beanName)) {
        //抛出循环依赖异常
        throw new BeanCurrentlyInCreationException(beanName);
    }
}

//循环依赖异常
public BeanCurrentlyInCreationException(String beanName) {
    super(beanName,
        "Requested bean is currently in creation: Is there an unresolvable
        circular reference?");
}
```

4.5、spring构造器循环依赖案例

创建类A

```
package com.javacode2018.cycledependency.demo1;

import org.springframework.stereotype.Component;

@Component
public class A {
    private B b;

    public A(B b) {
        this.b = b;
    }
}
```

创建类B

```
package com.javacode2018.cycledependency.demo1;

import org.springframework.stereotype.Component;

@Component
public class B {
    private A a;

    public B(A a) {
        this.a = a;
    }
}
```

启动类

```
package com.javacode2018.cycledependency.demo1;

import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan
public class MainConfig {
    public static void main(String[] args) {
        AnnotationConfigApplicationContext context = new
        AnnotationConfigApplicationContext();
        context.register(MainConfig.class);
        //刷新容器上下文，触发单例bean创建
        context.refresh();
        //关闭上下文
        context.close();
    }
}
```

运行上面的main方法，产生了异常，部分异常信息如下，说明创建bean a的时候出现了循环依赖，导致创建bean无法继续进行，以后大家遇到这个错误了，应该可以很快定位到问题了。

```
Caused by: org.springframework.beans.factory.BeanCurrentlyInCreationException:
Error creating bean with name 'a': Requested bean is currently in creation: Is
there an unresolvable circular reference?
    at
    org.springframework.beans.factory.support.DefaultSingletonBeanRegistry.beforeSin
gletonCreation(DefaultSingletonBeanRegistry.java:347)
```

5、setter方式详解

再看看setter的2个类的源码

```
public class A {
    private B b;

    public B getB() {
        return b;
    }

    public void setB(B b) {
        this.b = b;
    }
}

public class B {
    private A a;

    public A getA() {
        return a;
    }

    public void setA(A a) {
        this.a = a;
    }
}
```

大家试试通过硬编码的方式来相互注入，很简单吧，如下面这样

```
A a = new A();
B b = new B();
a.setB(b);
b.setA(a);
```

咱们通过硬编码的方式可以搞成功的，spring肯定也可以搞成功，确实，setter循环依赖，spring可以正常执行。

下面来看spring中setter循环依赖注入的流程。

6、spring中setter循环依赖注入流程

spring在创建单例bean的过程中，会用到三级缓存，所以需要先了解三级缓存。

6.1、三级缓存是哪三级？

spring中使用了3个map来作为三级缓存，每一级对应一个map

第几级缓存	对应的map	说明
第1级	Map<String, Object> singletonObjects	用来存放已经完全创建好的单例bean beanName->bean实例
第2级	Map<String, Object> earlySingletonObjects	用来存放早期的bean beanName->bean实例
第3级	Map<String, ObjectFactory<?>> singletonFactories	用来存放单例bean的ObjectFactory beanName->ObjectFactory实例

这3个map的源码位于

org.springframework.beans.factory.support.DefaultSingletonBeanRegistry 类中。

6.2、单例bean创建过程源码解析

代码入口

```
org.springframework.beans.factory.support.AbstractBeanFactory#doGetBean
```

step1: doGetBean

如下，这个方法首先会调用getSingleton获取bean，如果可以获取到，就会直接返回，否则会执行创建bean的流程

```
Project
AbstractBeanFactory.java x AbstractAutowireCapableBeanFactory.java x BeanPostProcessor.java x Default
AbstractBeanFactory > doGetBean()
    * @return an instance of the bean
    * @throws BeansException if the bean could not be created
    */
    /unchecked/
    protected <T> T doGetBean(final String name, @Nullable final Class<T> requiredType
        @Nullable final Object[] args, boolean typeCheckOnly) throws BeansException

        final String beanName = transformedBeanName(name);
        Object bean;

        // Eagerly check singleton cache for manually registered singletons.
        Object sharedInstance = getSingleton(beanName);
        if (sharedInstance != null && args == null) {
            if (logger.isTraceEnabled()) {
                if (isSingletonCurrentlyInCreation(beanName)) {
                    logger.trace("Returning eagerly cached instance of singleton be
```

step2: getSingleton(beanName, true)

源码如下，这个方法内部会调用 `getSingleton(beanName, true)` 获取bean，注意第二个参数是 `true`，这个表示是否可以获取早期的bean，这个参数为true，会尝试从三级缓存 `singletonFactories` 中获取bean，然后将三级缓存中获取到的bean丢到二级缓存中。

```
public Object getSingleton(String beanName) {
    return getSingleton(beanName, true);
}

protected Object getSingleton(String beanName, boolean allowEarlyReference) {
    //从第1级缓存中获取bean
    Object singletonObject = this.singletonObjects.get(beanName);
    //第1级中没有,且当前beanName在创建列表中
    if (singletonObject == null && isSingletonCurrentlyInCreation(beanName)) {
        synchronized (this.singletonObjects) {
            //从第2级缓存汇总获取bean
            singletonObject = this.earlySingletonObjects.get(beanName);
            //第2级缓存中没有 && allowEarlyReference为true，也就是说2级缓存中没有找到
            //bean且beanName在当前创建列表中的时候，才会继续往下走。
            if (singletonObject == null && allowEarlyReference) {
                //从第3级缓存中获取bean
                ObjectFactory<?> singletonFactory =
                this.singletonFactories.get(beanName);
                //第3级中有获取到了
                if (singletonFactory != null) {
                    //3级缓存汇总放的是ObjectFactory，所以会调用其getObject方法获取bean
                    singletonObject = singletonFactory.getObject();
                    //将3级缓存中的bean丢到第2级中
                    this.earlySingletonObjects.put(beanName, singletonObject);
                    //将bean从三级缓存中干掉
                    this.singletonFactories.remove(beanName);
                }
            }
        }
    }
    return singletonObject;
}
```

step3: getSingleton(String beanName, ObjectFactory<?> singletonFactory)

上面调用 `getSingleton(beanName, true)` 没有获取到bean，所以会继续走bean的创建逻辑，会走到下面代码，如下

```
AbstractBeanFactory.java
AbstractBeanFactory > doGetBean()

310
317
318
319
320 // Create bean instance.
321 if (mbd.isSingleton()) { 根据bean的定义判断是不是单例
322     sharedInstance = getSingleton(beanName, () -> {
323         try {
324             return createBean(beanName, mbd, args);
325         }
326         catch (BeansException ex) {
327             // Explicitly remove instance from singleton cache: It might have been put there
328             // eagerly by the creation process, to allow for circular reference resolution.
329             // Also remove any beans that received a temporary reference to the bean.
330             destroySingleton(beanName);
331             throw ex;
332         }
333     });
334     bean = getObjectForBeanInstance(sharedInstance, name, beanName, mbd);
335 }
```

进入 `getSingleton(String beanName, ObjectFactory<?> singletonFactory)`, 源码如下, 只留了重要的部分

```
public Object getSingleton(String beanName, ObjectFactory<?> singletonFactory) {
    //从第1级缓存中获取bean, 如果可以获取到, 则自己返回
    Object singletonObject = this.singletonObjects.get(beanName);
    if (singletonObject == null) {
        //将beanName加入当前创建列表中
        beforeSingletonCreation(beanName);
        //①: 创建单例bean
        singletonObject = singletonFactory.getObject();
        //将beanName从当前创建列表中移除
        afterSingletonCreation(beanName);
        //将创建好的单例bean放到1级缓存中, 并将其从2、3级缓存中移除
        addSingleton(beanName, singletonObject);
    }
    return singletonObject;
}
```

注意代码①, 会调用 `singletonFactory.getObject()` 创建单例bean, 我们回头看看 `singletonFactory` 这个变量的内容, 如下图, 可以看出主要就是调用 `createBean` 这个方法

```
// Create bean instance.
if (mbd.isSingleton()) {
    sharedInstance = getSingleton(beanName, () -> {
        try {
            return createBean(beanName, mbd, args);
        }
        catch (BeansException ex) {
            // Explicitly remove instance from singleton cache: It might have
            // eagerly by the creation process, to allow for circular referen
            // Also remove any beans that received a temporary reference to
            destroySingleton(beanName);
            throw ex;
        }
    });
    bean = getObjectForBeanInstance(sharedInstance, name, beanName, mbd);
}
```


下面我们进入 `createBean` 方法，这个内部最终会调用 `doCreateBean` 来创建bean，所以我们主要看 `doCreateBean`。

step4: doCreateBean

```
protected Object doCreateBean(final String beanName, final RootBeanDefinition
mbd, final @Nullable Object[] args)
    throws BeanCreationException {

    // ①: 创建bean实例，通过反射实例化bean，相当于new x()创建bean的实例
    BeanWrapper instanceWrapper = createBeanInstance(beanName, mbd, args);

    // bean = 获取刚刚new出来的bean
    Object bean = instanceWrapper.getWrappedInstance();

    // ②: 是否需要将早期的bean暴露出去，所谓早期的bean相当于这个bean就是通过new的方式创建了这个对象，但是这个对象还没有填充属性，所以是个半成品
    // 是否需要将早期的bean暴露出去，判断规则（bean是单例 && 是否允许循环依赖 && bean是否在正在创建的列表中）
    boolean earlySingletonExposure = (mbd.isSingleton() &&
    this.allowCircularReferences &&
    isSingletonCurrentlyInCreation(beanName));

    if (earlySingletonExposure) {
        //③: 调用addSingletonFactory方法，这个方法内部会将其丢到第3级缓存中，
        getEarlyBeanReference的源码大家可以看一下，内部会调用一些方法获取早期的bean对象，比如可以在这个里面通过aop生成代理对象
        addSingletonFactory(beanName, () -> getEarlyBeanReference(beanName, mbd,
        bean));
    }

    // 这个变量用来存储最终返回的bean
    Object exposedObject = bean;
    //填充属性，这里面会调用setter方法或者通过反射将依赖的bean注入进去
    populateBean(beanName, mbd, instanceWrapper);
    //④: 初始化bean，内部会调用BeanPostProcessor的一些方法，对bean进行处理，这里可以对bean进行包装，比如生成代理
    exposedObject = initializeBean(beanName, exposedObject, mbd);

    //早期的bean是否被暴露出去了
    if (earlySingletonExposure) {
        /**
         * ⑤: getSingleton(beanName, false)，注意第二个参数是false，这个为false的时候，
         * 只会从第1和第2级中获取bean，此时第1级中肯定是没有的（只有bean创建完毕之后才会放入1级缓存）
         */
        Object earlySingletonReference = getSingleton(beanName, false);
        /**
         * ⑥: 如果earlySingletonReference不为空，说明第2级缓存有这个bean，二级缓存中有这个bean，说明了什么？
         * 大家回头再去看看上面的分析，看一下什么时候bean会被放入2级缓存？
         * （若 bean存在三级缓存中 && beanName在当前创建列表的时候，此时其他地方调用了getSingleton(beanName, false)方法，那么bean会从三级缓存移到二级缓存）
         */
        if (earlySingletonReference != null) {
            //⑦: exposedObject==bean，说明bean创建好了之后，后期没有被修改

```

```

        if (exposedObject == bean) {
            //earlySingletonReference是从二级缓存中获取的，二级缓存中的bean来源于三
            级缓存，三级缓存中可能对bean进行了包装，比如生成了代理对象
            //那么这个地方就需要将 earlySingletonReference 作为最终的bean
            exposedObject = earlySingletonReference;
        } else if (!this.allowRawInjectionDespiteWrapping &&
hasDependentBean(beanName)) {
            //回头看看上面的代码，刚开始exposedObject=bean，
            // 此时能走到这里，说明exposedObject和bean不一样了，他们不一样了说明了什
            么？

            // 说明initializeBean内部对bean进行了修改
            // allowRawInjectionDespiteWrapping（默认是false）：是否允许早期暴露出
            去的bean(earlySingletonReference)和最终的bean不一致
            // hasDependentBean(beanName)：表示有其他bean以利于beanName
            // getDependentBeans(beanName)：获取有哪些bean依赖beanName
            String[] dependentBeans = getDependentBeans(beanName);
            Set<String> actualDependentBeans = new LinkedHashSet<>
            (dependentBeans.length);
            for (String dependentBean : dependentBeans) {
                //判断dependentBean是否已经被标记为创建了，就是判断dependentBean是
                否已经被创建了

                if
                (!removeSingletonIfCreatedForTypeCheckOnly(dependentBean)) {
                    actualDependentBeans.add(dependentBean);
                }
            }
            /**
             *
             * 能走到这里，说明早期的bean被别人使用了，而后面程序又将exposedObject做了
            修改

            * 也就是说早期创建的bean是A，这个A已经被有些地方使用了，但是A通过
            initializeBean之后可能变成了B，比如B是A的一个代理对象
            * 这个时候就坑了，别人已经用到的A和最终容器中创建完成的A不是同一个A对象了，
            那么使用过程中就可能存在问题了
            * 比如后期对A做了增强（Aop），而早期别人用到的A并没有被增强
            */
            if (!actualDependentBeans.isEmpty()) {
                //弹出异常（早期给别人的bean和最终容器创建的bean不一致了，弹出异常）
                throw new BeanCurrentlyInCreationException(beanName,"异常内容
                见源码。。。。");
            }
        }
    }

    return exposedObject;
}

```

上面的step1~step4，大家要反复看几遍，下面这几个问题搞清楚之后，才可以继续向下看，不懂的结
合源码继续看上面几个步骤

1、什么时候bean被放入3级缓存？

早期的bean被放入3级缓存

2、什么时候bean会被放入2级缓存？

当beanX还在创建的过程中，此时被加入当前beanName 创建列表了，但是这个时候bean并没有被创建完毕（bean被丢到一级缓存才算创建完毕），此时bean还是个半成品，这个时候其他bean需要用到beanX，此时会从三级缓存中获取到beanX，beanX会从三级缓存中丢到2级缓存中。

3、什么时候bean会被放入1级缓存？

bean实例化完毕，初始化完毕，属性注入完毕，bean完全组装完毕之后，才会被丢到1级缓存。

4、populateBean方法是干什么的？

填充属性的，比如注入依赖的对象。

6.3、下面来看A、B类setter循环依赖的创建过程

- 1、getSingleton("a", true) 获取a：会依次从3个级别的缓存中找a，此时3个级别的缓存中都没有a
- 2、将a丢到正在创建的beanName列表中（Set<String> singletonsCurrentlyInCreation）
- 3、实例化a：A a = new A();这个时候a对象是早期的a，属于半成品
- 4、将早期的a丢到三级缓存中（Map<String, ObjectFactory<?> > singletonFactories）
- 5、调用populateBean方法，注入依赖的对象，发现setB需要注入b
- 6、调用getSingleton("b", true) 获取b：会依次从3个级别的缓存中找a，此时3个级别的缓存中都没有b
- 7、将b丢到正在创建的beanName列表中
- 8、实例化b：B b = new B();这个时候b对象是早期的b，属于半成品
- 9、将早期的b丢到三级缓存中（Map<String, ObjectFactory<?> > singletonFactories）
- 10、调用populateBean方法，注入依赖的对象，发现setA需要注入a
- 11、调用getSingleton("a", true) 获取a：此时a会从第3级缓存中被移到第2级缓存，然后将其返回给b使用，此时a是个半成品（属性还未填充完毕）
- 12、b通过setA将11中获取的a注入到b中
- 13、b被创建完毕，此时b会从第3级缓存中被移除，然后被丢到1级缓存
- 14、b返回给a，然后b被通过A类中的setB注入给a
- 15、a的populateBean执行完毕，即：完成属性填充，到此时a已经注入到b中了
- 16、调用 `a = initializeBean("a", a, mbd)` 对a进行处理，这个内部可能对a进行改变，有可能导致a和原始的a不是同一个对象了
- 17、调用 `getSingleton("a", false)` 获取a，注意这个时候第二个参数是false，这个参数为false的时候，只会从前2级缓存中尝试获取a，而a在步骤11中已经被丢到了第2级缓存中，所以此时这个可以获取到a，这个a已经被注入给b了
- 18、此时判断注入给b的a和通过 `initializeBean` 方法产生的a是否是同一个a，不是同一个，则弹出异常

从上面的过程中我们可以得到一个非常非常重要的结论

当某个bean进入到2级缓存的时候，说明这个bean的早期对象被其他bean注入了，也就是说，这个bean还是半成品，还未完全创建好的时候，已经被别人拿去使用了，所以必须要有3级缓存，2级缓存中存放的是早期的被别人使用的对象，如果没有2级缓存，是无法判断这个对象在创建的过程中，是否被别人拿去使用了。

3级缓存是为了解决一个非常重要的问题：早期被别人拿去使用的bean和最终成型的bean是否是一个bean，如果不是同一个，则会产生异常，所以以后面试的时候被问到为什么需要用到3级缓存的时候，你只需要这么回答就可以了：**三级缓存是为了判断循环依赖的时候，早期暴露出去已经被别人使用的bean和最终的bean是否是同一个bean，如果不是同一个则弹出异常，如果早期的对象没有被其他bean使用，而后期被修改了，不会产生异常，如果没有三级缓存，是无法判断是否有循环依赖，且早期的bean被循环依赖中的bean使用了。。**

spring容器默认是不允许早期暴露给别人的bean和最终的bean不一致的，但是这个配置可以修改，而修改之后存在很大的分享，所以不要去改，通过下面这个变量控制

```
org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory#allowRawInjectionDespiteWrapping

private boolean allowRawInjectionDespiteWrapping = false;
```

6.4、模拟BeanCurrentlyInCreationException异常

来个登录接口ILogin

```
package com.javacode2018.cycledependency.demo2;

//登录接口
public interface ILogin {
}
```

来2个实现类

LoginA

这个上面加上 `@Component` 注解，且内部需要注入 `x`

```
package com.javacode2018.cycledependency.demo2;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class LoginA implements ILogin {
    @Autowired
    private X x;

    public X getX() {
        return x;
    }

    public void setX(X x) {
        this.x = x;
    }
}
```

LoginC，不需要spring来管理

```

package com.javacode2018.cycledependency.demo2;

//代理
public class LoginC implements ILogin {

    private ILogin target;

    public LoginC(ILogin target) {
        this.target = target;
    }
}

```

X类，有@Component，且需要注入ILogin对象，这个地方会注入LoginA，此时LoginA和X会参数循环依赖

```

package com.javacode2018.cycledependency.demo2;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class X {

    @Autowired
    private ILogin login;

    public ILogin getLogin() {
        return login;
    }

    public void setLogin(ILogin login) {
        this.login = login;
    }

}

```

添加一个BeanPostProcessor类，实现 postProcessAfterInitialization 方法，这个方法发现bean是loginA的时候，将其包装为LoginC返回，这个方法会在bean创建的过程中调用 initializeBean 时候被调用

```

package com.javacode2018.cycledependency.demo2;

import org.springframework.beans.BeansException;
import org.springframework.beans.factory.config.BeanPostProcessor;
import org.springframework.stereotype.Component;

@Component
public class MyBeanPostProcessor implements BeanPostProcessor {

    @Override
    public Object postProcessAfterInitialization(Object bean, String beanName)
        throws BeansException {
        if (beanName.equals("loginA")) {
            //loginA实现了ILogin
            return new LoginC((ILogin) bean);
        }
    }
}

```

```

        } else {
            return bean;
        }
    }
}

```

spring配置类

```

package com.javacode2018.cycledependency.demo2;

import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan
public class MainConfig {
    public static void main(String[] args) {
        AnnotationConfigApplicationContext context = new
AnnotationConfigApplicationContext();
        context.register(MainConfig.class);
        context.refresh();
        context.close();
    }
}

```

运行输出，产生了 `BeanCurrentlyInCreationException` 异常，是因为注入给x的是LoginA这个类的对象，而最后容器中beanname: loginA对应的是LoginC了，导致注入给别人的对象和最中的对象不一致了，产生了异常。

```

Exception in thread "main"
org.springframework.beans.factory.BeanCurrentlyInCreationException: Error
creating bean with name 'loginA': Bean with name 'loginA' has been injected into
other beans [x] in its raw version as part of a circular reference, but has
eventually been wrapped. This means that said other beans do not use the final
version of the bean. This is often the result of over-eager type matching -
consider using 'getBeanNamesForType' with the 'allowEagerInit' flag turned off,
for example.

```

7、案例：若只使用2级缓存会产生什么后果？

下面来个案例，通过在源码中设置断点的方式，来模拟二级缓存产生的后果。

添加A类

我们希望loginA在A类之前被创建好，所以这里用到了@DependsOn注解。

```
package com.javacode2018.cycledependency.demo3;

import org.springframework.context.annotation.DependsOn;
import org.springframework.stereotype.Component;

@Component
@DependsOn("loginA") //类A依赖于loginA, 但是又不想通过属性注入的方式强依赖
public class A {

}
```

接口ILogin

```
package com.javacode2018.cycledependency.demo3;

//登录接口
public interface ILogin {

}
```

来2个实现类, LoginA需要spring管理, LoginC不需要spring管理。

LoginA

```
package com.javacode2018.cycledependency.demo3;

import org.springframework.stereotype.Component;

@Component
public class LoginA implements ILogin {

}
```

LoginC

```
package com.javacode2018.cycledependency.demo3;

//代理
public class LoginC implements ILogin {

    private ILogin target;

    public LoginC(ILogin target) {
        this.target = target;
    }

}
```

MyBeanPostProcessor

负责将loginA包装为LoginC

```
package com.javacode2018.cycledependency.demo3;

import org.springframework.beans.BeansException;
```

```

import org.springframework.beans.factory.config.BeanPostProcessor;
import org.springframework.stereotype.Component;

@Component
public class MyBeanPostProcessor implements BeanPostProcessor {
    @Override
    public Object postProcessAfterInitialization(Object bean, String beanName)
        throws BeansException {
        if (beanName.equals("loginA")) {
            //loginA实现了ILogin
            return new LoginC((ILogin) bean);
        } else {
            return bean;
        }
    }
}

```

启动类MainConfig

```

package com.javacode2018.cycldependency.demo3;

import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan
public class MainConfig {
    public static void main(String[] args) {
        AnnotationConfigApplicationContext context = new
        AnnotationConfigApplicationContext();
        context.register(MainConfig.class);
        context.refresh();
        context.close();
    }
}

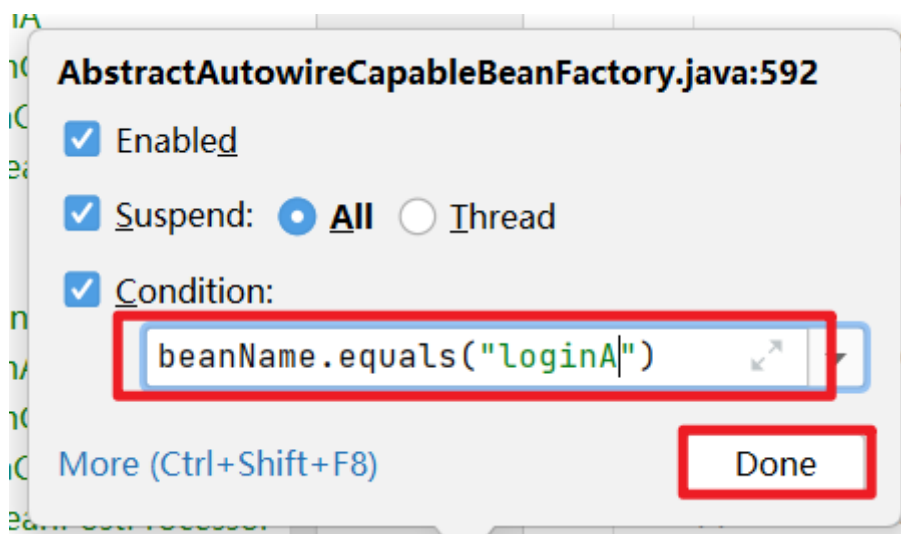
```

下面模拟只使用二级缓存的情况

在bean被放到三级缓存之后，下面的一行代码处设置断点，操作如下

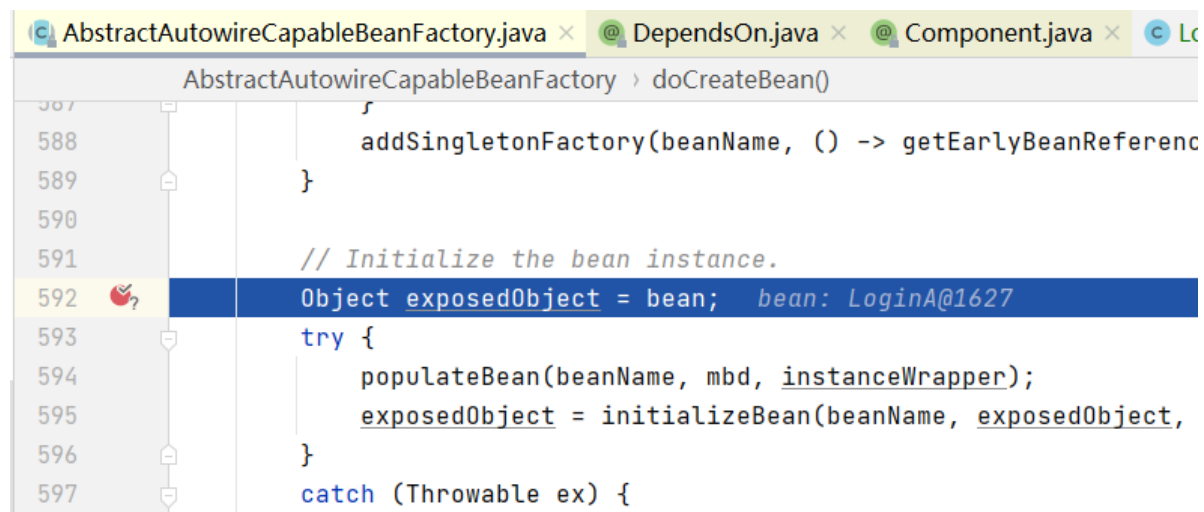


会弹出一个框，然后填入下面配置，这个配置表示满足条件的时候，这个断点才会起效



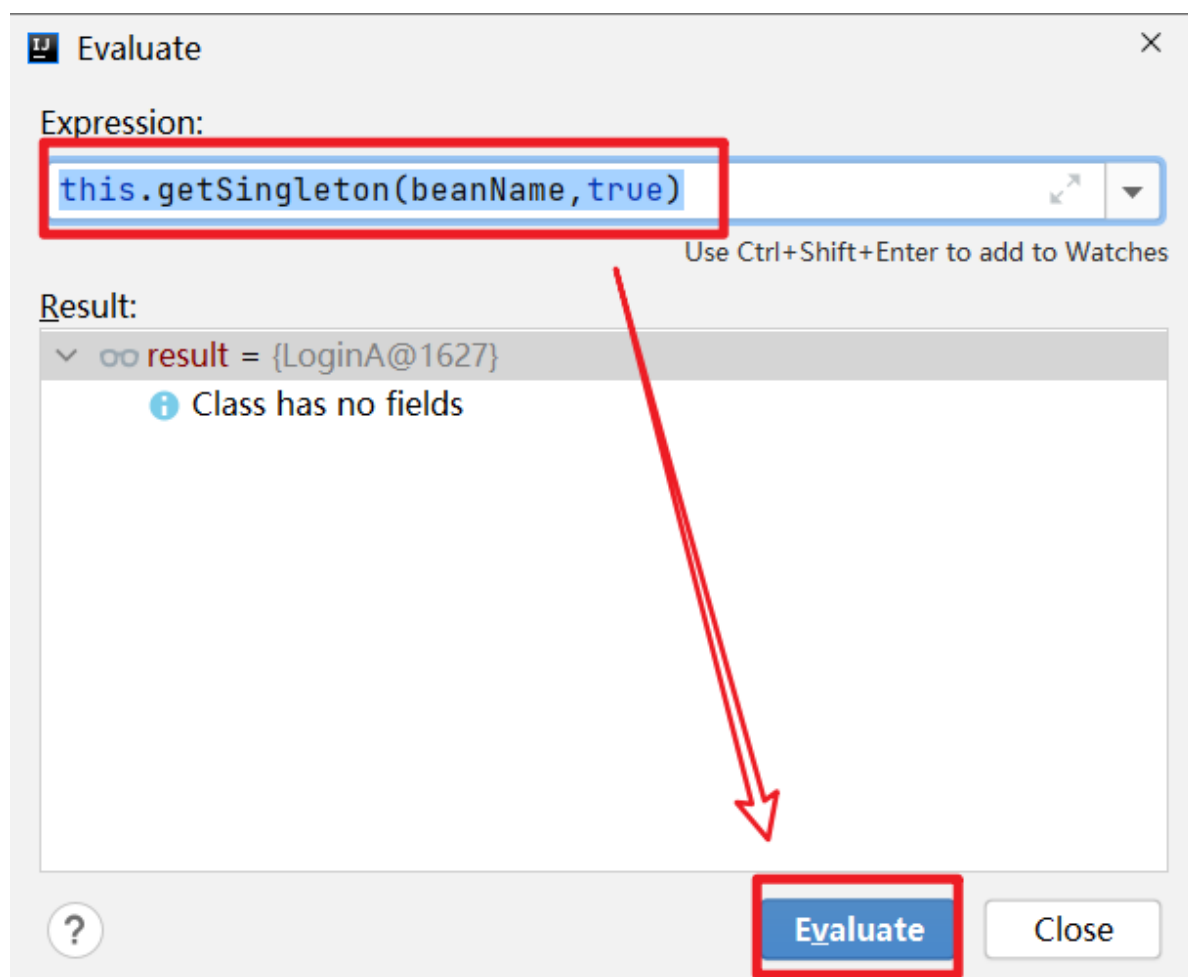
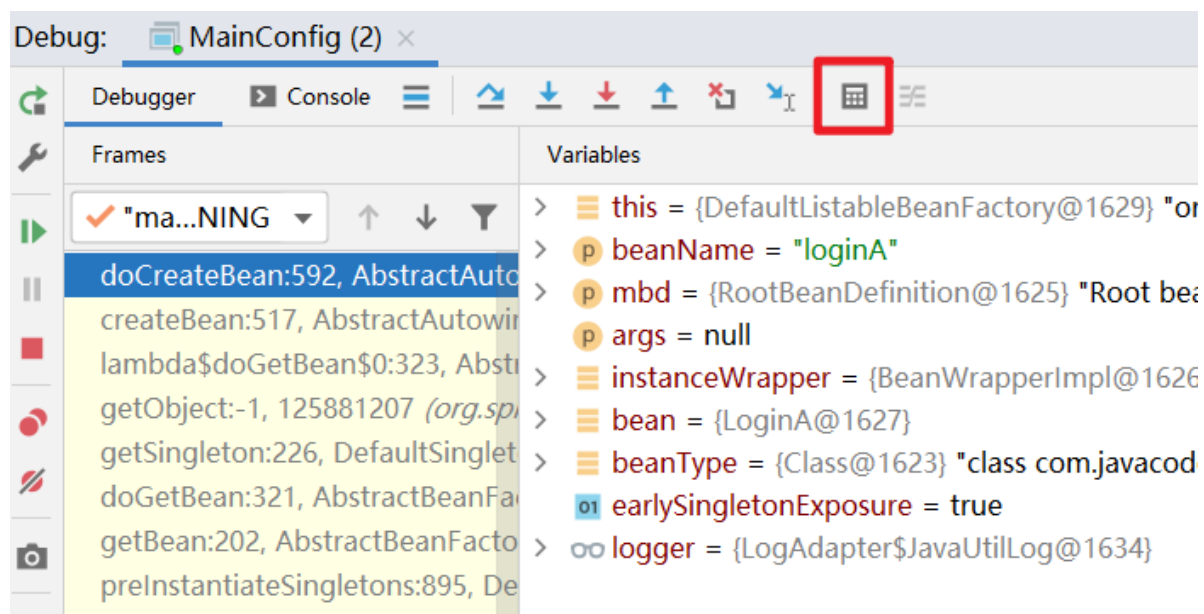
debug方式运行程序

走到了这个断点的位置，此时loginA已经被放到第3级缓存中，此时如果我们调用 `this.getSingleton(beanName, true)`, loginA会从第3级缓存移到第2级，这个时候就相当于只有2级缓存了，操作如下



点击下面的按钮，会弹出一个窗口，可以在窗口中执行代码，执行

`this.getSingleton(beanName, true)`，即将 `loginA` 从三级缓存放到2级缓存，这样相当于没有3级缓存了。



运行结果，最终也参数了 `BeanCurrentlyInCreationException` 异常，实际上这个程序并没有出现循环依赖的情况，但是如果只用了二级缓存，也出现了早期被暴露的bean和最终的bean不一致的问题所参数的异常。

```
Exception in thread "main"
org.springframework.beans.factory.BeanCurrentlyInCreationException
```

这个程序如果不进行干预，直接运行，是可以正常运行的，只有在3级缓存的情况才可以正常运行。

8、总结

今天的内容有点多，大家慢慢消化，有问题欢迎留言！

9、案例源码

git地址：

<https://gitee.com/javacode2018/spring-series>

本文案例对应源码：

`spring-series\lesson-008-junit`

大家star一下，所有系列代码都会在这个里面，还有所有原创文章的连接也在里面，方便查阅！！

10、Spring系列

1. [Spring系列第1篇：为何要学spring?](#)
2. [Spring系列第2篇：控制反转 \(IoC\) 与依赖注入 \(DI\)](#)
3. [Spring系列第3篇：Spring容器基本使用及原理](#)
4. [Spring系列第4篇：xml中bean定义详解\(-\)](#)
5. [Spring系列第5篇：创建bean实例这些方式你们都知道?](#)
6. [Spring系列第6篇：玩转bean scope，避免跳坑里！](#)
7. [Spring系列第7篇：依赖注入之手动注入](#)
8. [Spring系列第8篇：自动注入 \(autowire\) 详解，高手在于坚持](#)
9. [Spring系列第9篇：depend-on到底是干什么的?](#)
10. [Spring系列第10篇：primary可以解决什么问题?](#)
11. [Spring系列第11篇：bean中的autowire-candidate又是干什么的?](#)
12. [Spring系列第12篇：lazy-init：bean延迟初始化](#)
13. [Spring系列第13篇：使用继承简化bean配置\(abstract & parent\)](#)
14. [Spring系列第14篇：lookup-method和replaced-method比较陌生，怎么玩的?](#)
15. [Spring系列第15篇：代理详解 \(Java动态代理&cglib代理\) ?](#)
16. [Spring系列第16篇：深入理解java注解及spring对注解的增强 \(预备知识\)](#)
17. [Spring系列第17篇：@Configuration和@Bean注解详解\(bean批量注册\)](#)
18. [Spring系列第18篇：@ComponentScan、@ComponentScans详解\(bean批量注册\)](#)
19. [Spring系列第18篇：@import详解\(bean批量注册\)](#)
20. [Spring系列第20篇：@Conditional通过条件来控制bean的注册](#)
21. [Spring系列第21篇：注解实现依赖注入 \(@Autowired、@Resource、@Primary、@Qualifier\)](#)
22. [Spring系列第22篇：@Scope、@DependsOn、@ImportResource、@Lazy 详解](#)
23. [Spring系列第23篇：Bean生命周期详解](#)
24. [Spring系列第24篇：父子容器详解](#)
25. [Spring系列第25篇：@Value【用法、数据来源、动态刷新】](#)
26. [Spring系列第26篇：国际化详解](#)
27. [Spring系列第27篇：spring事件机制详解](#)
28. [Spring系列第28篇：Bean循环依赖详解](#)
29. [Spring系列第29篇：BeanFactory扩展 \(BeanFactoryPostProcessor、BeanDefinitionRegistryPostProcessor\)](#)
30. [Spring系列第30篇：jdk动态代理和cglib代理](#)
31. [Spring系列第31篇：aop概念详解](#)

32. [Spring系列第32篇：AOP核心源码、原理详解](#)
33. [Spring系列第33篇：ProxyFactoryBean创建AOP代理](#)
34. [Spring系列第34篇：@Aspect中@Pointcut 12种用法](#)
35. [Spring系列第35篇：@Aspect中5中通知详解](#)
36. [Spring系列第36篇：@EnableAspectJAutoProxy、@Aspect中通知顺序详解](#)
37. [Spring系列第37篇：@EnableAsync & @Async 实现方法异步调用](#)
38. [Spring系列第38篇：@Scheduled & @EnableScheduling定时器详解](#)
39. [Spring系列第39篇：强大的Spel表达式](#)
40. [Spring系列第40篇：缓存使用 \(@EnableCaching、@Cacheable、@CachePut、@CacheEvict、@Caching、@CacheConfig\)](#)
41. [Spring系列第41篇：@EnableCaching集成redis缓存](#)
42. [Spring系列第42篇：玩转JdbcTemplate](#)
43. [Spring系列第43篇：spring中编程式事务怎么用的？](#)
44. [Spring系列第44篇：详解spring声明式事务\(@Transactional\)](#)
45. [Spring系列第45篇：带你吃透Spring事务7种传播行为](#)
46. [Spring系列第46篇：Spring如何管理多数据源事务？](#)
47. [Spring系列第47篇：spring编程式事务源码解析](#)
48. [Spring系列第48篇：@Transaction 事务源码解析](#)
49. [Spring系列第49篇：通过Spring事务实现MQ中的事务消息](#)
50. [Spring系列第50篇：spring事务拦截器顺序如何控制？](#)
51. [Spring系列第51篇：导致 Spring 事务失效常见的几种情况](#)
52. [Spring系列第52篇：Spring实现数据库读写分离](#)
53. [Spring系列第53篇：Spring集成MyBatis](#)
54. [Spring系列第54篇：集成junit](#)
55. [Spring系列第55篇：spring上下文生命周期](#)

11、更多好文章

1. [Java高并发系列（共34篇）](#)
2. [MySql高手系列（共27篇）](#)
3. [Maven高手系列（共10篇）](#)
4. [Mybatis系列（共12篇）](#)
5. [聊聊db和缓存一致性常见的实现方式](#)
6. [接口幂等性这么重要，它是什么？怎么实现？](#)
7. [泛型，有点难度，会让很多人懵逼，那是因为你没有看这篇文章！](#)